Cryptography & Network Security

End-Sem Notes: Lectures 1–18 and the rest of the official syllabus

CSN15403, MNNIT Allahabad · All questions, past papers and practice sets are in the separate **Question Bank** PDF

★★ = asked in *both* past mid-sem papers ★ = asked once / high priority ▸ = not examined yet, likely in the end-sem

★ MUST KNOW TRAP highlight = write this sentence in the exam

Contents

1 What to Study First (End-Sem) 9

1.1 What the end-sem covers . . . 9
1.2 What the past papers say about the examiner . . . 9
1.3 What is most likely after the mid-sem ▸ . . . 10
1.4 Study order . . . 10

I Foundations and Classical Cryptography 10

2 Model for Network Security 10

2.1 Four tasks in designing a security service . . . 11

3 Network Access Security Model 11

4 Cryptography 12

4.1 Symmetric cipher model . . . 12
4.2 Types of cryptography . . . 13

5 Security of Encryption Schemes 13

6 Security Goals, Attacks and Cryptanalysis 14

6.1 Security trends . . . 14
6.2 Security goals (CIA) . . . 15
6.3 Attacks (Forouzan's taxonomy) ★ . . . 15
6.4 Security services and mechanisms (ITU-T X.800) ★ . . . 17
6.5 Kerckhoff's principle and cryptanalysis attacks ★ . . . 18
6.6 History notes from the slides . . . 20

7 Classical Encryption Techniques 21

7.1 Substitution technique . . . 21
7.2 Transposition technique . . . 21
7.3 Substitution vs. transposition . . . 22

8 Caesar Cipher 22

8.1 Letters as numbers (start at 0) . . . 22
8.2 Algorithm . . . 22
8.3 Worked example: encrypt "neso academy" ($k = 3$) . . . 23
8.4 Homework 1: decrypt "QHWZRUN VHFXULWB" ($k = 3$) . . . 23

9 Shift Cipher (the general form) 23

9.1 Pros and cons of the Caesar cipher . . . 24

10 Brute-Force Attack on the Caesar/Shift Cipher 24

11 Monoalphabetic Cipher 26

11.1 Permutations . . . 26
11.2 The cipher . . . 26
11.3 Breaking it: frequency analysis . . . 27

11.4 Pros, cons, countermeasure . . . 28
11.5 Homework: encrypt with a key phrase . . . 28

12 Playfair Cipher ★★ 29
12.1 Building the 5×5 key matrix . . . 29
12.2 Step 1: split the plaintext into digrams . . . 29
12.3 Step 2: encrypt each pair (three cases) ★ . . . 30
12.4 Worked examples (keyword MONARCHY) . . . 30
12.5 Solved question . . . 31
12.6 When I or J is in the keyword or the message ★ . . . 31
12.7 Decryption . . . 32
12.8 The slide version: Forouzan's key square . . . 32
12.9 Strength and weakness . . . 32

13 Hill Cipher 33
13.1 Encryption: matrix multiplication mod 26 . . . 33
13.2 Which matrices can be keys . . . 34
13.3 Encryption example: "pay more money" . . . 34
13.4 Decryption: finding K^{-1} . . . 34
13.5 Decryption example: RRLMWBKASPDH . . . 35
13.6 A 2 × 2 example . . . 35
13.7 The slide example: a 4 × 4 key (Forouzan) . . . 36
13.8 Why Hill hides letter frequencies . . . 36
13.9 Cryptanalysis: the known-plaintext attack . . . 36

14 Polyalphabetic Ciphers 37
14.1 Vigenère cipher ★ . . . 38
14.2 Autokey system ★ . . . 39
14.3 Vernam cipher . . . 40
14.4 The classical substitution ciphers compared . . . 40

15 One-Time Pad ★ 41
15.1 How it grew out of the earlier ciphers . . . 41
15.2 Definition and mechanism . . . 42
15.3 Example 1: letters . . . 42
15.4 Example 2: bits . . . 42
15.5 Why it cannot be broken . . . 42
15.6 Perfect secrecy: the derivation . . . 43
15.7 The rules, and what breaks if you bend them . . . 43
15.8 Two fundamental difficulties . . . 44
15.9 Exam-format one-time-pad numerical . . . 44

16 Transposition Ciphers 45
16.1 Rail fence . . . 45
16.2 Decrypting a rail fence . . . 45
16.3 Row–column transposition . . . 46
16.4 Two stages: more complex . . . 47

17 Traditional Ciphers: Remaining Topics (Lecture 5) 47
17.1 Multiplicative and affine ciphers ★ . . . 48
17.2 Breaking the affine cipher: two pairs give (k_1, k_2) . . . 49
17.3 Cryptanalysis of Vigenère: Kasiski test and index of coincidence . . . 50
17.4 Cryptanalysis of Hill: known-plaintext . . . 51
17.5 Rotor ciphers and Enigma . . . 52
17.6 Transposition: keyless, keyed, combined ★ . . . 53

17.7 Encryption and decryption keys of a transposition . . . 54
17.8 Cryptanalysis of transposition ciphers . . . 54
17.9 Stream vs. block ciphers . . . 55

18 Classical Ciphers at a Glance **55**

19 Steganography **56**
19.1 Historical techniques . . . 57
19.2 Drawbacks . . . 57
19.3 LSB steganography . . . 57
19.4 Watermarking vs. image steganography . . . 58
19.5 OpenStego demo . . . 58

20 Prime Numbers **59**
20.1 Why primes matter in cryptography . . . 59

II Mathematics of Cryptography 60

21 Modular Arithmetic **60**
21.1 Integer division and residues . . . 60
21.2 Congruence and residue classes . . . 60
21.3 Operations in $\mathbb{Z}_n$: why reducing first is allowed . . . 61
21.4 Additive and multiplicative inverses ★ . . . 62
21.5 The sets $\mathbb{Z}_n$, $\mathbb{Z}_n^*$, $\mathbb{Z}_p$ and $\mathbb{Z}_p^*$ . . . 62
21.6 Linear congruences $ax \equiv b \pmod{n}$ . . . 63

22 Modular Exponentiation ★ **64**
22.1 Tool 1: reduce the base first . . . 64
22.2 Tool 2: repeated squaring . . . 64
22.3 Square-and-multiply: both scanning orders and the cost ★ . . . 66

23 GCD and the Euclidean Algorithm **68**
23.1 Divisibility and common divisors . . . 68
23.2 Euclid's algorithm and why it works . . . 69
23.3 Method 1: the table (Q, A, B, R) . . . 69
23.4 Method 2: recursion . . . 70
23.5 Relatively prime numbers . . . 70

24 Extended Euclidean Algorithm ★★ **71**
24.1 The idea: carry every remainder as a combination of a and b . . . 71
24.2 The algorithm and the table (Forouzan) . . . 72
24.3 Multiplicative inverse with the table ★★ . . . 72
24.4 Linear Diophantine equations $ax + by = c$ ★ . . . 73

25 Relatively Prime Numbers, Euler's Totient and $\mathbb{Z}_n^*$ **75**
25.1 Relatively prime (co-prime) numbers . . . 75
25.2 Euler's totient function $\phi(n)$ ★ . . . 75
25.3 Computing $\phi(n)$: the four rules ★ . . . 76
25.4 The sets $\mathbb{Z}_n$ and $\mathbb{Z}_n^*$ . . . 77
25.5 Why cryptography cares about $\phi(n)$ . . . 77

26 Fermat's and Euler's Theorems, Inverses and Primitive Roots **78**
26.1 Fermat's little theorem and exponent reduction ★ . . . 78
26.2 Euler's theorem ★ . . . 79
26.3 Multiplicative inverses via Fermat and Euler ★ . . . 80
26.4 Order of an element ▸ . . . 81

26.5 Primitive roots ▸ . . . 81

27 Primes, Primality Testing, Factorization, CRT and Discrete Logs **83**
27.1 Primes and how many there are . . . 83
27.2 Primality testing: deterministic and probabilistic . . . 83
27.3 Miller–Rabin test ▸ . . . 84
27.4 Factorization . . . 86
27.5 Chinese remainder theorem ▸ . . . 86
27.6 Fast exponentiation: square-and-multiply ★ . . . 89
27.7 Discrete logarithms ▸ . . . 90

28 Algebraic Structures: Groups, Rings, Fields ★★ **91**
28.1 Groups . . . 91
28.2 Order, subgroups and cyclic subgroups . . . 92
28.3 Rings and fields . . . 92
28.4 Finite fields: why the size must be p^n . . . 93
28.5 GF(p) arithmetic and division ★ . . . 93
28.6 Why GF(2^n) instead of $\mathbb{Z}_{2^n}$ . . . 94
28.7 Polynomial representation and addition . . . 94
28.8 Irreducible polynomials: the modulus . . . 94
28.9 Polynomial long division over GF(2) ★ . . . 94
28.10 Multiplication in GF(2^n) ★ . . . 95
28.11 Multiplicative inverse in GF(2^n): extended Euclid on polynomials ★★ . . . 96
28.12 Small fields: GF(2^2) and GF(2^3) . . . 99

III Symmetric-Key Ciphers **99**

29 Modern Symmetric-Key Ciphers **99**
29.1 Modern block ciphers and padding . . . 99
29.2 Components of a modern block cipher . . . 100
29.3 Product ciphers: confusion, diffusion and rounds ★ . . . 101
29.4 Feistel ciphers ★ . . . 102
29.5 Non-Feistel ciphers . . . 104
29.6 Attacks on modern block ciphers . . . 104
29.7 Modern stream ciphers . . . 104

30 Data Encryption Standard (DES) **105**
30.1 DES structure ★ . . . 105
30.2 Initial and final permutations . . . 105
30.3 Rounds: mixer, swapper and the reverse cipher . . . 106
30.4 The DES function $f(R_{i-1}, K_i)$ ★ . . . 106
30.5 Key generation ★ . . . 108
30.6 DES analysis: avalanche and completeness . . . 110
30.7 Weaknesses of DES . . . 110
30.8 Multiple DES . . . 111
30.9 Security of DES in one line each . . . 111

31 Advanced Encryption Standard (AES) **112**
31.1 Parameters and data units . . . 112
31.2 Round structure . . . 112
31.3 SubBytes (substitution) ★ . . . 113
31.4 ShiftRows (permutation) ★ . . . 114
31.5 MixColumns (mixing) ★ . . . 115
31.6 AddRoundKey (key adding) ★ . . . 115

31.7 Key expansion (AES-128) ★ . . . 116
31.8 One complete round: FIPS-197 example . . . 117
31.9 Decryption . . . 117
31.10 Analysis of AES . . . 118
31.11 AES versus DES . . . 118

32 Block Cipher Modes of Operation **118**
32.1 What a mode must achieve ▸ . . . 119
32.2 Electronic codebook (ECB) mode ▸ . . . 119
32.3 Cipher block chaining (CBC) mode ★▸ . . . 119
32.4 Cipher feedback (CFB) mode ▸ . . . 120
32.5 Output feedback (OFB) mode ▸ . . . 121
32.6 Counter (CTR) mode ★▸ . . . 122
32.7 IV and nonce rules, and why block size matters ★▸ . . . 122
32.8 Error propagation, parallelism and padding compared ★▸ . . . 122
32.9 Worked example: the five modes on a toy 4-bit cipher ★▸ . . . 123
32.10 Ciphertext stealing . . . 124
32.11 Multiple encryption: double and triple DES . . . 124

IV Asymmetric-Key Cryptography **125**

33 Asymmetric-Key Cryptography and RSA ▸ **125**
33.1 Why asymmetric-key cryptography exists ▸ . . . 125
33.2 Trapdoor one-way functions ▸ . . . 125
33.3 RSA: key generation, encryption, decryption ▸ . . . 126
33.4 Why decryption works: proof of correctness ▸ . . . 126
33.5 The slide examples ▸ . . . 126
33.6 Exam-format RSA numericals ▸ . . . 127
33.7 Choosing e; why $e = 2$ never works ▸ . . . 128
33.8 Faster decryption with the CRT ▸ . . . 128
33.9 Attacks on RSA ▸ . . . 128
33.10 Recommendations . . . 129

34 ElGamal and Elliptic Curve Cryptosystems ▸ **129**
34.1 ElGamal cryptosystem ▸ . . . 129
34.2 Security of ElGamal ▸ . . . 130
34.3 Elliptic curves over the real numbers . . . 131
34.4 Elliptic curves over GF(p) ▸ . . . 131
34.5 Point addition and doubling over GF(p) ▸ . . . 132
34.6 Scalar multiplication and the ECDLP ▸ . . . 133
34.7 ECC simulating ElGamal ▸ . . . 133
34.8 Security of ECC and key size ▸ . . . 134

V Integrity, Authentication and Key Management **134**

35 Message Integrity and Hash Functions **134**
35.1 Integrity versus secrecy; message and digest ▸ . . . 134
35.2 The three security criteria ▸ . . . 135
35.3 The random oracle model ▸ . . . 135
35.4 The four birthday problems ▸ . . . 136
35.5 Attacks on an n-bit digest ▸ . . . 137
35.6 Iterated hash functions: the Merkle–Damgård scheme ▸ . . . 138
35.7 Compression functions: from scratch or from a block cipher ▸ . . . 139
35.8 SHA-512: overview and parameters ▸ . . . 140

35.9 SHA-512: padding and length field ▸ . . . 140
35.10 SHA-512: word expansion and the compression function ▸ . . . 141
35.11 Exercises on the SHA-512 operations ▸ . . . 142
35.12 Whirlpool . . . 142

36 Message Authentication: MDC, MAC, HMAC and CMAC **143**
36.1 Why a digest does not authenticate: MDC versus MAC ▸ . . . 143
36.2 Prefix, postfix and envelope MACs from a hash ▸ . . . 144
36.3 Security of a MAC ▸ . . . 145
36.4 Nested MAC and HMAC ★▸ . . . 145
36.5 CBC-MAC (Data Authentication Algorithm) and CMAC ▸ . . . 146

37 Digital Signatures **148**
37.1 Conventional versus digital signatures ▸ . . . 148
37.2 The signing and verifying process; why keys are needed ★▸ . . . 148
37.3 Signing the digest ▸ . . . 149
37.4 Services: authentication, integrity, nonrepudiation, but not confidentiality ★▸ . . . 150
37.5 Attacks on digital signatures; types of forgery ▸ . . . 151
37.6 RSA digital signature scheme ★▸ . . . 151
37.7 ElGamal digital signature scheme ★▸ . . . 152
37.8 Schnorr digital signature scheme ▸ . . . 154
37.9 Digital Signature Standard (DSS / DSA) ★▸ . . . 155
37.10 Elliptic Curve Digital Signature Algorithm (ECDSA) ▸ . . . 156
37.11 Security attacks on digital signature schemes ▸ . . . 157
37.12 Comparison of the five schemes . . . 158

38 Key Management and Key Distribution **159**
38.1 The symmetric-key distribution problem: the N^2 problem ▸ . . . 159
38.2 Key-distribution center (KDC) and session keys ▸ . . . 160
38.3 Session-key creation: a simple KDC protocol and its flaw ▸ . . . 161
38.4 The Needham–Schroeder protocol ▸ . . . 161
38.5 The Otway–Rees protocol ▸ . . . 163
38.6 Kerberos ★▸ . . . 163
38.7 Comparing the KDC-based protocols ▸ . . . 166
38.8 Symmetric-key agreement: Diffie–Hellman ★▸ . . . 166
38.9 Security of Diffie–Hellman: discrete logarithm and man-in-the-middle ★▸ . . . 168
38.10 Station-to-station protocol ▸ . . . 169

39 Public-Key Distribution, X.509 and PKI **171**
39.1 The public-key distribution problem; public announcement ▸ . . . 171
39.2 Trusted center and controlled trusted center ▸ . . . 171
39.3 Certification authority and certificates ▸ . . . 171
39.4 X.509 certificates ★▸ . . . 172
39.5 Certificate renewal and revocation ▸ . . . 173
39.6 Public-key infrastructure (PKI) ▸ . . . 173
39.7 Trust models and certificate chains ▸ . . . 174
39.8 KDC versus public-key distribution ▸ . . . 175

40 Entity Authentication **175**
40.1 What entity authentication is ▸ . . . 175
40.2 Verification categories and multi-factor authentication ▸ . . . 176
40.3 Fixed passwords: the four approaches ▸ . . . 176
40.4 One-time passwords and Lamport's hash chain ★▸ . . . 178
40.5 Challenge–response authentication ▸ . . . 180
40.6 Zero-knowledge authentication: the idea ▸ . . . 183

40.7 Fiat–Shamir protocol ★▸ . . . 183
40.8 Feige–Fiat–Shamir protocol ▸ . . . 185
40.9 Guillou–Quisquater protocol ▸ . . . 186
40.10 Biometrics ▸ . . . 187

VI Beyond the Core 189

41 Other Cryptographic Techniques: DNA, Quantum, Information Hiding 189
41.1 DNA cryptography: why use DNA at all ▸ . . . 190
41.2 DNA structure and complementary base pairing ▸ . . . 190
41.3 Fundamentals of DNA computing ▸ . . . 190
41.4 Encoding binary data and ASCII text as DNA ★▸ . . . 191
41.5 The DNA cryptography algorithm (slides 7–11) ★▸ . . . 192
41.6 Quantum cryptography: security from physics ▸ . . . 194
41.7 Properties of quantum information ▸ . . . 194
41.8 BB84 quantum key distribution ★▸ . . . 195
41.9 Why eavesdropping is detected: the intercept–resend attack ★▸ . . . 196
41.10 Quantum algorithms: Shor and Grover ★▸ . . . 196
41.11 Post-quantum cryptography ▸ . . . 197
41.12 Challenges of quantum cryptography ▸ . . . 198
41.13 Information hiding: what Lecture 18 adds to steganography ▸ . . . 198
41.14 Steganography in digital media ▸ . . . 199
41.15 Steganography techniques ★▸ . . . 199
41.16 Cryptography versus information hiding ★▸ . . . 201
41.17 Digital watermarking ★▸ . . . 202

42 Network Security Protocols and System Security 204
42.1 Where security can be placed in the stack ▸ . . . 204

43 E-mail Security: PGP and S/MIME 205
43.1 Why e-mail needs its own security ▸ . . . 205
43.2 PGP: the five services ★▸ . . . 205
43.3 PGP: the order of operations and why ★▸ . . . 206
43.4 PGP: key IDs and key rings ▸ . . . 207
43.5 PGP: the web of trust ▸ . . . 207
43.6 S/MIME and MIME ▸ . . . 208
43.7 PGP versus S/MIME ▸ . . . 209

44 IP Security (IPSec) 209
44.1 Why security at the IP level; applications and services ▸ . . . 209
44.2 Security associations, SAD and SPD ★▸ . . . 209
44.3 Transport mode and tunnel mode ★▸ . . . 210
44.4 Authentication Header (AH) ★▸ . . . 210
44.5 Anti-replay service ▸ . . . 211
44.6 Encapsulating Security Payload (ESP) ★▸ . . . 212
44.7 AH versus ESP ▸ . . . 212
44.8 Combining security associations ▸ . . . 213
44.9 Key management: Oakley, ISAKMP and IKE ★▸ . . . 213

45 Web Security: SSL/TLS, HTTPS and SET 214
45.1 Web security threats and the SSL/TLS architecture ★▸ . . . 214
45.2 The Record Protocol ★▸ . . . 215
45.3 Change Cipher Spec and Alert protocols ▸ . . . 215
45.4 The Handshake Protocol ★▸ . . . 216
45.5 Master secret and key derivation ▸ . . . 217

45.6 From SSL to TLS 1.3 ▸ . . . 217
45.7 HTTPS ▸ . . . 218
45.8 SET: purpose and participants ▸ . . . 218
45.9 SET: the dual signature ★▸ . . . 218
45.10 SET: purchase request and payment authorisation ▸ . . . 219

46 System Security: Intruders, Malicious Software, Firewalls and Trusted Systems 220
46.1 Intruders ▸ . . . 220
46.2 Intrusion detection ▸ . . . 220
46.3 Password management ▸ . . . 221
46.4 Malicious software: taxonomy ★▸ . . . 222
46.5 Viruses: phases, structure and types ★▸ . . . 222
46.6 Worms, bots and ransomware ▸ . . . 223
46.7 Virus countermeasures ▸ . . . 224
46.8 Firewalls: design goals and control techniques ★▸ . . . 224
46.9 Types of firewall ★▸ . . . 224
46.10 Bastion host and firewall configurations ★▸ . . . 225
46.11 Trusted systems: Bell–LaPadula and the reference monitor ★▸ . . . 226

47 Assignment Topics: Primality, Factoring, RSA Attacks, OAEP, Hash Families, Real-World Protocols 228

48 The AKS Primality Test 228
48.1 The problem and the theorem behind AKS ▸ . . . 228
48.2 The AKS algorithm (pseudo code) ★▸ . . . 229
48.3 Complexity, and why Miller–Rabin is used instead ▸ . . . 230

49 Factorization Algorithms 230
49.1 Measuring the cost: input size and L-notation ▸ . . . 230
49.2 Trial division and Fermat's method: when each is fast . . . 230
49.3 Pollard's $p - 1$ method ★▸ . . . 231
49.4 Pollard's rho method ★▸ . . . 232
49.5 The quadratic sieve ▸ . . . 233
49.6 The number field sieve ▸ . . . 233
49.7 Comparison and implications for RSA key sizes ▸ . . . 234

50 Attacks on RSA: Can Any of Them Succeed? 234
50.1 The question, the toy key and the reference key ▸ . . . 234
50.2 Attack 1: factoring n . . . 234
50.3 Attack 2: common modulus ★▸ . . . 235
50.4 Attack 3: Håstad's broadcast attack (low encryption exponent) ★▸ . . . 235
50.5 Attacks 4 and 5: short messages and plaintext–key relations ▸ . . . 236
50.6 Attack 6: the cycling attack ▸ . . . 236
50.7 Attack 7: Wiener's low private exponent attack ▸ . . . 236
50.8 Attack 8: chosen-ciphertext attack (multiplicative property) ▸ . . . 237
50.9 Attack 9: timing and other implementation attacks ▸ . . . 237
50.10 Verdict: no attack succeeds against correctly used RSA ★▸ . . . 238

51 Optimal Asymmetric Encryption Padding (OAEP) 238
51.1 Why textbook RSA must be padded ▸ . . . 238
51.2 OAEP encoding and decoding ★▸ . . . 238
51.3 What OAEP achieves ▸ . . . 239

52 Hash Function Families: MD, SHA-1, SHA-2, SHA-3 and Others 240
52.1 MD4 and MD5 ▸ . . . 240
52.2 SHA-0 and SHA-1 ▸ . . . 240
52.3 The SHA-2 family ▸ . . . 241

52.4 SHA-3 (Keccak) and the sponge construction ▸ . . . 241
52.5 RIPEMD-160, Whirlpool and BLAKE2 . . . 242
52.6 Comparison of the hash families ★▸ . . . 242

53 WhatsApp End-to-End Encryption (the Signal Protocol) 243
53.1 The problem and the keys ★▸ . . . 243
53.2 X3DH: agreeing on a key with an offline user ★▸ . . . 243
53.3 The Double Ratchet: a fresh key for every message ★▸ . . . 244
53.4 Groups, attachments and verifying keys . . . 245

54 Internet Banking and UPI Security 246
54.1 Threats and the controls of internet banking ▸ . . . 246
54.2 UPI: architecture and the path of a payment ★▸ . . . 246
54.3 Why the UPI design is secure ▸ . . . 247
54.4 Frauds and countermeasures ▸ . . . 247
54.5 Model answer: which UPI app is safer, and why? ▸ . . . 248

VII Revision 248

55 Formula and Algorithm Sheet 249

1 What to Study First (End-Sem)

1.1 What the end-sem covers

The end-sem paper covers the whole course: the eighteen lecture decks of Dr. J Sathish Kumar (CSN15403) and the official syllabus behind them. Lectures 1–14 were the mid-sem syllabus; Lectures 15–18 (message authentication and digital signatures, key management, entity authentication, and other techniques such as DNA and quantum cryptography) have never been examined, so they are the most likely source of new long questions. The official syllabus also names topics that the slides only touch or skip, namely block-cipher modes of operation, triple DES, MD5 and the SHA family, Kerberos and X.509, PGP and S/MIME, IPSec, SSL/TLS, SET, intruders, viruses, firewalls and trusted systems. These are taught in full in §32, §38 and §42, so that a question taken from the syllabus rather than from the slides is still covered. Finally, the teacher set eight assignment topics (WhatsApp end-to-end encryption, internet banking and UPI security, the AKS test, factorization methods, attacks on RSA, OAEP, other hash algorithms and attacks on signature schemes); a teacher who sets an assignment usually asks about it, so they have their own chapter, §47.

1.2 What the past papers say about the examiner

Both mid-sem papers (2024–25 and 2025–26, solved in the Question Bank) were taken almost word for word from the two standard textbooks: **Forouzan** (*Cryptography and Network Security*) and **Stallings**. The slides are mostly Forouzan's figures, so the notes teach the textbook method wherever a slide is thin, and the worked examples are phrased the way those exercises are phrased. The instruction "precise answer with proper justification will be considered to get complete marks" means that every numerical must show its full working (the extended-Euclid table, the square-and-multiply trace, the signature verification with both sides computed) and every protocol answer should have a diagram with numbered messages.

	Topic asked in the mid-sem papers	2024–25	2025–26	Where
★★	Playfair encryption	3	4	§12
★★	GF(2^8) inverse by the extended Euclidean algorithm	4	4	§28
★★	Extended Euclid: integer inverse, Diophantine equation	5	3	§24
★	One-time-pad version of Vigenère	–	8	§14, §15
★	AES: ShiftRows and AddRoundKey on a given state	–	6	§31
★	DES: Feistel structure, round function, key generation	5	–	§30
★	GF(p) division, Fermat exponent reduction	3	–	§26
★	Keyed transposition, attack types, cipher properties	5	–	§17, §6

Three patterns are worth carrying into the end-sem. Each paper had one block-cipher question (DES one year, AES the other), so expect one again, possibly on modes of operation this time. The extended Euclidean algorithm appeared twice in each paper, and in the end-sem it reappears inside RSA key generation, ElGamal and DSS signatures and the CRT, so it is still the single most reusable skill. Finally, classical ciphers carried a large share of the mid-sem marks; in an end-sem they are usually reduced to one short question.

1.3 What is most likely after the mid-sem ▸

The prediction below comes from the syllabus, the slides and the textbook exercises, not from an end-sem paper (none was available), so treat it as a ranking rather than a promise.

	Likely question	Where
★▸	RSA, ElGamal and DSS digital signatures: generate keys, sign, verify (small numbers); prove verification works	§37
★▸	Diffie–Hellman key agreement with numbers, the man-in-the-middle attack, station-to-station	§38
★▸	MAC vs MDC vs signature; HMAC and CMAC with diagrams	§36
★▸	Kerberos (draw and explain), KDC protocols, X.509 certificates and PKI	§38
★▸	Passwords (hashing, salting, Lamport one-time passwords), challenge–response, Fiat–Shamir zero knowledge, biometrics	§40
▸	RSA numericals, CRT, Miller–Rabin, ElGamal/ECC point arithmetic, birthday attack	§33–§35
▸	DNA cryptography, BB84 quantum key distribution, steganography vs watermarking	§41
▸	Modes of operation (ECB, CBC, CFB, OFB, CTR), double/triple DES	§32
▸	PGP, IPSec (AH vs ESP, transport vs tunnel), SSL/TLS handshake, SET dual signature, firewalls	§42
▸	Assignment topics: AKS, Pollard $p-1$ and rho, attacks on RSA, OAEP, SHA family, WhatsApp, UPI	§47

1.4 Study order

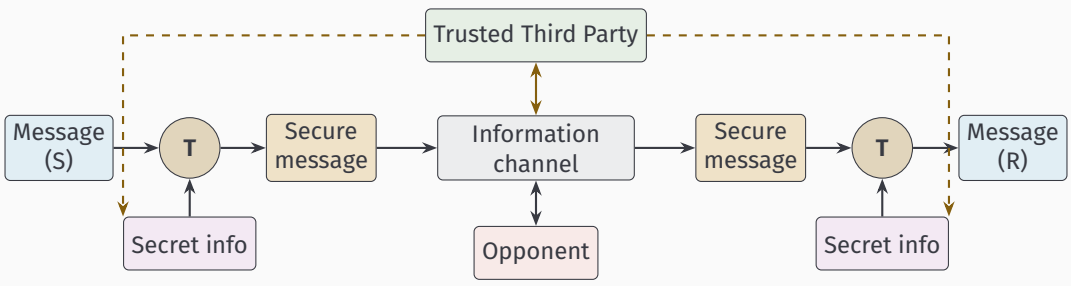
The order puts the most marks per hour first and reuses earlier skills in later topics.

1. **Number-theory toolkit** (§22, §24, §26, §27). Be fluent with square-and-multiply, the extended-Euclid table, Fermat/Euler exponent reduction and the CRT table; every public-key numerical below is built from these four.
2. **Public-key encryption** (§33, §34). RSA key generation, encryption and decryption; ElGamal; elliptic-curve point addition and doubling.
3. **Hash, MAC and signatures** (§35, §36, §37). The three resistance properties and the birthday bound, then HMAC/CMAC, then each signature scheme with a worked example and its proof of correctness.
4. **Key management and entity authentication** (§38, §40). Practise drawing every protocol from memory with numbered messages, and say in one line what each nonce, timestamp or signature prevents.
5. **Block ciphers** (§29–§32). DES round and key schedule, AES round operations, modes of operation.
6. **Remaining syllabus** (§41, §42, §47). Read once for theory questions; learn the comparison tables.
7. **Classical ciphers** (Part I). One pass of Playfair, Vigenère/one-time pad, Hill and transposition, since one short question is still likely.
8. Finally, the formula sheet (§55) the night before, and the timed mock end-sem paper in the Question Bank.

Part I
Foundations and Classical Cryptography

2 Model for Network Security

Two parties, a sender S and a receiver R, want to exchange messages over an insecure communication facility such as the Internet. The parties are called **principals**. Computer networking is concerned with getting the message from S to R; network security is concerned with what can go wrong on the way, because the path passes through links and routers that neither principal controls. Anyone with access to the path, called the **opponent** (eavesdropper, sniffer, attacker), can read, copy, change or block what is sent. The model below shows the general pattern that every protection scheme in this course follows.



The transformation and the secret. Instead of sending the plaintext message, S passes it through a **security-**

related transformation T. The transformation has two inputs: the message and a piece of **secret information** (normally a **key**, a string of bits that selects one particular transformation out of a whole family). Its output is the **secure message**, for example a ciphertext. At the other end R applies the complementary transformation with its own copy of the secret information and gets the original message back. Stallings sums up the model in one sentence: every technique for providing security has two components, a security-related transformation of the information to be sent and some secret information shared by the two principals.

Why T and not E. The box is labelled T rather than E (encryption) because encryption is only one kind of transformation. **Steganography**, which hides the message inside an innocent-looking carrier such as an image or an audio file so that its very existence is concealed, is another; appending an authentication code or a signature to the message is a third. The model covers all of them.

What the opponent can do. The opponent sits on the channel and can intercept the secure message, but since it was transformed with a secret he does not have, he cannot recover the content. Security therefore does *not* come from hiding how T works: the algorithms are public and standardised. It comes from two things: the strength of the algorithm, and the secrecy and length of the key. If the key can take only a few values, the opponent simply tries all of them (a brute-force attack, §10); every extra key bit doubles his work.

The trusted third party. A **trusted third party** (TTP) is an entity that both principals trust and that the opponent cannot impersonate. It may be needed to *distribute* the secret information to S and R while keeping it from the opponent (a key distribution centre), to *certify* public keys, or to *arbitrate* disputes between the principals about whether a message was really sent or received. A familiar example is the certificate authority (CA) that signs a bank's SSL/TLS certificate: when a customer opens the bank's website, the browser trusts the bank's public key only because a CA it already trusts vouches for it. The details come in key management (§38).

2.1 Four tasks in designing a security service

Each component of the model turns into one design task. A transformation is needed, so an algorithm must be designed; the algorithm needs secret information, so keys must be generated; both principals need the same (or matching) secret information, so it must be distributed; and the principals must use all of this in the same way, so a protocol must be fixed.

★ MUST KNOW : the four tasks (Stallings)

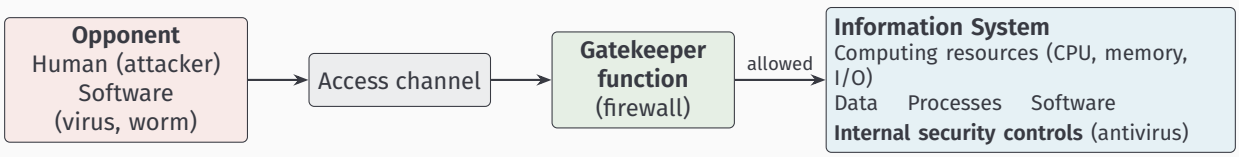
1. **Design an algorithm** for performing the security-related transformation (encryption at S, decryption at R), such that an opponent cannot defeat its purpose.
2. **Generate the secret information** to be used with the algorithm: keys large and random enough that guessing or exhaustive search is infeasible.
3. **Develop methods for the distribution and sharing** of the secret information, so that S and R obtain it without the opponent learning it.
4. **Specify a protocol** to be used by the two principals that makes use of the algorithm and the secret information to achieve a particular security service (e.g. SSL/TLS, IPsec).

Memory aid: *Algorithm → Key → Distribute → Protocol*.

Task 1 is where cryptographic design happens (the ciphers of the coming chapters). Task 2 is harder than it looks, because a key produced by a predictable process (a timestamp, a weak random-number generator) shrinks the effective key space no matter how long the key is. Task 3 is the *key distribution problem*: in symmetric cryptography both principals need the same key, and sending it over the channel it is meant to protect defeats the purpose; it is solved by a TTP, by public-key methods or by Diffie–Hellman. Task 4 exists because a strong algorithm with good keys can still be used insecurely: the protocol fixes the message formats, the order of messages and the checks each party performs, which is what SSL/TLS and IPsec standardise.

3 Network Access Security Model

The first model protects a *message in transit*. A second class of threats targets an *information system* itself: an opponent tries to get into a computer system through the network, to read or change the data stored there or to disrupt its service. Stallings divides such threats into **information access threats**, which intercept or modify data on behalf of users who should not have access to it, and **service threats**, which exploit flaws in services to stop legitimate users from using them.



The **opponent** may be a human (a hacker who breaks in for the challenge, a criminal after money, a disgruntled

ex-employee) or software (viruses, worms, Trojan horses), which may arrive over the network or on a USB stick. It reaches the system through the **access channel**, normally the Internet. The **information system** contains the assets to be protected: computing resources (processor, memory, I/O), data, running processes and installed software.

Defence is placed in two layers. The **gatekeeper function** is the first line, like a guard at a gate: it examines everything that comes in through the access channel and admits only what is allowed. Firewalls, packet filters, password-based login procedures and screening logic that rejects worms and viruses all belong here. Since no gate is perfect and some attacks start inside, the second line consists of **internal security controls**, which run inside the system and watch its activity and stored information to detect and contain whatever got past the gate: antivirus software, file-integrity checkers, intrusion detection and operating-system access permissions.

Insiders. An employee who misuses legitimate access never passes the gatekeeper, so only the internal controls can catch him. Insider attacks can do great damage, but the model is drawn mainly for the outside opponent, who must come through the access channel.

	Model for network security	Network access security model
Protects	a message while it travels from *S* to *R*	an information system and the resources inside it
Opponent	on the channel, between the principals	outside the system, trying to get in (or an insider)
Defence	security-related transformation with secret information (encryption, MAC, signature)	gatekeeper (firewall, login) plus internal security controls (antivirus, IDS)
Extra party	trusted third party for keys and disputes	none required

4 Cryptography

The Concise Oxford English Dictionary defines cryptography as “the art of writing or solving codes”. The definition used in this course (Stallings, quoted in the lecture) is more precise:

> Cryptography is the art or science encompassing the principles and methods of transforming an intelligible message into one that is unintelligible, and then retransforming that message back to its original form.

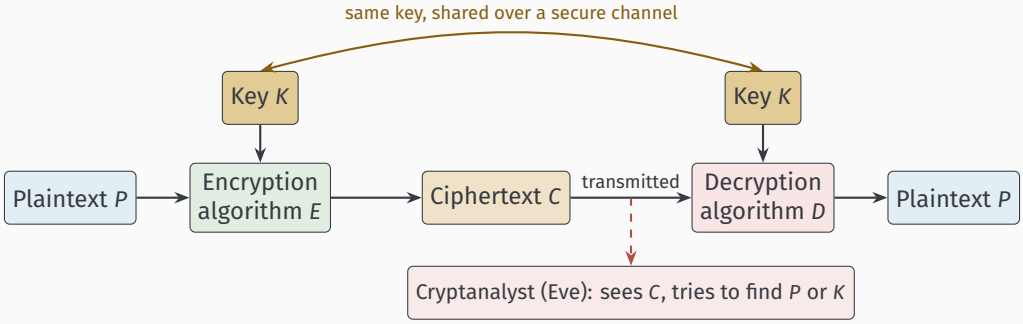
Intelligible means readable by anyone; **unintelligible** means meaningless to anyone who does not hold the key; **retransforming** means the process must be reversible by the intended receiver. The word “art” reflects that classical ciphers were designed by ingenuity; modern cryptography is a science built on mathematics and complexity theory.

Terminology. The original message is the **plaintext** (clear text) and the coded message is the **ciphertext**. Converting plaintext to ciphertext is **enciphering** or **encryption**; recovering the plaintext is **deciphering** or **decryption**. The scheme used is a **cryptographic system** or **cipher**, and the **key** is the input to the cryptographic algorithm that selects the particular transformation. The slides contrast **cryptography**, the science of creating coded messages, with **cryptanalysis**, the art of breaking them, i.e. recovering the plaintext or the key without being told the key. The two together form **cryptology**.

Scope today. Cryptography now covers much more than secret writing: mechanisms for ensuring integrity, techniques for exchanging secret keys, protocols for authenticating users, electronic auctions and elections, and digital cash. Forouzan groups the tools into three mechanisms: **symmetric-key encipherment**, **asymmetric-key encipherment** and **hashing**.

4.1 Symmetric cipher model

A symmetric (conventional, single-key) cipher has five ingredients (Stallings): the **plaintext**; the **encryption algorithm**, which performs substitutions and transformations on the plaintext; the **secret key**, also an input to the encryption algorithm, which determines the exact substitutions and transformations; the **ciphertext**, which depends on both the plaintext and the key, so that two different keys give two different ciphertexts for the same message; and the **decryption algorithm**, which is the encryption algorithm run in reverse with the same key.



$$C = E(K, P) \qquad P = D(K, C) = D\big(K, E(K, P)\big)$$

Both algorithms take **two** inputs: encryption takes P and K, decryption takes C and K. The equation on the right is the correctness requirement: D with key K must be the inverse of E with key K, so that applying one after the other returns the original input.

Requirements for secure use. Stallings states two. First, the encryption algorithm must be strong: an opponent who knows the algorithm and holds many ciphertexts, even together with their plaintexts, must be unable to decrypt other ciphertexts or to find the key. Second, sender and receiver must have obtained their copies of the key in a secure fashion and must keep it secure. The algorithms E and D are **public**; only K is secret (Kerckhoff's principle, §6). This is what makes symmetric encryption practical: since the algorithm need not be hidden, manufacturers can build cheap chips implementing it, and the only thing that must be protected is the key.

The key is everything. If the opponent recovers one plaintext, he has read one message. If he recovers the key, he can read *every* past and future message encrypted under it, which is why cryptanalysts aim at the key. In the lecture's toy example both ends use $K = 100$; such a key would be found instantly. Real symmetric keys are 128 or 256 bits long (2048 bits is the size of an RSA modulus, a different kind of key), so exhaustive search is out of reach (§5).

4.2 Types of cryptography

Cryptosystems are classified by the number of keys. In **symmetric-key** (private-key, secret-key, conventional) cryptography one secret key, known only to S and R, is used both to encrypt and to decrypt. In **asymmetric-key** (public-key) cryptography each user has a **key pair**: one key encrypts and a different but mathematically related key decrypts. One key of the pair is published (the **public key**) and the other is kept by its owner (the **private key**). The lecture's toy question, "can we encrypt with 100 and decrypt with 500?", has the answer *yes*, provided the two numbers satisfy a mathematical relationship that makes decryption undo encryption, and provided the private key cannot be computed from the public one in practice (RSA, §33, is the standard example).

	Symmetric (private / secret key)	**Asymmetric** (public key)
Keys	One shared key	Key pair: public + private
Use	Same key encrypts and decrypts	One key encrypts, the other (mathematically related) decrypts
Secrecy	Key known only to S and R	Public key published; private key kept by owner
Toy example	Encrypt 100, decrypt 100	Encrypt 100, decrypt 500
Based on	substitution and permutation of symbols	number theory (factoring, discrete logarithms)
Speed and use	fast; bulk data	slow; key exchange, signatures

Three dimensions (Stallings). A cryptographic system is characterised along three independent dimensions. (1) The **type of operation** used to turn plaintext into ciphertext: *substitution*, in which each element is mapped to another element, and *transposition*, in which elements are rearranged; no information may be lost, so that the operations can be undone, and most real systems, called *product systems*, use several stages of both. (2) The **number of keys**: one for symmetric, two for asymmetric. (3) The **way the plaintext is processed**: a *block cipher* processes one block of elements at a time and produces an output block for each input block, while a *stream cipher* processes the input continuously, one element at a time (§29).

5 Security of Encryption Schemes

Calling a cipher "secure" needs a definition: secure against an opponent with how much ciphertext, how much computing power and how much time? Two standards are used.

Unconditional security. An encryption scheme is unconditionally secure if the ciphertext generated by the scheme does not contain enough information to determine uniquely the corresponding plaintext, no matter how much ciphertext is available. The guarantee holds against an opponent with unlimited time and computing power, because the obstacle is not the amount of work but the absence of information: every candidate plaintext remains possible and equally likely after the ciphertext is seen. Shannon (1949) called this **perfect secrecy**.

A one-letter example shows how this is possible. Encrypt a single letter p with a shift cipher, $C = (p + k) \bmod 26$, where the key k is chosen uniformly at random from $0, \dots, 25$ and used only for this letter. For any ciphertext letter c and *any* candidate plaintext p there is exactly one key that maps p to c, namely $k = (c - p) \bmod 26$. Each candidate therefore has the same probability $1/26$ of producing the observed c, so seeing c tells the opponent nothing about p, and trying all 26 keys just lists all 26 letters. Extending this to a long message with a fresh random key for every letter gives the **one-time pad** (§15), the only practical unconditionally secure cipher. The price is visible in the argument: the key must be truly random, as long as the message, and never reused, which makes key distribution as hard as sending the message itself.

Computational security. Practical ciphers reuse a short key for long messages, so the ciphertext *does* determine the plaintext in principle; the opponent's problem is only the amount of work. Such a scheme is called

computationally secure if it meets either of two criteria:

★ MUST KNOW : the two definitions

Unconditionally secure: the ciphertext does not contain enough information to determine the plaintext uniquely, no matter how much ciphertext is available (holds even against unlimited computing power; example: one-time pad).

Computationally secure: either (1) the **cost** of breaking the cipher exceeds the **value** of the encrypted information, or (2) the **time** required to break the cipher exceeds the **useful lifetime** of the information.

The cost criterion says that breaking is pointless when it is unprofitable: spending $1,000,000 of computing to steal a $1,000 transfer is a loss. The time criterion says that breaking is pointless when it is too late: a price quote valid for 10 minutes that takes 50 years to crack is worthless when cracked. Almost all ciphers in use (AES, RSA) are only computationally secure, since unconditional security needs a key as long as all the data ever sent.

Estimating the work: brute force. The one attack that works against every cipher is **exhaustive key search**: try every key until the output is meaningful plaintext. If there are N keys and the correct one is equally likely to be any of them, the attacker finds it after $1, 2, \ldots, N$ trials with equal probability, so the expected number of trials is $(1 + 2 + \cdots + N)/N = (N + 1)/2 \approx N/2$. A b-bit key gives $N = 2^b$, so each extra bit doubles the work. The table gives the average search time, computed by script, at 10^9 decryptions per second (one fast processor) and 10^{13} per second (a large parallel machine).

Key	Number of keys N	Time at 10^9/s	Time at 10^{13}/s
Shift cipher (Caesar family)	25	1.3×10^{-8} s	1.3×10^{-12} s
56 bits (DES)	$2^{56} \approx 7.2 \times 10^{16}$	1.14 years	1.0 hour
128 bits (AES)	$2^{128} \approx 3.4 \times 10^{38}$	5.4×10^{21} years	5.4×10^{17} years
168 bits (triple DES)	$2^{168} \approx 3.7 \times 10^{50}$	5.9×10^{33} years	5.9×10^{29} years
256 bits (AES)	$2^{256} \approx 1.2 \times 10^{77}$	1.8×10^{60} years	1.8×10^{56} years
26-letter permutation (monoalphabetic)	$26! \approx 4.0 \times 10^{26}$	6.4×10^{9} years	6.4×10^{5} years

The universe is about 1.4×10^{10} years old, so a 128-bit key meets the time criterion for any information whatsoever, while a 56-bit key no longer does. Two cautions follow. Brute force also needs the attacker to *recognise* the correct plaintext; this is easy for English text but harder if the plaintext was compressed or is random binary data. And a large key space is necessary but not sufficient: the monoalphabetic cipher has 26! keys yet falls in minutes to frequency analysis, which never tries keys at all. Computational security is therefore never proved; a cipher is accepted as computationally secure when the best *known* attack is still too expensive.

	Unconditionally secure	Computationally secure
Opponent assumed	unlimited time and computing power	realistic resources and time
Why it cannot be broken	ciphertext carries no information about the plaintext	breaking costs more or takes longer than the information is worth
Key requirement	random, as long as the message, used once	short key (128–256 bits) reused for many messages
Proof	information-theoretic (Shannon)	none; based on the best known attack
Example	one-time pad	AES, RSA, DES (in its time)

Trap: what "secure" does and does not mean

The Caesar cipher is neither: it is broken in seconds (§10). Computationally secure does not mean "unbreakable"; it means "not worth breaking" or "not breakable in time", and it can be lost when computers get faster (DES) or a better attack is found. Write both criteria of computational security, joined by *either*, and use the words cost/value and time/useful lifetime.

6 Security Goals, Attacks and Cryptanalysis

Information is an asset with a value like any other asset, and like any other asset it must be protected. Lecture 3 builds the subject in Forouzan's order: what we want to achieve (the **security goals**), what can go wrong (the **attacks**), and what we use against it (the **services** and **mechanisms** of ITU-T X.800). The last part of this section turns to attacks on the cipher itself (**cryptanalysis**).

6.1 Security trends

The lecture opens with the yearly reports of CERT-In (the Indian Computer Emergency Response Team) to show that the threat is real and growing. The number of incidents it handled grew about 26-fold between 2017 and 2021, and the mix changed: in 2017 more than half of the incidents were website defacements, while from 2018 onwards

unauthorised network scanning, probing and exploitation of vulnerable services dominate. Most incidents are thus automated reconnaissance and malware, not the breaking of ciphers.

Year	Total	Scanning, probing, vuln. services	Virus, malicious code	Website defacements	Phishing
2017	53 117	9 383 (17.7%)	9 750 (18.4%)	**29 518 (55.6%)**	552
2018	208 456	**127 481 (61.2%)**	61 055 (29.3%)	16 655 (8.0%)	454
2019	394 499	**305 276 (77.4%)**	62 163 (15.8%)	24 366 (6.2%)	472
2020	1 158 208	**1 028 881 (88.8%)**	99 986 (8.6%)	25 969 (2.2%)	280
2021	1 402 809	**1 160 333 (82.7%)**	209 110 (14.9%)	27 408 (2.0%)	523

Remaining incidents: website intrusion and malware propagation, and "others". The 2021 slide lists scanning/probing (432 057) and vulnerable services (728 276) separately; they are added here. Every row was checked to sum to its total.

6.2 Security goals (CIA)

To be secure, information must be hidden from unauthorised access (**confidentiality**), protected from unauthorised change (**integrity**), and available to an authorised entity when it is needed (**availability**). These three goals are the CIA triad, and every attack below is classified by the goal it threatens.

Confidentiality. Confidentiality means that information is concealed from everyone who is not authorised to see it. It is the oldest goal and the one most people associate with security. In the military, concealment of sensitive information is the main concern; in industry, hiding information from competitors is vital; in banking, customers' accounts must be kept secret. Confidentiality applies both to *storage* and to *transmission*: when a piece of information is stored on or retrieved from a remote computer, it must be concealed while it travels.

Integrity. Information changes constantly: when a customer deposits or withdraws money, her balance must change. Integrity means that changes are made only by authorised entities and through authorised mechanisms. A violation need not be malicious; an interruption in the system, such as a power surge, can also corrupt information. Integrity is therefore about *detecting* unwanted change as much as preventing it.

Availability. Information that cannot be reached is useless. Availability means that information is accessible to authorised entities when it is needed. Since information is constantly updated, it must stay accessible to those allowed to update it, and its unavailability is as harmful as a loss of confidentiality or integrity: a bank whose customers cannot reach their accounts cannot do business.

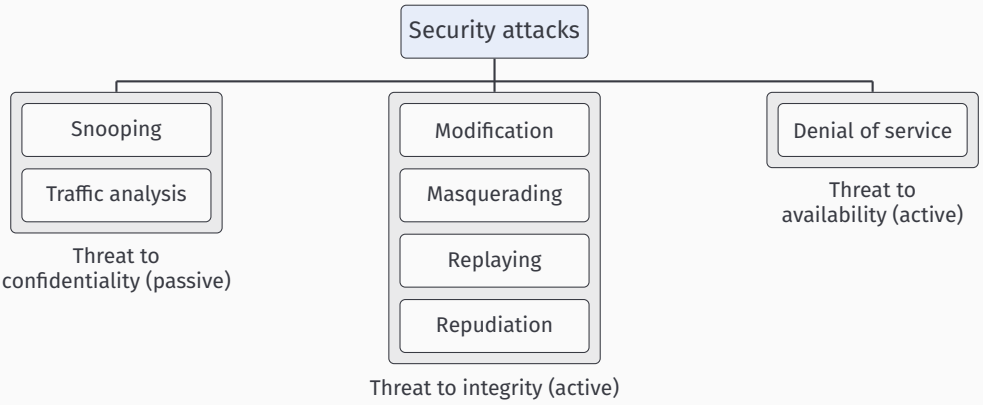
Confidentiality	hide information from unauthorised access, in storage *and* in transmission (military secrets, bank accounts).
Integrity	changes only by authorised entities through authorised mechanisms. Violation need not be malicious (a power surge can corrupt data).
Availability	information must be accessible to authorised entities when needed; unavailability is as harmful as a breach.

Stallings adds two goals that are often listed with the triad: **authenticity**, being able to verify that users are who they claim to be and that each message came from a trusted source, and **accountability**, being able to trace every action to the entity responsible (which is what non-repudiation supports).

6.3 Attacks (Forouzan's taxonomy) ★

▸ **Likely this year:** "Explain the security attacks that threaten confidentiality, integrity and availability" or "differentiate passive and active attacks with examples" (4–5 marks).

Forouzan groups the attacks by the goal they threaten. Two threaten confidentiality, four threaten integrity and one threatens availability.



Attacks threatening confidentiality.

- **Snooping** is unauthorised access to or interception of data. A file sent over the Internet may contain confidential information; an unauthorised entity intercepts the transmission and uses the contents for her own benefit. Snooping is prevented by making the data unintelligible to the interceptor with *encipherment*.
- **Traffic analysis** works even when the data is enciphered. By monitoring the traffic, the attacker learns the electronic addresses (e.g. e-mail addresses) of sender and receiver, the times, sizes and frequency of messages, and pairs of requests and responses, which help her guess the nature of the transaction. A burst of messages between a company and a law firm can reveal a takeover without a single message being read. Encryption does not stop it; traffic padding and routing control (below) do.

Attacks threatening integrity.

- **Modification**: after intercepting or accessing information, the attacker changes it to benefit herself. A customer sends her bank a transaction request; the attacker intercepts it and changes the type of transaction or the beneficiary. Simply *deleting* or *delaying* the message, to harm the system or profit from it, also counts as modification.
- **Masquerading** (spoofing) happens when the attacker impersonates somebody else, for example by stealing a customer's bank card and PIN and pretending to be that customer. She may also pretend to be the *receiver*: a user tries to contact his bank, but another site poses as the bank and collects his information (phishing).
- **Replaying**: the attacker obtains a copy of a message sent by a user and later sends it again. A person asks her bank to pay the attacker for a job; the attacker intercepts the request and resends it to receive a second payment. The slide's figure makes a further point: if Alice logs in by sending hash(password), Eve can sniff the hash and replay it without ever learning the password. Replay is stopped by making every message fresh (a sequence number, timestamp or nonce), not by encryption or hashing, since the replayed copy is a perfectly valid message.
- **Repudiation** is different from all the others because it is performed by one of the two parties to the communication, not by an outsider. The sender may later deny that she sent a message (a customer orders her bank to transfer money and later denies having made the request), or the receiver may deny having received it (a manufacturer is paid electronically and later claims he never received the payment).

Attack threatening availability.

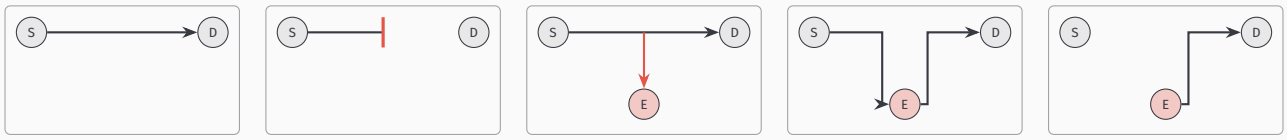
- **Denial of service** (DoS) slows down or totally interrupts the service of a system. The attacker may send so many bogus requests to a server that it crashes under the load; intercept and delete the server's responses so that the client believes the server is down; or intercept the clients' requests, so that the clients resend them many times and overload the system. When the flood comes from many machines at once (a botnet) it is a distributed DoS (DDoS).

Attack	Threatens	Type	What happens
Snooping	confidentiality	passive	unauthorised access/interception of data. Counter: encipherment.
Traffic analysis	confidentiality	passive	learning addresses, timing, request/response pairs from traffic even when it's encrypted.
Modification	integrity	active	intercept and change (or delete, delay) a message.
Masquerading (spoofing)	integrity	active	impersonating someone (stolen card + PIN; a fake bank site).
Replaying	integrity	active	capture a message and resend it later (a second payment).
Repudiation	integrity	active	a *party to the communication* denies having sent or received a message.
Denial of service	availability	active	slow down or stop the service (bogus requests; deleting responses).

Passive versus active attacks. In a **passive** attack the attacker's goal is only to obtain information: the attack does not modify data or harm the system, although the revealed information may harm the sender or receiver. Snooping and traffic analysis, the attacks on confidentiality, are passive. Since nothing in the system changes, there is nothing to notice, so a passive attack is *difficult to detect* until the victim finds out that confidential information has leaked. It can, however, be *prevented*, by encipherment of the data (plus padding against traffic analysis). An **active** attack changes the data or harms the system; the attacks on integrity and availability are active. Active attacks are normally *easier to detect than to prevent*: the change leaves evidence (a wrong digest, a duplicate sequence number, a server that stops answering), but the attacker can launch them in so many ways and places that blocking all of them in advance is impractical. The defensive goal is therefore to detect them and recover.

	Passive attack	Active attack
Goal	obtain information	change data or harm the system
Attacks	snooping, traffic analysis	modification, masquerading, replaying, repudiation, DoS
Threatens	confidentiality	integrity and availability
Detection	difficult (nothing changes)	relatively easy (effects are visible)
Strategy	prevent (encipherment, padding)	detect and recover (integrity checks, authentication, freshness)

Stallings' four-way picture (normal flow source → destination): **interruption** (availability), **interception** (confidentiality), **modification** (integrity), **fabrication** (authenticity / masquerade). The lecture shows the figure below: in each panel the information should flow from the source *S* to the destination *D*, and the third node is the attacker.



(a) Normal flow (b) Interruption (c) Interception (d) Modification (e) Fabrication

Interruption destroys the flow or makes it unusable (cutting a line, DoS), an attack on availability. **Interception** gives an unauthorised party access to the flow (wiretapping, copying files), an attack on confidentiality and the only passive one of the four. **Modification** lets the attacker capture and alter the flow before it reaches *D*, an attack on integrity. **Fabrication** inserts counterfeit messages that appear to come from *S*, an attack on authenticity (masquerade). In Stallings' own list of attacks the passive ones are *release of message contents* (Forouzan's snooping) and *traffic analysis*, and the active ones are *masquerade*, *replay*, *modification of messages* and *denial of service*.

Trap: repudiation is not an outsider attack

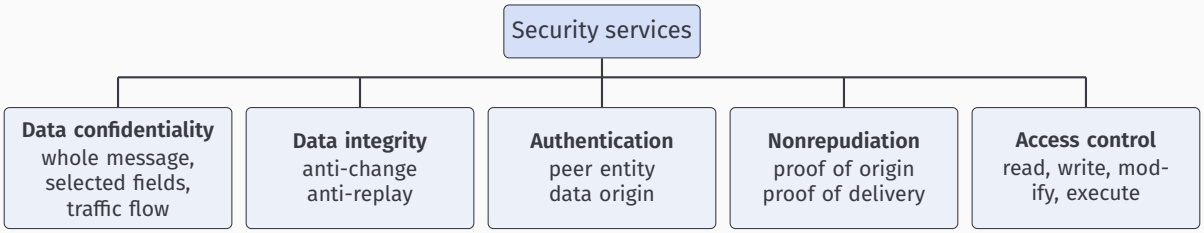
Every other attack is by a third party (Eve). Repudiation is done by Alice or Bob themselves. It is countered by digital signatures (non-repudiation), not encryption.

6.4 Security services and mechanisms (ITU-T X.800) ★

▸ **Likely this year:** "List the security services of X.800 and the mechanisms that provide each one" (Forouzan Table 1.1, 4–5 marks), or "which service/mechanism defeats replaying / repudiation / traffic analysis?"

The goals say what we want and the attacks say what can go wrong; a system designer also needs a standard vocabulary for the protection itself. The ITU-T (International Telecommunication Union, Telecommunication Standardization Sector) provides it in Recommendation **X.800**, the security architecture for OSI. It separates three notions (Stallings): a **security attack** is any action that compromises the security of information; a **security mechanism** is a process (or a device incorporating one) designed to detect, prevent or recover from an attack; and a **security service** is a processing or communication service that enhances the security of data processing and information transfer, is intended to counter attacks, and is implemented by one or more mechanisms. In short, a service is *what* is provided, a mechanism is *how* it is provided.

Security services. X.800 defines five services, two of which have two flavours each.



- **Data confidentiality** protects data from disclosure. It may cover the whole message or selected fields, and it includes protection against *traffic analysis* (traffic-flow confidentiality). It counters snooping and traffic analysis.
- **Data integrity** protects data from modification, insertion, deletion and replaying by an adversary. The *anti-change* part detects altered content; the *anti-replay* part detects an old valid message sent again.
- **Authentication** assures each party of the identity of the other. *Peer entity authentication* is used in connection-oriented communication: it authenticates the sender or receiver when the connection is established. *Data origin authentication* is used in connectionless communication: it authenticates the source of each piece of data. It counters masquerading.

- **Nonrepudiation** protects against a sender or receiver who later denies the communication. With *proof of origin*, the receiver can later prove the identity of the sender if she denies having sent the message; with *proof of delivery*, the sender can later prove that the data was delivered to the intended recipient. It counters repudiation.
- **Access control** protects against unauthorised access to data and resources; "access" covers reading, writing, modifying and executing programs. It counters intruders who try to use a system they are not entitled to.

Availability is not one of the five services. X.800 treats it as a property supported by other services (access control and authentication keep bogus users out) and by system management, which is why denial of service has no purely cryptographic cure.

Security mechanisms. X.800 also recommends eight mechanisms.

- **Encipherment** hides or covers data to provide confidentiality; it uses cryptography or steganography.
- **Data integrity**: a short check value, created by a specific process from the data itself, is appended to the data. The receiver computes a new check value from the received data and compares the two; any difference shows that the data was changed (hash, MAC, §35).
- **Digital signature**: the sender electronically signs the data with her private key and the receiver verifies the signature with the sender's public key. Since only the sender has the private key, the signature proves who signed and that the data has not changed since.
- **Authentication exchange**: two entities exchange messages to prove their identity to each other, for example by showing that one knows a secret that only it should know (§40).
- **Traffic padding** inserts bogus data into the traffic so that its volume and timing no longer reveal anything, which thwarts traffic analysis.
- **Routing control** selects, and keeps changing, the routes between sender and receiver so that the opponent cannot eavesdrop on one particular route.
- **Notarization** selects a trusted third party to control the communication between two entities. The notary records the request and the time, so that neither party can later deny it; this prevents repudiation.
- **Access control** uses methods (passwords, PINs, access lists) to prove that a user has the right to the data or resources of a system.

Relation between services and mechanisms. Each service is implemented by one or more mechanisms, and one mechanism can serve several services. Forouzan's Table 1.1 gives the relation; the last column links it back to the attacks.

Security service	Security mechanisms (Forouzan Table 1.1)	Main attacks countered
Data confidentiality	encipherment and routing control	snooping; traffic analysis (with traffic padding)
Data integrity	encipherment, digital signature, data integrity	modification; replaying (anti-replay)
Authentication	encipherment, digital signature, authentication exchange	masquerading
Nonrepudiation	digital signature, data integrity, notarization	repudiation
Access control	access control mechanism	unauthorised access to the system

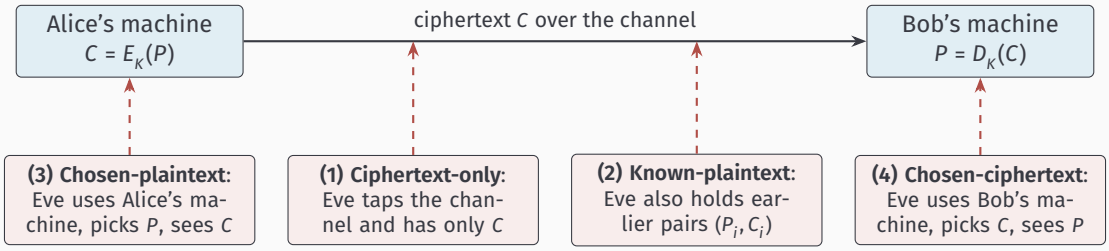
The entries can be derived from what each mechanism does. Encipherment appears three times because a message that decrypts correctly under a shared key also shows that it was produced by a key holder and not altered meaningfully. A digital signature appears three times because it binds the data to the one person who holds the private key: altering the data breaks the signature (integrity), a valid signature identifies the signer (authentication), and the signer cannot later deny it (nonrepudiation). Nonrepudiation needs more than integrity or encipherment with a *shared* key, since the receiver could have produced such a message himself; only a private-key signature or a notary provides evidence a third party will accept. Traffic padding is not listed in Forouzan's table, but it is the mechanism X.800 pairs with routing control for traffic-flow confidentiality.

6.5 Kerckhoff's principle and cryptanalysis attacks ★

★ **In the papers:** 2024–25 Q3(a): Eve types "abcdefghij" on Alice's computer and reads the output: which attack? (**chosen-plaintext**, since she chose the plaintext and saw its ciphertext).

Kerckhoff's principle. It may seem that a cipher would be more secure if both the algorithm and the key were hidden. Kerckhoff's principle says this is the wrong design target: one should always assume that the adversary, Eve, knows the encryption/decryption algorithm; the resistance of the cipher to attack must be based only on the secrecy of the key. Guessing the key should be so hard that there is no need to hide the algorithm. There are good reasons for this. Algorithms leak: they are built into products that can be bought and reverse-engineered, and every engineer who worked on them knows them. A published algorithm can be studied by the whole research community, so its weaknesses are found by friends rather than by enemies. And a key compromised can be replaced in minutes, while a compromised algorithm means rebuilding the whole system. The principle is clearest for modern ciphers: there are only a few standard algorithms (AES, RSA), but each has a key domain so large that the key cannot be found.

Cryptanalysis. Cryptography is the science of making secret codes; **cryptanalysis** is the science of breaking them, that is, recovering the plaintext or, better, the key without being given the key. Designers study it in order to measure how strong their ciphers are. The attacks are classified by *what Eve has* besides the algorithm, from least to most information.



Ciphertext-only attack. Eve has access only to some ciphertext and tries to find the key and the plaintext. She is assumed to know the algorithm and to be able to intercept ciphertext. This is the *most probable* attack, because intercepting ciphertext is all it needs, so a cipher must above all resist it. Three methods are used. In a **brute-force** (exhaustive-key-search) attack, Eve knows the algorithm and the key domain and decrypts the ciphertext with every possible key until the plaintext makes sense; it is prevented only by a very large key domain (§5). In a **statistical** attack, Eve uses inherent properties of the plaintext language: E is the most frequent letter in English, so she assumes the most frequent ciphertext character stands for E, and after a few such pairs she finds the key; it is prevented by ciphers that hide the characteristics of the language. In a **pattern** attack, Eve exploits patterns that a cipher leaves in the ciphertext even when it hides single-letter frequencies, so a good cipher must make its ciphertext look as random as possible.

Known-plaintext attack. Eve has some plaintext/ciphertext pairs, collected earlier, in addition to the ciphertext she wants to break. For example, Alice sent Bob a secret message but later made its contents public; Eve kept both and uses the pair to analyse Alice's next message, assuming Alice has not changed her key. The same methods as before apply, but with more information the attack is *easier*. It is *less likely*, because Alice may have changed her key or never disclosed an old message. The Hill cipher is the classic victim (§17).

Chosen-plaintext attack. The pairs are chosen by Eve herself. This can happen if Eve has access to Alice's computer: she types plaintext of her choice and intercepts the ciphertext it produces. She still does not have the key, which is normally embedded in the encryption software. Choosing the plaintext lets her design inputs that expose the key (all-a text, or the alphabet in order, which shows a transposition's permutation directly), so the attack is *much easier*, but *much less likely*.

Chosen-ciphertext attack. The mirror image: Eve chooses some ciphertext and has it decrypted, forming ciphertext/plaintext pairs. This can happen if Eve has access to Bob's computer. Public-key schemes must resist it, since textbook RSA does not (§33).

★ MUST KNOW : the four cryptanalysis attacks (Eve always knows the algorithm)

Ciphertext-only: only ciphertext (most likely; brute force, statistical, pattern). **Known-plaintext**: plus old (P, C) pairs under the same key. **Chosen-plaintext**: Eve picks P and sees C (access to Alice's machine). **Chosen-ciphertext**: Eve picks C and sees P (access to Bob's machine). Each step gives Eve more power and is less likely to happen.

Attack	Eve has	Note
Ciphertext-only	only ciphertext	most probable. Methods: brute force (key space must be huge), statistical (letter frequencies), pattern attacks.
Known-plaintext	earlier plaintext/ciphertext pairs (same key)	easier, less likely (key may have changed).
Chosen-plaintext	pairs for plaintexts she chose (access to Alice's machine)	much easier, much less likely.
Chosen-ciphertext	plaintexts for ciphertexts she chose (access to Bob's machine)	mirror of chosen-plaintext; must be resisted by public-key schemes.

To classify an attack in an exam, ask two questions: *whose machine does Eve use* (none, Alice's encryptor, Bob's decryptor), and *who picked the text* (nobody, Eve). Knowing old pairs without having chosen them is known-plaintext; choosing them is chosen-plaintext or chosen-ciphertext.

WORKED EXAMPLE : Mid-sem 2024–25 Q3(a)

Q. Eve secretly gets access to Alice's computer and, using its cipher, types "`abcdefghij`". The screen shows "CABDEHFGIJ". What type of attack is Eve launching?
A. Eve does not have the key, but she uses Alice's encryption machine, *chooses* the plaintext herself and observes the corresponding ciphertext. That is a **chosen-plaintext attack**. It is not known-plaintext, because the pair was not collected from earlier traffic but created by Eve; it is not chosen-ciphertext, because she used the encryptor, not Bob's decryptor. The choice of input is also typical: since the plaintext letters are all different and in alphabetical order, the ciphertext directly shows where each position went (here `abcde` → CABDE and `fghij` → HFGIJ, a keyed transposition with block size 5; part (b) is solved in the Question Bank).

6.6 History notes from the slides

Lecture 1 traces cryptography from hiding messages to the mathematics of today. The milestones, each with the idea it introduced:

- **3500 BC**, Sumerian cuneiform writing, and **1900 BC**, Egypt: the first known use of cryptography (non-standard hieroglyphs).
- **500–600 BC, Atbash** (Hebrew scribes): a substitution cipher using the reversed alphabet ($a \leftrightarrow z$, $b \leftrightarrow y$, ...). It has no key at all.
- **486 BC, scytale** (Greece): a strip of leather wound round a rod of fixed diameter, written along the rod and unwound; the letters are only rearranged, so it is a *transposition* cipher whose key is the rod's diameter.
- **60–50 BC, Caesar**: shift every letter by X positions ($X = 3$: A→D, B→E, ...), §8. Its weakness is *frequency analysis* (the slide dates it to about 1000 AD; the Arab scholar al-Kindi described it in the 9th century): a monoalphabetic substitution keeps the letter frequencies of the language, and there are only 25 shifts to search exhaustively. The slide's goal for designers follows: exhaustive key search is always possible, so it must be made practically infeasible.
- **1587, Vigenère**: a *polyalphabetic* cipher (one-to-many: the same plaintext letter becomes different ciphertext letters at different positions), §14. **1863, Kasiski** broke it: repeated strings in the ciphertext are usually a multiple of the key length apart, so the common divisor of their distances gives the key length m; the ciphertext is then written in m columns, each column is a Caesar cipher, and each is solved by frequency analysis.
- **1845, Morse code**: a representation of letters by on/off signal codes (an encoding, not a cipher, since there is no key).
- **1918, ADFGVX** (German army, WWI) and **1918, Enigma** (Arthur Scherbius, for confidential business documents and later the German forces in WWII): a rotor machine performing a different substitution for every letter, because the rotor wiring advances as you type, with no codebooks needed.
- **1949, Shannon**, "Communication Theory of Secrecy Systems": proved the one-time pad unbreakable (perfect secrecy, §5).
- **1970s, DES** (IBM, adopted as a US federal standard); **1976, Diffie–Hellman** key exchange, which solved the key-exchange problem and started public-key cryptography; **1977, RSA** (Rivest, Shamir, Adleman, MIT); **1985, ECC** (Miller at IBM and Koblitz at the University of Washington, independently); **1991, MD5** (Rivest); **2001, AES** (published 26 November 2001) and **SHA-512**.

ADFGVX (German army, WWI): a keyed 6 × 6 Polybius square (letters + digits, rows/columns labelled A D F G V X) followed by a **columnar transposition**. The square turns each character into a pair of the letters A, D, F, G, V, X (row label, column label), which were chosen because they are very different in Morse code; the transposition then separates the two halves of each pair, so that neither letter frequencies nor pairs survive. It combines substitution and transposition, the idea behind every modern block cipher.

	A	D	F	G	V	X
A	1	4	7	r	e	g
D	i	m	n	t	a	b
F	c	d	f	h	j	k
G	l	o	p	q	s	u
V	v	w	x	y	z	0
X	2	3	5	6	8	9

Square keyword `147regiment` (then the rest of a–z, 0–9). Encrypt `attackat1200am`: a → DV, t → DG, ... giving DVDGDGDVFAFXDVDGAAXAVXVXDVDD. Write it under the transposition key PRIVACY, read columns in alphabetical order of the key letters (A, C, I, P, R, V, Y):
DXXV GDAD DAAX DVDX VFGV GFAD DVVD (verified).

WORKED EXAMPLE : decryption, the slide's second example (ADFGX, keys `monkeys` and `zebras`)

Q. Decrypt ADDDF DDAXF XAGGF DXXAX FGXFG G, encrypted with square keyword `monkeys` and transposition key `zebras`.
Undo the transposition. The 26 letters fill 6 columns as 4 full rows plus 2 letters, so the columns under Z and E (the first two key letters) have 5 letters and the others 4. Number the key letters alphabetically: Z E B R A S → 6 3 2 4 1 5. Cut the ciphertext in that order: A (4) ADDD, B (4) FDDA, E (5) XFXAG, R (4) GFDX, S (4) XAXF, Z (5) GXFGG. Reading the grid row by row gives the intermediate text GXFGAX XFDFDA FXDDDX GAAXDF GG.

Undo the square. This example uses the 5 × 5 **ADFGX** square (letters only, i/j share a cell), keyword `monkeys` followed by the rest of the alphabet: rows A `monke`, D `ysabc`, F `dfghi`, G `lpqrt`, X `uvwxz`. The pairs GX FG AX XF DF DA FX DD DX GA AX DF GG give t h e w a y i s c l e a r, so the plaintext is **"the way is clear"** ✓ (script-checked).

Corrections to the notes / slides

- In the older year's slide the replaying box sat visually near the confidentiality group. This year's taxonomy figure (L3 slide 13) groups it correctly under integrity; use the table above.
- The ADFGVX decryption example (keys `monkeys/zebras`): the earlier version of these notes said the square from `monkeys` does not give "the way is clear". It does, once the square is the 5 × 5 ADFGX square shown on the slide (not a 6 × 6 ADFGVX one); see the worked example. Strictly, that example is the ADFGX cipher (March 1918), the predecessor of ADFGVX (June 1918).
- L1 history slide: DES was adopted in 1977 by the US National Bureau of Standards (NBS); the name NIST dates from 1988.
- L1 "Mono-alphabetic substitution" slide: "only $s = |A|$ keys (25 for the Roman alphabet)" is true for the *shift* cipher only ($|A| = 26$ shifts, of which 25 change the text). A general monoalphabetic substitution has 26! keys and is broken by frequency analysis, not by key search.
- L3 CERT-In 2019 pie chart: scanning/probing/vulnerable services is 305 276/394 499 = 77.4%, not 75%.

7 Classical Encryption Techniques

Plaintext sent in the open over a public channel invites both kinds of attack (§6): **passive** attacks, in which the opponent eavesdrops and reads the data, and **active** attacks, in which he alters or injects data. Encryption turns the plaintext into ciphertext so that an interceptor learns nothing from it.

Classical techniques are the ciphers used before computers, operating on the letters of a natural-language alphabet. All of them are now **obsolete**: they are easy to break and their keys are recovered quickly, often by hand. They are still studied for two reasons. Every modern cipher is built from the same two operations, *substitution* and *transposition*, only applied to bits and repeated over many rounds (§29); and the ways classical ciphers fail (small key space, preserved letter frequencies, linearity) are exactly the weaknesses that modern designs are built to avoid.

7.1 Substitution technique

In a **substitution** cipher each letter of the plaintext is **replaced** by another letter, number or symbol, while the *positions* of the letters stay the same. The cipher is a mapping from the plaintext alphabet to the ciphertext alphabet, and this mapping must be one-to-one: if two plaintext letters were sent to the same ciphertext letter, the receiver could not tell which one was meant, and decryption would be ambiguous. Decryption applies the inverse mapping.

Key (a mapping table): a→M, b→X, g→A, x→Z

Plaintext	b	a	g
	↓	↓	↓
Ciphertext	**X**	**M**	**A**

In the lecture's example the plaintext `bag` becomes XMA: each letter is looked up in the table and replaced. The mapping table *is* the key. Sender and receiver must share the same table, so this is symmetric encryption.

If a given plaintext letter is always replaced by the same ciphertext letter, wherever it occurs, the cipher is **monoalphabetic** (a one-to-one relationship between plaintext and ciphertext symbols). The Caesar cipher and the general monoalphabetic cipher are of this kind. If the same plaintext letter can be replaced by different ciphertext letters at different positions, the cipher is **polyalphabetic** (one-to-many), like Vigenère and the one-time pad (§14, §15); Playfair and Hill, which substitute groups of letters at once, are filed with them by Forouzan. A monoalphabetic substitution only renames the letters, so the ciphertext has the same frequency profile as the plaintext moved onto different symbols, and that is what frequency analysis exploits.

7.2 Transposition technique

A **transposition** cipher does not replace any letter. It applies a **permutation**, a rearrangement, to the positions of the plaintext letters, so the ciphertext consists of exactly the plaintext letters in a different order. The permutation is the key.

Plaintext `NESO` → `ESON`, `SONE`, `ONES`, `ENOS`, … e.g. `ESON` takes letters in order (2, 3, 4, 1); `SONE` takes (3, 4, 1, 2).

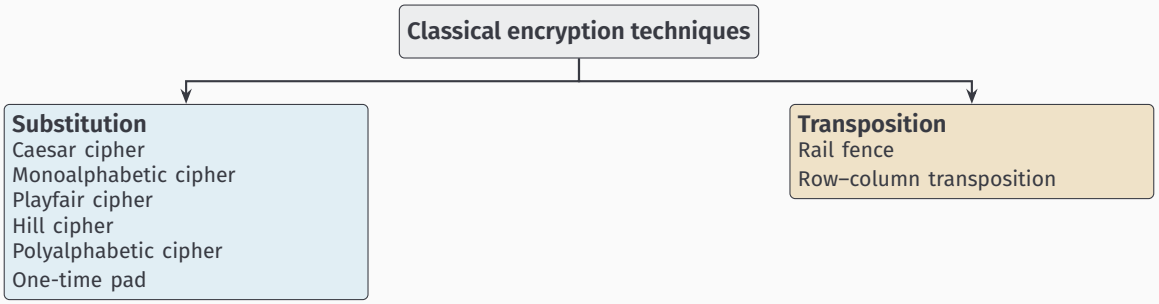
The number of possible keys for a block of n letters is the number of orderings of n positions: n choices for the first position, $n - 1$ for the second, and so on, giving $n!$. A 4-letter word has only $4! = 24$ arrangements, so short messages are easy to guess. Since $n!$ grows fast ($10! = 3\,628\,800$, $20! \approx 2.4 \times 10^{18}$), transposition is only effective for **long** plaintexts. Even then it has a built-in weakness: the letter frequencies of the ciphertext are *identical*

to those of the plaintext, so a cryptanalyst sees at once that a transposition was used (English frequencies, but no English words) and can rebuild the text by anagramming common pairs such as `th` and `he`. Rail fence and row–column transposition (Transposition Ciphers section) and keyed transposition (§17) are the standard examples.

7.3 Substitution vs. transposition

The two techniques attack the structure of the plaintext from opposite sides: substitution hides *which* letters occur, transposition hides *where* they occur. Neither alone hides everything, which is why practical ciphers combine them (ADFGVX in 1918, every round of DES and AES today).

	Substitution	**Transposition**
What changes	The letters themselves	The order of the letters
Positions	Unchanged	Permuted
Letters in ciphertext	New letters / symbols	Exactly the plaintext letters
Letter frequencies	Moved onto different symbols	Identical to plaintext
Key	a mapping of the alphabet (e.g. a shift)	a permutation of positions
Weak against	frequency analysis (monoalphabetic)	anagramming; short messages ($n!$ small)



8 Caesar Cipher

The Caesar cipher is the earliest known and simplest substitution cipher, used by Julius Caesar for military messages. Each letter of the alphabet is replaced by the letter standing three places further down the alphabet, and the alphabet is treated as a circle, so that after `z` it wraps round to `a`:

```
plain:  a b c d e f g h i j k l m n o p q r s t u v w x y z
cipher: D E F G H I J K L M N O P Q R S T U V W X Y Z A B C
```

Since every letter is moved by the same amount and a letter is always replaced by the same letter, it is a monoalphabetic substitution. Stallings' classic example: `meet me after the toga party` → `PHHW PH DIWHU WKH WRJD SDUWB`.

8.1 Letters as numbers (start at 0)

To write the rule as a formula, each letter is replaced by its position in the alphabet, starting from 0:

A	B	C	D	E	F	G	H	I	J	K	L	M
0	1	2	3	4	5	6	7	8	9	10	11	12
N	O	P	Q	R	S	T	U	V	W	X	Y	Z
13	14	15	16	17	18	19	20	21	22	23	24	25

Numbering from 0 matches the remainders mod 26, which are exactly $0, 1, \ldots, 25$. The wrap-around from `z` to `a` is then simply reduction modulo 26, i.e. arithmetic in $\mathbb{Z}_{26}$ (§21). If the letters were numbered 1 to 26, `z` would be 26, which reduces to 0, a number that stands for no letter.

8.2 Algorithm

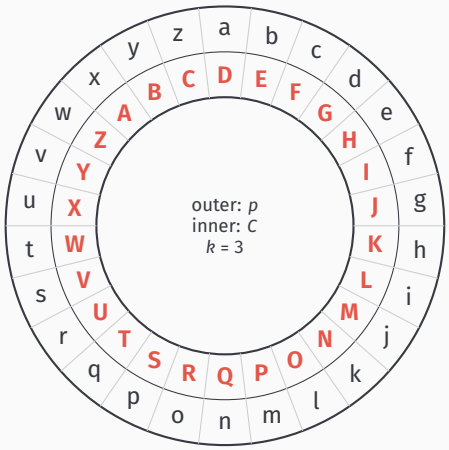
With p the number of a plaintext letter, C the number of the ciphertext letter and k the key (the shift), encryption adds the key and decryption subtracts it, both modulo 26.

★ MUST KNOW : Caesar / shift cipher

$$C = E(p, k) = (p + k) \bmod 26$$
$$p = D(C, k) = (C - k) \bmod 26$$

Caesar cipher: $k = 3$. Shift cipher: any $k \in \{1, \ldots, 25\}$.

Why decryption works. Decrypting the encryption gives $((p + k) \bmod 26 - k) \bmod 26 = p \bmod 26$, because reducing modulo 26 before or after the subtraction gives the same remainder. Since $0 \le p \le 25$, this is exactly p.
Computing the remainder. mod gives the remainder, not the quotient: 27 mod 26 = 1, because 27 = 1×26+1. During decryption $C - k$ can be negative; the remainder must lie in $0, \ldots, 25$, so add 26: −2 mod 26 = 24, because −2 = (−1) × 26 + 24.
Convention. Plaintext is written in lowercase and ciphertext in UPPERCASE, which keeps the two apart in a worked answer. For letters away from the end of the alphabet one can simply count k letters forward; the formula matters at the wrap-around.



8.3 Worked example: encrypt "neso academy" ($k = 3$)

WORKED EXAMPLE : Caesar encryption (lecture example)

Q. Encrypt "neso academy" using the Caesar cipher.
A. Convert each letter to its number, add 3, reduce mod 26, convert back:

Plaintext	n	e	s	o	a	c	a	d	e	m	y
p	13	4	18	14	0	2	0	3	4	12	24
$p + 3$	16	7	21	17	3	5	3	6	7	15	**27**
mod26	16	7	21	17	3	5	3	6	7	15	**1**
Ciphertext	**Q**	**H**	**V**	**R**	**D**	**F**	**D**	**G**	**H**	**P**	**B**

Answer: **QHVR DFDGHPB**. The only wrap-around is y: 24 + 3 = 27, and 27 = 26 × 1 + **1**, so $C = 1$ = B. (Counting on: y → z → a → **b**.) For the other letters you can just count three letters forward. **Check:** $B = 1$, $1 - 3 = -2 \equiv 24 =$ y ✓.

8.4 Homework 1: decrypt "QHWZRUN VHFXULWB" ($k = 3$)

WORKED EXAMPLE : Caesar decryption (lecture homework)

Q. Decrypt "QHWZRUN VHFXULWB" using the Caesar cipher.
A. Apply $p = (C - 3) \bmod 26$ to every letter:

Ciphertext	Q	H	W	Z	R	U	N	V	H	F	X	U	L	W	B
C	16	7	22	25	17	20	13	21	7	5	23	20	11	22	1
$C - 3$	13	4	19	22	14	17	10	18	4	2	20	17	8	19	−2
mod26	13	4	19	22	14	17	10	18	4	2	20	17	8	19	**24**
Plaintext	n	e	t	w	o	r	k	s	e	c	u	r	i	t	y

Answer: **network security**. Note the last letter: 1 − 3 = −2, add 26 ⇒ 24 = y.

9 Shift Cipher (the general form)

The Caesar cipher has no real key: the shift is always 3, so anyone who knows the method can read every message. The obvious repair is to let the shift vary and keep it secret. The **shift cipher** uses the same two formulas with a secret key k; Forouzan calls it the **additive cipher**, because encryption is addition in $\mathbb{Z}_{26}$. **Caesar cipher = shift cipher with $k = 3$.**

How many keys? A key k and the key $k + 26$ give the same cipher, since $(p + k + 26) \bmod 26 = (p + k) \bmod 26$; so every key can be reduced modulo 26 (a shift of 29 is a Caesar shift of 3), and there are only 26 different keys, 0 to 25. One of them is useless:

- $k = 0$ is useless: $C = (p + 0) \bmod 26 = p$, so the ciphertext equals the plaintext. (Homework: "Neso", $k = 0 \Rightarrow$ NESO.)
- $k = 26$ is the same as $k = 0$. So there are only **25 usable keys**.

Decryption is encryption with $26 - k$**.** Subtracting k modulo 26 is the same as adding its additive inverse $26 - k$: $(C - k) \bmod 26 = (C + 26 - k) \bmod 26$. Decrypting the Caesar cipher is therefore the same as encrypting with key 23, and an attacker who tries all forward shifts tries all decryption keys too. This is why the forward and backward

shifts in the brute-force table below always add up to 26.

Encrypting twice does not help. Shifting by k_1 and then by k_2 gives $(p + k_1 + k_2)$ mod 26, which is a single shift by $(k_1 + k_2)$ mod 26. Double encryption with two shift keys is just another shift cipher, still with 25 keys to search. (In algebraic terms, the shift ciphers are closed under composition.)

Example: "Neso", $k = 4$: N 13+4=17 →**R**, e 4+4=8 →**I**, s 18+4=22 →**W**, o 14+4=18 →**S** ⇒ **RIWS**.

Homework 2: "cryptography", $k = 4$:

Plaintext	c	r	y	p	t	o	g	r	a	p	h	y
p	2	17	24	15	19	14	6	17	0	15	7	24
$p + 4$ mod 26	6	21	**2**	19	23	18	10	21	4	19	11	**2**
Ciphertext	G	V	C	T	X	S	K	V	E	T	L	C

Answer: **GVCTXSKVETLC** (both y's wrap: 24 + 4 = 28 ≡ 2).

In code: the 0–25 numbering is a convention. With ASCII (A=65, a=97), shift to 0 first, then back: $C = \big((p - 65 + k) \bmod 26\big) + 65$ for uppercase (use 97 for lowercase).

WORKED EXAMPLE : Forouzan Example 3.3 (additive cipher)

Q. Use the additive cipher with key = 15 to encrypt the message "hello". Decrypt the result to verify.
Encryption, $C = (p + 15)$ mod 26:

Plaintext	h	e	l	l	o
p	7	4	11	11	14
$p + 15$	22	19	**26**	**26**	**29**
mod26	22	19	0	0	3
Ciphertext	**W**	**T**	**A**	**A**	**D**

Ciphertext	W	T	A	A	D
C	22	19	0	0	3
$C - 15$	7	4	−15	−15	−12
mod26	7	4	11	11	14
Plaintext	h	e	l	l	o

Ciphertext **WTAAD**; decryption with $p = (C - 15)$ mod 26 (adding 26 to the negative values) returns **hello** ✓. Note that the double l gives the double A: a monoalphabetic cipher keeps repeated letters visible.

9.1 Pros and cons of the Caesar cipher

Advantages. The cipher is **simple** and **easy to implement**: it needs one addition per letter and can be done by hand or written in a few lines of code.

Disadvantages. The lecture lists three, and together they are exactly what makes a brute-force attack succeed.

1. **The encryption and decryption algorithms are known.** By itself this is normal (Kerckhoff's principle, §6); it means the attacker knows the cipher is a shift and only has to find k.
2. **There are only 25 keys to try**, so the cipher is vulnerable to a **brute-force attack**: trying all of them takes seconds.
3. **The language of the plaintext is known and easily recognisable**, so the attacker can tell when he has hit the right key, because exactly one trial produces readable text.

A fourth weakness follows from being monoalphabetic with a single shift: letter frequencies survive, so even without trying keys, one correct guess of a single plaintext–ciphertext letter pair reveals the key (statistical attack below).

10 Brute-Force Attack on the Caesar/Shift Cipher

A **brute-force** (exhaustive key search) attack tries every possible key until readable plaintext appears. For the shift cipher this is trivial. There are 25 keys and the right one is equally likely to be any of them, so on average the attacker needs $(1 + 2 + \cdots + 25)/25 = 13$ trials and at worst 25. By hand that is a few minutes of counting; a computer doing 10^9 trials per second needs about 10^{-8} seconds. Compare $2^{128} \approx 3.4 \times 10^{38}$ keys for AES (§5): the attack is the same, only the size of the key space differs.

★ MUST KNOW : when brute force works (Stallings)

Three conditions make brute-force cryptanalysis of the Caesar cipher easy: (1) the encryption and decryption algorithms are known; (2) there are only 25 keys to try; (3) the language of the plaintext is known and easily recognisable. A cipher defeats brute force by making (2) false: a key space of 2^{128} or more.

The lecture example. Ciphertext: SQDYMZK. Shift every letter forward by 0, 1, 2, ..., 25 and look for a real word:

Fwd	Back	Result
0	26	SQDYMZK
1	25	TREZNAL
2	24	USFAOBM
3	23	VTGBPCN
4	22	WUHCQDO
5	21	XVIDREP
6	20	YWJESFQ
7	19	ZXKFTGR
8	18	AYLGUHS
9	17	BZMHVIT
10	16	CANIWJU
11	15	DBOJXKV*
12	14	ECPKYLW

Fwd	Back	Result
13	13	FDQLZMX
14	**12**	**GERMANY**
15	11	HFSNBOZ
16	10	IGTOCPA
17	9	JHUPDQB
18	8	KIVQERC
19	7	LJWRFSD
20	6	MKXSGTE
21	5	NLYTHUF
22	4	OMZUIVG*
23	3	PNAVJWH
24	2	QOBWKXI
25	1	RPCXLYJ

* Recomputed; the notes had these rows slightly wrong (see corrections box).

Forward + back = 26 in every row, because shifting forward 14 is the same as shifting back 12: $C + 14 \equiv C - 12 \pmod{26}$ (§9). Only one row is English, **GERMANY**, so the attacker now has the plaintext *and* the key.

Which number is the key? The sender encrypted with k = **12** (G 6 + 12 = 18 = S, ..., Y 24 + 12 = 36 ≡ 10 = K). The 14 is the forward shift that *undoes* it. The lecture calls 14 "the key"; be clear which one you mean in an exam: write "encryption key k = 12; decryption is a forward shift of 14 (a backward shift of 12)".

Decryption versus cryptanalysis. Decryption is done by the receiver, who has the key and reverses the encryption; cryptanalysis is done by an attacker who does not have the key and recovers the plaintext and/or the key, here by brute force. The table above is cryptanalysis, not decryption.

A caution. Brute force assumes that exactly one trial gives meaningful text. For a long ciphertext this is practically certain, but a very short ciphertext (three or four letters) may decrypt to two different words under two keys, and then the attacker cannot decide without more ciphertext or context.

WORKED EXAMPLE : Forouzan Example 3.4 (brute force)

Q. Eve has intercepted the ciphertext UVACLYFZLJBYL. Show how she can use a brute-force attack to break the cipher.
A. Eve knows the cipher is additive and that the key domain is $\{1, \ldots, 25\}$. She decrypts with $p = (C - k) \bmod 26$ for $k = 1, 2, 3, \ldots$ and stops at the first meaningful text:

Key	Plaintext	Key	Plaintext
1	tuzbkxeykiaxk	5	pqvxgtaugewtg
2	styajwdxjhzwj	6	opuwfsztfdvsf
3	rsxzivcwigyvi	**7**	**notverysecure**
4	qrwyhubvhfxuh		

The plaintext is **"not very secure"** and the key is **k = 7**. **Check:** n = 13, 13 + 7 = 20 = U, the first ciphertext letter ✓. (The remaining keys 8–25 were also tried by script; none gives English.)

Statistical attack. The shift cipher also falls without trying keys. Since every letter is shifted by the same k, a single correct pair is enough: if ciphertext letter c stands for plaintext letter p, then $k = (c - p) \bmod 26$. The attacker counts the ciphertext letters and assumes the most frequent one stands for e, the most common letter in English (about 12.5% of letters on the slide's chart; then t, a, o). If the result is not readable, the next most frequent letter is tried as e, or the top letter as t. The attack needs a reasonably long ciphertext, because the frequencies of a short text can differ from the averages.

WORKED EXAMPLE : Forouzan Example 3.5 (statistical attack)

Q. Eve has intercepted the following ciphertext. Using a statistical attack, find the plaintext.
XLILSYWIMWRSAJSVWEPIJSVJSYVQMPPMSRHSPPEVWMXMWASVX
LQSVILYVVCFIJSVIXLIWIPPIVVIGIMZIWQSVISJJIVW
A. Count the 92 ciphertext letters. The most frequent are I (14 times), V (13) and S (12). Assume I → e: $k = (8 - 4) \bmod 26 = 4$. Decrypt with $p = (C - 4) \bmod 26$:

the house is now for sale for four million dollars it is worth more hurry before the seller receives more offers

The text is English, so the key is **4**. **Check:** t = 19, 19 + 4 = 23 = X ✓. (Taking V as e would give k = 17 and gibberish; V actually stands for r, which is unusually common in this text.)

Trap: the usual slips with shift ciphers

The key space is 25, not 26, since $k = 0$ changes nothing. Show the wrap-around step ($27 \bmod 26 = 1$) and turn a negative value into a remainder by adding 26. In a brute-force answer, draw the table, box the readable row and state the *encryption* key (12 in the SQDYMZK example, not 14). Know the three cons of the Caesar cipher, and do not call "the algorithm is public" a violation of Kerckhoff's principle: it is what the principle assumes.

Corrections to the notes / slides

- "neso academy" encrypts to QHVR DFDGHPB. The notes' final answer QHVRDFDGHB drops the P.
- Brute-force table: row 11 is DBOJXKV and row 22 is OMZUIVG. Row 12 (ECPKYLW) was already correct.
- The notes call con #1 a "Kerckhoffs violation". It isn't: public algorithms are normal (see the intro sheet). The problem is a public algorithm *plus* only 25 keys.
- Slide typo: "used *be* Julius Caesar" → "used *by*".
- Shift-cipher slide: "Key = 2, 3, 4, 5, ..." leaves out $k = 1$, which is a valid key; the usable keys are $1, \ldots, 25$.

11 Monoalphabetic Cipher

The Caesar (shift) cipher has two weaknesses that make it useless. Its key space has only 25 useful keys, so an attacker simply tries them all. Worse, every letter is shifted by the *same* amount, so the shift found for a single letter decrypts the whole message. The monoalphabetic cipher removes both weaknesses at once: instead of shifting the alphabet, it replaces each letter by an *arbitrary* other letter, chosen independently for each of the 26 letters. The only constraint is that the replacement must be one-to-one, so that the receiver can undo it.

11.1 Permutations

A permutation of a finite set S is an ordered sequence of all the elements of S, with each element appearing exactly once. The set $S = \{a, b, c\}$ has six permutations: `abc`, `acb`, `bac`, `bca`, `cab`, `cba`. In general a set of n elements has $n! = n(n-1)\cdots 2 \cdot 1$ permutations, because the first position can be filled in n ways, the second in $n-1$ ways (one element is used up), and so on down to one way for the last position. For example $2! = 2$ and $4! = 24$.

Permutations matter here because the key of a monoalphabetic cipher *is* a permutation of the alphabet: writing the ciphertext letters for $a, b, \ldots, z$ in order gives a sequence in which each of the 26 letters appears exactly once. The number of possible keys is therefore the number of permutations of 26 letters, $26!$.

11.2 The cipher

In a **monoalphabetic substitution cipher** Alice and Bob agree on a table that pairs every plaintext letter with one ciphertext letter. The bottom row of the table, the **cipher line**, can be any permutation of the 26 letters. To encrypt, each plaintext letter is looked up in the top row and replaced by the letter below it; to decrypt, each ciphertext letter is found in the bottom row and replaced by the letter above it. The relationship between plaintext and ciphertext symbols is **one-to-one**: a plaintext letter always becomes the same ciphertext letter, and no two plaintext letters share a ciphertext letter. If two letters shared one, the receiver could not tell which was meant, so the table would not be decryptable.

The key space has $26! \approx 4.03 \times 10^{26} \approx 2^{88}$ keys, compared with 25 for the shift cipher. A computer testing 10^{12} keys per second would need about 13 million years to try them all, so the slide's conclusion is that the large key space "would seem to eliminate brute-force techniques". The shift ciphers are just 26 of these $26!$ permutations, those in which the cipher line is the alphabet rotated. Although a permutation is involved, the monoalphabetic cipher is still a **substitution** cipher: the permutation is applied to the *alphabet* to build the key, while the letters of the message keep their positions. Rearranging the message itself would be a transposition.

Example (lecture). Under one key, "Neso" becomes DULA. The distance each letter moves is different, so learning one pair of the table says nothing about the others; with Caesar the distance is always 3.

Plaintext	N (13)	E (4)	S (18)	O (14)
Ciphertext	**D** (3)	**U** (20)	**L** (11)	**A** (0)
Shift $(C - p) \bmod 26$	16	16	19	12

WORKED EXAMPLE : the slide's key table (Forouzan)

Q. Using the key below, encrypt "this message is easy to encrypt but hard to find the key", and show how Bob decrypts.

Plain	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
Cipher	N	O	A	T	R	B	E	C	F	U	X	D	Q	G	Y	L	K	H	V	I	J	M	P	Z	S	W

A. Remove the spaces and replace each letter by the one below it: t→I, h→C, i→F, s→V, so “this” → ICFV; “message” → QRVVNER; and so on. The ciphertext is

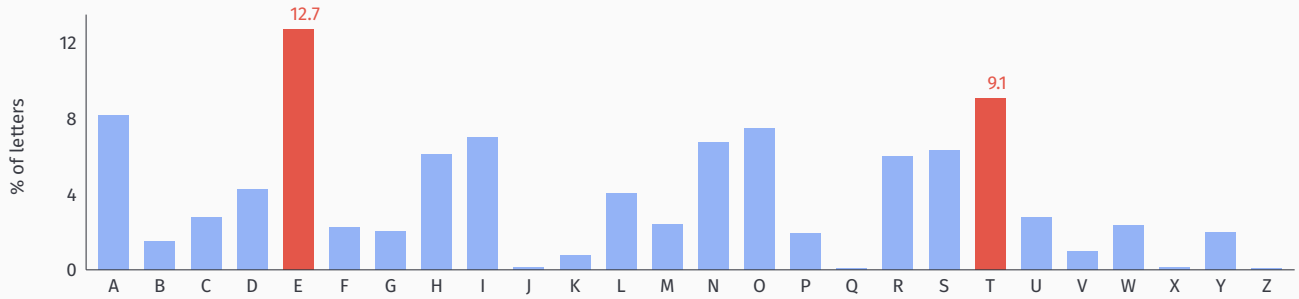
ICFVQRVVNERFVRNVSIYRGAHSLIOJICNHTIYBFGTICRXRS

Bob finds each ciphertext letter in the bottom row and reads the letter above it (I→t, C→h, F→i, V→s, …), which recovers the plaintext. Note that the three s of “message is” all became V and every e became R: the frequency pattern of the plaintext survives, which is the subject of the next subsection.

11.3 Breaking it: frequency analysis

The huge key space does not protect the cipher, because the attacker never searches through keys. A monoalphabetic cipher only *renames* letters. If the key maps x to $\pi(x)$, then every occurrence of x in the plaintext becomes an occurrence of $\pi(x)$ in the ciphertext and nothing else becomes $\pi(x)$, so $\pi(x)$ occurs in the ciphertext exactly as often as x occurs in the plaintext. The histogram of ciphertext letters is the histogram of plaintext letters with the bars relabelled. A monoalphabetic cipher does not change the frequency of characters, so each letter of the key can be found separately by matching frequencies, and the attack needs a few dozen decisions instead of 26! guesses. This is a **statistical** (frequency) attack, and it needs only the ciphertext, which makes it a ciphertext-only attack, the most likely kind in practice.

The attack works because English letters are far from equally common. The relative frequencies (lecture chart, from a large corpus) are:



Ranked from most to least common, the letters are

e t a o i n s h r d l c u m w f g y p b v k j x q z.

The letter e alone is 12.7% of English text and t is 9.1%; the nine letters e t a o i n s h r together make up about 70% of it, while j, x, q, z are each below 0.2%. Pairs and triples are just as uneven. The most common **digrams** (two-letter groups) are th, he, in, er, an, re, ed, on, es, st, en, at, to, nt, ha, nd, ou, ea, ng, as, or, ti, is, et, it, ar, te, se, hi, of, and the most common **trigrams** are the, ing, and, her, ere, ent, tha, nth, was, eth, for, dth (lecture table, from Forouzan).

The attack therefore proceeds as follows. Eve counts the ciphertext letters and guesses that the most frequent one stands for e and the next for t. She then tests the guesses against digrams and trigrams: if t has been placed, a frequent pattern “t?e” is almost certainly the, which reveals h. Each new letter makes more words partly readable, and the remaining letters are filled in by recognising words. The statistics only become reliable with enough text: with about 100 letters the top two or three ranks usually match English, while the lower ranks are shuffled and have to be fixed by word guessing. For a very short message the counts may mislead completely.

WORKED EXAMPLE : the lecture’s frequency attack (Stallings’ ciphertext, 120 letters)

Q. The following ciphertext was produced by a monoalphabetic cipher. Use a statistical attack to find the plaintext.

UZQSOVUOHXMOPVGPOZPEVSGZWSZOPFPESXUDBMETSXAIZ
VUEPHZHMDZSHZOWSFPAPPDTSVPQUZWYMXUZUHSX
EPYEPOPDZSZUFPOMBZWPFUPZHMDJUDTMOHMQ

A. Step 1: count. The 120 letters occur as follows (C, K, L, N, R do not occur):

Letter	**P**	**Z**	S	U	O	M	H	D	E	V	X	F	W	Q	T	A	B	G	Y	I	J
Count	16	14	10	10	9	8	7	6	6	5	5	4	4	3	3	2	2	2	2	1	1

Step 2: the two peaks. P is 16/120 = 13.33% and Z is 14/120 = 11.67%, close to the 12.7% of e and 9.1% of t. Guess P = e and Z = t.
Step 3: digrams and trigrams. The digram ZW occurs 3 times, and ZWP once. With the guesses this is “t?e”, so ZWP = “the” and W = h. Then ZWS = “th?”, which fits “that”, so S = a.

Step 4: fill in. With e, t, h, a placed, 16 + 14 + 4 + 10 = 44 of the 120 letters (37%) are known and words become guessable. For example, ZWSZ OPFPESX now reads "that ?e?e?a?", which suggests "that several" and gives O = s, F = v, E = r, X = l. Each guess is checked against the rest of the text, and the remaining letters follow in the same way. The result is

it was disclosed yesterday that several informal but direct contacts have been made with political representatives of the viet cong in moscow

and the recovered key (cipher → plain) is

Cipher	A	B	D	E	F	G	H	I	J	M	O	**P**	Q	S	T	U	V	**W**	X	Y	**Z**
Plain	b	f	n	r	v	y	c	u	g	o	s	**e**	w	a	m	i	d	**h**	l	p	**t**

Check: applying this table to all 120 letters gives the sentence above. Note that S (a) and U (i) tie at 10, ahead of O (s) at 9: below the top two ranks, a short text does not follow the English ranking exactly, which is why steps 3–4 are needed.

11.4 Pros, cons, countermeasure

Advantages. The monoalphabetic cipher is as simple to use as the Caesar cipher, but its key space of 26! makes brute force impossible, and knowing one letter's substitute reveals nothing about the others. It is therefore much more secure than the Caesar cipher against an attacker who only tries keys.

Disadvantages. Since the ciphertext reflects the frequency data of the original alphabet, the cipher is easy to break with frequency analysis; the guessing attack above needs only a short ciphertext and some knowledge of English letter frequencies. The key is also long (a full table of 26 letters) and awkward to share and memorise, which is why practical versions derive it from a key phrase (see the homework below).

Countermeasure: homophones. A **homophonic substitution** gives each plaintext letter several possible ciphertext symbols, called **homophones**, and the encryptor picks one of them at random for each occurrence. The number of homophones is made proportional to the letter's frequency: e (about 13%) gets about 13 symbols, while z gets one. Since each of the 13 symbols for e is then used for roughly 12.7/13 ≈ 1% of the text, every symbol appears with about the same frequency and the peaks in the histogram disappear. The ciphertext alphabet must be larger than 26, for example the numbers 00 to 99. Decryption is still unique, because each symbol belongs to only one plaintext letter.

Homophones flatten the *single-letter* frequencies, but they do not make the cipher secure. Each plaintext letter still affects only one ciphertext symbol, so multi-letter patterns survive: t is still very often followed by h, so the symbols of t are often followed by symbols of h, and the attacker can group the symbols that behave alike and then proceed as before. Two better remedies were developed from this observation: encrypting *several letters together* (Playfair, Hill), and using *several cipher alphabets* chosen by a key (the polyalphabetic ciphers of §14).

★ MUST KNOW : monoalphabetic cipher

Key = any permutation of the 26 letters: $26! \approx 4 \times 10^{26}$ keys (Caesar: 25), so brute force is impossible. But the mapping is one-to-one and fixed, so **letter frequencies are preserved** and the cipher falls to frequency analysis: English ranking e t a o i n s h r d l …, top digram th, top trigram the. Homophones flatten single-letter frequencies but not multi-letter patterns.

11.5 Homework: encrypt with a key phrase

Since a random 26-letter table is hard to remember, the key is often derived from a key phrase: write the phrase without spaces, delete every letter that has already appeared, and use the result as the cipher line. If the phrase does not contain all 26 letters, the unused letters are appended in alphabetical order (exactly as in the Playfair square below).

WORKED EXAMPLE : lecture homework

Q. Encrypt "Attack postponed to tomorrow and do not use our secret paper until further info" with a monoalphabetic cipher whose key is "The quick brown fox jumps over the lazy dog" (ignore the second and later occurrences of a letter in the key).
A. Removing spaces and repeated letters from the key phrase leaves THEQUICKBROWNFXJMPSVLAZYDG. The phrase is a pangram, so this already contains all 26 letters and is the whole cipher line:

Plain	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
Cipher	T	H	E	Q	U	I	C	K	B	R	O	W	N	F	X	J	M	P	S	V	L	A	Z	Y	D	G

Substituting letter by letter (spaces kept only for readability):

TVVTEO JXSVJXFUQ VX VXNXPPXZ TFQ QX FXV LSU XLP SUEPUV JTJUP LFVBW ILPVKUP BFIX

Check on the first word: a→T, t→V, t→V, a→T, c→E, k→O gives TVVTEO. The doubled `tt` became the doubled VV, another pattern a monoalphabetic cipher cannot hide.

12 Playfair Cipher ★★

★ **In the papers:** 2024–25 Q2 (3 marks): key "GUIDANCE", message "The key is hidden under the door pad". 2025–26 Q3 (4 marks): given key matrix, "Insta Reels Pe Time Waste Na Kar", filler X, I/J in one cell. Both solved in the Question Bank (past papers).

▸ **Likely this year:** A third Playfair encryption or decryption (3–4 marks) with a doubled letter, an odd length and an I/J; or "why is Playfair harder to break than a monoalphabetic cipher, and how is it still broken?" (2–3 marks).

The monoalphabetic cipher fails because every letter is encrypted on its own, so the letter frequencies of English pass straight into the ciphertext. The **Playfair cipher** attacks this at the root: it encrypts the plaintext **two letters at a time**, and the substitute of a letter depends on the letter it is paired with. It is the first literal **digram substitution** cipher, where a **digram** is a pair of consecutive letters treated as one unit. It was invented in 1854 by **Charles Wheatstone** but bears the name of **Lord Playfair**, who promoted its use, so it is also called the Playfair square or the Wheatstone–Playfair cipher. It is a manual **symmetric** technique that needs only pencil and paper, and it was used as a field cipher by the British army in World War I and by the Allied forces in World War II.

Caesar and the monoalphabetic cipher are **single-letter** ciphers: a plaintext letter always becomes the same ciphertext letter. Playfair is a **multiple-letter** cipher: the same plaintext letter becomes different ciphertext letters depending on its partner (in the solved question below, e becomes D, O or N), so single-letter frequencies are hidden. The slides (Forouzan) describe the same property in polyalphabetic terms: the key stream is a sequence of pairs $k = [(k_1, k_2), (k_3, k_4), \dots]$, each determined by a plaintext pair and the square, and $C_i = k_i$.

12.1 Building the 5×5 key matrix

The secret key is an arrangement of 25 letters in a 5×5 matrix. Since the alphabet has 26 letters and the square has 25 cells, **I and J share one cell**. J is chosen because it is one of the rarest English letters (0.15%) and a J written as I is easily corrected from context. A keyword makes the arrangement easy to remember and to share:

1. Write the keyword into the square row by row, **skipping every letter that has already appeared**.
2. Fill the remaining cells with the unused letters **in alphabetical order**.
3. Treat I and J as one letter throughout. If the keyword contains I or J, the I/J cell is filled at that point and any later I or J is a repeat.

With the keyword **MONARCHY** the first eight cells hold M O N A R C H Y (highlighted), and the rest of the alphabet follows: B, D, E, F, G, I/J, K, L, P, Q, S, T, U, V, W, X, Z.
Any arrangement of the 25 letters is a valid key, so there are $25! \approx 1.55 \times 10^{25}$ possible squares and brute force is as hopeless as for the monoalphabetic cipher. (Rotating all rows, or all columns, cyclically gives the same cipher, because the rules only use "same row", "same column", "to the right" and "below"; this divides the count by 25, which changes nothing in practice.)

M	O	N	A	R
C	H	Y	B	D
E	F	G	I/J	K
L	P	Q	S	T
U	V	W	X	Z

12.2 Step 1: split the plaintext into digrams

Before encryption the plaintext is prepared: spaces are removed, every `j` is replaced by `i`, and the letters are split into pairs from left to right. Two adjustments are made while pairing:

- If the two letters of a pair are the **same**, a filler ("bogus") letter, normally x, is inserted between them, and the pairing continues from the second of the two letters.
- If, after all insertions, **one letter is left over** at the end, a filler x is appended.

The first rule exists because the encryption rules below are defined only for two *different* cells. Two equal letters occupy the same cell, which lies in the same row and in the same column at once, so the row rule and the column rule would both apply and disagree, and the rectangle rule would have no rectangle. Splitting doubles also hides a give-away pattern: doubled letters (`ll, ee, ss, oo, tt`) are common in English, and if a double became a double in the ciphertext, an attacker would see where they are. The second rule exists simply because the cipher works on whole pairs. Only a double *inside* one pair needs a filler; a double that falls across two pairs is harmless.

Plaintext	Digrams	Why
attack	`at ta ck`	nothing to fix (`tt` is split across two pairs)
neso academy	`ne so ac ad em yx`	11 letters: `y` is left over, add `x`
balloon	`ba lx lo on`	`ll` in one pair: `l x`, then re-pair `l o o n`

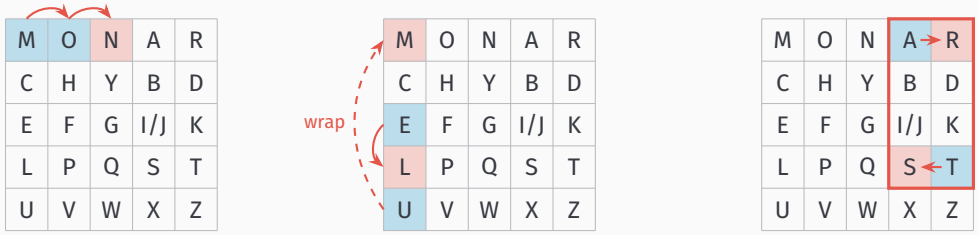
In “balloon” the inserted `x` also moves the `oo` across a pair boundary, so it needs no filler of its own. The filler must be removed by the receiver after decryption, so it should be a letter that rarely occurs next to itself; if the doubled letter is `x` itself, another rare letter such as `q` or `z` is used.

12.3 Step 2: encrypt each pair (three cases) ★

▸ **Likely this year:** Every Playfair question: state the three rules and apply them; “why is the rectangle rule its own inverse” is a good 1–2 mark theory question.

Each prepared pair is located in the square, and it falls into exactly one of three cases:

1. **Same row:** each letter is replaced by the letter to its **right**, wrapping from the last column to the first.
2. **Same column:** each letter is replaced by the letter **below** it, wrapping from the bottom row to the top.
3. **Rectangle** (different row and different column): the two letters are opposite corners of a rectangle. Each letter is replaced by the letter **in its own row** and in the **other letter’s column**, i.e. by the other corner on its own row. Order matters: `at`→RS but `ta`→SR.



blue = plaintext pair, red = ciphertext

Why each rule can be undone. The receiver must be able to tell which rule was used and reverse it, so each rule must turn its own kind of pair into a pair of the *same* kind, one-to-one.

- **Same row.** Moving one cell to the right with wrap-around is a cyclic rotation of the five letters of that row. A rotation is one-to-one and is undone by rotating to the left. Both ciphertext letters stay in the row and stay different, so the receiver sees a same-row pair and knows to move left. The wrap-around is essential: without it the last letter of a row would have no substitute.
- **Same column.** The same argument holds with “down” and “up”.
- **Rectangle.** If the plaintext letters are at (row r_1, column c_1) and (r_2, c_2) with $r_1 \neq r_2$ and $c_1 \neq c_2$, the ciphertext letters are at (r_1, c_2) and (r_2, c_1), the other two corners of the same rectangle. They are again in different rows and columns, so the receiver recognises the case, and applying the same rule to them gives (r_1, c_1) and (r_2, c_2), the original letters. The rectangle rule is **its own inverse**, which is why decryption uses it unchanged.

As a result, encryption is a one-to-one map on the $25 \times 24 = 600$ ordered pairs of distinct letters, which is exactly what decryption needs (checked by script for all four squares in this section). Three side effects follow from the rules, and cryptanalysts use them: a letter is never replaced by itself (every rule moves it to another cell); a reversed pair gives the reversed ciphertext (if AB→XY then BA→YX, as `at`/`ta` shows); and a ciphertext pair never consists of two equal letters.

12.4 Worked examples (keyword MONARCHY)

WORKED EXAMPLE : the lecture examples

“attack” → `at ta ck`
- `at`: rectangle → RS
- `ta`: rectangle → SR
- `ck`: rectangle (C row 2, K row 3) → DE

Answer: **RSSRDE**

“mosque” → `mo sq ue`
- `mo`: same row → ON
- `sq`: same row → TS
- `ue`: same column, U wraps to top → ML

Answer: **ONTSML**

Rapid fire (each digram on its own):

Digram	oa	ab	gk	dc	rz	zr	mz	rm
Case	row	column	row	row	column	column	rectangle	row
Wrap?			K→E	D→C	Z→R	Z→R		R→M
Answer	**NR**	**BI**	**IE**	**CH**	**DR**	**RD**	**RU**	**MO**

(BI/IE: the I/J cell can be written as I or J. rz/zr give DR/RD: the reversal property.)

12.5 Solved question

WORKED EXAMPLE : Neso solved question

Q. Encrypt "Hide the gold under the carpet" with the Playfair cipher, keyword "Neso Academy".

N	E	S	O	A
C	D	M	Y	B
F	G	H	I/J	K
L	P	Q	R	T
U	V	W	X	Z

Keyword letters: N E S O A C D M Y

A. The message has 25 letters (odd), so the last t gets a filler x. No pair has a repeated letter.

hi de th eg ol du nd er th ec ar pe tx

Pair	Case	Out	Pair	Case	Out
hi	row	IK	er	rect.	OP
de	column	GD	th	rect.	QK
th	rect.	QK	ec	rect.	ND
eg	column	DP	ar	rect.	OT
ol	rect.	NR	pe	column	VD
du	rect.	CV	tx	rect.	RZ
nd	rect.	EC			

Answer: **IKGDQKDPNRCVECOPQKNDOTVDRZ**

The plaintext e became D (in de, eg, pe), O (in er) and N (in ec): this is the multiple-letter property that hides single-letter frequencies. But the same *pair* always gives the same output (th→QK both times), because Playfair is a fixed substitution on digrams. This is why Playfair can still be attacked using digram frequencies (see the last subsection).

12.6 When I or J is in the keyword or the message ★

▸ **Likely this year:** Both past questions contained an I ("hidden", "Insta"); the I/J cell and an ii double are where marks are lost.

Since I and J share one cell, the cleanest procedure is to **replace every J by I before doing anything else**, in the keyword and in the message. Three consequences follow.

- In the **keyword**, the first I or J takes the I/J cell; a later I or J is a **repeat** and is skipped, like any other repeated letter.
- In the **message**, a pair ij or ji becomes ii, which is a **double** and needs a filler.
- The ciphertext never contains J, because the cell is written as I. The receiver decides between i and j from context.

WORKED EXAMPLE : I/J in both keyword and message

Q. Encrypt "meet me in fiji" with the keyword **JUPITER**.

I/J	U	P	T	E
R	A	B	C	D
F	G	H	K	L
M	N	O	Q	S
V	W	X	Y	Z

Keyword: J U P I T E R

Matrix. J goes into the first cell as I/J. The I in "JUPITER" is a repeat of that cell, so it is skipped. Then come U P T E R and the rest of the alphabet.

Digrams. meetmeinfiji → swap J: meetmeinfiii

- me et: the ee is split across two pairs, so **no filler**.
- fi, then ii (was ji) is a double: ix; then the last i is alone: ix.

me et me in fi ix ix

Pair	Case		Out	Pair	Case		Out
me	rectangle		SI	fi	column	F→M, I→R	MR
et	row	E wraps to I	IE	ix	rectangle	I→P, X→V	PV
me	rectangle		SI	ix	rectangle		PV
in	rectangle	I→U, N→M	UM				

Ciphertext: **SIIESIUMMRPVPV**. **Check:** decrypting gives me et me in fi ix ix. Dropping the fillers gives "meet me in fiii", and context says **meet me in fiji**.

12.7 Decryption

Decryption prepares the ciphertext into pairs (no fillers are needed, since a ciphertext pair is never doubled and the length is even) and reverses each rule: a same-row pair moves one cell **left**, a same-column pair moves one cell **up** (both with wrap-around), and a rectangle pair uses the **same** rule as encryption, because that rule is its own inverse. Finally the receiver removes the fillers: an x between two equal letters or at the very end is almost certainly a filler, while a genuine x (as in "taxi") is kept. This last step needs judgement, which is a small weakness of the scheme.

WORKED EXAMPLE : lecture homework

Q. Decrypt ODZFQSEZSONTSW with the keyword "Neso App".

N	E	S	O	A
P	B	C	D	F
G	H	I/J	K	L
M	Q	R	T	U
V	W	X	Y	Z

Keyword letters: N E S O A P

Pair	Case	Plain
OD	column (up; O wraps to bottom)	yo
ZF	column (up)	ua
QS	rectangle	re
EZ	rectangle	aw
SO	row (left)	es
NT	rectangle	om
SW	rectangle	ex

A. yo ua re aw es om ex → drop the final filler → **you are awesome**. **Check:** encrypting yo (column, down) gives OD again.

12.8 The slide version: Forouzan's key square

The slides (following Forouzan) do not use a keyword. The secret key is simply a 5 × 5 arrangement of the 25 letters, and the example key is built by dropping the letters into the matrix **diagonally, starting from the top right-hand corner**: A; then B, C; then D, E, F; and so on along each anti-diagonal (alternate diagonals shaded below). The rules are exactly the same.

WORKED EXAMPLE : slide example (Forouzan)

start

L	G	D	B	A
Q	M	H	E	C
U	R	N	I/J	F
X	V	S	O	K
Z	Y	W	T	P

Q. Encrypt "hello" with the key square shown.
A. Pairing gives he, ll, o; the ll needs a filler, giving he lx lo.

- he: same row (row 2): h→E, e→C (no wrap) ⇒ EC.
- lx: same column (column 1): l→Q, x→Z ⇒ QZ.
- lo: rectangle: l (row 1) takes o's column → B; o (row 4) takes l's column → X ⇒ BX.

Ciphertext: **ECQZBX** (matches the slide).

12.9 Strength and weakness

Strength. Playfair is far stronger than a monoalphabetic cipher for three reasons. First, there are 26 × 26 = 676 possible digrams instead of 26 letters, so an attacker must identify individual digrams, which needs a 676-entry frequency table and much more ciphertext. Second, the frequencies of digrams are much flatter than those of letters: the most common digram, th, makes up only about 3–4% of all digrams, compared with 12.7% for the letter e, so the peaks that give a monoalphabetic cipher away are low and close together. Third, single-letter frequencies are hidden, because a letter's substitute depends on its partner. With 25! possible squares, brute force is also impossible. For these reasons Playfair was long considered unbreakable and served as a field cipher in both world wars.

Weakness. Playfair is still a *monoalphabetic cipher on digrams*: a given plaintext pair always produces the same ciphertext pair (th→QK twice in the solved question). The frequencies of digrams are therefore preserved, only slightly disturbed by the inserted fillers, and a cryptanalyst can use a ciphertext-only attack based on digram frequencies to find the key. A few hundred letters of ciphertext are generally enough. The structure of the rules helps the attacker further: reversed pairs give reversed ciphertext pairs (a frequent pair such as er/re shows up as XY/YX), no letter maps to itself, and a known plaintext pair reveals which letters share a row, a column or a rectangle, from which the square can be rebuilt. Much of the structure of the plaintext language survives, which is why later ciphers moved to larger blocks (Hill) or to many alphabets (Vigenère).

★ MUST KNOW : Playfair

Key: 5 × 5 square; keyword without repeats, then the rest of the alphabet in order; **I/J share one cell** (replace J by I first). Inventor **Wheatstone** (1854), promoter **Lord Playfair**; a **digram** (multiple-letter) cipher; 25! keys.
Prepare: split into pairs; a double inside a pair gets a filler x between the letters (re-pair from there); an odd last letter gets a filler x.
Encrypt: same row → right; same column → below (both wrap); rectangle → own row, other letter's column.
Decrypt: left; up; rectangle unchanged (it is its own inverse); then drop the fillers.
Security: hides single-letter frequencies, but each digram always gives the same output, so digram frequencies leak.

Corrections to the notes / slides

- **Frequency example:** Gemini's copy of the ciphertext has an extra Z. The correct text (above) has 120 letters, with P 16 times and Z 14 times (the notes said 131 letters, P 18 times). In the notes, the step "S = h, ZSZ = that" is wrong: W = h (from ZWP = the) and S = a.
- **Short frequency example** ("GZGEWVGRNCP" → "we execute plan") is left out. The letters in the notes don't produce that sentence (11 letters vs. 13, and the given mapping yields "ewexcteplan"). Check the video at 11:08 if you need it.
- **Slide L5 (monoalphabetic key):** the slide's ciphertext ICFVQRVVNEFVRN… has 44 letters for a 45-letter plaintext. With the slide's own key, "message" is QRVVNER, so the slide dropped an R; the correct ciphertext is ICFVQRVVNE**R**FVRNVSIYRGAHSLIOJICNHTIYBFGTICRXRS.
- Neso App homework: F→A is not a wrap (F is in row 2). Only O→Y wraps.
- Video 3 (about 08:00): the speaker says "W" / "MW" but means O / mo. The slide is right: mo→ON.
- Not covered in the video: if the filler itself would double (e.g. xx), a different filler such as q or z is normally used.

13 Hill Cipher

▸ **Likely this year:** Hill encryption with a 2 × 2 or 3 × 3 key, then K^{-1} and decryption (5–8 marks); or the known-plaintext attack $K = P^{-1}C$ (3–4 marks).

Playfair hides single-letter frequencies, but it works only on pairs, its rules are fixed geometric moves, and each digram always has the same substitute, so digram statistics still leak. The **Hill cipher**, published by the mathematician **Lester Hill in 1929**, generalises the idea to blocks of any size m and replaces the geometric rules by linear algebra modulo 26. Each ciphertext letter of a block is a weighted sum of *all* m plaintext letters of that block, so the same plaintext letter becomes different ciphertext letters depending on its neighbours, and for $m \geq 3$ even digram frequencies are hidden. It is a **multiletter** (polygraphic) substitution cipher: $m = 2$ encrypts digraphs, $m = 3$ trigraphs, general m polygraphs. Because every character of a block contributes to the encryption of every other character of the block, Forouzan classifies Hill as a **block cipher**, while the letter-by-letter ciphers before it are stream ciphers.

The mathematics needed is matrix multiplication modulo 26 for encryption, and the determinant and the multiplicative inverse modulo 26 for decryption.

13.1 Encryption: matrix multiplication mod 26

The letters are numbered $a = 0, \ldots, z = 25$. The plaintext is cut into blocks of m letters (the last block is padded with a filler such as x), and each block is written as a **row vector** $P = (P_1\ P_2\ \cdots\ P_m)$. The key is an $m \times m$ matrix K with entries in $\mathbb{Z}_{26}$. A block is encrypted by multiplying it by the key and reducing every entry modulo 26; decryption multiplies by the inverse matrix.

★ MUST KNOW : Hill cipher

Encrypt: $C = PK \bmod 26$ Decrypt: $P = CK^{-1} \bmod 26$, $K^{-1} = (\det K)^{-1} \cdot \operatorname{adj}(K) \bmod 26$

Decryption works because $CK^{-1} = (PK)K^{-1} = P(KK^{-1}) = PI = P$. The key is valid only if $\gcd(\det K, 26) = 1$.

For $m = 3$ the multiplication written out is

$$(C_1\ C_2\ C_3) = (P_1\ P_2\ P_3)\begin{pmatrix} K_{11} & K_{12} & K_{13} \\ K_{21} & K_{22} & K_{23} \\ K_{31} & K_{32} & K_{33} \end{pmatrix} \bmod 26$$

$$\begin{aligned} C_1 &= (P_1K_{11} + P_2K_{21} + P_3K_{31}) \bmod 26 \\ C_2 &= (P_1K_{12} + P_2K_{22} + P_3K_{32}) \bmod 26 \\ C_3 &= (P_1K_{13} + P_2K_{23} + P_3K_{33}) \bmod 26 \end{aligned}$$

The sizes are $(1 \times 3)(3 \times 3) = (1 \times 3)$, and each C_j is the plaintext row multiplied entry by entry with column j of K and summed. The equations show directly that every ciphertext letter depends on every plaintext letter of the block.

PK **vs.** *KP*. This lecture and Forouzan use $C = PK$ (plaintext as a row). Some books (and some exam papers) write the plaintext as a **column** vector: then $C = KP$ and decryption is $P = K^{-1}C$. The two conventions give different ciphertexts for the same key (column-vector encryption with K is row-vector encryption with the transpose K^T), so state the convention and use it consistently.

13.2 Which matrices can be keys

Not every square matrix has an inverse modulo 26, and a key without one is useless, because Bob could not decrypt. The test comes from the **adjugate** $\operatorname{adj}(K)$, the transpose of the matrix of cofactors $C_{ij} = (-1)^{i+j} M_{ij}$, where the minor M_{ij} is the determinant of K with row i and column j deleted. Over the integers every square matrix satisfies

$$K \cdot \operatorname{adj}(K) = \det(K) \cdot I$$

(for the key in the next example, $K \cdot \operatorname{adj}(K) = -939\,I$, checked). If $\det K$ has a multiplicative inverse modulo 26, multiplying both sides by it gives $K \cdot \big((\det K)^{-1} \operatorname{adj}(K)\big) \equiv I$, which is the formula for K^{-1}. Conversely, if K^{-1} exists then $\det(K)\det(K^{-1}) = \det(I) = 1 \pmod{26}$, so $\det K$ must be invertible. Hence a Hill key is valid if and only if $\gcd(\det K \bmod 26, 26) = 1$, i.e. $\det K \bmod 26$ is odd and not 13. Only 12 of the 26 values qualify: $1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25$.

When the condition fails, encryption is not one-to-one. With $K = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}$ ($\det K = 2$), the blocks aa $= (0, 0)$ and na $= (13, 0)$ both encrypt to $(0, 0) =$ AA, because $13 \cdot 2 = 26 \equiv 0$. Bob, receiving AA, cannot know which was sent.

Key space. The slides count $26^{m \times m}$ matrices ($26^9 \approx 5.4 \times 10^{12}$ for $m = 3$, $26^{16} \approx 4.4 \times 10^{22}$ for $m = 4$). Only the invertible ones are keys (157,248 of the 456,976 matrices for $m = 2$), but the count grows so fast with m that brute force on a large key is hopeless.

13.3 Encryption example: "pay more money"

WORKED EXAMPLE : lecture / Stallings

Q. Encrypt "pay more money" with the Hill cipher, $C = PK \bmod 26$, $K = \begin{pmatrix} 17 & 17 & 5 \\ 21 & 18 & 21 \\ 2 & 2 & 19 \end{pmatrix}$.

A. $m = 3$, so the 12 letters form four blocks `pay | mor | emo | ney` (no padding needed). Block 1, pay $= (15\ 0\ 24)$:

$$C_1 = 15 \cdot 17 + 0 \cdot 21 + 24 \cdot 2 = 255 + 0 + 48 = 303 = 11 \cdot 26 + \mathbf{17} \quad \to \text{R}$$
$$C_2 = 15 \cdot 17 + 0 \cdot 18 + 24 \cdot 2 = 255 + 0 + 48 = 303 \equiv \mathbf{17} \quad \to \text{R}$$
$$C_3 = 15 \cdot 5 + 0 \cdot 21 + 24 \cdot 19 = 75 + 0 + 456 = 531 = 20 \cdot 26 + \mathbf{11} \quad \to \text{L}$$

Block	P	PK (before mod)	mod 26	Ciphertext
pay	(15, 0, 24)	(303, 303, 531)	(17, 17, 11)	**RRL**
mor	(12, 14, 17)	(532, 490, 677)	(12, 22, 1)	**MWB**
emo	(4, 12, 14)	(348, 312, 538)	(10, 0, 18)	**KAS**
ney	(13, 4, 24)	(353, 341, 605)	(15, 3, 7)	**PDH**

Answer: **RRLMWBKASPDH**. **Check:** decryption below returns `pay more money`.

13.4 Decryption: finding K^{-1}

By the previous subsection, $K^{-1} = (\det K)^{-1} \cdot \operatorname{adj}(K) \bmod 26$, and the computation is done in four steps, each of which should be shown in an exam answer: (1) $\det K \bmod 26$; (2) its multiplicative inverse modulo 26, which exists only if $\gcd(\det K, 26) = 1$; (3) $\operatorname{adj}(K) \bmod 26$; (4) the product of (2) and (3), reduced modulo 26, followed by the check $KK^{-1} \equiv I$.

Step 1: determinant (expand along row 1, signs + − +):

$$\det K = 17 \begin{vmatrix} 18 & 21 \\ 2 & 19 \end{vmatrix} - 17 \begin{vmatrix} 21 & 21 \\ 2 & 19 \end{vmatrix} + 5 \begin{vmatrix} 21 & 18 \\ 2 & 2 \end{vmatrix} = 17(300) - 17(357) + 5(6) = 5100 - 6069 + 30 = -939$$

$-939 = -36 \cdot 26 - 3$, so $\det K \equiv -3 \equiv -3 + 26 = \mathbf{23} \pmod{26}$. Since 23 is odd and not 13, the key is valid.

Step 2: inverse of 23 mod 26. Find x with $23x \bmod 26 = 1$ by trying $x = 1, 2, 3, \ldots$ (or by the extended Euclidean algorithm, §24):

x	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	**17**
$23x \bmod 26$	23	20	17	14	11	8	5	2	25	22	19	16	13	10	7	4	**1**

$23 \times 17 = 391 = 15 \cdot 26 + 1$, so $\mathbf{23^{-1} \equiv 17}$.

Step 3: adjugate, using the 5×5 shortcut. Computing nine cofactors and transposing is error-prone; the lecture's grid produces the adjugate directly, with the signs and the transpose built in.

1. Copy the first two columns to the right, then the first two rows to the bottom (5×5).
2. Delete the first row and first column. That leaves a 4×4 grid.
3. Slide a 2×2 window over the grid. Each window gives $ad - bc$. **Columns 1–2 of the grid give row 1 of** adj K (windows at rows 1–2, 2–3, 3–4); columns 2–3 give row 2; columns 3–4 give row 3. The signs come out automatically.

17	17	5	17	17
21	18	21	21	18
2	2	19	2	2
17	17	5	17	17
21	18	21	21	18

$\Rightarrow$

18	21	21	18
2	19	2	2
17	5	17	17
18	21	21	18

first window: $18 \cdot 19 - 21 \cdot 2 = 300$

$$\operatorname{adj} K = \begin{pmatrix} 18\cdot 19 - 21\cdot 2 & 2\cdot 5 - 19\cdot 17 & 17\cdot 21 - 5\cdot 18 \\ 21\cdot 2 - 21\cdot 19 & 19\cdot 17 - 2\cdot 5 & 5\cdot 21 - 17\cdot 21 \\ 21\cdot 2 - 18\cdot 2 & 2\cdot 17 - 2\cdot 17 & 17\cdot 18 - 17\cdot 21 \end{pmatrix} = \begin{pmatrix} 300 & -313 & 267 \\ -357 & 313 & -252 \\ 6 & 0 & -51 \end{pmatrix}$$

$$\equiv \begin{pmatrix} 14 & -1 & 7 \\ -19 & 1 & -18 \\ 6 & 0 & -25 \end{pmatrix} \equiv \begin{pmatrix} 14 & 25 & 7 \\ 7 & 1 & 8 \\ 6 & 0 & 1 \end{pmatrix} \pmod{26}$$

(Reduce mod 26, then add 26 to each negative: $-1 \to 25$, $-19 \to 7$, $-18 \to 8$, $-25 \to 1$.) The shortcut agrees with the cofactor definition: for example the entry in row 1, column 2 is the cofactor $C_{21} = -\begin{vmatrix} 17 & 5 \\ 2 & 19 \end{vmatrix} = -(323 - 10) = -313$.

Step 4: combine.

$$K^{-1} = 17 \begin{pmatrix} 14 & 25 & 7 \\ 7 & 1 & 8 \\ 6 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 238 & 425 & 119 \\ 119 & 17 & 136 \\ 102 & 0 & 17 \end{pmatrix} \equiv \boxed{\begin{pmatrix} 4 & 9 & 15 \\ 15 & 17 & 6 \\ 24 & 0 & 17 \end{pmatrix}} \pmod{26}$$

Check: $KK^{-1} = \begin{pmatrix} 443 & 442 & 442 \\ 858 & 495 & 780 \\ 494 & 52 & 365 \end{pmatrix} \equiv \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} = I$ ✓ ($443 = 17 \cdot 26 + 1$, $442 = 17 \cdot 26$, ...)

13.5 Decryption example: RRLMWBKASPDH

WORKED EXAMPLE : lecture

Q. Decrypt RRLMWBKASPDH with the key of the previous example.
A. Multiply each ciphertext block by K^{-1} found above and reduce mod 26:

Block	C	CK^{-1} (before mod)	mod 26	Plaintext
RRL	(17, 17, 11)	(587, 442, 544)	(15, 0, 24)	**pay**
MWB	(12, 22, 1)	(402, 482, 329)	(12, 14, 17)	**mor**
KAS	(10, 0, 18)	(472, 90, 456)	(4, 12, 14)	**emo**
PDH	(15, 3, 7)	(273, 186, 362)	(13, 4, 24)	**ney**

e.g. $P_1 = 17 \cdot 4 + 17 \cdot 15 + 11 \cdot 24 = 68 + 255 + 264 = 587 = 22 \cdot 26 + 15 \to$ p. Answer: **pay more money**.

13.6 A 2×2 example

For $m = 2$ the inverse has a closed form. If $K = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$, then $\det K = ad - bc$ and $\operatorname{adj} K = \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$ (swap the diagonal, negate the other two entries), so

$$K^{-1} = (ad - bc)^{-1} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix} \bmod 26.$$

WORKED EXAMPLE : new, Forouzan exercise style

Q. Using the Hill cipher with $C = PK \bmod 26$ and $K = \begin{pmatrix} 5 & 8 \\ 17 & 3 \end{pmatrix}$, encrypt "exam". Find K^{-1} and decrypt the result.

Encrypt. Blocks ex = (4, 23) and am = (0, 12).

Block	PK	mod 26
ex	$(4\cdot5+23\cdot17,\ 4\cdot8+23\cdot3)=(411,101)$	$(21,23)=$ VX
am	$(0\cdot5+12\cdot17,\ 0\cdot8+12\cdot3)=(204,36)$	$(22,10)=$ WK

Ciphertext: **VXWK**. ($411=15\cdot26+21$, $101=3\cdot26+23$, $204=7\cdot26+22$.)
Inverse. $\det K=5\cdot3-8\cdot17=15-136=-121=-5\cdot26+9\equiv9$; $\gcd(9,26)=1$, so the key is valid. $9^{-1}\equiv3$ since $9\cdot3=27=26+1$.

$$K^{-1}=3\begin{pmatrix}3&-8\\-17&5\end{pmatrix}=\begin{pmatrix}9&-24\\-51&15\end{pmatrix}\equiv\begin{pmatrix}9&2\\1&15\end{pmatrix}\pmod{26}.$$

Check: $KK^{-1}=\begin{pmatrix}5\cdot9+8\cdot1&5\cdot2+8\cdot15\\17\cdot9+3\cdot1&17\cdot2+3\cdot15\end{pmatrix}=\begin{pmatrix}53&130\\156&79\end{pmatrix}\equiv I$ ✓ ($53=2\cdot26+1$, $130=5\cdot26$, $156=6\cdot26$, $79=3\cdot26+1$).
Decrypt. VX: $(21\cdot9+23\cdot1,\ 21\cdot2+23\cdot15)=(212,387)\equiv(4,23)=$ ex. WK: $(22\cdot9+10\cdot1,\ 22\cdot2+10\cdot15)=(208,194)\equiv(0,12)=$ am. Plaintext: **exam** ✓.

13.7 The slide example: a 4×4 key (Forouzan)

WORKED EXAMPLE : slide example, corrected

Q. Encrypt "code is ready" with the 4×4 key K below ($C=PK \bmod 26$), and show that K^{-1} recovers it.
A. Removing spaces gives 11 letters, so a bogus z is added and the blocks are code | isre | adyz, stacked as the rows of a 3×4 matrix P. Multiplying all blocks at once:

$$\underbrace{\begin{pmatrix}2&14&3&4\\8&18&17&4\\0&3&24&25\end{pmatrix}}_{P}\underbrace{\begin{pmatrix}9&7&11&13\\4&7&5&6\\2&21&14&9\\3&23&21&8\end{pmatrix}}_{K}=\begin{pmatrix}92&267&218&169\\190&631&500&397\\135&1100&876&434\end{pmatrix}\equiv\underbrace{\begin{pmatrix}14&7&10&13\\8&7&6&7\\5&8&18&18\end{pmatrix}}_{C}\pmod{26}$$

For example $C_{24}=8\cdot13+18\cdot6+17\cdot9+4\cdot8=104+108+153+32=397=15\cdot26+7$. Reading C row by row gives **OHKNIHGHFISS**. With the slide's inverse

$$K^{-1}=\begin{pmatrix}2&15&22&3\\15&0&19&3\\9&9&3&11\\17&0&4&7\end{pmatrix},$$

$CK^{-1} \bmod 26$ gives back P (codeisreadyz), and Bob drops the bogus z. ($\det K\equiv3 \pmod{26}$, so the key is valid.) The slide prints OHKNIHGKLISS; two of its entries are wrong (see the corrections box).

13.8 Why Hill hides letter frequencies

In $C_j=\sum_i P_iK_{ij}$ every ciphertext letter of a block depends on every plaintext letter of the block. Two consequences follow. First, a plaintext letter has no fixed substitute: what it becomes depends on its position in the block and on the other letters there. Second, changing a single plaintext letter changes up to all m ciphertext letters of its block: changing P_1 by δ changes each C_j by δK_{1j}, which is non-zero modulo 26 whenever K_{1j} is invertible. With the lecture key, whose first row 17, 17, 5 consists of invertible numbers, pay→RRL but bay→NNT, and paz→TTE: one changed letter, three changed ciphertext letters. Since every ciphertext letter mixes m plaintext letters, the peaks of single-letter frequencies are smeared out, and for $m\geq3$ digram statistics are destroyed too.

What Hill does *not* hide is the block itself: it is a fixed substitution on blocks, so the same block always gives the same ciphertext block. In effect it is a monoalphabetic cipher on an alphabet of 26^m "letters" (676 for $m=2$, 17,576 for $m=3$). As the slides put it, Hill ciphers do not preserve the statistics of the plaintext: Eve cannot run frequency analysis on single letters, digrams or trigrams; an analysis of blocks of size m might work, but a plaintext rarely contains enough repeated blocks for it.

13.9 Cryptanalysis: the known-plaintext attack

Brute force and frequency analysis both fail, but the Hill cipher has a fatal structural weakness: it is **linear**. Suppose Eve knows m and has m pairs of plaintext/ciphertext blocks, which may come from the same message or different messages but must be distinct. She writes the plaintext blocks as the rows of an $m\times m$ matrix P and the corresponding ciphertext blocks as the rows of C. Each row satisfies $c_i=p_iK$, and matrix multiplication works row by row, so the stacked matrices satisfy $C=PK$. If P is invertible modulo 26, multiplying on the *left* by P^{-1}

gives $P^{-1}C = P^{-1}PK = K$:

$$\boxed{K = P^{-1}C \bmod 26}$$

The order matters, because matrix multiplication is not commutative (CP^{-1}, as printed on the slide, is a different matrix). If P is not invertible, Eve uses a different set of m blocks; if she does not know m, she tries $m = 2, 3, \ldots$ and checks each candidate key by decrypting more ciphertext. A **chosen-plaintext** attack is even easier: encrypting the unit blocks `baa`, `aba`, `aab` gives the rows of K directly. With the lecture key they encrypt to RRF = (17, 17, 5), VSV = (21, 18, 21) and CCT = (2, 2, 19), exactly the three rows.

WORKED EXAMPLE : known-plaintext attack (Stallings)

Q. A 2 × 2 Hill cipher ($C = PK$) encrypts "hillcipher" to HCRZSSXNSP. Find the key.
A. The pairs are `hi`→HC, `ll`→RZ, `ci`→SS, ph→XN, `er`→SP. Take the first two:

$$P = \begin{pmatrix} 7 & 8 \\ 11 & 11 \end{pmatrix} (\texttt{hi}, \texttt{ll}), \qquad C = \begin{pmatrix} 7 & 2 \\ 17 & 25 \end{pmatrix} (\text{HC}, \text{RZ}).$$

$\det P = 7 \cdot 11 - 8 \cdot 11 = -11 \equiv 15$; $\gcd(15, 26) = 1$, so P is invertible, and $15^{-1} \equiv 7$ ($15 \cdot 7 = 105 = 4 \cdot 26 + 1$).

$$P^{-1} = 7\begin{pmatrix} 11 & -8 \\ -11 & 7 \end{pmatrix} = \begin{pmatrix} 77 & -56 \\ -77 & 49 \end{pmatrix} \equiv \begin{pmatrix} 25 & 22 \\ 1 & 23 \end{pmatrix},$$

$$K = P^{-1}C = \begin{pmatrix} 25 \cdot 7 + 22 \cdot 17 & 25 \cdot 2 + 22 \cdot 25 \\ 1 \cdot 7 + 23 \cdot 17 & 1 \cdot 2 + 23 \cdot 25 \end{pmatrix} = \begin{pmatrix} 549 & 600 \\ 398 & 577 \end{pmatrix} \equiv \begin{pmatrix} 3 & 2 \\ 8 & 5 \end{pmatrix}$$

($549 = 21 \cdot 26 + 3$, $600 = 23 \cdot 26 + 2$, $398 = 15 \cdot 26 + 8$, $577 = 22 \cdot 26 + 5$.) **Check** with the unused third pair: `ci` = (2, 8) gives $(2 \cdot 3 + 8 \cdot 8,\ 2 \cdot 2 + 8 \cdot 5) = (70, 44) \equiv (18, 18) = \text{SS}$ ✓.

The slide's 3 × 3 example (worked in §17) is the same computation with three known blocks; $\det P \equiv 9$, so P is invertible:

$$P = \begin{pmatrix} 5 & 7 & 10 \\ 13 & 17 & 7 \\ 0 & 5 & 4 \end{pmatrix}, \quad C = \begin{pmatrix} 3 & 6 & 0 \\ 14 & 16 & 9 \\ 3 & 17 & 11 \end{pmatrix}, \quad P^{-1} = \begin{pmatrix} 21 & 14 & 1 \\ 0 & 8 & 25 \\ 13 & 3 & 8 \end{pmatrix}, \quad K = P^{-1}C = \begin{pmatrix} 2 & 3 & 7 \\ 5 & 7 & 9 \\ 1 & 2 & 11 \end{pmatrix}.$$

The lesson carries over to modern ciphers: a cipher whose output is a linear function of its input is broken by a handful of known pairs, which is why DES and AES contain non-linear S-boxes (§30, §31).

Corrections to the notes / slides

- All of Gemini's numbers for this batch are correct (Hill encryption, det, adjoint, K^{-1}, decryption, Vigenère, autokey; checked by script).
- Slide typos noted by Gemini: for `ney`, the slide shows the old vector (348, 312, 538) instead of (353, 341, 605). For RRL, the slide shows (17 17 14) labelled C; it should be (17 17 11) and the output is P.
- **Slide L5 (cryptanalysis of Hill):** "Because $C = PK$, Eve can use the relationship $K = CP^{-1}$" is wrong; it must be $K = P^{-1}C$ (the slide's own figure on the next page does compute $P^{-1}C$). For the slide's data, CP^{-1} gives a different, wrong matrix.
- **Slide L5 (Hill, "code is ready"):** with the slide's K, two entries of the printed C are wrong: $C_{24} = 397 \bmod 26 = 7$ (slide: 11) and $C_{31} = 135 \bmod 26 = 5$ (slide: 11). The correct ciphertext is `OHKNIHGHFISS`; the slide's text `OHKNIHGKLISS` matches neither its own matrix nor the correct one. The slide's K^{-1} is correct.

14 Polyalphabetic Ciphers

Monoalphabetic ciphers fall to frequency analysis because every letter always gets the same substitute. Playfair and Hill hide single-letter frequencies by encrypting blocks, but each block still has one fixed substitute. A **polyalphabetic** cipher attacks the problem from the other side: it keeps encrypting one letter at a time, but uses **different** monoalphabetic rules at different positions, so each occurrence of a character may have a different substitute; the relationship between plaintext and ciphertext characters is one-to-many. All polyalphabetic ciphers share two features:

1. A **set of related monoalphabetic substitution rules** is used.
2. A **key** determines which rule is used for each letter.

Since the occurrences of a frequent letter such as e are spread over several ciphertext letters, and each ciphertext letter collects several different plaintext letters, the ciphertext histogram is much flatter than the plaintext one, and Eve cannot use single-letter frequency statistics. How secure the cipher is then depends entirely on how the sequence of rules, the **key stream**, is produced. The four ciphers below are successive answers to that question:

repeat a keyword (Vigenère), continue the keyword with the plaintext (autokey), use an independent key stream of bits (Vernam), and finally a truly random key used once (the one-time pad, §15).

14.1 Vigenère cipher ★

★ **In the papers:** 2025–26 Q1 (8 marks, the biggest question so far): one-time-pad version of Vigenère, encrypt "sendmore-money" with a key stream, then find the key that turns the ciphertext into "cashnotneeded". 2024–25 Q1(c): characters changed.

▸ **Likely this year:** Encrypt or decrypt with a keyword (3–4 marks); find the key from a known plaintext ($K_i = C_i - P_i$); "explain the Kasiski test" (2–3 marks).

The Vigenère cipher (16th century) uses as its rule set the **26 Caesar ciphers** with shifts 0 to 25, each named by the letter that a is shifted to: key letter d means shift 3. The key is a keyword $k_1 k_2 \dots k_m$ of length m, and it is **repeated** as often as needed to cover the message, so plaintext letter i is encrypted with the Caesar cipher chosen by key letter i mod m. In the **Vigenère tableau**, a 26 × 26 table whose row k is the alphabet shifted by k, encryption is a lookup: the row of the key letter and the column of the plaintext letter meet at the ciphertext letter.

$$C_i = (P_i + K_{i \bmod m}) \bmod 26 \qquad P_i = (C_i - K_{i \bmod m}) \bmod 26$$

The key stream is $(k_1, \dots, k_m, k_1, \dots, k_m, \dots)$. Spaces are removed first, so the keyword runs continuously across word boundaries.

WORKED EXAMPLE : Stallings / lecture

Q. Encrypt `wearediscoveredsaveyourself` (27 letters) with the Vigenère keyword `deceptive` (9 letters).
A. The key `deceptive` is repeated three times.

Key	d	e	c	e	p	t	i	v	e	d	e	c	e	p
K	3	4	2	4	15	19	8	21	4	3	4	2	4	15
Plain	w	e	a	r	e	d	i	s	c	o	v	e	r	e
P	22	4	0	17	4	3	8	18	2	14	21	4	17	4
(P+K) mod 26	25	8	2	21	19	22	16	**13**	6	17	25	6	21	19
Cipher	**Z**	**I**	**C**	**V**	**T**	**W**	**Q**	**N**	**G**	**R**	**Z**	**G**	**V**	**T**

Key	t	i	v	e	d	e	c	e	p	t	i	v	e
K	19	8	21	4	3	4	2	4	15	19	8	21	4
Plain	d	s	a	v	e	y	o	u	r	s	e	l	f
P	3	18	0	21	4	24	14	20	17	18	4	11	5
(P+K) mod 26	22	**0**	21	25	7	**2**	16	24	**6**	**11**	12	**6**	9
Cipher	**W**	**A**	**V**	**Z**	**H**	**C**	**Q**	**Y**	**G**	**L**	**M**	**G**	**J**

Answer: **ZICVTWQNGRZGVTWAVZHCQYGLMGJ** (highlighted cells wrapped past 25). **Check:** $Z - d = 25 - 3 = 22 =$ w.

Why it is stronger. The same plaintext letter gets different ciphertext letters: the six e's of the message (positions 2, 5, 12, 14, 19, 25) become `I, T, G, T, H, M`. Conversely, the most common ciphertext letter, G, stands for c, e, r and l. A plaintext letter can have up to m different substitutes, so a longer keyword with distinct letters flattens the frequencies more.

Vigenère is m interleaved Caesar ciphers. Writing the message in rows of length m puts all letters encrypted with the same key letter into one column (Forouzan's view of Vigenère as a combination of m additive ciphers). Each column is an ordinary Caesar cipher:

Key letter (shift)	d (3)	e (4)	c (2)	e (4)	p (15)	t (19)	i (8)	v (21)	e (4)
Row 1 (plain/cipher)	w/Z	e/I	a/C	**r/V**	**e/T**	**d/W**	i/Q	s/N	c/G
Row 2	o/R	v/Z	e/G	**r/V**	**e/T**	**d/W**	s/A	a/V	v/Z
Row 3	e/H	y/C	o/Q	u/Y	r/G	s/L	e/M	l/G	f/J

Cryptanalysis. The weakness is that the **key repeats**. Breaking Vigenère has two parts: find the key length m, then find the key. For the first part the **Kasiski test** looks for repeated segments of at least three letters in the ciphertext. If the same plaintext fragment occurs at two positions whose distance is a multiple of m, it is encrypted with the same key letters both times and produces the same ciphertext fragment. In the example, the plaintext `red` at positions 4–6 and 13–15 meets the key letters `ept` both times and gives VTW twice (bold in the table); the distance is 13 − 4 = 9 = m. A repeat of three or more letters is unlikely to be a coincidence, so the cryptanalyst assumes $m \mid d$ for each distance d, and with several repeats m divides $\gcd(d_1, d_2, \dots)$. (The slide example, with distances 100, 48, 60, 8 and gcd 4, is in §17.) For the second part, the ciphertext is split into m pieces (letters $1, m{+}1, 2m{+}1, \dots$; letters $2, m{+}2, \dots$; and so on). The whole ciphertext does not preserve the single-

letter frequencies of the plaintext, but each piece does, because each piece is a Caesar cipher; each is broken by frequency analysis (its most common letter is probably e) or by trying its 26 shifts.

Known plaintext. If Eve knows some plaintext, the key falls out immediately, since $K_i = (C_i - P_i) \bmod 26$. From `ZICVTWQNGRZG` and `wearediscove` she gets 25 − 22 = 3 (d), 8 − 4 = 4 (e), ..., i.e. `deceptivedec`, which shows both the key and its period. The same subtraction answers the past-paper question of finding a key that maps a ciphertext to a chosen plaintext (the Question Bank (past papers)).

WORKED EXAMPLE : slide example (Forouzan)

Q. Encrypt "She is listening" with the Vigenère keyword "PASCAL".
A. The initial key stream is (15, 0, 18, 2, 0, 11), repeated as often as needed.

Plaintext	s	h	e	i	s	l	i	s	t	e	n	i	n	g
P	18	07	04	08	18	11	08	18	19	04	13	08	13	06
Key stream	15	00	18	02	00	11	15	00	18	02	00	11	15	00
$P + K$	**33**	07	22	10	18	22	23	18	**37**	06	13	19	**28**	06
C	07	07	22	10	18	22	23	18	11	06	13	19	02	06
Ciphertext	**H**	**H**	**W**	**K**	**S**	**W**	**X**	**S**	**L**	**G**	**N**	**T**	**C**	**G**

Ciphertext: **HHWKSWXSLGNTCG**. **Check:** $H - P = 7 - 15 = -8 \equiv 18 = s$. The two s at positions 1 and 5 became H and S, and the two H at positions 1–2 come from different letters (s, h).

14.2 Autokey system ★

★ **In the papers:** 2024–25 Q1(d): one plaintext change alters **2** ciphertext characters in auto-key (the letter itself and the one it keys).

▸ **Likely this year:** An autokey encryption (keyword version or Forouzan's single initial key, 2–4 marks); "how many ciphertext characters change" or "why is autokey still breakable" (1–2 marks).

Vigenère's weakness is the period, so the obvious fix is a key that never repeats. Vigenère himself proposed the **autokey system**: the keyword is followed by the **plaintext itself**, so the key is a **running key** as long as the message. In the lecture (Stallings) version with a keyword of length m, $k_{i+m} = P_i$. The slides (Forouzan) use the special case $m = 1$: the first subkey k_1 is a secret number agreed by Alice and Bob, the second subkey is the value of the first plaintext letter, the third is the second plaintext letter, and so on, so $k = (k_1, P_1, P_2, \ldots)$. In both versions

$$C_i = (P_i + k_i) \bmod 26 \qquad P_i = (C_i - k_i) \bmod 26.$$

Bob knows only the keyword, so he must decrypt **strictly from left to right**: each recovered plaintext letter becomes the key for a later position.

```
Key    d e c e p t i v e w e a r e d i s c o v e r e d s a v
Plain  w e a r e d i s c o v e r e d s a v e y o u r s e l f
Cipher Z I C V T W Q N G K Z E I I G A S X S T S L V V W L A
```

Answer: **ZICVTWQNGKZEIIGASXSTSLVVWLA**. The first 9 letters match Vigenère (same key); after that the key is the plaintext (`w e a r e ...`).

Why it improves on Vigenère. The key has no period, so there are no repeats at multiples of a fixed distance, the Kasiski test finds nothing, and the ciphertext cannot be split into Caesar columns. Each key letter also depends on the message, so two messages under the same keyword quickly use different keys.

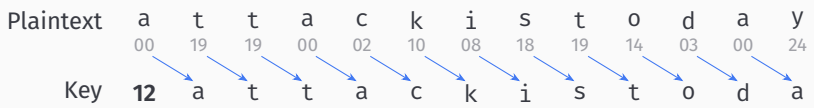
Why it is still breakable. The key is ordinary English text: key letters have English frequencies, so **key and plaintext share the same letter statistics**, and pairs of frequent letters (e+e, e+t, ...) dominate the ciphertext; statistical attacks still apply. Forouzan's version is weaker still: once k_1 is known, $P_1 = C_1 - k_1$ is the next key, and the whole message unrolls. Since k_1 has only 26 values, the autokey cipher is as vulnerable to a brute-force attack as the additive cipher.

Effect of one change. Changing plaintext letter P_i changes C_i and also the one ciphertext letter that P_i keys, C_{i+m} (C_{i+1} in Forouzan's version): exactly **2** characters, which is the past-paper answer. (Changing the `r` at position 4 of the example changes ciphertext positions 4 and 13 only.) An *error* in one ciphertext letter behaves differently: Bob decrypts a wrong P_i, uses it as the key at position $i + m$, gets that letter wrong too, and the error propagates to every m-th letter until the end of the message. In Vigenère an error stays in one letter.

WORKED EXAMPLE : slide example (Forouzan)

Q. Alice and Bob use an autokey cipher with initial key $k_1 = 12$. Encrypt "Attack is today". Then show how Eve breaks it without the key.

A. The key stream is 12 followed by the plaintext values, shifted one place:



Plaintext	a	t	t	a	c	k	i	s	t	o	d	a	y
P	00	19	19	00	02	10	08	18	19	14	03	00	24
Key stream	**12**	00	19	19	00	02	10	08	18	19	14	03	00
P + *k*	12	19	**38**	19	02	12	18	**26**	**37**	**33**	17	03	24
C	12	19	12	19	02	12	18	00	11	07	17	03	24
Ciphertext	**M**	**T**	**M**	**T**	**C**	**M**	**S**	**A**	**L**	**H**	**R**	**D**	**Y**

Ciphertext: **MTMTCMSALHRDY**. Each plaintext letter becomes the key one position later (arrows), and the last letter y keys nothing.
Brute force. Eve tries each k_1 and unrolls the chain $P_1 = C_1 - k_1$, $P_2 = C_2 - P_1$, ...: $k_1 = 11$ gives `bsuzdjjrunezz`, $k_1 = 13$ gives `zusbblhtspcbx`, and only $k_1 = 12$ gives readable text, `attackistoday`. At most 26 trials are needed.

14.3 Vernam cipher

The autokey key is not independent of the message (it *is* the message), so its statistics leak. The next step was taken by **Gilbert Vernam**, an AT&T engineer, in **1918**: use a key stream **as long as the message** that **does not repeat** and has no statistical relationship to the plaintext, so that no pattern is left for a statistical attack. Vernam's cipher works on **binary bits**, not letters, because it was built for teletype machines that send each character as a short binary code. Each plaintext bit is combined with one key bit by exclusive-OR.

$$C_i = P_i \oplus K_i \qquad P_i = C_i \oplus K_i$$

Here $\oplus$ is XOR: the output is 1 if the two bits differ and 0 if they are equal. Decryption works because XOR undoes itself:

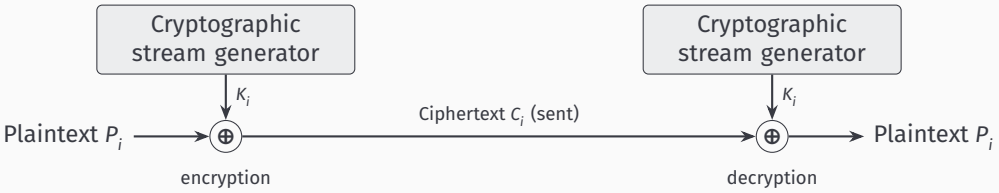
$$C_i \oplus K_i = P_i \oplus K_i \oplus K_i = P_i \oplus 0 = P_i$$

a	b	$a \oplus b$
0	0	0
0	1	1
1	0	1
1	1	0

P 1011
K 0110
C 1101
$C \oplus K$ 1011 = P

(bit example is mine, not from the lecture)

XOR is addition modulo 2, so Vernam is the Vigenère idea applied to the two-letter alphabet $\{0, 1\}$: each key bit selects one of two "Caesar ciphers" (keep the bit, or flip it). XOR is the natural choice for two reasons. It is its own inverse, so the same operation and the same key stream encrypt and decrypt. And if a key bit is 0 or 1 with probability $\frac{1}{2}$ each, independently of everything else, the ciphertext bit is also 0 or 1 with probability $\frac{1}{2}$ whatever the plaintext bit is, so a single ciphertext bit reveals nothing about the plaintext bit.



Both sides need the **same, synchronised key stream**: the receiver's generator must produce exactly the bits the sender used, in the same order.

Weakness. Vernam's machine read the key from a **loop of paper tape**, so the key eventually **repeated**, and with enough ciphertext, and some known or probable plaintext, the cipher can be broken. The reason is that whenever two stretches of plaintext are encrypted with the same key bits, the key cancels: $C_1 \oplus C_2 = (P_1 \oplus K) \oplus (P_2 \oplus K) = P_1 \oplus P_2$. For example, with P_1 = `10110010`, P_2 = `01100111` and K = `11010100`, the ciphertexts are C_1 = `01100110` and C_2 = `10110011`, and $C_1 \oplus C_2$ = `11010101` = $P_1 \oplus P_2$, with no trace of K. The XOR of two English texts is full of structure; guessing a probable word in one plaintext reveals the matching part of the other, and then $K = C \oplus P$. **Fix:** make the key truly random, as long as the message, and never reused. That is the **one-time pad** (§15), which gives perfect secrecy.

14.4 The classical substitution ciphers compared

Each cipher in this chapter was designed to remove the weakness that broke the one before it, and the summary below is the usual answer to a "differentiate" question.

★ MUST KNOW : polyalphabetic ciphers

Vigenère: $C_i = (P_i + K_{i \bmod m}) \bmod 26$, keyword repeated; weakness: the period m (Kasiski, then m Caesar ciphers). **Autokey:** key = keyword followed by the plaintext ($k_{i+m} = P_i$; Forouzan: k_1, then $k_i = P_{i-1}$); no period, but the key is English, and with a single k_1 only 26 keys; one plaintext change alters 2 ciphertext letters. **Vernam (1918):** $C_i = P_i \oplus K_i$ on bits, key stream as long as the message; weakness: the tape loop repeats, and reused key bits cancel ($C_1 \oplus C_2 = P_1 \oplus P_2$). Random, never-reused key = one-time pad.

Cipher	Unit	Key	Keys	Fixed substitute?	Main attack
Shift (Caesar)	letter	shift k	25	yes	brute force
Monoalphabetic	letter	permutation	26!	yes	letter frequencies
Playfair	digram	5×5 square	25!	per digram	digram frequencies
Hill	m letters	invertible $m \times m$ matrix	$< 26^{m^2}$	per block	known plaintext
Vigenère	letter	keyword, repeated	26^m	no (period m)	Kasiski + Caesar
Autokey	letter	keyword + plaintext	26^m	no (no period)	key is English
Vernam	bit	bit stream (tape)	–	no	reused key stream
One-time pad	letter/bit	random, used once	–	no	none (perfect)

Trap: the usual slips in Hill and Vigenère answers

In Hill, state the convention ($C = PK$ with row vectors, or $C = KP$ with columns) and keep it for decryption; check $\gcd(\det K, 26) = 1$ *before* computing the inverse; turn negative entries into 0–25 by adding 26; and always verify $KK^{-1} \equiv I$. In the known-plaintext attack the inverse goes on the left: $K = P^{-1}C$. In Vigenère, remove the spaces first so the keyword runs continuously across words; in autokey, decrypt strictly left to right.

Corrections to the notes / slides

- The notes put "key and plaintext share letter frequencies" under Vigenère cryptanalysis. That point is really about the autokey system, where the key *is* plaintext, so it is placed there above.
- Slide L5 (autokey) is correct: `attackistoday` with $k_1 = 12$ gives `MTMTCMSALHRDY`; slide L5 (Vigenère, PASCAL) is correct: `HHWKSWXSLGNTCG` (both checked by script).

15 One-Time Pad ★

★ **In the papers:** 2025–26 Q1 (8 marks): OTP-Vigenère with key stream 9 0 1 7 23 15 21 14 11 11 2 8 9 (Stallings 2.x). Part (b), finding a key that maps the ciphertext to another plaintext, is exactly why an OTP is unbreakable: every plaintext of the same length is possible.

15.1 How it grew out of the earlier ciphers

Every substitution cipher so far was broken through the same flaw: the key is reused, so some regularity of the plaintext survives into the ciphertext. A monoalphabetic cipher applies one fixed substitution to the whole message, so the letter frequencies of English pass straight through. Vigenère rotates through m substitutions, so once m is known the ciphertext splits into m interleaved Caesar ciphers (§17 shows how the Kasiski test finds m). The autokey cipher removes the period by continuing the key with the plaintext itself, but then the key is English text and carries English statistics. Vernam's cipher (1918) works on bits with a long key stream punched on paper tape, but the tape was a loop, so the key eventually repeated and a long enough message could be broken.

Shannon's study of secrecy showed what the key must look like for *no* regularity to survive: each plaintext symbol must be encrypted with a key symbol chosen **at random from the whole key domain**, independently of every other symbol. Even the weak additive cipher becomes perfect under this rule. It is broken easily because the same shift is used for every letter; but if the first letter is shifted by 04, the second by 02, the third by 21, and each shift is drawn at random from $00, 01, \ldots, 25$, then nothing links one ciphertext letter to another or to the plaintext. This is the **one-time pad** (OTP), the last substitution technique of the course and an improvement to the Vernam cipher (Stallings credits the improvement to Joseph Mauborgne; Forouzan's slides credit Vernam).

Cipher	Key	What goes wrong
Vigenère	short keyword, **repeated**	repeats every m letters ⇒ split into Caesar ciphers
Autokey	keyword + plaintext	key is English ⇒ has English letter frequencies
Vernam	long key stream (paper tape)	tape loops, so the key **eventually repeats**
One-time pad	**truly random**, **same length** as message, **used once**	nothing to exploit

15.2 Definition and mechanism

In a one-time pad the key is a **truly random** sequence that is **as long as the message**, so it never has to be repeated. The key is used to encrypt and decrypt **a single message** and is then **discarded**; every new message needs a new random key of its own length. The encryption rule itself is unchanged from Vigenère (letters) or Vernam (bits): only the key is different. Stallings calls the result "the ultimate in security", because such a scheme is unbreakable.

★ MUST KNOW : one-time pad

Key: truly random, same length as the message, used once, then destroyed.
Letters: $C_i = (P_i + K_i) \bmod 26$, $P_i = (C_i - K_i) \bmod 26$. **Bits**: $C_i = P_i \oplus K_i$, $P_i = C_i \oplus K_i$.
Perfect secrecy: $\Pr[M \mid C] = \Pr[M]$; the ciphertext reveals nothing about the plaintext. The OTP is the *only* cipher with this property (key at least as long as the message is necessary).
Two difficulties: (1) making large quantities of random keys; (2) key distribution and protection.

Decryption recovers the plaintext because the key cancels: $(P_i + K_i) - K_i \equiv P_i \pmod{26}$, and $(P_i \oplus K_i) \oplus K_i = P_i \oplus 0 = P_i$, since XOR of a bit with itself is 0. The receiver therefore needs exactly the same key, in the same order, aligned with the same letter positions; this requirement is the source of all the practical difficulties discussed below.

15.3 Example 1: letters

Plaintext `attackatdawn`, random key `qvhzmxkbplwe` (12 key letters for 12 plaintext letters). Each letter is shifted by its own key value, and four sums pass 25 and wrap:

Plain	a	t	t	a	c	k	a	t	d	a	w	n
P	0	19	19	0	2	10	0	19	3	0	22	13
Key	q	v	h	z	m	x	k	b	p	l	w	e
K	16	21	7	25	12	23	10	1	15	11	22	4
$P + K$	16	**40**	**26**	25	14	**33**	10	20	18	11	**44**	17
mod 26	16	14	0	25	14	7	10	20	18	11	18	17
Cipher	**Q**	**O**	**A**	**Z**	**O**	**H**	**K**	**U**	**S**	**L**	**S**	**R**

Ciphertext **QOAZOHKUSLSR**. The receiver computes $(C - K) \bmod 26$ letter by letter to get `attackatdawn` back, then destroys the key. The two `t`'s of `att` became O and A, and the two S's of the ciphertext came from different letters (`d` and `w`): equal letters do not produce equal letters, and equal ciphertext letters do not mean equal plaintext letters.

15.4 Example 2: bits

"HI" in 8-bit ASCII, XORed with a random 16-bit key. Each ciphertext bit is 1 exactly where the plaintext bit and the key bit differ:

Plaintext (H, I)	`01001000`	`01001001`	
Key	`10110101`	`01101100`	
Ciphertext $P \oplus K$	`11111101`	`00100101`	
$C \oplus K$	`01001000`	`01001001`	= "HI" again

15.5 Why it cannot be broken

The one-time pad produces random output that has **no statistical relationship to the plaintext**. Since the ciphertext contains no information whatsoever about the plaintext, there is simply no way to break the code, and the security of the scheme comes **entirely from the randomness of the key**. This can be seen at three levels.

One letter. Suppose the attacker sees the ciphertext letter Q (= 16). Every plaintext letter could have produced it, each with exactly one key letter, namely $K = (16 - P) \bmod 26$:

If the plaintext was	a	b	c	d	e	...	z
then the key was	q	p	o	n	m	...	r

Since the key letter was drawn at random, all 26 cases are **equally likely**, and seeing Q tells the attacker nothing. The same argument shows that the ciphertext letters themselves are uniformly distributed: $\Pr[C_i = c] = \sum_p \Pr[P_i = p] \cdot \Pr[K_i = c - p] = \sum_p \Pr[P_i = p] \cdot \frac{1}{26} = \frac{1}{26}$, whatever the plaintext statistics. The ciphertext has a flat frequency histogram (no peak for e to aim at), and since the key never repeats, the Kasiski and index-of-coincidence tests of §17 find nothing.

The whole message. For a message of n letters there are 26^n keys, exactly as many as there are n-letter strings, and every string is the decryption of the ciphertext under exactly one key. The same ciphertext therefore decrypts to anything of the same length:

Ciphertext	Key tried	Result
QOAZOHKUSLSR	qvhzmxkbplwe	**attack at dawn**
QOAZOHKUSLSR	zkhikhruztku	**retreat at six**
KXFUDM	kemubc	**attack**
KXFUDM	htaqqj	**defend**

A brute-force search produces *every* meaningful message of that length, with nothing to say which one was sent. Compare the Caesar cipher, where only one of the 25 tries makes sense (GERMANY): there the key space is far smaller than the message space, so wrong keys give gibberish and the right one stands out.

15.6 Perfect secrecy: the derivation

Definition (Shannon, 1949). A cipher has **perfect secrecy** if, for every plaintext M and every ciphertext C that can occur,

$$\Pr[M \mid C] = \Pr[M].$$

$\Pr[M]$ is what the attacker believes about the message before interception (for example, that "no attack today" is the usual message), and $\Pr[M \mid C]$ is what she believes after seeing C. Equality means the ciphertext changes nothing: given a ciphertext from a perfectly secure cipher, absolutely nothing is revealed about the plaintext.

The one-time pad meets the definition. Fix any plaintext M and ciphertext C of length n. Exactly one key maps M to C, namely $K = C - M \pmod{26}$ letter by letter (or $K = C \oplus M$ for bits). Every key is equally likely, with probability $1/26^n$, so

$$\Pr[C \mid M] = \Pr[K = C - M] = \frac{1}{26^n} \quad \text{for every } M.$$

Averaging over the messages, $\Pr[C] = \sum_M \Pr[M]\Pr[C \mid M] = \frac{1}{26^n}\sum_M \Pr[M] = \frac{1}{26^n}$. Bayes' rule then gives

$$\Pr[M \mid C] = \frac{\Pr[C \mid M]\,\Pr[M]}{\Pr[C]} = \frac{(1/26^n)\,\Pr[M]}{1/26^n} = \Pr[M].$$

In words: every key is equally likely, and for each candidate plaintext exactly one key turns it into the observed C, so C "points" equally strongly to every plaintext, and the attacker's probabilities stay where they were. For a one-bit pad with $\Pr[M = 0] = 0.9$: $\Pr[C = 1] = 0.9 \cdot 0.5 + 0.1 \cdot 0.5 = 0.5$, and $\Pr[M = 0 \mid C = 1] = 0.5 \cdot 0.9/0.5 = 0.9$; Eve's best guess after interception is exactly her guess before it.

Why the key must be as long as the message. Suppose there were fewer keys than possible messages. Decrypting a given C under all keys yields at most as many candidate plaintexts as there are keys, so some message M with $\Pr[M] > 0$ is not among them and has $\Pr[M \mid C] = 0$: the ciphertext has ruled it out, and secrecy is not perfect. For example, a Caesar shift used for a two-letter message cannot turn aa into AB, because one shift maps equal letters to equal letters; seeing AB eliminates aa, bb, ... Hence perfect secrecy needs at least as many keys as messages, i.e. at least as much random key as message. This is Shannon's bound, and it is the reason the OTP is the **only** practical cipher with perfect secrecy.

Unconditional versus computational security. Perfect secrecy is the **unconditionally secure** case of §5: no amount of ciphertext or computing power helps, because the information is simply not there. Every other cipher in this course (DES, AES, RSA, ...) is only **computationally** secure: it can be broken in principle, but doing so costs more time or money than the information is worth.

Trap: what perfect secrecy does not give

The OTP hides the *content* among messages of the *same length*; it does not hide the length (if Alice's answer is "YES" or "NO", a 3-letter ciphertext gives it away). It also gives no integrity: flipping a ciphertext bit flips the same plaintext bit, and Bob cannot detect it. Integrity needs a hash or MAC (§35).

15.7 The rules, and what breaks if you bend them

Rule 1: never reuse a key (the "two-time pad"). Encrypt a second message holdposition with the *same* key qvhzmxkbplwe; the result is C_2 = XJSCBLCJITKR. Subtracting the two ciphertexts cancels the key,

$$C_1 - C_2 = (P_1 + K) - (P_2 + K) = P_1 - P_2 \pmod{26},$$

and indeed both differences equal TFIXNWILKSIA. What remains is two English texts combined with each other, which has strong structure (common words, frequent letters) that the attacker can attack without knowing any

key. Worse, if the attacker knows or guesses P_1 ("attack at dawn" is a stock phrase), she recovers the key $K = C_1 - P_1 =$ `qvhzmxkbplwe` and then $P_2 = C_2 - K =$ **hold position**. With bits the same cancellation is $C_1 \oplus C_2 = P_1 \oplus P_2$. Historically, Soviet pads that were reused in the 1940s were partly read by the US VENONA project for exactly this reason.

Rule 2: the key must be truly random. If the key is text taken from a book (a **running key**), it has English letter frequencies and can be attacked like the autokey. If it is generated by a formula from a short seed, the cipher is really a stream cipher: at best *computationally* secure, never perfect, because the number of possible key streams is only the number of seeds.

Rule 3: the key must be at least as long as the message. A shorter key has to repeat, and then the cipher is back to Vigenère or Vernam with their weaknesses; the derivation above shows that perfect secrecy is then impossible.

Known plaintext does not hurt a correctly used OTP. If the attacker learns one plaintext and its ciphertext, she gets $K = C - P$, but only for that message, and that key is never used again. For every other cipher a known pair helps break future traffic (§17); here it is worthless.

15.8 Two fundamental difficulties

In theory the one-time pad is the end of the story; in practice it has two fundamental difficulties.

1. The practical problem of making large quantities of random keys. Every message needs fresh random key material of the same size, so a system carrying a lot of traffic needs millions of truly random characters on a regular basis. Pseudorandom generators do not qualify (Rule 2), and good physical sources of randomness are slow.
2. Even more daunting is the problem of key distribution and protection. For every message, sender and receiver must hold an identical key of the same length, delivered and stored securely, and destroyed after use. If a secure channel able to carry that much key existed, the message could simply be sent over it instead.

The second point is softened by *timing*: the key can be carried in advance, by a trusted courier, at a moment and by a route the parties choose, after which the messages can travel over any insecure channel. The slide's example is a president who sends a trusted envoy with a random key to another president before sending a completely secret message. This is why Forouzan calls the OTP "a perfect cipher, but almost impossible to implement commercially".

Advantages. Perfect secrecy: no attack, however much ciphertext or computing power the attacker has, can extract information about the plaintext. Encryption and decryption are trivial additions or XORs. **Disadvantages.** The key is as long as all the traffic ever sent, must be truly random, must reach the receiver securely in advance and must never be reused; it gives no integrity. Because of these difficulties, the one-time pad is of limited utility, and is useful primarily for low-bandwidth channels requiring very high security (diplomatic and military links).

15.9 Exam-format one-time-pad numerical

WORKED EXAMPLE : Stallings 2.x, mid-sem 2025–26 Q1

Q. This problem uses a one-time pad version of the Vigenère cipher: each plaintext letter is shifted by the next number of the key stream `9 0 1 7 23 15 21 14 11 11 2 8 9`. (a) Encrypt `sendmoremoney`. (b) Using the ciphertext from (a), find a key stream that makes it decrypt to `cashnotneeded`. (8 marks)
(a) $C_i = (P_i + K_i) \bmod 26$, letters numbered from $a = 0$:

Plain	s	e	n	d	m	o	r	e	m	o	n	e	y
P	18	4	13	3	12	14	17	4	12	14	13	4	24
K	9	0	1	7	23	15	21	14	11	11	2	8	9
$P+K$	**27**	4	14	10	**35**	**29**	**38**	18	23	25	15	12	**33**
mod 26	1	4	14	10	9	3	12	18	23	25	15	12	7
Cipher	**B**	**E**	**O**	**K**	**J**	**D**	**M**	**S**	**X**	**Z**	**P**	**M**	**H**

Ciphertext: **BEOKJDMSXZPMH**.
(b) The key that turns C into a chosen plaintext P' is $K'_i = (C_i - P'_i) \bmod 26$:

C	B 1	E 4	O 14	K 10	J 9	D 3	M 12	S 18	X 23	Z 25	P 15	M 12	H 7
P'	c 2	a 0	s 18	h 7	n 13	o 14	t 19	n 13	e 4	e 4	d 3	e 4	d 3
$C - P'$	−1	4	−4	3	−4	−11	−7	5	19	21	12	8	4
K'	**25**	**4**	**22**	**3**	**22**	**15**	**19**	**5**	**19**	**21**	**12**	**8**	**4**

Key stream: **25 4 22 3 22 15 19 5 19 21 12 8 4**. Check, first letter: $(2 + 25) \bmod 26 = 1 =$ B ✓.
Conclusion to write. The same ciphertext decrypts to `sendmoremoney` under one key and to `cashnotneeded` under another, and both keys are equally likely random streams. An attacker who tries

all keys gets every 13-letter message and cannot tell which was sent, so the one-time pad cannot be broken by brute force: it has perfect secrecy.

16 Transposition Ciphers

Substitution ciphers replace letters, and every one of them up to the one-time pad was attacked through the statistics of the replaced letters. A **transposition cipher** takes the opposite approach: some sort of **permutation is applied to the plaintext letters**. The letters themselves are **not replaced**; only their **positions** change, so a symbol in the first position of the plaintext may appear in the tenth position of the ciphertext. A **permutation** of n positions is a rearrangement in which every position receives exactly one letter; there are $n!$ of them, and the key of a transposition cipher is a permutation or a rule (depth, column order) that generates one.

Why transposition keeps the letter frequencies. Every plaintext letter appears in the ciphertext exactly once, merely at another position, so the ciphertext is an *anagram* of the plaintext: the two contain the same letters the same number of times, and their single-letter frequencies are identical. (Script check: the ciphertext of the Kill Corona example below has exactly the letter counts of its padded plaintext.) This gives the cryptanalyst an immediate test. If a ciphertext is unreadable but its letter counts look like normal English (e about 12.7%, t 9.1%, a 8.2%, and j, q, x, z almost absent), the letters are genuine and only their order is scrambled: the cipher is a **transposition**. A monoalphabetic substitution shows the same *shape* of histogram but on the wrong letters (the tallest bar is not e), and a polyalphabetic cipher flattens it. The frequency count identifies a transposition but does not break it; the attacker still has to find the order, using digram statistics (th, he, in, ...) to decide which letters belong next to each other. The simplest transposition is the rail fence.

★ MUST KNOW : transposition

Substitution changes the *letters*; transposition changes their *positions*, so **letter frequencies are preserved** (a frequency count that looks like plain English means transposition).
Rail fence, depth D: zig-zag with period $2(D-1)$, read row by row. Row–column: write row by row, read column by column, **key = order of the columns**; two stages = do it again on the ciphertext.

16.1 Rail fence

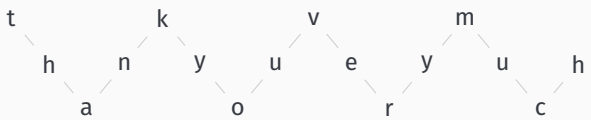
In the rail fence technique the plaintext is **written down as a sequence of diagonals and then read off as a sequence of rows**. The letters are written on D rows (the **depth**), moving down one row per letter from row 1 to row D and then back up to row 1, in a zig-zag; spaces and punctuation are dropped. The ciphertext is row 1, followed by row 2, and so on. Going down takes $D-1$ steps and coming back takes $D-1$ steps, so the row pattern repeats every $2(D-1)$ letters.

Example: "neso academy is the best", depth 2 (20 letters):



Row 1: NSAAEYSHBS Row 2: EOCDMITEET ⇒ **NSAAEYSHBSEOCDMITEET**

Depth 3, "Thank you very much" (16 letters). The path visits rows 1, 2, 3, 2, 1, 2, 3, ... and repeats every $2(D-1) = 4$ letters:



Rows: TKVM + HNYUEYUH + AORC ⇒ **TKVMHNYUEYUHAORC**. The middle row holds twice as many letters as the outer rows, because the zig-zag passes through it on the way down and again on the way up.

Homework ("neso academy is the best", depth 3): the rows are NAESB, EOCDMITEET and SAYHS, so the ciphertext is **NAESBEOCDMITEETSAYHS**.

Security. The only secret is the depth, and it can only be one of $2, 3, \ldots, L-1$ for an L-letter message, so an attacker simply tries every depth; the frequencies are English, which tells her at once that she is facing a transposition. The rail fence is therefore a scrambling rule rather than a real cipher (Forouzan files it under *keyless* transpositions, §17).

16.2 Decrypting a rail fence

The receiver knows the depth D. The difficulty is that the ciphertext does not show where one row ends and the next begins, so the first job is to work out **how many letters each row holds**; after that the zig-zag can be rebuilt

and read along its path.

1. Count the letters L. The zig-zag pattern of rows repeats every $2(D-1)$ letters.
2. Write the **row number of every position** (1, 2, ..., D, ..., 2, 1, 2, ...) and **count** how many times each row appears.
3. **Cut** the ciphertext into pieces of those lengths: the first piece is row 1, the next is row 2, and so on.
4. Put each piece back into its row's positions, left to right. Then **read along the zig-zag** (position 1, 2, 3, ...).

The counts without writing the pattern. Write $L = q \cdot 2(D-1) + r$ with $0 \le r < 2(D-1)$. Each complete cycle visits row 1 and row D once and every middle row twice, and within a cycle row i is visited at positions i and $2D - i$. So row 1 holds $q + [r \ge 1]$ letters, row D holds $q + [r \ge D]$, and a middle row i holds $2q + [r \ge i] + [r \ge 2D - i]$, where $[\cdot]$ is 1 if the condition holds and 0 otherwise. For $L = 17$, $D = 4$: $17 = 2 \cdot 6 + 5$, giving 3, 5, 6, 3, as in Example 3 below.

Example 1: depth 2. NSAAEYSHBSEOCDMITEET (20 letters). The pattern is 1 2 1 2 ..., so each row has 10 letters: row 1 = NSAAEYSHBS, row 2 = EOCDMITEET. Take letters alternately from row 1 and row 2: N**E**S**O**A**C**... ⇒ **nesoacademyisthebest**. (Depth-2 shortcut: split in half and interleave. With an odd length, row 1 gets the extra letter.)

Example 2: depth 3. TKVMHNYUEYUHAORC (16 letters). The pattern repeats every $2(3-1) = 4$:

Position	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Row	1	2	3	2	1	2	3	2	1	2	3	2	1	2	3	2

Row 1 appears 4 times, row 2 8 times, row 3 4 times. Cut: TKVM | HNYUEYUH | AORC. Fill the row-1 positions (1, 5, 9, 13) with T K V M, the row-2 positions with H N Y U E Y U H, and the row-3 positions with A O R C. Reading positions 1 to 16 gives **thankyouverymuch**.

Example 3: depth 4, unequal rows (the tricky case). CGIRORYSYTAHFNPPU (17 letters). The pattern repeats every $2(4-1) = 6$:

Position	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
Row	1	2	3	4	3	2	1	2	3	4	3	2	1	2	3	4	3

Counts: row 1 = 3, row 2 = 5, row 3 = 6, row 4 = 3 (total 17 ✓). Cut: CGI | RORYS | YTAHFN | PPU. Put the pieces back:



Read along the line: **cryptographyisfun** ("cryptography is fun"). The same steps, with row counts 5, 10, 5, turn the homework answer above back into nesoacademyisthebest.

Trap: unequal rows

Do not assume the rows are equal, or that the middle rows are equal to each other. In Example 3 rows 2 and 3 have different lengths (5 and 6) because the message stops partway through a cycle, after the zig-zag has passed row 3 on its way up but before it reaches row 2.

16.3 Row–column transposition

The rail fence has too few keys to be of any use. A more complex scheme uses a **rectangle**: the message is **written row by row** into a rectangle of c columns and **read column by column**, but the columns are read in a **secret order**. The **key** is that order of the columns, written as a number above each column (the column marked 1 is read first). Leftover cells in the last row are filled with agreed filler letters (x, y, z), so that every column has the same height and the receiver can remove the fillers after decryption.

Example: "Kill Corona Virus at twelve am tomorrow" (33 letters), 7 columns × 5 rows = 35 cells, so 2 fillers (y, z). Key **4 3 1 2 5 6 7**.

4	3	**1**	2	5	6	7
k	i	l	l	c	o	r
o	n	a	v	i	r	u
s	a	t	t	w	e	l
v	e	a	m	t	o	m
o	r	r	o	w	y	z

numbers above = read order

Order	Column	Letters
1	col 3	LATAR
2	col 4	LVTMO
3	col 2	INAER
4	col 1	KOSVO
5	col 5	CIWTW
6	col 6	OREOY
7	col 7	RULMZ

Answer: **LATARLVTMOINAERKOSVOCIWTWOREOYRULMZ**

Key space and weakness. With c columns there are $c!$ reading orders ($7! = 5040$ here), multiplied by the choice of c, so brute force by hand is already tedious. The weakness is structural. Consecutive ciphertext letters come from one column, i.e. from plaintext positions exactly c apart: the first five ciphertext letters are plaintext letters 3, 10, 17, 24, 31. An attacker who guesses the number of rows (here 5, a divisor of 35) cuts the ciphertext into 5-letter pieces, stands them side by side as columns and rearranges the columns until the rows read as English, guided by common digrams; Forouzan calls this the **pattern attack** (§17). This is why the lecture says a single transposition "can be reconstructed".

WORKED EXAMPLE : decrypting a row–column transposition

Q. Decrypt LATARLVTMOINAERKOSVOCIWTWOREOYRULMZ, written row by row into 7 columns and read in the column order **4 3 1 2 5 6 7**.
Column height. $L = 35$ letters in 7 columns, so every column holds $35/7 = 5$ letters (5 rows).
Refill the columns in key order. The first 5 letters belong to the column numbered 1 (column 3), the next 5 to the column numbered 2 (column 4), and so on:

Piece	LATAR	LVTMO	INAER	KOSVO	CIWTW	OREOY	RULMZ
Goes to column	3	4	2	1	5	6	7

Read row by row. Column 1 is now KOSVO, column 2 INAER, column 3 LATAR, and so on; reading the rebuilt grid across its rows gives `killcor | onaviru | sattwel | veamtom | orrowyz`. Removing the fillers yz, the plaintext is **kill corona virus at twelve am tomorrow**. Check: re-encrypting gives the received ciphertext ✓.

16.4 Two stages: more complex

A single transposition leaves the regular pattern just described. The remedy is to **feed the ciphertext back in** and transpose again, usually with the same key:

4	3	**1**	2	5	6	7
L	A	T	A	R	L	V
T	M	O	I	N	A	E
R	K	O	S	V	O	C
I	W	T	W	O	R	E
O	Y	R	U	L	M	Z

Read columns 3, 4, 2, 1, 5, 6, 7:
`TOOTR AISWU AMKWY LTRIO RNVOL LAORM VECEZ`

Answer: **TOOTRAISWUAMKWYLTRIORNVOLLAORMVECEZ**. The full key is now: the rectangle size, the column order and the number of stages. To decrypt, the receiver applies the single-stage decryption of the worked example twice (the second stage is undone first).

Why two stages are stronger. After one stage, ciphertext positions 1–5 hold plaintext letters 3, 10, 17, 24, 31: a run with constant step 7. After two stages they hold plaintext letters 17, 32, 8, 26, 7 (computed by composing the two permutations), and no constant step appears anywhere in the ciphertext. Cutting the ciphertext into columns therefore no longer produces fragments of plaintext rows, and the attacker must undo both permutations at once instead of one rectangle at a time. Two stages are still a single permutation of the 35 positions (so the letter frequencies are still those of English), but it is not a *columnar* permutation: it lies outside the small, regular family that the pattern attack searches. §17 repeats this analysis on Forouzan's keyed columnar example.

Corrections to the notes / slides

- "Thank you very much", depth 3, is TKVMHNYUEYUH**AORC**. The notes' final string ends in AORE, but their own row reading gives AORC.
- Row–column: the speaker says "R U L M C" for column 7, but the grid (and slide) give RULMZ. All other results were checked by script and are correct.
- The OTP formulas, Shannon and $P(M \mid C) = P(M)$ come from Gemini's notes, not the slides. They are standard and correct.

17 Traditional Ciphers: Remaining Topics (Lecture 5)

The college lecture follows Forouzan's chapter 3, which treats the classical ciphers as arithmetic in $\mathbb{Z}_{26}$ (letters $a = 0, \ldots, z = 25$) and pays more attention to how each cipher is *broken*. This section collects what the Neso videos did not cover: the multiplicative and affine ciphers and the attack on them, the cryptanalysis of Vigenère and Hill, rotor machines, Forouzan's family of transposition ciphers with their attacks, and the stream/block classification. Throughout, Eve knows the algorithm (Kerckhoff's principle, §6) and only the key is secret.

17.1 Multiplicative and affine ciphers ★

★ **In the papers:** 2024–25 Q1(a,b): one plaintext change alters only **1** ciphertext character in multiplicative and affine ciphers (each letter is enciphered on its own).

The additive (shift) cipher has only 26 keys, one of them useless, so it falls to brute force. The obvious variation is to *multiply* instead of add. In the **multiplicative cipher** the plaintext and ciphertext are integers in $\mathbb{Z}_{26}$ and the key k is an integer in $\mathbb{Z}_{26}^*$:

$$C = (P \times k) \bmod 26, \qquad P = (C \times k^{-1}) \bmod 26.$$

Why the key must be in $\mathbb{Z}_{26}^*$. Encryption must be one-to-one, otherwise two plaintext letters share a ciphertext letter and Bob cannot tell which was sent. If $d = \gcd(k, 26) > 1$, then P and $P + 26/d$ encrypt to the same letter, since $k(P + 26/d) = kP + 26 \cdot (k/d) \equiv kP$. With $k = 2$, for example, a (0) and n (13) both become A, and b and o both become C. If $\gcd(k, 26) = 1$, k has a multiplicative inverse k^{-1} modulo 26 (extended Euclid, §24), so multiplying by k^{-1} undoes the encryption. The usable keys are therefore exactly the elements of

$$\mathbb{Z}_{26}^* = \{1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25\}$$

(the odd numbers except 13), and there are $\phi(26) = \phi(2)\phi(13) = 1 \times 12 = 12$ of them (§25). The key domain is even smaller than the additive one.

The affine cipher combines the two: the first key multiplies, the second adds,

$$C = (P \times k_1 + k_2) \bmod 26, \qquad P = \left((C - k_2) \times k_1^{-1}\right) \bmod 26,$$

where $k_1 \in \mathbb{Z}_{26}^*$ and $k_2 \in \mathbb{Z}_{26}$. Decryption reverses the two steps in the opposite order: adding the additive inverse $-k_2$ undoes the addition, and multiplying by k_1^{-1} undoes the multiplication. The additive cipher is the special case $k_1 = 1$ and the multiplicative cipher the case $k_2 = 0$.

Why the affine key space is 312. k_1 can be any of the 12 elements of $\mathbb{Z}_{26}^*$ and k_2 any of the 26 elements of $\mathbb{Z}_{26}$, giving $12 \times 26 = 312$ pairs. No two pairs give the same substitution: if (k_1, k_2) and (k_1', k_2') encrypted every letter identically, then $P = 0$ would give $k_2 = k_2'$ and $P = 1$ would then give $k_1 = k_1'$. So the affine cipher has exactly 312 keys (311 excluding the identity (1, 0); a script check finds 312 distinct substitution tables). This is a tiny subset of the 26! monoalphabetic substitutions: brute force over 312 keys is trivial, and since each letter always maps to the same letter, the cipher also keeps the English letter frequencies.

One plaintext change changes one ciphertext letter. Each letter is enciphered on its own, with the same key and without reference to its neighbours, so changing P_i changes C_i and nothing else. This holds for the additive, multiplicative and affine ciphers alike.

Cipher	Encrypt	Decrypt / key space
Additive (shift)	$C = (P + k) \bmod 26$	$P = (C - k) \bmod 26$; 26 keys
Multiplicative	$C = (P \times k) \bmod 26$	$P = (C \times k^{-1}) \bmod 26$; $k \in \mathbb{Z}_{26}^*$: **12 keys**
Affine	$C = (Pk_1 + k_2) \bmod 26$	$P = ((C - k_2)k_1^{-1}) \bmod 26$; $12 \times 26 =$ **312** keys

★ MUST KNOW : multiplicative and affine

k (or k_1) must be in $\mathbb{Z}_{26}^* = \{1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25\}$, i.e. $\gcd(k, 26) = 1$, so that k^{-1} exists. Inverses: $3 \leftrightarrow 9$, $5 \leftrightarrow 21$, $7 \leftrightarrow 15$, $11 \leftrightarrow 19$, $17 \leftrightarrow 23$, 1 and 25 are their own.
Key spaces: additive 26, multiplicative 12, affine $12 \times 26 = 312$. Two plaintext/ciphertext pairs determine an affine key.

WORKED EXAMPLE : Forouzan, "hello" with the multiplicative and affine ciphers

Q. (a) Encrypt "hello" with the multiplicative cipher, key 7. (b) Encrypt "hello" with the affine cipher, key pair (7, 2). (c) Decrypt ZEBBW with the key pair (7, 2).
Since $\gcd(7, 26) = 1$, both keys are valid; $7^{-1} \bmod 26 = 15$ because $7 \times 15 = 105 = 4 \times 26 + 1$, and the additive inverse of 2 is $-2 \equiv 24$.

Letter	P	$7P$	(a) mod 26	$7P + 2$	mod 26	(b) C
h	7	49	23 X	51	25	Z
e	4	28	2 C	30	4	E
l	11	77	25 Z	79	1	B
l	11	77	25 Z	79	1	B
o	14	98	20 U	100	22	W

(a) **XCZZU**. (b) **ZEBBW**.
(c) $P = (C - 2) \times 15 \bmod 26$:

Cipher	C	$C-2$	$\times 15$	mod 26	Plain
Z	25	23	345	7	h
E	4	2	30	4	e
B	1	−1	−15	11	l
B	1	−1	−15	11	l
W	22	20	300	14	o

Plaintext **hello** ✓. The doubled l gives a doubled B: the cipher is monoalphabetic.

17.2 Breaking the affine cipher: two pairs give (k_1, k_2)

Brute force and frequency analysis both work on an affine ciphertext, but the cipher's linearity allows something faster. Each known plaintext/ciphertext letter pair (P, C) gives one linear congruence in the two unknowns,

$$P \times k_1 + k_2 \equiv C \pmod{26}.$$

Two pairs give two congruences, and subtracting them eliminates k_2:

$$(P_1 - P_2)\, k_1 \equiv C_1 - C_2 \pmod{26}.$$

If $\gcd(P_1 - P_2, 26) = 1$, multiplying by $(P_1 - P_2)^{-1}$ gives a unique k_1, and then $k_2 = C_1 - P_1 k_1 \bmod 26$. The same solution in matrix form, as on the slides, is $\binom{k_1}{k_2} = \begin{pmatrix} P_1 & 1 \\ P_2 & 1 \end{pmatrix}^{-1} \binom{C_1}{C_2}$. Finally the candidate must be checked: a k_1 outside $\mathbb{Z}_{26}^*$ cannot be an affine key, which means the data did not come from an affine cipher (or one of the pairs is wrong). If $\gcd(P_1 - P_2, 26) = 2$, the congruence has two solutions k_1 and $k_1 + 13$ (or none); one of them is even and is rejected, so normally one key remains. Where the pairs come from decides the name of the attack: *chosen-plaintext* if Eve types the plaintext herself, *known-plaintext* if she merely learns it, and *ciphertext-only* if she guesses that the two most frequent ciphertext letters stand for e and t.

WORKED EXAMPLE : slide example, chosen-plaintext attack on the affine cipher

Q. Eve intercepts PWUFFOGWCHFDWIWEJOUUNJORSMDWRHVCMWJUPVCCG. She briefly gets access to Alice's computer and has time to type only "et". She is not sure which of two algorithms is the affine cipher: algorithm 1 gives WC, algorithm 2 gives WF. Find the key and the plaintext.
Algorithm 1 (e = 4 → W = 22, t = 19 → C = 2):

$$4k_1 + k_2 \equiv 22, \qquad 19k_1 + k_2 \equiv 2 \pmod{26}.$$

Subtract: $15k_1 \equiv 2 - 22 = -20 \equiv 6$. Since $15^{-1} = 7$ $(15 \times 7 = 105 = 4 \times 26 + 1)$, $k_1 = 6 \times 7 = 42 \bmod 26 = 16$, and $k_2 = 22 - 4 \times 16 = -42 \equiv 10$. In matrix form: $\det\left(\begin{smallmatrix} 4 & 1 \\ 19 & 1 \end{smallmatrix}\right) = 4 - 19 = -15 \equiv 11$, $11^{-1} = 19$, inverse $= 19 \left(\begin{smallmatrix} 1 & -1 \\ -19 & 4 \end{smallmatrix}\right) \equiv \left(\begin{smallmatrix} 19 & 7 \\ 3 & 24 \end{smallmatrix}\right)$, and $\left(\begin{smallmatrix} 19 & 7 \\ 3 & 24 \end{smallmatrix}\right)\binom{22}{2} = \binom{432}{114} \equiv \binom{16}{10}$. **Rejected**: $\gcd(16, 26) = 2$, so 16 has no inverse in $\mathbb{Z}_{26}^*$ (indeed $16 \times 0 \equiv 16 \times 13 \equiv 0$), and algorithm 1 is not the affine cipher.
Algorithm 2 (t = 19 → F = 5): $15k_1 \equiv 5 - 22 = -17 \equiv 9$, so $k_1 = 9 \times 7 = 63 \bmod 26 = \mathbf{11}$ and $k_2 = 22 - 4 \times 11 = -22 \equiv \mathbf{4}$. The coefficient matrix is the same as for algorithm 1 (same plaintext "et"), so with the same inverse: $\left(\begin{smallmatrix} 19 & 7 \\ 3 & 24 \end{smallmatrix}\right)\binom{22}{5} = \binom{453}{186} \equiv \binom{11}{4}$. Accepted: $\gcd(11, 26) = 1$.
Decrypt. The decryption key is the pair of inverses $(k_1^{-1}, -k_2) = (19, 22)$, since $11 \times 19 = 209 = 8 \times 26 + 1$: $P = 19(C + 22) \bmod 26$. First letters: P = 15: $19 \times 37 = 703 \equiv 1$ = b; W = 22: $19 \times 44 = 836 \equiv 4$ = e; U = 20: $19 \times 42 = 798 \equiv 18$ = s; F = 5: $19 \times 27 = 513 \equiv 19$ = t. The whole text is **best time of the year is spring when flowers bloom**. Check: encrypting it with $(11, 4)$ gives the intercepted ciphertext ✓.

WORKED EXAMPLE : new, known pair with an even difference

Q. An affine cipher encrypts "go" as MA. Find the key and decrypt MAAXLESG.
Equations. g = 6 → M = 12 and o = 14 → A = 0: $6k_1 + k_2 \equiv 12$ and $14k_1 + k_2 \equiv 0$. Subtracting the first from the second: $8k_1 \equiv -12 \equiv 14 \pmod{26}$.
Solve. $\gcd(8, 26) = 2$ divides 14, so divide everything by 2: $4k_1 \equiv 7 \pmod{13}$. $4^{-1} \bmod 13 = 10$ $(40 = 3 \times 13 + 1)$, so $k_1 \equiv 70 \equiv 5 \pmod{13}$, i.e. $k_1 \in \{5, 18\}$ modulo 26. 18 is even, hence not in $\mathbb{Z}_{26}^*$: $k_1 = \mathbf{5}$, and $k_2 = 12 - 6 \times 5 = -18 \equiv \mathbf{8}$.
Decrypt with $5^{-1} = 21$ $(105 = 4 \times 26 + 1)$: $P = 21(C - 8) \bmod 26$.

C	M 12	A 0	A 0	X 23	L 11	E 4	S 18	G 6
$21(C-8)$	84	−168	−168	315	63	−84	210	−42
mod 26	6	14	14	3	11	20	2	10
P	g	o	o	d	l	u	c	k

Plaintext **good luck**. Check: $5 \times 6 + 8 = 38 \equiv 12$ = M ✓.

17.3 Cryptanalysis of Vigenère: Kasiski test and index of coincidence

Vigenère ciphers, like all polyalphabetic ciphers, do not preserve the frequency of characters: each plaintext letter is shifted by one of m different amounts, so the peaks of English are spread out. Their weakness is the *period*. Every m-th letter is shifted by the same key letter, so the ciphertext is really m interleaved additive ciphers (§14). Cryptanalysis therefore has two parts: **finding the length m of the key**, and then **finding the key itself**.

The Kasiski test. English repeats short fragments constantly (`the`, `ing`, `and`, `tion`). When the same fragment occurs twice at a distance that is a multiple of m, both occurrences line up with the same key letters, so they are enciphered identically and the ciphertext shows a repeated segment. The Kasiski test therefore searches the ciphertext for repeated segments of **at least three characters** and measures the distance d between them; the cryptanalyst assumes that m divides d. With several repeats at distances $d_1, d_2, \ldots, d_n$, m divides all of them, so

$$m \mid \gcd(d_1, d_2, \ldots, d_n).$$

A chance repetition (two different plaintext fragments under different key letters that happen to give the same three ciphertext letters) is rare, which is why short repeats of one or two letters are ignored; a distance that does not share the common factor is treated as such a coincidence and left out of the gcd.

Finding the key. Once m is known, the ciphertext is divided into m pieces: piece j holds letters $j, j+m, j+2m, \ldots$. All letters of one piece were shifted by the same key letter, so each piece is an additive cipher on a sample of English and is broken by the methods for the additive cipher: frequency analysis, or simply trying all 26 shifts. The whole ciphertext does not preserve the single-letter frequencies of the plaintext, but each piece does. The deciphered pieces are then interleaved one character at a time to give the plaintext; if it does not make sense, another m is tried.

The index of coincidence (Friedman) finds m without needing repeats. The IC of a text of N letters, in which letter i occurs f_i times, is the probability that two letters picked at random from it are equal:

$$\mathrm{IC} = \frac{\sum_i f_i(f_i - 1)}{N(N-1)}.$$

For English it is about $\sum p_i^2 \approx 0.066$ (0.0655 from the standard frequency table), while for uniformly random letters it is $1/26 \approx 0.038$. A monoalphabetic substitution only renames the letters, so it leaves the IC unchanged; a Vigenère ciphertext mixes m alphabets, so its IC falls towards 0.038 as m grows. To find m, split the ciphertext into m pieces for each candidate m and average the pieces' IC: for the right m (and its multiples) every piece is a single Caesar cipher and its IC jumps to about 0.066.

WORKED EXAMPLE : slide example, Forouzan's Vigenère ciphertext

Q. Find the key length and the key of the 175-letter Vigenère ciphertext

```
LIOMWGFEGGDVWGHHCQUCRHRWAGWIOWQLKGZETKKMEVLWPCZVGTHVTSGXQOVGCSVETQLTJSU
MVWVEUVLXEWSLGFZMVVWLGYHCUSWXQHKVGSHEEVFLCFDGVSUMPHKIRZDMPHHBVWVWJWIXGF
                    WLTSHGJOUEEHHVUCFVGOWICQLTJSUXGLW
```

Kasiski. Repetitions of three-character segments (positions as on the slide, counted from 0):

String	First index	Second index	Difference
JSU	68	168	100
SUM	69	117	48
VWV	72	132	60
MPH	119	127	8

(QLT, LTJ and TJS also repeat at distance 100; they are part of the same repeated block QLTJSU.) $100 = 4 \cdot 25$, $48 = 4 \cdot 12$, $60 = 4 \cdot 15$, $8 = 4 \cdot 2$, so gcd = 4 and $m \in \{1, 2, 4\}$; try $m = 4$ first.
Index of coincidence of the pieces for each candidate m:

m	1	2	3	4	5	6
average IC of the m pieces	0.053	0.056	0.050	**0.077**	0.059	0.053

Only $m = 4$ (and its multiple 8, IC 0.069) reaches the English level; the values exceed 0.066 because each piece is only 43–44 letters long. Both tests agree: **m = 4**.
Break each piece. Piece C1 is letters 1, 5, 9, ..., piece C2 letters 2, 6, 10, ...:

Piece	Most frequent	Shift if top = e	Best shift (all 26 tried)	Key letter
C1	G (8 of 44)	2	2	c
C2	S (8 of 44)	14	14	o
C3	H (7 of 44)	3	3	d
C4	M, E, V (6 each of 43)	8 (if M)	4	e

For C4 the single most frequent letter is ambiguous, so all 26 shifts are tried and the one whose output has the most English-like letter distribution wins: shift 4 turns M, E, V into i, a, r, three common letters. The key is **code** (2, 14, 3, 4). Interleaving the deciphered pieces gives the slide's plaintext: *"Julius Caesar used a cryptosystem in his war, which is now referred to as Caesar cipher. It is a shift cipher with the key set to three. Each character in the plaintext is shifted three characters to create ciphertext."* (The text the slide actually encrypted contained a few typos; see the corrections box.)

Why neither test works on the one-time pad. Its key has no period, so repeated ciphertext segments are pure coincidences whose distances have no common factor, and every way of splitting the ciphertext gives pieces with the random IC of about 0.038.

17.4 Cryptanalysis of Hill: known-plaintext

Why the Hill cipher resists the earlier attacks. A brute-force attack on a Hill cipher is extremely difficult, because the key is an $m \times m$ matrix and each entry can take one of 26 values: the key domain has size $26^{m \times m}$ ($26^4 = 456\,976$ matrices for $m = 2$, of which 157 248 are invertible; $26^9 \approx 5.4 \times 10^{12}$ for $m = 3$). Statistical attacks fail too. Each ciphertext letter $C_j = P_1 k_{1j} + P_2 k_{2j} + \cdots + P_m k_{mj}$ depends on *all* m letters of its block, so the same plaintext letter is enciphered differently depending on its neighbours, and Eve cannot run frequency analysis on single letters, digrams or trigrams. Only whole blocks of m letters that repeat at block-aligned positions repeat in the ciphertext, and a plaintext rarely has many such repeats.

Why it falls to a known-plaintext attack. The cipher is *linear*: every block satisfies $C = PK \pmod{26}$. Suppose Eve knows m and has the plaintext/ciphertext pairs of at least m distinct blocks (from the same message or from different ones). She writes the m plaintext blocks as the rows of an $m \times m$ matrix P and the corresponding ciphertext blocks as the rows of C. Because each row satisfies the cipher equation, the whole matrices satisfy $C = PK$, and if P is invertible modulo 26 (i.e. $\gcd(\det P, 26) = 1$),

$$\boxed{K = P^{-1}C \pmod{26}}.$$

If P is not invertible, Eve chooses a different set of m pairs. If she does not know m, she tries $m = 2, 3, \ldots$ (provided m is not very large) and tests each candidate key on one more known block. With K she computes K^{-1} and reads every message encrypted with that key. The lesson carried into modern ciphers is that **linearity is fatal**: DES and AES add non-linear S-boxes precisely so that no such matrix equation exists.

WORKED EXAMPLE : Stallings, 2×2 Hill from a known plaintext

Q. The plaintext "friday" is encrypted with a 2×2 Hill cipher (row-vector convention $C = PK$) to give PQCFKU. Find the key.
Pairs. fr $(5, 17) \to$ PQ $(15, 16)$; id $(8, 3) \to$ CF $(2, 5)$; ay $(0, 24) \to$ KU $(10, 20)$. Use the first two:

$$P = \begin{pmatrix} 5 & 17 \\ 8 & 3 \end{pmatrix}, \qquad C = \begin{pmatrix} 15 & 16 \\ 2 & 5 \end{pmatrix}.$$

Invert P. $\det P = 5 \cdot 3 - 17 \cdot 8 = -121 \equiv 9 \pmod{26}$; $\gcd(9, 26) = 1$ and $9^{-1} = 3$ ($27 = 26 + 1$).

$$P^{-1} = 3 \begin{pmatrix} 3 & -17 \\ -8 & 5 \end{pmatrix} = \begin{pmatrix} 9 & -51 \\ -24 & 15 \end{pmatrix} \equiv \begin{pmatrix} 9 & 1 \\ 2 & 15 \end{pmatrix}.$$

Key.

$$K = P^{-1}C = \begin{pmatrix} 9 \cdot 15 + 1 \cdot 2 & 9 \cdot 16 + 1 \cdot 5 \\ 2 \cdot 15 + 15 \cdot 2 & 2 \cdot 16 + 15 \cdot 5 \end{pmatrix} = \begin{pmatrix} 137 & 149 \\ 60 & 107 \end{pmatrix} \equiv \begin{pmatrix} 7 & 19 \\ 8 & 3 \end{pmatrix}.$$

Check with the third pair: $(0, 24)K = (24 \cdot 8,\ 24 \cdot 3) = (192, 72) \equiv (10, 20) =$ KU ✓.

WORKED EXAMPLE : slide example, $m = 3$

Q. Eve knows $m = 3$ and has intercepted three plaintext/ciphertext block pairs (not necessarily from the same

message): $(05,07,10) \to (03,06,00)$, $(13,17,07) \to (14,16,09)$, $(00,05,04) \to (03,17,11)$. Find K.

$$P = \begin{pmatrix} 5 & 7 & 10 \\ 13 & 17 & 7 \\ 0 & 5 & 4 \end{pmatrix}, \quad C = \begin{pmatrix} 3 & 6 & 0 \\ 14 & 16 & 9 \\ 3 & 17 & 11 \end{pmatrix}, \quad \det P = 451 \equiv 9, \quad 9^{-1} = 3.$$

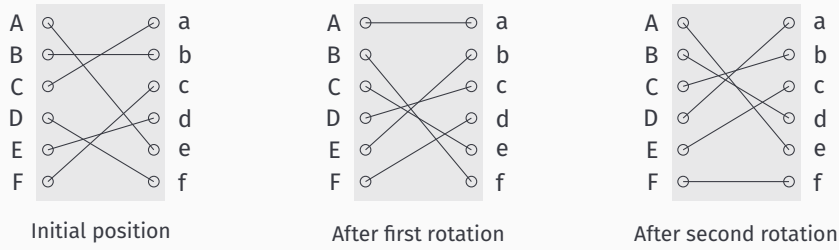
With the adjugate of P reduced mod 26,

$$\operatorname{adj} P \equiv \begin{pmatrix} 7 & 22 & 9 \\ 0 & 20 & 17 \\ 13 & 1 & 20 \end{pmatrix}, \qquad P^{-1} = 3 \cdot \operatorname{adj} P \equiv \begin{pmatrix} 21 & 14 & 1 \\ 0 & 8 & 25 \\ 13 & 3 & 8 \end{pmatrix}, \qquad K = P^{-1}C \equiv \begin{pmatrix} 2 & 3 & 7 \\ 5 & 7 & 9 \\ 1 & 2 & 11 \end{pmatrix}.$$

(All entries mod 26, verified: $PP^{-1} \equiv I$ and $PK \equiv C$.) Now Eve has the key and can break any ciphertext encrypted with it.

17.5 Rotor ciphers and Enigma

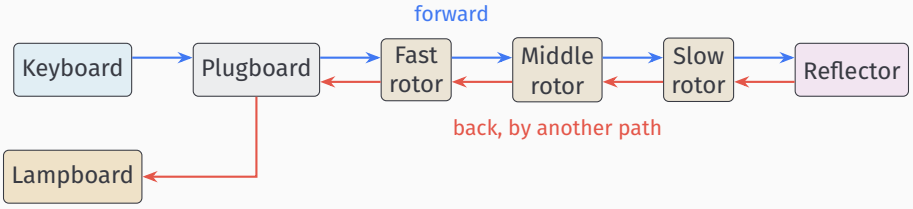
Rotor cipher. The one-time pad is not practical, but monoalphabetic substitution is far too weak. One step toward more secure encipherment is the **rotor cipher**: it uses the idea of monoalphabetic substitution, but **changes the mapping between plaintext and ciphertext characters after every character**. The mapping is realised by a **rotor**, a disc with one contact per letter on each face and a fixed internal wiring that connects each input contact to an output contact; the wiring is one permutation of the alphabet. After each character the rotor turns by one position, so every wire meets different letters and the next character is encrypted with a shifted permutation. The key is the initial position of the rotor (and the wiring itself if it is secret). Since the same letter is enciphered differently at different positions, the rotor cipher is **polyalphabetic**, and it is a stream cipher whose key stream is the sequence of rotor positions.



Example (the slide's 6-letter rotor; plaintext on the right, ciphertext on the left). In the initial position b is wired to B and e to A; after the first rotation e is wired to C; after the second, e is wired to A again. If the rotor were stationary (a monoalphabetic substitution), "bee" would be encrypted as BAA. With the rotor turning after each letter it becomes BCA: the two e's are encrypted as different characters. A single rotor of 26 contacts repeats its sequence of permutations every 26 letters, which is still a short period.

The Enigma machine was invented by Arthur Scherbius in 1918 for businesses that needed to send confidential documents without codebooks; it was modified by the German army and used extensively in World War II. It is based on the principle of rotor ciphers, with three rotors connected so that the **wiring changes as you type**. Its main parts are:

- a **keyboard** for typing the plaintext and a **lampboard** on which the ciphertext letter lights up;
- a **plugboard** that swaps chosen pairs of letters on the way in and again on the way out;
- three **rotors**: the *fast* rotor turns one position with every key press, the *middle* rotor turns once per revolution of the fast rotor, and the *slow* rotor once per revolution of the middle one (like the wheels of an odometer), so the sequence of substitutions repeats only after about $26^3 = 17\,576$ letters;
- a **reflector**, which sends the current back through the three rotors by a different path.



In the slide's example the key T is pressed: the plugboard turns it into K, the three wheels carry it to the reflector, which returns it on another contact, the wheels carry it back to W, and the plugboard turns W into G, so lamp G lights.

Consequences of the reflector. The current path is reversible, so with the same settings pressing G lights T: encryption and decryption are the same operation, which made the machine convenient. But the reflector never connects a contact to itself, so the return path always ends on a different letter: **Enigma never encrypts a letter to itself**. This was a real weakness. Allied cryptanalysts used it to place a guessed plaintext word (a *crib*) against the ciphertext, rejecting every position where some letter would have mapped to itself, and then searched the rotor settings with machines (the *bombes*). The key of a day was the choice and order of the rotors, their ring settings and starting positions, and the plugboard pairs.

17.6 Transposition: keyless, keyed, combined ★

★ **In the papers:** 2024–25 Q3(b): "abcdefghij" → "CABDEHFGIJ" with a keyed transposition: the pattern repeats every 5 letters, so the permutation key has size **5**.

A transposition cipher does not substitute one symbol for another; it changes the location of the symbols (§16). Forouzan sorts transposition ciphers by how the permutation is produced.

Keyless transposition. Simple transposition ciphers used in the past are keyless: the permutation comes from the shape of a table, not from a key. There are two methods. In the first, the text is written into a table *column by column* and transmitted *row by row*; the rail fence is this method with a 2-row table. To send "meet me at the park", Alice writes the letters in a zig-zag over two lines and sends the first line followed by the second: **MEMATEAKETETHPR**. Bob divides the ciphertext in half (here the second half, ETETHPR, has one character fewer than the first, MEMATEAK), uses the halves as the two rows and reads in zig-zag. In the second method, the text is written *row by row* and transmitted *column by column*. With an agreed 4 columns Alice writes `meet|meat|thep|ark` and sends **MMTAEEHREAEKTTP**; Bob writes the ciphertext column by column and reads it row by row.

The permutation of the second method has a visible pattern. Writing each plaintext position under the ciphertext position it moves to,

Plaintext position	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15
Ciphertext position	01	05	09	13	02	06	10	14	03	07	11	15	04	08	12

the second character has moved to the fifth position, the third to the ninth, and so on. The permutation falls into runs (01, 05, 09, 13), (02, 06, 10, 14), (03, 07, 11, 15), (04, 08, 12), in each of which adjacent numbers differ by 4, the number of rows. Such regularity is what the pattern attack below exploits.

Keyed (block) transposition. Instead of permuting the whole text at once, the plaintext is divided into groups of a predetermined size, called **blocks**, and the characters of each block are permuted separately with a **permutation key**. Alice and Bob agree to send "Enemy attacks tonight" in blocks of five, adding a bogus character z to fill the last block: `enemy attac kston ightz`. With the encryption key **(3 1 4 5 2)**, *position i of the ciphertext block takes the character at position key[i] of the plaintext block*: the third character becomes the first, the first becomes the second, the fourth the third, and so on. So `enemy` → `EEMYN`, and the ciphertext is **EEMYN TAACT TKONS HITZG**. Bob divides the ciphertext into 5-character groups and applies the key in the reverse direction. Since letters are only moved, changing one plaintext letter changes exactly one ciphertext letter.

The past-paper question. Eve types "abcdefghij" on Alice's computer and sees "CABDEHFGIJ". This is a **chosen-plaintext attack**, and the choice of plaintext is clever: all letters are different, so each output letter reveals its source position directly. `abcde` → `CABDE` and `fghij` → `HFGIJ` both take input positions 3, 1, 2, 4, 5, so the pattern repeats every 5 letters: the key has size **5**, the encryption key is **(3 1 2 4 5)**, and the decryption key is its inverse **(2 3 1 4 5)**.

Combining the two approaches (keyed columnar transposition). More recent transposition ciphers combine the two approaches to achieve better scrambling. Encryption is done in three steps: the text is **written into a table row by row**, the **columns are permuted** with the key, and the new table is **read column by column**. The first and third steps give a keyless *global* reordering, and the second a *blockwise keyed* reordering (each row is a block). Such ciphers are called **keyed columnar** or simply **columnar transposition ciphers**. With the key (3 1 4 5 2):

Written row by row

e	n	e	m	y
a	t	t	a	c
k	s	t	o	n
i	g	h	t	z

Columns permuted (3 1 4 5 2)

E	E	M	Y	N
T	A	A	C	T
T	K	O	N	S
H	I	T	Z	G

Read column by column

```
ETTH EAKI MAOT YCNZ NTSG
```

Ciphertext **ETTHEAKIMAOTYCNZNTSG**. Bob reverses the steps: he writes the ciphertext column by column, permutes the columns with the decryption key, and reads row by row.

Trap: two ways to write a column key

The Neso row–column key (§16) writes above each column its *rank in the reading order* (4 3 1 2 5 6 7 means column 3 is read first). Forouzan's key lists, for each output position, the *source column* (3 1 4 5 2 means column 3 goes first). The two notations are inverse permutations of each other: the Neso key 4 3 1 2 5 6 7 is Forouzan's (3 4 2 1 5 6 7). State which convention you use.

17.7 Encryption and decryption keys of a transposition

A single key is used in two directions for the column exchange: downward for encryption and upward for decryption. It is customary to create two keys from it, one for encryption and one for decryption, each stored as a table with one entry per column. **Each entry shows the source column number; the destination column number is understood from the position of the entry.** The encryption key (3 1 4 5 2) says that column 3 of the source becomes column 1 of the destination; the decryption key (2 5 1 3 4) says that column 2 of the source becomes column 1 of the destination.

Inverting a key. Encryption sends source column e_i to destination i. Decryption must send it back: the column now at position i must return to position e_i, i.e. the decryption key has the entry i at position e_i. Hence the algorithm on the slide:

```
for index = 1 to Column:  DecKey[EncKey[index]] ← index
```

By hand: write the index $1, 2, \ldots$ under the encryption key, swap the two rows, and sort the pairs by the (former) key row; the index row becomes the decryption key. For the slide's key (2 6 3 1 4 7 5):

Encryption key	2	6	3	1	4	7	5	Sorted by key	1	2	3	4	5	6	7
Index	1	2	3	4	5	6	7	Index = decryption key	**4**	**1**	**3**	**5**	**7**	**2**	**6**

Check: key value 1 stood at index 4, so DecKey[1] = 4; key value 2 stood at index 1, so DecKey[2] = 1. Likewise (3 1 4 5 2) inverts to (2 5 1 3 4).

Transposition as a matrix. The plaintext and ciphertext blocks can be written as $1 \times m$ row vectors of letter values and the key as an $m \times m$ **permutation matrix**, in which every row and every column has exactly one 1 and the rest are 0s. Column j of the key has its 1 in row key[j], so multiplying a block by it copies input position key[j] to output position j. Encryption is $C = PK$ and decryption $P = CK^{-1}$, as in the Hill cipher; a transposition is a Hill cipher whose matrix only moves letters. The inverse needs no computation: $K^{-1} = K^T$. Indeed $(KK^T)_{ij} = \sum_l K_{il}K_{jl}$; for $i = j$ the single 1 of row i meets itself and the sum is 1, while for $i \neq j$ the 1s of rows i and j lie in different columns (each column has only one 1) and the sum is 0. With the key (3 1 4 5 2) and the four blocks of "enemyattackstonightz" as rows:

$$\begin{pmatrix} 04 & 13 & 04 & 12 & 24 \\ 00 & 19 & 19 & 00 & 02 \\ 10 & 18 & 19 & 14 & 13 \\ 08 & 06 & 07 & 19 & 25 \end{pmatrix} \begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix} = \begin{pmatrix} 04 & 04 & 12 & 24 & 13 \\ 19 & 00 & 00 & 02 & 19 \\ 19 & 10 & 14 & 13 & 18 \\ 07 & 08 & 19 & 25 & 06 \end{pmatrix}$$

that is `EEMYN TAACT TKONS HITZG`, the keyed block transposition above; multiplying by K^T returns `enemy attac kston ightz`.

17.8 Cryptanalysis of transposition ciphers

Transposition ciphers are vulnerable to several kinds of ciphertext-only attack.

Statistical attack. Single-letter frequency analysis does not reveal the key, but since the letters are unchanged it immediately shows that the cipher is a transposition (and what language the plaintext is in); digram and trigram statistics then guide the reordering.

Brute-force attack. Eve can try all possible keys, but if the key length is unknown their number is huge, $1! + 2! + 3! + \cdots + L!$ for an L-letter ciphertext. A better approach is to guess the number of columns. If the table is full, the number of columns divides L: for $L = 20 = 2 \times 2 \times 5$ it is one of $1, 2, 4, 5, 10, 20$, where 1 column (no change) is out of the question and 20 (a single row) is unlikely. For each candidate c only $c!$ orders remain ($5! = 120$ for $c = 5$).

Pattern attack. The ciphertext of a keyed (columnar) transposition has repeated patterns. For the ciphertext `ETTHEAKIMAOTYCNZNTSG` above, the plaintext positions of the ciphertext letters are

```
03 08 13 18  01 06 11 16  04 09 14 19  05 10 15 20  02 07 12 17
```

The first ciphertext character comes from the third plaintext character, and within each group of four adjacent numbers differ by 5, the number of columns. If Eve knows or guesses that there are 5 columns, she organises the ciphertext in groups of four characters (each group is one column of the table), and permuting the groups ($5! = 120$ orders at most, far fewer with digram clues) reveals the plaintext.

Double transposition. Double transposition ciphers make the cryptanalyst's job much harder. The whole algorithm is applied twice, normally with the same key (a different key can be used in each step). Encrypting the middle text ETTHEAKIMAOTYCNZNTSG again with (3 1 4 5 2) gives the ciphertext **TIYTEAOZHMCSEANGTKTN**, whose plaintext positions are

```
13 16 05 07 03 06 14 20 18 04 10 12 01 09 15 17 08 11 19 02
```

No constant difference survives, so grouping the ciphertext by columns no longer gives pieces of plaintext rows. The single-letter frequency attack still works (the letters are still those of the plaintext), but the pattern attack is now much more difficult. As in §16, the double transposition is still one permutation of the 20 positions; what it removes is the regular columnar structure that the pattern attack depends on.

17.9 Stream vs. block ciphers

Symmetric ciphers fall into two broad categories. In a **stream cipher**, encryption and decryption are done **one symbol (a character or a bit) at a time**. There is a plaintext stream $P = P_1P_2P_3\ldots$, a ciphertext stream $C = C_1C_2C_3\ldots$ and a key stream $K = (k_1, k_2, k_3, \ldots)$, and

$$C_1 = E_{k_1}(P_1), \quad C_2 = E_{k_2}(P_2), \quad C_3 = E_{k_3}(P_3), \ \ldots$$

The ciphers differ only in how the key stream is made: the additive cipher repeats one value ($k_i = k$), the monoalphabetic cipher uses one fixed table, Vigenère repeats m values, the autokey takes its values from the plaintext, the rotor cipher uses the sequence of rotor positions, and the one-time pad uses fresh random values.

In a **block cipher**, a group of plaintext symbols of size m ($m > 1$) is encrypted **together**, creating a ciphertext group of the same size. A single key is used to encrypt the whole block, even if the key is made of several values, and **each ciphertext block depends on the whole plaintext block**. Playfair ($m = 2$: the output for a letter depends on its partner) and Hill ($m \times m$ key) are block ciphers, and so are the modern DES and AES (64- and 128-bit blocks, §29). In practice the two ideas are combined: a block cipher encrypts the blocks of a long message one after another, so at the level of single blocks it is a block cipher, but considering each block as one unit the message is processed as a stream of blocks.

The difference shows in how far one plaintext change spreads, which past papers ask directly:

Cipher	Ciphertext characters changed	Reason
Additive, multiplicative, affine	1	monoalphabetic, letter by letter
Vigenère, one-time pad	1	stream: one key value per letter
Autokey	2	P_i is also the key of a later letter
Keyed transposition	1	the letter is moved, not changed
Playfair	2 (usually)	the digram is one block
Hill with $m \times m$ key	up to m	every output letter uses the whole block

Trap: Vigenère is a stream cipher

Even though the key repeats every m letters, Vigenère encrypts one character at a time, each with its own key element: a **stream** cipher. Playfair works on pairs, Hill on m-letter blocks: **block** ciphers.

18 Classical Ciphers at a Glance

The table collects every classical cipher of the course with its key and the weakness that breaks it. The pattern is consistent: a cipher is broken through whatever structure the key fails to destroy, whether that is a tiny key space, preserved letter or digram frequencies, a period, linearity or a regular position pattern.

Cipher	Type	Key	Main weakness
Caesar	substitution, 1 letter	shift 3 (fixed)	algorithm known, no real key
Shift	substitution, 1 letter	shift 1–25	only 25 keys: brute force
Multiplicative	substitution, 1 letter	$k \in \mathbb{Z}_{26}^*$ (12)	brute force; frequencies preserved
Affine	substitution, 1 letter	(k_1, k_2): 312 keys	two known pairs give the key
Monoalphabetic	substitution, 1 letter	permutation (26!)	letter frequencies preserved
Playfair	substitution, 2 letters	5×5 keyword matrix	same pair → same output
Hill	substitution, m letters	invertible $m \times m$ matrix	linear (known plaintext)
Vigenère	polyalphabetic	repeating keyword	key repeats: Kasiski/IC, then Caesars
Autokey	polyalphabetic	keyword + plaintext	key has English frequencies
Vernam	polyalphabetic, bits (XOR)	key stream	tape key repeated
Rotor, Enigma	polyalphabetic, stream	wiring, rotor positions	finite period; Enigma never maps a letter to itself
One-time pad	substitution	random, once, message length	none (perfect secrecy); key generation and distribution
Rail fence	transposition	depth	few possibilities
Row–column	transposition	column order (+ stages)	one stage can be reconstructed
Keyed / columnar	transposition	permutation of m positions	pattern attack (double transposition resists it)

Transposition ciphers keep the letter frequencies of the plaintext unchanged (only positions move); substitution ciphers keep the positions and change the letters.

Corrections to the notes / slides

- Kasiski slide: "the cryptanalyst assumes that $d \mid m$" and "$\gcd(d_1, \dots, d_n)/m$" should read $m \mid d$ and $m \mid \gcd(d_1, \dots, d_n)$: the key length divides the distances, not the other way round.
- Kasiski example: the indices 68/168, 69/117, 72/132, 119/127 count from 0 (from 1 they are 69/169, 70/118, 73/133, 120/128); the differences 100, 48, 60, 8 are unaffected.
- Kasiski example: the slide's rows P1–P4 are not the decryptions of C1–C4 from the 21st letter of each piece onward. The ciphertext was made from a copy of the sentence with typos (decrypted with key `code`, "shift" lacks its `f`, "shifted" ends in `r`, the `t` of "to" is missing and an extra `a` precedes "ciphertext"), while P1–P4 were written from the corrected sentence. The key `code` and the method are right.
- Affine attack: "The square matrix and its inverse are the same" means they are the same as in the first attempt (both use the plaintext "et"); $\left(\begin{smallmatrix} 4 & 1 \\ 19 & 1 \end{smallmatrix}\right)$ is not its own inverse, which is $\left(\begin{smallmatrix} 19 & 7 \\ 3 & 24 \end{smallmatrix}\right)$.
- Keyless transposition slide: the mapping row shows 13 under plaintext position 8 and the run (02, 06, 10, 13); both should be 14. The runs are (01, 05, 09, 13), (02, 06, 10, 14), (03, 07, 11, 15), (04, 08, 12).
- Double transposition slide: the ciphertext is `TIYTEAOZHMCSEANG`**`T`**`KTN`, not …`EANGYKTN`; in the position list the seventh entry is **14**, not a second 10.
- "Keys" slide: "one for encryption and one for direction" should be "one for decryption". Enigma slide: the inventor is Arthur *Scherbius*.
- Stream/block slide lists "Polyalphabetic cipher" under block ciphers while listing Vigenère under stream ciphers. By the slide's own definitions, polyalphabetic ciphers that encrypt one letter at a time (Vigenère, autokey, rotor) are stream ciphers.

19 Steganography

Encryption hides the *meaning* of a message but not the fact that a secret message is being sent. Ciphertext looks like random gibberish, so it draws attention: an observer immediately knows that something is worth intercepting, may block it, or may pressure the sender and receiver for the key. **Steganography** (Greek "covered writing") solves a different problem. It **conceals the existence of the message** by hiding it inside an innocent-looking **cover** such as a text, an image or an audio file, so that an observer does not suspect that any secret communication is taking place. It is **not an encryption scheme**: the message is not transformed into something unintelligible but merely hidden, and its protection lies in going unnoticed. **Cryptography**, by contrast, renders the message unintelligible to outsiders by various transformations of the text.

Cryptography hides the meaning of a message; steganography hides its existence.

	Cryptography	**Steganography**
Hides	the **meaning**	the **existence**
Attacker sees	a scrambled message; knows something secret is being sent	an ordinary-looking text/image; doesn't suspect anything
Output	ciphertext	a normal-looking **cover** (text, image, audio) with the message inside

Steganography is the other kind of "security-related transformation" T in the model for network security (§2): the transformation applied to the message before transmission does not have to be encryption.

Examples from the lecture. The simplest text steganography arranges words so that some of their letters spell the real message, here the first letter of each word (an *acrostic*, or null cipher). The cover sentence reads as ordinary if odd text, and the secret is the rule "read the first letters":

- **S**imply **e**ncrypt **c**orrect **r**eading **e**xactly **t**wice. ⇒ **Secret**
- **T**ell **h**enry **a**lice **n**ot **k**ane / **Y**esterday **o**urplan **u**ndone! ⇒ **Thank You**

19.1 Historical techniques

Before computers, messages were hidden physically in ordinary documents. Stallings lists four classical techniques; each leaves the document readable as normal text and reveals the hidden marks only under a special treatment that an inspector would not think of applying.

Character marking	Chosen letters of a printed or typewritten text are overwritten faintly in pencil. The marks are not visible unless the paper is held at an angle to bright light.
Invisible ink	Written with a substance (e.g. lemon juice) that leaves no visible trace until heat or a chemical is applied.
Pin punctures	Tiny pin holes on selected letters. They are not visible unless the paper is held up in front of a light.
Typewriter correction ribbon	Text typed between the lines with a correction ribbon. It is visible only under a strong light.

Modern steganography hides data in digital media instead. Stallings' example is a high-resolution photograph (Kodak Photo CD, 2048×3072 pixels of 24 bits): changing only the least significant bit of each colour byte does not visibly affect the picture, yet hides $2048 \times 3072 \times 3 = 18\,874\,368$ bits, about 2.3 MB, in a single snapshot. This LSB method is explained below.

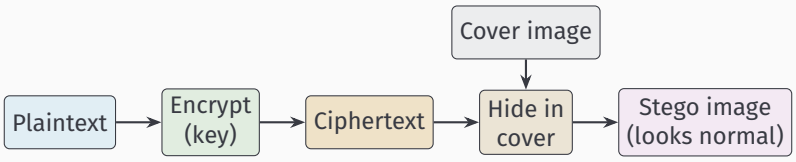
19.2 Drawbacks

Compared with encryption, steganography has two drawbacks.

1. **Lots of overhead.** A large cover is needed to hide relatively few bits of message: a whole sentence for six letters in the acrostic above, a whole image for a few kilobytes.
2. **Once the system is discovered, it becomes virtually worthless.** The security rests entirely on the method staying unknown, which contradicts Kerckhoff's principle (§6). Unlike encryption, there is no secret key left to protect the message once the method is known. This can be softened by making the insertion depend on a key (for example, a key that selects which pixels carry the bits).

Advantage. Steganography can be used by parties who have something to lose if the *fact* of their secret communication (not necessarily its content) is discovered. Encryption flags the traffic as important or secret, and may mark the sender or receiver as someone with something to hide; a stego image is just a photograph.

Best of both. A message can be **first encrypted and then hidden** using steganography. The hiding conceals that communication is taking place, and if the hiding is discovered the attacker still only has ciphertext, which is protected by the key. Encrypted data also looks random, like the noise in the least significant bits it replaces, which makes the hidden data harder to detect statistically.



19.3 LSB steganography

A byte $b_7 b_6 \ldots b_1 b_0$ has the value $128b_7 + 64b_6 + \cdots + 2b_1 + b_0$. The leftmost bits are the **most significant bits** (MSB), because changing them changes the value a lot; the rightmost bits are the **least significant bits** (LSB), which change it the least:

$$1010\,10\,\underbrace{01}_{\text{LSBs}} \qquad \text{bit weights: } 128\ 64\ 32\ 16\ 8\ 4\ \underline{2\ 1}$$

Changing the lowest k bits of a byte changes its value by at most $2^k - 1$. For $k = 2$ that is at most 3 out of 255, about 1.2% of the range, which the eye cannot see in a colour.

RGB model. In a 24-bit colour image each pixel is stored as three bytes, one each for the intensity of **Red**, **Green** and **Blue** (0–255).

$$\text{Red } (255,0,0) = (11111111, 00000000, 00000000) \quad \text{Green } (0,255,0) \quad \text{Blue } (0,0,255)$$



(0, 255, 0), G = 11111111



(0, 252, 0), G = 11111100

The two boxes look identical, but the last 2 bits of their green values differ. **Those LSBs can therefore store hidden information**: the sender overwrites them with message bits, and the image looks unchanged. Using the 2 LSBs of each of R, G and B hides $2 \times 3 =$ **6 bits per pixel**, and no channel value changes by more than 3. To extract the message the receiver simply reads the last 2 bits of each channel, in the agreed order of pixels, and concatenates them.

WORKED EXAMPLE : hiding bits in one pixel

Q. Hide the six bits `11 01 10` in the pixel (100, 50, 200) using the 2 LSBs of each channel. By how much does each value change?
Write each value in binary and replace its last two bits with the next two message bits:

Channel	Value	Binary	Replace last 2 bits	New value
R	100	01100100	01100111	103 (+3)
G	50	00110010	00110001	49 (−1)
B	200	11001000	11001010	202 (+2)

New pixel **(103, 49, 202)**; every channel changes by at most 3, as predicted. Check: the last two bits of 103, 49, 202 are `11`, `01`, `10` ✓.

Capacity. With 6 bits per pixel an 800×600 image holds $800 \cdot 600 \cdot 6 = 2{,}880{,}000$ bits = 360,000 bytes. With only the single LSB of each channel (3 bits per pixel) a 1024×768 image holds $1024 \cdot 768 \cdot 3 = 2{,}359{,}296$ bits = 294,912 bytes. The lecture's 17-byte message "Kill Corona Virus" is $17 \times 8 = 136$ bits, which needs only $\lceil 136/6 \rceil = 23$ pixels at 6 bits per pixel.

Blue orchids: the lecture shows the image without and with hidden information side by side, and they cannot be told apart.

Detection. Looking at the image reveals nothing, but a statistical analysis may. In a natural photograph the LSBs are correlated with the rest of the image, while embedded data (especially encrypted data) is uniformly random, so the LSB plane of a stego image has slightly different statistics; a changed file format or file size is a further clue (see the OpenStego demo below). The study of detecting hidden data is called **steganalysis**.

19.4 Watermarking vs. image steganography

Both techniques embed extra information in an image, but for opposite purposes. A **watermark** is embedded to **prove ownership** or copyright: the logo or owner information is not secret, it is often visible (like the `nesoacademy.org` mark on the lecture's laptop image), and it must survive cropping and compression so that it can still be shown later. **Image steganography** embeds a message for **secret communication**: its **existence** must stay hidden, and robustness matters less than going unnoticed.

	Watermarking	**Image steganography**
Purpose	prove **ownership** / copyright	**secret communication**
Is the mark secret?	no; it's often visible (like the `nesoacademy.org` logo)	yes; its **existence** is hidden
Hidden data	logo, owner info	any message or file

19.5 OpenStego demo

OpenStego is a free tool that embeds a file in an image and extracts it again. The lecture uses it in both directions:

1. **Hide Data:** message file `My Message.txt` ("Kill Corona Virus", 17 bytes) + cover file `Blue Orchids.jpg` (58.4 KB) → stego file `BO.png` (824 KB). No password. "Message embedded into 1 cover file(s)."
2. **Extract Data:** input `BO.png` → output folder. "Message file extracted successfully", and the text file comes back unchanged.

Why PNG, and why so much bigger (not explained in the video): JPEG compression is lossy, so it slightly changes pixel values when the image is saved, which would wipe out the hidden LSBs. The stego image is therefore saved in the lossless PNG format, which stores every pixel exactly and is much larger (58.4 KB → 824 KB). That size jump is itself a hint that something is going on. Leaving the password empty means the extraction needs no key, so anyone who suspects the file and has the tool can read the message; with a password OpenStego encrypts the data before hiding it, which is the encrypt-then-hide combination above.

20 Prime Numbers

Why the course turns to mathematics. Classical ciphers substitute and move letters. Modern cryptography treats plaintext and ciphertext as *numbers*, and most encryption is based heavily on **number theory** and **abstract algebra**; calculus and trigonometry make no major contribution. The concepts needed are the division and Euclidean algorithms and the extended Euclidean algorithm (§24), modular arithmetic (§21), groups, rings, fields and finite fields with polynomial arithmetic (§28), Euler's totient (§25), Fermat's and Euler's theorems (§26), and primality testing, the Chinese remainder theorem and discrete logarithms (§27). The lecture also notes that this is cryptography for classical computers; quantum computers will need different assumptions, because a quantum algorithm (Shor's) factors efficiently. Primes come first because almost everything else is built on them.

Definitions. A **prime** is an integer greater than 1 with exactly two divisors, 1 and itself: it cannot be divided by any other number without leaving a remainder. A **composite** number has more than two divisors. **1 is neither**, because it has only one divisor. Examples: 2, 3, 5, 7 are prime; $9 = 3^2$ (divisors 1, 3, 9) and $33 = 3 \cdot 11$ (divisors 1, 3, 11, 33) are composite.

Facts. 2 is the smallest prime and the **only even prime**, since every larger even number has 2 as a third divisor. Except for 2 and 5, every prime ends in the digit 1, 3, 7 or 9: a number ending in 0, 2, 4, 6 or 8 is divisible by 2, and one ending in 5 is divisible by 5. (The converse fails: 33 and 39 end in 3 and 9 but are composite.)

Every number has prime factors. By the **fundamental theorem of arithmetic**, every integer $n > 1$ can be written as a product of primes in exactly one way, apart from the order of the factors. A prime is its own factorisation; 1 is not a factor to be listed, since it is not prime.

n	factorisation	n	factorisation
10	$2 \cdot 5$	37	37 (prime)
11	11 (prime)	308	$2^2 \cdot 7 \cdot 11$
100	$2^2 \cdot 5^2$ (9 divisors)	14688	$2^5 \cdot 3^3 \cdot 17$

The number of divisors follows from the factorisation: a divisor of $2^2 \cdot 5^2$ is $2^a 5^b$ with $a, b \in \{0, 1, 2\}$, giving $3 \times 3 = 9$ divisors of 100 (1, 2, 4, 5, 10, 20, 25, 50, 100).

20.1 Why primes matter in cryptography

Many encryption algorithms, RSA first among them (§33), are based on prime numbers, because of an asymmetry in the work involved. **Multiplying** two large primes is **very fast**: two 300-digit numbers are multiplied in a fraction of a millisecond. The **reverse**, **factoring** the product back into its primes, is **extremely computer-intensive**: no known classical algorithm does it in polynomial time, and for a product of two well-chosen 1024-bit primes all the computers in the world would take far too long (§27 covers the factoring methods).

$$p, q \xrightarrow{\text{easy}} n = p \cdot q \qquad\qquad n \xrightarrow{\text{very hard}} p, q$$

A function that is easy to compute but infeasible to reverse is called a **one-way function**; if a secret (here the primes p and q) makes the reverse easy, it is a **trapdoor one-way function**, which is exactly what a public-key cipher needs. Cryptographic algorithms therefore use **large** primes, hundreds of digits long. The lecture closes with six numbers to test, from 4 to 34 digits: 5393, 27644437, 4398042316799, 1125899839733759, 18014398241046527 and 1298074214633706835075030044377087. **All six are prime** (checked with the Miller–Rabin test of §27); trial division would need about $\sqrt{n}$ steps, which is hopeless for the last one, and this is why fast probabilistic primality tests matter.

★ MUST KNOW : primes

Prime: > 1 with exactly two divisors (1 and itself); 1 is not prime; 2 is the only even prime. Every $n > 1$ factors uniquely into primes, e.g. $14688 = 2^5 \cdot 3^3 \cdot 17$.
Security of RSA-type systems rests on **factoring the product** $n = pq$ of two large primes being infeasible, while multiplying p and q is easy.

Corrections to the notes / slides

- **Slide wording:** "Factoring very large prime numbers is very hard" should say factoring the *product* of two large primes. A prime can't be factored at all.
- The factorisation table writes $11 = 1^1 \times 11^1$ and lists 1 as a prime factor. 1 is not prime; 11 and 37 are simply prime.
- The notes call watermarks "typically visible". The lecture's example is visible, but watermarks can also be invisible. The real difference is the purpose (ownership vs. secret communication).
- Fermat's and Euler's theorems need $\gcd(a, p) = 1$ / $\gcd(a, n) = 1$; the notes omitted this condition.
- Everything else checked: TOMATO → (a) (by script), $N(N-1)/2$, all six numbers prime, factorisations correct.

Part II
Mathematics of Cryptography

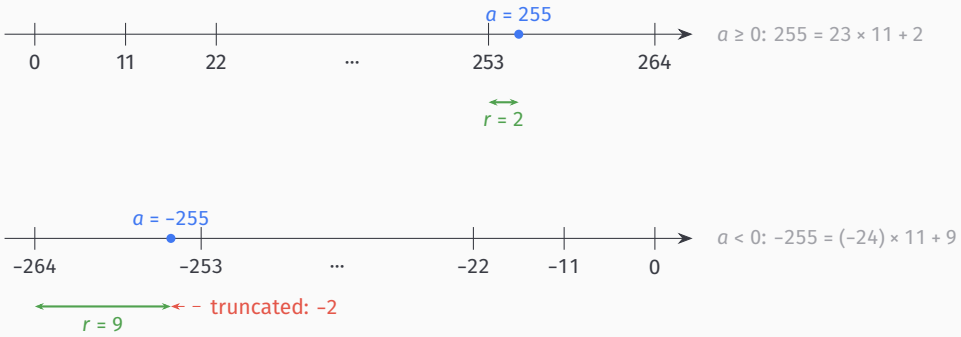
21 Modular Arithmetic

Modular arithmetic is integer arithmetic in which only the remainder after division by a fixed positive integer n is kept, so that counting wraps around to 0 after reaching $n-1$. Cryptography needs it for two reasons. First, a cipher must map a finite alphabet onto itself: shifting Y by 3 has to land on a letter again, which is achieved by wrapping around after 25. Second, public-key systems compute with numbers that are hundreds of digits long, and modular arithmetic keeps every intermediate result below the modulus however many operations are performed. This section builds the arithmetic from the division algorithm and ends with the two questions that recur in every later chapter: when can we divide modulo n, and how do we solve $ax \equiv b \pmod{n}$.

21.1 Integer division and residues

The set of integers $\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$ is closed under the three binary operations cryptography uses (addition, subtraction and multiplication) but not under division, since $7/2$ is not an integer. Integer division therefore has two outputs instead of one. For any integer a and any positive integer n there are unique integers q (the quotient) and r (the remainder) with $a = q \times n + r$ and $0 \le r < n$. This statement is the **division algorithm**. For $a = 255$ and $n = 11$ it gives $255 = 23 \times 11 + 2$, so $q = 23$ and $r = 2$.

The pair (q, r) is unique because of the condition $0 \le r < n$: if $a = qn + r = q'n + r'$, then $r - r' = (q' - q)n$ is a multiple of n lying strictly between $-n$ and n, so it is 0. Geometrically, qn is the largest multiple of n that does not exceed a, and r is the distance from that multiple up to a. The picture settles the case of a negative a as well: the largest multiple of n not exceeding a then lies to the *left* of a, further from zero, so r is still non-negative.



Negative dividends. C, Java and many calculators divide by truncating towards zero, which for a negative a returns a negative quotient and a negative remainder: $-255 = (-23) \times 11 + (-2)$. Such an r violates $0 \le r < n$. The fix on the slide is to decrement q by 1 and add n to r, which leaves the right-hand side unchanged because $(q-1)n + (r+n) = qn + r$: $-255 = (-24) \times 11 + 9$, so the remainder is 9. In practice the rule is: **if the remainder comes out negative, add the modulus**. (Python's `%` already does this: `-255 % 11` is 9.)

The modulo operator. Modular arithmetic keeps only the second output of the division algorithm. $a \bmod n$ is the remainder r of dividing a by n; n is called the modulus and r the residue. The residue always lies in the **set of least residues modulo** n, $\mathbb{Z}_n = \{0, 1, 2, \dots, n-1\}$; for example $\mathbb{Z}_2 = \{0, 1\}$, $\mathbb{Z}_6 = \{0, 1, \dots, 5\}$ and $\mathbb{Z}_{11} = \{0, 1, \dots, 10\}$. The slide examples are $27 \bmod 5 = 2$, $36 \bmod 12 = 0$, $-18 \bmod 14 = 10$ (truncated remainder -4, plus 14) and $-7 \bmod 10 = 3$ (truncated remainder -7, plus 10). A 12-hour clock is arithmetic modulo 12 that writes 12 in place of 0: 15:00 shows 3 because $15 \bmod 12 = 3$, and 23:00 shows 11.

Divisibility. When the remainder is zero, $a = q \times n$ and we say that n *divides* a, written $n \mid a$; otherwise $n \nmid a$. Thus $4 \mid 32$ because $32 = 8 \times 4$, while $8 \nmid 42$ because $42 = 5 \times 8 + 2$. The properties of divisibility and the greatest common divisor are developed in §23.

21.2 Congruence and residue classes

The mod operator sends infinitely many integers to the same residue: 2, 12, 22 and -8 all leave remainder 2 on division by 10. In cryptography it is this common remainder that matters, not the particular integer, so we need a relation that says "same remainder" without reducing anything. Two integers are congruent modulo n, written $a \equiv b \pmod{n}$, if $n \mid (a - b)$. Equivalently, $a = kn + b$ for some integer k, or $a \bmod n = b \bmod n$.

The last equivalence is what makes the definition useful, and it follows from the division algorithm. Write $a = q_1 n + r_1$ and $b = q_2 n + r_2$ with $0 \le r_1, r_2 < n$. Then $a - b = (q_1 - q_2)n + (r_1 - r_2)$, and since $|r_1 - r_2| < n$, the difference is a multiple of n exactly when $r_1 = r_2$. As a result the fastest validity test for a congruence is to subtract and check

divisibility; neither side has to be reduced.

Congruence must be read differently from both equality and the operator. $10 \equiv -2 \pmod{12}$ is true although $10 \neq -2$, and $38 \equiv 14 \pmod{12}$ is true although $38 \bmod 12 = 2$, not 14. The operator "mod" returns one specific number in $\mathbb{Z}_n$, while "$\equiv \ldots \pmod{n}$" states that two integers lie in the same class and allows any representative on either side. The eight statements from the lecture video:

Statement	$a - b$	Divisible by m?	Verdict
$38 \equiv 2 \pmod{12}$	36	yes	**valid** (and $38 \bmod 12 = 2$)
$38 \equiv 14 \pmod{12}$	24	yes	**valid** (but $38 \bmod 12 = 2$, not 14)
$5 \equiv 0 \pmod{5}$	5	yes	**valid**
$10 \equiv 2 \pmod{6}$	8	no	**invalid**
$13 \equiv 3 \pmod{13}$	10	no	**invalid**
$2 \equiv -3 \pmod{5}$	5	yes	**valid**
$-8 \equiv 7 \pmod{5}$	−15	yes	**valid**
$-3 \equiv -8 \pmod{5}$	5	yes	**valid**

Congruence is an equivalence relation. It is reflexive ($a \equiv a$, since $n \mid 0$), symmetric (if $n \mid a - b$ then $n \mid b - a$) and transitive (if $n \mid a - b$ and $n \mid b - c$, then n divides their sum $a - c$). Any equivalence relation splits its set into disjoint classes of mutually related elements. The residue class $[a]_n$ is the set of all integers congruent to a modulo n, that is, $[a]_n = \{\ldots, a - 2n, a - n, a, a + n, a + 2n, \ldots\}$. Modulo n there are exactly n classes, one for each possible remainder. For $n = 5$:

$$[0] = \{\ldots, -15, -10, -5, 0, 5, 10, 15, \ldots\} \quad [1] = \{\ldots, -14, -9, -4, 1, 6, 11, 16, \ldots\} \quad [2] = \{\ldots, -13, -8, -3, 2, 7, 12, 17, \ldots\}$$
$$[3] = \{\ldots, -12, -7, -2, 3, 8, 13, 18, \ldots\} \quad [4] = \{\ldots, -11, -6, -1, 4, 9, 14, 19, \ldots\}$$

$\mathbb{Z}_n$ is obtained by choosing the smallest non-negative member of each class, which is why its elements are called the *least residues*. Any member of a class can stand in for any other inside a congruence, so $-8 \equiv 2 \equiv 12 \equiv 22 \pmod{10}$ all describe the single element 2 of $\mathbb{Z}_{10}$. The slides draw $\mathbb{Z}$ as a line and $\mathbb{Z}_n$ as a circle of n points: moving n steps along the line brings you back to the same point of the circle.

21.3 Operations in $\mathbb{Z}_n$: why reducing first is allowed

Addition, subtraction and multiplication in $\mathbb{Z}_n$ are defined by computing in $\mathbb{Z}$ and mapping the result back with mod n. The slide examples: adding 7 to 14 in $\mathbb{Z}_{15}$ gives $21 \bmod 15 = 6$; subtracting 11 from 7 in $\mathbb{Z}_{13}$ gives $-4 \bmod 13 = 9$; multiplying 11 by 7 in $\mathbb{Z}_{20}$ gives $77 \bmod 20 = 17$. The difficulty is size. The product $1{,}723{,}345 \times 2{,}124{,}945$ already has 13 digits; in RSA the operands have over 300 digits, and a power such as C^d computed in full would have more digits than there are atoms in the universe. We must reduce the operands *before* operating, and that needs a proof that the result does not change.

Derivation. Write $a = q_1 n + r_1$ and $b = q_2 n + r_2$, where $r_1 = a \bmod n$ and $r_2 = b \bmod n$. Then

$$a \pm b = (q_1 \pm q_2)\, n + (r_1 \pm r_2), \qquad a \times b = (q_1 q_2 n + q_1 r_2 + q_2 r_1)\, n + r_1 r_2.$$

In each case the result differs from $r_1 \pm r_2$ (respectively $r_1 r_2$) by a multiple of n, so both have the same remainder. This proves the three properties on the slides:

$$(a + b) \bmod n = [(a \bmod n) + (b \bmod n)] \bmod n, \qquad (a - b) \bmod n = [(a \bmod n) - (b \bmod n)] \bmod n,$$
$$(a \times b) \bmod n = [(a \bmod n) \times (b \bmod n)] \bmod n.$$

More generally, if $a \equiv a'$ and $b \equiv b' \pmod{n}$, then $a \pm b \equiv a' \pm b'$ and $ab \equiv a'b'$: congruence is compatible with $+$, $-$ and $\times$, so any operand may be replaced by any member of its residue class, including a negative one when that is smaller. Applying the product rule $k - 1$ times gives $a^k \equiv (a \bmod n)^k \pmod{n}$, which is the starting point of modular exponentiation (§22).

The slides apply the properties with $a = 1{,}723{,}345 \equiv 8$ and $b = 2{,}124{,}945 \equiv 9 \pmod{11}$: $(a + b) \bmod 11 = (8 + 9) \bmod 11 = 6$, $(a - b) \bmod 11 = (8 - 9) \bmod 11 = 10$, and $(a \times b) \bmod 11 = (8 \times 9) \bmod 11 = 6$. Inputs taken from $\mathbb{Z}$ rather than $\mathbb{Z}_n$ are handled the same way: $(17 + 27) \bmod 14 = 2$, $(12 - 43) \bmod 13 = 8$ and $(123 \times (-10)) \bmod 19 = 5$. Two more: $(123 \times 456) \bmod 7 = (4 \times 1) \bmod 7 = 4$, and $3^{100} \bmod 10 = (3^4)^{25} \bmod 10 = 1$ because $3^4 = 81 \equiv 1$.

Powers of 10. Applying the third property k times gives $10^k \bmod x = (10 \bmod x)^k \bmod x$, so $10^k \equiv 1$ modulo 3 and modulo 9 (hence a number is congruent to its digit sum modulo 9), and $10^k \equiv 3^k \pmod{7}$.

There is no division rule. Cancelling a common factor is not allowed in general: $6 \equiv 16 \pmod{10}$, but halving both sides gives $3 \not\equiv 8 \pmod{10}$. In modular arithmetic "dividing by a" means multiplying by an inverse of a, and the next subsection shows that such an inverse exists only under a condition.

Properties of $\mathbb{Z}_n$. Addition and multiplication modulo n inherit from $\mathbb{Z}$ the commutative laws, the associative laws and the distributive law $a \times (b + c) \equiv a \times b + a \times c$, with identity 0 for addition and 1 for multiplication. Subtraction is neither commutative nor associative ($5 - 3 \neq 3 - 5$). In the language of §28, $\langle \mathbb{Z}_n, +, \times \rangle$ is a commutative ring.

21.4 Additive and multiplicative inverses ★

▸ **Likely this year:** "Find all additive / multiplicative inverse pairs in $\mathbb{Z}_n$" or "why has 8 no inverse in $\mathbb{Z}_{10}$?" (2–3 marks); an inverse is also one step of every Hill, affine and RSA numerical.

Encryption must be undone, and in modular arithmetic undoing an addition or a multiplication means applying an inverse. In $\mathbb{Z}_n$, b is the additive inverse of a if $a + b \equiv 0 \pmod{n}$, and the multiplicative inverse of a if $a \times b \equiv 1 \pmod{n}$. The multiplicative inverse is written a^{-1}.

Additive inverses always exist. The additive inverse of a in $\mathbb{Z}_n$ is $n - a$ (and 0 is its own), because $a + (n - a) = n \equiv 0$. It is unique in $\mathbb{Z}_n$, and it is what defines subtraction: $a - b \equiv a + (n - b)$. The six pairs in $\mathbb{Z}_{10}$ are $(0,0)$, $(1,9)$, $(2,8)$, $(3,7)$, $(4,6)$ and $(5,5)$. The shift cipher is the standard application: encryption adds the key k modulo 26, so decryption adds its additive inverse $26 - k$, and Caesar's key 3 is undone by adding 23.

Multiplicative inverses may not exist. In $\mathbb{Z}_{10}$ there are only three pairs, $(1,1)$, $(3,7)$ and $(9,9)$, since $3 \times 7 = 21 \equiv 1$ and $9 \times 9 = 81 \equiv 1$; the elements 0, 2, 4, 5, 6 and 8 have no inverse. The multiplication table of $\mathbb{Z}_{10}$ (from the slides) shows why. A row contains a 1 only when its element is co-prime to 10. The row of 8 contains only even numbers, because $8x - 10k$ is even for all integers x and k, so $8x$ can never be congruent to 1. The same row also repeats values ($8 \times 1 \equiv 8 \times 6 \equiv 8$), so multiplication by 8 destroys information and cannot be undone; this is precisely why the key of a multiplicative or affine cipher must be co-prime to 26.

×	0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	0	0
1	0	**1**	2	3	4	5	6	7	8	9
2	0	2	4	6	8	0	2	4	6	8
3	0	3	6	9	2	5	8	**1**	4	7
4	0	4	8	2	6	0	4	8	2	6
5	0	5	0	5	0	5	0	5	0	5
6	0	6	2	8	4	0	6	2	8	4
7	0	7	4	**1**	8	5	2	9	6	3
8	0	8	6	4	2	0	8	6	4	2
9	0	9	8	7	6	5	4	3	2	**1**

Multiplication table of $\mathbb{Z}_{10}$: only the rows of 1, 3, 7, 9 (co-prime to 10) contain a 1.

Theorem. a has a multiplicative inverse modulo n if and only if $\gcd(a, n) = 1$; the inverse is then unique in $\mathbb{Z}_n$. The proof has three short parts.

- *Only if.* Suppose $ab \equiv 1 \pmod{n}$, that is, $ab - kn = 1$ for some integer k. Any common divisor of a and n divides the left-hand side, hence divides 1, so $\gcd(a, n) = 1$.
- *If.* Suppose $\gcd(a, n) = 1$. Bézout's identity, which the extended Euclidean algorithm makes constructive (§24), gives integers s and t with $s \times n + t \times a = 1$. Reducing modulo n removes the first term, so $t \times a \equiv 1 \pmod{n}$ and $a^{-1} = t \bmod n$.
- *Uniqueness.* If b and b' are both inverses of a, then $b \equiv b(ab') = (ba)b' \equiv b'$.

Hence 8^{-1} does not exist in $\mathbb{Z}_{10}$ ($\gcd(10, 8) = 2$) and 12^{-1} does not exist modulo 26 ($\gcd(26, 12) = 2$), while $11^{-1} \bmod 26 = 19$ ($11 \times 19 = 209 = 8 \times 26 + 1$) and $23^{-1} \bmod 100 = 87$ ($23 \times 87 = 2001$); both are computed with the table in §24. For a prime modulus every non-zero element is co-prime to it and therefore invertible: in $\mathbb{Z}_{11}$ the pairs are $(1,1)$, $(2,6)$, $(3,4)$, $(5,9)$, $(7,8)$ and $(10,10)$. In $\mathbb{Z}_8$ the invertible elements are the odd ones, and each is its own inverse:

a in $\mathbb{Z}_8$	0	1	2	3	4	5	6	7
$-a$ (additive inverse)	0	7	6	5	4	3	2	1
a^{-1} (multiplicative inverse)	–	1	–	3	–	5	–	7

The same condition reappears throughout the course. Hill decryption needs $(\det K)^{-1} \bmod 26$, so a valid key matrix has $\gcd(\det K, 26) = 1$; RSA needs $d = e^{-1} \bmod \phi(n)$, so e is chosen with $\gcd(e, \phi(n)) = 1$.

★ MUST KNOW

$a \equiv b \pmod{n} \iff n \mid (a - b)$. The operands of +, –, × may be reduced at any time; there is no division, only multiplication by an inverse.

additive inverse: $n - a$ (always exists), $\quad a^{-1} \bmod n$ exists $\iff \gcd(a, n) = 1$.

21.5 The sets $\mathbb{Z}_n$, $\mathbb{Z}_n^*$, $\mathbb{Z}_p$ and $\mathbb{Z}_p^*$

Since multiplicative inverses exist only for residues co-prime to n, cryptography works with two sets. $\mathbb{Z}_n = \{0, 1, \ldots, n - 1\}$ contains every residue; addition and subtraction never leave it, and every element has an additive inverse. $\mathbb{Z}_n^* = \{a \in \mathbb{Z}_n : \gcd(a, n) = 1\}$ contains exactly the residues that have multiplicative inverses. It is

closed under multiplication, because a product of two numbers co-prime to n is again co-prime to n, so multiplication and division never leave it. The slide examples are $\mathbb{Z}_6^* = \{1,5\}$, $\mathbb{Z}_7^* = \{1,2,3,4,5,6\}$ and $\mathbb{Z}_{10}^* = \{1,3,7,9\}$, and the rule they state is: use $\mathbb{Z}_n$ when additive inverses are needed and $\mathbb{Z}_n^*$ when multiplicative inverses are needed. When the modulus is a prime p, every non-zero residue is co-prime to p, so $\mathbb{Z}_p^* = \{1,2,\dots,p-1\}$ is $\mathbb{Z}_p$ without 0; for example $\mathbb{Z}_{13} = \{0,1,\dots,12\}$ and $\mathbb{Z}_{13}^* = \{1,2,\dots,12\}$. In $\mathbb{Z}_p$ one can add, subtract, multiply and divide by any non-zero element, which makes it the field GF(p) (§28); this is why Diffie–Hellman, ElGamal and elliptic-curve cryptography use a prime modulus. The number of elements of $\mathbb{Z}_n^*$ is Euler's totient $\phi(n)$ (§25).

21.6 Linear congruences $ax \equiv b \pmod{n}$

Problem. Several cryptanalytic steps reduce to solving $ax \equiv b \pmod{n}$ for x, for example recovering an affine-cipher key from a known plaintext–ciphertext pair or solving the equations of a known-plaintext attack on the Hill cipher. Over the rationals the answer would be $x = b/a$; modulo n, division by a is possible only when a is invertible, so solvability depends on $d = \gcd(a,n)$.

Case $d = 1$. Multiplying both sides by a^{-1} gives the unique solution $x \equiv b \times a^{-1} \pmod{n}$. Forouzan's example is $3x+4 \equiv 6 \pmod{13}$. Adding -4 (the additive inverse of 4) to both sides gives $3x \equiv 2$; since $\gcd(3,13) = 1$ and $3^{-1} = 9$ ($3 \times 9 = 27 \equiv 1$), $x \equiv 2 \times 9 = 18 \equiv \mathbf{5} \pmod{13}$. Check: $3 \times 5 + 4 = 19 \equiv 6$.

General case: derivation. The congruence says $ax - b = kn$ for some integer k, that is, $b = ax - kn$. Since d divides both a and n, it divides the right-hand side, so **if $d \nmid b$ there is no solution**. If $d \mid b$, write $a = da'$, $b = db'$ and $n = dn'$. Dividing $ax - b = kn$ by d shows that $ax \equiv b \pmod{n}$ is equivalent to $a'x \equiv b' \pmod{n'}$, and now $\gcd(a', n') = 1$, because dividing two numbers by their gcd leaves co-prime numbers. The reduced congruence therefore has the unique solution $x_0 = b' \times (a')^{-1} \bmod n'$, and the solutions in $\mathbb{Z}$ are all $x \equiv x_0 \pmod{n'}$. Inside $\mathbb{Z}_n$ these are

$$x = x_0, \quad x_0 + \tfrac{n}{d}, \quad x_0 + 2\tfrac{n}{d}, \quad \dots, \quad x_0 + (d-1)\tfrac{n}{d},$$

since the next candidate, $x_0 + d \cdot \frac{n}{d} = x_0 + n$, wraps around to x_0. So **if $d \mid b$ there are exactly d solutions modulo n**. Forouzan states the procedure as three steps: divide the whole congruence, modulus included, by d; multiply by the inverse of a/d modulo n/d to get x_0; add multiples of n/d.

For $10x \equiv 2 \pmod{15}$, $d = \gcd(10,15) = 5$ and $5 \nmid 2$, so there is no solution: every value of $10x - 15k$ is a multiple of 5. For $14x \equiv 12 \pmod{18}$, $d = 2$ divides 12, so there are two solutions. The reduced congruence is $7x \equiv 6 \pmod{9}$; $7^{-1} \bmod 9 = 4$ ($28 \equiv 1$), so $x_0 = 6 \times 4 = 24 \bmod 9 = 6$, and the solutions are **6** and $6 + 9 = \mathbf{15}$. Check: $14 \times 6 = 84 = 4 \times 18 + 12$ and $14 \times 15 = 210 = 11 \times 18 + 12$.

WORKED EXAMPLE : linear congruence (Forouzan style)

Q. Find all solutions in $\mathbb{Z}_{21}$ of (a) $15x \equiv 9 \pmod{21}$ (b) $15x \equiv 10 \pmod{21}$.
A. (a) $d = \gcd(15,21)$: $21 = 1 \times 15 + 6$, $15 = 2 \times 6 + 3$, $6 = 2 \times 3 + 0$, so $d = 3$. Since $3 \mid 9$, there are exactly 3 solutions. Divide everything by 3: $5x \equiv 3 \pmod{7}$. Since $5 \times 3 = 15 = 2 \times 7 + 1$, $5^{-1} \bmod 7 = 3$, and $x_0 = 3 \times 3 = 9 \bmod 7 = 2$. Add multiples of $n/d = 7$:

x	$15x$	$15x \bmod 21$
$x_0 = 2$	$30 = 1 \times 21 + 9$	9 ✓
$2 + 7 = 9$	$135 = 6 \times 21 + 9$	9 ✓
$2 + 14 = 16$	$240 = 11 \times 21 + 9$	9 ✓

Answer: $x \in \{2, 9, 16\}$.
(b) Again $d = 3$, but $3 \nmid 10$. Every value of $15x - 21k$ is a multiple of 3 and so can never equal 10. **No solution.**

Trap: the usual slips

- Writing $38 \bmod 12 = 14$. The operator returns the least residue, 2; "$38 \equiv 14 \pmod{12}$" is a true congruence, not a value of the operator.
- Leaving a negative remainder: $-18 \bmod 14 = 10$, not -4.
- Dividing a and b by d but not the modulus. From $14x \equiv 12 \pmod{18}$ the correct reduction is $7x \equiv 6 \pmod{9}$, not $\pmod{18}$, and the answer then has $d = 2$ solutions modulo 18.
- Quoting an inverse without checking $\gcd(a,n) = 1$ first. If the gcd is not 1, write "no inverse exists" and give the gcd as the reason.

Corrections to the notes / slides

- All numbers in this section were recomputed by script (residues, the eight congruences, the $\mathbb{Z}_n$ operations, the inverse tables and the three linear congruences).
- Lecture 4, "Find all multiplicative inverse pairs in $\mathbb{Z}_{11}$": the slide says "seven pairs" but lists six, and **six is correct**: $(1,1)$, $(2,6)$, $(3,4)$, $(5,9)$, $(7,8)$, $(10,10)$. All 10 non-zero elements are invertible; 1 and 10 are their own inverses and the other 8 form 4 pairs.
- Lecture 2 says "when we use a computer or a calculator, r and q are negative when a is negative". That holds for truncating division (C, Java, most calculators); Python's `//` and `%` already give $q = -24$, $r = 9$ for -255 and 11.
- The lecture video calls 14 and 25 "remainders" in $38 \equiv 14 \pmod{12}$ and $35 \equiv 25 \pmod{10}$. These are valid congruences, but the remainders are 2 and 5. The video also states the multiplicative inverse without its condition $\gcd(a, n) = 1$; the slides give it.

22 Modular Exponentiation ★

★ **In the papers:** 2024–25 Q5(ii): $100^{95} \bmod 9$, $658^{17} \bmod 17$, $145^{102} \bmod 101$. ▸ This year every RSA/ElGamal numerical also needs it (§33).

Modular exponentiation is exponentiation performed over a modulus: $a^b \bmod m$ means $(a^b) \bmod m$, the remainder of the whole power; the modulus does not apply to the exponent. It is the core operation of public-key cryptography. RSA encrypts with $C = P^e \bmod n$ and decrypts with $P = C^d \bmod n$, Diffie–Hellman computes $g^x \bmod p$, ElGamal and DSS are built from the same operation, and elliptic-curve cryptography uses its additive analogue, double-and-add. The lecture introduces it with $2^{33} \bmod 30$ and $3^{100} \bmod 29$.

Computing the power first and reducing at the end fails for two separate reasons. The first is **size**: 3^{100} already has 48 digits, and with a 2048-bit exponent the power would have roughly 10^{616} digits, far beyond any memory. The second is **time**: multiplying by a one step at a time needs $b - 1$ multiplications, which for $b \approx 2^{2048}$ is impossible. The size problem is solved by reducing modulo m after every multiplication, which the product rule of §21 allows and which keeps every intermediate value below m^2. The time problem is solved by repeated squaring, which needs only about $2 \log_2 b$ multiplications. The lecture packages these ideas as two tools: reduce the base first, then build the power by repeated squaring, reducing after every multiplication.

22.1 Tool 1: reduce the base first

If $a \equiv r \pmod m$, then $a^b \equiv r^b \pmod m$. This follows from the product rule: a factor may be replaced by any congruent number, and a^b is a product of b factors equal to a. The base may therefore be replaced by any member of its residue class, and the best choice is the one with the smallest absolute value: the residue r itself, or $r - m$ when $r > m/2$ (for example $23 \equiv -7 \pmod{30}$). Two special cases make some questions trivial. If $a \equiv 1$, then $a^b \equiv 1$ for every b. If $a \equiv -1$, then $a^b \equiv (-1)^b$, which is 1 for even b and $-1 \equiv m - 1$ for odd b. A negative final result is turned into a residue by adding m.

Problem	Base becomes	Working	Answer
$23^3 \bmod 30$	$23 \equiv -7$	$(-7)^3 = 49 \times (-7) = -133 = -5 \times 30 + 17$	**17**
$31^{500} \bmod 30$	$31 \equiv 1$	$1^{500} = 1$, however large the exponent	**1**
$242^{329} \bmod 243$	$242 \equiv -1$	329 is odd, so $(-1)^{329} = -1 \equiv -1 + 243$	**242**
$11^7 \bmod 13$	$11 \equiv -2$	$(-2)^7 = -128 = -10 \times 13 + 2$	**2**

The first row can be checked by brute force, $23^3 = 12167 = 405 \times 30 + 17$; for large exponents no such check exists, which is why the method matters. The lecture reduces -133 in two steps ($-133 \to -13 \to -13 + 30 = 17$); formally $-133 = -5 \times 30 + 17$, and the result is the same.

The exponent may not be reduced modulo m. The exponent counts how many factors are multiplied; it is an ordinary integer, not an element of $\mathbb{Z}_m$, and the product rule says nothing about it. One counterexample settles it: $2^5 = 32 \equiv 2 \pmod 3$, but $5 \bmod 3 = 2$ and $2^2 = 4 \equiv 1 \pmod 3$. Powers do repeat, but with a different period: by Euler's theorem (§26) $a^{\phi(m)} \equiv 1$ when $\gcd(a, m) = 1$, so the exponent may be reduced modulo $\phi(m)$ (modulo $p - 1$ for a prime p), never modulo m. Practice Q41 is built on this trap.

22.2 Tool 2: repeated squaring

When the base is not close to 0 or ±1, the first tool does not help: $88 \bmod 187$ is 88 or -99, and neither is small. The second tool attacks the number of multiplications instead. Write the exponent in binary, $b = \sum_i b_i 2^i$ with each $b_i \in \{0, 1\}$. Then

$$a^b = \prod_{i\,:\,b_i=1} a^{2^i}, \qquad \text{e.g.} \quad a^7 = a^4 \cdot a^2 \cdot a^1, \qquad a^{100} = a^{64} \cdot a^{32} \cdot a^4,$$

and the powers a^{2^i} are cheap because each is the square of the previous one: $a^{2^{i+1}} = \left(a^{2^i}\right)^2$. The lecture's procedure follows directly.

1. Write b as a sum of powers of 2, which is its binary form: $7 = 4 + 2 + 1$, $100 = 64 + 32 + 4$.
2. Build the table $a^1, a^2, a^4, \ldots$ up to the largest power of 2 that does not exceed b. Each entry is the square of the previous one, reduced modulo m; use the negative representative whenever it is smaller in size.
3. Multiply the entries whose exponents add up to b, reducing modulo m after every multiplication.

For a^{100} this means 6 squarings ($a^2, a^4, \ldots, a^{64}$) and 2 multiplications, 8 operations instead of 99. The general count is derived in the next subsection.

Example 1: $88^7 \bmod 187$ **(**$7 = 4 + 2 + 1$**)**

Power	Computation	mod 187
88^1		88
88^2	$88 \times 88 = 7744 = 41 \times 187 + 77$	77
88^4	$77 \times 77 = 5929 = 31 \times 187 + 132$	132

$88^7 = 88^4 \cdot 88^2 \cdot 88^1 \equiv 132 \cdot 77 \cdot 88$. The lecture multiplies all three first ($894{,}432 = 4783 \times 187 + 11$); reducing after each product keeps the numbers small: $132 \times 77 = 10164 \equiv 66$, then $66 \times 88 = 5808 \equiv$ **11**. The lecturer asks to bookmark this result. In the RSA lecture it is the encryption of $P = 88$ with $e = 7$, $n = 187$, and the homework $11^{23} \bmod 187 = 88$ (practice Q37) is the matching decryption with $d = 23$ (§33).

Example 2: the last two digits of 29^5

The last k digits of a positive integer N are $N \bmod 10^k$, because $N = q \times 10^k + r$ where r is the number formed by those digits. So the question asks for $29^5 \bmod 100$, with $5 = 4 + 1$:

Power	Computation	mod 100	or
29^1		**29**	−71
29^2	$29 \times 29 = 841$	41	−59
29^4	$41 \times 41 = 1681$	81	**−19**

$29^5 = 29^4 \cdot 29^1 \equiv (-19)(29) = -551 \equiv -51 \equiv$ **49**. A second route confirms it: $29^2 \cdot 29^2 \cdot 29 \equiv 41 \cdot 41 \cdot 29 = 48749$, which ends in 49. In each row the representative with the smaller absolute value is used: 29 rather than −71, and −19 rather than 81.

Example 3: $3^{100} \bmod 29$ **(**$100 = 64 + 32 + 4$**, binary 1100100)**

Power	Computation	mod 29	or	Use
3^1		3	−26	3
3^2	3×3	9	−20	9
3^4	$9 \times 9 = 81$	23	−6	−6 ← needed
3^8	$(-6)^2 = 36$	7	−22	7
3^{16}	$7 \times 7 = 49$	20	−9	−9
3^{32}	$(-9)^2 = 81$	23	−6	−6 ← needed
3^{64}	$(-6)^2 = 36$	7	−22	**7** ← needed

$3^{100} = 3^{64} \cdot 3^{32} \cdot 3^4 \equiv 7 \cdot (-6) \cdot (-6) = 252 = 8 \times 29 + 20$, so the answer is **20**. The column of squares repeats ($-6 \to 7 \to -9 \to -6 \to 7$), because the powers of any element modulo m eventually cycle. *Preview of Fermat's theorem (§26):* since 29 is prime, $3^{28} \equiv 1$, so $3^{100} = 3^{3 \cdot 28 + 16} \equiv 3^{16} \equiv 20$ ✓.

Example 4: $23^{16} \bmod 30$ **(the exponent is a power of 2)**

When $b = 2^k$, the table is the whole computation: square k times, reducing each time.

$$23^{16} = \left(\left((23^2)^2\right)^2\right)^2 : \quad 23 \equiv -7 \xrightarrow{2} 49 \equiv 19 \equiv -11 \xrightarrow{2} 121 \equiv 1 \xrightarrow{2} 1 \xrightarrow{2} \mathbf{1}$$

Once a square reaches 1, every further square stays 1.

Lecture homework: (1) $11^{23} \bmod 187$; (2) the last three digits of 175^{209}. Full solutions are in the practice section (Q37, Q38).

22.3 Square-and-multiply: both scanning orders and the cost ★

▸ **Likely this year:** "Compute a^x mod n by square-and-multiply, showing every step" (2–3 marks), inside every RSA/ElGamal numerical; sometimes "write the algorithm and state its complexity".

The table method is how the computation is done by hand. Written as an algorithm it is called **square-and-multiply** (binary exponentiation), and it can scan the bits of the exponent in either direction. Both orders perform the same number of operations and give the same result; Forouzan prints the right-to-left version and Stallings the left-to-right one, so both are worth knowing. Below, $b = (b_{k-1} \dots b_1 b_0)_2$ has k bits.

Right-to-left (Forouzan, Alg. 9.7)

```
SqMul_RL(a, b, n):
  y ← 1
  for i ← 0 to k-1:
    if b_i = 1: y ← (y × a) mod n
    a ← (a × a) mod n
    (not needed after the last bit)
  return y
```

Left-to-right (Stallings)

```
SqMul_LR(a, b, n):
  f ← 1
  for i ← k-1 downto 0:
    f ← (f × f) mod n
    if b_i = 1: f ← (f × a) mod n
  return f
```

Right-to-left. The bits are scanned from the least significant bit b_0 upwards, with two variables: y, the result so far (initially 1), and a running square, which holds a^{2^i} at step i. If $b_i = 1$, the running square is multiplied into y; then it is squared for the next step. This is the lecture's table computed on the fly: the running square walks down the column $a, a^2, a^4, \dots$, and y collects the rows that are needed. After step i the invariant is $y = a^{(b \bmod 2^{i+1})}$, so after the last bit $y = a^b$. For 88^7 mod 187 ($7 = 111_2$):

i	b_i	Multiplication (init. $y = 1$)	Squaring (init. $a = 88$)
0	1	$y = 1 \times 88 = 88$	$a = 88^2 = 7744 \equiv 77$
1	1	$y = 88 \times 77 = 6776 = 36 \times 187 + 44 \to 44$	$a = 77^2 = 5929 \equiv 132$
2	1	$y = 44 \times 132 = 5808 = 31 \times 187 + 11 \to$ **11**	(not needed)

The same answer as Example 1. Forouzan's own example, 17^{22} mod 21 = 4, is traced in §27.

Left-to-right. The bits are scanned from the most significant bit down, with a single variable f: at every bit f is squared, and if the bit is 1 it is also multiplied by a. To see why this works, let c be the number formed by the bits processed so far (Stallings tracks it as an extra column). Initially $c = 0$ and $f = a^0 = 1$. Processing a bit appends it to c, which doubles c and adds the bit: $c \leftarrow 2c + b_i$. Squaring f doubles its exponent and multiplying by a adds 1 to it, so the invariant $f \equiv a^c \pmod n$ holds after every step, and after the last bit $c = b$. For 3^{100} mod 29 ($100 = 1100100_2$):

i	b_i	c	Square	Multiply by 3 (if $b_i = 1$)
6	1	1	$1^2 = 1$	$1 \times 3 = 3$
5	1	3	$3^2 = 9$	$9 \times 3 = 27$
4	0	6	$27^2 = 729 = 25 \times 29 + 4 \to 4$	–
3	0	12	$4^2 = 16$	–
2	1	25	$16^2 = 256 = 8 \times 29 + 24 \to 24$	$24 \times 3 = 72 \to 14$
1	0	50	$14^2 = 196 = 6 \times 29 + 22 \to 22$	–
0	0	100	$22^2 = 484 = 16 \times 29 + 20 \to$ **20**	–

The result, 20, agrees with Example 3. The left-to-right order always multiplies by the same fixed base a, which is cheap when a is small (primality tests often use $a = 2$) and needs only one variable. The right-to-left order computes the squarings independently of the bits, so they can be precomputed or run in parallel with the multiplications, and it matches the hand table.

Cost: why it is $O(\log b)$. Let the exponent have $k = \lfloor \log_2 b \rfloor + 1$ bits. Both orders perform one squaring per bit, of which one is free (squaring $f = 1$ at the start, or the unused last squaring), so $k - 1$ squarings in all. They perform one multiplication per 1-bit, at most k, and one of these is again free (a multiplication by 1). The total is therefore at most $2(k-1) = 2\lfloor \log_2 b \rfloor$ modular multiplications, and about $1.5 \log_2 b$ on average, since about half the bits of a random exponent are 1. For 3^{100} (7 bits, three 1-bits) this is 6 squarings and 2 multiplications, 8 in all, against 99 for repeated multiplication. Each modular multiplication works on numbers below n and needs one reduction, so with schoolbook arithmetic it costs $O((\log n)^2)$ bit operations, and the whole exponentiation costs $O(\log b \cdot (\log n)^2)$, which is polynomial in the length of the input. Repeated multiplication needs $b - 1 \approx 2^k$ steps, exponential in the length k of the exponent. For a 2048-bit exponent the difference is about 4000 multiplications against 2^{2048}.

This asymmetry is what public-key cryptography is built on: computing a^x mod n is fast, while undoing it (the e-th root modulo n in RSA, the discrete logarithm in Diffie–Hellman) has no comparably fast method (§27). Since

the cost depends on the 1-bits, the common RSA public exponent $e = 65537 = 2^{16}+1 = 10000000000000001_2$ needs only 16 squarings and one multiplication. The same dependence enables an attack: a multiplication happens only at a 1-bit, so the time or power consumed by a decryption leaks the bits of the private exponent d unless the implementation runs in constant time (§33).

★ MUST KNOW

$a^b \bmod m$: reduce the **base** modulo m (to the smallest representative, possibly negative); **never reduce the exponent modulo** m (with $\gcd(a, m) = 1$ it may be reduced modulo $\phi(m)$, §26). Square-and-multiply: square at every bit, multiply at every 1-bit, reduce after every step; at most $2\lfloor \log_2 b \rfloor$ multiplications, i.e. $O(\log b)$.

WORKED EXAMPLE : 2024–25 Q5(ii), by base reduction and repeated squaring

Q. Find (a) $100^{95} \bmod 9$ (b) $658^{17} \bmod 17$ (c) $145^{102} \bmod 101$.
(a) $100 = 11 \times 9 + 1$, so $100 \equiv 1 \pmod 9$ and $100^{95} \equiv 1^{95} = \mathbf{1}$.
(b) $658 = 38 \times 17 + 12$ and $12 \equiv -5 \pmod{17}$. The exponent is $17 = 16 + 1 = 10001_2$.

Power	Computation	mod 17
12^1		$12 \equiv -5$
12^2	$(-5)^2 = 25 = 17 + 8$	8
12^4	$8^2 = 64 = 3 \times 17 + 13$	$13 \equiv -4$
12^8	$(-4)^2 = 16$	$16 \equiv -1$
12^{16}	$(-1)^2 = 1$	1

$658^{17} \equiv 12^{16} \cdot 12^1 \equiv 1 \times 12 = \mathbf{12}$.
(c) $145 = 101 + 44$, so the base is 44. The exponent is $102 = 64 + 32 + 4 + 2 = 1100110_2$.

Power	Computation	mod 101
44^1		44
44^2	$44^2 = 1936 = 19 \times 101 + 17$	**17** ←
44^4	$17^2 = 289 = 2 \times 101 + 87$	$87 \equiv$ **−14** ←
44^8	$(-14)^2 = 196 = 101 + 95$	$95 \equiv -6$
44^{16}	$(-6)^2 = 36$	36
44^{32}	$36^2 = 1296 = 12 \times 101 + 84$	$84 \equiv$ **−17** ←
44^{64}	$(-17)^2 = 289 \equiv 87$	$87 \equiv$ **−14** ←

Multiply the marked rows, reducing each time: $44^{64} \cdot 44^{32} \equiv (-14)(-17) = 238 = 2 \times 101 + 36 \to 36$; $36 \cdot 44^4 \equiv 36 \times (-14) = -504 = -5 \times 101 + 1 \to 1$; $1 \cdot 44^2 \equiv \mathbf{17}$.
Answers: 1, 12, 17. Check with Fermat's theorem (§26): 17 and 101 are prime, so $12^{17} \equiv 12 \pmod{17}$, and $44^{100} \equiv 1 \pmod{101}$ gives $44^{102} \equiv 44^2 \equiv 17$ ✓.

WORKED EXAMPLE : square-and-multiply, left-to-right (Stallings' example)

Q. Using the square-and-multiply algorithm, compute $7^{560} \bmod 561$. Show c and f after every step and count the multiplications.
A. $560 = 512 + 32 + 16 = 1000110000_2$ (10 bits). Start with $c = 0$, $f = 1$; at each bit square f, and multiply by 7 when $b_i = 1$.

i	b_i	c	Square ($f^2 \bmod 561$)	×7 (if $b_i = 1$)
9	1	1	1	$f = 7$
8	0	2	49	–
7	0	4	$2401 = 4 \times 561 + 157 \to 157$	–
6	0	8	$24649 = 43 \times 561 + 526 \to 526$	–
5	1	17	$276676 = 493 \times 561 + 103 \to 103$	$721 = 561 + 160 \to 160$
4	1	35	$25600 = 45 \times 561 + 355 \to 355$	$2485 = 4 \times 561 + 241 \to 241$
3	0	70	$58081 = 103 \times 561 + 298 \to 298$	–
2	0	140	$88804 = 158 \times 561 + 166 \to 166$	–
1	0	280	$27556 = 49 \times 561 + 67 \to 67$	–
0	0	560	$4489 = 8 \times 561 + 1 \to \mathbf{1}$	–

Answer: $7^{560} \bmod 561 = \mathbf{1}$, using 9 non-trivial squarings and 2 non-trivial multiplications (11 instead of 559). The final $c = 560$ confirms the bits were read correctly. The result is notable: $561 = 3 \times 11 \times 17$ is composite, yet $7^{560} \equiv 1$ as if 561 were prime, because 561 is a Carmichael number, which fools the Fermat primality test (§27).

Trap: the usual slips

- Reducing the exponent modulo m, as in "117 mod 19 = 3, so $5^{117} \equiv 5^3 \pmod{19}$". Only the base is reduced modulo m.
- Reading the bits in the wrong direction. Right-to-left starts at the least significant bit; left-to-right starts at the most significant bit, and the c column must end at b.
- Multiplying all the needed powers before reducing. It is not wrong, but the numbers grow quickly; reduce after every product.
- Writing -7^3 for $(-7)^3$. With an even exponent the brackets change the sign: $(-7)^2 = 49$ but $-7^2 = -49$. Give the final answer as a residue in $\{0, \ldots, m-1\}$.

Corrections to the notes / slides

- All of Gemini's numbers are correct (checked by script: 17, 1, 242, 2, 11, 49, 20, 1). The two traces, Stallings' table for 7^{560} mod 561 and the past-paper values 1, 12, 17 were also recomputed.
- Slide glitches noted by Gemini: in Example 2 the summary line briefly reads "88^7 mod 187 = 49" (copy–paste error; it should be 29^5 mod 100 = 49). Example 3's slide stops at "252 mod 29" (the answer, 20, is only spoken).
- −133 mod 30: the lecture writes the intermediate step as −13 (truncated division), then adds 30. Formally $-133 = -5 \times 30 + 17$. Same answer.
- The slide writes -7^3 for $(-7)^3$. Strictly $-7^3 = -(7^3)$; the brackets matter when the exponent is even (e.g. $(-7)^2 = 49$ but $-7^2 = -49$).

23 GCD and the Euclidean Algorithm

The previous section showed that a is invertible modulo n exactly when $\gcd(a, n) = 1$, and that $ax \equiv b \pmod{n}$ is solvable exactly when $\gcd(a, n)$ divides b. Cryptography therefore keeps asking for greatest common divisors: is the Hill key's determinant co-prime to 26, is the RSA exponent e co-prime to $\phi(n)$, does a congruence have a solution. The school method, factorising both numbers, is useless here, because factoring large numbers is exactly the problem RSA relies on being infeasible. Euclid's algorithm (about 300 BC) computes the gcd using only divisions with remainder, and its number of steps grows with the number of digits rather than with the size of the numbers. Its extension in the next section also produces the inverse.

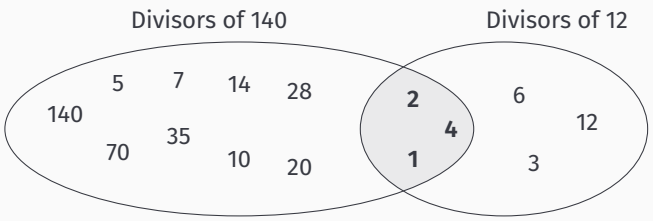
23.1 Divisibility and common divisors

If $a = q \times n$ for some integer q, that is, if dividing a by n leaves remainder 0, then n **divides** a, written $n \mid a$, and n is a *divisor* (factor) of a. For example $4 \mid 32$, while $8 \nmid 42$ because $42 = 5 \times 8 + 2$. Lecture 2 lists four properties, and each follows from the definition in one line.

- **Property 1:** if $a \mid 1$, then $a = \pm 1$, since no other integer divides 1.
- **Property 2:** if $a \mid b$ and $b \mid a$, then $a = \pm b$. Write $b = ka$ and $a = lb$; then $a = lka$, so $lk = 1$ and $k = l = \pm 1$ (for $a \neq 0$).
- **Property 3 (transitivity):** if $a \mid b$ and $b \mid c$, then $a \mid c$, because $b = ka$ and $c = lb$ give $c = (lk)a$. For example $3 \mid 15$ and $15 \mid 45$, so $3 \mid 45$.
- **Property 4 (linear combinations):** if $a \mid b$ and $a \mid c$, then $a \mid (mb + nc)$ for all integers m, n, because $b = ka$ and $c = la$ give $mb + nc = (mk + nl)a$. For example $3 \mid 15$ and $3 \mid 9$, so $3 \mid (2 \times 15 + 4 \times 9) = 66$.

Property 4 is the engine of Euclid's algorithm, and it is also behind every "no solution" argument in this chapter: a common divisor of a and n divides every combination $ax + ny$.

Every positive integer has 1 and itself as divisors. A prime has exactly these two, and a composite number has more. The greatest common divisor $\gcd(a, b)$ of two positive integers is the largest integer that divides both; it is also called the highest common factor (HCF). For small numbers it can be found by listing divisors. The slide's example is 140 and 12, whose common divisors are 1, 2 and 4, so $\gcd(140, 12) = 4$. The lecture video does the same for $\gcd(12, 33) = 3$ (common divisors $\{1, 3\}$), $\gcd(25, 150) = 25$ and $\gcd(13, 31) = 1$. Listing all divisors amounts to factoring, so the method cannot scale to cryptographic sizes.



common divisors $\{1, 2, 4\}$: $\gcd(140, 12) = 4$

23.2 Euclid's algorithm and why it works

The algorithm rests on two facts stated on the slides.

$$\textbf{Fact 1: } \gcd(a, 0) = a, \qquad \textbf{Fact 2: } \gcd(a, b) = \gcd(b, r), \text{ where } r = a \bmod b.$$

Why Fact 1 holds. Every integer divides 0 (since $0 = 0 \times d$), so the common divisors of a and 0 are just the divisors of a, and the largest of these is a itself.

Why Fact 2 holds. Divide: $a = qb + r$. If d divides both a and b, then by Property 4 it divides $a - qb = r$. Conversely, if d divides both b and r, then it divides $qb + r = a$. The pairs (a, b) and (b, r) therefore have exactly the same set of common divisors, and so the same greatest common divisor. The argument works for any multiple qb; the remainder is used because it makes the new pair as small as possible.

The algorithm. Apply Fact 2 repeatedly, replacing (a, b) by $(b, a \bmod b)$, until the second number is 0; then Fact 1 gives the answer. The slide's example:

$$\gcd(36, 10) = \gcd(10, 6) = \gcd(6, 4) = \gcd(4, 2) = \gcd(2, 0) = \mathbf{2}.$$

Termination. Each step replaces the second number b by $r = a \bmod b$, and $0 \le r < b$. The second numbers therefore form a strictly decreasing sequence of non-negative integers, $b > r_1 > r_2 > \cdots$, which must reach 0 after at most b steps; the last non-zero term is the gcd.

Speed. The bound of b steps is far too pessimistic: the remainder at least halves every two steps. Consider three consecutive remainders, where $r_{i+2} = r_i \bmod r_{i+1}$. If $r_{i+1} \le r_i/2$, then $r_{i+2} < r_{i+1} \le r_i/2$. Otherwise $r_{i+1} > r_i/2$, the quotient is 1, and $r_{i+2} = r_i - r_{i+1} < r_i/2$. Either way two steps halve the remainder, so at most about $2 \log_2 b$ divisions are needed, which is linear in the number of digits. A 1024-bit b therefore needs at most about 2048 divisions, while listing divisors would need about 2^{1024} trial divisions. The slowest inputs are consecutive Fibonacci numbers, where every quotient is 1: $\gcd(89, 55)$ takes 9 divisions.

Forouzan writes the algorithm with two variables r_1 and r_2 that hold the current pair:

```
r1 ← a;  r2 ← b;                          (initialisation)
while (r2 > 0)
{
   q ← r1 / r2;                           (integer quotient)
   r ← r1 - q × r2;
   r1 ← r2;  r2 ← r;
}
gcd(a, b) ← r1
```

★ MUST KNOW

$\gcd(a, b) = \gcd(b,\ a \bmod b)$ and $\gcd(a, 0) = a$. Divide, shift ($B \to A$, $R \to B$), and stop when the second number is 0: the gcd is the last non-zero remainder. At most about $2 \log_2 b$ divisions.

23.3 Method 1: the table (Q, A, B, R)

The lecture video lays the divisions out as a table, which is the format to use in an exam.

1. Draw the columns Q (quotient), A, B and R (remainder), and start with A = the larger number and B = the smaller.
2. In each row compute $Q = \lfloor A/B \rfloor$ and $R = A \bmod B$, so that $A = Q \times B + R$.
3. In the next row B moves into A and R moves into B (the diagonal arrows below). This is Fact 2.
4. Stop when $B = 0$, since division by 0 is impossible; write X for Q and R. The gcd is the A of that row (Fact 1).

Q	A	B	R	
2	33	12	9	$33 = 2 \cdot 12 + 9$
1	12	9	3	$12 = 1 \cdot 9 + 3$
3	9	3	0	$9 = 3 \cdot 3 + 0$ (don't stop yet!)
X	3	0	X	$B = 0$: stop. $\gcd(12, 33) = \mathbf{3}$

The other lecture examples, and Forouzan's slide example with the columns named q, r_1, r_2, r (the same table under different names):

gcd(750, 900)

Q	A	B	R
1	900	750	150
5	750	150	0
X	**150**	0	X

gcd(252, 105)

Q	A	B	R
2	252	105	42
2	105	42	21
2	42	21	0
X	**21**	0	X

gcd(2740, 1760) (Lecture 2)

q	r_1	r_2	r
1	2740	1760	980
1	1760	980	780
1	980	780	200
3	780	200	180
1	200	180	20
9	180	20	0
	20	0	

Shortcut. The gcd is the **last non-zero remainder** (9, 3, **3**, then 0; or 42, **21**, then 0). The extra row with $B = 0$ is only where the lecture reads it off. The Lecture 2 class exercise, gcd(25, 60), runs $60 = 2 \times 25 + 10$, $25 = 2 \times 10 + 5$, $10 = 2 \times 5 + 0$, giving **5**.

23.4 Method 2: recursion

Facts 1 and 2 are already a recursive definition, and the second lecture video writes them as a function:

```
Prerequisite: a > b
Euclid_GCD(a, b):
  if b = 0 then
    return a;                          (Fact 1)
  else
    return Euclid_GCD(b, a mod b);  (Fact 2)
```

gcd(50, 12)
= gcd(12, 50 mod 12) = gcd(12, 2)
= gcd(2, 12 mod 2) = gcd(2, 0)
= **2**

gcd(83, 19)
= gcd(19, 7) (83 = 4 · 19 + 7)
= gcd(7, 5) (19 = 2 · 7 + 5)
= gcd(5, 2) (7 = 1 · 5 + 2)
= gcd(2, 1) (5 = 2 · 2 + 1)
= gcd(1, 0) = **1** (2 = 2 · 1 + 0)

Both methods perform exactly the same divisions, so either may be used. The prerequisite $a > b$ is a convenience, not a necessity. If the smaller number comes first, the first call swaps them at the cost of one wasted step: gcd(12, 33) → gcd(33, 12 mod 33) = gcd(33, 12), since 12 mod 33 = 12. In the exam it is cleaner to write the larger number first.

23.5 Relatively prime numbers

Two integers a and b are relatively prime (co-prime) if $\gcd(a, b) = 1$, as for 83 and 19 above. The condition is about the pair: neither number has to be prime, as gcd(8, 9) = 1 shows. Cryptography needs the notion for three reasons. First, $a^{-1} \bmod n$ exists only if $\gcd(a, n) = 1$ (§21); this is the Hill key condition $\gcd(\det K, 26) = 1$ and the affine-cipher condition on the multiplier. Second, RSA key generation chooses e with $\gcd(e, \phi(n)) = 1$, so that $d = e^{-1} \bmod \phi(n)$ exists. Third, the number of residues co-prime to n is Euler's totient $\phi(n)$ (§25). Euclid's algorithm decides co-primality; the extended algorithm (§24) runs the same divisions while tracking two extra columns and produces the inverse itself, instead of the trial-and-error table used for $23^{-1} \bmod 26$ in the Hill cipher.

WORKED EXAMPLE : GCD by the Euclidean algorithm (Forouzan exercise)

Q. Find (a) gcd(88, 220) (b) gcd(300, 42) (c) gcd(24, 320) (d) gcd(401, 700). Which pairs are relatively prime?
A. Larger number first; each line is one row $A = Q \times B + R$.

Pair	Divisions	gcd
(a) 220, 88	$220 = 2 \times 88 + 44$, $88 = 2 \times 44 + 0$	**44**
(b) 300, 42	$300 = 7 \times 42 + 6$, $42 = 7 \times 6 + 0$	**6**
(c) 320, 24	$320 = 13 \times 24 + 8$, $24 = 3 \times 8 + 0$	**8**

(d) needs a longer table:

Q	A	B	R	
1	700	401	299	700 = 1 × 401 + 299
1	401	299	102	401 = 1 × 299 + 102
2	299	102	95	299 = 2 × 102 + 95
1	102	95	7	102 = 1 × 95 + 7
13	95	7	4	95 = 13 × 7 + 4
1	7	4	3	7 = 1 × 4 + 3
1	4	3	1	4 = 1 × 3 + 1
3	3	1	0	3 = 3 × 1 + 0
X	**1**	0	X	

Answers: 44, 6, 8, 1. Only 401 and 700 are relatively prime, so 401 has an inverse modulo 700 and the other pairs have none. **Check:** $88 = 2 \times 44$ and $220 = 5 \times 44$ with $\gcd(2, 5) = 1$; similarly $300 = 50 \times 6$, $42 = 7 \times 6$ and $320 = 40 \times 8$, $24 = 3 \times 8$, with co-prime cofactors.

Lecture homework: $\gcd(1005, 105)$ (table method) and $\gcd(529, 123)$ (recursive method). Solutions are in practice Q43.

Trap: reading the table

- Stop when $B = 0$, not when $R = 0$. In the row where R first becomes 0, the answer is that row's B, which moves into A in the next row; reading A from the row with $R = 0$ gives a wrong answer (9 instead of 3 for $\gcd(33, 12)$).
- Swapping the meaning of the two outputs: "A mod B" gives the remainder R, while "A divided by B" gives the quotient Q. The table needs both.
- Show every division as $A = Q \times B + R$ in the exam; the marks are for the steps, not only for the final number.

Corrections to the notes / slides

- All results are correct (checked by script): 3, 25, 1, 3, 150, 21, 2, 1 from the videos, and $\gcd(36, 10) = 2$, $\gcd(2740, 1760) = 20$, $\gcd(25, 60) = 5$, $\gcd(140, 12) = 4$ from the Lecture 2 slides.
- Verbal slips only: "divisors of 13" means 33 (video 1); "a = 9" means 19 (video 2). The slides are right.
- Wording: "A mod B" gives the *remainder*; "A divided by B" gives the quotient Q. The table needs both.

24 Extended Euclidean Algorithm ★★

★ **In the papers:** 2024–25 Q7 (5 marks): pseudocode + 23^{-1} in $\mathbb{Z}_{100}$ with "value in iteration k" sub-questions. 2025–26 Q5 (3 marks): solve $19x + 13y = 20$. The same table, with polynomials, gives the GF(2^8) inverse asked in both years (§28), and d in RSA (§33).

Euclid's algorithm decides whether $\gcd(a, n) = 1$, and hence whether $a^{-1} \bmod n$ exists, but it does not produce the inverse. The extended Euclidean algorithm does. Given two integers a and b, the extended Euclidean algorithm computes $\gcd(a, b)$ and, at the same time, integers s and t such that $s \times a + t \times b = \gcd(a, b)$. That such s and t always exist is **Bézout's identity**; the algorithm proves it by constructing them. One table then answers four kinds of exam question: the inverse $b^{-1} \bmod n$, the solutions of a linear Diophantine equation $ax + by = c$, the solutions of a linear congruence (§21), and the RSA private key $d = e^{-1} \bmod \phi(n)$. With polynomials instead of integers the same table gives inverses in GF(2^8).

24.1 The idea: carry every remainder as a combination of a and b

By hand, s and t can be found by back-substitution. For $a = 19$ and $b = 13$, Euclid's divisions are $19 = 1 \times 13 + 6$ and $13 = 2 \times 6 + 1$. Solving them from the last line upwards gives

$$1 = 13 - 2 \times 6 = 13 - 2 \times (19 - 1 \times 13) = 3 \times 13 - 2 \times 19,$$

so $s = -2$ and $t = 3$. This works, but it requires storing every quotient, running the divisions a second time in reverse, and keeping track of signs, which is where most errors happen. The extended algorithm does the same bookkeeping forwards, in a single pass.

The key observation is that every remainder in Euclid's sequence is an integer combination of a and b. The first two numbers of the sequence are a and b themselves, and trivially

$$r_1 = a = 1 \cdot a + 0 \cdot b, \qquad r_2 = b = 0 \cdot a + 1 \cdot b,$$

which gives the starting values $s_1 = 1$, $s_2 = 0$, $t_1 = 0$, $t_2 = 1$. Suppose that in some row the **invariant** $r_1 = s_1 a + t_1 b$ and $r_2 = s_2 a + t_2 b$ holds. The next remainder is $r = r_1 - q r_2$, and substituting the two combinations gives

$$r = (s_1 a + t_1 b) - q\,(s_2 a + t_2 b) = (s_1 - q s_2)\, a + (t_1 - q t_2)\, b.$$

So if s and t are updated by exactly the same rule as r, namely $s = s_1 - qs_2$ and $t = t_1 - qt_2$, the new remainder satisfies $r = sa + tb$. After the shift $(r_1, r_2) \leftarrow (r_2, r)$, $(s_1, s_2) \leftarrow (s_2, s)$, $(t_1, t_2) \leftarrow (t_2, t)$ the invariant holds again, so by induction it holds in every row. The r columns are Euclid's algorithm unchanged, so the loop ends with $r_1 = \gcd(a, b)$ and $r_2 = 0$, and the invariant for r_1 then reads $\gcd(a, b) = s_1 a + t_1 b$. This proves Bézout's identity for all a and b.

24.2 The algorithm and the table (Forouzan)

The slides give the algorithm as three identical update blocks, one for each pair of variables:

```
r1 ← a;  r2 ← b;
s1 ← 1;  s2 ← 0;                                        (initialisation)
t1 ← 0;  t2 ← 1;
while (r2 > 0)
{
   q ← r1 / r2;
   r ← r1 - q × r2;  r1 ← r2;  r2 ← r;                  (updating r's)
   s ← s1 - q × s2;  s1 ← s2;  s2 ← s;                  (updating s's)
   t ← t1 - q × t2;  t1 ← t2;  t2 ← t;                  (updating t's)
}
gcd(a, b) ← r1;  s ← s1;  t ← t1
```

On paper each pass of the loop is one row of a table with the columns $q \mid r_1\ r_2\ r \mid s_1\ s_2\ s \mid t_1\ t_2\ t$. In every row the three blocks do the same thing: the new value is (first) − $q\times$ (second), and in the next row the second value moves into the first column and the new value into the second. A final row, with no quotient, shows the values after the loop, and the answer is read from its *first* columns. The slide example, $a = 161$, $b = 28$:

q	r_1	r_2	r	s_1	s_2	s	t_1	t_2	t
5	161	28	21	1	0	1	0	1	−5
1	28	21	7	0	1	−1	1	−5	6
3	21	7	0	1	−1	4	−5	6	−23
	7	0		**−1**	4		**6**	−23	

$\gcd(161, 28) = 7$, $s = -1$, $t = 6$: $(-1)(161) + 6(28) = -161 + 168 = 7$ ✓. The invariant can be checked in every row: $21 = 1 \times 161 - 5 \times 28$, $7 = -1 \times 161 + 6 \times 28$ and $0 = 4 \times 161 - 23 \times 28$. The last of these gives a free self-check: in the final row $s_2 a + t_2 b = 0$, and (s_2, t_2) always equals $\pm(b/d, -a/d)$ with $d = \gcd(a, b)$; here $(4, -23) = (28/7, -161/7)$. In an inverse table (next subsection) this means the final t_2 is $\pm n$, so any other value there reveals an arithmetic slip.

The other slide examples test the edge cases. For $a = 17$, $b = 0$ the loop never runs, because $r_2 = 0$ from the start, and the initial values give $\gcd(17, 0) = 17$, $s = 1$, $t = 0$. For $a = 0$, $b = 45$ one iteration with $q = 0$ swaps the pair, giving $\gcd(0, 45) = 45$, $s = 0$, $t = 1$. The slide exercise $a = 84$, $b = 320$ also starts with the smaller number, and the first row ($q = 0$) simply swaps it:

q	r_1	r_2	r	s_1	s_2	s	t_1	t_2	t
0	84	320	84	1	0	1	0	1	0
3	320	84	68	0	1	−3	1	0	1
1	84	68	16	1	−3	4	0	1	−1
4	68	16	4	−3	4	−19	1	−1	5
4	16	4	0	4	−19	80	−1	5	−21
	4	0		**−19**	80		**5**	−21	

$\gcd(84, 320) = 4$, $s = -19$, $t = 5$: $-19 \times 84 + 5 \times 320 = -1596 + 1600 = 4$ ✓, as on the slide. The final $(s_2, t_2) = (80, -21) = (320/4, -84/4)$ confirms it.

24.3 Multiplicative inverse with the table ★★

▸ **Likely this year:** "Find b^{-1} in $\mathbb{Z}_n$ using the extended Euclidean algorithm; give the number of iterations / r or t in iteration k" (5 marks), and $d = e^{-1} \bmod \phi(n)$ in every RSA question.

To find $b^{-1} \bmod n$, run the algorithm with $a = n$, that is, $r_1 = n$ and $r_2 = b$. At the end $s \times n + t \times b = \gcd(n, b)$. If the gcd is 1, reduce this equation modulo n; the slides do it in three lines:

$$(s \times n + b \times t) \bmod n = 1 \;\Rightarrow\; [(s \times n) \bmod n] + [(b \times t) \bmod n] = 1 \;\Rightarrow\; (b \times t) \bmod n = 1.$$

So the multiplicative inverse of b is the final t_1, mapped into $\mathbb{Z}_n$. The s columns are never used and are dropped, leaving $q \mid r_1\ r_2\ r \mid t_1\ t_2\ t$. If the final r_1 is not 1, then $\gcd(n, b) > 1$ and no inverse exists (§21); the algorithm detects this by itself. A negative t_1 is normal and is mapped into $\mathbb{Z}_n$ by adding n. The slide's version of the algorithm, which is the pseudocode asked in 2024–25 Q7:

```
r1 ← n;  r2 ← b;  t1 ← 0;  t2 ← 1;
while (r2 > 0)
{
   q ← r1 / r2;
   r ← r1 - q × r2;  r1 ← r2;  r2 ← r;
   t ← t1 - q × t2;  t1 ← t2;  t2 ← t;
}
if (r1 = 1) then b^-1 ← t1 mod n
else ``no inverse''
```

The three slide examples:

$11^{-1} \bmod 26$

q	r_1	r_2	r	t_1	t_2	t
2	26	11	4	0	1	−2
2	11	4	3	1	−2	5
1	4	3	1	−2	5	−7
3	3	1	0	5	−7	26
	1	0		**−7**	26	

$23^{-1} \bmod 100$

q	r_1	r_2	r	t_1	t_2	t
4	100	23	8	0	1	−4
2	23	8	7	1	−4	9
1	8	7	1	−4	9	−13
7	7	1	0	9	−13	100
	1	0		**−13**	100	

$12^{-1} \bmod 26$

q	r_1	r_2	r	t_1	t_2	t
2	26	12	2	0	1	−2
6	12	2	0	1	−2	13
	2	0		−2	13	

$11^{-1} \equiv -7 \equiv \mathbf{19} \pmod{26}$ (check: $11 \times 19 = 209 = 8 \times 26 + 1$); $23^{-1} \equiv -13 \equiv \mathbf{87} \pmod{100}$ (check: $23 \times 87 = 2001$); $12^{-1} \bmod 26$: the final $r_1 = 2$, so $\gcd(26, 12) = 2$ and **no inverse exists**. In the third table $t_1 = -2$ still satisfies $12 \times (-2) \equiv 2 \pmod{26}$, the gcd rather than 1, which is exactly what the invariant predicts.

WORKED EXAMPLE : 2024–25 Q7 (5 marks), inverse with "iteration k" sub-questions

Q. Write the pseudocode for finding the multiplicative inverse of b in $\mathbb{Z}_n$, and use it to find 23^{-1} in $\mathbb{Z}_{100}$. State (a) the number of iterations, (b) the remainder in iteration 2, (c) the value of t in iteration 4, (d) the iteration whose t becomes the inverse.
A. Pseudocode: as above ($r_1 = n$, $r_2 = b$, $t_1 = 0$, $t_2 = 1$; loop while $r_2 > 0$; inverse = t_1 mod n if $r_1 = 1$). An *iteration* is one pass of the while loop, i.e. one row that has a quotient; the row shows the values at the start of the pass and the r, t computed in it.

Iteration	q	r_1	r_2	r	t_1	t_2	t
1	4	100	23	8	0	1	−4
2	2	23	8	**7**	1	−4	9
3	1	8	7	1	−4	9	**−13**
4	7	7	1	0	9	−13	**100**
(end)		**1**	0		**−13**	100	

Since $r_1 = 1$, $\gcd(100, 23) = 1$ and the inverse exists. (a) **4 iterations**. (b) The remainder in iteration 2 is **7** ($23 = 2 \times 8 + 7$). (c) t in iteration 4 is **100**, which is $\pm n$ as the self-check predicts. (d) Iteration 3 computes $t = -13$, which becomes the final t_1; mapped into $\mathbb{Z}_{100}$: $23^{-1} = -13 + 100 = \mathbf{87}$. **Check:** $23 \times 87 = 2001 = 20 \times 100 + 1$ ✓.

WORKED EXAMPLE : inverse in $\mathbb{Z}_{180}$ (Forouzan exercise)

Q. Using the extended Euclidean algorithm, find the multiplicative inverses of 7, 132 and 24 in $\mathbb{Z}_{180}$, where they exist.
A. For $b = 7$:

Iteration	q	r_1	r_2	r	t_1	t_2	t
1	25	180	7	5	0	1	−25
2	1	7	5	2	1	−25	26
3	2	5	2	1	−25	26	−77
4	2	2	1	0	26	−77	180
(end)		**1**	0		**−77**	180	

$r_1 = 1$, so the inverse exists: $7^{-1} = -77 + 180 = \mathbf{103}$. Check: $7 \times 103 = 721 = 4 \times 180 + 1$ ✓. For $b = 132$ the r columns run $180 = 1 \times 132 + 48$, $132 = 2 \times 48 + 36$, $48 = 1 \times 36 + 12$, $36 = 3 \times 12 + 0$, ending with $r_1 = 12 \neq 1$: **no inverse**. For $b = 24$: $180 = 7 \times 24 + 12$, $24 = 2 \times 12 + 0$, again $r_1 = 12$: **no inverse**. Both share the factor 12 with 180.

24.4 Linear Diophantine equations $ax + by = c$ ★

▸ **Likely this year:** "Find the particular and general solutions of $ax + by = c$" (3 marks), as in 2025–26 Q5; occasionally a word problem with non-negative solutions.

A linear Diophantine equation in two variables is $ax + by = c$, where a, b, c are integers and only integer solutions x, y are wanted. Over the reals such an equation has a whole line of solutions; restricting to integers leaves either none or infinitely many, and the extended Euclidean algorithm decides which and finds them all. The three facts below are derived from Bézout's identity, with $d = \gcd(a, b)$.

No solution unless $d \mid c$. For every pair of integers x, y, d divides $ax + by$ (Property 4 of §23). If $d \nmid c$, no choice of x, y can make $ax + by$ equal to c. For example $15x + 12y = 13$ has no solution, because $\gcd(15, 12) = 3$ and $3 \nmid 13$.

A particular solution when $d \mid c$. Divide the equation by d: $a'x + b'y = c'$ with $a' = a/d$, $b' = b/d$, $c' = c/d$ and $\gcd(a', b') = 1$. The extended algorithm on (a', b') gives s and t with $a's + b't = 1$, and multiplying by c' gives $a'(c's) + b'(c't) = c'$. So $x_0 = (c/d)s$, $y_0 = (c/d)t$ is a solution. (Running the algorithm on (a, b) directly gives the same s, t, since $sa + tb = d$ is the same equation multiplied by d.)

The general solution. Let (x, y) be any other solution. Subtracting $a'x_0 + b'y_0 = c'$ from $a'x + b'y = c'$ gives $a'(x - x_0) = b'(y_0 - y)$. So b' divides $a'(x - x_0)$, and since b' is co-prime to a', it divides $x - x_0$. (This step, *Euclid's lemma*, follows from Bézout as well: multiplying $a's + b't = 1$ by $m = x - x_0$ gives $m = s\,a'm + t\,b'm$, and b' divides both terms.) Hence $x = x_0 + kb'$ for some integer k, and substituting back gives $y = y_0 - ka'$. Conversely every k gives a solution, because $a'(x_0 + kb') + b'(y_0 - ka') = a'x_0 + b'y_0$. These are the slide's formulas:

$$\text{particular: } x_0 = \frac{c}{d}s, \quad y_0 = \frac{c}{d}t, \qquad\qquad \text{general: } x = x_0 + k\frac{b}{d}, \quad y = y_0 - k\frac{a}{d}, \quad k \in \mathbb{Z}.$$

The solutions are evenly spaced: x steps by b/d while y steps by a/d in the opposite direction, which keeps $ax + by$ constant. The linear congruence $ax \equiv b \pmod{n}$ of §21 is the same problem in disguise, since it means $ax + ny = b$ for some integer y; the condition $d \mid b$ and the spacing n/d of its d solutions are this result read modulo n.

★ MUST KNOW

$s \cdot a + t \cdot b = \gcd(a, b)$: update r, s, t by the same rule (new = first $-q\times$ second), stop when $r_2 = 0$, read r_1, s_1, t_1. Inverse: run with $r_1 = n$, $r_2 = b$; if the final $r_1 = 1$, then $b^{-1} = t_1 \bmod n$, otherwise no inverse. $ax + by = c$: solvable iff $d = \gcd(a, b) \mid c$; $x = \frac{c}{d}s + k\frac{b}{d}$, $y = \frac{c}{d}t - k\frac{a}{d}$.

The slide examples. For $21x + 14y = 35$: $d = \gcd(21, 14) = 7$ and $7 \mid 35$, so there are infinitely many solutions. Dividing by 7 gives $3x + 2y = 5$, and the algorithm on $(3, 2)$ gives $s = 1$, $t = -1$. The particular solution is $x_0 = 5 \times 1 = 5$, $y_0 = 5 \times (-1) = -5$, and the general solution is $x = 5 + 2k$, $y = -5 - 3k$: $(5, -5), (7, -8), (9, -11), \ldots$, each of which satisfies $21x + 14y = 35$. For cashing a Rs. 100 cheque in Rs. 20 and Rs. 5 notes, $20x + 5y = 100$: $d = 5$ divides 100, the reduced equation is $4x + y = 20$, and $4s + t = 1$ gives $s = 0$, $t = 1$. The particular solution is $x_0 = 0 \times 20 = 0$, $y_0 = 1 \times 20 = 20$, and the general one is $x = k$, $y = 20 - 4k$. Only non-negative note counts make sense, so $0 \le k \le 5$: $(0, 20), (1, 16), (2, 12), (3, 8), (4, 4), (5, 0)$.

WORKED EXAMPLE : 2025–26 Q5 (3 marks), solve $19x + 13y = 20$

Q. Find the particular and general solutions of $19x + 13y = 20$.
A. Run the extended Euclidean algorithm on $a = 19$, $b = 13$:

q	r_1	r_2	r	s_1	s_2	s	t_1	t_2	t
1	19	13	6	1	0	1	0	1	−1
2	13	6	1	0	1	−2	1	−1	3
6	6	1	0	1	−2	13	−1	3	−19
	1	0		**−2**	13		**3**	−19	

$d = \gcd(19, 13) = 1$, $s = -2$, $t = 3$ (check: $-2 \times 19 + 3 \times 13 = -38 + 39 = 1$). Since $1 \mid 20$, solutions exist.
Particular: $x_0 = (c/d)s = 20 \times (-2) = \mathbf{-40}$, $y_0 = (c/d)t = 20 \times 3 = \mathbf{60}$. Check: $19(-40) + 13(60) = -760 + 780 = 20$ ✓.
General: $x = -40 + 13k$, $y = 60 - 19k$, $k \in \mathbb{Z}$.

k	0	1	2	3	4	5
(x, y)	$(-40, 60)$	$(-27, 41)$	$(-14, 22)$	$(\mathbf{-1, 3})$	$(12, -16)$	$(25, -35)$

The smallest solution is $(-1, 3)$ ($k = 3$), so the general solution can also be written $x = -1 + 13k$, $y = 3 - 19k$. Check: $19(-1) + 13(3) = -19 + 39 = 20$ ✓.

WORKED EXAMPLE : non-negative solutions (Forouzan-style word problem)

Q. A post office sells only 39-cent and 15-cent stamps. Find the numbers of stamps a customer can buy to pay exactly \$2.70 of postage.
A. Let x and y be the numbers of 39-cent and 15-cent stamps: $39x + 15y = 270$, with $x, y \ge 0$. $d = \gcd(39, 15) = 3$

$(39 = 2 \times 15 + 9,\ 15 = 1 \times 9 + 6,\ 9 = 1 \times 6 + 3,\ 6 = 2 \times 3)$, and $3 \mid 270$. Dividing by 3: $13x + 5y = 90$.

q	r_1	r_2	r	s_1	s_2	s	t_1	t_2	t
2	13	5	3	1	0	1	0	1	−2
1	5	3	2	0	1	−1	1	−2	3
1	3	2	1	1	−1	2	−2	3	−5
2	2	1	0	−1	2	−5	3	−5	13
	1	0		**2**	−5		**−5**	13	

$s = 2$, $t = -5$ $(2 \times 13 - 5 \times 5 = 1)$. Particular: $x_0 = 90 \times 2 = 180$, $y_0 = 90 \times (-5) = -450$. General: $x = 180 + 5k$, $y = -450 - 13k$. Non-negative counts need $180 + 5k \geq 0$, i.e. $k \geq -36$, and $-450 - 13k \geq 0$, i.e. $k \leq -34.6$, so $k \in \{-36, -35\}$. **Answer:** $(x, y) = (0, 18)$ **or** $(5, 5)$: eighteen 15-cent stamps, or five of each. Check: $15 \times 18 = 270$ and $39 \times 5 + 15 \times 5 = 195 + 75 = 270$ ✓.

Trap: where marks are lost

- For an inverse, put the **modulus first** ($r_1 = n$). Starting with $r_1 = b$ makes the inverse appear in the s column, which the t-only table has dropped.
- Read the answer from the final row's *first* columns (r_1, s_1, t_1), not from t_2 (which is $\pm n$) or from the last computed t.
- A negative t_1 is not an error: add n (in RSA, add $\phi(n)$, never n). Check $b \times b^{-1} \bmod n = 1$ before writing the answer.
- If the final $r_1 \neq 1$, write "no inverse, since gcd = r_1"; do not report t_1.
- Show the full table with headed columns; the marks are awarded per row, and the "iteration k" sub-questions can only be answered from it.

Corrections to the notes / slides

- All slide tables are correct (recomputed by script): $(161, 28) \to 7, -1, 6$; $(17, 0) \to 17, 1, 0$; $(0, 45) \to 45, 0, 1$; $(84, 320) \to 4, -19, 5$; $11^{-1} \bmod 26 = 19$; $23^{-1} \bmod 100 = 87$; $12^{-1} \bmod 26$ does not exist; $21x + 14y = 35$ and the cheque solutions.
- Lecture 4, Diophantine slide: "if $d \nmid c$, the equation has no solution" has the "not" stroke hand-drawn on the slide. A text copy of the older slides lost it and reads "if $d \mid c$, the equation has no solution", which is wrong; the condition for *no* solution is $d \nmid c$.

25 Relatively Prime Numbers, Euler's Totient and $\mathbb{Z}_n^*$

25.1 Relatively prime (co-prime) numbers

Two positive integers a and b are relatively prime (co-prime) if $\gcd(a, b) = 1$, that is, if their only common positive divisor is 1. Neither number has to be prime: the condition is about the pair, not about each number. Since $12 = 2^2 \cdot 3$ and $77 = 7 \cdot 11$ share no prime factor, they are co-prime although both are composite, while $15 = 3 \cdot 5$ and $21 = 3 \cdot 7$ are not, because 3 divides both. If p is prime, every integer from 1 to $p - 1$ is co-prime to p, since the only divisors of p are 1 and p itself.

Cryptography cares about this property for one reason: a **has a multiplicative inverse modulo** n **exactly when** $\gcd(a, n) = 1$. If $ax \equiv 1 \pmod{n}$, then $ax - kn = 1$ for some integer k, so any common divisor of a and n divides 1. Conversely, if $\gcd(a, n) = 1$, the extended Euclidean algorithm (§24) produces s, t with $sa + tn = 1$, and reducing modulo n gives $sa \equiv 1$. Co-primality is therefore always checked with the Euclidean algorithm, never by eye.

a, b	Divisors / Euclid steps	GCD	Co-prime?	Note
4, 13	$\{1, 2, 4\}$, $\{1, 13\}$	1	yes	composite + prime
15, 21	$\{1, 3, 5, 15\}$, $\{1, 3, 7, 21\}$	3	**no**	share the factor 3
11, 17		1	yes	both prime
11, 21		1	yes	prime + composite $(21 = 3 \cdot 7)$
12, 77		1	yes	**both composite**
83, 19	$83 = 4 \cdot 19 + 7$, $19 = 2 \cdot 7 + 5$, $7 = 5 + 2$, $5 = 2 \cdot 2 + 1$	1	yes	
790, 121	$790 = 6 \cdot 121 + 64$, $121 = 64 + 57$, $64 = 57 + 7$, $57 = 8 \cdot 7 + 1$	1	yes	lecture homework

25.2 Euler's totient function $\phi(n)$ ★

▸ **Likely this year:** $\phi(n)$ for a given n (Forouzan: "find $\phi(29)$, $\phi(32)$, $\phi(80)$, $\phi(100)$, $\phi(101)$"), and $\phi(n) = (p - 1)(q - 1)$ inside every RSA question.

Many results in public-key cryptography (Euler's theorem, RSA key generation, inverses by exponentiation) need to know *how many* residues modulo n are invertible. Since invertibility is the same as being co-prime to n, this

count gets its own name. Euler's totient (phi) function $\phi(n)$ is the number of integers in $\{1, 2, \ldots, n-1\}$ that are relatively prime to n. Zero and negative numbers are not counted. The lecture videos write it as $\Phi(n)$; Forouzan and the slides write $\phi(n)$.

By hand, $\phi(n)$ is found by computing $\gcd(k, n)$ for every k from 1 to $n-1$ and counting the 1s. For $n = 8$:

k	1	2	3	4	5	6	7	
$\gcd(k, 8)$	1	2	1	4	1	2	1	
co-prime?	✓	×	✓	×	✓	×	✓	$\Rightarrow \phi(8) = \mathbf{4}$

In the same way $\phi(5) = 4$ and $\phi(11) = 10$, since every number below a prime is co-prime to it. This direct count needs $n-1$ GCDs, which is useless for $n = 1000$ and hopeless for a 1024-bit RSA modulus, so in practice $\phi(n)$ is always computed from the prime factorisation of n using the rules below.

25.3 Computing $\phi(n)$: the four rules ★

The slides (following Forouzan) give four rules. Each one follows from counting which numbers *fail* to be co-prime to n, and the derivations are short enough to reproduce in an exam.

Rule 1: $\phi(1) = 1$ **by convention** (the slides write $\phi(1) = 0$; see the corrections box). It never matters in practice.

Rule 2: for a prime p**,** $\phi(p) = p - 1$**.** The only divisors of p are 1 and p, so none of $1, \ldots, p-1$ shares a factor with p. For example $\phi(7) = 6$, $\phi(13) = 12$, $\phi(31) = 30$.

Rule 3: for a prime power, $\phi(p^k) = p^k - p^{k-1}$**.** A number shares a factor with p^k exactly when it is a multiple of p, because p is the only prime in p^k. Among $1, \ldots, p^k$ the multiples of p are $p, 2p, \ldots, p^{k-1} \cdot p$, which is p^{k-1} numbers, and removing them leaves $p^k - p^{k-1} = p^{k-1}(p-1)$. For example $\phi(8) = 8 - 4 = 4$, $\phi(25) = 25 - 5 = 20$, and $\phi(49) = 49 - 7 = 42$.

Rule 4: if $\gcd(m, n) = 1$**, then** $\phi(mn) = \phi(m)\,\phi(n)$ (the function is *multiplicative*). By the Chinese remainder theorem (§27), each x modulo mn corresponds to exactly one pair (x mod m, x mod n), and x is co-prime to mn exactly when it is co-prime to both m and n. The co-prime x therefore correspond to pairs chosen from $\phi(m)$ values and $\phi(n)$ values, giving $\phi(m)\phi(n)$ of them. The condition $\gcd(m, n) = 1$ is essential: $\phi(4) = 2$ but $\phi(2)\phi(2) = 1$.

Two distinct primes: $\phi(pq) = (p-1)(q-1)$**.** This is the RSA case, and it is worth deriving directly. Among $1, \ldots, pq$, the numbers sharing a factor with pq are the multiples of p (there are q of them: $p, 2p, \ldots, qp$) and the multiples of q (there are p of them). The number pq itself is in both lists and has been removed twice, so

$$\phi(pq) = pq - q - p + 1 = (p-1)(q-1).$$

For example $\phi(21) = 2 \cdot 6 = 12$, $\phi(77) = 6 \cdot 10 = 60$, and $\phi(187) = \phi(11 \cdot 17) = 10 \cdot 16 = 160$.

General formula. Combining rules 3 and 4, if $n = p_1^{e_1} p_2^{e_2} \cdots p_k^{e_k}$ with distinct primes p_i, then

$$\phi(n) = \left(p_1^{e_1} - p_1^{e_1-1}\right)\left(p_2^{e_2} - p_2^{e_2-1}\right)\cdots\left(p_k^{e_k} - p_k^{e_k-1}\right) = n\left(1 - \tfrac{1}{p_1}\right)\left(1 - \tfrac{1}{p_2}\right)\cdots\left(1 - \tfrac{1}{p_k}\right).$$

The second form works for every n and contains rules 2 and 3 and the pq formula as special cases. Only the *distinct* primes appear in the product; the exponents are already accounted for by the factor n. As a consequence, computing $\phi(n)$ is exactly as hard as factoring n, which is the reason RSA can publish n but must keep $\phi(n)$ secret.

★ MUST KNOW

$$\phi(p) = p - 1, \qquad \phi(p^k) = p^k - p^{k-1}, \qquad \phi(pq) = (p-1)(q-1)\ (p \neq q), \qquad \phi(n) = n \prod_{p \mid n}\left(1 - \tfrac{1}{p}\right).$$

$\phi(mn) = \phi(m)\phi(n)$ **only when** $\gcd(m, n) = 1$. Always factorise n completely first.

Example: $\phi(35)$, with $35 = 5 \times 7$: $(5-1)(7-1) = 4 \times 6 = \mathbf{24}$. The grid below checks it by crossing out the multiples of 5 and of 7 among $1, \ldots, 34$; none is a multiple of both, since the first common multiple is 35.

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34

red = multiple of 5 (6 numbers), blue = multiple of 7 (4 numbers); $34 - 10 = 24$ white cells

Trap: repeated prime factors

$25 = 5 \times 5$ is **not** a product of two distinct primes, so $(5-1)(5-1) = 16$ is wrong. Use rule 3: $\phi(25) = 25 - 5 = \mathbf{20}$. (Check: among $1, \ldots, 24$ only 5, 10, 15, 20 share a factor with 25, leaving $24 - 4 = 20$.) The slides make the same point with $\phi(49)$: writing $\phi(49) = \phi(7)\phi(7) = 36$ is wrong because 7 and 7 are not co-prime; the correct value is $49 - 7 = 42$.

WORKED EXAMPLE : $\phi(n)$ from the factorisation (Forouzan / slide style)

Q. Find (a) $\phi(29)$ (b) $\phi(32)$ (c) $\phi(80)$ (d) $\phi(100)$ (e) $\phi(101)$ (f) $\phi(240)$ (g) $\phi(1000)$ (h) $\phi(7000)$.
A. Factorise first, then apply the rules.

n	Factorisation	Working	$\phi(n)$
29	prime	$29-1$	**28**
32	2^5	$2^5-2^4=32-16$	**16**
80	$2^4\cdot 5$	$(2^4-2^3)(5-1)=8\cdot 4$	**32**
100	$2^2\cdot 5^2$	$(4-2)(25-5)=2\cdot 20$	**40**
101	prime	$101-1$	**100**
240	$2^4\cdot 3\cdot 5$	$(16-8)(3-1)(5-1)=8\cdot 2\cdot 4$	**64**
1000	$2^3\cdot 5^3$	$1000\cdot\frac{1}{2}\cdot\frac{4}{5}$ (or $\phi(8)\phi(125)=4\cdot 100$)	**400**
7000	$2^3\cdot 5^3\cdot 7$	$7000\cdot\frac{1}{2}\cdot\frac{4}{5}\cdot\frac{6}{7}=7000\cdot\frac{24}{70}$	**2400**

Check: a direct count of the co-prime $k<n$ by script gives the same eight values. Lecture homework: $\phi(369)=\phi(3^2)\phi(41)=6\cdot 40=$ **240** and $\phi(372)=\phi(2^2)\phi(3)\phi(31)=2\cdot 2\cdot 30=$ **120**.

$\phi(n)$ **is even for every** $n>2$**.** If $\gcd(k,n)=1$ then also $\gcd(n-k,n)=1$, so the co-prime numbers below n come in pairs $\{k,n-k\}$. The two members of a pair can only coincide if $n=2k$, but then $\gcd(k,n)=k$, which equals 1 only for $n=2$. Hence for $n>2$ the co-prime numbers split into disjoint pairs and their count is even; in $\mathbb{Z}_{14}^*$ the pairs are $\{1,13\}$, $\{3,11\}$, $\{5,9\}$.

25.4 The sets $\mathbb{Z}_n$ and $\mathbb{Z}_n^*$

$\mathbb{Z}_n=\{0,1,\ldots,n-1\}$ is the set of residues modulo n. Every element has an additive inverse in it ($a+(n-a)\equiv 0$), but not every element has a multiplicative inverse. Since cryptosystems such as the affine cipher, Hill cipher and RSA need to *divide*, they work in the smaller set

$$\mathbb{Z}_n^*=\{\,a\in\mathbb{Z}_n:\gcd(a,n)=1\,\},$$

which is exactly the set of residues that have multiplicative inverses. By definition its size is $|\mathbb{Z}_n^*|=\phi(n)$. The slide's rule of thumb: use $\mathbb{Z}_n$ when additive inverses are needed and $\mathbb{Z}_n^*$ when multiplicative inverses are needed. $\mathbb{Z}_n^*$ is closed under multiplication modulo n, because a product of two numbers co-prime to n is again co-prime to n (no prime factor of n can appear in it). Together with the inverses this makes $\langle\mathbb{Z}_n^*,\times\rangle$ a group of order $\phi(n)$ (§28), and the theorems of the next section are statements about this group. The slides single out the two sets with a *prime* modulus, $\mathbb{Z}_p$ and $\mathbb{Z}_p^*$. When $n=p$ is prime, $\mathbb{Z}_p^*=\{1,2,\ldots,p-1\}$ contains every non-zero residue (e.g. $\mathbb{Z}_{13}^*=\{1,\ldots,12\}$), so every non-zero element of $\mathbb{Z}_p$ can be divided by, which is what makes $\mathbb{Z}_p$ the field GF(p).

n	$\mathbb{Z}_n^*$	$\lvert\mathbb{Z}_n^*\rvert=\phi(n)$
6	$\{1,5\}$	$\phi(2)\phi(3)=2$
7	$\{1,2,3,4,5,6\}$	$7-1=6$
10	$\{1,3,7,9\}$	$\phi(2)\phi(5)=4$
14	$\{1,3,5,9,11,13\}$	$\phi(2)\phi(7)=1\cdot 6=6$ (slide Example 5)
21	$\{1,2,4,5,8,10,11,13,16,17,19,20\}$	$\phi(3)\phi(7)=2\cdot 6=12$
26	the 12 odd numbers except 13	$\phi(2)\phi(13)=12$ (valid keys/determinants mod 26)

25.5 Why cryptography cares about $\phi(n)$

Three later topics rest on $\phi(n)$. First, Euler's theorem (§26) states that $a^{\phi(n)}\equiv 1 \pmod n$ for every $a\in\mathbb{Z}_n^*$, which allows exponents to be reduced modulo $\phi(n)$ and inverses to be computed as $a^{\phi(n)-1}$. Second, RSA chooses $n=pq$ and computes $\phi(n)=(p-1)(q-1)$; the keys satisfy $ed\equiv 1 \pmod{\phi(n)}$, and Euler's theorem then makes decryption undo encryption. In the standard example $n=187=11\cdot 17$, $\phi(187)=160$ and $7\cdot 23=161\equiv 1 \pmod{160}$, which is why $88^7 \bmod 187=11$ and $11^{23}\bmod 187=88$ undo each other. Third, the number of primitive roots of a prime p is $\phi(p-1)$, which matters for Diffie–Hellman and ElGamal.

Security. Knowing $\phi(n)$ for an RSA modulus is equivalent to knowing its factors. From $n=pq$ and $\phi(n)=n-(p+q)+1$ one gets $p+q=n-\phi(n)+1$, and p,q are then the roots of $z^2-(p+q)z+n=0$. For $n=143$ and $\phi(n)=120$: $p+q=24$, $pq=143$, so $p,q=$ **11, 13**. This is why $\phi(n)$ is part of the private key.

Corrections to the notes / slides

- All values recomputed by script: $\phi(5) = 4$, $\phi(8) = 4$, $\phi(11) = 10$, $\phi(35) = 24$, $\phi(25) = 20$, $\phi(49) = 42$, $\phi(240) = 64$, $\phi(1000) = 400$, $\phi(7000) = 2400$, $|\mathbb{Z}_{14}^*| = 6$, $|\mathbb{Z}_{21}^*| = 12$. The homework answers in the old notes ($\phi(369) = 240$, $\phi(372) = 120$) are correct.
- Slide L10-06 gives $\phi(1) = 0$, which follows Forouzan's "integers smaller than n" wording. The standard convention is $\phi(1) = 1$ (count $1 \le k \le n$). It only affects $n = 1$ and never matters in RSA.
- The lecture video's "case 3" ("either or both numbers composite") makes the product formula $n \prod(1 - 1/p)$ look like a last resort. It actually works for every n, and the prime and pq formulas are special cases of it.

26 Fermat's and Euler's Theorems, Inverses and Primitive Roots

Public-key cryptography computes powers $a^b \bmod n$ with exponents hundreds of digits long, and it needs inverses modulo n. Square-and-multiply (§22) makes such powers feasible, but the two theorems of this section do more: they say that the powers of a modulo n repeat with a known period, so a huge exponent can be replaced by a small one, and they give a formula for the inverse. The same periodicity, studied element by element, leads to the order of an element and to primitive roots, which Diffie–Hellman and ElGamal are built on.

26.1 Fermat's little theorem and exponent reduction ★

★ **In the papers:** 2024–25 Q5(ii) (part of 3 marks): $100^{95} \bmod 9$, $658^{17} \bmod 17$, $145^{102} \bmod 101$.

First version. If p is prime and a is an integer not divisible by p, then $a^{p-1} \equiv 1 \pmod p$. Both conditions must be checked before the theorem is used: for a composite modulus or for $p \mid a$ it says nothing.

Why it is true. Multiply each of $1, 2, \dots, p-1$ by a and reduce modulo p. No product is 0, because p divides neither factor, and no two products are equal, because $ai \equiv aj$ would mean $p \mid a(i-j)$ and hence $p \mid (i-j)$, i.e. $i = j$. The $p-1$ products are therefore just $1, \dots, p-1$ in a different order; for $p = 7$, $a = 3$ they are $3, 6, 2, 5, 1, 4$. Multiplying all of them together gives $a^{p-1}(p-1)! \equiv (p-1)! \pmod p$, and since $(p-1)!$ is co-prime to p it can be cancelled, leaving $a^{p-1} \equiv 1$.

Second version. If p is prime, then $a^p \equiv a \pmod p$ for every integer a. This removes the condition on a. If $p \nmid a$, it is the first version multiplied by a; if $p \mid a$, both sides are $\equiv 0$.

p, a	Conditions	Working	Result
5, 2	prime, $5 \nmid 2$	$2^4 = 16 = 3 \cdot 5 + 1$	$\equiv 1$ ✓
13, 11	prime, $13 \nmid 11$	$11 \equiv -2$, $(-2)^{12} = \left((-2)^4\right)^3 = 16^3 \equiv 3^3 = 27 \equiv 1$	$\equiv 1$ ✓
6, 2	**6 not prime**	$2^5 = 32 = 5 \cdot 6 + 2$	$\equiv 2 \neq 1$: theorem silent
11, 5 (homework)	prime, $11 \nmid 5$	$5^2 \equiv 3$, $5^4 \equiv 9$, $5^8 \equiv 81 \equiv 4$, $5^{10} = 5^8 \cdot 5^2 \equiv 4 \cdot 3 = 12$	$\equiv 1$ ✓

Exponent reduction. Since $a^{p-1} \equiv 1$, the powers of a modulo p repeat with period dividing $p-1$. Writing the exponent as $b = q(p-1) + r$ with $r = b \bmod (p-1)$ gives

$$a^b = \left(a^{p-1}\right)^q a^r \equiv 1^q \cdot a^r = a^{\,b \bmod (p-1)} \pmod p \qquad (p \text{ prime}, \ p \nmid a).$$

The slides' examples follow directly: $6^{10} \bmod 11 = 1$ because the exponent is exactly $p-1$, and $3^{12} \bmod 11 = 3^{11} \cdot 3 \equiv 3 \cdot 3 = 9$ by the second version (equivalently $12 \bmod 10 = 2$ and $3^2 = 9$). In practice the base is reduced modulo p first, then the exponent modulo $p-1$, and the small power that remains is computed by repeated squaring.

Trap: what to reduce, and by what

The **base** is reduced modulo p; the **exponent** is reduced modulo $p-1$, never modulo p. And the modulus must be prime: in $100^{95} \bmod 9$ the modulus $9 = 3^2$ is composite, so Fermat does not apply. There the base reduction alone finishes the job ($100 \equiv 1 \pmod 9$), or Euler's theorem with $\phi(9) = 6$ can be used.

WORKED EXAMPLE : exponent reduction by Fermat (Forouzan exercise)

Q. Using Fermat's little theorem, find (a) $5^{15} \bmod 13$ (b) $15^{18} \bmod 17$ (c) $456^{17} \bmod 17$.
A.

	Conditions	Base	Exponent	Result
(a)	13 prime, $13 \nmid 5$	5	$15 = 1 \cdot 12 + 3 \to 3$	$5^3 = 125 = 9 \cdot 13 + 8 \Rightarrow$ **8**
(b)	17 prime, $17 \nmid 15$	$15 \equiv -2$	$18 = 1 \cdot 16 + 2 \to 2$	$(-2)^2 =$ **4**
(c)	17 prime	$456 = 26 \cdot 17 + 14 \to 14$	second version: $a^{17} \equiv a$	**14**

Check (by script): $5^{15} \bmod 13 = 8$, $15^{18} \bmod 17 = 4$, $456^{17} \bmod 17 = 14$.

Fermat as a primality test. Read backwards, the theorem gives a test: if $a^{n-1} \not\equiv 1 \pmod n$ for some a with $\gcd(a,n)=1$, then n is certainly composite (e.g. $2^{14} \bmod 15 = 4$). The converse fails: $2^{340} \equiv 1 \pmod{341}$ although $341 = 11 \cdot 31$. The Fermat test, Carmichael numbers and Miller–Rabin are covered in §27.

26.2 Euler's theorem ★

▸ **Likely this year:** Exponent reduction modulo $\phi(n)$ for a composite n, and "why does RSA decryption work?" (Euler is its basis).

Fermat's theorem needs a prime modulus, but RSA works modulo $n = pq$. Euler's theorem is the generalisation to any modulus. **First version:** if $\gcd(a,n)=1$, then $a^{\phi(n)} \equiv 1 \pmod n$. When $n = p$ is prime, $\phi(p) = p-1$ and this is exactly Fermat's theorem.

Why it is true. The proof is the same permutation argument, run over $\mathbb{Z}_n^*$ (§25) instead of $\{1,\dots,p-1\}$. Multiplying the $\phi(n)$ elements of $\mathbb{Z}_n^*$ by a permutes them, because a is invertible; for $n=10$, $a=3$ the set $\{1,3,7,9\}$ becomes $\{3,9,1,7\}$. Taking the product of all elements on both sides gives $a^{\phi(n)} \cdot \Pi \equiv \Pi$, and the product Π is invertible, so $a^{\phi(n)} \equiv 1$.

The co-prime condition is genuinely needed. If $\gcd(a,n) = d > 1$, every power of a is a multiple of d, and so is every number $\equiv a^k \pmod n$; such a number can never be $\equiv 1$. The lecture homework $11^{20} \bmod 33$ shows this: the powers of 11 alternate $11, 22, 11, 22, \dots$, so $11^{20} \equiv 22 \neq 1$ although $\phi(33) = 20$.

a, n	gcd	Working	Result
3, 10	1	$\phi(10)=4$; $3^4 = 81 = 8\cdot 10 + 1$	$\equiv 1$ ✓
2, 10	**2**	$2^4 = 16 \equiv 6$	$\neq 1$: theorem silent
10, 11	1	$\phi(11)=10$; $10 \equiv -1$, $(-1)^{10} = 1$	$\equiv 1$ ✓
6, 35	1	$\phi(35)=24$; slide example $6^{24} \bmod 35$	$\equiv 1$ ✓
11, 33 (homework)	**11**	$\phi(33)=20$; powers alternate $11, 22$	$11^{20} \equiv 22 \neq 1$

Exponent reduction. Exactly as with Fermat, $\gcd(a,n)=1$ and $b = q\phi(n) + r$ give $a^b \equiv a^{b \bmod \phi(n)} \pmod n$. The work is in computing $\phi(n)$, which needs the factorisation of n.

Second version. For $n = pq$ (and more generally any squarefree n), $a^{k\phi(n)+1} \equiv a \pmod n$ for every integer a and every $k \geq 0$. This removes the co-prime condition, which RSA needs because a message M might, by bad luck, share the factor p with n. The proof works one prime at a time. Modulo p: if $p \nmid a$, then $a^{k\phi(n)} = \left(a^{p-1}\right)^{k(q-1)} \equiv 1$ by Fermat, so $a^{k\phi(n)+1} \equiv a$; if $p \mid a$, both sides are $\equiv 0$. The same holds modulo q. Since the distinct primes p and q both divide $a^{k\phi(n)+1} - a$, so does pq. For example $11^{21} = 11^{\phi(33)+1} \equiv 11 \pmod{33}$ although $\gcd(11,33) \neq 1$. The argument breaks when a prime appears squared: for $n=4$, $a=2$, $k=1$ the claim would be $2^3 \equiv 2$, but $2^3 = 8 \equiv 0 \pmod 4$.

Why RSA decryption works. RSA picks e and d with $ed \equiv 1 \pmod{\phi(n)}$, i.e. $ed = k\phi(n)+1$. Decrypting $C = M^e$ gives $C^d = M^{ed} = M^{k\phi(n)+1} \equiv M \pmod n$ by the second version, for every message $M < n$. In the example $n = 187$, $e = 7$, $d = 23$: $ed = 161 = 1 \cdot 160 + 1$, and indeed $88^7 \bmod 187 = 11$ and $11^{23} \bmod 187 = 88$.

★ MUST KNOW

	First version	Second version (no co-prime condition)
Fermat (p prime)	$a^{p-1} \equiv 1 \pmod p$ if $p \nmid a$	$a^p \equiv a \pmod p$
Euler	$a^{\phi(n)} \equiv 1 \pmod n$ if $\gcd(a,n)=1$	$a^{k\phi(n)+1} \equiv a \pmod n$ ($n = pq$, squarefree)
Reduction	$a^b \equiv a^{b \bmod (p-1)} \pmod p$ and	$a^b \equiv a^{b \bmod \phi(n)} \pmod n$ (co-prime a)
Inverse	$a^{-1} \equiv a^{p-2} \pmod p$ and	$a^{-1} \equiv a^{\phi(n)-1} \pmod n$ (co-prime a)

WORKED EXAMPLE : exponent reduction by Euler

Q. Using Euler's theorem, find (a) $11^{2024} \bmod 35$ (b) the last two digits of 3^{1003} (c) $20^{62} \bmod 77$ (slide example).
A.

	gcd, $\phi(n)$	Exponent	Remaining power	Result
(a)	$\gcd(11,35)=1$, $\phi(35) = 4\cdot 6 = 24$	$2024 = 84\cdot 24 + 8$	$11^2 = 121 \equiv 16$, $11^4 \equiv 16^2 = 256 \equiv 11$, $11^8 \equiv 11^2 \equiv 16$	**16**
(b)	$\gcd(3,100)=1$, $\phi(100) = 40$	$1003 = 25\cdot 40 + 3$	$3^3 = 27$	**27**
(c)	$\gcd(20,77)=1$, $\phi(77) = 6\cdot 10 = 60$	$62 = 1\cdot 60 + 2$	$20^2 = 400 = 5\cdot 77 + 15$	**15**

For (b), "last two digits" means the residue modulo 100. **Check** (by script): $11^{2024} \bmod 35 = 16$, $3^{1003} \bmod 100 = 27$, $20^{62} \bmod 77 = 15$.

26.3 Multiplicative inverses via Fermat and Euler ★

★ **In the papers:** 2024–25 Q5(iii): $7^{-1} \bmod 17$, $15^{-1} \bmod 99$ (does not exist).

The multiplicative inverse of a modulo n is the residue a^{-1} with $a \cdot a^{-1} \equiv 1 \pmod{n}$; it exists if and only if $\gcd(a, n) = 1$ (derived in §25). It depends on the modulus as well as on a: 2^{-1} is 3 modulo 5 but 4 modulo 7. For small n it can be found by trial, multiplying a by $1, 2, 3, \ldots$ until the product is $\equiv 1$:

a mod n	gcd	Search	Inverse	Note
3 mod 5	1	$3 \cdot 2 = 6 \equiv 1$	**2**	so $2^{-1} \bmod 5 = 3$ as well
2 mod 11	1	$2 \cdot 6 = 12 \equiv 1$	**6**	
4 mod 5	1	$4 \cdot 4 = 16 \equiv 1$	**4**	its own inverse ($4 \equiv -1$)
2 mod 7	1	$2 \cdot 4 = 8 \equiv 1$	**4**	not 3: depends on n
5 mod 10	**5**	$5k \bmod 10 \in \{0, 5\}$ only	**none**	

Inverse by exponentiation. Fermat's theorem already contains the inverse. If p is prime and $p \nmid a$, then $a \cdot a^{p-2} = a^{p-1} \equiv 1$, so

$$a^{-1} \equiv a^{p-2} \pmod{p}, \qquad \text{and in the same way} \qquad a^{-1} \equiv a^{\phi(n)-1} \pmod{n} \text{ if } \gcd(a, n) = 1.$$

This replaces the extended Euclidean algorithm by one modular exponentiation, which is convenient when the modulus is prime or when $\phi(n)$ is known. The price is that for a composite n the factorisation is needed to get $\phi(n)$. The slides' examples (L10-15 and L10-20), all checked by script:

Fermat (n prime)		Euler (n composite)		
$8^{-1} \bmod 17 = 8^{15}$	$= 15$	$8^{-1} \bmod 77 = 8^{59}$	$= 29$	$\phi(77) = 60$
$5^{-1} \bmod 23 = 5^{21}$	$= 14$	$7^{-1} \bmod 15 = 7^{7}$	$= 13$	$\phi(15) = 8$
$60^{-1} \bmod 101 = 60^{99}$	$= 32$	$60^{-1} \bmod 187 = 60^{159}$	$= 53$	$\phi(187) = 160$
$22^{-1} \bmod 211 = 22^{209}$	$= 48$	$71^{-1} \bmod 100 = 71^{39}$	$= 31$	$\phi(100) = 40$
$2^{-1} \bmod 13 = 2^{11}$	$= 7$	$3^{-1} \bmod 20 = 3^{7}$	$= 7$	$\phi(20) = 8$

Each answer is checked by one multiplication, e.g. $8 \cdot 15 = 120 = 7 \cdot 17 + 1$ and $7 \cdot 13 = 91 = 6 \cdot 15 + 1$.

Trap: the inverse may not exist

Before using any formula, compute $\gcd(a, n)$. If it is not 1 there is no inverse, and the exam expects exactly that answer with the reason. Blindly applying Euler's formula still produces a number, which is wrong: for $15^{-1} \bmod 99$, $\phi(99) = 60$ and $15^{59} \bmod 99 = 36$, but $15 \cdot 36 = 540 \equiv 45 \not\equiv 1 \pmod{99}$.

WORKED EXAMPLE : 2024–25 Q5(ii) and (iii), as it should be written

Q. (ii) Find the values of $100^{95} \bmod 9$, $658^{17} \bmod 17$ and $145^{102} \bmod 101$. (iii) Find the values of $7^{-1} \bmod 17$ and $15^{-1} \bmod 99$.

A. (ii) $100^{95} \bmod 9$. The modulus 9 is not prime, so Fermat's theorem cannot be used; reduce the base instead. $100 = 11 \cdot 9 + 1$, so $100 \equiv 1 \pmod 9$ and $100^{95} \equiv 1^{95} = \mathbf{1}$. (Euler agrees: $\gcd(100, 9) = 1$, $\phi(9) = 6$, $95 \bmod 6 = 5$, $1^5 = 1$.)

$658^{17} \bmod 17$. 17 is prime, so by the second version of Fermat's theorem $a^{17} \equiv a \pmod{17}$. $658 = 38 \cdot 17 + 12$, hence $658^{17} \equiv 658 \equiv \mathbf{12}$.

$145^{102} \bmod 101$. 101 is prime and $101 \nmid 145$. Base: $145 \equiv 44$. Exponent: $102 = 1 \cdot 100 + 2$, and $44^{100} \equiv 1$ by Fermat. So $145^{102} \equiv 44^2 = 1936 = 19 \cdot 101 + 17$, i.e. **17**.

(iii) $7^{-1} \bmod 17$. 17 is prime and $17 \nmid 7$, so $7^{-1} \equiv 7^{17-2} = 7^{15} \pmod{17}$. With $15 = 8 + 4 + 2 + 1$:

7^1	7^2	7^4	7^8	$7^{15} = 7^8 \cdot 7^4 \cdot 7^2 \cdot 7$
7	$49 \equiv 15$	$15^2 = 225 \equiv 4$	$4^2 = 16$	$16 \cdot 4 = 64 \equiv 13$; $13 \cdot 15 = 195 \equiv 8$; $8 \cdot 7 = 56 \equiv \mathbf{5}$

So $7^{-1} \bmod 17 = \mathbf{5}$. Check: $7 \cdot 5 = 35 = 2 \cdot 17 + 1$ ✓.

$15^{-1} \bmod 99$. $\gcd(99, 15)$: $99 = 6 \cdot 15 + 9$, $15 = 1 \cdot 9 + 6$, $9 = 1 \cdot 6 + 3$, $6 = 2 \cdot 3 + 0$, so $\gcd = 3 \neq 1$. Every number $15x - 99k$ is a multiple of 3 and can never equal 1, therefore $15^{-1} \bmod 99$ **does not exist**.

Which method? All four methods give the same inverse when it exists; they differ in what they need. The summary below is the decision to make in an exam.

Method	Formula	When to use
Trial	$a \cdot 1, a \cdot 2, \ldots$ until $\equiv 1$	Small n only (mod 26 ciphers, GF(7) division).
Fermat	$a^{-1} = a^{p-2} \bmod p$	Prime modulus; compute with square-and-multiply.
Euler	$a^{-1} = a^{\phi(n)-1} \bmod n$	Composite n whose factorisation (hence $\phi(n)$) is known.
Extended Euclid	$sa + tn = 1 \Rightarrow a^{-1} = s \bmod n$	The general method: works for any n without factoring, and detects "no inverse" (final gcd $\neq 1$). The past papers ask for it explicitly (2024–25 Q7, §24); RSA uses it to find d.

26.4 Order of an element ▸

▸ **Likely this year:** "Find the order of every element of $\mathbb{Z}_7^*$ / $\mathbb{Z}_{10}^*$" (Forouzan), usually as the first step of a primitive-root question.

Euler's theorem says that $a^{\phi(n)} \equiv 1$ for every $a \in \mathbb{Z}_n^*$, but many elements return to 1 much earlier. The order of a modulo n, $\operatorname{ord}(a)$, is the smallest positive integer i with $a^i \equiv 1 \pmod{n}$. It exists for every $a \in \mathbb{Z}_n^*$, and Euler's theorem guarantees $\operatorname{ord}(a) \le \phi(n)$. After $a^{\operatorname{ord}(a)} = 1$ the powers start over, so the sequence $a, a^2, a^3, \ldots$ is periodic with period exactly $\operatorname{ord}(a)$.

The key property: $a^m \equiv 1 \iff \operatorname{ord}(a) \mid m$. Write $m = q \cdot \operatorname{ord}(a) + r$ with $0 \le r < \operatorname{ord}(a)$. Then $a^m = \left(a^{\operatorname{ord}(a)}\right)^q a^r \equiv a^r$, and by the minimality of the order $a^r \equiv 1$ only for $r = 0$. Two consequences follow. First, since $a^{\phi(n)} \equiv 1$, **the order always divides** $\phi(n)$ (this is Lagrange's theorem for the group $\mathbb{Z}_n^*$), so to find an order only the divisors of $\phi(n)$ need to be tried. Second, exponents can be reduced modulo $\operatorname{ord}(a)$, which is sharper than reducing modulo $\phi(n)$: $\operatorname{ord}(2) = 3$ modulo 7, so $2^{100} \equiv 2^{100 \bmod 3} = 2^1 = 2 \pmod 7$.

The table lists $a^i \bmod 7$ for the whole group $\mathbb{Z}_7^*$ (slide L11-30); the shaded cell is the first 1 in each row, and its column is the order. Every order is a divisor of $\phi(7) = 6$, and only $a = 3$ and $a = 5$ reach the maximum 6.

a	$i=1$	2	3	4	5	6	$\operatorname{ord}(a)$
1	**1**	1	1	1	1	1	1
2	2	4	**1**	2	4	1	3
3	3	2	6	4	5	**1**	**6**
4	4	2	**1**	4	2	1	3
5	5	4	6	2	3	**1**	**6**
6	6	**1**	6	1	6	1	2

Two more slide examples (L11-27, L11-29): in $\mathbb{Z}_{10}^* = \{1,3,7,9\}$ the orders are $1, 4, 4, 2$ ($3^2 = 9$, $3^4 = 81 \equiv 1$), and in $\mathbb{Z}_8^* = \{1,3,5,7\}$ they are $1, 2, 2, 2$, because $3^2 = 9$, $5^2 = 25$ and $7^2 = 49$ are all $\equiv 1 \pmod 8$.

26.5 Primitive roots ▸

▸ **Likely this year:** "Find all primitive roots of 7 / 11 / 13", "is 3 a primitive root of 17?", "how many primitive roots does 17 have?" (needed for Diffie–Hellman and ElGamal).

The idea, for a prime p. Take the multiplicative group $\mathbb{Z}_p^* = \{1, 2, \ldots, p-1\}$, which is $\mathbb{Z}_p$ without 0. Pick an element g and write down its powers $g^1, g^2, g^3, \ldots \pmod p$. Two things can happen. Either the powers run through *every* element of $\mathbb{Z}_p^*$ before returning to 1, or they hit 1 early and then repeat a smaller cycle. In the first case g is called a primitive root of p: its powers $g^1, \ldots, g^{p-1}$ produce each of $1, \ldots, p-1$ exactly once. Modulo 7, for example:

	g^1	g^2	g^3	g^4	g^5	g^6	
$g = 3$	3	2	6	4	5	1	all six elements: **primitive root**
$g = 2$	2	4	1	2	4	1	only $\{2, 4, 1\}$: **not** a primitive root

Since the powers start repeating as soon as they reach 1, the number of different powers equals the order of g. Producing all $p-1$ elements therefore means exactly that $\operatorname{ord}(g) = p-1$, the largest order possible. When such a g exists, every element of the group is a power of it, so the group is *cyclic* and g is a *generator*. Every element y is then g^x for exactly one $x \in \{1, \ldots, p-1\}$, and that x is the discrete logarithm of y (§27).

Why cryptography needs them. Diffie–Hellman and ElGamal publish p and a base g, keep an exponent x secret, and publish $g^x \bmod p$. If g had a small order d, then g^x could take only d different values, and Eve could find a matching exponent by trying d values. Choosing a primitive root makes all $p-1$ values possible, so the search is as large as it can be.

The same idea for any n. Nothing changes except the group. For a general n the group is $\mathbb{Z}_n^*$, the numbers below n that are coprime to n, and it has $\phi(n)$ elements instead of $p-1$. So $g \in \mathbb{Z}_n^*$ is a primitive root of n if $\operatorname{ord}(g) = \phi(n)$, which means $g^1, \ldots, g^{\phi(n)}$ produce every element of $\mathbb{Z}_n^*$ once each. Modulo 10, $\mathbb{Z}_{10}^* = \{1,3,7,9\}$ and the powers of 3 are $3, 9, 7, 1$, so 3 is a primitive root of 10. The powers never produce 0, 2, 4, 5, 6 or 8, and they are not supposed to: a power of a number coprime to n is itself coprime to n, so only $\mathbb{Z}_n^*$ can ever be generated, never all of $\mathbb{Z}_n$. For a prime this is the same statement as before, since $\mathbb{Z}_p^*$ is $\mathbb{Z}_p$ without 0 and $\phi(p) = p-1$.

Unlike primes, many composite n have *no* primitive root at all, because every element returns to 1 too early. Modulo 8, the group $\mathbb{Z}_8^* = \{1,3,5,7\}$ has $\phi(8) = 4$ elements, but 3^2, 5^2 and 7^2 are all $\equiv 1$, so no order reaches 4 (slide L11-28). Modulo 15, every element satisfies $a^4 \equiv 1$, because $a^2 \equiv 1 \pmod 3$ and $a^4 \equiv 1 \pmod 5$, so no order reaches $\phi(15) = 8$. The complete answer is a theorem worth memorising:

★ MUST KNOW : primitive roots

g is a primitive root of $n \iff \operatorname{ord}(g) = \phi(n) \iff$ the powers of g produce all of $\mathbb{Z}_n^*$ (for a prime: all of $1, \dots, p-1$). Primitive roots exist **only** for $n = 2,\ 4,\ p^k,\ 2p^k$ (p an odd prime). Every prime has them; 9, 10, 18 have them; 8, 12, 15 do not.
When they exist there are exactly $\phi(\phi(n))$ of them; for a prime p that is $\phi(p-1)$.

Test 1: list the powers. Multiply by g repeatedly, reducing each time; for example $3^4 \equiv 6 \cdot 3 = 18 \equiv 4 \pmod 7$, so you never need $3^6 = 729$. If a 1 appears before the $(p-1)$-th power, stop: g is not a primitive root.

Test	Powers $g^1, g^2, \dots \pmod p$	Verdict
2 mod 5	2, 4, 3, 1	order $4 = p-1$: **yes**
3 mod 5	3, 4, 2, 1	**yes**
4 mod 5	4, **1**	order 2: no
3 mod 7	3, 2, 6, 4, 5, 1	**yes**
2 mod 7	2, 4, **1**	order 3: no
2 mod 11 (homework)	2, 4, 8, 5, 10, 9, 7, 3, 6, 1	**yes**

The homework "primitive roots of 5" is answered by the first three rows: 1 never works ($1^k = 1$), 2 and 3 do, and 4 does not, so the primitive roots of 5 are **2 and 3**.

Test 2: the fast test. Listing takes up to $p-1$ multiplications. It is faster to use the fact that the order of g must divide $p-1$: if g is not a primitive root, its order is a proper divisor of $p-1$, and every proper divisor divides $(p-1)/q$ for some prime factor q of $p-1$. So it is enough to check those few exponents:

$$g \text{ is a primitive root of } p \iff g^{(p-1)/q} \not\equiv 1 \pmod p \text{ for every prime } q \text{ dividing } p-1.$$

For $p = 7$, $p - 1 = 6 = 2 \cdot 3$, so check g^3 and g^2. For $g = 3$: $3^3 = 27 \equiv 6$ and $3^2 = 9 \equiv 2$, neither is 1, so 3 is a primitive root. For $g = 2$: $2^3 = 8 \equiv 1$, so 2 is not.

Finding all of them from one. Once one primitive root g is known, the others are the powers g^k with $\gcd(k, p-1) = 1$. The reason is that g^k has order $\frac{p-1}{\gcd(k,p-1)}$, which equals $p-1$ exactly when the gcd is 1. Counting those k gives the $\phi(p-1)$ from the box: 5 has $\phi(4) = 2$ (2, 3), 7 has $\phi(6) = 2$ (3, 5), 11 has $\phi(10) = 4$ (2, 6, 7, 8), and 13 has $\phi(12) = 4$ (2, 6, 7, 11).

WORKED EXAMPLE : all primitive roots of a prime

Q. Find all the primitive roots of 11 (i.e. of the group $G = \langle \mathbb{Z}_{11}^*, \times \rangle$).
A. $\phi(11) = 10 = 2 \cdot 5$, so a primitive root g must satisfy $g^{10/2} = g^5 \not\equiv 1$ and $g^{10/5} = g^2 \not\equiv 1$. Try $g = 2$: $2^2 = 4 \not\equiv 1$ and $2^5 = 32 = 2 \cdot 11 + 10 \equiv 10 \not\equiv 1$, so **2 is a primitive root**. (For contrast, $g = 3$ fails: $3^5 = 243 = 22 \cdot 11 + 1 \equiv 1$.)
The other primitive roots are 2^k with $\gcd(k, 10) = 1$, i.e. $k \in \{1, 3, 7, 9\}$:

k	1	3	7	9
$2^k \bmod 11$	2	8	$128 = 11 \cdot 11 + 7 \to 7$	$512 = 46 \cdot 11 + 6 \to 6$

The primitive roots of 11 are **2, 6, 7, 8**. **Check:** their number is $\phi(10) = 4$ ✓, and the full list of powers of 2 is 2, 4, 8, 5, 10, 9, 7, 3, 6, 1, all ten residues.

WORKED EXAMPLE : fast test and counting

Q. Show that 3 is a primitive root of 17 but 2 is not. How many primitive roots does 17 have? List them.
A. $\phi(17) = 16 = 2^4$, whose only prime factor is 2, so g is a primitive root iff $g^{16/2} = g^8 \not\equiv 1 \pmod{17}$. For $g = 3$: $3^2 = 9$, $3^4 = 81 \equiv 13$, $3^8 \equiv 13^2 = 169 \equiv 16 \not\equiv 1$, so **3 is a primitive root**. For $g = 2$: $2^4 = 16$, $2^8 \equiv 16^2 = 256 = 15 \cdot 17 + 1 \equiv 1$, so **2 is not** (its order is 8). The number of primitive roots is $\phi(16) = \mathbf{8}$. They are 3^k for odd k: $3^1, 3^3, \dots, 3^{15} \equiv 3, 10, 5, 11, 14, 7, 12, 6$, i.e. **3, 5, 6, 7, 10, 11, 12, 14**.

Corrections to the notes / slides

- All values recomputed by script. The slides' inverse examples (L10-15: 15, 14, 32, 48; L10-20: 29, 13, 53, 31) and examples $6^{10} \bmod 11 = 1$, $3^{12} \bmod 11 = 9$, $6^{24} \bmod 35 = 1$, $20^{62} \bmod 77 = 15$ are correct. The video homework answers are correct: $5^{10} \equiv 1 \pmod{11}$, $11^{20} \equiv 22 \pmod{33}$, 2 is a primitive root of 11, the primitive roots of 5 are 2 and 3.
- Slide L10-16 says the second version of Euler's theorem "removes the condition that a and n should be coprime". That is true only for squarefree n (such as $n = pq$ in RSA). It fails for $n = 4$, $a = 2$: $2^{\phi(4)+1} = 2^3 = 8 \equiv 0 \neq 2$.
- Slide L10-18 solves $20^{62} \bmod 77$ with the second version, but $\gcd(20, 77) = 1$, so the first version already gives $20^{62} = 20^{60} \cdot 20^2 \equiv 400 \equiv 15$.
- Video notation: "-2^{12}" and "-1^{10}" mean $(-2)^{12}$ and $(-1)^{10}$. Without brackets $-2^{12} = -(2^{12})$; write the brackets in the exam.
- The general definition of a primitive root (any n) was stated in the video but not explained: only $n = 2, 4, p^k, 2p^k$ have primitive roots, and then there are $\phi(\phi(n))$ of them.

27 Primes, Primality Testing, Factorization, CRT and Discrete Logs

Asymmetric-key cryptography is built on a handful of facts about primes, and this chapter collects them. RSA needs two large random primes, so we need a fast way to decide whether a 1024-bit number is prime; this is *primality testing*. RSA is secure only because nobody can split $n = pq$ back into p and q; this is the hardness of *factorization*. RSA decryption is made about four times faster by working modulo p and q separately and gluing the results with the *Chinese remainder theorem*. Every public-key operation is a modular power, computed by *square-and-multiply*. Finally, Diffie–Hellman and ElGamal rely on the fact that undoing a modular power, the *discrete logarithm*, is infeasible.

27.1 Primes and how many there are

A positive integer is prime if it has exactly two divisors, 1 and itself; a *composite* has more than two divisors, and the number 1 (exactly one divisor) is neither. Two integers are *coprime* (relatively prime) when their gcd is 1, so a prime p is coprime to every integer from 1 to $p - 1$, which is why $\mathbb{Z}_p^* = \{1, \dots, p-1\}$.

Infinitely many primes. Suppose the primes were only $p_1, \dots, p_k$, and let $N = p_1 p_2 \cdots p_k + 1$. Dividing N by any p_i leaves remainder 1, so N either is a new prime or has a prime factor outside the list; either way the list was incomplete. Note that N itself need not be prime: $2 \cdot 3 \cdot 5 \cdot 7 \cdot 11 \cdot 13 + 1 = 30031 = 59 \times 509$, but 59 and 509 are new primes.

Counting primes. The function $\pi(n)$ counts the primes $\le n$: $\pi(1) = 0$, $\pi(2) = 1$, $\pi(3) = 2$, $\pi(10) = 4$, $\pi(20) = 8$, $\pi(50) = 15$, $\pi(100) = 25$. For large n there is no formula, only bounds (Lagrange found the lower one, Gauss the upper):

$$\frac{n}{\ln n} < \pi(n) < \frac{n}{\ln n - 1.08366}.$$

For $n = 10^6$: $n/\ln n = 72{,}382.4$ and $n/(\ln n - 1.08366) = 78{,}543.2$, so $72{,}383 \le \pi(10^6) \le 78{,}543$; the actual count is 78,498. The practical consequence is that primes thin out only slowly: near N about one number in $\ln N$ is prime. For 1024-bit numbers $\ln 2^{1024} \approx 710$, so if we test random *odd* 1024-bit numbers, about one in 355 is prime, and generating an RSA prime takes a few hundred primality tests. This is why the test itself must be fast.

Checking a number by $\sqrt{n}$. If $n = ab$ with $1 < a \le b$, then $a^2 \le ab = n$, so $a \le \sqrt{n}$. Therefore a composite n always has a prime factor $\le \lfloor \sqrt{n} \rfloor$, and it is enough to try the primes up to $\lfloor \sqrt{n} \rfloor$.

- Is 97 prime? $\lfloor \sqrt{97} \rfloor = 9$, and none of 2, 3, 5, 7 divides 97, so **97 is prime**.
- Is 301 prime? $\lfloor \sqrt{301} \rfloor = 17$, so we would try 2, 3, 5, 7, 11, 13, 17; but already $7 \mid 301$ ($301 = 7 \times 43$), so **301 is composite**.

Mersenne and Fermat numbers. These were early attempts at formulas that produce primes. A *Mersenne number* is $M_p = 2^p - 1$. If the exponent factors as ab, then $2^a - 1$ divides $2^{ab} - 1$ (since $x - 1 \mid x^b - 1$ with $x = 2^a$), so M_p can be prime only for prime p. The condition is not sufficient: $M_2, M_3, M_5, M_7, M_{13}, M_{17}, M_{19} = 3, 7, 31, 127, 8191, 131071, 524287$ are prime, but $M_{11} = 2047 = 23 \times 89$ and $M_{23} = 8388607 = 47 \times 178481$. A *Fermat number* is $F_n = 2^{2^n} + 1$. Fermat saw that $F_0, \dots, F_4 = 3, 5, 17, 257, 65537$ are prime and guessed that all are, but Euler found $F_5 = 4294967297 = 641 \times 6700417$, and no larger Fermat number is known to be prime. $F_4 = 65537 = 2^{16} + 1$ survives in practice as the standard RSA public exponent, because its binary form has only two 1-bits, which makes encryption with square-and-multiply cheap.

27.2 Primality testing: deterministic and probabilistic

The problem is to decide, for a number of several hundred digits, whether it is prime. A *deterministic* algorithm always gives the correct answer; a *probabilistic* algorithm is much faster but may declare a composite number

"prime" with a small, controllable probability. Its "composite" answers are always correct, because they come with a witness.

Divisibility test (deterministic). Try every $r = 2, 3, \ldots, \lfloor\sqrt{n}\rfloor$ and return "composite" as soon as $r \mid n$, otherwise "prime". It is correct by the $\sqrt{n}$ argument above, but its cost is exponential in the size of the input: an n_b-bit number has $\sqrt{n} \approx 2^{n_b/2}$ candidate divisors. For a 200-bit n this is 2^{100} bit operations; at 2^{30} operations per second that takes 2^{70} seconds, about 3.7×10^{13} years.

AKS (deterministic, polynomial). In 2002 Agrawal, Kayal and Saxena gave the first deterministic test whose time is polynomial in the number of bits, $O((\log_2 n)^{12})$ bit operations, based on the identity $(x-a)^p \equiv x^p - a \pmod p$, which holds exactly when p is prime. It settled a theoretical question, but in practice the probabilistic tests below are far faster.

Fermat test (probabilistic). Fermat's little theorem says that if n is prime then $a^{n-1} \equiv 1 \pmod n$ for every a with $n \nmid a$. Read in reverse: if $a^{n-1} \not\equiv 1 \pmod n$, then n is certainly composite; if $a^{n-1} \equiv 1$, then n is only *probably* prime, because some composites also satisfy it. For example $341 = 11 \times 31$ gives $2^{340} \equiv 1 \pmod{341}$ (since $2^{10} = 1024 = 3 \cdot 341 + 1$), so base 2 is fooled, but base 3 exposes it: $3^{340} \equiv 56 \pmod{341}$.

Carmichael numbers. Trying more bases does not always help. A *Carmichael number* is a composite n with $a^{n-1} \equiv 1 \pmod n$ for *every* a coprime to n, so the Fermat test fails for all such bases. The smallest is $561 = 3 \times 11 \times 17$ (then 1105, 1729, 2465, 2821). The reason is visible from the factors: $561 - 1 = 560$ is divisible by $3-1$, $11-1$ and $17-1$, so for each prime factor p, $a^{560} = (a^{p-1})^{560/(p-1)} \equiv 1 \pmod p$, and a number that is $\equiv 1$ modulo 3, 11 and 17 is $\equiv 1$ modulo 561 (by the CRT below). In general n is Carmichael exactly when it is squarefree and $(p-1) \mid (n-1)$ for every prime $p \mid n$ (Korselt's criterion). Only bases sharing a factor with n catch it, e.g. $3^{560} \equiv 375 \pmod{561}$, and for large n such bases are too rare to find by chance.

Square-root test. If $n = p$ is prime and $x^2 \equiv 1 \pmod p$, then $p \mid (x-1)(x+1)$, and since p is prime it divides one of the two factors, so $x \equiv \pm 1 \pmod p$. Hence modulo a prime, the only square roots of 1 are $+1$ and -1 (written $n-1$ for -1). A composite modulus usually has extra, *nontrivial* square roots of 1, and finding one proves that n is composite.

n	Square roots of 1 mod n	Comment
7 (prime)	1, 6	only ± 1
17 (prime)	1, 16	only ± 1
8 (composite)	1, 3, 5, 7	extra roots 3, 5 $\Rightarrow$ composite
15 (composite)	1, 4, 11, 14	extra roots 4, 11
22 (composite)	1, 21	only ± 1 although composite
561 (composite)	1, 67, 188, 254, 307, 373, 494, 560	$2^3 = 8$ roots (3 prime factors)

The test is one-way: $n = 22 = 2 \times 11$ has only ± 1, because modulo 2 there is a single square root of 1. For an odd n with r distinct prime factors there are 2^r square roots of 1 (a choice of ± 1 modulo each prime factor, glued by the CRT). A nontrivial root also *factors* n: since $n \mid (x-1)(x+1)$ but divides neither factor, $\gcd(x \pm 1, n)$ are proper factors; for $x = 67$, $\gcd(66, 561) = 33$ and $\gcd(68, 561) = 17$.

27.3 Miller–Rabin test ▸

▸ **Likely this year:** "Use the Miller–Rabin test with base 2 to check whether 561 (or 61, 341) is prime": write $n - 1 = m \cdot 2^k$, then the T table.

The Fermat test only looks at the final value a^{n-1}, and Carmichael numbers make that value 1 for every base. Miller–Rabin also looks at the values *on the way* to a^{n-1} and applies the square-root test to them, which catches Carmichael numbers too.

Idea. Write $n - 1 = m \cdot 2^k$ with m odd (n odd, so $k \geq 1$). Then a^{n-1} is obtained from a^m by squaring k times:

$$a^m,\ a^{2m},\ a^{4m},\ \ldots,\ a^{2^k m} = a^{n-1} \pmod n,$$

each term being the square of the previous one. If n is prime, the last term is 1 by Fermat. Walking backwards from it, every term whose square is 1 must be ± 1, by the square-root test. So for a prime n the sequence either starts with $a^m \equiv 1$ (then it is all 1s), or it contains a -1 somewhere before the last position, after which it is all 1s. If the sequence reaches $+1$ *without* a -1 just before it, the previous term is a nontrivial square root of 1 and n is composite; if it never reaches ± 1, Fermat already fails.

★ MUST KNOW : Miller–Rabin (base a)

Find m odd and k with $n - 1 = m \times 2^k$. Set $T \leftarrow a^m \bmod n$. If $T = \pm 1$, return **prime**.
Repeat at most $k - 1$ times: $T \leftarrow T^2 \bmod n$; if $T = +1$ return **composite** (the previous T was a nontrivial square root of 1); if $T = -1$ return **prime**.
If the loop ends without a decision, return **composite**.

The loop stops after $k-1$ squarings because the k-th square is a^{n-1} itself: if the previous T is not ± 1, then either its square is 1 (a nontrivial root) or it is not 1 (Fermat fails), so n is composite in both cases and nothing is gained by computing it. "Prime" here means "probably prime": a composite n passes for at most $1/4$ of the bases (such bases are called *strong liars*), so t independent random bases leave an error probability of at most 4^{-t}, e.g. $4^{-40} \approx 10^{-24}$. The *recommended primality test* (Forouzan) is therefore trial division by small primes first, which cheaply removes most composites, followed by Miller–Rabin with several bases.

WORKED EXAMPLE : Miller–Rabin on 561 and 61 (Forouzan, base 2)

Q. Use the Miller–Rabin test with base 2 to decide whether (a) 561 and (b) 61 are prime.
(a) $n-1 = 560 = 35 \times 2^4$, so $m = 35$, $k = 4$, $a = 2$, and at most $k-1 = 3$ squarings. First $2^{35} \bmod 561$ by repeated squaring: $2^8 = 256$, $2^{16} = 256^2 \bmod 561 = 460$, $2^{32} = 460^2 \bmod 561 = 103$, so $2^{35} = 2^{32} \cdot 2^3 \equiv 103 \times 8 = 824 \equiv 263$.

Step	Computation	T	Decision
Init	$2^{35} \bmod 561$	263	not ± 1, continue
1	$263^2 = 69169 \bmod 561$	166	not ± 1, continue
2	$166^2 = 27556 \bmod 561$	67	not ± 1, continue
3	$67^2 = 4489 \bmod 561$	**1**	$+1 \Rightarrow$ **composite**

561 is composite. Justification: $67^2 \equiv 1$ with $67 \neq \pm 1$, so 67 is a nontrivial square root of 1, impossible modulo a prime. (Indeed $561 = 3 \times 11 \times 17$; and $\gcd(66, 561) = 33$, $\gcd(68, 561) = 17$.) Note that one more squaring gives $2^{560} \equiv 1$, so 561 *passes* the Fermat test; Miller–Rabin catches it only because it inspects the step from 67 to 1.

(b) $n-1 = 60 = 15 \times 2^2$, so $m = 15$, $k = 2$, at most one squaring. $2^{15} = 2^8 \cdot 2^4 \cdot 2^2 \cdot 2 \equiv 12 \times 16 \times 4 \times 2 = 1536 = 25 \cdot 61 + 11 \equiv 11 \pmod{61}$.

Step	Computation	T	Decision
Init	$2^{15} \bmod 61$	11	not ± 1, continue
1	$11^2 = 121 \bmod 61$	$60 = -1$	$-1 \Rightarrow$ **prime**

61 passes (probably prime), and 61 is indeed prime. Check: $2^{60} = (2^{30})^2 \equiv (-1)^2 = 1$, consistent with Fermat.

WORKED EXAMPLE : Fermat test vs Miller–Rabin on 341 (new)

Q. Show that 341 passes the Fermat test with base 2, then apply the Miller–Rabin test with base 2. What can you conclude?
Fermat test. $2^{10} = 1024 = 3 \times 341 + 1$, so $2^{10} \equiv 1 \pmod{341}$ and $2^{340} = (2^{10})^{34} \equiv 1$. 341 passes the Fermat test with base 2.
Miller–Rabin. $340 = 85 \times 2^2$, so $m = 85$, $k = 2$, at most one squaring. $2^{85} = (2^{10})^8 \cdot 2^5 \equiv 1 \cdot 32 = 32$.

Step	Computation	T	Decision
Init	$2^{85} \bmod 341$	32	not ± 1, continue
1	$32^2 = 1024 \bmod 341$	**1**	$+1 \Rightarrow$ **composite**

341 is composite: 32 is a nontrivial square root of 1 modulo 341. Base 2 is a Fermat liar for 341 but not a strong liar. The root even gives the factors: $\gcd(31, 341) = 31$ and $\gcd(33, 341) = 11$, so $341 = 11 \times 31$.

WORKED EXAMPLE : a prime with a long sequence (new)

Q. Apply the Miller–Rabin test with base 2 to $n = 97$.
$96 = 3 \times 2^5$, so $m = 3$, $k = 5$, at most 4 squarings.

Step	Computation	T	Decision
Init	$2^3 \bmod 97$	8	continue
1	$8^2 = 64$	64	continue
2	$64^2 = 4096 = 42 \cdot 97 + 22$	22	continue
3	$22^2 = 484 = 5 \cdot 97 - 1$	$96 = -1$	**prime**

97 passes (and is prime, as the $\sqrt{97}$ check showed earlier).

The remaining slide runs and two instructive edge cases:

n, base	$n-1 = m \cdot 2^k$	T sequence	Result
27, 2	$13 \cdot 2^1$	11 (no loop, $k-1=0$)	composite (correct)
4033, 2	$63 \cdot 2^6$	3521, **4032** ($=-1$)	"prime" (wrong: 37×109)
4033, 3	$63 \cdot 2^6$	3551, 2443, 3442, 2443, 3442, 2443	composite (correct)
2047, 2	$1023 \cdot 2^1$	**1**	"prime" (wrong: $M_{11} = 23 \times 89$)
25, 7	$3 \cdot 2^3$	18, **24** ($=-1$)	"prime" (wrong: 5^2)

Bases 2 for 4033 and 2047, and 7 for 25, are strong liars; this is exactly why the test is repeated with several bases (4033 fools base 2 but not base 3).

27.4 Factorization

Fundamental theorem of arithmetic. Every integer $n > 1$ can be written as a product of primes, $n = p_1^{e_1} p_2^{e_2} \cdots p_k^{e_k}$, and this representation is unique apart from the order of the factors. Uniqueness lets us read gcd and lcm directly from the exponents. A common divisor can use each prime at most as often as *both* numbers contain it, and a common multiple must use it at least as often as *either* does:

$$\gcd(a,b) = \prod p_i^{\min(a_i,b_i)}, \qquad \operatorname{lcm}(a,b) = \prod p_i^{\max(a_i,b_i)}, \qquad \gcd(a,b) \times \operatorname{lcm}(a,b) = a \times b,$$

the last because $\min(a_i,b_i) + \max(a_i,b_i) = a_i + b_i$ for every prime. Example: $2160 = 2^4 \cdot 3^3 \cdot 5$ and $1200 = 2^4 \cdot 3 \cdot 5^2$ give $\gcd = 2^4 \cdot 3 \cdot 5 = 240$ and $\operatorname{lcm} = 2^4 \cdot 3^3 \cdot 5^2 = 10800$; check $240 \times 10800 = 2{,}592{,}000 = 2160 \times 1200$. This method is useless for large numbers, since it needs the factorization, which is exactly the hard part; Euclid's algorithm finds the gcd without it.

Why factorization matters. Finding $\phi(n)$, and with it the RSA private key, is as hard as factoring $n = pq$. Primality testing is easy (Miller–Rabin), but finding the factors of a number already known to be composite is believed to be hard, and RSA lives in the gap between the two problems.

Trial division. Divide n by $a = 2, 3, 4, \ldots$ while $a \le \sqrt{n}$; whenever $a \mid n$, output a, replace n by n/a, and try the same a again (to catch repeated factors). If $n > 1$ when the loop ends, the remaining n is a prime factor. Examples: $1233 = 3^2 \times 137$, and $1523357784 = 2^3 \times 3^2 \times 13 \times 37 \times 43987$. Like the divisibility test, it costs up to $\sqrt{n} = 2^{n_b/2}$ divisions, so it only works when n has small factors.

Fermat's factorization method. Any odd $n = ab$ (with $a \ge b$) is a difference of two squares: with $x = (a+b)/2$ and $y = (a-b)/2$ we get $x^2 - y^2 = ab = n$. Conversely, if we find x with $w = x^2 - n$ a perfect square y^2, then $n = (x+y)(x-y)$. The method starts at $x = \lceil \sqrt{n} \rceil$ and increases x by 1 until $x^2 - n$ is a square. Since the search ends at $x = (a+b)/2$, it is fast exactly when the two factors are close to $\sqrt{n}$.

x	$w = x^2 - 5959$	Square?	
78	125	no	($\lceil \sqrt{5959} \rceil = 78$)
79	282	no	
80	441	yes, 21^2	$5959 = (80+21)(80-21) =$ **101×59**

For 8051 it succeeds immediately ($90^2 - 8051 = 49 = 7^2$, so $8051 = 97 \times 83$), while $4033 = 37 \times 109$ needs 10 steps ($x = 64, \ldots, 73$; $73^2 - 4033 = 1296 = 36^2$). The security lesson is that RSA primes p and q must not be close to each other.

Other methods (named on the slides). *Pollard* $p-1$ finds a prime factor p quickly when $p-1$ has only small prime factors, by computing $\gcd(a^{B!} - 1, n)$. *Pollard rho* iterates $x \leftarrow x^2 + c \bmod n$ and finds a factor p in about $\sqrt{p}$ steps from a repetition modulo p, detected with a gcd. The *quadratic sieve* and the *number field sieve* build many congruences $x^2 \equiv y^2 \pmod{n}$ (a generalisation of Fermat's idea) and are sub-exponential; the number field sieve is the fastest known method for large RSA moduli, but it is still far too slow for 2048-bit n. (A *sieve of Eratosthenes*, by contrast, is the school method for *listing* all primes up to a bound by crossing out multiples.)

27.5 Chinese remainder theorem ▸

▸ **Likely this year:** Forouzan style "Solve $x \equiv 2 \pmod 3$, $x \equiv 3 \pmod 5$, $x \equiv 2 \pmod 7$" with the full M, M_i, M_i^{-1} table and the inverses by extended Euclid; or a word problem / RSA speed-up.

Problem. We are given several congruences in one unknown with different moduli,

$$x \equiv a_1 \pmod{m_1}, \quad x \equiv a_2 \pmod{m_2}, \quad \ldots, \quad x \equiv a_k \pmod{m_k},$$

and want all x that satisfy them simultaneously. The Chinese remainder theorem (CRT) states that if the moduli are pairwise coprime, the system has exactly one solution modulo $M = m_1 m_2 \cdots m_k$, and it gives a formula for it. For example $x \equiv 2 \pmod 3$, $x \equiv 3 \pmod 5$, $x \equiv 2 \pmod 7$ has the solution $x = 23$ (and $23 + 105t$).

Idea. Build x as a sum of k terms in which the i-th term is $\equiv a_i$ modulo m_i and $\equiv 0$ modulo every other modulus. Then modulo m_i all terms but the i-th vanish and $x \equiv a_i$, for each i at once. A number divisible by every modulus except m_i is $M_i = M/m_i$; multiplying it by its inverse modulo m_i turns it into 1 modulo m_i without disturbing the zeros, and multiplying by a_i gives the required value.

★ MUST KNOW : CRT in four steps

1. $M = m_1 \times m_2 \times \cdots \times m_k$ (the common modulus). 2. $M_i = M/m_i$ for each i.
3. M_i^{-1} = the inverse of M_i **modulo** m_i (reduce M_i mod m_i first, then extended Euclid).
4. $x = \left(a_1 M_1 M_1^{-1} + a_2 M_2 M_2^{-1} + \cdots + a_k M_k M_k^{-1}\right) \bmod M$.

Why it works. For $j \neq i$, m_j divides M_i, so the term $a_i M_i M_i^{-1}$ is $\equiv 0 \pmod{m_j}$. Modulo m_i the term equals $a_i \cdot (M_i M_i^{-1}) \equiv a_i \cdot 1$. Therefore $x \equiv a_i \pmod{m_i}$ for every i. Step 3 needs $\gcd(M_i, m_i) = 1$, and this is exactly where pairwise coprimality is used: M_i is a product of moduli each coprime to m_i.

Why the solution is unique modulo M. If x and x' both solve the system, every m_i divides $x - x'$; since the m_i are pairwise coprime, their product M divides $x - x'$. So all solutions are $x + tM$, and there is exactly one in $\{0, \ldots, M-1\}$. Equivalently, the map $x \mapsto (x \bmod m_1, \ldots, x \bmod m_k)$ is a one-to-one correspondence between $\mathbb{Z}_M$ and $\mathbb{Z}_{m_1} \times \cdots \times \mathbb{Z}_{m_k}$, which is what the applications below exploit.

Edge case: moduli not coprime. Then the theorem does not apply, and the system may have no solution or several. For instance $x \equiv 1 \pmod 4$ and $x \equiv 2 \pmod 6$ has no solution, because the first forces x odd and the second forces x even.

WORKED EXAMPLE : the standard CRT question (Forouzan, slide example)

Q. Solve $x \equiv 2 \pmod 3$, $x \equiv 3 \pmod 5$, $x \equiv 2 \pmod 7$ using the Chinese remainder theorem.
Step 1. The moduli 3, 5, 7 are pairwise coprime, so the CRT applies. $M = 3 \times 5 \times 7 = 105$.
Steps 2–3.

i	a_i	m_i	$M_i = M/m_i$	$M_i \bmod m_i$	$M_i^{-1} \bmod m_i$	$a_i M_i M_i^{-1}$
1	2	3	35	2	2	$2 \cdot 35 \cdot 2 = 140$
2	3	5	21	1	1	$3 \cdot 21 \cdot 1 = 63$
3	2	7	15	1	1	$2 \cdot 15 \cdot 1 = 30$

Inverses (extended Euclid with $r_1 = m_i$, $r_2 = M_i \bmod m_i$, $t_1 = 0$, $t_2 = 1$). For $35^{-1} \bmod 3 = 2^{-1} \bmod 3$:

q	r_1	r_2	r	t_1	t_2	t
1	3	2	1	0	1	−1
2	2	1	0	1	−1	3
	1	0		**−1**	3	

$r_1 = 1$, so the inverse exists and equals $t_1 = -1 \equiv \mathbf{2} \pmod 3$ (check $2 \cdot 2 = 4 \equiv 1$). Since $21 \equiv 1 \pmod 5$ and $15 \equiv 1 \pmod 7$, the other two inverses are 1.
Step 4. $x = (140 + 63 + 30) \bmod 105 = 233 \bmod 105 = \mathbf{23}$.
Check. $23 = 7 \cdot 3 + 2$, $23 = 4 \cdot 5 + 3$, $23 = 3 \cdot 7 + 2$ ✓. All solutions: $x = 23 + 105t$.

WORKED EXAMPLE : CRT with non-trivial inverses (new)

Q. Find the smallest positive x with $x \equiv 3 \pmod 7$, $x \equiv 5 \pmod{11}$, $x \equiv 7 \pmod{13}$.
Step 1. 7, 11, 13 are distinct primes, hence pairwise coprime. $M = 7 \times 11 \times 13 = 1001$.
Steps 2–3.

i	a_i	m_i	M_i	$M_i \bmod m_i$	$M_i^{-1} \bmod m_i$	$a_i M_i M_i^{-1}$
1	3	7	143	3	5	$3 \cdot 143 \cdot 5 = 2145$
2	5	11	91	3	4	$5 \cdot 91 \cdot 4 = 1820$
3	7	13	77	12	12	$7 \cdot 77 \cdot 12 = 6468$

Inverses by extended Euclid (columns $q, r_1, r_2, r, t_1, t_2, t$):

$3^{-1} \bmod 7$

q	r_1	r_2	r	t_1	t_2	t
2	7	3	1	0	1	−2
3	3	1	0	1	−2	7
	1	0		**−2**	7	

$3^{-1} \bmod 11$

q	r_1	r_2	r	t_1	t_2	t
3	11	3	2	0	1	−3
1	3	2	1	1	−3	4
2	2	1	0	−3	4	−11
	1	0		**4**	−11	

So $143^{-1} \equiv 3^{-1} \equiv -2 \equiv \mathbf{5} \pmod 7$ (check $3 \cdot 5 = 15 \equiv 1$) and $91^{-1} \equiv 3^{-1} \equiv \mathbf{4} \pmod{11}$ (check $3 \cdot 4 = 12 \equiv 1$). For the third, $77 \equiv 12 \equiv -1 \pmod{13}$, and $(-1)^{-1} = -1$, so $77^{-1} \equiv \mathbf{12} \pmod{13}$.
Step 4. $x = (2145 + 1820 + 6468) \bmod 1001 = 10433 \bmod 1001 = \mathbf{423}$.
Check. $423 = 60 \cdot 7 + 3$, $423 = 38 \cdot 11 + 5$, $423 = 32 \cdot 13 + 7$ ✓.

Trap: inverses modulo the wrong number

M_i^{-1} is the inverse modulo m_i (the small modulus), not modulo M. Reduce M_i modulo m_i before running extended Euclid, but multiply the *unreduced* M_i in step 4; using $M_i \bmod m_i$ in the final sum breaks the zeros modulo the other moduli.

Congruences with a coefficient, $c\,x \equiv b \pmod m$**.** The CRT formula needs every congruence in the form $x \equiv a_i \pmod{m_i}$, so each one is first solved for x. If $\gcd(c, m) = 1$, multiply both sides by $c^{-1} \bmod m$; the modulus does not change. If $d = \gcd(c, m) > 1$, the congruence has a solution only when $d \mid b$, and then dividing *everything* by d gives $\frac{c}{d}x \equiv \frac{b}{d} \pmod{\frac{m}{d}}$: the modulus shrinks to m/d, because $m \mid c(x - x')$ only forces $\frac{m}{d} \mid (x - x')$. Dividing c and b by d but keeping the old modulus m silently throws away $d - 1$ of the d solutions modulo m.

WORKED EXAMPLE : CRT after reducing the coefficients (video example 2, corrected)

Q. Solve $4x \equiv 5 \pmod 9$ and $2x \equiv 6 \pmod{20}$.
First congruence. $\gcd(4, 9) = 1$ and $4^{-1} \bmod 9 = 7$ (since $4 \cdot 7 = 28 = 3 \cdot 9 + 1$). Multiplying by 7: $x \equiv 35 \equiv 8 \pmod 9$.
Second congruence. $\gcd(2, 20) = 2$ and $2 \mid 6$, so it is solvable; dividing by 2 gives $x \equiv 3 \pmod{10}$, which modulo 20 means $x \equiv 3$ *or* $13 \pmod{20}$.
CRT on $x \equiv 8 \pmod 9$**,** $x \equiv 3 \pmod{10}$**.** $\gcd(9, 10) = 1$, $M = 90$.

i	a_i	m_i	M_i	$M_i \bmod m_i$	$M_i^{-1} \bmod m_i$	$a_i M_i M_i^{-1}$
1	8	9	10	1	1	$8 \cdot 10 \cdot 1 = 80$
2	3	10	9	9	9	$3 \cdot 9 \cdot 9 = 243$

$x = (80 + 243) \bmod 90 = 323 \bmod 90 = \mathbf{53}$, so the complete solution is $\mathbf{x \equiv 53 \pmod{90}}$, i.e. $x \in \{53, 143\}$ modulo 180.
Check. $4 \cdot 53 = 212 = 23 \cdot 9 + 5$ and $2 \cdot 53 = 106 = 5 \cdot 20 + 6$ ✓; $4 \cdot 143 = 572 = 63 \cdot 9 + 5$ and $2 \cdot 143 = 286 = 14 \cdot 20 + 6$ ✓.
The video kept modulus 20 after dividing ($x \equiv 3 \pmod{20}$) and used $M = 180$, $M_1^{-1} = 20^{-1} \bmod 9 = 5$, $M_2^{-1} = 9^{-1} \bmod 20 = 9$, getting $x = 1043 \bmod 180 = 143$. That value is correct but it is only one of the two solutions modulo 180; the branch $x \equiv 13 \pmod{20}$ gives the other, 53.

More slide examples (all checked):

System	M	M_i	M_i^{-1}	x
$x \equiv 3\ (7),\ 3\ (13),\ 0\ (12)$	1092	156, 84, 91	4, 11, 7	**276**
$S \equiv 1\ (11),\ 12\ (13),\ 2\ (17)$	2431	221, 187, 143	1, 8, 5	**155**
$z \equiv 61\ (99),\ 65\ (98),\ 69\ (97)$	941094	9506, 9603, 9702	50, 97, 49	**457**

Applications.

- *Word problems.* "Find an integer that leaves remainder 3 when divided by 7 and by 13 but is divisible by 12" is the first row above: $x = 276$ ($276 = 12 \times 23$, $276 = 39 \cdot 7 + 3 = 21 \cdot 13 + 3$).
- *Secret sharing.* The secret S is handed out as residues modulo 11, 13, 17, 19: shares 1, 12, 2, 3. Any 3 shares determine S uniquely modulo the product of their moduli, which exceeds S, so CRT recovers it; every one of the four 3-subsets gives $S = 155$ (the second row uses $\{1, 12, 2\}$).
- *Arithmetic on big numbers with small registers.* To compute $z = 123 + 334$ on a machine that holds only numbers below 100, represent $123 \equiv 24, 25, 26$ and $334 \equiv 37, 40, 43$ modulo $99, 98, 97$; add componentwise to get $z \equiv 61, 65, 69$; CRT gives $z = 457$, correct because $457 < M = 941094$ and the solution is unique modulo M.
- *RSA decryption speed-up.* Since $n = pq$ with p, q coprime, the plaintext $P = C^d \bmod n$ is determined by $P \bmod p$ and $P \bmod q$. These are computed with exponents reduced by Fermat, $d_p = d \bmod (p-1)$ and $d_q = d \bmod (q-1)$, on numbers half as long, and are then combined by CRT. A modular multiplication on half-length numbers costs about a quarter as much and the exponents are half as long, so the two half-size powers together are roughly four times faster than one full-size power.

WORKED EXAMPLE : RSA decryption with CRT (new)

Q. In RSA with $p = 7$, $q = 11$, $n = 77$, $e = 13$, $d = 37$, decrypt $C = 41$ using the CRT.
Reduced exponents. $d_p = 37 \bmod 6 = 1$, $d_q = 37 \bmod 10 = 7$.
Half-size powers. $P_p = 41^1 \bmod 7 = 6$.

$P_q = 41^7 \bmod 11 = 8^7 \bmod 11$: $8^2 = 64 \equiv 9$, $8^4 \equiv 81 \equiv 4$, $8^7 = 8^4 \cdot 8^2 \cdot 8 \equiv 4 \cdot 9 \cdot 8 = 288 \equiv 2$.
CRT. Solve $P \equiv 6 \pmod 7$, $P \equiv 2 \pmod{11}$ with $M = 77$:
$M_1 = 11 \equiv 4 \pmod 7$, $4^{-1} \bmod 7 = 2$; $M_2 = 7$, $7^{-1} \bmod 11 = 8$ ($7 \cdot 8 = 56 = 5 \cdot 11 + 1$).
$P = (6 \cdot 11 \cdot 2 + 2 \cdot 7 \cdot 8) \bmod 77 = (132 + 112) \bmod 77 = 244 \bmod 77 = \mathbf{13}$.
Check. Directly, $41^{37} \bmod 77 = 13$, and re-encrypting, $13^{13} \bmod 77 = 41$ ✓.

27.6 Fast exponentiation: square-and-multiply ★

▸ **Likely this year:** Needed inside every RSA / ElGamal / Diffie–Hellman numerical: show the bit table for $a^x \bmod n$.

Problem. Public-key cryptography computes $y = a^x \bmod n$ with x and n of 1024 bits or more. Multiplying by a again and again needs $x - 1$ multiplications, about 2^{1024}, which is impossible. (The size of intermediate results is the smaller problem: reducing modulo n after every multiplication keeps them below n^2.)

Idea. Write the exponent in binary, $x = x_{n_b-1}2^{n_b-1} + \cdots + x_1 2^1 + x_0 2^0$ with each $x_i \in \{0, 1\}$. Then

$$a^x = \prod_{i\,:\,x_i=1} a^{2^i}, \qquad \text{e.g.} \quad a^{22} = a^{16} \cdot a^4 \cdot a^2 \quad (22 = 10110_2).$$

The powers $a, a^2, a^4, a^8, \ldots$ are each the square of the previous one, so they cost one squaring each, and we multiply into the result only those whose bit is 1.

Mechanism (Forouzan, right-to-left). Start with $y = 1$. Scan the bits from x_0 (least significant) upwards: if $x_i = 1$, set $y \leftarrow a \times y \bmod n$; then square, $a \leftarrow a^2 \bmod n$ (not needed after the last bit). Return y. An n_b-bit exponent needs at most $n_b - 1$ squarings and n_b multiplications, so the cost is $O(n_b)$ modular multiplications, linear in the number of bits instead of exponential. A 1024-bit exponent needs at most about 2048 multiplications. The public exponent $e = 65537 = 10000000000000001_2$ needs 16 squarings and a single non-trivial multiplication, which is why it is the usual choice.

$17^{22} \bmod 21$ **(slide example),** $22 = 10110_2$

i	x_i	Multiplication (init. $y = 1$)	Squaring (init. $a = 17$)
0	0	–	$a = 17^2 \bmod 21 = 16$
1	1	$y = 1 \times 16 \bmod 21 = 16$	$a = 16^2 \bmod 21 = 4$
2	1	$y = 16 \times 4 \bmod 21 = 1$	$a = 4^2 \bmod 21 = 16$
3	0	–	$a = 16^2 \bmod 21 = 4$
4	1	$y = 1 \times 4 \bmod 21 = \mathbf{4}$	(not needed)

So $17^{22} \bmod 21 = 4$, using 4 squarings and 3 multiplications instead of 21 multiplications. Section 22 computes the same kind of power left-to-right (scanning from the most significant bit, square then multiply by a); both give the same answer, and either is acceptable in the exam as long as every intermediate value is shown.

WORKED EXAMPLE : RSA encryption and decryption by square-and-multiply (Forouzan numbers)

Q. In RSA, $n = 77$, $e = 13$, $d = 37$. Encrypt $M = 5$ and verify by decrypting, using the square-and-multiply method.
Encryption $C = 5^{13} \bmod 77$, $13 = 1101_2$ ($x_0 = 1, x_1 = 0, x_2 = 1, x_3 = 1$):

i	x_i	Multiplication ($y = 1$)	Squaring ($a = 5$)
0	1	$y = 1 \times 5 = 5$	$a = 5^2 = 25$
1	0	–	$a = 25^2 = 625 \bmod 77 = 9$
2	1	$y = 5 \times 9 = 45$	$a = 9^2 = 81 \bmod 77 = 4$
3	1	$y = 45 \times 4 = 180 \bmod 77 = \mathbf{26}$	(not needed)

Decryption $M = 26^{37} \bmod 77$, $37 = 100101_2$:

i	x_i	Multiplication ($y = 1$)	Squaring ($a = 26$)
0	1	$y = 26$	$a = 26^2 = 676 \bmod 77 = 60$
1	0	–	$a = 60^2 = 3600 \bmod 77 = 58$
2	1	$y = 26 \times 58 = 1508 \bmod 77 = 45$	$a = 58^2 = 3364 \bmod 77 = 53$
3	0	–	$a = 53^2 = 2809 \bmod 77 = 37$
4	0	–	$a = 37^2 = 1369 \bmod 77 = 60$
5	1	$y = 45 \times 60 = 2700 \bmod 77 = \mathbf{5}$	(not needed)

Answer. $C = 26$, and decryption returns $M = 5$ ✓.

27.7 Discrete logarithms ▸

▸ **Likely this year:** "Find x such that $3^x \equiv 7 \pmod{17}$" via a table of powers, using the log rules; or theory: why the discrete log is hard and which systems rely on it.

Problem. Square-and-multiply computes $y = a^x \bmod n$ quickly. The reverse question is: given a, y and n, find x. Over the real numbers this is $\log_a y$; modulo n it is the *discrete logarithm*, and no comparably fast method is known for it.

Primitive roots (reminder). The order of a in $\mathbb{Z}_n^*$ is the smallest $i \geq 1$ with $a^i \equiv 1$, and a is a *primitive root* when its order equals $\phi(n)$; details and examples are in §26. For a prime p, $\mathbb{Z}_p^*$ always has primitive roots, and if g is one then $g^1, g^2, \ldots, g^{p-1}$ are all different and run through every element $1, \ldots, p-1$ exactly once. Hence every $y \in \mathbb{Z}_p^*$ equals g^x for exactly one $x \in \{1, \ldots, p-1\}$; this x is the *discrete logarithm* of y to the base g, written $x = L_g y$ (the modulus is understood). In $\mathbb{Z}_7^*$ the primitive roots are 3 and 5, giving two logarithm tables:

y	1	2	3	4	5	6
$x = L_3 y \pmod 7$	6	2	1	4	5	3
$x = L_5 y \pmod 7$	6	4	5	2	1	3

With the table, $4 \equiv 3^x \pmod 7$ gives $x = L_3 4 = 4$ ($3^4 = 81 \equiv 4$), and $6 \equiv 5^x \pmod 7$ gives $x = L_5 6 = 3$ ($5^3 = 125 \equiv 6$).

Rules. Since $g^{p-1} \equiv 1$, exponents of g only matter modulo $p - 1 = \phi(p)$, so discrete logs obey the usual logarithm laws with arithmetic modulo $\phi(n)$:

$$L_g 1 \equiv 0, \qquad L_g(x \cdot y) \equiv L_g x + L_g y, \qquad L_g(x^k) \equiv k \cdot L_g x \pmod{\phi(n)}.$$

The second rule follows from $g^{Lx} \cdot g^{Ly} = g^{Lx+Ly}$. (The table writes $L_g 1 = 6$ rather than 0, because it lists exponents from 1 to $p-1$; $6 \equiv 0 \pmod 6$.) Example in $\mathbb{Z}_7^*$: $L_3(4 \cdot 5) = L_3 6 = 3$, and $L_3 4 + L_3 5 = 4 + 5 = 9 \equiv 3 \pmod 6$ ✓.

Edge case: base not a primitive root. Then the powers of a do not cover the whole group, so $a^x \equiv y$ may have no solution or several. In $\mathbb{Z}_7^*$, 2 has order 3 (2^x cycles through 2, 4, 1), so $2^x \equiv 3 \pmod 7$ has no solution, while $2^x \equiv 4$ has $x = 2$ and $x = 5$.

Why it is hard. The only general method on the slides is *exhaustive search*: for $x = 1, 2, \ldots, n-1$, test whether $y = a^x \bmod n$ and return the first x that works. That is up to $p - 1$ exponentiations, exponential in the number of bits, while computing a^x needs only $O(\log p)$ multiplications. Better algorithms exist (baby-step giant-step in about $\sqrt{p}$ steps; index calculus, sub-exponential), but for a well-chosen 2048-bit prime they are still infeasible, roughly as hard as factoring a 2048-bit RSA modulus. Modular exponentiation is therefore a *one-way function*, and this asymmetry is the basis of **Diffie–Hellman** (both parties publish g^a and g^b and share g^{ab}; an eavesdropper would need $a = L_g(g^a)$) and of **ElGamal** (the private key d hides behind the public $e_2 = e_1^d \bmod p$). For example, with $p = 17$, $g = 3$, $a = 4$, $b = 7$: the public values are $3^4 \equiv 13$ and $3^7 \equiv 11$, and the shared key is $11^4 \equiv 13^7 \equiv 4 \pmod{17}$.

WORKED EXAMPLE : discrete logarithms modulo 17 (new)

Q. (a) Show that 3 is a primitive root of 17 and tabulate $L_3 y$ for $\mathbb{Z}_{17}^*$. (b) Solve $3^x \equiv 7 \pmod{17}$. (c) Using only $L_3 2$ and $L_3 7$ and the rules, find $L_3 14$. (d) Solve $3^x \equiv 8 \pmod{17}$ from $L_3 2$.

(a) Successive powers $3^x \bmod 17$, each 3× the previous:

x	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
$3^x \bmod 17$	3	9	10	13	5	15	11	16	14	8	7	4	12	2	6	1

The first power equal to 1 is 3^{16}, so $\operatorname{ord}(3) = 16 = \phi(17)$ and 3 is a primitive root (all 16 values differ). Reading the table backwards:

y	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
$L_3 y$	16	14	1	12	5	15	11	10	2	3	7	13	4	9	6	8

(b) $x = L_3 7 =$ **11**. Check: $3^{11} \bmod 17 = 7$ (from the power table).
(c) $L_3 14 = L_3(2 \cdot 7) \equiv L_3 2 + L_3 7 = 14 + 11 = 25 \equiv$ **9** $\pmod{16}$. Check: $3^9 \bmod 17 = 14$ ✓.
(d) $L_3 8 = L_3(2^3) \equiv 3 \cdot L_3 2 = 42 \equiv$ **10** $\pmod{16}$. Check: $3^{10} \bmod 17 = 8$ ✓.

Corrections to the notes / slides

- Slide (Fermat test): "561 = 33 × 17" is true but is not the prime factorisation; 561 = 3 × 11 × 17, the smallest Carmichael number.
- Slide (AKS): the example computes $(\log_2 200)^{12} \approx 3.9 \times 10^{10}$ bit operations for a 200-bit n (the printed 39,547,615,483 is not even that value exactly; it is ≈ 39,787,815,242). For a 200-bit number $\log_2 n = 200$, so the bound $(\log_2 n)^{12}$ is $200^{12} \approx 4.1 \times 10^{27}$: still polynomial, but not "40 seconds".
- Slide (recommended test, 4033): the list of trial divisors "2, 3, 5, 7, 11, 17, 23" skips 13 and 19 (neither divides 4033). A full divisibility test up to $\lfloor\sqrt{4033}\rfloor = 63$ would find 37 and settle the question; the recommended test uses trial division by *small* primes only.
- Slide (Fermat test): "If x is a prime, then $a^{n-1} \equiv 1$" should read "if n is a prime".
- Slide $\phi(1) = 0$ follows Forouzan's "integers smaller than n"; the standard convention is $\phi(1) = 1$. It never matters in RSA. (Totient chapter.)
- Euler's second version $a^{k\phi(n)+1} \equiv a \pmod{n}$ holds for squarefree n (such as $n = pq$ in RSA), not for every n; it fails for $n = 4$, $a = 2$. (See §26.)

28 Algebraic Structures: Groups, Rings, Fields ★★

Every cipher so far has done arithmetic on a set of integers: the additive cipher adds mod 26, and the multiplicative cipher multiplies mod 26 and needs an inverse to decrypt. Modern ciphers such as AES work on bytes, and they need to add, multiply and divide bytes without ever leaving the set of bytes. A set together with the operations defined on it is called an **algebraic structure**. Three of them matter here: a **group** has one operation that can always be undone, a **ring** has two operations of which only the first can always be undone, and a **field** has two operations that can both be undone (except division by zero). This section builds up to the finite fields GF(p) and GF(2^n), because AES does all its byte arithmetic in GF(2^8), and the GF(2^8) inverse was asked in both past papers (8 marks in total).

28.1 Groups

A **group** $G = \langle S, \bullet \rangle$ is a set S with one binary operation $\bullet$ (a rule that combines any two elements into a third) satisfying four properties:

1. **Closure**: if $a, b \in G$, then $a \bullet b \in G$.
2. **Associativity**: $(a \bullet b) \bullet c = a \bullet (b \bullet c)$ for all a, b, c.
3. **Identity**: there is an element e with $e \bullet a = a \bullet e = a$ for every a.
4. **Inverse**: every a has an element a' with $a \bullet a' = a' \bullet a = e$.

If in addition $a \bullet b = b \bullet a$ for all a, b (**commutativity**), the group is called **abelian** or commutative.

The inverse property is the reason groups matter in cryptography: it gives a second operation for free. "Undo b" is just $a \bullet b'$. In a group under addition, b' is $-b$ and the derived operation is subtraction; in a group under multiplication, b' is b^{-1} and the derived operation is division. A group therefore supports one *pair* of operations, $(+, -)$ or $(\times, \div)$, which is exactly the shape of an encryption/decryption pair.

$\langle \mathbb{Z}_n, + \rangle$ is an abelian group. Closure holds because the result is reduced mod n and so lands back in $\mathbb{Z}_n = \{0, \ldots, n-1\}$; associativity and commutativity are inherited from ordinary addition; the identity is 0; and the inverse of a is $n - a$, since $a + (n - a) = n \equiv 0$. For example, in $\mathbb{Z}_{10}$ the inverse of 3 is 7, so $4 - 3$ is computed as $4 + 7 = 11 \equiv 1$.

$\langle \mathbb{Z}_n^*, \times \rangle$ is an abelian group. $\mathbb{Z}_n^*$ is the set of residues that are coprime to n, for example $\mathbb{Z}_{10}^* = \{1, 3, 7, 9\}$. Closure holds because a product of two numbers coprime to n is again coprime to n; the identity is 1; and every element has an inverse because $\gcd(a, n) = 1$ lets the extended Euclidean algorithm (Section 24) find s, t with $sn + ta = 1$, i.e. $ta \equiv 1 \pmod{n}$. Plain $\mathbb{Z}_n$ under $\times$ is *not* a group: 0 has no inverse, and neither does any a sharing a factor with n (in $\mathbb{Z}_{10}$, $2y$ is always even and so never ≡ 1). Removing exactly these elements is what the star in $\mathbb{Z}_n^*$ does.

Reading an operation table. For a small set, the operation can be given as a table whose entry in row x, column y is $x \bullet y$. Closure means every entry belongs to the set. The identity is the element whose row repeats the column header. The inverse of x is the column in which the identity appears in row x. Commutativity means the table is symmetric about its main diagonal. Associativity cannot be seen at a glance and has to be checked on triples.

•	a	b	c	d
a	a	b	c	d
b	b	c	d	a
c	c	d	a	b
d	d	a	b	c

In the slide's table on $\{a, b, c, d\}$, every entry is in the set, so closure holds. Row a copies the header, so a is the identity. The identity appears (shaded) in row b under d, in row c under c and in row d under b, giving the inverse pairs (a, a), (b, d), (c, c). The table is symmetric, so the group is abelian, and all 64 triples are associative (e.g. $(a \bullet b) \bullet c = a \bullet (b \bullet c) = d$). It is in fact $\langle \mathbb{Z}_4, + \rangle$ with $a, b, c, d = 0, 1, 2, 3$.

Permutation group. The set is all $3! = 6$ permutations of three positions, and the operation $\circ$ is **composition**: apply one permutation, then the other. Write $[a\ b\ c]$ for the permutation sending $1 \to a$, $2 \to b$, $3 \to c$. Composition is read right to left, so $[3\ 1\ 2] \circ [1\ 3\ 2]$ means "first [1 3 2], then [3 1 2]": $1 \to 1 \to 3$, $2 \to 3 \to 2$, $3 \to 2 \to 1$, which

gives [3 2 1] (the slide's example). Closure holds because two rearrangements in a row are again a rearrangement; composition of functions is associative; the identity is [1 2 3]; and every permutation can be undone, so it has an inverse. The group is **not abelian**: [1 3 2] ∘ [2 1 3] = [3 1 2], but [2 1 3] ∘ [1 3 2] = [2 3 1].

Why a double transposition is no stronger. A keyed transposition cipher with block length k encrypts each block by applying a key permutation π. Applying a second transposition π_2 (same block length) after π_1 is, by closure, the same as applying the single permutation $\pi_2 \circ \pi_1$, which is again one of the $k!$ possible keys. The attacker never needs to find π_1 and π_2 separately: recovering the one equivalent permutation breaks the cipher, and the key space is still $k!$. The same closure argument shows that two additive (shift) ciphers in a row are just one shift cipher.

28.2 Order, subgroups and cyclic subgroups

A group is **finite** if its set has finitely many elements, and its **order** $|G|$ is that number of elements: $|\mathbb{Z}_6| = 6$, $|\mathbb{Z}_{10}^*| = 4$.

A **subgroup** H of G is a subset of G that is itself a group under the *same* operation as G. It follows that the two share the same identity, that $a \bullet b$ of two members of H stays in H, and that the inverse of a member of H is in H. The trivial group $\langle\{e\}, \bullet\rangle$ and G itself are always subgroups of G. "Same operation" is the condition that catches students: $\mathbb{Z}_{10}$ is a subset of $\mathbb{Z}_{12}$, but $\langle\mathbb{Z}_{10}, +\rangle$ is **not** a subgroup of $\langle\mathbb{Z}_{12}, +\rangle$, because one adds mod 10 and the other mod 12 (e.g. 7 + 5 is 2 in one and 0 in the other).

Cyclic subgroups. In a group, the power a^k means $a \bullet a \bullet \ldots \bullet a$ (k times), with $a^0 = e$; in an additive group, a^k therefore means $k \cdot a \bmod n$. In a finite group, the powers $a^0, a^1, a^2, \ldots$ cannot all be different, so they eventually return to e and then repeat. The set of powers is closed under the operation and contains each member's inverse, so it is a subgroup, called the **cyclic subgroup generated by** a. If it is the whole of G, then G is a **cyclic group** and a is a **generator**.

	$\langle\mathbb{Z}_6, +\rangle$ (powers = $k \cdot a \bmod 6$)		$\langle\mathbb{Z}_{10}^*, \times\rangle$ (powers = $a^k \bmod 10$)		
a	generated subgroup	ord(a)	a	$a^0, a^1, a^2, \ldots$	ord(a)
0	{0}	1	1	1	1
1	{0, 1, 2, 3, 4, 5} = G	6	3	1, 3, 9, 7 (= G)	4
2	{0, 2, 4}	3	7	1, 7, 9, 3 (= G)	4
3	{0, 3}	2	9	1, 9	2
4	{0, 4, 2}	3			
5	{0, 5, 4, 3, 2, 1} = G	6			

So $\langle\mathbb{Z}_6, +\rangle$ has four cyclic subgroups, {0}, {0, 2, 4}, {0, 3} and G, and is cyclic with generators 1 and 5. $\langle\mathbb{Z}_{10}^*, \times\rangle$ has three, {1}, {1, 9} and G, and is cyclic with generators 3 and 7.

Lagrange's theorem and the order of an element. If H is a subgroup of a finite group G, then $|H|$ divides $|G|$. The table agrees: the subgroup sizes 1, 2, 3, 6 all divide 6, and 1, 2, 4 divide 4. The **order of an element**, ord(a), is the order of the cyclic subgroup it generates, which is the smallest $k \geq 1$ with $a^k = e$. Applying Lagrange to that subgroup shows ord(a) divides $|G|$, and therefore $a^{|G|} = e$ for every a. With $G = \mathbb{Z}_n^*$ and $|G| = \phi(n)$ this is exactly Euler's theorem $a^{\phi(n)} \equiv 1 \pmod{n}$ (Section 26). Lagrange also rules out orders: $\mathbb{Z}_7^*$ has 6 elements, so no element can have order 4 or 5 (the actual orders of $1, \ldots, 6$ are $1, 3, 6, 3, 6, 2$).

28.3 Rings and fields

A **ring** $R = \langle\{\ldots\}, +, \times\rangle$ has two operations. The first must satisfy all five properties (it is an abelian group), the second only closure and associativity, and the second must **distribute** over the first: $a \times (b + c) = (a \times b) + (a \times c)$ and $(a + b) \times c = (a \times c) + (b \times c)$. If the second operation is also commutative, the ring is a **commutative ring**. $\langle\mathbb{Z}, +, \times\rangle$ is a commutative ring: addition satisfies all five properties, multiplication satisfies closure, associativity and commutativity, and $5 \times (3 + 2) = (5 \times 3) + (5 \times 2) = 25$. A ring supports $(+, -)$ and $\times$, but not division: $12 \div 5 = 2.4$ is not in $\mathbb{Z}$, so most elements of $\mathbb{Z}$ have no multiplicative inverse.

A **field** $F = \langle\{\ldots\}, +, \times\rangle$ is a commutative ring in which the second operation also satisfies all five properties, except that the identity of the first operation (0) has no inverse under the second. A field therefore supports both pairs, $(+, -)$ and $(\times, \div)$, with the single exception of division by 0. The exception cannot be avoided: distributivity gives $0 \times a = (0 + 0) \times a = 0 \times a + 0 \times a$, so $0 \times a = 0$ for every a, and nothing multiplies 0 into 1.

★ MUST KNOW : which properties each structure needs

Structure	First operation (+)	Second operation (×)	Supports	Typical set
Group	closure, associativity, identity, inverse (+ commutativity: abelian)	–	(+, −) or (×, ÷)	$\mathbb{Z}_n$; $\mathbb{Z}_n^*$
Ring	all 5	closure, associativity (+ commutativity: commutative ring); distributes over +	(+, −) and ×	$\mathbb{Z}$
Field	all 5	all 5, except 0 has no inverse; distributes over +	(+, −) and (×, ÷)	$\mathbb{Z}_p$, GF(2^n)

28.4 Finite fields: why the size must be p^n

Zero divisors kill inverses. Suppose $a \times b = 0$ with $a \neq 0$ and $b \neq 0$. If a had an inverse a^{-1}, then $b = a^{-1} \times (a \times b) = a^{-1} \times 0 = 0$, a contradiction; so a has no inverse. This one argument explains every "not a field" result in this chapter. For a composite $n = ab$ with $1 < a, b < n$, we get $a \times b \equiv 0 \pmod{n}$, so $\mathbb{Z}_n$ is only a commutative ring. For a prime p no such factorisation exists, every non-zero element is coprime to p and has an inverse, and $\mathbb{Z}_p$ is a field.

Galois' result. A finite field must have p^n elements, p prime and $n \geq 1$; it is written GF(p^n) (Galois field). The reason is short. Add 1 to itself repeatedly: $1, 1+1, 1+1+1, \ldots$. The field is finite, so some sum $k \cdot 1$ must equal 0; let p be the smallest such k. This p must be prime, because $p = ab$ with $a, b < p$ would give $(a \cdot 1)(b \cdot 1) = 0$ with both factors non-zero, which the zero-divisor argument forbids. So the field contains a copy of $\mathbb{Z}_p$. Every element can then be written uniquely as $a_0 e_0 + a_1 e_1 + \cdots + a_{n-1} e_{n-1}$ with coefficients $a_i \in \mathbb{Z}_p$ for some fixed basis $e_0, \ldots, e_{n-1}$ (the field is an n-dimensional vector space over $\mathbb{Z}_p$). There are p choices for each of the n coefficients, giving p^n elements. For GF(2^n) the basis will be $1, x, \ldots, x^{n-1}$ and the coefficients will be bits, which is exactly the polynomial representation used below.

28.5 GF(p) arithmetic and division ★

★ In the papers: 2024–25 Q5(i): $5 \div 3$ in GF(7).

GF(p) is the set $\mathbb{Z}_p = \{0, 1, \ldots, p-1\}$ with addition and multiplication mod p. Subtraction is addition of the additive inverse, $a - b = a + (p - b)$, and division is multiplication by the multiplicative inverse, $a \div b = a \times b^{-1}$, which exists for every $b \neq 0$ because p is prime. The inverse is found with the extended Euclidean algorithm, by inspection for small p (look for $b \cdot y \equiv 1$), or by Fermat as $b^{-1} \equiv b^{p-2}$.

The smallest field, **GF(2)** = {0, 1}, is the one every bit lives in: addition and subtraction are both **XOR** ($1 + 1 = 0$, and $-1 = 1$), multiplication is **AND**, and 1 is its own inverse under both operations. In **GF(5)** the additive inverse pairs are $(1, 4), (2, 3)$ and the multiplicative ones $(1, 1), (2, 3), (4, 4)$; in **GF(7)** they are $(1, 6), (2, 5), (3, 4)$ and $(1, 1), (2, 4), (3, 5), (6, 6)$.

WORKED EXAMPLE : 2024–25 Q5(i), "In the field GF(7), find the result of $5 \div 3$."

Division in a field means multiplication by the inverse: $5 \div 3 = 5 \times 3^{-1} \bmod 7$.
Inverse of 3. Extended Euclid with $r_1 = 7$, $r_2 = 3$:

q	r_1	r_2	r	t_1	t_2	t
2	7	3	1	0	1	−2
3	3	1	0	1	−2	7
	1	0		**−2**	7	

gcd = 1, so the inverse exists: $3^{-1} \equiv -2 \equiv 5 \pmod 7$ (check $3 \times 5 = 15 = 2 \cdot 7 + 1$; Fermat agrees, $3^5 = 243 \equiv 5$).
Divide. $5 \div 3 = 5 \times 5 = 25 = 3 \cdot 7 + 4 \equiv$ **4**.
Check: $4 \times 3 = 12 \equiv 5 \pmod 7$ ✓.
A second one in the same style: in GF(13), $4 \div 9 = 4 \times 9^{-1} = 4 \times 3 =$ **12**, since $9 \times 3 = 27 \equiv 1$; check $12 \times 9 = 108 = 8 \cdot 13 + 4$ ✓.

Trap: "÷" is not integer division

$5 \div 3$ in GF(7) is not 1.67 and not 1; it is the element y with $3y \equiv 5$. The same past-paper question then asked for 15^{-1} mod 99, which does not exist because $\gcd(15, 99) = 3$: $\mathbb{Z}_{99}$ is not a field, so the inverse is only guaranteed in GF(p).

28.6 Why GF(2^n) instead of $\mathbb{Z}_{2^n}$

Computers store numbers as n-bit words ($n = 8, 16, 32, \ldots$), so the natural set is $\{0, \ldots, 2^n - 1\}$ with modulus 2^n. But 2^n is not prime, so $\mathbb{Z}_{2^n}$ has zero divisors and is not a field. In $\mathbb{Z}_{256}$, for example, $2 \times 128 = 256 \equiv 0$, and indeed no even byte has an inverse, because $2y$ stays even mod 256 and can never be 1. Half of all bytes could not be divided by.

There are two ways out. The first is to use GF(p) with p the largest prime below 2^n. This works but wastes values: for $n = 4$ the prime is 13, so 13, 14 and 15 are unusable, and for $n = 8$ it is 251, so 251–255 are unusable; an arbitrary byte of data could not even be represented. The second, which AES uses, is to keep all 2^n words and *redefine* addition and multiplication so that the field properties hold. Galois guarantees that a field of size 2^n exists ($p = 2$), and it is built from polynomials.

28.7 Polynomial representation and addition

An n-bit word $a_{n-1} \ldots a_1 a_0$ stands for the polynomial $a_{n-1}x^{n-1} + \cdots + a_1 x + a_0$, whose coefficients are bits, i.e. elements of GF(2). The term x^i is the i-th term and the bit a_i is its coefficient, so the word records which powers of x are present. For example, `10011001` $= x^7 + x^4 + x^3 + 1$. In the other direction, $x^5 + x^2 + x$ as an 8-bit word is found by supplying the missing terms, $0x^7 + 0x^6 + 1x^5 + 0x^4 + 0x^3 + 1x^2 + 1x + 0$, which is `00100110`. There are therefore two levels of arithmetic: the coefficients are combined in GF(2), and the polynomials are combined modulo a fixed polynomial in GF(2^n).

Addition and subtraction add the coefficients of equal powers in GF(2), so $x^k + x^k = 0$. Because $-1 \equiv 1 \pmod 2$, we have $-B = B$, so $A - B = A + B$: addition and subtraction are the same operation. Adding two polynomials of degree at most $n - 1$ never raises the degree, so no reduction is needed. Since adding coefficients mod 2 bit by bit is exactly XOR, addition in GF(2^n) is the XOR of the two words, every element is its own additive inverse ($a \oplus a = 0$), and the identity is 0. Slide example: $(x^5 + x^2 + x) \oplus (x^3 + x^2 + 1) = x^5 + x^3 + x + 1$, because the two x^2 terms cancel; in bits, `00100110` $\oplus$ `00001101` $=$ `00101011`.

28.8 Irreducible polynomials: the modulus

Multiplying two polynomials of degree at most $n-1$ can produce degree up to $2n-2$, which no longer fits in n bits. So, as with integers mod p, the product is divided by a fixed polynomial $m(x)$ of degree n and only the remainder (degree $< n$) is kept. The modulus must be **irreducible**: it cannot be written as a product of two polynomials of lower degree (at least 1) with coefficients in GF(2). An irreducible polynomial plays the role of a prime. If instead $m = f \cdot g$, then $f \otimes g \equiv 0 \pmod m$ with $f, g \neq 0$, and the zero-divisor argument shows f has no inverse, so the structure would only be a ring.

For degree 2 or 3, a polynomial is reducible exactly when it has a factor of degree 1, i.e. a root in GF(2). It has the root 0 if it has no constant term, and it has the root 1 if it has an even number of terms. So $x^2 + 1 = (x + 1)^2$ is reducible (two terms), while $x^2 + x + 1$ (constant term, three terms) is irreducible. From degree 4 on, a product of two irreducible quadratics also has to be ruled out, e.g. $x^4 + x^2 + 1 = (x^2 + x + 1)^2$ has no root but is still reducible.

Degree	Irreducible polynomials over GF(2)
1	x, $x + 1$
2	$x^2 + x + 1$
3	$x^3 + x + 1$, $x^3 + x^2 + 1$
4	$x^4 + x + 1$, $x^4 + x^3 + 1$, $x^4 + x^3 + x^2 + x + 1$
5	$x^5 + x^2 + 1$, $x^5 + x^3 + 1$, $x^5 + x^3 + x^2 + x + 1$, $x^5 + x^4 + x^2 + x + 1$, $x^5 + x^4 + x^3 + x + 1$, $x^5 + x^4 + x^3 + x^2 + 1$

AES uses $m(x) = x^8 + x^4 + x^3 + x + 1$, the bit pattern `1 0001 1011` $=$ `0x11B`, one of the 30 irreducible polynomials of degree 8.

28.9 Polynomial long division over GF(2) ★

★ **In the papers:** Every row of the GF(2^8) inverse table (2024–25 Q4, 2025–26 Q2) is one such division, and so is the verification step. This is where marks are lost.

Long division of polynomials over GF(2) is ordinary long division in which every subtraction is an XOR. Terms of equal power cancel, there are no borrows and no negative coefficients, and no fractions appear because every leading coefficient is 1. The procedure is:

1. Write the dividend and the divisor in descending powers, and write the dividend in columns, one column per power, leaving gaps for missing powers.
2. Divide the leading term of the current dividend by the leading term of the divisor, $x^a / x^b = x^{a-b}$. This is the next term of the quotient.
3. Multiply the *whole* divisor by x^{a-b} (add $a - b$ to every exponent) and write it under the current line, each term

in the column of its power.

4. XOR the two lines: a power present in both lines cancels, and a power present in only one line drops down. The leading term always cancels.
5. Repeat on the new line while its degree is at least the degree of the divisor. The last line is the **remainder**, and the quotient is the sum of the terms found in step 2.

WORKED EXAMPLE : reduce $x^{12} + x^7 + x^2$ modulo $m(x) = x^8 + x^4 + x^3 + x + 1$ (Forouzan)

$$
\begin{array}{llllllllll}
x^{12} & & +x^7 & & & & +x^2 & & & \text{dividend} \\
x^{12} & +x^8 & +x^7 & +x^5 & +x^4 & & & & & \oplus\, x^4 \cdot m \\
\hline
 & x^8 & & +x^5 & +x^4 & & +x^2 & & & \\
 & x^8 & & & +x^4 & +x^3 & & +x & +1 & \oplus\, 1 \cdot m \\
\hline
 & & & x^5 & & +x^3 & +x^2 & +x & +1 & \text{remainder}
\end{array}
$$

Quotient $x^4 + 1$, **remainder** $x^5 + x^3 + x^2 + x + 1$. In the first step $x^{12}/x^8 = x^4$, so m is shifted up by 4; the x^{12} and x^7 terms cancel, and x^8, x^5, x^4 come down together with the untouched x^2. In the second step $x^8/x^8 = 1$.

The same division in bits (shift `100011011` under the leading 1 and XOR; a fast way to check): `1000010000100` ⊕ `1000110110000` = `100110100`; `100110100` ⊕ `100011011` = `000101111` = `0x2F`.

Trap: the usual slips in GF(2) division

Shifting only the leading term of the divisor instead of all of it; aligning terms by position on the page instead of by power; writing $x^k + x^k = 2x^k$ instead of 0; and stopping when the remainder is 0 rather than when its degree is below the divisor's.

28.10 Multiplication in GF(2^n) ★

▸ **Likely this year:** "Multiply {57} by {83}" or "find {02} · {87}" (Stallings/FIPS-197, needed for AES MixColumns), or Forouzan's $P_1 \otimes P_2$.

To multiply, expand the product term by term ($x^i \cdot x^j = x^{i+j}$, and coefficient products in GF(2) are AND), add all the partial products with XOR so that equal powers cancel in pairs, and then reduce modulo $m(x)$ by long division; the remainder is the answer.

WORKED EXAMPLE : $(x^5 + x^2 + x) \otimes (x^7 + x^4 + x^3 + x^2 + x)$ in GF(2^8), long method

Let $P_1 = x^5 + x^2 + x$ and $P_2 = x^7 + x^4 + x^3 + x^2 + x$. One partial product per term of P_1:

$$
\begin{aligned}
x^5 \cdot P_2 &= x^{12} + x^9 + x^8 + x^7 + x^6 \\
x^2 \cdot P_2 &= x^9 + x^6 + x^5 + x^4 + x^3 \\
x \cdot P_2 &= x^8 + x^5 + x^4 + x^3 + x^2
\end{aligned}
$$

XOR: $x^9, x^8, x^6, x^5, x^4, x^3$ each appear twice and cancel, leaving $x^{12} + x^7 + x^2$. Reducing this modulo $m(x)$ was the long division above, so $P_1 \otimes P_2 = \mathbf{x^5 + x^3 + x^2 + x + 1}$, i.e. {26} ⊗ {9E} = {2F}.

The shortcut: multiply by x repeatedly. Multiplying a polynomial by x raises every exponent by one, which is a left shift of the word by one bit. If the top bit (x^7) was 0, the result still has degree at most 7 and nothing else is needed. If the top bit was 1, the result contains x^8, which must be reduced. Since $m(x) = x^8 + (x^4 + x^3 + x + 1) \equiv 0$ and minus equals plus, $x^8 \equiv x^4 + x^3 + x + 1$, so the reduction is: drop the overflow bit and XOR with `00011011` = `0x1B`. This step is called **xtime**. Any product $a \otimes b$ is then the XOR of $a \otimes x^i$ over the 1-bits i of b, and $a \otimes x^i$ is obtained by applying xtime i times. Because every intermediate result is reduced straight away, no number ever exceeds 8 bits, which is why this is the version used in software and hardware.

WORKED EXAMPLE : the same product by the shortcut (slide table)

Build $x^i \otimes P_2$ for $i = 0, \ldots, 5$, each row being xtime of the row above:

Power	Operation	Result	Byte	Reduced?
$x^0 \otimes P_2$		$x^7 + x^4 + x^3 + x^2 + x$	9E	No
$x^1 \otimes P_2$	$x \otimes (x^7 + x^4 + x^3 + x^2 + x)$	$x^5 + x^2 + x + 1$	27	Yes
$x^2 \otimes P_2$	$x \otimes (x^5 + x^2 + x + 1)$	$x^6 + x^3 + x^2 + x$	4E	No
$x^3 \otimes P_2$	$x \otimes (x^6 + x^3 + x^2 + x)$	$x^7 + x^4 + x^3 + x^2$	9C	No
$x^4 \otimes P_2$	$x \otimes (x^7 + x^4 + x^3 + x^2)$	$x^5 + x + 1$	23	Yes
$x^5 \otimes P_2$	$x \otimes (x^5 + x + 1)$	$x^6 + x^2 + x$	46	No

Row 1 in bits: 10011110 shifted is 1 00111100, the top bit overflowed, so 00111100 ⊕ 00011011 = 00100111 (27). Row 4 likewise: 00111000 ⊕ 00011011 = 00100011 (23).
$P_1 = x^5 + x^2 + x$ uses rows 5, 2 and 1: $(x^6 + x^2 + x) \oplus (x^6 + x^3 + x^2 + x) \oplus (x^5 + x^2 + x + 1) = \mathbf{x^5 + x^3 + x^2 + x + 1}$, i.e. 46 ⊕ 4E ⊕ 27 = 2F ✓ (same as the long method).

WORKED EXAMPLE : {57} • {83} and {57} • {13} in GF(2^8) (Stallings / FIPS-197)

{57} = 01010111 = $x^6 + x^4 + x^2 + x + 1$. Repeated xtime (a 1 in the top bit means XOR 1B after the shift):

i	0	1	2	3	4	5	6	7
{57} • x^i	57	AE	47	8E	07	0E	1C	38
reduced?		no	yes	no	yes	no	no	no

(e.g. i = 2: AE = 10101110 → 01011100 ⊕ 00011011 = 01000111 = 47.)
{83} = 10000011 = $x^7 + x + 1$, so {57} • {83} = 38 ⊕ AE ⊕ 57 = **C1** = $x^7 + x^6 + 1$.
{13} = 00010011 = $x^4 + x + 1$, so {57} • {13} = 07 ⊕ AE ⊕ 57 = **FE**.
Check (long method for 57·83): the product is $x^{13} + x^{11} + x^9 + x^8 + x^6 + x^5 + x^4 + x^3 + 1$; dividing by $m(x)$ gives quotient $x^5 + x^3$ and remainder $x^7 + x^6 + 1$ ✓.
MixColumns use. AES only multiplies by 02 and 03: {02} • a = xtime(a) and {03} • a = xtime(a) ⊕ a. For example, {02} • {87}: 10000111 → 00001110 ⊕ 00011011 = 00010101 = **15**; and {03} • {6E} = DC ⊕ 6E = **B2** (Section 31).

28.11 Multiplicative inverse in GF(2^n): extended Euclid on polynomials ★★

★ **In the papers:** 2024–25 Q4 (4 marks): inverse of x^5; 2025–26 Q2 (4 marks): inverse of x^7, both mod $x^8 + x^4 + x^3 + x + 1$.

Division in GF(2^n) needs the inverse a^{-1} with $a \otimes a^{-1} \equiv 1 \pmod m$. The idea is the same as for integers. The extended Euclidean algorithm on $m(x)$ and $a(x)$ produces polynomials s, t with $s \cdot m + t \cdot a = \gcd(m, a)$. Since m is irreducible and a is a non-zero polynomial of smaller degree, the gcd is 1, so $t \cdot a = 1 + s \cdot m \equiv 1 \pmod m$, and t is the inverse. As with integer inverses, only the t column is needed.

The table is Forouzan's integer table with two changes: the quotient q comes from a polynomial long division, and every subtraction is an XOR. Start with $r_1 = m$, $r_2 = a$, $t_1 = 0$, $t_2 = 1$, and in each row compute

$$q, r = \text{quotient and remainder of } r_1 \div r_2, \qquad t = t_1 - q \cdot t_2 = t_1 \oplus (q \cdot t_2),$$

then shift $(r_1, r_2) \leftarrow (r_2, r)$ and $(t_1, t_2) \leftarrow (t_2, t)$. Stop when $r_2 = 0$. The product $q \cdot t_2$ is an ordinary polynomial product (expand, XOR equal powers); it needs no reduction, because the t values never reach degree n until the very last row.

★ MUST KNOW : GF(2^n) inverse recipe and its three checks

$r_1 = m$, $r_2 = a$, $t_1 = 0$, $t_2 = 1$; each row: q, r from $r_1 \div r_2$; $t = t_1 \oplus q \cdot t_2$; shift; stop at $r_2 = 0$. **Answer** = t_1 **of the final row.**
Checks: (1) the final r_1 must be 1; (2) the last t computed always equals m itself (the last row says $t \cdot a$ is a multiple of m, and a shares no factor with m); (3) multiply $a \otimes t_1$ and reduce by m: the remainder must be 1.

Small example (slide): $(x^2 + 1)^{-1}$ in GF(2^4) modulo $x^4 + x + 1$. The divisions are $(x^4 + x + 1) \div (x^2 + 1)$ and $(x^2 + 1) \div x$:

$x^4 + x + 1$	dividend
$x^4 + x^2$	$\oplus\, x^2 \cdot r_2$
$x^2 + x + 1$	
$x^2 + 1$	$\oplus\, 1 \cdot r_2$
x	remainder

$x^2 + 1$	dividend
x^2	$\oplus\, x \cdot r_2$
1	remainder

so the quotients are $x^2 + 1$, x, x (the last row is $x \div 1$). The t values are $0 \oplus (x^2 + 1) \cdot 1 = x^2 + 1$, then $1 \oplus x(x^2 + 1) = x^3 + x + 1$, then $(x^2 + 1) \oplus x(x^3 + x + 1) = x^4 + x + 1 = m$ ✓.

q	r_1	r_2	r	t_1	t_2	t
x^2+1	x^4+x+1	x^2+1	x	0	1	x^2+1
x	x^2+1	x	1	1	x^2+1	x^3+x+1
x	x	1	0	x^2+1	x^3+x+1	x^4+x+1
	1	0		**x^3+x+1**	x^4+x+1	

$(x^2 + 1)^{-1} = \mathbf{x^3 + x + 1}$. Check: $(x^2 + 1)(x^3 + x + 1) = x^5 + x^2 + x + 1$, and $x^5 + x^2 + x + 1 \oplus x \cdot m = 1$ ✓.

WORKED EXAMPLE : 2024–25 Q4, "In GF(2^8), find the inverse of (x^5) modulo ($x^8 + x^4 + x^3 + x + 1$)." (4 marks)

Setup. $r_1 = m = x^8 + x^4 + x^3 + x + 1$, $r_2 = x^5$, $t_1 = 0$, $t_2 = 1$.
Divisions (one per row of the table):

Row 1: $(x^8 + x^4 + x^3 + x + 1) \div x^5$

$x^8 + x^4 + x^3 + x + 1$		dividend
x^8		$\oplus\ x^3 \cdot r_2$
$x^4 + x^3 + x + 1$		remainder

$q = x^3$, $r = x^4 + x^3 + x + 1$.

Row 2: $x^5 \div (x^4 + x^3 + x + 1)$

x^5		dividend
$x^5 + x^4$	$+ x^2 + x$	$\oplus\ x \cdot r_2$
x^4	$+ x^2 + x$	
$x^4 + x^3$	$+ x + 1$	$\oplus\ 1 \cdot r_2$
$x^3 + x^2$	$+ 1$	remainder

$q = x + 1$, $r = x^3 + x^2 + 1$.

Row 3: $(x^4 + x^3 + x + 1) \div (x^3 + x^2 + 1)$

$x^4 + x^3 + x + 1$		dividend
$x^4 + x^3 + x$		$\oplus\ x \cdot r_2$
1		remainder

$q = x$, $r = 1$.

Row 4: $(x^3 + x^2 + 1) \div 1$

Dividing by 1 gives the dividend back:
$q = x^3 + x^2 + 1$, $r = 0$.

t **values** ($t = t_1 \oplus q \cdot t_2$):

- Row 1: $0 \oplus x^3 \cdot 1 = x^3$.
- Row 2: $1 \oplus (x + 1) \cdot x^3 = 1 \oplus (x^4 + x^3) = x^4 + x^3 + 1$.
- Row 3: $x^3 \oplus x \cdot (x^4 + x^3 + 1) = x^3 \oplus (x^5 + x^4 + x) = x^5 + x^4 + x^3 + x$.
- Row 4: $(x^3 + x^2 + 1)(x^5 + x^4 + x^3 + x) = (x^8 + x^7 + x^6 + x^4) \oplus (x^7 + x^6 + x^5 + x^3) \oplus (x^5 + x^4 + x^3 + x) = x^8 + x$, so $t = (x^4 + x^3 + 1) \oplus (x^8 + x) = x^8 + x^4 + x^3 + x + 1 = m$ ✓.

Table.

q	r_1	r_2	r	t_1	t_2	t
x^3	$x^8+x^4+x^3+x+1$	x^5	x^4+x^3+x+1	0	1	x^3
$x+1$	x^5	x^4+x^3+x+1	x^3+x^2+1	1	x^3	x^4+x^3+1
x	x^4+x^3+x+1	x^3+x^2+1	1	x^3	x^4+x^3+1	$x^5+x^4+x^3+x$
x^3+x^2+1	x^3+x^2+1	1	0	x^4+x^3+1	$x^5+x^4+x^3+x$	m
	1	0		$\mathbf{x^5+x^4+x^3+x}$	m	

The final $r_1 = 1$, so the inverse exists and equals the final t_1: $\boxed{(x^5)^{-1} = x^5 + x^4 + x^3 + x}$ (byte 3A).

Verification. $x^5 \otimes (x^5 + x^4 + x^3 + x) = x^{10} + x^9 + x^8 + x^6$, reduced modulo m:

$x^{10} + x^9 + x^8 + x^6$		dividend
x^{10}	$+ x^6 + x^5 + x^3 + x^2$	$\oplus\ x^2 \cdot m$
$x^9 + x^8$	$+ x^5 + x^3 + x^2$	
x^9	$+ x^5 + x^4 + x^2 + x$	$\oplus\ x \cdot m$
x^8	$+ x^4 + x^3 + x$	
x^8	$+ x^4 + x^3 + x + 1$	$\oplus\ 1 \cdot m$
	1	remainder

The remainder is 1, so the answer is correct ✓.

WORKED EXAMPLE : 2025–26 Q2, "Find multiplicative inverse of x^7 mod ($x^8 + x^4 + x^3 + x + 1$)." (4 marks)

Setup. $r_1 = m = x^8 + x^4 + x^3 + x + 1$, $r_2 = x^7$, $t_1 = 0$, $t_2 = 1$.
Divisions:

Row 1: $(x^8 + x^4 + x^3 + x + 1) \div x^7$

$x^8 + x^4 + x^3 + x + 1$		dividend
x^8		$\oplus\ x \cdot r_2$
$x^4 + x^3 + x + 1$		remainder

$q = x,\ r = x^4 + x^3 + x + 1.$

Row 2: $x^7 \div (x^4 + x^3 + x + 1)$

x^7		dividend
$x^7 + x^6 + x^4 + x^3$		$\oplus\ x^3 \cdot r_2$
$x^6 + x^4 + x^3$		
$x^6 + x^5 + x^3 + x^2$		$\oplus\ x^2 \cdot r_2$
$x^5 + x^4 + x^2$		
$x^5 + x^4 + x^2 + x$		$\oplus\ x \cdot r_2$
x		remainder

$q = x^3 + x^2 + x,\ r = x.$

Row 3: $(x^4 + x^3 + x + 1) \div x$

$x^4 + x^3 + x + 1$		dividend
x^4		$\oplus\ x^3 \cdot r_2$
$x^3 + x + 1$		
x^3		$\oplus\ x^2 \cdot r_2$
$x + 1$		
x		$\oplus\ 1 \cdot r_2$
1		remainder

$q = x^3 + x^2 + 1,\ r = 1.$

Row 4: $x \div 1$

$q = x,\ r = 0.$

t **values** ($t = t_1 \oplus q \cdot t_2$):

- Row 1: $0 \oplus x \cdot 1 = x$.
- Row 2: $1 \oplus (x^3 + x^2 + x) \cdot x = 1 \oplus (x^4 + x^3 + x^2) = x^4 + x^3 + x^2 + 1$.
- Row 3: $(x^3 + x^2 + 1)(x^4 + x^3 + x^2 + 1) = (x^7 + x^6 + x^5 + x^3) \oplus (x^6 + x^5 + x^4 + x^2) \oplus (x^4 + x^3 + x^2 + 1) = x^7 + 1$, so $t = x \oplus (x^7 + 1) = x^7 + x + 1$.
- Row 4: $(x^4 + x^3 + x^2 + 1) \oplus x(x^7 + x + 1) = (x^4 + x^3 + x^2 + 1) \oplus (x^8 + x^2 + x) = x^8 + x^4 + x^3 + x + 1 = m$ ✓.

Table.

q	r_1	r_2	r	t_1	t_2	t
x	$x^8+x^4+x^3+x+1$	x^7	x^4+x^3+x+1	0	1	x
x^3+x^2+x	x^7	x^4+x^3+x+1	x	1	x	$x^4+x^3+x^2+1$
x^3+x^2+1	x^4+x^3+x+1	x	1	x	$x^4+x^3+x^2+1$	x^7+x+1
x	x	1	0	$x^4+x^3+x^2+1$	x^7+x+1	m
	1	0		**x^7+x+1**	m	

The final $r_1 = 1$, so $\boxed{(x^7)^{-1} = x^7 + x + 1}$ (byte 83, i.e. $\{80\}^{-1} = \{83\}$).

Verification. $x^7 \otimes (x^7 + x + 1) = x^{14} + x^8 + x^7$, reduced modulo m:

$x^{14} + x^8 + x^7$		dividend
$x^{14} + x^{10} + x^9 + x^7 + x^6$		$\oplus\ x^6 \cdot m$
$x^{10} + x^9 + x^8 + x^6$		
$x^{10} + x^6 + x^5 + x^3 + x^2$		$\oplus\ x^2 \cdot m$
$x^9 + x^8 + x^5 + x^3 + x^2$		
$x^9 + x^5 + x^4 + x^2 + x$		$\oplus\ x \cdot m$
$x^8 + x^4 + x^3 + x$		
$x^8 + x^4 + x^3 + x + 1$		$\oplus\ 1 \cdot m$
1		remainder

The remainder is 1 ✓. (A quicker check uses the reductions $x^8 \equiv x^4 + x^3 + x + 1$, $x^{14} \equiv x^7 + x^4 + x^3 + x$, from repeated xtime: $x^{14} + x^8 + x^7 \equiv (x^7 + x^4 + x^3 + x) + (x^4 + x^3 + x + 1) + x^7 = 1$.)

Trap: where the GF(2^8) inverse goes wrong

In row 1, $m \div x^5$ gives remainder $x^4 + x^3 + x + 1$ (only x^8 cancels), not 0 and not x^4. In the t column, $q \cdot t_2$ is a polynomial *product* that must be expanded before the XOR with t_1. The answer is t_1 in the last line, not the last t (which is m). And if a remainder ever has degree ≥ the divisor's, the division was stopped too early.

28.12 Small fields: GF(2^2) and GF(2^3)

GF(2^2) has the four 2-bit words `00`, `01`, `10`, `11` = 0, 1, x, $x+1$. The only irreducible polynomial of degree 2 is x^2+x+1, so $x^2 \equiv x+1$. Addition is XOR with identity `00`; multiplication has identity `01`:

⊕	00	01	10	11
00	00	01	10	11
01	01	00	11	10
10	10	11	00	01
11	11	10	01	00

⊗	00	01	10	11
00	00	00	00	00
01	00	01	10	11
10	00	10	11	01
11	00	11	01	10

For example $x \otimes x = x^2 \equiv x+1$ (`11`), $x \otimes (x+1) = x^2 + x \equiv 1$, so x and $x+1$ are inverses, and $(x+1)^2 = x^2 + 1 \equiv x$.

GF(2^3) with the slide's modulus x^3+x^2+1 (so $x^3 \equiv x^2+1$). Addition is again XOR (e.g. `011` ⊕ `101` = `110`). Multiplication, with the inverse pairs shaded:

⊗	000	001	010	011	100	101	110	111
000	000	000	000	000	000	000	000	000
001	000	001	010	011	100	101	110	111
010	000	010	100	110	101	111	001	011
011	000	011	110	101	001	010	111	100
100	000	100	101	001	111	011	010	110
101	000	101	111	010	011	110	100	001
110	000	110	001	111	010	100	011	101
111	000	111	011	100	110	001	101	010

Inverse pairs: $1 \leftrightarrow 1$, $x \leftrightarrow x^2+x$, $x+1 \leftrightarrow x^2$, $x^2+1 \leftrightarrow x^2+x+1$. Each non-zero row contains `001` exactly once, which is the field property "every non-zero element has exactly one inverse" made visible. One entry worked out: $x \otimes x^2 = x^3 \equiv x^2+1$, so `010` ⊗ `100` = `101`.

Corrections to the notes / slides

- Slide definition of a field: the second operation "satisfies all five properties … except that the identity of the first operation has no inverse". Read this as: every element except 0 has a multiplicative inverse.
- Irreducible-polynomial table (slide 35): degree 5 lists only five polynomials and misses $x^5 + x^3 + 1$; there are six (all listed above, checked by script).
- GF(2^3) multiplication table (slide 47): the binary entries are right, but two polynomial labels are wrong. $x \otimes x$ = `100` is x^2 (slide prints (x)), and $(x+1) \otimes (x^2+x+1)$ = `100` is x^2 (slide prints (x)).

Part III

Symmetric-Key Ciphers

29 Modern Symmetric-Key Ciphers

The classical ciphers of the earlier sections work on letters of a 26-symbol alphabet. A computer, however, has to protect text, numbers, images, audio and video alike, and all of these are stored as bits. Modern ciphers are therefore **bit-oriented**: the plaintext, the ciphertext and the key are bit strings, and the operations are bit operations (permutation of bit positions, table lookup on small groups of bits, XOR). They come in two families, exactly as before: **block ciphers**, which encrypt a fixed-size block of n bits at a time, and **stream ciphers**, which encrypt a stream r bits at a time.

29.1 Modern block ciphers and padding

A **modern block cipher** encrypts an n-bit plaintext block with a k-bit key into an n-bit ciphertext block, and decryption must be the exact inverse, so the encryption mapping has to be a one-to-one function on the 2^n possible blocks. Since the cipher only accepts whole blocks, a message whose length is not a multiple of n is first **padded**. If $|M|$ is the message length and $|Pad|$ the padding length, we need $|M| + |Pad| \equiv 0 \pmod{n}$, which

gives directly

$$|Pad| = (-|M|) \bmod n.$$

Only the last block carries padding; every other block is a full block of message bits.

WORKED EXAMPLE : padding (slide example)

Q. How many padding bits must be added to a message of 100 characters if 8-bit ASCII is used and the block cipher accepts 64-bit blocks?
A. The message is $100 \times 8 = 800$ bits. Then $|Pad| = (-800) \bmod 64$. Since $800 = 12 \times 64 + 32$, we get $-800 \equiv -32 \equiv 32 \pmod{64}$, so **32 bits** of padding (e.g. 0s) are added. The padded plaintext is 832 bits = **13** blocks of 64 bits, and the cipher is applied 13 times; only the 13th block contains padding.
Check: $832/64 = 13$ exactly.
Q (variant). 75 characters of 8-bit ASCII, 128-bit blocks (AES)? **A.** $|M| = 600$, $|Pad| = (-600) \bmod 128 = 640 - 600 =$ **40** bits, giving 640 bits = **5 blocks**.

Substitution or transposition at bit level. A block cipher can in principle be designed either as a transposition (rearrange the n bits) or as a substitution (replace the whole n-bit block by another). A transposition only moves bits, so it *preserves the number of 1s*: if the ciphertext has ten 1s, so does the plaintext. Eve then only has to try the $\binom{64}{10} = 151\,473\,214\,816 \approx 2^{37}$ blocks with ten 1s instead of all 2^{64} blocks. A substitution has no such invariant, which is why a modern block cipher is designed as a substitution cipher; transposition units appear only as components inside it.

Full-size key ciphers and why they are impractical. A **full-size key** cipher allows every possible mapping as a key. A full-size key transposition cipher on n bits has $n!$ keys, and a full-size key substitution cipher has $(2^n)!$ keys (every permutation of the 2^n blocks). For $n = 64$ the key would need $\lceil \log_2 64! \rceil = 296$ bits for transposition and $\log_2 (2^{64})! \approx 2^{70}$ bits for substitution, i.e. a key larger than any storage. Real ciphers are therefore **partial-size key** ciphers: DES uses a 56-bit key, so it realises only 2^{56} of the $(2^{64})!$ possible substitutions. The design task is to make those 2^{56} mappings look like randomly chosen ones, which is done by combining small components.

29.2 Components of a modern block cipher

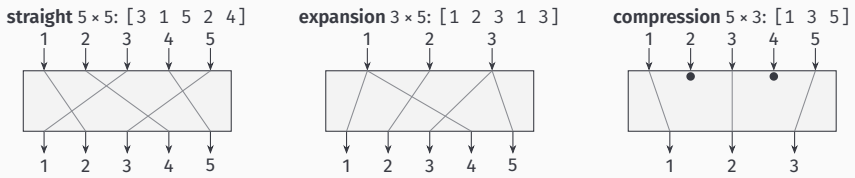
A modern block cipher is not built as one huge substitution table. It is a combination of **transposition units (P-boxes)**, **substitution units (S-boxes)** and a few simple operations (XOR, circular shift, swap, split and combine), each small enough to be implemented and analysed.

P-boxes ★

▸ **Likely this year:** "Differentiate straight, expansion and compression P-boxes; which are invertible and why?" or "Design an 8 × 8 permutation table that …".

A **P-box** (permutation box) is the bit-level analogue of a transposition cipher: it moves bits to new positions. It is described by a **permutation table** with one entry per *output* position; entry j says which input bit is copied to output j. Three types occur.

- A **straight P-box** has n inputs and n outputs, and its table is a permutation of $1..n$. There are $n!$ straight P-boxes (all 6 for $n = 3$). The DES initial permutation is a 64×64 straight P-box (table starting `58 50 42 34 …`).
- An **expansion P-box** has n inputs and $m > n$ outputs. Its table has m entries, of which $m - n$ are repeats, because some inputs are wired to two outputs. The slides' 12×16 example `01 09 10 11 12 01 02 03 03 04 05 06 07 08 09 12` repeats inputs 1, 3, 9 and 12.
- A **compression P-box** has n inputs and $m < n$ outputs. Its table has m entries drawn from $1..n$ with $n - m$ values missing: those inputs are **blocked** and never reach the output. The slides' 32×24 example blocks inputs 7, 8, 9, 15, 16, 23, 24 and 25.



Invertibility. A straight P-box is invertible: since every input appears exactly once in the table, each output position can be sent back to where it came from. The inverse table is found by writing, for each input position i, the output position that holds it. A compression P-box is *not* invertible, because a blocked input bit is simply lost and the inverse would have to invent its value. An expansion P-box is *not* invertible either: an input copied to two outputs would have two candidate sources in the reverse direction, and if the received bits disagree there

is no definite answer (more precisely, its reverse is a compression, not an inverse). Only straight P-boxes are invertible. This is exactly why AES (all components invertible) uses only straight permutations, while DES (Feistel) can afford an expansion P-box inside its round function.

WORKED EXAMPLE : designing and inverting a straight P-box (slide example)

Q. Design an 8×8 permutation table for a straight P-box that moves the two middle bits (bits 4 and 5) of the input to the two ends (bits 1 and 8) of the output, without changing the relative positions of the other bits. Give the inverse table and apply the P-box to `10011100`.
A. Output bit 1 must take input bit 4 and output bit 8 must take input bit 5. The remaining outputs 2–7 take inputs 1, 2, 3, 6, 7, 8 in their original order. Hence the table is

`[4 1 2 3 6 7 8 5].`

Inverse: input 1 went to output 2, input 2 to 3, input 3 to 4, input 4 to 1, input 5 to 8, input 6 to 5, input 7 to 6, input 8 to 7, so the inverse table is `[2 3 4 1 8 5 6 7]`.
Apply: input `10011100` has bits $b_1..b_8 = 1,0,0,1,1,1,0,0$. Output $= b_4 b_1 b_2 b_3 b_6 b_7 b_8 b_5 =$ **11001001**.
Check: applying the inverse table to `11001001` returns `10011100`; the number of 1s (four) is unchanged, as it must be for any P-box.

S-boxes

An **S-box** (substitution box) is a miniature substitution cipher on a few bits: it takes an n-bit input and produces an m-bit output, where m and n need not be equal. It is normally given as a **lookup table**: some input bits select the row, the rest select the column, and the cell holds the output. In DES, for instance, a 6-bit input chooses one of 4 rows by its two outer bits and one of 16 columns by its four middle bits, and the cell is a 4-bit value.

Linear versus nonlinear. Write the inputs as $x_1..x_n$ and the outputs as $y_1..y_m$. The S-box is **linear** if every output is an XOR of inputs, $y_i = a_{i1}x_1 \oplus a_{i2}x_2 \oplus \cdots \oplus a_{in}x_n$ with constants $a_{ij} \in \{0,1\}$; then the whole box is a matrix product over GF(2). For example, $y_1 = x_1 \oplus x_2 \oplus x_3$, $y_2 = x_1$ is linear, with matrix $\left(\begin{smallmatrix}1&1&1\\1&0&0\end{smallmatrix}\right)$. If any output contains a product of inputs, such as $y_1 = x_1x_3 \oplus x_2$, the box is **nonlinear**. A cipher built only from linear boxes, XORs and P-boxes would be linear as a whole, so every ciphertext bit would be a fixed XOR of plaintext and key bits, and a handful of known plaintext–ciphertext pairs would give a solvable linear system for the key. The S-boxes are the only nonlinear part of a block cipher, and they must be nonlinear.

Invertibility. An S-box can be inverted only if it is a one-to-one map, which requires $m = n$ (a $6 \to 4$ box maps 64 inputs onto 16 outputs, so each output has 4 preimages). The slides' 3×3 example uses the leftmost bit as the row and the two rightmost bits as the column:

enc	00	01	10	11
0	011	101	111	100
1	000	010	001	110

dec	00	01	10	11
0	100	110	101	000
1	011	001	111	010

Input `001` (row 0, column 01) encrypts to `101`, and `101` (row 1, column 01) in the decryption table gives back `001`; a script confirms that the two tables undo each other for all 8 inputs.

XOR, circular shift, swap, split and combine

Exclusive-or is addition in GF(2^n), and five properties make it the standard way to mix in a key: **closure** (the XOR of two n-bit words is an n-bit word), **associativity**, **commutativity**, an **identity** $00\ldots0$, and an **inverse** for every word, namely the word itself, since $x \oplus x = 00\ldots0$. The last property is the important one: whatever is XORed in can be removed by XORing the same value again, $(x \oplus k) \oplus k = x$. A key mixed in this way is called a **whitener**.

A **circular shift** rotates an n-bit word left or right by k positions. It is invertible, since a left shift by k is undone by a right shift by k, and shifting by k is the same as shifting by k mod n. A **swap** is the special circular shift by $k = n/2$, exchanging the left and right halves; doing it twice restores the word, so it is **self-invertible**. A **split** cuts an n-bit word into two $n/2$-bit halves and a **combine** concatenates two halves; each is the inverse of the other.

29.3 Product ciphers: confusion, diffusion and rounds ★

▸ **Likely this year:** One-line theory: "Define confusion and diffusion; which component provides which?" or "What is the avalanche effect?"

A single small component is easy to analyse and therefore easy to break; a full-size substitution table is secure but impossible to store. Shannon's answer was the **product cipher**: a cipher that alternates substitutions, per-

mutations and key mixing, so that simple and cheap parts combine into a mapping that behaves like a random one. He named the two properties such a cipher must have.

Diffusion hides the relationship between the ciphertext and the plaintext: each ciphertext bit should depend on many or all plaintext bits, so that changing one plaintext bit changes many ciphertext bits. This frustrates an attacker who uses ciphertext statistics (letter or bit frequencies) to guess the plaintext, because any local pattern of the plaintext is spread over the whole block.

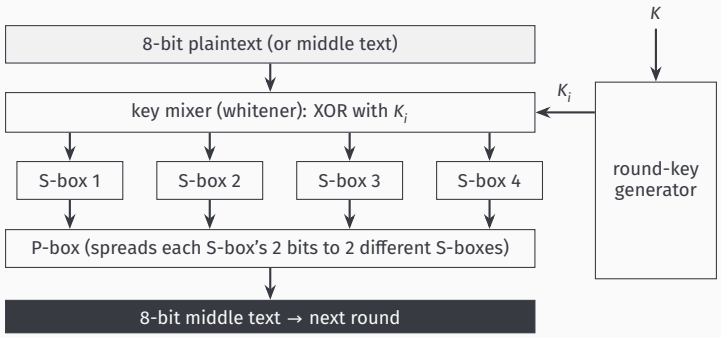
Confusion hides the relationship between the ciphertext and the key: each ciphertext bit should depend on the key in a complicated way, so that changing one key bit changes most ciphertext bits. This frustrates an attacker who uses known ciphertext to solve for the key.

Why S-boxes give confusion. The key enters the cipher only through XOR, and XOR and P-boxes are linear over GF(2). If the cipher had no S-boxes, every ciphertext bit would be a fixed XOR of plaintext bits and key bits, and a few plaintext–ciphertext pairs would give a linear system whose solution is the key. The nonlinear S-box placed after the key mixing breaks this: its output is a nonlinear function of (data $\oplus$ key), so after a few rounds the ciphertext is a high-degree nonlinear function of the key that cannot be solved for. That is precisely confusion.

Why P-boxes give diffusion. Suppose one input bit of a round changes. It enters one S-box, whose output may change in all its bits (say 2 bits for a 2×2 box). The P-box then sends those changed bits to *different* S-boxes of the next round, where each again changes up to 2 bits, so the change can grow from 1 bit to 2, 4, 8, ... bits in successive rounds. Without the P-box the changed bits would stay inside the same S-box forever and the change would never spread; the permutation between rounds is what carries it across the block. That is diffusion.

Avalanche. The observable result of good confusion and diffusion is the **avalanche effect**: flipping one bit of the plaintext or of the key flips about half of the ciphertext bits. Section 30 shows DES going from 1 differing bit to 29 differing bits out of 64.

Rounds. One layer of S-boxes and P-boxes spreads a bit change only a little, so the layer is repeated. An **iterated product cipher** applies the same **round** N times; each round consists of a **key mixer** (whitener, XOR with a round key), a layer of S-boxes and a P-box. The text between two rounds is called the **middle text**, and a **key schedule** (round-key generator) derives the round keys $K_1, \ldots, K_N$ from the cipher key, so that each round mixes in a different key. Decryption runs N inverse rounds with the round keys in reverse order.



One round of the slides' 8-bit product cipher (the slide shows two such rounds).

29.4 Feistel ciphers ★

★ **In the papers:** 2024–25 Q6 (5 marks) opens with "what Feistel structure is prevalent in DES": the derivation below is that part of the answer.

Modern block ciphers fall into two classes. **Feistel ciphers** (DES) use invertible, self-invertible *and noninvertible* components; **non-Feistel ciphers** (AES) use only invertible components. The problem the Feistel design solves is this: noninvertible components such as an expansion P-box or a $6 \to 4$ S-box mix bits very effectively, but a cipher must be decryptable, and a noninvertible component cannot be run backwards. Feistel's idea is to collect all the noninvertible parts into one function f and never invert it. Its output is only ever *XORed* onto the data, and since XOR is self-inverse, the receiver can cancel it by computing the same f value again and XORing once more.

Step 1: the idea in its simplest form. Let f depend only on the key. Then

$$\text{encryption: } C = P \oplus f(K), \qquad \text{decryption: } C \oplus f(K) = P \oplus f(K) \oplus f(K) = P \oplus 00\ldots0 = P.$$

Nothing here requires f^{-1}; decryption simply computes $f(K)$ forwards.

WORKED EXAMPLE : the slides' first Feistel example

Q. Plaintext and ciphertext are 4 bits, the key is 3 bits. f takes the first and third bits of the key, reads them as a decimal number, squares it, and writes the result as 4 bits. Encrypt and decrypt $P = 0111$ with $K = 101$.

A. First and third key bits: $1, 1 \to (11)_2 = 3$; $3^2 = 9 = 1001$, so $f(101) = 1001$.
Encryption: $C = P \oplus f(K) = 0111 \oplus 1001 =$ **1110**.
Decryption: $C \oplus f(K) = 1110 \oplus 1001 = 0111 = P$ ✓.
f is noninvertible (e.g. keys 101 and 111 both give 1001, since bit 2 is ignored), yet decryption works because only f, never f^{-1}, is needed.

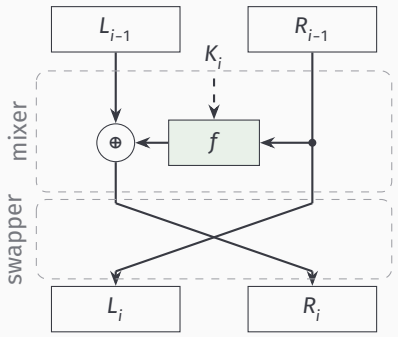
Step 2: let the data enter f. A function of the key alone adds nothing per block (it is a fixed XOR, i.e. a one-key Vernam). To make f depend on the data while keeping decryption possible, the block is **split** into a left half L and a right half R; R feeds f and L is XORed with the output. This unit is the **mixer**:

$$L_2 = L_1 \oplus f(R_1, K), \qquad R_2 = R_1.$$

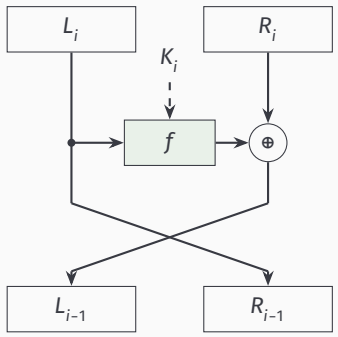
The receiver gets $R_2 = R_1$ unchanged, so it can recompute exactly the same $f(R_1, K)$ and cancel it: $L_2 \oplus f(R_2, K) = L_1 \oplus f(R_1, K) \oplus f(R_1, K) = L_1$. The mixer is therefore **self-invertible**: applying it twice with the same key is the identity.

Step 3: the swapper and more rounds. The mixer has one flaw: R is never changed, so half of the plaintext appears in clear in the ciphertext. The fix is to add a **swapper** after the mixer, so that the next round modifies the other half, and to repeat for several rounds. One Feistel round i is then

$$L_i = R_{i-1}, \qquad R_i = L_{i-1} \oplus f(R_{i-1}, K_i).$$



encryption round:
$L_i = R_{i-1}$, $R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$



inverse round (same K_i), f only forwards:
$R_{i-1} = L_i$, $L_{i-1} = R_i \oplus f(L_i, K_i)$

★ MUST KNOW : why a Feistel round is invertible even if f is not

Given the round output (L_i, R_i), the first equation hands back $R_{i-1} = L_i$ directly. Knowing R_{i-1}, the receiver can *recompute* $f(R_{i-1}, K_i)$ forwards and XOR it away:

$$R_i \oplus f(L_i, K_i) = L_{i-1} \oplus f(R_{i-1}, K_i) \oplus f(R_{i-1}, K_i) = L_{i-1}.$$

f is only ever evaluated, never inverted, so it may be many-to-one (expansion P-box, $6 \to 4$ S-boxes). Since the mixer and the swapper are each self-inverse, decryption is the same structure run with the round keys in reverse order $(K_N, \ldots, K_1)$.

In the slides' two-round final design (slide L7-29), both rounds end with a swapper, so the decryption circuit applies the swapper first and then the mixer, round keys K_2 then K_1. DES removes the swapper from the *last* round; then encryption and decryption are literally the same algorithm and only the key order changes. The toy below uses that DES convention.

WORKED EXAMPLE : two-round Feistel cipher with a noninvertible f

Q. An 8-bit Feistel cipher has 4-bit halves and two rounds; round 1 has a swapper, round 2 does not (DES convention). The round function is $f(R, K) = (R \oplus K)^2 \bmod 16$. Encrypt $L_0R_0 = 1010\ 0111$ with $K_1 = 0100$, $K_2 = 1001$, then decrypt with the same algorithm and the keys reversed. Show that f is not invertible.
A. f is not invertible: $0011^2 = 9$ and $0101^2 = 25 \equiv 9 \pmod{16}$, so two inputs give 1001; in fact $x^2 \bmod 16$ only takes the values 0, 1, 4, 9.

Round	input L, R	$R \oplus K$	f	$L \oplus f$	output (after swap?)
Enc 1 ($K_1 = 0100$)	1010, 0111	0011	1001	0011	swap → 0111, 0011
Enc 2 ($K_2 = 1001$)	0111, 0011	1010	0100	0011	no swap → **0011, 0011**
Dec 1 ($K_2 = 1001$)	0011, 0011	1010	0100	0111	swap → 0011, 0111
Dec 2 ($K_1 = 0100$)	0011, 0111	0011	1001	1010	no swap → **1010, 0111**

Ciphertext = **0011 0011**; decryption returns 1010 0111 = P ✓. Each decryption round recomputes exactly the

f value of the matching encryption round (1001 and 0100), because the half fed to f is passed through unchanged.

29.5 Non-Feistel ciphers

A **non-Feistel cipher** uses only invertible components, and every component of the encryption has a matching inverse component in the decryption. This constrains the parts: S-boxes must have as many output bits as input bits, and expansion and compression P-boxes are not allowed, because they cannot be inverted. Since there is no f whose output is XORed in, there is also no need to split the block into halves; each round transforms the whole block. The slides' 8-bit product cipher above is a non-Feistel cipher: XOR (self-invertible), 2×2 invertible S-boxes and a straight P-box, so each round is invertible and decryption runs the inverse rounds with keys K_2, K_1. AES is the real example.

	Feistel (DES)	**Non-Feistel** (AES)
Components	invertible, self-invertible and noninvertible	only invertible ($n \times n$ S-boxes, straight P-boxes)
Block handling	split into L, R; mixer $L \oplus f(R, K)$, then swapper	whole block in every round
Decryption	same structure, round keys reversed; f never inverted	inverse components in reverse order, round keys reversed

29.6 Attacks on modern block ciphers

Brute force on the key is normally infeasible because keys are long (but 2^{56} for DES is not). The serious attacks exploit the structure of the rounds.

- **Differential cryptanalysis** (Biham and Shamir) is a **chosen-plaintext** attack. Eve submits pairs of plaintexts with a chosen difference $P_1 \oplus P_2$ and studies the output difference $C_1 \oplus C_2$; a weak cipher propagates certain differences with high probability, which reveals key bits. The slides' extreme case is a cipher that is only an XOR with the key: $C_1 \oplus C_2 = (P_1 \oplus K) \oplus (P_2 \oplus K) = P_1 \oplus P_2$, so the difference passes through untouched and is independent of the key. Nonlinear S-boxes are what destroy this predictability.
- **Linear cryptanalysis** (Matsui) is a **known-plaintext** attack. If an S-box is not fully nonlinear, some XOR of input and output bits equals an XOR of key bits with probability noticeably different from $1/2$; collecting many plaintext–ciphertext pairs and counting how often such an equation holds reveals those key bits.

29.7 Modern stream ciphers

A **modern stream cipher** encrypts and decrypts r bits at a time: the plaintext $P = p_n \ldots p_2 p_1$, ciphertext $C = c_n \ldots c_2 c_1$ and key stream $K = k_n \ldots k_2 k_1$ consist of r-bit words, and $c_i = p_i \oplus k_i$, $p_i = c_i \oplus k_i$. Encryption and decryption are a single XOR, so the cipher is fast and needs no padding; all of its security lies in the **key stream**, which must look random and must never be reused. Stream ciphers are classified by how the key stream is produced.

Synchronous stream ciphers. In a **synchronous** stream cipher the key stream is *independent* of the plaintext and the ciphertext; it is generated from the key alone, and sender and receiver must stay in step (synchronised) with each other. The ideal case is the **one-time pad**: a truly random key stream as long as the message, used once, giving perfect secrecy, at the price that a key as long as all the traffic must be shared in advance. Practical synchronous ciphers replace the random pad by a pseudorandom generator such as a **feedback shift register** (FSR). An FSR has m one-bit cells $b_{m-1} \ldots b_1 b_0$; at each step b_0 is output as a key bit, all cells shift right, and the new leftmost bit is computed by a **feedback function** of the old cells. In a **linear** FSR (LFSR) the feedback is an XOR of selected cells, so it costs one gate. With a well-chosen feedback an m-cell LFSR runs through all $2^m - 1$ nonzero states before repeating, the maximum period. Because it is linear, however, about $2m$ consecutive key-stream bits are enough to solve for the feedback taps and predict the rest, so real designs use a **nonlinear** FSR or combine several LFSRs nonlinearly.

WORKED EXAMPLE : LFSR key stream (Forouzan style)

Q. Create a 4-cell LFSR with feedback $b_4 = b_1 \oplus b_0$ (the new b_3). Show the output for 20 shifts if the seed $b_3 b_2 b_1 b_0$ is 0001, and find the period.
A. Each step outputs b_0, then the state becomes $(b_1 \oplus b_0)\, b_3\, b_2\, b_1$.

step	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
state	0001	1000	0100	0010	1001	1100	0110	1011	0101	1010	1101	1110	1111	0111	0011	**0001**
out b_0	1	0	0	0	1	0	0	1	1	0	1	0	1	1	1	1

At step 15 the state is the seed again, so the key stream repeats with period **15** $= 2^4 - 1$ (maximum: every nonzero state appears once). The 20 output bits are **1000 1001 1010 1111 0001**.
Check: the bits 16–20 (1 0001) repeat bits 1–5 (1 0001), as the period predicts.

Nonsynchronous (self-synchronising) stream ciphers. In a **nonsynchronous** stream cipher each key-stream element depends on previous plaintext or ciphertext elements. The usual form is **self-synchronising**: the key stream is computed from the key and the last few ciphertext bits (as in CFB mode). If a ciphertext bit is corrupted or lost, the receiver's key stream is wrong only while the bad bit is inside its window, and decryption recovers by itself once the window has passed.

Error behaviour. In a synchronous cipher a flipped ciphertext bit spoils only the matching plaintext bit, but a lost or inserted bit shifts the two key streams against each other and garbles everything after it until the parties resynchronise. In a self-synchronising cipher a single error spoils a few following bits as well, but a lost bit is recovered from automatically.

Corrections to the notes / slides

- Feistel example (L7 slide 25): the problem says f uses the *first and third* key bits, but the solution "extracts the first and second bits to get 11". For $K = 101$ the first and second bits are $10 = 2$, giving $f = 0100$; only the first and third bits give $11 = 3$ and the stated $f = 1001$, $C = 1110$. Use first and third.
- Stream ciphers (L7 slide 39): the line reads "In a synchronous stream cipher the key is independent of the plaintext or ciphertext"; it is easily read as "asynchronous" (the old version of these notes did so). The key-independent class is **synchronous**. Some books (Stallings) call the *other* class, the self-synchronising ciphers, "asynchronous", so do not mix the words up.
- Nonlinear S-box (L7 slide 18): over GF(2), $x^3 = x$ and $x^2 = x$, so the slide's $y_1 = x_1^3 + x_2 = x_1 + x_2$ is actually linear; the box is nonlinear because of the product term x_1x_2 in $y_2 = x_1^2 + x_1x_2 + x_3$.

30 Data Encryption Standard (DES)

The **Data Encryption Standard** is the symmetric-key block cipher published by NIST (then NBS) in 1977, based on IBM's Lucifer. It encrypts a **64-bit block** with a **56-bit key** through **16 Feistel rounds**. The key is normally supplied as 64 bits, of which 8 are parity bits that are dropped, and a key schedule derives sixteen **48-bit round keys** $K_1, \ldots, K_{16}$ from it. Decryption is done by the **reverse cipher**, which is the same algorithm with the round keys used in the order $K_{16}, \ldots, K_1$. DES has since been replaced by AES because its key is too short, but its structure is the standard example of a Feistel cipher (Section 29).

30.1 DES structure ★

★ **In the papers:** 2024–25 Q6 (5 marks): Feistel structure, block diagrams of the DES algorithm and the round function, pseudocode for the S-box substitution and key generation.

Encryption consists of an **initial permutation** (IP) of the 64-bit plaintext, sixteen rounds, and a **final permutation** (FP). After IP the block is split into a left half L_0 and a right half R_0 of 32 bits each. Each round i takes (L_{i-1}, R_{i-1}) and produces (L_i, R_i) with a Feistel **mixer** followed by a **swapper**:

$$L_i = R_{i-1}, \qquad R_i = L_{i-1} \oplus f(R_{i-1}, K_i),$$

where f is the DES function described below. Round 16 has no swapper, and after it the halves are combined and passed through FP to give the 64-bit ciphertext. The block diagrams of the whole algorithm, of one round and of the key generator are drawn in the exam answer at the end of the key-generation subsection.

30.2 Initial and final permutations

IP and FP are keyless **straight P-boxes** (64×64) that are inverses of each other: $\mathrm{FP} = \mathrm{IP}^{-1}$. The IP table is read row by row; entry j is the input bit that goes to output position j.

Initial permutation

58	50	42	34	26	18	10	02
60	52	44	36	28	20	12	04
62	54	46	38	30	22	14	06
64	56	48	40	32	24	16	08
57	49	41	33	25	17	09	01
59	51	43	35	27	19	11	03
61	53	45	37	29	21	13	05
63	55	47	39	31	23	15	07

Final permutation

40	08	48	16	56	24	64	32
39	07	47	15	55	23	63	31
38	06	46	14	54	22	62	30
37	05	45	13	53	21	61	29
36	04	44	12	52	20	60	28
35	03	43	11	51	19	59	27
34	02	42	10	50	18	58	26
33	01	41	09	49	17	57	25

So output bit 1 of IP is input bit 58, and input bit 1 lands in output position 40; correspondingly the FP table has 40 in position 1 (FP puts it back).

Why they add no security. Both permutations are fixed, keyless and published. An attacker who knows a plaintext can apply IP to it himself, and can apply $\mathrm{IP} = \mathrm{FP}^{-1}$ to the ciphertext, which strips both permutations off and leaves exactly the 16 rounds to attack. IP and FP have no cryptographic significance; they are usually explained as a convenience for loading 8-bit bytes into 1970s hardware.

WORKED EXAMPLE : initial permutation (Forouzan style)

Q. Find the output of the initial permutation for the input `0x0002 0000 0000 0001` and show that FP restores it.
A. The input has only two 1-bits: hex digit 4 is 2 = 0010, i.e. bit 12 + 3 = **15**, and the last bit, **64**. In the IP table, 15 is entry 63 and 64 is entry 25, so the output has 1s exactly in positions 25 and 63:

$$\text{IP}(0002000000000001) = \texttt{0x0000 0080 0000 0002}.$$

(Bit 25 is the leading bit of hex digit 7, giving 8; bit 63 is the third bit of hex digit 16, giving 2.)
Check: in the FP table, entry 15 is 63 and entry 64 is 25, so FP moves bits 63 and 25 back to positions 15 and 64.

30.3 Rounds: mixer, swapper and the reverse cipher

Each round is a Feistel round (Section 29): the **mixer** replaces L_{i-1} by $L_{i-1} \oplus f(R_{i-1}, K_i)$ and leaves R_{i-1} alone, and the **swapper** exchanges the halves. The mixer is invertible because of the XOR: given the output, the receiver still has R_{i-1}, recomputes $f(R_{i-1}, K_i)$ and XORs it away. The mixer and the swapper are each **self-inverse**: applying either twice (the mixer with the same key) gives back the input.

Why decryption is the same algorithm with keys reversed. Write encryption as the sequence of operations

$$\text{IP},\ M_{K_1},\ S,\ M_{K_2},\ S,\ \ldots,\ M_{K_{15}},\ S,\ M_{K_{16}},\ \text{FP},$$

where M_K is the mixer with key K and S the swapper. The inverse of a sequence applies the inverses of its steps in reverse order, and here $\text{FP}^{-1} = \text{IP}$, $M_K^{-1} = M_K$, $S^{-1} = S$ and $\text{IP}^{-1} = \text{FP}$. Decryption is therefore

$$\text{IP},\ M_{K_{16}},\ S,\ M_{K_{15}},\ S,\ \ldots,\ S,\ M_{K_1},\ \text{FP}.$$

This is exactly the encryption algorithm with the round keys in the order $K_{16}, \ldots, K_1$. The match works only because round 16 has no swapper: with a swapper there, the inverse would start with S and would no longer have the same shape as the cipher. The last round has no swapper so that the reverse cipher is the cipher itself with reversed round keys. (Equivalently, Stallings lets all 16 rounds swap and adds a 32-bit swap before FP; the two descriptions give the same ciphertext.)

The slides' pseudocode for the cipher (Forouzan):

```
Cipher (plainBlock[64], RoundKeys[16, 48], cipherBlock[64])
{
    permute (64, 64, plainBlock, inBlock, InitialPermutationTable)
    split (64, 32, inBlock, leftBlock, rightBlock)
    for (round = 1 to 16)
    {
        mixer (leftBlock, rightBlock, RoundKeys[round])
        if (round ≠ 16) swapper (leftBlock, rightBlock)
    }
    combine (32, 64, leftBlock, rightBlock, outBlock)
    permute (64, 64, outBlock, cipherBlock, FinalPermutationTable)
}
```

Here mixer does $T \leftarrow f(\text{rightBlock}, K)$, leftBlock $\leftarrow$ leftBlock $\oplus\ T$, and swapper exchanges the two blocks through a temporary copy.

30.4 The DES function $f(R_{i-1}, K_i)$ ★

The DES function is the heart of the cipher and the only place where the key enters. It applies the 48-bit round key to the 32-bit right half and returns 32 bits, through four steps.

1. Expansion P-box (32 → 48). R_{i-1} has 32 bits but K_i has 48, and the S-boxes need 6-bit inputs, so R_{i-1} is expanded first. It is viewed as eight 4-bit sections; each section is widened to 6 bits by adding, on its left, the last bit of the previous section and, on its right, the first bit of the next section (cyclically, so section 1 borrows bit 32 and section 8 borrows bit 1):

32	01	02	03	04	05		04	05	06	07	08	09
08	09	10	11	12	13		12	13	14	15	16	17
16	17	18	19	20	21		20	21	22	23	24	25
24	25	26	27	28	29		28	29	30	31	32	01

The 16 bits that appear twice are the first and fourth bits of every section (1, 4, 5, 8, ..., 29, 32). This duplication is deliberate: a bit at the edge of a section reaches two neighbouring S-boxes, so changing it changes the input

of two S-boxes, which is what makes the avalanche grow quickly. Being an expansion P-box, this step is not invertible, which a Feistel cipher can afford.

2. Whitener. The 48-bit expanded half is XORed with the 48-bit round key K_i. This is the only point at which the key influences the round.

3. S-boxes (48 → 32). The 48 bits are cut into eight 6-bit groups, and group j enters S-box S_j, which outputs 4 bits. The S-boxes do the real mixing (confusion), being the only nonlinear part of DES. Each S-box is a table of 4 rows and 16 columns. For a 6-bit input $b_1b_2b_3b_4b_5b_6$:

$$\text{row} = (b_1b_6)_2 \in \{0..3\}, \qquad \text{column} = (b_2b_3b_4b_5)_2 \in \{0..15\},$$

and the cell is the 4-bit output. The outer bits pick one of four substitutions and the middle bits are substituted by it; every row of every S-box is a permutation of 0..15 (checked by script), so each output value has exactly 4 preimages and the box as a whole is not invertible. The eight S-boxes are different. S-box 1:

S1	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
2	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

4. Straight P-box (32 → 32). The 32 S-box output bits are permuted by a straight P-box; entry j is the input bit sent to output j, so the seventh input bit becomes the second output bit:

16	07	20	21	29	12	28	17
01	15	23	26	05	18	31	10
02	08	24	14	32	27	03	09
19	13	30	06	22	11	04	25

Its job is diffusion: the 4 output bits of each S-box are sent to positions that feed different S-boxes in the next round.

WORKED EXAMPLE : S-box lookup

▸ **Likely this year:** "The input to S-box 1 is `100011`. What is the output?" (Forouzan's standard S-box question.)

Q. (a) The input to S-box 1 is `100011`. What is the output? (b) The input to S-box 8 is `000000`. What is the output?

A. (a) Outer bits b_1b_6 = 1 1, so row = $(11)_2$ = 3. Middle bits $b_2..b_5$ = 0001, so column = 1. In S1, row 3, column 1 holds 12, so the output is **1100**.

(b) Outer bits 0 0 ⇒ row 0; middle bits 0000 ⇒ column 0. S8 row 0 begins 13 2 8 4 ..., so the output is 13 = **1101**.

More S1 lookups for practice: `000000` → row 0, col 0 → 14 = 1110; `111111` → row 3, col 15 → 13 = 1101; `011011` → row 1, col 13 → 5 = 0101.

Trap: row and column bits

The row is the *first and last* bit (not the first two), and the column is the *middle four*. Numbering starts at 0 for both rows and columns. Writing the answer as a decimal (12) without converting it to 4 bits (`1100`) loses the mark that the output is a 4-bit word.

WORKED EXAMPLE : one full DES round (Forouzan's trace: P = 123456ABCD132536, key AABB09182736CCDD)

Q. After the initial permutation, L_0 = 14A7D678 and R_0 = 18CA18AD; the first round key is K_1 = 194CD072DE8C. Compute $f(R_0, K_1)$, L_1 and R_1.

A. R_0 = 0001 1000 1100 1010 0001 1000 1010 1101. Expansion (each 4-bit section borrows its neighbours' edge bits) gives $E(R_0)$ = 8F16540F155A; XOR with K_1 gives 965A847DCBD6. The eight 6-bit groups go through the S-boxes:

S-box	1	2	3	4	5	6	7	8
$E(R_0)$ group	100011	110001	011001	010100	000011	110001	010101	011010
K_1 group	000110	010100	110011	010000	011100	101101	111010	001100
XOR (input)	100101	100101	101010	000100	011111	011100	101111	010110
row, col	3, 2	3, 2	2, 5	0, 2	1, 15	0, 14	3, 7	0, 11
output	8	10	15	14	6	5	7	14

S-box output = 8AFE657E; the straight P-box gives $f(R_0, K_1)$ = 4EDF35EC. Then

$$L_1 = R_0 = \text{18CA18AD}, \qquad R_1 = L_0 \oplus f = \text{14A7D678} \oplus \text{4EDF35EC} = \text{5A78E394}.$$

Check: this is the round-1 output in Forouzan's table; continuing all 16 rounds (round 16 output `19BA9212 CF26B472`, no swap) and FP gives the ciphertext `C0B7A8D05F3A829C`, and running the same program with the keys reversed returns the plaintext.

30.5 Key generation ★

The round-key generator must turn one 56-bit cipher key into sixteen different 48-bit round keys, cheaply and so that every key bit influences many rounds. It works in four steps.

1. **Parity drop (64 → 56).** The key is supplied as 64 bits in which bits 8, 16, 24, ..., 64 are parity bits (each key byte has odd parity). A compression permutation (PC-1) discards these 8 bits and permutes the remaining 56.
2. **Split.** The 56 bits are split into two 28-bit halves C_0 and D_0.
3. **Shift.** In round i both halves are circularly shifted left, by **1 bit in rounds 1, 2, 9 and 16** and by 2 bits in all other rounds, giving C_i and D_i.
4. **Compression P-box (56 → 48).** $C_i \| D_i$ passes through a compression P-box (PC-2) that blocks 8 of the 56 bits and permutes the other 48; the result is K_i.

Why it is built this way. Because the halves are rotated before each compression, a different selection and arrangement of key bits reaches the S-boxes in every round; each key bit ends up in 12 to 15 of the 16 round keys (about 14 on average, counted by script). The shift amounts add up to $4 \times 1 + 12 \times 2 = 28$, a full rotation of a 28-bit half, so $C_{16} = C_0$ and $D_{16} = D_0$. This lets the decryption side generate $K_{16}, \ldots, K_1$ on the fly by *right* shifts starting from the original halves.

WORKED EXAMPLE : first round key (same key as the trace above)

Q. For the key `AABB09182736CCDD`, find C_0, D_0, C_1, D_1 and K_1.
A. Parity drop gives the 56-bit key `C3C033A33F0CFA`, so C_0 = `C3C033A`, D_0 = `33F0CFA`. Round 1 shifts by 1 bit:

$$C_0 = 1100\,0011\,1100\,0000\,0011\,0011\,1010 \xrightarrow{\text{rotl 1}} C_1 = 1000\,0111\,1000\,0000\,0110\,0111\,0101 = \texttt{8780675}$$
$$D_0 = 0011\,0011\,1111\,0000\,1100\,1111\,1010 \xrightarrow{\text{rotl 1}} D_1 = 0110\,0111\,1110\,0001\,1001\,1111\,0100 = \texttt{67E19F4}$$

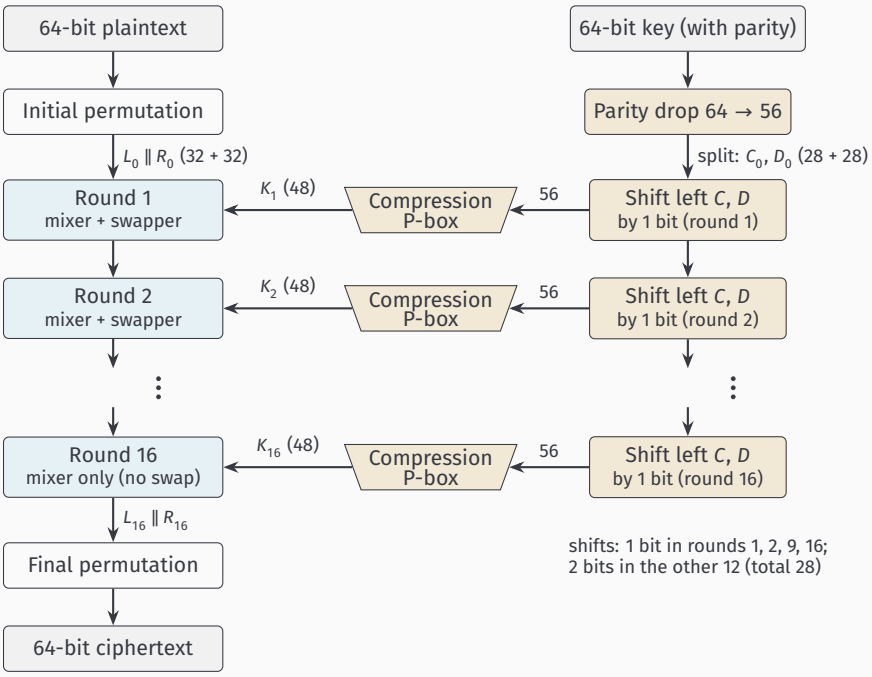
The compression P-box applied to $C_1 \| D_1$ gives K_1 = `194CD072DE8C`, the key used in the round trace above.
Check: in each half the leading bit moved to the end (e.g. C_0 starts with 1 and C_1 ends with 1).

WORKED EXAMPLE : 2024–25 Q6 (5 marks), answer as it should be written

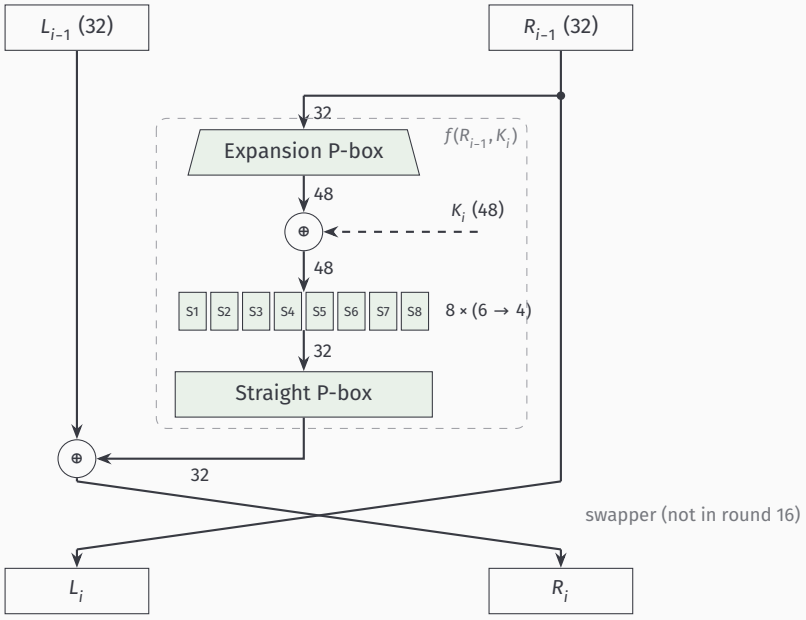
Q. In DES, what Feistel structure is prevalent? Use flow diagrams and key generation to define, with block diagrams, (1) the algorithm for DES and (2) the algorithm for the round function. How would you write the pseudocode for the substitution block and key generation?
A. Feistel structure. DES is a Feistel cipher: the 64-bit block (after IP) is split into 32-bit halves, and each of the 16 rounds applies a mixer and a swapper,

$$L_i = R_{i-1}, \qquad R_i = L_{i-1} \oplus f(R_{i-1}, K_i), \qquad i = 1..16 \text{ (no swap in round 16).}$$

All noninvertible parts (expansion P-box, 6 → 4 S-boxes) are inside f, whose output is only XORed in. Since $R_{i-1} = L_i$ is available to the receiver, $L_{i-1} = R_i \oplus f(L_i, K_i)$; so f never has to be inverted, and decryption is the same algorithm with keys $K_{16}, \ldots, K_1$.
(1) DES algorithm and key generation.



(2) Round i and the round function $f(R_{i-1}, K_i)$.



Inside f: the expansion P-box widens R_{i-1} from 32 to 48 bits, the whitener XORs in K_i, eight S-boxes reduce 48 bits to 32 (row = outer bits, column = middle 4 bits), and a straight P-box permutes the 32 bits.

Pseudocode (Forouzan style; bit arrays are indexed from 1).

```
substitute (inBlock[48], outBlock[32], SubstitutionTables[8, 4, 16])
{
   for (i = 1 to 8)
   {
      b ← (i − 1) × 6                          // start of group i
      row ← 2× inBlock[b+1] + inBlock[b+6]
      col ← 8× inBlock[b+2] + 4× inBlock[b+3]
            + 2× inBlock[b+4] + inBlock[b+5]
      value ← SubstitutionTables[i][row][col]
      o ← (i − 1) × 4
      outBlock[o+1] ← value / 8;  value ← value mod 8
      outBlock[o+2] ← value / 4;  value ← value mod 4
      outBlock[o+3] ← value / 2;  value ← value mod 2
      outBlock[o+4] ← value
   }
}
```

```
Key_Generator (keyWithParities[64], RoundKeys[16, 48], ShiftTable[16])
{
   permute (64, 56, keyWithParities, cipherKey, ParityDropTable)
   split (56, 28, cipherKey, leftKey, rightKey)
   for (round = 1 to 16)
   {
      shiftLeft (leftKey, ShiftTable[round])
      shiftLeft (rightKey, ShiftTable[round])
      combine (28, 56, leftKey, rightKey, preRoundKey)
      permute (56, 48, preRoundKey, RoundKeys[round], KeyCompressionTable)
   }
}
```

ShiftTable = [1, 1, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 1]; shiftLeft is a circular left shift of a 28-bit half.

Check: the substitute routine reproduces the S-box output 8AFE657E of the round trace above (tested by script).

★ MUST KNOW : DES at a glance

64-bit block, 56-bit key (64 with parity), 16 Feistel rounds, 48-bit round keys. Round: $L_i = R_{i-1}$, $R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$, no swap in round 16. f: expansion 32 → 48, XOR K_i, 8 S-boxes 6 → 4 (row = $b_1 b_6$, column = $b_2..b_5$), straight P-box 32 → 32. Keys: parity drop 64 → 56, split 28 + 28, rotate left 1 bit in rounds 1, 2, 9, 16 and 2 bits otherwise, compression 56 → 48. Decryption: same algorithm, keys $K_{16}..K_1$.

30.6 DES analysis: avalanche and completeness

Two properties are expected of a good block cipher. The **avalanche effect** means that a small change in the plaintext (or the key) should change many ciphertext bits, about half of them. **Completeness** means that each ciphertext bit should depend on many plaintext bits. In DES both come from the combination of S-boxes (confusion) and P-boxes, including the expansion that feeds each edge bit to two S-boxes (diffusion), and tests show DES is strong in both.

WORKED EXAMPLE : avalanche in DES (Forouzan's example)

Q. Encrypt the plaintexts 0000000000000000 and 0000000000000001 (differing in one bit) with the key 22234512987ABB23. How many ciphertext bits differ?
A. The ciphertexts are 4789FD476E82A5F1 and 0A4ED5C15A63FEA3; their XOR has **29** one-bits, so a 1-bit change of the plaintext changed 29 of the 64 ciphertext bits. The number of differing bits after each round (script) shows how fast the difference spreads:

round	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
bits differing	1	6	20	29	30	33	32	29	32	39	33	28	30	31	30	29

After only 3 to 4 rounds the difference is already near the ideal of 32; the remaining rounds keep it there.

30.7 Weaknesses of DES

Design of the S-boxes and P-boxes. Critics point out a few irregularities, none of which has led to a practical attack:

- In S-box 4, the last three output bits can be derived in the same way as the first output bit by complementing some of the input bits.
- Two specially chosen inputs to the S-box array can produce the same output.
- The same output of a single round can be obtained by changing bits in only three neighbouring S-boxes.
- The initial and final permutations have no security benefit, and in the expansion P-box the first and fourth bits of every 4-bit section are repeated.

Key size. The real weakness is the 56-bit key. A brute-force attack on one ciphertext block must try up to $2^{56} \approx 7.2 \times 10^{16}$ keys. At 10^6 keys per second on each of 10^6 parallel chips, i.e. 10^{12} keys per second, the whole key space takes $2^{56}/10^{12} \approx 72\,058$ seconds ≈ **20 hours** (10 hours on average, since the key is found halfway on average). Networked computers simulate parallel hardware: the slides quote 3500 Internet computers finding an RSA Laboratories challenge key in 120 days, so 42 000 machines would need $3500 \times 120/42\,000 = 10$ days.

Weak keys. A **weak key** is one for which all 16 round keys are the same. This happens when, after the parity drop, each 28-bit half is all 0s or all 1s, because rotating such a half changes nothing. There are 4 weak keys (verified by script: each gives a single distinct round key):

Key before parity drop (64 bits)	Actual key: C_0, D_0 (hex)
0101 0101 0101 0101	0000000 0000000
1F1F 1F1F 0E0E 0E0E	0000000 FFFFFFF
E0E0 E0E0 F1F1 F1F1	FFFFFFF 0000000
FEFE FEFE FEFE FEFE	FFFFFFF FFFFFFF

With all round keys equal, the key order $K_{16}, \ldots, K_1$ is the same as $K_1, \ldots, K_{16}$, so the reverse cipher *is* the cipher: encrypting twice with a weak key returns the plaintext, $E_K(E_K(P)) = P$.

Semi-weak keys. A **semi-weak key** creates only 2 distinct round keys, each used 8 times (each half follows the alternating pattern 0101 … or 1010 …, or is constant, with at least one half alternating). They come in **6 pairs** (k_1, k_2) whose round-key sequences are each other's reverse, so $E_{k_2}(E_{k_1}(P)) = P$: one key of the pair decrypts what the other encrypts. The pairs (all checked by script) are:

First key	Second key	First key	Second key
01FE 01FE 01FE 01FE	FE01 FE01 FE01 FE01	1FFE 1FFE 0EFE 0EFE	FE1F FE1F FE0E FE0E
1FE0 1FE0 0EF1 0EF1	E01F E01F F10E F10E	011F 011F 010E 010E	1F01 1F01 0E01 0E01
01E0 01E0 01F1 01F1	E001 E001 F101 F101	E0FE E0FE F1FE F1FE	FEE0 FEE0 FEF1 FEF1

Possible weak keys. Forouzan lists **48 possible weak keys**, which create only 4 distinct round keys, each used 4 times (halves repeating with period 4, such as 0011 0011 ...). Since 2^{56} keys contain only $4+12+48 = 64$ such keys, a randomly chosen key is weak with probability 2^{-50}, so these keys matter only if they are chosen on purpose.

Key complement. Let $\bar{x}$ be the bitwise complement. Then $E_{\bar{K}}(\bar{P}) = \overline{E_K(P)}$. Proof: complementing K complements every round key, and complementing the data complements R_{i-1} and hence its expansion; then $E(\bar{R})\oplus\bar{K}_i = E(R)\oplus K_i$, so the S-box inputs and f are unchanged, and $\bar{L}\oplus f = \overline{L\oplus f}$, so every half stays complemented through all rounds (checked by script). As a result, a chosen-plaintext attacker who has $C_1 = E_K(P)$ and $C_2 = E_K(\bar{P})$ can test two keys with one encryption (K and $\bar{K}$), halving the brute-force work to 2^{55}.

30.8 Multiple DES

The obvious fix for a short key is to encrypt several times with different keys. This only helps because DES is **not a group**: if for every K_1, K_2 there were a K_3 with $E_{K_2}(E_{K_1}(P)) = E_{K_3}(P)$, double encryption would just be single DES with another key. It has been shown that no such K_3 exists in general, so cascading DES really produces new mappings.

Double DES. $C = E_{K_2}(E_{K_1}(P))$ and $P = D_{K_1}(D_{K_2}(C))$, with a 112-bit key (K_1, K_2). One would expect 2^{112} trials for a brute-force attack, but the **meet-in-the-middle attack** (a known-plaintext attack) needs only about 2^{57}. The derivation starts from the middle text M, the value between the two encryptions:

$$M = E_{K_1}(P) \quad \text{and also} \quad M = D_{K_2}(C).$$

Given one known pair (P, C), Eve encrypts P under all 2^{56} values of K_1 and stores the results M in a table sorted by M. She then decrypts C under all 2^{56} values of K_2 and looks up each $D_{K_2}(C)$ in the table. Every match gives a candidate pair (K_1, K_2), since it makes both equations hold. The cost is $2^{56}+2^{56} = 2^{57}$ DES operations (plus memory for 2^{56} entries), instead of $2^{56}\times 2^{56} = 2^{112}$, because the two keys are searched *one after the other* rather than in all combinations. Some matches are false: 2^{112} key pairs map to only 2^{64} middle values, so about $2^{112}/2^{64} = 2^{48}$ pairs fit one (P, C) by chance; testing the candidates on a second known pair (P', C') leaves about $2^{48}/2^{64} < 1$ false survivor, so the right key is identified. Double DES raises the work only from 2^{56} to 2^{57}, not to 2^{112}.

Triple DES with two keys. $C = E_{K_1}(D_{K_2}(E_{K_1}(P)))$ and $P = D_{K_1}(E_{K_2}(D_{K_1}(C)))$, the **EDE** (encrypt–decrypt–encrypt) form. The key is 112 bits, and a meet-in-the-middle attack now has to split three stages into one and two, which leaves a 2^{112} search on one side, so the attack no longer helps. The middle stage is a *decryption* for compatibility: with $K_1 = K_2$ the first two stages cancel and 3DES becomes single DES, so a 3DES device can talk to an old DES device (checked by script). Two-key 3DES is used in PEM and in the financial standard ANS X9.17 / ISO 8732.

Triple DES with three keys. $C = E_{K_3}(D_{K_2}(E_{K_1}(P)))$ with a 168-bit key. Meet-in-the-middle reduces its effective strength to about 2^{112}, which is still far beyond brute force; it is used in PGP and S/MIME.

30.9 Security of DES in one line each

- **Brute force:** 2^{56} keys (2^{55} with the complement property), feasible with special hardware or networked computers.
- **Differential cryptanalysis:** DES can be broken with 2^{47} chosen plaintexts; its S-boxes were evidently designed against this attack, which is why it is no better than brute force in practice.
- **Linear cryptanalysis:** DES can be broken with 2^{43} known plaintexts, a newer attack the designers did not anticipate.

Corrections to the notes / slides

- The slides' **substitute** pseudocode (L8 slide 16, as in Forouzan) declares inBlock[32], outBlock[48]; the sizes are swapped (48 bits in, 32 bits out). It also indexes inBlock[$i \times 6 + 1$] with $i = 1..8$, which reaches position 49 for $i = 8$; the group offset must be $(i - 1) \times 6$ (and $(i - 1) \times 4$ for outBlock), as in the corrected version in the Q6 answer (verified by script).
- Semi-weak key table (L8 slide 23), row 3: `01E0 01E1 01F1 01F1` should be `01E0 01E0 01F1 01F1`. `E1` differs from `E0` only in the parity bit, so it is the same 56-bit key, but it violates odd parity.
- Key-size slide (L8 slide 21): "In 1977 a team of researchers used 3500 computers ... RSA Laboratories" should read 1997 (RSA's DES Challenge); DES itself was only published in 1977.

31 Advanced Encryption Standard (AES)

Why DES had to be replaced. By the late 1990s the 56-bit DES key was within reach of brute force: a 2^{56} search is about 7.2×10^{16} trials, and a purpose-built machine (EFF, 1998) found a DES key in a few days. Triple DES fixes the key length but runs three DES encryptions per block, which is slow in software, and it keeps DES's 64-bit block, which is too small for encrypting large volumes of data under one key. NIST therefore ran an open competition, judged on three criteria (*security, cost, implementation*), and in 2000 chose **Rijndael** by Daemen and Rijmen. It was published as the **Advanced Encryption Standard** (FIPS 197) in December 2001.

What kind of cipher it is. AES is a symmetric-key, non-Feistel block cipher that encrypts 128-bit blocks with a 128-, 192- or 256-bit key in 10, 12 or 14 rounds. Non-Feistel means that, unlike DES, the round does not split the block into halves and hide a one-way function inside a swap. Instead every step of the round is itself invertible, and decryption runs the inverse steps in reverse order. Such a design is called a substitution–permutation network: substitution (SubBytes) gives confusion, and permutation plus mixing (ShiftRows, MixColumns) give diffusion.

31.1 Parameters and data units

The block is always 128 bits and every round key is always 128 bits, whatever the cipher-key length. Only the number of rounds N_r depends on the key size, because a longer key needs more rounds before every key bit has influenced every state bit. Since one round key is added before round 1 and one in each of the N_r rounds, the number of round keys is $N_r + 1$, and each round key is 4 words.

Version	Key (bits)	Rounds N_r	Round keys $N_r + 1$	Key words $4(N_r + 1)$	Brute force
AES-128	128	10	11	44 (w_0–w_{43})	2^{128}
AES-192	192	12	13	52 (w_0–w_{51})	2^{192}
AES-256	256	14	15	60 (w_0–w_{59})	2^{256}

Data units. AES names five units. A *bit* is a binary digit; a *byte* is 8 bits and is treated as an element of GF(2^8), written as two hex digits; a *word* is 4 bytes (32 bits), seen either as a row or as a column of 4 bytes; a *block* is 16 bytes (128 bits), $b_0 b_1 \ldots b_{15}$; and the *state* is the block arranged as a 4×4 matrix of bytes $s_{r,c}$, which is what every transformation actually acts on. Each column of the state is one word, so the state is also four words $w_0 w_1 w_2 w_3$ side by side.

Block to state. The 16 bytes fill the state **column by column**: bytes $b_0..b_3$ form column 0 (top to bottom), $b_4..b_7$ column 1, and so on, so that

$$s_{r,c} = b_{r+4c}, \qquad \text{equivalently} \qquad b_i \to s_{i \bmod 4,\, \lfloor i/4 \rfloor}.$$

The reverse mapping reads the state column by column to get the output block. This column-major order matters because MixColumns and AddRoundKey work on columns, so four consecutive input bytes are processed together.

Trap: filling the state row-wise

Writing the 16 bytes *across* the rows is the most common slip. The first four bytes go *down* column 0. With the wrong order every later step (ShiftRows, MixColumns, the key words as columns) gives a wrong answer.

WORKED EXAMPLE : text block to state (slide Example 1, from Forouzan)

Q. Show the state for the text block "AES uses a matrix", letters coded A = 00 to Z = 25 in hex, padding with Z.

A. Removing spaces gives 14 letters, so two bogus Z's make 16: AESUSESAMATRIXZZ. Each letter's number is written in **hex** (for example M = 12 = 0C, R = 17 = 11, X = 23 = 17, Z = 25 = 19):

Text	A	E	S	U	S	E	S	A	M	A	T	R	I	X	Z	Z
Hex	00	04	12	14	12	04	12	00	0C	00	13	11	08	17	19	19

Filling column by column (first four bytes down column 0):

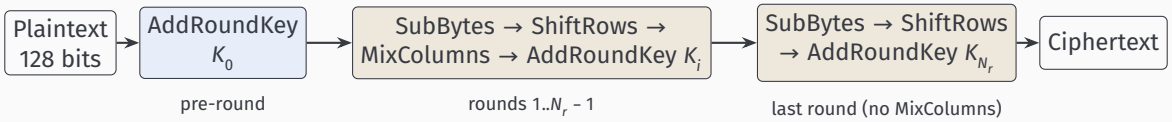
$$\text{State} = \begin{pmatrix} 00 & 12 & 0C & 08 \\ 04 & 04 & 00 & 17 \\ 12 & 12 & 13 & 19 \\ 14 & 00 & 11 & 19 \end{pmatrix}$$

Check: $s_{1,3} = b_{1+12} = b_{13} = 17$ (the X). The slide prints 23 here; see the corrections box.

31.2 Round structure

Encryption of AES-128 is one key addition followed by 10 rounds:

1. **Pre-round:** AddRoundKey with round key $K_0 = w_0..w_3$ (the cipher key itself).
2. **Rounds 1 to $N_r - 1$:** SubBytes, ShiftRows, MixColumns, AddRoundKey (with $K_i = w_{4i}..w_{4i+3}$).
3. **Last round N_r:** SubBytes, ShiftRows, AddRoundKey, **without MixColumns**.



Why a key addition comes first. Every step except AddRoundKey is public and keyless. If the cipher began with SubBytes, an attacker who knows the plaintext could compute that step himself, so it would add work but no secrecy. Adding the key first (key whitening) means that from the very first non-linear step onward, nothing is computable without the key.

Why the last round has no MixColumns. MixColumns is linear over GF(2^8), so $\mathrm{MC}(Y) \oplus K = \mathrm{MC}\big(Y \oplus \mathrm{MC}^{-1}(K)\big)$. If the last round ended with MixColumns and then AddRoundKey, an attacker could apply InvMixColumns to the ciphertext and obtain $Y \oplus \mathrm{MC}^{-1}(K_{N_r})$, which is just the same cipher without the final MixColumns under a different (equally unknown) key. The final MixColumns therefore adds no security. Leaving it out costs nothing and makes the decryption algorithm have the same shape as encryption, which allows the "equivalent inverse cipher" to reuse the same hardware.

★ MUST KNOW

AES: block 128, key 128/192/256, rounds 10/12/14, round keys $N_r + 1$ (always 128 bits each), $4(N_r + 1)$ words. Pre-round AddRoundKey; rounds 1..$N_r - 1$: **SubBytes, ShiftRows, MixColumns, AddRoundKey**; last round drops MixColumns. The state is filled **column by column**, $s_{r,c} = b_{r+4c}$.

31.3 SubBytes (substitution) ★

▸ **Likely this year:** "Find SubBytes of `0x53`" or "apply SubBytes to the given state using the S-box"; or derive one S-box entry from the GF(2^8) inverse and the affine transform.

Problem and idea. A cipher built only from XORs, shifts and matrix multiplications is linear, and a linear cipher falls to solving a system of equations. SubBytes is the one non-linear step in AES. Each of the 16 bytes is replaced independently by looking it up in one fixed 16 × 16 table, the S-box. Because the same table is used for every byte, equal input bytes always give equal output bytes; SubBytes is an *intrabyte* transformation, since a byte's output depends on that byte alone.

Lookup rule. Write the byte as two hex digits xy. The **left digit x selects the row** and the **right digit y selects the column**; the entry there is the output. For example `53`: row 5, column 3 gives `ED`, and `00`: row 0, column 0 gives `63`.

$x\backslash y$	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

AES S-box (FIPS-197), generated by script from the definition below and checked entry by entry.

How the S-box is built. The table is not arbitrary; each entry is computed in two steps from the byte a viewed as a polynomial in GF(2^8) with modulus $m(x) = x^8 + x^4 + x^3 + x + 1$ (11B).

1. **Multiplicative inverse:** $b = a^{-1}$ in GF(2^8), with `00` mapped to `00` (it has no inverse). The inverse map is highly non-linear, which is what resists linear and differential cryptanalysis. This is why AES works in GF(2^8) and not in $\mathbb{Z}_{256}$: in $\mathbb{Z}_{256}$ every even byte lacks an inverse, so the map would not be a permutation.

2. **Affine transformation over GF(2):** with $c = 63 = 0110\,0011$,

$$b'_i = b_i \oplus b_{(i+4) \bmod 8} \oplus b_{(i+5) \bmod 8} \oplus b_{(i+6) \bmod 8} \oplus b_{(i+7) \bmod 8} \oplus c_i, \qquad i = 0..7.$$

This step breaks the simple algebraic form $a \mapsto a^{-1}$ and removes its fixed points (00 ↦ 00 and 01 ↦ 01). As a result the AES S-box has no fixed point ($S(a) \neq a$ for all 256 bytes) and none with $S(a) = \bar{a}$.

InvSubBytes uses the inverse table: undo the affine map, then take the GF(2^8) inverse. Without that table, find the output in the S-box and read off its row and column (for example ED sits at row 5, column 3, so InvSubBytes(ED) = 53).

WORKED EXAMPLE : one S-box entry from the definition

Q. Show that SubBytes(53) = ED by computing it, not by looking it up.

A. *Step 1.* $53 = x^6 + x^4 + x + 1$. Its inverse modulo $m(x)$, by the extended Euclidean algorithm in GF(2^8), is $CA = x^7 + x^6 + x^3 + x$ (check: $53 \cdot CA = 01$). So $b = CA = 1100\,1010$, that is $b_7..b_0 = 1, 1, 0, 0, 1, 0, 1, 0$.
Step 2. Apply $b'_i = b_i \oplus b_{i+4} \oplus b_{i+5} \oplus b_{i+6} \oplus b_{i+7} \oplus c_i$ (indices mod 8, $c = 63$, so $c_0 = c_1 = c_5 = c_6 = 1$):

i	b_i	b_{i+4}	b_{i+5}	b_{i+6}	b_{i+7}	c_i	b'_i
0	0	0	0	1	1	1	**1**
1	1	0	1	1	0	1	**0**
2	0	1	1	0	1	0	**1**
3	1	1	0	1	0	0	**1**
4	0	0	1	0	1	0	**0**
5	0	1	0	1	0	1	**1**
6	1	0	1	0	0	1	**1**
7	1	1	0	0	1	0	**1**

Reading $b'_7..b'_0 = 1110\,1101$ gives **SubBytes(53) = ED**, which matches row 5, column 3 of the table.

WORKED EXAMPLE : SubBytes of a state (slide Example 2)

Q. Apply SubBytes to the state of Example 1.

A. Each byte is looked up independently (row = left digit, column = right digit): 00→63, 04→F2, 12→C9, 14→FA, 0C→FE, 13→7D, 11→82, 08→30, 17→F0, 19→D4.

$$\begin{pmatrix} 00 & 12 & 0C & 08 \\ 04 & 04 & 00 & 17 \\ 12 & 12 & 13 & 19 \\ 14 & 00 & 11 & 19 \end{pmatrix} \xrightarrow{\text{SubBytes}} \begin{pmatrix} 63 & C9 & FE & 30 \\ F2 & F2 & 63 & \mathbf{F0} \\ C9 & C9 & 7D & D4 \\ FA & 63 & 82 & D4 \end{pmatrix}$$

The two 04's in row 1 both become F2, showing that SubBytes alone does not hide repeated bytes; ShiftRows and MixColumns are needed for that.

Single-bit changes spread: 5A and 5B differ in one bit, but S(5A) = BE and S(5B) = 39 differ in four bits (BE ⊕ 39 = 87 = 1000 0111).

31.4 ShiftRows (permutation) ★

★ **In the papers:** 2025–26 Q4(a) (part of 6 marks): ShiftRows on a given state. Solved in full after AddRoundKey below.

Problem and idea. SubBytes changes each byte on its own, and MixColumns (next) mixes only the four bytes inside one column. Without a step that moves bytes *between* columns, the four columns would never interact, and AES would be four independent 32-bit ciphers running side by side, each attackable separately. ShiftRows is that step: it permutes bytes so that each column's four bytes end up in four different columns.

Mechanism. Row r of the state is rotated cyclically left by r bytes: row 0 is unchanged, row 1 moves left by 1, row 2 by 2, row 3 by 3. In symbols $s'_{r,c} = s_{r,(c+r) \bmod 4}$. The permutation is at the *byte* level: bytes change position but the bits inside a byte are untouched, unlike DES, whose P-boxes permute single bits. Since the four bytes of input column c lie in rows 0, 1, 2, 3, they are shifted by 0, 1, 2, 3 places and land in four different columns; after the next MixColumns every output column then depends on all four input columns. InvShiftRows rotates row r *right* by r bytes.

$$\begin{pmatrix} s_{00} & s_{01} & s_{02} & s_{03} \\ s_{10} & s_{11} & s_{12} & s_{13} \\ s_{20} & s_{21} & s_{22} & s_{23} \\ s_{30} & s_{31} & s_{32} & s_{33} \end{pmatrix} \xrightarrow{\text{ShiftRows}} \begin{pmatrix} s_{00} & s_{01} & s_{02} & s_{03} \\ s_{11} & s_{12} & s_{13} & s_{10} \\ s_{22} & s_{23} & s_{20} & s_{21} \\ s_{33} & s_{30} & s_{31} & s_{32} \end{pmatrix}$$

Applied to the SubBytes output of Example 2 (slide Example 3):

$$\begin{pmatrix} 63 & C9 & FE & 30 \\ F2 & F2 & 63 & F0 \\ C9 & C9 & 7D & D4 \\ FA & 63 & 82 & D4 \end{pmatrix} \xrightarrow{\text{ShiftRows}} \begin{pmatrix} 63 & C9 & FE & 30 \\ F2 & 63 & F0 & F2 \\ 7D & D4 & C9 & C9 \\ D4 & FA & 63 & 82 \end{pmatrix}$$

31.5 MixColumns (mixing) ★

▸ **Likely this year:** "Perform MixColumns on one column" (one or all four output bytes), or "show that the given column maps to …".

Problem and idea. SubBytes is intrabyte and ShiftRows only exchanges bytes, so neither makes a byte depend on its neighbours. MixColumns is the *interbyte* step: it replaces each column by a combination of all four of its bytes, so that changing one input byte changes all four output bytes. To guarantee that each new byte differs from the others even when all four inputs are equal, each input is first multiplied by a different constant and then the products are added. This is a matrix multiplication over GF(2^8), where "add" is XOR and "multiply" is polynomial multiplication modulo 11B:

$$\begin{pmatrix} s'_0 \\ s'_1 \\ s'_2 \\ s'_3 \end{pmatrix} = \begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix} \begin{pmatrix} s_0 \\ s_1 \\ s_2 \\ s_3 \end{pmatrix}, \qquad \text{InvMixColumns: } \begin{pmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{pmatrix}.$$

Each row of the matrix is the row above rotated right by one. Written out, $s'_0 = 02 \cdot s_0 \oplus 03 \cdot s_1 \oplus s_2 \oplus s_3$, $s'_1 = s_0 \oplus 02 \cdot s_1 \oplus 03 \cdot s_2 \oplus s_3$, $s'_2 = s_0 \oplus s_1 \oplus 02 \cdot s_2 \oplus 03 \cdot s_3$, $s'_3 = 03 \cdot s_0 \oplus s_1 \oplus s_2 \oplus 02 \cdot s_3$. The two matrices multiply to the identity in GF(2^8) (checked by script), which is why InvMixColumns undoes MixColumns.

Why the constants 01, 02, 03. They make encryption cheap, because multiplying by these needs only a shift and XORs. Multiplying by 02 is multiplying by x: shift the byte left by one bit, and if the bit shifted out (the old top bit b_7) was 1, the result has an x^8 term, which is reduced by XOR with 1B (since $x^8 \equiv x^4 + x^3 + x + 1$). This operation is called **xtime**. Multiplying by 03 = $x + 1$ is then $(02 \cdot b) \oplus b$. The matrix is also chosen so that the total number of non-zero bytes in an input column plus its output column is at least 5 (branch number 5), so one changed input byte always changes all four output bytes.

★ MUST KNOW : xtime

$02 \cdot b = (b \ll 1)$, then XOR 1B if the top bit of b was 1. $\quad 03 \cdot b = (02 \cdot b) \oplus b$. $\quad 01 \cdot b = b$.

WORKED EXAMPLE : one column (FIPS-197 test column)

Q. Apply MixColumns to the column $(DB, 13, 53, 45)^T$.

A. First the products by 02 and 03 of every input byte:

b	binary	shift left	top bit 1?	$02 \cdot b$ / $03 \cdot b = (02 \cdot b) \oplus b$
DB	1101 1011	1011 0110 = B6	yes: B6 ⊕ 1B	AD / 76
13	0001 0011	0010 0110 = 26	no	26 / 35
53	0101 0011	1010 0110 = A6	no	A6 / F5
45	0100 0101	1000 1010 = 8A	no	8A / CF

Then each output byte is a row of the matrix times the column, XORed left to right:

$$s'_0 = 02{\cdot}DB \oplus 03{\cdot}13 \oplus 53 \oplus 45 = AD \oplus 35 \oplus 53 \oplus 45 = 98 \oplus 53 \oplus 45 = CB \oplus 45 = \mathbf{8E}$$
$$s'_1 = DB \oplus 02{\cdot}13 \oplus 03{\cdot}53 \oplus 45 = DB \oplus 26 \oplus F5 \oplus 45 = FD \oplus F5 \oplus 45 = 08 \oplus 45 = \mathbf{4D}$$
$$s'_2 = DB \oplus 13 \oplus 02{\cdot}53 \oplus 03{\cdot}45 = DB \oplus 13 \oplus A6 \oplus CF = C8 \oplus A6 \oplus CF = 6E \oplus CF = \mathbf{A1}$$
$$s'_3 = 03{\cdot}DB \oplus 13 \oplus 53 \oplus 02{\cdot}45 = 76 \oplus 13 \oplus 53 \oplus 8A = 65 \oplus 53 \oplus 8A = 36 \oplus 8A = \mathbf{BC}$$

Answer: $(DB, 13, 53, 45) \to (8E, 4D, A1, BC)$, the FIPS-197 test value. Check: every output byte changed, as the branch-number argument predicts.

A second check comes from the exam state below: column 0 after ShiftRows is $(87, 6E, 46, A6)$. With $02 \cdot 87 = 0E \oplus 1B = 15$, $03 \cdot 6E = DC \oplus 6E = B2$, the first output byte is $15 \oplus B2 \oplus 46 \oplus A6 = 47$, and the whole column becomes $(47, 37, 94, ED)$, which is exactly column 0 of the state given in part (b) of that question (Stallings' figures chain ShiftRows → MixColumns → AddRoundKey).

31.6 AddRoundKey (key adding) ★

★ **In the papers:** 2025–26 Q4(b) (part of 6 marks): AddRoundKey of a given state with a given key matrix.

Problem and idea. The three steps above are fixed and public, so on their own they would be a keyless scrambling that anyone can undo. AddRoundKey is the only step that uses the key: it XORs the 128-bit round key into the state. It proceeds column by column, like MixColumns, but where MixColumns multiplies each column by a constant matrix, AddRoundKey *adds* a key word to it: in round i, column c becomes $s_c \oplus w_{4i+c}$. Equivalently, write the round key as a 4×4 matrix whose columns are the four words and XOR the two matrices entry by entry.

Inverse. Addition and subtraction in GF(2^8) are both XOR, and $x \oplus k \oplus k = x$, so AddRoundKey is its own inverse; decryption uses the same operation with the same round key.

Doing the XOR by hand. XOR one hex digit at a time, through its 4-bit binary form: $4 \oplus A = 0100 \oplus 1010 = 1110 = E$. Useful shortcuts: $x \oplus x = 0$, $x \oplus 0 = x$, $x \oplus F$ flips all four bits.

WORKED EXAMPLE : 2025–26 Q4 (6 marks), Stallings' AES example

Q. In AES, the state is given below. (a) Show the ShiftRows transformation. (b) After MixColumns, assume the state and the round key are as given. Perform AddRoundKey.

A. (a) Row r is rotated left by r bytes: row 0 unchanged; row 1 `EC 6E 4C 90` → `6E 4C 90 EC`; row 2 `4A C3 46 E7` → `46 E7 4A C3`; row 3 `8C D8 95 A6` → `A6 8C D8 95`.

$$\begin{pmatrix} 87 & F2 & 4D & 97 \\ EC & 6E & 4C & 90 \\ 4A & C3 & 46 & E7 \\ 8C & D8 & 95 & A6 \end{pmatrix} \xrightarrow{\text{ShiftRows}} \begin{pmatrix} 87 & F2 & 4D & 97 \\ \mathbf{6E} & \mathbf{4C} & \mathbf{90} & \mathbf{EC} \\ \mathbf{46} & \mathbf{E7} & \mathbf{4A} & \mathbf{C3} \\ \mathbf{A6} & \mathbf{8C} & \mathbf{D8} & \mathbf{95} \end{pmatrix}$$

(b) AddRoundKey is the byte-wise XOR of the state and the key matrix, $s'_{r,c} = s_{r,c} \oplus k_{r,c}$:

$$\begin{pmatrix} 47 & 40 & A3 & 4C \\ 37 & D4 & 70 & 9F \\ 94 & E4 & 3A & 42 \\ ED & A5 & A6 & BC \end{pmatrix} \oplus \begin{pmatrix} AC & 19 & 28 & 57 \\ 77 & FA & D1 & 5C \\ 66 & DC & 29 & 00 \\ F3 & 21 & 41 & 6A \end{pmatrix} = \begin{pmatrix} \mathbf{EB} & \mathbf{59} & \mathbf{8B} & \mathbf{1B} \\ \mathbf{40} & \mathbf{2E} & \mathbf{A1} & \mathbf{C3} \\ \mathbf{F2} & \mathbf{38} & \mathbf{13} & \mathbf{42} \\ \mathbf{1E} & \mathbf{84} & \mathbf{E7} & \mathbf{D6} \end{pmatrix}$$

Working, digit by digit (left digits, then right digits):

```
47⊕AC: 4⊕A=E, 7⊕C=B → EB   40⊕19: 5, 9 → 59   A3⊕28: 8, B → 8B   4C⊕57: 1, B → 1B
37⊕77: 4, 0 → 40            D4⊕FA: 2, E → 2E   70⊕D1: A, 1 → A1   9F⊕5C: C, 3 → C3
94⊕66: F, 2 → F2            E4⊕DC: 3, 8 → 38   3A⊕29: 1, 3 → 13   42⊕00: 4, 2 → 42
ED⊕F3: 1, E → 1E            A5⊕21: 8, 4 → 84   A6⊕41: E, 7 → E7   BC⊕6A: D, 6 → D6
```

For example $47 \oplus AC$: $0100\,0111 \oplus 1010\,1100 = 1110\,1011 = EB$. Check: XORing the answer with the key again returns the given state (for instance $EB \oplus AC = 47$), because AddRoundKey is self-inverse.

Trap: AddRoundKey

Do not add bytes as integers ($47 + AC = F3$ is wrong; the answer is $47 \oplus AC = EB$). And in the ShiftRows part, shift *left*, by the row index starting from 0; rotating right, or shifting row 1 by 2, is the usual slip.

31.7 Key expansion (AES-128) ★

▸ **Likely this year:** "Given the cipher key, find w_4–w_7 (the first round key)", Forouzan/Stallings style, using the S-box table.

Problem. AES-128 needs 11 round keys of 128 bits, that is 44 words $w_0..w_{43}$, but the user supplies only one 128-bit key. The expansion must be cheap, must spread every key bit into many round-key bits, and must not be linear; otherwise knowing part of one round key would give the others by simple algebra, and keys with patterns (like DES's weak keys) would produce repeating round keys.

Mechanism. The first four words are the cipher key itself, taken 4 bytes at a time: $w_0 = k_0k_1k_2k_3$, $w_1 = k_4..k_7$, and so on. Each later word is the word four positions back XORed with the previous word, except that every fourth word passes the previous word through a non-linear function first:

$$w_i = \begin{cases} w_{i-4} \oplus w_{i-1}, & i \bmod 4 \neq 0, \\ w_{i-4} \oplus t_i, \quad t_i = \text{SubWord}\big(\text{RotWord}(w_{i-1})\big) \oplus \text{RCon}[i/4], & i \bmod 4 = 0. \end{cases}$$

- **RotWord** rotates the 4 bytes of a word left by one: $[a_0, a_1, a_2, a_3] \to [a_1, a_2, a_3, a_0]$. It is ShiftRows applied to a single row.
- **SubWord** applies the S-box to each of the 4 bytes. It is SubBytes on a single word and supplies the non-linearity.
- **RCon** (round constant) is a word whose rightmost three bytes are zero and whose left byte is $x^{j-1} \bmod m(x)$ for round j. Each value is `02` times the previous one in GF(2^8), so `01, 02, 04, 08, 10, 20, 40, 80`, then

$02 \cdot 80$ overflows and is reduced to $00 \oplus 1B = 1B$, and $02 \cdot 1B = 36$. RCon is different in every round, which breaks the symmetry between rounds, so that no key gives identical round keys.

Round j	1	2	3	4	5	6	7	8	9	10
RCon[j]	01	02	04	08	10	20	40	80	1B	36

(each followed by 00 00 00)

Round key K_j is the four words $w_{4j}, w_{4j+1}, w_{4j+2}, w_{4j+3}$, placed as the *columns* of a 4×4 matrix, so that AddRoundKey can XOR column c with w_{4j+c}. For AES-192 and AES-256 the rule is the same with 6 or 8 words per step instead of 4 (t is computed when $i \bmod 6 = 0$ or $i \bmod 8 = 0$); AES-256 additionally applies SubWord alone when $i \bmod 8 = 4$.

WORKED EXAMPLE : first round key (FIPS-197 / Stallings key)

Q. The AES-128 cipher key is 2B7E1516 28AED2A6 ABF71588 09CF4F3C. Find w_4, w_5, w_6, w_7.
A. w_0 = 2B7E1516, w_1 = 28AED2A6, w_2 = ABF71588, w_3 = 09CF4F3C.
$i = 4$ is a multiple of 4, so compute t_4 from w_3:

RotWord(w_3)	09 CF 4F 3C → CF 4F 3C 09
SubWord	S(CF) = 8A (row C, col F), S(4F) = 84, S(3C) = EB, S(09) = 01 ⇒ 8A84EB01
⊕ RCon[1]	8A84EB01 ⊕ 01000000 = t_4 = 8B84EB01

Then

$$w_4 = w_0 \oplus t_4 = \texttt{2B7E1516} \oplus \texttt{8B84EB01} = \mathbf{A0FAFE17}$$
$$w_5 = w_1 \oplus w_4 = \texttt{28AED2A6} \oplus \texttt{A0FAFE17} = \mathbf{88542CB1}$$
$$w_6 = w_2 \oplus w_5 = \texttt{ABF71588} \oplus \texttt{88542CB1} = \mathbf{23A33939}$$
$$w_7 = w_3 \oplus w_6 = \texttt{09CF4F3C} \oplus \texttt{23A33939} = \mathbf{2A6C7605}$$

As a matrix (words as columns) the round-1 key is

$$K_1 = \begin{pmatrix} \texttt{A0} & \texttt{88} & \texttt{23} & \texttt{2A} \\ \texttt{FA} & \texttt{54} & \texttt{A3} & \texttt{6C} \\ \texttt{FE} & \texttt{2C} & \texttt{39} & \texttt{76} \\ \texttt{17} & \texttt{B1} & \texttt{39} & \texttt{05} \end{pmatrix}.$$

Check: these are the FIPS-197 Appendix A values; continuing the same way gives t_8 = 52386BE5 (RotWord(2A6C7605) = 6C76052A, SubWord = 50386BE5, RCon = 02), w_8 = F2C295F2, and finally w_{43} = B6630CA6.

31.8 One complete round: FIPS-197 example

Putting the four steps together on the standard test vector (plaintext 3243F6A8 885A308D 313198A2 E0370734, the key above) shows what a full round looks like; the mock paper's AES question (Q8) takes its matrices from this trace. The plaintext fills the state column by column, the pre-round XORs it with K_0 (the cipher key), and round 1 then gives:

$$\underbrace{\begin{pmatrix} \texttt{19} & \texttt{A0} & \texttt{9A} & \texttt{E9} \\ \texttt{3D} & \texttt{F4} & \texttt{C6} & \texttt{F8} \\ \texttt{E3} & \texttt{E2} & \texttt{8D} & \texttt{48} \\ \texttt{BE} & \texttt{2B} & \texttt{2A} & \texttt{08} \end{pmatrix}}_{\text{after } K_0} \xrightarrow{\text{Sub}} \begin{pmatrix} \texttt{D4} & \texttt{E0} & \texttt{B8} & \texttt{1E} \\ \texttt{27} & \texttt{BF} & \texttt{B4} & \texttt{41} \\ \texttt{11} & \texttt{98} & \texttt{5D} & \texttt{52} \\ \texttt{AE} & \texttt{F1} & \texttt{E5} & \texttt{30} \end{pmatrix} \xrightarrow{\text{Shift}} \begin{pmatrix} \texttt{D4} & \texttt{E0} & \texttt{B8} & \texttt{1E} \\ \texttt{BF} & \texttt{B4} & \texttt{41} & \texttt{27} \\ \texttt{5D} & \texttt{52} & \texttt{11} & \texttt{98} \\ \texttt{30} & \texttt{AE} & \texttt{F1} & \texttt{E5} \end{pmatrix} \xrightarrow{\text{Mix}} \begin{pmatrix} \texttt{04} & \texttt{E0} & \texttt{48} & \texttt{28} \\ \texttt{66} & \texttt{CB} & \texttt{F8} & \texttt{06} \\ \texttt{81} & \texttt{19} & \texttt{D3} & \texttt{26} \\ \texttt{E5} & \texttt{9A} & \texttt{7A} & \texttt{4C} \end{pmatrix} \xrightarrow{\oplus K_1} \begin{pmatrix} \texttt{A4} & \texttt{68} & \texttt{6B} & \texttt{02} \\ \texttt{9C} & \texttt{9F} & \texttt{5B} & \texttt{6A} \\ \texttt{7F} & \texttt{35} & \texttt{EA} & \texttt{50} \\ \texttt{F2} & \texttt{2B} & \texttt{43} & \texttt{49} \end{pmatrix}$$

After all ten rounds the ciphertext is 3925841D 02DC09FB DC118597 196A0B32; a Python AES implementation reproduces this and decrypts it back.

31.9 Decryption

Since every step is invertible, decryption applies the inverse steps in reverse order with the round keys in reverse order (K_{N_r} first, K_0 last). In the original design (Forouzan's figure) this gives:

1. AddRoundKey with $K_{N_r} = w_{40}..w_{43}$.
2. Each of the first $N_r - 1$ decryption rounds: InvShiftRows, InvSubBytes, AddRoundKey, InvMixColumns.
3. Last decryption round: InvShiftRows, InvSubBytes, AddRoundKey with K_0 (no InvMixColumns).

InvShiftRows rotates right, InvSubBytes uses the inverse S-box, InvMixColumns uses the matrix [0E 0B 0D 09] and AddRoundKey is its own inverse. Because InvShiftRows and InvSubBytes commute (one moves bytes, the other changes each byte on its own) and InvMixColumns is linear, the steps can be reordered into the same

pattern as encryption (InvSubBytes, InvShiftRows, InvMixColumns, AddRoundKey) if the middle round keys are passed through InvMixColumns; this is the *equivalent inverse cipher*. Unlike DES, AES decryption is a different algorithm from encryption, and InvMixColumns, with its larger constants, is slower than MixColumns.

31.10 Analysis of AES

Brute force. A 128-bit key needs up to 2^{128} trials against DES's 2^{56}, so if DES can be broken in t seconds, AES-128 needs $2^{128}/2^{56} \cdot t = 2^{72}\,t$ seconds. The 192- and 256-bit versions widen the gap further. The best published shortcut (biclique attack, 2011) lowers the AES-128 cost only to about 2^{126}, which is still far out of reach.

No weak keys. DES has keys whose 16 round keys are all equal, so encrypting twice decrypts. In AES, SubWord makes the key schedule non-linear and RCon makes each round's step different, so no such keys are known.

Diffusion and statistical attacks. The steps work together to spread a single change over the whole block in two rounds. In round 1 a changed byte stays one byte through SubBytes and ShiftRows, then MixColumns spreads it to all 4 bytes of its column. In round 2 ShiftRows sends those 4 bytes to 4 different columns, and MixColumns spreads each of them over its column, so all 16 bytes have changed (checked by script: 4 bytes after one round, 16 after two). With 10 or more rounds the ciphertext shows no frequency pattern of the plaintext, and statistical analysis has failed.

Differential and linear cryptanalysis. AES was designed after these attacks were known; the S-box (GF(2^8) inverse) has low differential and linear probabilities, and the branch number of MixColumns forces many active S-boxes over a few rounds, so no such attack works on full AES.

Simplicity and cost. The steps are byte operations, table lookups and XORs, so AES is fast on cheap 8-bit processors with little memory as well as on 32-bit machines (where SubBytes, ShiftRows and MixColumns can be merged into table lookups), and many processors have AES instructions.

31.11 AES versus DES

	DES	AES
Structure	Feistel (round function need not be invertible)	non-Feistel SPN (every step invertible)
Block size	64 bits	128 bits
Key size	56 bits (64 with parity)	128, 192 or 256 bits
Rounds	16	10, 12 or 14
Round key	48 bits, 16 keys	128 bits, $N_r + 1$ keys
Works on	bits (expansion, P-boxes)	bytes in GF(2^8) (state matrix)
Non-linearity	eight 6-to-4-bit S-boxes	one 8-to-8-bit S-box (inverse + affine)
Permutation	bit-level P-boxes	byte-level ShiftRows
Decryption	same algorithm, keys reversed	inverse steps, keys reversed
Brute force	2^{56} (broken in practice)	2^{128} or more
Weak keys	4 weak, 12 semi-weak	none known

Corrections to the notes / slides

- **Example 1, letter X.** The slide writes X as 23. X is letter 23 in *decimal*, which is 17 in hex (the same slide correctly converts M = 12 → 0C and R = 17 → 11). With 17, SubBytes gives F0; the slide's 26 (in Examples 2 and 3) is $S(23)$. If the exam reuses the slide's state, say which value you used.
- **S-box, row B, column 1.** The slide's table prints CB; the correct FIPS-197 entry is $S(B1) = C8$. CB already appears at row 5, column 9 ($S(59) = CB$), and an S-box is a permutation, so it cannot appear twice. The inverse table on the next slide is right ($InvS(C8) = B1$). All other 255 entries of the slide match.
- **ShiftRows rows.** The slide numbers the rows 1 to 4 ("Row 1: no shifting, Row 2: 1-byte left shift, ..."), while the pseudocode and these notes number them 0 to 3. Either way the top row is not shifted and the bottom row moves left by 3.

32 Block Cipher Modes of Operation

A block cipher such as DES (§30) or AES (§31) is defined for exactly one n-bit block: $C = E_K(P)$ with $|P| = n$ (64 or 128 bits). Real messages are kilobytes to gigabytes long, have arbitrary length, and sometimes arrive one character at a time. A **mode of operation** is the rule that says how the block cipher is applied to such a message: how the message is cut into blocks, how successive blocks are linked, and what extra input (an initialisation vector or a counter) is used. NIST (SP 800-38A) standardises five confidentiality modes, ECB, CBC, CFB, OFB and CTR, and they are the content of this section.

Notation. The plaintext is split into blocks $P_1, P_2, \ldots, P_N$ and the ciphertext is $C_1, \ldots, C_N$; E_K and D_K are encryption and decryption of one block under key K; $\oplus$ is bitwise XOR. An **initialisation vector** (IV) is an extra n-bit block, known to both parties and normally sent in the clear with the ciphertext, that makes encryptions of the same message differ. A **nonce** (number used once) is a value that must never repeat under the same key.

32.1 What a mode must achieve ▸

▸ **Likely this year:** "Why are modes of operation needed? Classify them into block and stream modes" (2–3 marks).

Applying the cipher block by block is not enough, for three reasons. First, the same plaintext block always gives the same ciphertext block under one key, so any repetition in the message (headers, blank areas of an image, a field that recurs in every record) shows up in the ciphertext. Second, some applications cannot wait for a full block or cannot afford padding: a terminal sends one 8-bit character at a time, and a disk sector or a database field must keep its length. Third, the modes behave very differently when a ciphertext bit is damaged in transit, when blocks must be processed in parallel, or when a single block in the middle of a file must be read without decrypting everything before it.

The five modes fall into two families. In the **block modes** (ECB, CBC) the plaintext itself passes through E_K, so the message must be padded to a multiple of n bits (§29) and the receiver needs D_K. In the **stream modes** (CFB, OFB, CTR) the block cipher is used only to produce a pseudorandom **key stream**, which is XORed with the plaintext exactly as in a stream cipher. These modes need no padding, the ciphertext is as long as the plaintext, and both sides use only E_K, because the receiver regenerates the same key stream and XORs it away.

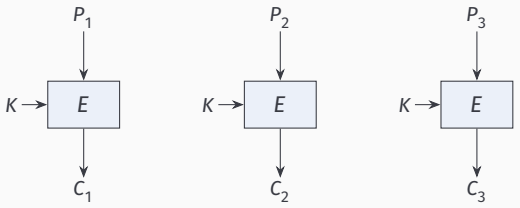
32.2 Electronic codebook (ECB) mode ▸

▸ **Likely this year:** "Explain ECB with a diagram; why is it insecure for long messages?" (3 marks).

In ECB each block is encrypted independently with the same key:

$$C_i = E_K(P_i), \qquad P_i = D_K(C_i), \qquad i = 1, \dots, N.$$

For a fixed key the mode is a huge lookup table that assigns one ciphertext block to every one of the 2^n plaintext blocks, hence the name codebook.



decryption: same picture with C_i on top, D in the box, P_i below

Security. Since equal plaintext blocks give equal ciphertext blocks, ECB leaks the pattern of the message: an encrypted bitmap image still shows the outline of the picture, because every run of identical pixel blocks becomes a run of identical ciphertext blocks. The blocks are also independent of their position, so Eve can delete, repeat or reorder whole ciphertext blocks without detection; if each record of a payroll file fills one block, she can copy the ciphertext block holding her colleague's salary into her own record (a **block replay** or substitution attack).

Advantages. ECB is the simplest mode. Encryption and decryption of different blocks are independent, so both can be done in parallel and any block can be decrypted on its own (random access). A transmission error in C_i garbles only P_i. **Disadvantages.** It leaks equality of blocks, it allows block substitution, and it needs padding. ECB is therefore used only for data shorter than one block that is random anyway, such as a session key encrypted under a master key.

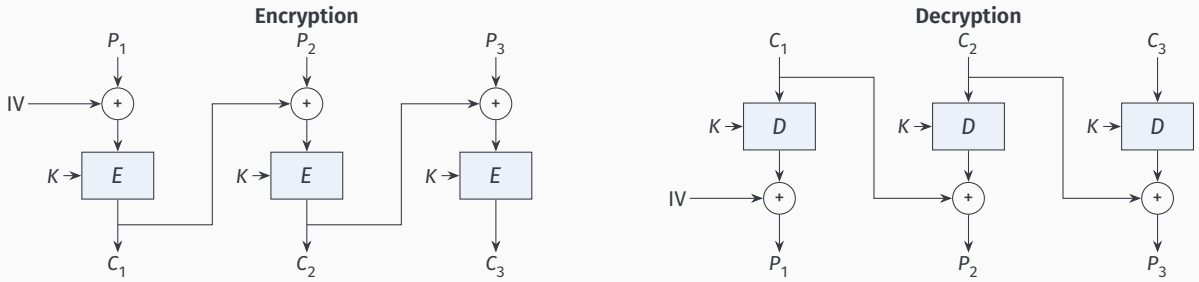
32.3 Cipher block chaining (CBC) mode ★▸

▸ **Likely this year:** "Draw and explain CBC encryption and decryption; how does an error propagate?" (5 marks).

Idea. To make equal plaintext blocks encrypt differently, CBC mixes each plaintext block with the *previous ciphertext block* before encrypting it. Since the previous ciphertext block depends on everything before it, C_i depends on $P_1, \dots, P_i$, and the first block is mixed with the IV:

$$C_0 = \mathrm{IV}, \qquad C_i = E_K(P_i \oplus C_{i-1}), \qquad P_i = D_K(C_i) \oplus C_{i-1}.$$

Why decryption works. Applying D_K to C_i undoes the encryption: $D_K(C_i) = D_K\big(E_K(P_i \oplus C_{i-1})\big) = P_i \oplus C_{i-1}$. The receiver already holds C_{i-1} (it is the previous ciphertext block, or the IV for $i = 1$), so XORing it in again cancels it and leaves P_i.



The IV. The IV must be known to the receiver, so it is sent with the ciphertext (it is not secret), but it must be **unpredictable**, i.e. chosen at random for every message. If the same IV were reused, two messages that start with the same block would have the same C_1, which leaks that they agree. A predictable IV is worse, because it allows a chosen-plaintext check of a guess: suppose an old message had $C_1 = E_K(\text{IV} \oplus P_1)$ and Eve wants to test whether $P_1 = G$. If she knows the IV′ of the next message and can get it to start with $P'_1 = \text{IV}' \oplus \text{IV} \oplus G$, then

$$C'_1 = E_K(\text{IV}' \oplus P'_1) = E_K(\text{IV} \oplus G),$$

which equals the old C_1 exactly when $G = P_1$ (this is the BEAST attack on TLS 1.0, whose IV was the last ciphertext block of the previous record). The IV must also arrive **unmodified**: since $P_1 = D_K(C_1) \oplus \text{IV}$, flipping bit j of the IV flips bit j of P_1 and nothing else, so an attacker who can change the IV can change chosen bits of the first block. Stallings therefore recommends protecting the IV, for example by sending it ECB-encrypted or under the message's MAC.

Error propagation (derivation). Suppose one bit of C_i is flipped in transit, giving C'_i.

- $P'_i = D_K(C'_i) \oplus C_{i-1}$: the decryption of a changed block is unrelated to $D_K(C_i)$, so the whole block P_i is **garbled** (about half its bits wrong).
- $P'_{i+1} = D_K(C_{i+1}) \oplus C'_i = P_{i+1} \oplus (C_i \oplus C'_i)$: block $i+1$ has **exactly the same bit flipped** as the ciphertext.
- $P_{i+2} = D_K(C_{i+2}) \oplus C_{i+1}$ uses neither C_i nor C'_i, so it and all later blocks are **correct**.

CBC is therefore self-recovering after two blocks. The predictable flip in P_{i+1} is a real weakness: Eve can flip a chosen bit of one block at the price of garbling the block before it, so CBC gives no integrity by itself. An error in a *plaintext* block P_i at the sender is different: it changes C_i and through the chain every later ciphertext block, but the receiver's decryption undoes the chain and recovers exactly the transmitted plaintext, so only P_i (which was wrong at the source) is wrong at the end.

Advantages. Identical plaintext blocks give different ciphertext blocks, so patterns are hidden; the last block depends on the whole message, which is the basis of CBC-MAC. Decryption is parallel and allows random access, because P_i needs only C_i and C_{i-1}, which are both in the received ciphertext. **Disadvantages.** Encryption is strictly sequential (each block waits for the previous ciphertext); the message must be padded; an attacker can make controlled bit flips; and the IV must be random and protected. CBC is the standard mode for bulk encryption in older protocols (IPSec ESP, TLS up to 1.2, PGP-style systems with 3DES or AES).

32.4 Cipher feedback (CFB) mode ▸

▸ **Likely this year:** "Explain s-bit CFB with a shift-register diagram; why does decryption use E?" (4 marks).

Problem and idea. A terminal or a serial line produces data in small units, say $s = 8$ bits, and each unit should be sent as soon as it is typed, without waiting for a full block and without padding. CFB turns the block cipher into a **self-synchronising stream cipher** (§29): the cipher encrypts a shift register, s bits of its output are XORed with the next s-bit plaintext segment, and the resulting ciphertext segment is fed back into the register.

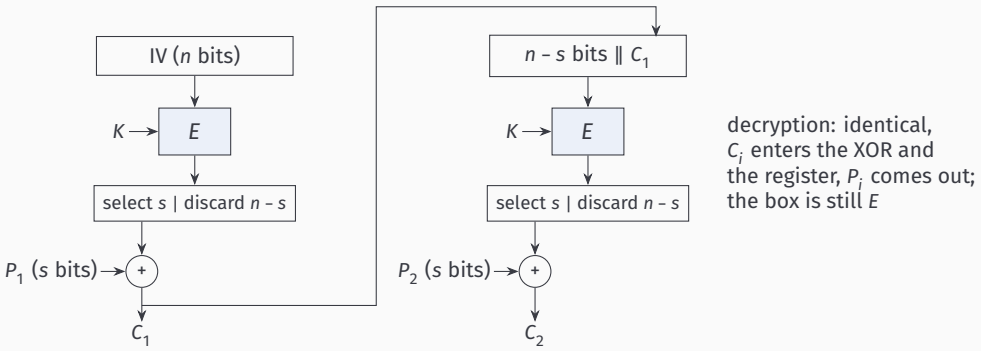
Mechanism. An n-bit shift register SR starts as the IV. For each s-bit segment:

$$O_i = E_K(SR_i), \qquad C_i = P_i \oplus \text{MSB}_s(O_i), \qquad SR_{i+1} = \text{LSB}_{n-s}(SR_i) \,\|\, C_i,$$

where MSB_s takes the leftmost s bits, the remaining $n - s$ output bits are discarded, and the register is shifted left by s bits with C_i entering on the right. Decryption runs the same register and the same E_K:

$$P_i = C_i \oplus \text{MSB}_s\big(E_K(SR_i)\big).$$

Why decryption uses E_K, not D_K. The receiver must cancel the mask $\text{MSB}_s(E_K(SR_i))$, not invert a block encryption. Its register holds the same IV and then the same ciphertext segments as the sender's, since it is filled with received ciphertext, so it computes the same mask with E_K, and $C_i \oplus \text{mask} = P_i \oplus \text{mask} \oplus \text{mask} = P_i$. With $s = n$ (full-block CFB) the register is simply the previous ciphertext block: $C_i = P_i \oplus E_K(C_{i-1})$, $C_0 = \text{IV}$.



Error propagation (derivation). A flipped bit in C_i affects decryption in two ways. Directly, $P_i = C_i \oplus \text{mask}$, so the **same bit** of P_i is flipped. Indirectly, C_i' enters the receiver's shift register and stays there while it is shifted from the right end to the left end, which takes n/s further segments; during that time the register differs from the sender's, E_K of it is unrelated to the correct mask, and those n/s segments are **garbled**. After that C_i' has been shifted out and decryption is correct again. In total one bit error damages $1 + n/s$ segments; for full-block CFB that is one bit of P_i plus the whole of P_{i+1}. The same window explains self-synchronisation: if a whole segment is lost or inserted, the receiver's register is correct again after n/s segments.

IV and cost. The IV need not be secret but must be unpredictable and must not be reused: with the same IV the first mask repeats, so $C_1 \oplus C_1' = P_1 \oplus P_1'$ for two messages. Each segment costs one full block encryption, so with $s = 8$ and AES ($n = 128$) CFB is 16 times slower than CBC. Encryption is sequential, since SR_{i+1} needs C_i; decryption can be parallel, since all ciphertext segments, and hence all register contents, are known at the start.

Advantages. CFB needs no padding, sends each s-bit unit immediately, and recovers by itself from errors and lost segments. **Disadvantages.** It is slow for small s, an error spreads over n/s further segments, and a bit flip in C_i gives a controlled flip in P_i.

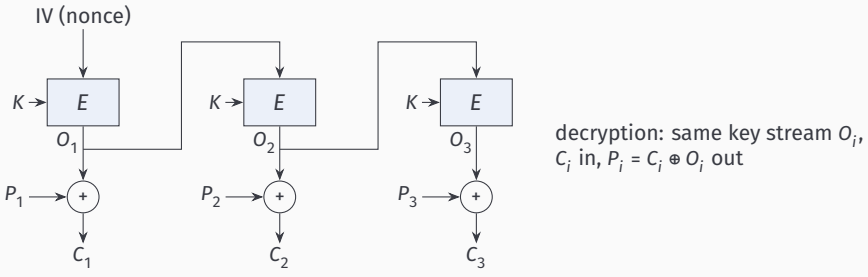
32.5 Output feedback (OFB) mode ▸

▸ **Likely this year:** "Differentiate CFB and OFB; why must the OFB IV never be reused?" (3–4 marks).

Idea. OFB feeds back the *output* of the cipher instead of the ciphertext. The key stream then depends only on the key and the IV, not on the message, which makes OFB a **synchronous stream cipher**: the key stream can be computed before the message exists, and a transmission error cannot spread. In the full-block form standardised by NIST,

$$O_0 = \text{IV}, \qquad O_i = E_K(O_{i-1}), \qquad C_i = P_i \oplus O_i, \qquad P_i = C_i \oplus O_i.$$

Decryption generates the same $O_1, O_2, \ldots$ from the same IV and key and XORs them away; again only E_K is used. If the last block is short, only the leading bits of the last O_N are used, so no padding is needed.



Error propagation. Since $P_i' = C_i' \oplus O_i$ and the key stream does not depend on the ciphertext, a flipped bit in C_i flips only the same bit of P_i; nothing else is affected. This makes OFB attractive for noisy channels (satellite links, digitised voice and video), where a few wrong bits are tolerable but a garbled block is not. The price is twofold. The mode is fully **malleable**: Eve can flip any chosen plaintext bit by flipping the ciphertext bit, so a MAC is needed (§35). And as in every synchronous stream cipher, a *lost* or inserted bit shifts the key stream against the ciphertext and garbles everything after it until the parties resynchronise.

Never reuse the IV. If two messages P and P' are encrypted with the same key and IV, they use the same key stream O_i, and Eve, who sees both ciphertexts, computes

$$C_i \oplus C_i' = (P_i \oplus O_i) \oplus (P_i' \oplus O_i) = P_i \oplus P_i'.$$

The key stream cancels, so she obtains the XOR of the two plaintexts without knowing the key, exactly the "two-time pad" failure of a one-time pad used twice (§15). If she knows or guesses one plaintext, she gets the other: $P_i' = C_i \oplus C_i' \oplus P_i$. The OFB IV is therefore a **nonce**: it need not be secret or random, but it must be unique for each message under a key.

Key-stream cycles. Since $O_i = E_K(O_{i-1})$ iterates one fixed permutation of the 2^n blocks, the key stream eventually returns to the IV and repeats. For a random permutation the cycle through a given IV has expected length about 2^{n-1}, which is harmless for $n = 64$ or 128, but a short cycle would be a disaster: in the toy cipher of the worked example, $0000 \to 0100 \to 1111 \to 0000$ is a cycle of length 3, so the IV 0000 would repeat the key stream after three blocks. The older s-bit form of OFB (feeding back only $s < n$ bits) has much shorter expected cycles, about $2^{n/2}$, which is why only full-block feedback is recommended.

Advantages. No error propagation, no padding, and the key stream can be precomputed. **Disadvantages.** No parallelism or random access (block i needs $O_1, \ldots, O_{i-1}$ first), malleability, loss of synchronisation on lost bits, and catastrophic failure on IV reuse.

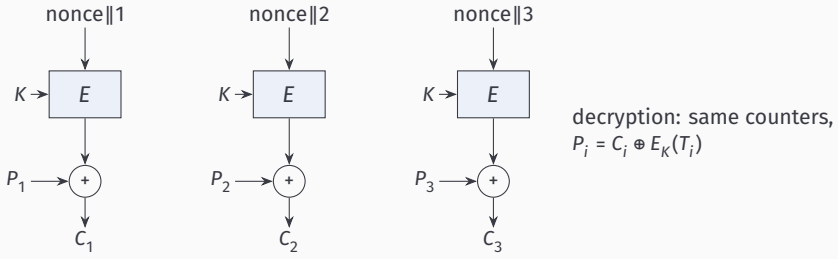
32.6 Counter (CTR) mode ★▸

▸ **Likely this year:** "Explain CTR mode and list its advantages" (4 marks); CTR versus OFB.

Idea. CTR keeps the stream-cipher structure of OFB but removes the chain from the key stream: block i of the key stream is the encryption of a **counter block** T_i that is different for every block and every message. Usually T_i is a per-message nonce concatenated with a block counter that starts at 1 (or 0) and is incremented for each block:

$$T_i = \text{nonce} \parallel i, \qquad C_i = P_i \oplus E_K(T_i), \qquad P_i = C_i \oplus E_K(T_i).$$

Decryption is the same computation, because XORing with the same mask twice cancels it. A short last block uses only the leading bits of $E_K(T_N)$, so there is no padding.



Advantages (Stallings lists six).

- **Hardware efficiency**: the blocks are independent, so many can be encrypted or decrypted in parallel; the throughput is limited only by the amount of parallel hardware.
- **Software efficiency**: the independence also suits pipelined and multi-core processors.
- **Preprocessing**: $E_K(T_i)$ does not depend on the data, so the key stream can be computed in advance and encryption reduces to one XOR per block when the data arrive.
- **Random access**: block i is decrypted from C_i and T_i alone, without touching blocks $1, \ldots, i-1$, which suits files and disks.
- **Provable security**: CTR is at least as secure as the other modes, under the same assumption on the block cipher.
- **Simplicity**: only E_K is ever used, so D_K need not be implemented.

Requirement. A counter block must **never be used twice with the same key**, in the same message or in different messages. If it were, the same mask would be XORed onto two plaintext blocks and $C_i \oplus C_i' = P_i \oplus P_i'$ would leak, exactly as for a reused OFB IV. This is why the nonce part changes for every message and the counter part is long enough never to wrap around within one message. Errors behave as in OFB: a flipped ciphertext bit flips only the same plaintext bit, and the mode is malleable, so CTR is normally combined with a MAC; AES-GCM, the default cipher of TLS 1.3, is CTR encryption plus a polynomial MAC.

32.7 IV and nonce rules, and why block size matters ★▸

▸ **Likely this year:** "What happens if the IV is reused in OFB/CTR? Show that $C \oplus C'$ leaks $P \oplus P'$" (3 marks).

The modes differ in what they demand of the IV, and every requirement comes from one of the attacks above.

Mode	Requirement on IV / counter	What goes wrong if it is violated
ECB	none (no IV)	equal blocks always visible
CBC	random, unpredictable; protected against change	reuse shows equal first blocks; predictable IV allows guess checking (BEAST); a changed IV flips chosen bits of P_1
CFB	unpredictable, not reused	reuse gives $C_1 \oplus C_1' = P_1 \oplus P_1'$
OFB	nonce: unique per message	reuse gives the same key stream: $C \oplus C' = P \oplus P'$ for every block
CTR	every counter block unique under the key	a repeated counter block gives $C_i \oplus C_j' = P_i \oplus P_j'$

Block size and the birthday bound. Even with perfect IVs, CBC leaks once two ciphertext blocks happen to be equal. If $C_i = C_j$, then $E_K(P_i \oplus C_{i-1}) = E_K(P_j \oplus C_{j-1})$; since E_K is a permutation, $P_i \oplus C_{i-1} = P_j \oplus C_{j-1}$, i.e. $P_i \oplus P_j = C_{i-1} \oplus C_{j-1}$, which Eve can compute. The ciphertext blocks behave like random n-bit values, so by the birthday problem (§35) such a collision is likely after about $1.18 \times 2^{n/2}$ blocks. For a 64-bit cipher (DES, 3DES) that is about 2^{32} blocks = 32 GB under one key, which is reached in practice (the Sweet32 attack on 3DES in TLS, 2016); for AES ($n = 128$) it is 2^{64} blocks, far beyond any session. Keys for 64-bit ciphers must therefore be changed long before 2^{32} blocks.

32.8 Error propagation, parallelism and padding compared ★▸

▸ **Likely this year:** "A bit error occurs in C_i. Which plaintext blocks are affected in ECB, CBC, CFB, OFB and CTR?" (5 marks).

★ MUST KNOW : the five modes

ECB	$C_i = E_K(P_i)$	$P_i = D_K(C_i)$
CBC	$C_i = E_K(P_i \oplus C_{i-1})$, $C_0 = \text{IV}$	$P_i = D_K(C_i) \oplus C_{i-1}$
CFB	$C_i = P_i \oplus \text{MSB}_s(E_K(SR_i))$, $SR_{i+1} = \text{LSB}_{n-s}(SR_i) \| C_i$	$P_i = C_i \oplus \text{MSB}_s(E_K(SR_i))$
OFB	$O_i = E_K(O_{i-1})$, $O_0 = \text{IV}$, $C_i = P_i \oplus O_i$	$P_i = C_i \oplus O_i$
CTR	$C_i = P_i \oplus E_K(T_i)$, $T_i = \text{nonce} \| i$	$P_i = C_i \oplus E_K(T_i)$

One flipped bit in C_i damages: ECB: block i; CBC: block i garbled + same bit of block $i+1$; CFB: same bit of segment i + the next n/s segments garbled; OFB, CTR: only the same bit of block i.

Mode	Type	Uses D_K?	Parallel enc./dec.	Padding	Typical use
ECB	block	yes	yes / yes	yes	single random blocks (keys)
CBC	block	yes	no / yes	yes	bulk data, CBC-MAC
CFB	stream, self-synchronising	no	no / yes	no	character streams, terminals
OFB	stream, synchronous	no	no / no (precomputable)	no	noisy channels (satellite, audio)
CTR	stream, synchronous	no	yes / yes	no	high-speed links, disks, AES-GCM

Random access to block i is possible in ECB, CBC (from C_{i-1}, C_i), CFB (from the preceding n/s segments) and CTR, but not in OFB, which must run the chain from the IV.

32.9 Worked example: the five modes on a toy 4-bit cipher ★▸

▸ **Likely this year:** "Using the given 4-bit block cipher, encrypt P in ECB, CBC, CFB, OFB and CTR mode" (5–8 marks).

The toy cipher has a 4-bit block and a 4-bit key: $E_K(x) = S(x \oplus K)$, where S is row 0 of DES S-box S1, a permutation of the sixteen 4-bit values. With $K = 0001$ the whole cipher is the table below (hex digits; e.g. $E_K(\text{A}) = S(1010 \oplus 0001) = S(1011) = 1100 = \text{C}$).

x	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
$S(x)$	E	4	D	1	2	F	B	8	3	A	6	C	5	9	0	7
$E_K(x) = S(x \oplus 1)$	4	E	1	D	F	2	8	B	A	3	C	6	9	5	7	0
$D_K(y)$, inverse of E_K	F	2	5	9	0	D	B	E	6	C	8	7	A	3	1	4

WORKED EXAMPLE : same plaintext, five modes

Q. Using the toy cipher with $K = 0001$, encrypt $P = 1010\ 1010\ 0011$ (three 4-bit blocks) in ECB, CBC, full-block CFB, OFB and CTR mode, with IV = 0110; in CTR the counter blocks are $T_1 = \text{IV}$, $T_i = T_{i-1} + 1 \bmod 16$. Decrypt the CBC ciphertext to check it, and say what each ciphertext reveals.

A. Every E_K value is read from the table.

	ECB		CBC				CFB (input C_{i-1})		
i	P_i	$C_i = E_K(P_i)$	C_{i-1}	$P_i \oplus C_{i-1}$	C_i	check $D_K(C_i) \oplus C_{i-1}$	$E_K(C_{i-1})$	P_i	C_i
1	1010	1100	0110 (IV)	1100	1001	$1100 \oplus 0110 = 1010$	$E_K(0110) = 1000$	1010	0010
2	1010	1100	1001	0011	1101	$0011 \oplus 1001 = 1010$	$E_K(0010) = 0001$	1010	1011
3	0011	1101	1101	1110	0111	$1110 \oplus 1101 = 0011$	$E_K(1011) = 0110$	0011	0101

	OFB			CTR		
i	$O_i = E_K(O_{i-1})$	P_i	$C_i = P_i \oplus O_i$	T_i	$E_K(T_i)$	$C_i = P_i \oplus E_K(T_i)$
1	$E_K(0110) = 1000$	1010	0010	0110	1000	0010
2	$E_K(1000) = 1010$	1010	0000	0111	1011	0001
3	$E_K(1010) = 1100$	0011	1111	1000	1010	1001

Results: ECB **1100 1100 1101**; CBC **1001 1101 0111**; CFB **0010 1011 0101**; OFB **0010 0000 1111**; CTR **0010 0001 1001**.

Observations. (1) ECB gives $C_1 = C_2 = 1100$, so Eve sees that $P_1 = P_2$ without knowing the key; every other mode hides the repetition. (2) CFB, OFB and CTR all start with $C_1 = P_1 \oplus E_K(\text{IV}) = 0010$, because all three use $E_K(\text{IV})$ as the first mask; they differ from block 2 on, where the next cipher input is C_1, O_1 and T_2 respectively. (3) OFB's $C_2 = 0000$ only means that O_2 happened to equal P_2.

Check: the CBC decryption column returns $1010, 1010, 0011 = P$.

WORKED EXAMPLE : error propagation on the toy cipher

Q. In each of the five ciphertexts above, the leftmost bit of C_2 is flipped in transit. Decrypt and state which plaintext bits are wrong.
A. The changed block is $C_2' = C_2 \oplus 1000$.

Mode	C_2'	Decryption of the affected blocks	Received P	Wrong bits ($\oplus P$)
ECB	0100	$P_2' = D_K(0100) = 0000$	1010 **0000** 0011	0000 1010 0000
CBC	0101	$P_2' = D_K(0101) \oplus C_1 = 1101 \oplus 1001 = 0100$; $P_3' = D_K(C_3) \oplus C_2' = 1110 \oplus 0101 = 1011$	1010 **0100 1011**	0000 1110 1000
CFB	0011	$P_2' = C_2' \oplus E_K(C_1) = 0011 \oplus 0001 = 0010$; $P_3' = C_3 \oplus E_K(C_2') = 0101 \oplus 1101 = 1000$	1010 **0010 1000**	0000 1000 1011
OFB	1000	$P_2' = C_2' \oplus O_2 = 1000 \oplus 1010 = 0010$	1010 **0010** 0011	0000 1000 0000
CTR	1001	$P_2' = C_2' \oplus E_K(T_2) = 1001 \oplus 1011 = 0010$	1010 **0010** 0011	0000 1000 0000

ECB garbles block 2 only; CBC garbles block 2 and flips exactly the leftmost bit of block 3; full-block CFB flips exactly the leftmost bit of block 2 and garbles block 3; OFB and CTR flip only the leftmost bit of block 2. Block 1 is correct in every mode.
Check: this matches the derived rules (CBC: i garbled, $i+1$ same bit; CFB: same bit, then $n/s = 1$ block garbled).

WORKED EXAMPLE : counter reuse in CTR

Q. By mistake the second message Q = 1111 0000 0101 is encrypted in CTR mode with the same key and the same counter blocks 0110, 0111, 1000. What can Eve learn from the two ciphertexts?
A. The masks are again 1000, 1011, 1010, so $C' = Q \oplus$ masks = 0111 1011 1111. Eve XORs the two ciphertexts block by block: 0010⊕0111 = 0101, 0001⊕1011 = 1010, 1001⊕1111 = 0110. This is exactly $P \oplus Q$ = 0101 1010 0110, obtained without the key. If she knows P (a standard header, say), she recovers $Q = C \oplus C' \oplus P$ completely.
Check: 1010 ⊕ 1111 = 0101, 1010 ⊕ 0000 = 1010, 0011 ⊕ 0101 = 0110 ✓.

32.10 Ciphertext stealing

Padding makes the ECB or CBC ciphertext longer than the plaintext, which is unacceptable when the ciphertext has to fit in the same space, for example a disk sector or a fixed-length database field. **Ciphertext stealing** avoids padding by "stealing" the missing bits of the last block from the ciphertext of the block before it. Let P_N have $m < n$ bits, and let head_m and tail_{n-m} denote the first m and the last $n-m$ bits. CBC with ciphertext stealing (Forouzan) processes blocks $1, \dots, N-2$ normally and then computes

$$X = E_K(P_{N-1} \oplus C_{N-2}), \qquad C_N = \text{head}_m(X), \qquad Y = (P_N \,\|\, 0^{n-m}) \oplus X, \qquad C_{N-1} = E_K(Y).$$

The sender transmits $C_1, \dots, C_{N-1}$ and the m-bit C_N, so the ciphertext is exactly as long as the plaintext. The receiver computes $Y = D_K(C_{N-1})$. The last $n-m$ bits of Y equal those of X (they were XORed with zeros), so $X = C_N \,\|\, \text{tail}_{n-m}(Y)$ can be rebuilt; then $P_N = \text{head}_m(Y) \oplus C_N$ and $P_{N-1} = D_K(X) \oplus C_{N-2}$. The tail of X that is never transmitted travels inside C_{N-1}; that is the "stolen" ciphertext. The method needs at least two blocks (checked by script on random inputs).

32.11 Multiple encryption: double and triple DES

Section 30 derives the meet-in-the-middle attack and the triple-DES variants in full; only the results are collected here. Double DES, $C = E_{K_2}(E_{K_1}(P))$, has a 112-bit key, but the attacker encrypts a known P under all 2^{56} keys K_1, stores the middle values, decrypts C under all 2^{56} keys K_2 and looks for a match, so the work is $2^{56} + 2^{56} = 2^{57}$ DES operations instead of 2^{112}. The memory is the real obstacle: 2^{56} entries of a 64-bit middle value and a 56-bit key are $2^{56} \times 15$ bytes $\approx 1.08 \times 10^{18}$ bytes, about one exabyte. Triple DES with two keys, $C = E_{K_1}(D_{K_2}(E_{K_1}(P)))$, has the middle stage as a *decryption* so that $K_1 = K_2$ gives single DES and a 3DES device stays compatible with a DES device; any split of its three stages leaves a 2^{112} search on one side. Triple DES with three keys has a 168-bit key and meet-in-the-middle security of about 2^{112}.

Scheme	Key bits	Brute force	Meet-in-the-middle
DES	56	2^{56}	—
Double DES	112	2^{112}	2^{57} time, 2^{56} blocks of memory
3DES, two keys (EDE)	112	2^{112}	no gain (still $\approx 2^{112}$)
3DES, three keys	168	2^{168}	$\approx 2^{112}$

Triple DES is a block cipher in its own right (64-bit block), so it is used with one of the five modes, normally CBC (3DES-CBC in PGP, S/MIME and IPSec), and it inherits the 64-bit birthday limit above.

Corrections to the notes / slides

- No class slides cover this topic; the notes follow Stallings, Forouzan and NIST SP 800-38A. The books differ in notation, which matters when an exam question is copied from one of them. Forouzan writes r for the CFB segment size that Stallings calls s, and describes OFB with an r-bit shift register like CFB; Stallings' newer editions and NIST use full-block OFB, $O_i = E_K(O_{i-1})$, as here. Forouzan initialises the CTR counter with the IV and increments it per block; Stallings writes a counter that starts from a nonce. Both describe the same modes.
- A frequent exam mistake is to draw D_K at the receiver in CFB, OFB or CTR. These modes use only E_K at both ends, which is also why the block function in them need not even be invertible.

Part IV
Asymmetric-Key Cryptography

33 Asymmetric-Key Cryptography and RSA ▸

▸ **Likely this year:** New this year, so expect one Forouzan-style numerical: "p, q, e given: find n, $\phi(n)$, d; encrypt P; decrypt C" (4–6 marks), plus a "why" question (proof of correctness, why $e = 2$ fails, one attack).

33.1 Why asymmetric-key cryptography exists ▸

▸ **Likely this year:** "Differentiate symmetric-key and asymmetric-key cryptography" / "Why is symmetric cryptography still needed?" (2–3 marks).

In symmetric-key cryptography both parties use the same secret key, so before Alice and Bob can talk privately they must already share a secret. This is the **key distribution problem**: the key has to travel over a secure channel that usually does not exist, and in a group of N users every pair needs its own key, so $N(N-1)/2$ secret keys must be created and stored (4950 for 100 users). Symmetric cryptography also cannot prove who sent a message, because anyone who can decrypt can also encrypt.

Asymmetric-key (public-key) cryptography removes the shared secret. Bob generates a **key pair**: a **public key** that he announces to everyone and a **private key** that only he knows. The public key locks; the private key unlocks. Anyone can encrypt for Bob with his public key, but only his private key decrypts, so Alice needs no earlier contact with Bob and N users need only N key pairs. Formally $C = f(K_{\text{public}}, P)$ and $P = g(K_{\text{private}}, C)$, where f is used only for encryption and g only for decryption, and knowing f must not reveal g. The secret is now carried by the receiver alone; for Bob to reply, he uses *Alice's* public key. Plaintext and ciphertext are treated as **numbers**, and encryption is a number-theoretic function rather than substitution and permutation of symbols.

Asymmetric cryptography does **not** replace symmetric cryptography, because it is far slower: one RSA operation is a modular exponentiation with 1024–2048-bit numbers, while AES needs a few table lookups and XORs per byte (RSA is roughly a thousand times slower). It is therefore used where symmetric cryptography cannot work: **exchanging a secret session key, authentication and digital signatures**. In practice the two are combined: a random AES key is sent encrypted under the receiver's RSA key, and the bulk data is encrypted with AES under that key.

	Symmetric-key	Asymmetric-key
Keys	one shared secret key	public/private key pair
Keys for N users	$N(N-1)/2$	N pairs
Secret held by	both parties	receiver only
Based on	substitution, permutation	number theory (factoring, discrete log)
Speed	fast (bulk data)	slow (keys, signatures)
Examples	DES, AES	RSA, ElGamal, ECC

33.2 Trapdoor one-way functions ▸

▸ **Likely this year:** "Define a trapdoor one-way function; give the RSA example (2 marks).

The design problem is to publish f while keeping f^{-1} hidden. A **one-way function** (OWF) is a function $y = f(x)$ that is easy to compute, while computing $x = f^{-1}(y)$ from y is computationally infeasible. A **trapdoor one-way function** (TOWF) has a third property: given *y and a secret* (the trapdoor), x is easy to compute. A public-key cipher needs exactly this: the public key lets everyone evaluate f, and the trapdoor is the private key.

Multiplication is one-way. For large primes p, q, computing $n = p \times q$ is easy, but recovering p and q from n is the **factorization problem**, for which no polynomial-time (classical) algorithm is known.

$y = x^k \bmod n$ **is a trapdoor one-way function.** Given x, k and n, fast exponentiation (§22) computes y in about $2\log_2 k$ multiplications; given only y, k and n, taking the k-th root modulo n is believed to be as hard as factoring n. The trapdoor is k' with $kk' \equiv 1 \pmod{\phi(n)}$: then $x = y^{k'} \bmod n$. Computing k' needs $\phi(n) = (p-1)(q-1)$, and computing $\phi(n)$ needs p and q, so the trapdoor exists only for whoever chose the primes. RSA is this function with $k = e$ and $k' = d$.

33.3 RSA: key generation, encryption, decryption ▸

▸ **Likely this year:** Write the three algorithms, then run them on small primes.

RSA (Rivest, Shamir, Adleman, 1977) is the most widely used public-key cryptosystem. Bob builds the trapdoor from two primes and publishes only their product; Alice encrypts by raising the plaintext to the public exponent, and Bob decrypts by raising the ciphertext to the private exponent.

★ MUST KNOW : RSA

Key generation (Bob)

1. Choose two large primes $p \neq q$; compute $n = p \times q$ and $\phi(n) = (p-1)(q-1)$.
2. Choose e with $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$.
3. Compute $d = e^{-1} \bmod \phi(n)$ with the extended Euclidean algorithm, so $e \times d \equiv 1 \pmod{\phi(n)}$.
4. Public key (e, n), announced. Private key d, kept secret (with p, q, $\phi(n)$).

Encryption (Alice): $C = P^e \bmod n$, with $0 \le P < n$. **Decryption (Bob)**: $P = C^d \bmod n$.

Encryption and decryption are each one fast exponentiation, so both are polynomial in the size of n. Eve sees C, e and n and must compute $P = \sqrt[e]{C} \bmod n$, which is exponential for her unless she can factor n. The plaintext must be smaller than n, because the result is only known modulo n: two plaintexts that differ by n give the same ciphertext. A long message is cut into blocks, each a number less than n.

33.4 Why decryption works: proof of correctness ▸

▸ **Likely this year:** "Prove that RSA decryption recovers the plaintext" (2–3 marks).

Since d is the inverse of e modulo $\phi(n)$, $ed = 1 + k\,\phi(n)$ for some integer k. Decryption computes

$$C^d \equiv (P^e)^d = P^{ed} = P^{1+k\phi(n)} = P \cdot \left(P^{\phi(n)}\right)^k \pmod{n}.$$

Case 1: $\gcd(P, n) = 1$ (almost every P). Euler's theorem (§26) gives $P^{\phi(n)} \equiv 1 \pmod{n}$, so $C^d \equiv P \cdot 1^k = P \pmod{n}$. Since $0 \le P < n$, the reduced result is exactly P, not just something congruent to it.

Case 2: $\gcd(P, n) \neq 1$. Euler's theorem does not apply, but the result still holds. Since $0 < P < pq$, P is a multiple of exactly one prime, say $p \mid P$ and $q \nmid P$. Modulo p, $P \equiv 0$, so $P^{ed} \equiv 0 \equiv P$. Modulo q, Fermat's little theorem gives $P^{q-1} \equiv 1$, and $ed = 1 + k(p-1)(q-1)$, so $P^{ed} = P \cdot (P^{q-1})^{k(p-1)} \equiv P \pmod{q}$. Since $P^{ed} \equiv P$ modulo both p and q, and p, q are coprime, the CRT (§27) gives $P^{ed} \equiv P \pmod{pq}$. (This is Euler's "second version", $P^{k\phi(n)+1} \equiv P \pmod{n}$ for $n = pq$ and every P.) Check: $n = 221 = 13 \cdot 17$, $e = 5$, $d = 77$, $P = 65 = 5 \cdot 13$: $65^5 \bmod 221 = 182$ and $182^{77} \bmod 221 = 65$ ✓. Such a P is no practical concern, because finding one would give $\gcd(P, n) = p$ and factor n.

The two algebraic structures. RSA works in two places (§28). Encryption and decryption use the **public ring** $R = \langle \mathbb{Z}_n, +, \times \rangle$: P and C can be any element of $\mathbb{Z}_n$ (Case 2 shows non-invertible elements also work), and exponentiation is repeated multiplication, which anyone can do since n is public. Key generation uses the **private group** $G = \langle \mathbb{Z}^*_{\phi(n)}, \times \rangle$: e must belong to this group, i.e. $\gcd(e, \phi(n)) = 1$, so that its inverse d exists in it. Only Bob can build G, because knowing $\phi(n)$ is as good as knowing the factors: $p+q = n - \phi(n) + 1$, and p, q are the roots of $x^2 - (p+q)x + n = 0$. For $n = 187$, $\phi = 160$: $p + q = 28$, $x^2 - 28x + 187 = 0$ gives $x = (28 \pm 6)/2 = 17, 11$.

33.5 The slide examples ▸

▸ **Likely this year:** Verify one row by hand, e.g. "with $n = 77$, $e = 13$, find d and encrypt 5.

The lecture uses the key $p = 7$, $q = 11$, $e = 13$: $n = 77$, $\phi(n) = 60$, and $d = 37$ because $13 \times 37 = 481 = 8 \cdot 60 + 1$. Alice encrypts 5 and John encrypts 63 with the same public key $(13, 77)$; Bob decrypts both with $d = 37$. The table also has the class homework key ($n = 187$) and a small textbook key ($n = 33$); all values were recomputed.

p, q	n	$\phi(n)$	e	d	Encrypt	Decrypt
7, 11	77	60	13	37	$5^{13} \bmod 77 = 26$	$26^{37} \bmod 77 = 5$
7, 11	77	60	13	37	$63^{13} \bmod 77 = 28$	$28^{37} \bmod 77 = 63$
17, 11	187	160	7	23	$88^7 \bmod 187 = 11$	$11^{23} \bmod 187 = 88$
3, 11	33	20	7	3	$5^7 \bmod 33 = 14$	$14^3 \bmod 33 = 5$

For the slide key, the d-table (§24) on $(60, 13)$ has quotients $4, 1, 1, 1, 1, 2$ and ends with $r_1 = 1$, $t_1 = -23$, so $d = -23 + 60 = 37$. A negative t_1 is normal; add $\phi(n)$, never n.

33.6 Exam-format RSA numericals ▸

▸ **Likely this year:** Exactly this layout: $\phi(n)$, gcd check, d-table, two square-and-multiply tables, a check line.

Every exponentiation below uses repeated squaring (§22): write the exponent as a sum of powers of 2, square repeatedly (reducing mod n each time), then multiply the needed powers, reducing after each product.

WORKED EXAMPLE : Stallings/Forouzan, the class homework key

Q. In RSA, $p = 17$, $q = 11$, $e = 7$. Find n, $\phi(n)$ and the private key d. Encrypt $P = 88$ and show that decryption recovers it.
Keys. $n = 17 \times 11 = 187$; $\phi(n) = 16 \times 10 = 160$. Since $160 = 2^5 \cdot 5$ and 7 is prime, $\gcd(7, 160) = 1$, so $e = 7$ is valid. $d = 7^{-1} \bmod 160$:

q	r_1	r_2	r	t_1	t_2	t
22	160	7	6	0	1	−22
1	7	6	1	1	−22	23
6	6	1	0	−22	23	−160
	1	0		**23**	−160	

$r_1 = 1$ (inverse exists), $d = t_1 = \mathbf{23}$. Check: $7 \times 23 = 161 = 160 + 1$ ✓. Public key $(7, 187)$, private key 23.
Encrypt $C = 88^7 \bmod 187$, with $7 = 4 + 2 + 1$:

Power	Computation	mod 187
88^1		88
88^2	$88^2 = 7744 = 41 \cdot 187 + 77$	77
88^4	$77^2 = 5929 = 31 \cdot 187 + 132$	132

$88^7 = 88^4 \cdot 88^2 \cdot 88^1$: $132 \cdot 77 = 10164 \equiv 66$; $66 \cdot 88 = 5808 \equiv 11$. $\Rightarrow C = \mathbf{11}$.
Decrypt $P = 11^{23} \bmod 187$, with $23 = 16 + 4 + 2 + 1$:

Power	Computation	mod 187
11^1		11
11^2	$11^2 = 121$	121
11^4	$121^2 = 14641 = 78 \cdot 187 + 55$	55
11^8	$55^2 = 3025 = 16 \cdot 187 + 33$	33
11^{16}	$33^2 = 1089 = 5 \cdot 187 + 154$	154

$154 \cdot 55 = 8470 \equiv 55$; $55 \cdot 121 = 6655 \equiv 110$; $110 \cdot 11 = 1210 \equiv 88$. $\Rightarrow P = \mathbf{88}$ ✓ (the original plaintext).

WORKED EXAMPLE : new, Forouzan exercise style (only n given)

Q. Bob's RSA public key is $(e, n) = (5, 221)$. (a) Find his private key. (b) Alice sends $P = 100$; find C. (c) Show Bob's decryption.
(a) Eve cannot factor a 1024-bit n, but 221 factors by trial division: $221 = 13 \times 17$. So $\phi(n) = 12 \times 16 = 192 = 2^6 \cdot 3$, and $\gcd(5, 192) = 1$. $d = 5^{-1} \bmod 192$:

q	r_1	r_2	r	t_1	t_2	t
38	192	5	2	0	1	−38
2	5	2	1	1	−38	77
2	2	1	0	−38	77	−192
	1	0		**77**	−192	

$d = \mathbf{77}$. Check: $5 \times 77 = 385 = 2 \cdot 192 + 1$ ✓.
(b) $5 = 4 + 1$: $100^2 = 10000 = 45 \cdot 221 + 55$; $55^2 = 3025 = 13 \cdot 221 + 152$, so $100^4 \equiv 152$. $C = 152 \cdot 100 = 15200 = 68 \cdot 221 + 172 \Rightarrow C = \mathbf{172}$.
(c) $P = 172^{77} \bmod 221$, with $77 = 64 + 8 + 4 + 1$:

Power	Computation	mod 221
172^1		172
172^2	$172^2 = 29584 = 133 \cdot 221 + 191$	191
172^4	$191^2 = 36481 = 165 \cdot 221 + 16$	16
172^8	$16^2 = 256 = 221 + 35$	35
172^{16}	$35^2 = 1225 = 5 \cdot 221 + 120$	120
172^{32}	$120^2 = 14400 = 65 \cdot 221 + 35$	35
172^{64}	$35^2 \equiv 120$ (as above)	120

$172^{64} \cdot 172^{8} = 120 \cdot 35 = 4200 = 19 \cdot 221 + 1 \equiv 1$; $1 \cdot 16 = 16$; $16 \cdot 172 = 2752 = 12 \cdot 221 + 100$. $\Rightarrow P =$ **100** ✓.

33.7 Choosing e; why $e = 2$ never works ▸

▸ **Likely this year:** "Why is $e = 2$ not allowed? or "is $e = 3$ valid for $p = 19$, $q = 23$? (1–2 marks).

Encryption cost depends on the number of bits and of 1-bits in e, so a small e with few 1-bits makes encryption cheap. The standard choice is $e = 2^{16} + 1 = 65537$: it is prime (so it is coprime to $\phi(n)$ unless a prime factor of $\phi(n)$ is 65537 itself), its binary form 1 0000 0000 0000 0001 needs only 16 squarings and one multiplication, and it is large enough to defeat the low-exponent attacks below. The private exponent d is then about as long as n, so decryption is the slow side (which the CRT speeds up).

$e = 2$ (or any even e) is invalid. Since p and q are odd primes, $p - 1$ and $q - 1$ are even, so $\phi(n)$ is even (in fact divisible by 4). Then $\gcd(2, \phi(n)) = 2 \neq 1$, e is not in $\mathbb{Z}^*_{\phi(n)}$, and no d exists. The cause is that squaring is not one-to-one: x and $n - x$ have the same square, so $C = P^2$ cannot be uniquely undone. The same check catches a Forouzan exercise: $p = 19$, $q = 23$, $e = 3$ gives $\phi(n) = 18 \cdot 22 = 396 = 3 \cdot 132$, so $\gcd(3, 396) = 3$ and $e = 3$ is not allowed for this n.

Trap: the usual RSA slips

d is the inverse of e modulo $\phi(n)$, never modulo n (for $(7, 187)$ the answer is 23; $7^{-1} \bmod 187 = 107$ is useless). Check $\gcd(e, \phi(n)) = 1$ before running the table. The plaintext must satisfy $P < n$. Encrypt with the *receiver's* public key. Reduce mod n after every multiplication, and turn a negative t_1 into $t_1 + \phi(n)$.

33.8 Faster decryption with the CRT ▸

▸ **Likely this year:** "Decrypt using the Chinese remainder theorem" on a small key, or "why is CRT decryption about 4 times faster".

Decryption is slow because d and n are both full size. Bob, however, knows p and q, so he can work modulo each prime separately and combine the answers with the CRT (§27). Modulo p the exponent can be reduced: since $C^{p-1} \equiv 1 \pmod p$ (Fermat), $C^d \equiv C^{d \bmod (p-1)} \pmod p$. Bob computes

$$d_p = d \bmod (p-1), \quad d_q = d \bmod (q-1), \qquad P_p = C^{d_p} \bmod p, \quad P_q = C^{d_q} \bmod q,$$

and then the unique $P < n$ with $P \equiv P_p \pmod p$ and $P \equiv P_q \pmod q$:

$$P = \left(P_p \cdot q \cdot (q^{-1} \bmod p) + P_q \cdot p \cdot (p^{-1} \bmod q)\right) \bmod n.$$

Each half uses a modulus of half the length (a multiplication costs about $1/4$) and an exponent of half the length (half as many steps), so each half costs about $1/8$ of the full exponentiation and both together about $1/4$: decryption is roughly **4 times faster**. Only the key owner can use it, since it needs p and q.

WORKED EXAMPLE : CRT decryption of the previous ciphertext

Q. With $p = 13$, $q = 17$, $d = 77$, decrypt $C = 172$ using the CRT.
$d_p = 77 \bmod 12 = 5$, $d_q = 77 \bmod 16 = 13$; $C_p = 172 \bmod 13 = 3$, $C_q = 172 \bmod 17 = 2$.
$P_p = 3^5 \bmod 13 = 243 \bmod 13 = 9$. $P_q = 2^{13} \bmod 17$: $2^4 = 16 \equiv -1$, $2^8 \equiv 1$, so $2^{13} = 2^8 \cdot 2^4 \cdot 2 \equiv -2 \equiv 15$.
$q^{-1} \bmod p = 17^{-1} \bmod 13 = 4^{-1} \bmod 13 = 10$ $(4 \cdot 10 = 40 = 3 \cdot 13 + 1)$; $p^{-1} \bmod q = 13^{-1} \bmod 17 = 4$ $(13 \cdot 4 = 52 = 3 \cdot 17 + 1)$.
$P = 9 \cdot 17 \cdot 10 + 15 \cdot 13 \cdot 4 = 1530 + 780 = 2310 = 10 \cdot 221 + 100 \Rightarrow P =$ **100** ✓, the same as $172^{77} \bmod 221$, with exponents 5 and 13 instead of 77.

33.9 Attacks on RSA ▸

▸ **Likely this year:** "Explain any two attacks on RSA" or "why must p and q be large / not close / why OAEP".

No devastating attack on correctly used RSA is known; every known attack exploits a weak choice of parameters or a careless use of textbook RSA. The slide's list, each with the reason it works:

- **Factorization.** If Eve factors n she computes $\phi(n)$ and then $d = e^{-1} \bmod \phi(n)$, exactly as Bob did. The whole security rests on n being too large to factor.
- **Chosen-ciphertext.** RSA is multiplicative: $(P_1 P_2)^e = P_1^e P_2^e$. To decrypt C, Eve picks X, asks Bob to decrypt the innocent-looking $Y = C \cdot X^e \bmod n$, receives $Z = Y^d = P \cdot X$, and computes $P = Z \cdot X^{-1}$. With $(7, 187)$, $C = 11$, $X = 2$: $Y = 11 \cdot 128 \bmod 187 = 99$, $Z = 99^{23} \bmod 187 = 176$, $P = 176 \cdot 94 \bmod 187 = 88$ ✓.
- **Low encryption exponent.** If e is small and $P^e < n$, no reduction happens and $P = \sqrt[e]{C}$ over the integers ($e = 3$, $n = 187$, $P = 5$: $C = 125$, $\sqrt[3]{125} = 5$). **Coppersmith's** theorem extends this to finding small unknown parts of P. **Broadcast**: if the same P goes to e receivers with moduli $n_1, \ldots, n_e$, the CRT gives $P^e \bmod n_1 \cdots n_e$, which

equals P^e because $P^e < n_1 \cdots n_e$, so an integer root reveals P (with $e = 3$, $n = 55,391,1189$ and $P = 42$: ciphertexts $3, 189, 370$, CRT gives $74088 = 42^3$). **Related messages** (Franklin–Reiter) and **short pad** attacks similarly recover messages that are linearly related or padded with too few random bits.

- **Low decryption exponent.** A small d speeds decryption but Wiener's attack recovers d from e/n by continued fractions when $d < \frac{1}{3}n^{1/4}$. A **revealed** d is worse than it looks: n with one pair (e, d) lets anyone factor n, so all keys on that modulus are dead.
- **Plaintext attacks.** *Short message*: if P comes from a small set (a PIN), Eve encrypts every candidate with the public key and compares, since textbook RSA is deterministic. *Cyclic*: C is a permutation of $\mathbb{Z}_n$, so re-encrypting $C, C^e, C^{e^2}, \ldots$ eventually returns to C, and the value just before C is P; with $(7, 33)$ and $C = 14$: $14 \to 20 \to 26 \to 5 \to 14$, so $P = 5$. *Unconcealed*: some P satisfy $P^e \equiv P$ (always 0, 1, $n - 1$); their number is $(1 + \gcd(e - 1, p - 1))(1 + \gcd(e - 1, q - 1))$, which for the slide key $(13, 77)$ is $7 \cdot 3 = 21$ messages (e.g. $10^{13} \equiv 10$), because $e - 1 = 12$ is a multiple of $p - 1 = 6$.
- **Common modulus.** If two users share n with exponents e_1, e_2, $\gcd(e_1, e_2) = 1$, and the same P is sent to both, Eve finds $se_1 + te_2 = 1$ and gets $C_1^s C_2^t = P^{se_1 + te_2} = P$. With $n = 187$, $e_1 = 3$, $e_2 = 7$, $P = 5$: $C_1 = 125$, $C_2 = 146$, $s = -2$, $t = 1$, $P = (125^{-1})^2 \cdot 146 = 3^2 \cdot 146 = 1314 \equiv 5$ ✓. Also, any user of the shared n can factor it from his own (e, d).
- **Implementation (timing, power).** Square-and-multiply does an extra multiplication for every 1-bit of d, so the decryption time or power trace leaks the bits of d. Defences: constant-time exponentiation, random delays, or *blinding* (decrypt $C \cdot r^e$, then multiply by r^{-1}).

33.10 Recommendations

Each recommendation blocks one of the attacks above.

- n at least **1024 bits** (about 309 decimal digits; 2048 today), so each of p, q at least **512 bits** (about 154 digits). This defeats factorization.
- p and q must not be close to each other, otherwise Fermat factoring ($n = a^2 - b^2$ with $a \approx \sqrt{n}$) finds them quickly. Both $p - 1$ and $q - 1$ should have a large prime factor (against Pollard's $p - 1$ method), and p/q should not be close to a fraction with small numerator and denominator.
- The modulus must not be shared between users (common-modulus attack).
- $e = 2^{16} + 1$ or an integer close to it (against low-exponent attacks, while keeping encryption fast).
- If d leaks, Bob must change n, e and d, since one (e, d) pair reveals the factors of n.
- Messages must be padded with **OAEP** (Optimal Asymmetric Encryption Padding). OAEP mixes the message with a fresh random value r through two hash-based masks ($M_1 = P \oplus G(r)$, $M_2 = r \oplus H(M_1)$) and encrypts $M_1 \| M_2$. This makes RSA probabilistic (same P, different C each time), fills the block so $P^e > n$, and destroys the multiplicative structure, blocking the short-message, broadcast, cyclic-style and chosen-ciphertext tricks.

Corrections to the notes / slides

- Slide on trapdoor functions: "there is not a polynomial time solution" to factoring should read *no polynomial-time classical algorithm is known*; it is not proven impossible (Shor's quantum algorithm is polynomial).
- Slide on RSA size: a 512-bit prime has 154 or 155 decimal digits (2^{511} has 154, 2^{512} has 155); 1024 bits is 309 digits, as stated.

34 ElGamal and Elliptic Curve Cryptosystems ▸

▸ **Likely this year:** New this year: an ElGamal encrypt/decrypt with a small prime, "find all points on $E_p(a, b)$", one point addition or doubling, and "why must r be fresh / why ECC has smaller keys".

34.1 ElGamal cryptosystem ▸

▸ **Likely this year:** "Explain ElGamal key generation, encryption and decryption; prove decryption is correct; encrypt P with p, e_1, d, r given" (4–5 marks).

RSA rests on factoring. ElGamal rests on a different hard problem, the **discrete logarithm problem** (DLP, §27). Let p be a large prime and e_1 a **primitive root** of p, i.e. an element whose powers $e_1^1, e_1^2, \ldots, e_1^{p-1}$ run through all of $\mathbb{Z}_p^* = \{1, \ldots, p - 1\}$ (§26). Computing $e_2 = e_1^d \bmod p$ is easy by square-and-multiply, but given e_1, e_2 and p, finding $d = \log_{e_1} e_2 \bmod p$ is infeasible for large p. So $d \mapsto e_1^d$ is a one-way function, and Bob's private key is the exponent d hidden inside the public e_2.

The idea of encryption is to multiply P by a **one-time mask** e_2^r that only Bob can rebuild. Alice picks a fresh random r and sends two numbers: $C_1 = e_1^r$, a "hint" that reveals nothing about r (DLP again), and $C_2 = P \cdot e_2^r$, the masked plaintext. Bob rebuilds the mask from the hint with his secret: $C_1^d = e_1^{rd} = (e_1^d)^r = e_2^r$. Eve would need either d or r, and both are discrete logarithms. This is the Diffie–Hellman idea turned into a cipher: $e_2^r = e_1^{dr}$ is a shared value that Alice computes from (r, e_2) and Bob from (d, C_1).

★ MUST KNOW : ElGamal

Key generation (Bob): choose a large prime p and a primitive root e_1 of p; choose d with $1 \le d \le p-2$; compute $e_2 = e_1^d \bmod p$. Public key (e_1, e_2, p), private key d.
Encryption (Alice): choose a **fresh** random $r \in \mathbb{Z}_p^*$; $C_1 = e_1^r \bmod p$, $C_2 = (P \times e_2^r) \bmod p$. Send (C_1, C_2).
Decryption (Bob): $P = \left[C_2 \times (C_1^d)^{-1}\right] \bmod p$, and $(C_1^d)^{-1} = C_1^{p-1-d} \bmod p$.

Proof of correctness. $C_2 \times (C_1^d)^{-1} = (P \cdot e_2^r)(e_1^{rd})^{-1} = P \cdot e_2^r (e_2^r)^{-1} = P \pmod p$, because $e_1^{rd} = (e_1^d)^r = e_2^r$. The inverse exists since $C_1^d \in \mathbb{Z}_p^*$.

The inverse shortcut. By Fermat, $C_1^{p-1} \equiv 1$, so $C_1^d \cdot C_1^{p-1-d} \equiv 1$, i.e. $(C_1^d)^{-1} = C_1^{p-1-d}$. One exponentiation replaces an exponentiation followed by an extended-Euclid inverse.

Properties. ElGamal is **probabilistic**: the same P encrypted twice gives different ciphertexts because r changes, so Eve cannot test guesses by re-encrypting them (unlike textbook RSA). The price is **ciphertext expansion**: the ciphertext is the pair (C_1, C_2), each as long as p, so it is **twice the size** of the plaintext. Like RSA, it is slow and is used for short data such as session keys.

WORKED EXAMPLE : the slide example

Q. Bob chooses $p = 11$, $e_1 = 2$, $d = 3$. Find his public key. Alice encrypts $P = 7$ with $r = 4$. Find the ciphertext and show Bob's decryption.
Keys. 2 is a primitive root of 11 (its powers are $2, 4, 8, 5, 10, 9, 7, 3, 6, 1$). $e_2 = 2^3 \bmod 11 = 8$. Public key $(2, 8, 11)$, private key 3.
Encrypt. $C_1 = 2^4 \bmod 11 = 16 \bmod 11 = 5$. $e_2^r = 8^4 = 4096 = 372 \cdot 11 + 4$, so $C_2 = 7 \cdot 4 = 28 \bmod 11 = 6$. Ciphertext $(\mathbf{5}, \mathbf{6})$.
Decrypt. $C_1^d = 5^3 = 125 = 11 \cdot 11 + 4 \equiv 4$; $4^{-1} \bmod 11 = 3$ ($4 \cdot 3 = 12$). $P = 6 \cdot 3 = 18 \bmod 11 = \mathbf{7}$ ✓.
Shortcut. $(C_1^d)^{-1} = 5^{11-1-3} = 5^7$: $5^2 \equiv 3$, $5^4 \equiv 9$, $5^7 = 9 \cdot 3 \cdot 5 = 135 = 12 \cdot 11 + 3 \equiv 3$ ✓.

WORKED EXAMPLE : new exam-style example

Q. In ElGamal, $p = 23$, $e_1 = 5$, Bob's private key $d = 6$. (a) Find e_2. (b) Encrypt $P = 20$ with $r = 3$. (c) Decrypt, and verify with the shortcut.
(a) $5^2 = 25 \equiv 2$, $5^4 \equiv 4$, $e_2 = 5^6 = 5^4 \cdot 5^2 \equiv 4 \cdot 2 = \mathbf{8}$. Public key $(5, 8, 23)$. (5 is a primitive root of 23: its order is 22.)
(b) $C_1 = 5^3 = 125 = 5 \cdot 23 + 10 \Rightarrow C_1 = 10$. $e_2^r = 8^3 = 512 = 22 \cdot 23 + 6 \equiv 6$. $C_2 = 20 \cdot 6 = 120 = 5 \cdot 23 + 5 \Rightarrow C_2 = 5$. Ciphertext $(\mathbf{10}, \mathbf{5})$.
(c) $C_1^d = 10^6$: $10^2 = 100 \equiv 8$, $10^4 \equiv 64 \equiv 18$, $10^6 = 18 \cdot 8 = 144 \equiv 6$ (this is e_2^r again, as the proof says). $6^{-1} \bmod 23 = 4$ ($6 \cdot 4 = 24$). $P = 5 \cdot 4 = \mathbf{20}$ ✓.
Shortcut: $10^{23-1-6} = 10^{16}$: $10^8 = 18^2 = 324 = 14 \cdot 23 + 2 \equiv 2$, $10^{16} \equiv 4$ ✓ (the same inverse).

34.2 Security of ElGamal ▸

▸ **Likely this year:** "Why must Alice use a fresh r for every message?" with the algebra.

Low-modulus attack. If p is small, Eve solves the discrete logarithm $d = \log_{e_1} e_2$ once (by search or a sub-exponential algorithm), stores d, and decrypts every message sent to Bob for as long as he keeps the key. With the slide key she tries $2^1 = 2$, $2^2 = 4$, $2^3 = 8 = e_2$ and has $d = 3$. Hence p must be at least 1024 bits (about 309 digits).

Known-plaintext attack when r is reused. The mask e_2^r depends only on r, so two messages encrypted with the same r share the same mask. If Eve learns one plaintext P and sees both ciphertexts $C_2 = P \cdot e_2^r$ and $C_2' = P' \cdot e_2^r$, she removes the mask without knowing d or r:

$$e_2^r = C_2 \times P^{-1} \bmod p, \qquad P' = C_2' \times (e_2^r)^{-1} \bmod p.$$

In the example above ($p = 23$, $e_2 = 8$, $r = 3$), suppose Alice also sends $P' = 17$ with the same r: $C_2' = 17 \cdot 6 = 102 \equiv 10$. Eve, knowing $P = 20$ and $C_2 = 5$: $20^{-1} \bmod 23 = 15$, so $e_2^r = 5 \cdot 15 = 75 \equiv 6$; then $P' = 10 \cdot 6^{-1} = 10 \cdot 4 = 40 \equiv 17$ ✓. The repeated C_1 even tells Eve that r was reused. Alice must use a fresh random r for every message.

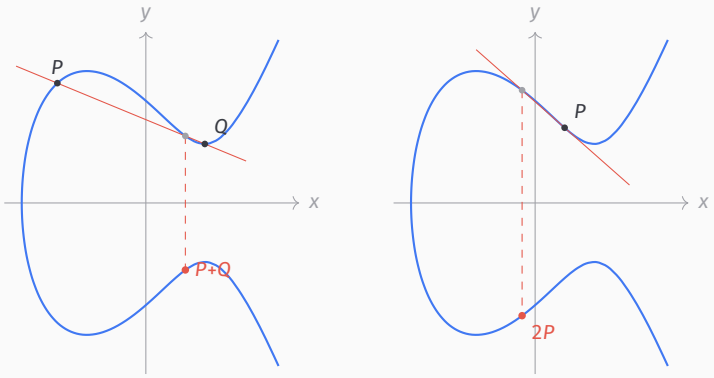
	RSA	ElGamal
Hard problem	factoring $n = pq$	discrete logarithm mod p
Public / private key	(e, n) / d	(e_1, e_2, p) / d
Encryption	deterministic, $C = P^e$	probabilistic, fresh r each time
Ciphertext size	same as plaintext	two numbers: twice the plaintext
Key generation	two primes, $d = e^{-1} \bmod \phi(n)$	one prime, $e_2 = e_1^d$
Main misuse	small e, no padding, shared n	reused r, small p

34.3 Elliptic curves over the real numbers

RSA and ElGamal are secure, but their security costs large keys (1024 bits and more). **Elliptic curve cryptography** (ECC) gets the same security with much smaller keys by replacing the group $\mathbb{Z}_p^*$ with the group of points on an elliptic curve, where the discrete logarithm is harder.

An **elliptic curve** is the set of points (x, y) satisfying the cubic $y^2 = x^3 + ax + b$. It is symmetric about the x-axis, since $(x, -y)$ satisfies the equation whenever (x, y) does. The curve must be **non-singular**, meaning $x^3 + ax + b$ has three distinct roots, which is equivalent to $4a^3 + 27b^2 \neq 0$. A singular curve has a cusp or a self-crossing where the tangent is undefined, and the group law below breaks there; for example $a = -3, b = 2$ gives $4(-27) + 27 \cdot 4 = 0$ and $x^3 - 3x + 2 = (x - 1)^2(x + 2)$ has a double root. The slide's curve $y^2 = x^3 - 3x + 3$ has $4a^3 + 27b^2 = -108 + 243 = 135 \neq 0$ and one real root, so it is a single connected curve.

Adding points (chord-and-tangent rule). To add $P \neq Q$, draw the line through P and Q; a line meets a cubic in three points, so it cuts the curve at a third point, and $R = P + Q$ is the **reflection** of that third point in the x-axis. To double P, use the **tangent** at P (the line through P and a point infinitely close to it), take its second intersection and reflect it. If the line is vertical ($Q = -P$, the reflection of P), there is no third point; we say it meets the curve at the **point at infinity** O, which is the identity: $P + (-P) = O$, $P + O = P$. With this rule the points form an abelian group (§28).



addition: chord doubling: tangent

Deriving the formulas. Let the line through $P = (x_1, y_1)$ have slope s: $y = s(x - x_1) + y_1$. Substituting into $y^2 = x^3 + ax + b$ gives a cubic in x whose expanded form starts $x^3 - s^2x^2 + \cdots = 0$. Its three roots are the x-coordinates of the three intersection points, and the sum of the roots of a monic cubic is minus the x^2 coefficient, so $x_1 + x_2 + x_3 = s^2$, giving $x_3 = s^2 - x_1 - x_2$. The third point is $(x_3,\ s(x_3 - x_1) + y_1)$; reflecting it gives $y_3 = s(x_1 - x_3) - y_1$. For addition, s is the chord slope $(y_2 - y_1)/(x_2 - x_1)$. For doubling, s is the tangent slope: differentiating $y^2 = x^3 + ax + b$ implicitly gives $2y\,y' = 3x^2 + a$, so $s = (3x_1^2 + a)/(2y_1)$, and $x_2 = x_1$. The constant b vanishes on differentiation, which is why the doubling slope contains a and never b.

34.4 Elliptic curves over GF(p) ▸

▸ **Likely this year:** Find all the points on $E_{13}(1, 1)$ / $E_{11}(1, 6)$" and find the inverse of a point" (3–4 marks).

Cryptography cannot use real numbers (rounding errors, unbounded size), so the same equation is taken modulo a prime $p > 3$. The curve $E_p(a, b)$ is the set of all pairs $(x, y) \in \mathbb{Z}_p \times \mathbb{Z}_p$ with

$$y^2 \equiv x^3 + ax + b \pmod{p}, \qquad a, b \in \mathbb{Z}_p, \quad 4a^3 + 27b^2 \not\equiv 0 \pmod{p},$$

together with the point at infinity O. The curve" is now a finite scatter of points, but the addition formulas derived above still define a group, with every division done as multiplication by an inverse modulo p.

Finding the points. For each $x = 0, 1, \ldots, p - 1$, compute $w = (x^3 + ax + b) \bmod p$, which must equal y^2. If $w = 0$ there is one point $(x, 0)$. If w is a nonzero **quadratic residue** (a perfect square mod p), there are two points $(x, \sqrt{w})$ and $(x, p - \sqrt{w})$. If w is not a square, no point has that x. Make the table of squares $y^2 \bmod p$ first; since $(p - y)^2 \equiv y^2$, only $y = 0, \ldots, (p - 1)/2$ are needed.

Negatives. The inverse of $P = (x, y)$ is $-P = (x, -y) = (x, p - y)$: the same x, only y flipped (the reflection in the x-axis). A point with $y = 0$ is its own inverse. Points therefore come in vertical pairs $(x, y), (x, p - y)$.

Hasse's theorem. The number of points $\#E$ (including O) satisfies $p + 1 - 2\sqrt{p} \leq \#E \leq p + 1 + 2\sqrt{p}$: about half the x values give a square and those give two points each, so #E is close to $p + 1$. It is a quick check on a point count.

WORKED EXAMPLE : Forouzan/slide, all points of $E_{13}(1,1)$

Q. Find all points on $E_{13}(1,1)$: $y^2 = x^3 + x + 1 \pmod{13}$. List the inverse pairs.
Valid curve: $4 + 27 = 31 \equiv 5 \neq 0 \pmod{13}$. **Squares mod 13:** $y = 0..6$ give $y^2 = 0, 1, 4, 9, 3, 12, 10$, so the nonzero squares are $\{1, 3, 4, 9, 10, 12\}$ with roots $1 \to \pm 1$, $3 \to \pm 4$, $4 \to \pm 2$, $9 \to \pm 3$, $10 \to \pm 6$, $12 \to \pm 5$.

x	$x^3 + x + 1$	w	square?	points
0	1	1	yes	(0, 1), (0, 12)
1	3	3	yes	(1, 4), (1, 9)
2	11	11	no	–
3	31	5	no	–
4	69	4	yes	(4, 2), (4, 11)
5	131	1	yes	(5, 1), (5, 12)
6	223	2	no	–
7	351	0	$y = 0$	(7, 0)
8	521	1	yes	(8, 1), (8, 12)
9	739	11	no	–
10	1011	10	yes	(10, 6), (10, 7)
11	1343	4	yes	(11, 2), (11, 11)
12	1741	12	yes	(12, 5), (12, 8)

17 points + O = 18. Each row's two points are inverses (e.g. $4 + 9 = 13$, so $(1, 9) = -(1, 4)$); $(7, 0)$ is its own inverse. $x = 2, 3, 6, 9$ give non-squares. Hasse: $14 \pm 2\sqrt{13} = [6.8,\ 21.2]$ contains 18 ✓.

WORKED EXAMPLE : new, all points of $E_{11}(1,6)$

Q. Find all points on $y^2 = x^3 + x + 6 \pmod{11}$ and verify Hasse's bound.
Valid: $4 + 27 \cdot 36 = 976 = 88 \cdot 11 + 8 \neq 0$. **Squares mod 11:** $y = 0..5$ give $0, 1, 4, 9, 5, 3$; nonzero squares $\{1, 3, 4, 5, 9\}$ with roots $1 \to \pm 1$, $3 \to \pm 5$, $4 \to \pm 2$, $5 \to \pm 4$, $9 \to \pm 3$.

x	x^3+x+6	w	sq?	points	x	x^3+x+6	w	sq?	points
0	6	6	no	–	6	228	8	no	–
1	8	8	no	–	7	356	4	yes	(7, 2), (7, 9)
2	16	5	yes	(2, 4), (2, 7)	8	526	9	yes	(8, 3), (8, 8)
3	36	3	yes	(3, 5), (3, 6)	9	744	7	no	–
4	74	8	no	–	10	1016	4	yes	(10, 2), (10, 9)
5	136	4	yes	(5, 2), (5, 9)					

12 points + O = 13. Hasse: $12 \pm 2\sqrt{11} = [5.4,\ 18.6]$ contains 13 ✓. Since 13 is prime, every point except O generates the whole group (Lagrange).

34.5 Point addition and doubling over GF(p) ▸

▸ **Likely this year:** "Add P and Q / find $2P$ on $E_p(a, b)$" (2–3 marks), always with modular inverses.

★ MUST KNOW : EC group law over GF(p)

For $P = (x_1, y_1)$, $Q = (x_2, y_2)$, $R = P + Q = (x_3, y_3)$ with

$$x_3 = s^2 - x_1 - x_2 \bmod p, \qquad y_3 = s(x_1 - x_3) - y_1 \bmod p,$$

$$s = \frac{y_2 - y_1}{x_2 - x_1} \bmod p \ \ (P \neq Q,\ \text{addition}), \qquad s = \frac{3x_1^2 + a}{2y_1} \bmod p \ \ (P = Q,\ \text{doubling}).$$

Division means multiplying by the inverse mod p. Special cases: $P + O = P$; if $x_1 = x_2$ and $y_2 = -y_1$ then $P + Q = O$; doubling a point with $y_1 = 0$ gives O.

WORKED EXAMPLE : one addition and two doublings

Q1 (Forouzan style). On $E_{13}(1, 1)$, add $P = (4, 2)$ and $Q = (10, 6)$.
$s = (6 - 2)(10 - 4)^{-1} = 4 \cdot 6^{-1} = 4 \cdot 11 = 44 \equiv 5$ $(6 \cdot 11 = 66 = 5 \cdot 13 + 1)$.
$x_3 = 25 - 4 - 10 = 11$; $y_3 = 5(4 - 11) - 2 = -37 \equiv -37 + 39 = 2$. **P + Q = (11, 2)**, which is in the point list ✓.
Q2. On $E_{13}(1, 1)$, find $2P$ for $P = (1, 4)$.
$s = (3 \cdot 1^2 + 1)(2 \cdot 4)^{-1} = 4 \cdot 8^{-1} = 4 \cdot 5 = 20 \equiv 7$ $(8 \cdot 5 = 40 = 3 \cdot 13 + 1)$.
$x_3 = 49 - 1 - 1 = 47 \equiv 8$; $y_3 = 7(1 - 8) - 4 = -53 \equiv -53 + 65 = 12$. **2P = (8, 12)** ✓.

Q3 (slide). On $E_{17}(2,2)$: $y^2 = x^3 + 2x + 2$, find $2P$ for $P = (5,1)$.
$s = (3 \cdot 5^2 + 2)(2 \cdot 1)^{-1} = 77 \cdot 2^{-1} \equiv 9 \cdot 9 = 81 \equiv 13$ $(77 = 4 \cdot 17 + 9,\ 2^{-1} = 9)$.
$x_3 = 169 - 5 - 5 = 159 = 9 \cdot 17 + 6 \equiv 6$; $y_3 = 13(5 - 6) - 1 = -14 \equiv 3$. $\mathbf{2P} = (\mathbf{6}, \mathbf{3})$.
Check: $3^2 = 9$ and $6^3 + 2 \cdot 6 + 2 = 230 = 13 \cdot 17 + 9$ ✓. (This curve has 19 points and $(5,1)$ generates all of them.)

Trap: point arithmetic slips

The doubling slope uses a (from $3x^2 + a$), never b. Compute s as an integer mod p with a modular inverse, never as a fraction or decimal. $-P = (x, p - y)$ flips only y. Use the addition formula only for $P \neq Q$ (it would divide by $x_2 - x_1 = 0$). Reduce negative results by adding multiples of p, and check the answer lies on the curve.

34.6 Scalar multiplication and the ECDLP ▸

▸ **Likely this year:** "Compute kP for small k and "state the ECDLP / Hasse's theorem.

Multiplying a point by an integer k means adding it to itself: $kP = P + P + \cdots + P$ (k times). It is computed by **double-and-add**, the analogue of square-and-multiply: scan the bits of k, double at every bit and add P at every 1-bit, so kP costs about $\log_2 k$ doublings. The slides' two examples on $E_{13}(1,1)$, recomputed:

- $4 \cdot (1,4) = 2 \cdot (2 \cdot (1,4)) = 2 \cdot (8,12)$: $s = (3 \cdot 64 + 1) \cdot 24^{-1} \equiv 11 \cdot 11^{-1} = 1$, $x_3 = 1 - 16 \equiv 11$, $y_3 = 1 \cdot (8 - 11) - 12 = -15 \equiv 11$, so $4 \cdot (1,4) = (\mathbf{11}, \mathbf{11})$ ✓.
- $3 \cdot (8,1) = 2 \cdot (8,1) + (8,1)$: doubling gives $s = 193 \cdot 2^{-1} \equiv 11 \cdot 7 \equiv 12$, $(11,2)$; then $(11,2) + (8,1)$: $s = (-1)(-3)^{-1} \equiv 12 \cdot 10^{-1} = 12 \cdot 4 \equiv 9$, $x_3 = 81 - 19 \equiv 10$, $y_3 = 9(11 - 10) - 2 = 7$, so $3 \cdot (8,1) = (\mathbf{10}, \mathbf{7})$ ✓.

$(1,4)$ has order 18, so it generates all of $E_{13}(1,1)$; its multiples (useful for checking the examples below) are:

k	1	2	3	4	5	6	7	8	9
$k(1,4)$	(1,4)	(8,12)	(0,12)	(11,11)	(5,1)	(10,6)	(12,8)	(4,2)	(7,0)
k	17	16	15	14	13	12	11	10	18
$k(1,4)$	(1,9)	(8,1)	(0,1)	(11,2)	(5,12)	(10,7)	(12,5)	(4,11)	O

The second row is the negative of the first ($(18 - k)P = -kP$), and $9P = (7,0)$ has $y = 0$, so $18P = 2 \cdot 9P = O$.

Elliptic curve discrete logarithm problem (ECDLP). Given a point P (ideally a generator) and $T = dP$, find d with $1 \le d \le \#E$. Computing T from d takes about $\log_2 d$ doublings, but no shortcut is known for the reverse: on a well-chosen curve the best attack is a generic one such as **Pollard's rho**, which needs about $\sqrt{\#E}$ steps. On the toy curve the table answers it at once ($T = (4,2) \Rightarrow d = 8$); for a 160-bit curve, $\sqrt{\#E} \approx 2^{80}$ steps is infeasible.

34.7 ECC simulating ElGamal ▸

▸ **Likely this year:** "Explain ECC encryption/decryption; verify the given example" (the slide's $E_{67}(2,3)$ example).

ElGamal works in any group where the discrete log is hard, so it carries over by a dictionary: multiplication mod p becomes point addition, exponentiation becomes scalar multiplication, and the inverse becomes the negative point. The plaintext must be encoded as a point P on the curve.

★ MUST KNOW : EC ElGamal

Keys (Bob): choose $E_p(a,b)$, a point e_1 on it, and an integer d; compute $e_2 = d \times e_1$. Public $(E_p(a,b), e_1, e_2)$, private d.
Encrypt (Alice): fresh random r; $C_1 = r \times e_1$, $C_2 = P + r \times e_2$.
Decrypt (Bob): $P = C_2 - (d \times C_1)$, where subtracting means adding the negative point.
Proof: $C_2 - dC_1 = P + r(de_1) - d(re_1) = P + (rd)e_1 - (dr)e_1 = P + O = P$.

WORKED EXAMPLE : the slide example on $E_{67}(2,3)$, every step

Q. Bob selects $E_{67}(2,3)$, $e_1 = (2,22)$, $d = 4$. Alice sends $P = (24,26)$ with $r = 2$. Find e_2, C_1, C_2, and verify decryption.

Step	Slope s (mod 67)	x_3	y_3	Result
$2e_1 = 2(2,22)$	$14 \cdot 44^{-1} = 14 \cdot 32 \equiv 46$	$2112 \equiv 35$	$-1540 \equiv 1$	$(35,1)$
$4e_1 = 2(35,1)$	$3677 \cdot 2^{-1} \equiv 59 \cdot 34 \equiv 63$	$3899 \equiv 13$	$1385 \equiv 45$	$(13,45)$
$2e_2 = 2(13,45)$	$509 \cdot 90^{-1} \equiv 40 \cdot 35 \equiv 60$	$3574 \equiv 23$	$-645 \equiv 25$	$(23,25)$
$P + 2e_2$	$(-1)(-1)^{-1} = 1$	$-46 \equiv 21$	$-23 \equiv 44$	$(21,44)$
$C_2 + (23,42)$	$(-2) \cdot 2^{-1} \equiv 66$	$4312 \equiv 24$	$-242 \equiv 26$	$(24,26)$

Keys: $e_2 = 4e_1 = (13,45)$. **Encrypt**: $C_1 = 2e_1 = (35,1)$, $re_2 = (23,25)$, $C_2 = (24,26) + (23,25) = (21,44)$.

Decrypt: $d \times C_1 = 4 \cdot (35, 1) = 8e_1 = 2e_2 = (23, 25)$ (two doublings, rows 2–3). Its negative is $(23, 67 - 25) = (23, 42)$, and $(21, 44) + (23, 42) = (\mathbf{24, 26}) = P$ ✓.
Inverses used: $44 \cdot 32 = 1408 = 21 \cdot 67 + 1$; $2 \cdot 34 = 68$; $23 \cdot 35 = 805 = 12 \cdot 67 + 1$ ($90 \equiv 23$).

WORKED EXAMPLE : new exam-style example on $E_{13}(1, 1)$

Q. Bob uses $E_{13}(1, 1)$ with $e_1 = (1, 4)$ and private key $d = 3$. Alice encrypts the point $P = (4, 2)$ with $r = 2$. Find Bob's public key, the ciphertext, and verify decryption.

Step	Slope s (mod 13)	x_3	y_3	Result
$2e_1 = 2(1, 4)$	$4 \cdot 8^{-1} = 4 \cdot 5 \equiv 7$	$47 \equiv 8$	$-53 \equiv 12$	$(8, 12)$
$e_2 = (8, 12) + (1, 4)$	$(-8)(-7)^{-1} = 5 \cdot 6^{-1} = 5 \cdot 11 \equiv 3$	0	12	$(0, 12)$
$2e_2 = 2(0, 12)$	$1 \cdot 24^{-1} = 11^{-1} = 6$	$36 \equiv 10$	$-72 \equiv 6$	$(10, 6)$
$C_2 = (4, 2) + (10, 6)$	$4 \cdot 6^{-1} \equiv 5$	11	$-37 \equiv 2$	$(11, 2)$
$2C_1 = 2(8, 12)$	$193 \cdot 24^{-1} \equiv 11 \cdot 6 \equiv 1$	$-15 \equiv 11$	$-15 \equiv 11$	$(11, 11)$
$3C_1 = (11, 11) + (8, 12)$	$1 \cdot (-3)^{-1} = 10^{-1} = 4$	$-3 \equiv 10$	$-7 \equiv 6$	$(10, 6)$
$C_2 + (10, 7)$	$5 \cdot (-1)^{-1} = 5 \cdot 12 \equiv 8$	$43 \equiv 4$	$54 \equiv 2$	$(4, 2)$

Public key $(E_{13}(1, 1), (1, 4), (0, 12))$. **Ciphertext** $C_1 = 2e_1 = (\mathbf{8, 12})$, $C_2 = (\mathbf{11, 2})$.
Decrypt: $d \times C_1 = 3 \cdot (8, 12) = (10, 6)$, the same point as $r \times e_2$; negative $(10, 7)$; $(11, 2) + (10, 7) = (\mathbf{4, 2})$ ✓. In multiples of e_1 (table above): $e_2 = 3e_1$, $C_1 = 2e_1$, $P = 8e_1$, $C_2 = (8 + 6)e_1 = 14e_1$, and $14 - 6 = 8$ ✓.

34.8 Security of ECC and key size ▸

▸ **Likely this year:** "Why does ECC need smaller keys than RSA for the same security? (2 marks).

To decrypt, Eve needs r (then $P = C_2 - r \times e_2$) or d (then $P = C_2 - d \times C_1$). Finding r from $C_1 = r \times e_1$, or d from $e_2 = d \times e_1$, is the ECDLP, which for large p (or large n in GF(2^n)) is infeasible by Pollard's rho. As in ElGamal, r must be fresh for every message.

Why ECC keys are smaller. For factoring and for the discrete log in $\mathbb{Z}_p^*$ there are **sub-exponential** algorithms (the number field sieve, index calculus) that exploit the arithmetic structure of integers, so RSA and ElGamal need 1024-bit and larger moduli to stay ahead. On a well-chosen elliptic curve no such structure is known, and only generic attacks costing about $\sqrt{\#E}$ work, i.e. $2^{k/2}$ for a k-bit curve. A 160-bit curve therefore costs about 2^{80} steps, the same effort as factoring a 1024-bit RSA modulus: ECC over GF(2^n) or GF(p) with a 160-bit modulus gives the same security as RSA with a 1024-bit modulus (and 256-bit ECC ≈ 3072-bit RSA). Smaller keys mean less computation, bandwidth and storage, which suits smart cards and mobile devices.

Corrections to the notes / slides

- ElGamal block diagram: the encryption box labels both lines C_1; the second line, $(e_2^r \cdot P)$ mod p, is C_2.
- $E_{67}(2, 3)$ example, step 8: "He uses $2 \times C_1$" should read $d \times C_1 = 4 \times C_1 = (23, 25)$. With $C_1 = 2e_1$, $2 \times C_1 = 4e_1 = e_2 = (13, 45)$, not $(23, 25)$ (recomputed). The same misprint is in Forouzan.
- ElGamal security slide: 1024 bits is about **309** decimal digits (as on the RSA slide), not 300.

Part V
Integrity, Authentication and Key Management

35 Message Integrity and Hash Functions

35.1 Integrity versus secrecy; message and digest ▸

▸ **Likely this year:** "Why is a hash, not encryption, used for integrity?" and "how is integrity checked with a digest?" (2–3 marks).

Every scheme so far gave *secrecy*: Eve cannot read the message. Many documents need the opposite guarantee. Alice's will, for example, need not be encrypted, because after her death anyone may read it; what must be preserved is its *integrity*, meaning that nobody can change its contents without the change being noticed. On paper this is achieved by putting Alice's fingerprint at the bottom of the document: Eve can read the will, but she cannot alter it or write a false one, because she cannot forge the fingerprint.

The electronic equivalent of the document–fingerprint pair is the **message–digest pair**. The message is passed through a **cryptographic hash function** h, which maps a message of any length to a short fixed-length string

$h(M)$, called the **message digest**. The digest is a compressed image of the message that plays the role of the fingerprint. To check integrity later, the receiver runs the same hash function on the message he now holds and compares the new digest with the stored one: if the two digests are equal, the message has not been changed; if they differ, the message was changed and is discarded.

Why a hash and not encryption. Encryption hides the message but does not stop Eve from altering the ciphertext; flipping a bit of a one-time-pad or stream-cipher ciphertext flips the same bit of the plaintext, and Bob has no way to tell a changed plaintext from a genuine one. A hash works the other way round: it hides nothing (the message can be public), but since any change to the message produces a different digest, any change is detected. The two services are therefore independent, and a message can need one, the other, or both.

The digest itself must be safe. The check only works if the stored or received digest is genuine. If Eve can replace both M and $h(M)$, she simply sends M' with $h(M')$ and the comparison passes. Hence the digest is kept in a safe place or sent over an authenticated channel; binding the digest to a secret key gives the MAC of the next lecture.

35.2 The three security criteria ▸

▸ **Likely this year:** "Define and differentiate preimage, second preimage and collision resistance" (3–4 marks), possibly with a figure of what Eve is given and must find.

A cryptographic hash function must make three kinds of forgery infeasible. They differ in how much Eve is given at the start, and therefore in how much freedom she has.

Preimage resistance. Given a hash function h and a digest $y = h(M)$, it must be extremely difficult for Eve to find *any* message M' with $h(M') = y$. If h were not preimage resistant, Eve could intercept just the digest $h(M)$, construct a message M' that hashes to it, and send M' to Bob as if it were M. In other words, the hash must be one-way.

Second preimage resistance. Given a specific message M *and* its digest $h(M)$, it must be extremely difficult to find another message $M' \neq M$ with $h(M') = h(M)$. This is the criterion that stops forgery of a particular message: if Alice sends M and $h(M)$ to Bob, Eve cannot create a different message with the same digest and substitute it. Eve here has more information than in the preimage case (she also knows one valid preimage), but she still has to hit one fixed target value.

Collision resistance. It must be extremely difficult to find *any* two messages $M \neq M'$ with $h(M) = h(M')$. Here Eve is given nothing and is free to choose both messages from scratch; such a pair is called a **collision**. This is the danger when Eve prepares two documents herself (say a fair contract and an unfair one with the same digest), gets Alice to approve the fair one, and later swaps in the other.

Criterion	Eve is given	Eve must find	Attack it stops
Preimage resistance	h and a digest $y = h(M)$	any M' with $h(M') = y$	preimage
Second preimage resistance	h, a message M and $h(M)$	$M' \neq M$ with $h(M') = h(M)$	second preimage
Collision resistance	only h	any pair $M \neq M'$ with $h(M) = h(M')$	collision

Collision resistance implies second preimage resistance. Suppose Eve had a method that, given any M, produces a second preimage M'. She could then pick any M she likes, run the method, and output the pair (M, M'), which is a collision. So a function that is *not* second-preimage resistant cannot be collision resistant; equivalently, a collision-resistant function is automatically second-preimage resistant. The converse fails, because finding a collision is the easier task (Eve chooses both sides), and the birthday analysis below shows by how much: a collision costs about $2^{n/2}$ hash evaluations, a second preimage about 2^n.

Why a compressor or a checksum is not a cryptographic hash. A lossless compression method such as ZIP or StuffIt also shrinks the message, but it is reversible: anyone can decompress the output and get the original, so it is not even a function to a fixed-length output, and it fails preimage resistance trivially. A checksum (adding up the bytes, a CRC) is fixed-length but linear and easy to invert: given a target checksum, Eve can compute what to change in any message to reach it, so it is neither preimage nor second-preimage resistant. Checksums detect accidental errors, not deliberate ones.

★ MUST KNOW : the three criteria

Preimage: given y, find any M' with $h(M') = y$. **Second preimage**: given M, find $M' \neq M$ with $h(M') = h(M)$. **Collision**: find any $M \neq M'$ with $h(M) = h(M')$.
Cost for an ideal n-bit hash: preimage and second preimage $\approx 2^n$, collision $\approx 2^{n/2}$. Collision resistance $\Rightarrow$ second preimage resistance.

35.3 The random oracle model ▸

▸ **Likely this year:** "Explain the random oracle model; why can the oracle not use a formula?" (2–3 marks).

To analyse attacks we need a model of a perfect hash function, against which real designs are compared. The

random oracle model (Bellare and Rogaway, 1993) describes such an ideal function as an oracle that keeps a table of the messages it has already hashed and obeys three rules:

1. When a *new* message of any length is given, the oracle creates a fixed-length digest that is a truly random string of 0s and 1s, gives it out, and records the message and digest in its table.
2. When a message is given for which a digest already exists, the oracle simply returns the digest in the table. This makes the function deterministic: the same message always gets the same digest.
3. The digest for a new message must be chosen independently of all previous digests. This implies that the oracle cannot use a formula or an algorithm to calculate the digest.

Example (slides 20–23). An oracle with 16-bit digests already holds three entries. It is asked to hash AB1234CD8765BDAD, which is not in its table, so it flips a fair coin 16 times and gets HHTHHHTTHTHHTTTH. Reading H as 1 and T as 0 gives 1101 1100 1011 0001 = DCB1, which it returns and records. When it is then asked for 4523AB1352CDEF45126, which is already in the table, it does not flip again but returns the stored digest 13AB.

Message	Digest
4523AB1352CDEF45126	13AB
723BAE38F2AB3457AC	02CA
AB1234CD8765BDAD	DCB1 (new)
AB45CD1048765412AAAB6662BE	A38B

Why a formula violates rule 3. Suppose the oracle used $h(M) = M \bmod n$ and had already issued $h(M_1)$ and $h(M_2)$. For the new message $M_3 = M_1 + M_2$ anyone can predict the digest without asking, since

$$h(M_3) = (M_1 + M_2) \bmod n = [\, h(M_1) + h(M_2) \,] \bmod n.$$

The digest of M_3 is therefore not independent of the earlier digests, which breaks rule 3. Real hash functions are algorithms, so they can only approximate the oracle; the model tells us the best security an n-bit design can hope for.

What the model gives an attacker. With an n-bit digest there are $N = 2^n$ possible digests, and each new message receives one of them uniformly at random. Some values may be chosen several times and some never. Eve knows the algorithm and n, so every attack reduces to drawing random digests and waiting for a match, which is exactly a birthday problem.

35.4 The four birthday problems ▸

▸ **Likely this year:** Derive the probability formula for one problem and get k for $P = 1/2$; or "how many people for a shared birthday" (23).

All four problems use a random variable that is uniform over N values (0 to $N - 1$; $N = 365$ for birthdays) and ask for the minimum number of samples k that makes an event *likely*, meaning probability $P \geq 1/2$. Every derivation uses the approximation $1 - x \approx e^{-x}$ for small x.

Problem 1: a sample equals a predefined value. What is the minimum number of students such that it is likely that at least one has a *predefined* birthday (say 1 January)? A single sample misses the target with probability $1 - 1/N$, and the k samples are independent, so all of them miss with probability $(1 - 1/N)^k \approx e^{-k/N}$. Hence

$$P_1 \approx 1 - e^{-k/N}, \qquad k \approx \ln \frac{1}{1-P} \times N \xrightarrow{P=1/2} k \approx \ln 2 \times N = 0.69N.$$

For $N = 365$: $0.693 \times 365 = 252.99$, so $k = 253$.

Problem 2: a sample equals one chosen from the set. What is the minimum number such that it is likely that at least one student has the same birthday as a student *selected by the professor*? The selected student is one of the k, so only the other $k - 1$ can match, and the problem is Problem 1 with $k - 1$ samples:

$$P_2 \approx 1 - e^{-(k-1)/N}, \qquad k \approx \ln 2 \times N + 1 = 0.69N + 1; \qquad N = 365 : \; k = 254.$$

Problem 3: any two samples are equal (the birthday paradox). What is the minimum number such that it is likely that *at least two* students share a birthday? It is easier to count the complement, that all k values are different. The first value is free, the second must avoid 1 value, the i-th must avoid $i - 1$ values, so

$$\Pr[\text{all different}] = \prod_{i=1}^{k-1} \left(1 - \frac{i}{N}\right) \approx \prod_{i=1}^{k-1} e^{-i/N} = e^{-(1+2+\cdots+(k-1))/N} = e^{-k(k-1)/2N}.$$

Therefore

$$P_3 \approx 1 - e^{-k(k-1)/2N}, \qquad k \approx \left\{2 \ln \frac{1}{1-P}\right\}^{1/2} N^{1/2} \xrightarrow{P=1/2} k \approx \sqrt{2 \ln 2}\,\sqrt{N} = 1.18\sqrt{N},$$

using $k(k-1) \approx k^2$. For $N = 365$: $1.18 \times 19.1 = 22.5$, so $k = 23$ (exactly, 23 people give $P = 0.507$ and 22 give 0.476). The answer is a "paradox" only because intuition compares each person with *one* date, whereas k people form $k(k-1)/2 = 253$ pairs, each a chance to match.

Problem 4: a sample of one set equals a sample of another. Two classes each have k students. What is the minimum k such that it is likely that some student of the first class shares a birthday with some student of the second? Treat the k values of set 1 as (nearly) distinct. Each value of set 2 then misses all of them with probability $1 - k/N$, and the k values of set 2 all miss with probability $(1 - k/N)^k \approx e^{-k^2/N}$. Hence

$$P_4 \approx 1 - e^{-k^2/N}, \qquad k \approx \left\{\ln \frac{1}{1-P}\right\}^{1/2} N^{1/2} \xrightarrow{P=1/2} k \approx \sqrt{\ln 2}\,\sqrt{N} = 0.83\sqrt{N}; \qquad N = 365 : \ k = 16.$$

(Exactly, $k = 16$ gives $P = 0.504$.)

Problem	Probability P	General k	k at $P = \frac{1}{2}$	$N = 365$	Attack
1	$1 - e^{-k/N}$	$\ln \frac{1}{1-P} \cdot N$	$0.69N$	253	preimage
2	$1 - e^{-(k-1)/N}$	$\ln \frac{1}{1-P} \cdot N + 1$	$0.69N + 1$	254	second preimage
3	$1 - e^{-k(k-1)/2N}$	$\{2 \ln \frac{1}{1-P}\}^{1/2} N^{1/2}$	$1.18\sqrt{N}$	23	collision
4	$1 - e^{-k^2/N}$	$\{\ln \frac{1}{1-P}\}^{1/2} N^{1/2}$	$0.83\sqrt{N}$	16	alternate collision

The constants are $0.69 = \ln 2$, $1.18 = \sqrt{2 \ln 2}$ and $0.83 = \sqrt{\ln 2}$. The key comparison is that k **grows like N in Problems 1 and 2 but only like $\sqrt{N}$ in Problems 3 and 4.** Problems 1 and 2 model the preimage and second preimage attacks, Problems 3 and 4 the collision attacks, so a collision attack is far cheaper than a preimage attack on the same hash.

Trap: rounding the constant

With 0.69 exactly, $0.69 \times 365 = 251.85$, which would suggest 252; the table value 253 comes from $\ln 2 = 0.6931$ ($0.6931 \times 365 = 252.99$). Use 0.693 when the answer must be an exact head count. Also do not mix up Problems 1 and 3: "someone has *my* birthday" needs 253 people, "*some two* share a birthday" needs only 23.

WORKED EXAMPLE : birthday numbers for a 16-bit digest

Q. A hash function produces 16-bit digests. Using the birthday results, how many random messages must Eve hash for a probability of at least 1/2 of (a) matching a given digest, (b) finding any two messages with the same digest, (c) a match between two lists of k messages each?
A. $N = 2^{16} = 65\,536$ and $\sqrt{N} = 2^8 = 256$.
(a) Problem 1: $k \approx 0.69 \times 65\,536 \approx 45\,220$ messages (order 2^{16}).
(b) Problem 3: $k \approx 1.18 \times 256 \approx$ **302** messages (order 2^8).
(c) Problem 4: $k \approx 0.83 \times 256 \approx$ **213** messages in each list.
Check: (b) is about 150 times cheaper than (a), as $2^{16}/2^8 = 256$ predicts up to the constants.

35.5 Attacks on an n-bit digest ▸

▸ **Likely this year:** "A hash uses a 64-bit digest; how many digests must Eve create to find a collision (or the original message) with $P > 0.5$, and how long does it take?" (3–4 marks).

Assume the hash behaves like a random oracle with n-bit digests, so its outputs are uniform over $N = 2^n$ values, and that Eve knows the algorithm and n. Each attack below is a loop that creates messages, hashes them, and compares digests; its success probability is the matching birthday problem.

Preimage attack. Eve has intercepted a digest $D = h(M)$ and wants any M' with $h(M') = D$. She creates k messages $M[1], \ldots, M[k]$, computes $T = h(M[i])$ for each, and returns $M[i]$ as soon as $T = D$. The target D is a predefined value, so this is Problem 1: $P = 1 - e^{-k/N}$, and success with $P \geq 1/2$ needs $k = 0.69 \times 2^n$ digests, which is of order 2^n.

Second preimage attack. Eve has intercepted both M and $D = h(M)$ and wants $M' \neq M$ with $h(M') = D$. She creates $k - 1$ further messages and compares each digest with D. The target is one of the k samples (the given M), so this is Problem 2: $P = 1 - e^{-(k-1)/N}$ and $k = 0.69 \times 2^n + 1$, again of order 2^n.

Collision attack. Eve wants any $M \neq M'$ with $h(M) = h(M')$. She creates k messages, stores the digests $D[1], \ldots, D[k]$, and compares each new digest with all earlier ones. Any two equal digests suffice, so this is Problem 3: $P = 1 - e^{-k(k-1)/2N}$ and $k = 1.18 \times 2^{n/2}$, of order only $2^{n/2}$. The square root comes from the number of pairs: k digests give about $k^2/2$ pairs, and once $k^2/2$ is comparable to 2^n a coincidence is likely.

Alternate collision attack. A random collision is usually useless, because the two colliding messages are meaningless strings. A real forger needs a collision between a *genuine* message M (which Alice will approve) and a *bogus* message M' (which Eve will later substitute), both meaningful. Eve therefore makes k variants of each without changing their meaning (an extra space after a word, a synonym, "will not" versus "won't", a redundant

phrase): $M_1, \ldots, M_k$ and $M'_1, \ldots, M'_k$. She hashes both lists and looks for $h(M_i) = h(M'_j)$. A match must be *across* the two lists, so this is Problem 4: $P = 1 - e^{-k^2/N}$ and $k = 0.83 \times 2^{n/2}$ variants of each, still of order $2^{n/2}$. Variants are cheap to produce: if a letter has $n/2$ independent places where one of two equivalent forms can be used, it has $2^{n/2}$ variants.

Attack	Birthday problem	k for $P = 1/2$	Order
Preimage	1	0.69×2^n	2^n
Second preimage	2	$0.69 \times 2^n + 1$	2^n
Collision	3	$1.18 \times 2^{n/2}$	$2^{n/2}$
Alternate collision	4	$0.83 \times 2^{n/2}$	$2^{n/2}$

WORKED EXAMPLE : a 64-bit digest (slides 27 and 32)

Q. A cryptographic hash function uses a 64-bit digest. (a) How many digests must Eve create to find the original message with probability more than 0.5, and how long does it take at 2^{30} (about one billion) digests per second? (b) How many digests must she create to find two messages with the same digest with probability more than 0.5, and how long does it take at 2^{20} (about one million) digests per second? Is a 64-bit digest secure?
A. $n = 64$, $N = 2^{64}$.
(a) Preimage attack (Problem 1): $k = 0.69 \times 2^{64} \approx 1.27 \times 10^{19}$ digests.
Time $= 0.69 \times 2^{64}/2^{30} = 0.69 \times 2^{34}$ s $= 1.185 \times 10^{10}$ s $\div (3.156 \times 10^7$ s/year$) \approx$ **376 years**. Secure against the preimage attack.
(b) Collision attack (Problem 3): $k = 1.18 \times 2^{64/2} = 1.18 \times 2^{32} \approx 5.07 \times 10^9$ digests.
Time $= 1.18 \times 2^{32}/2^{20} = 1.18 \times 2^{12}$ s $= 4833$ s $\approx$ **1.34 hours**. **Not** secure against the collision attack, even with a computer a thousand times slower.
Check: the collision attack is cheaper by about 2^{32} in digests, exactly the gap between 2^n and $2^{n/2}$.

What digest length is needed. Since the cheapest generic attack costs about $2^{n/2}$ hash evaluations, an n-bit hash offers only $n/2$ bits of security against collisions. To demand at least 2^{80} work, the digest must have $n/2 \geq 80$, i.e. $n \geq 160$ bits, which is why SHA-1 has 160 bits and the SHA-2 family 224–512 bits. **MD5's 128-bit digest** gives only 2^{64} collision security: $1.18 \times 2^{64} \approx 2.2 \times 10^{19}$ digests, about 640 years on one machine at 2^{30} per second, but well within reach of a large cluster. In addition, structural attacks (Wang et al., 2004) now find MD5 collisions in minutes on a PC, so MD5 must not be used where collisions matter.

WORKED EXAMPLE : new exam-style numerical

Q. A hash function produces 40-bit digests and Eve can compute 2^{20} digests per second. (a) How long does a preimage attack with success probability 1/2 take? (b) How long does a collision attack take? (c) If Eve hashes 2^{20} messages, what is the probability that two of them collide?
A. $N = 2^{40}$.
(a) $k = 0.69 \times 2^{40}$; time $= 0.69 \times 2^{20}$ s $= 723\,517$ s $\approx$ **8.4 days**.
(b) $k = 1.18 \times 2^{20} \approx 1.24 \times 10^6$; time $= 1.18$ s, **about one second**.
(c) $P = 1 - e^{-k(k-1)/2N} \approx 1 - e^{-2^{40}/2^{41}} = 1 - e^{-0.5} =$ **0.39**.
Check: (b) is (a) divided by roughly 2^{20}, consistent with the orders 2^{40} and 2^{20}.

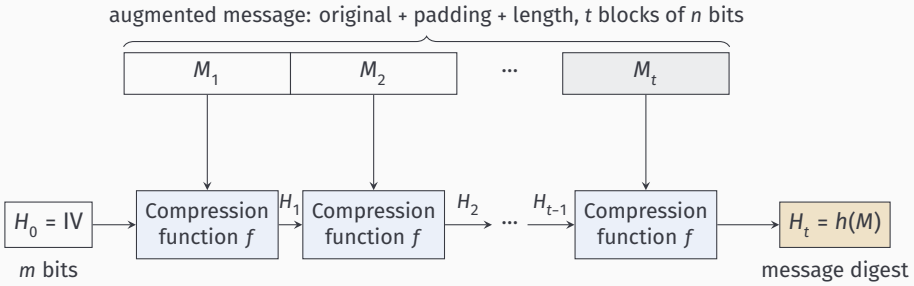
WORKED EXAMPLE : how many variants for an alternate collision

Q. Eve attacks a 64-bit hash with the alternate collision attack. How many variants of the genuine and the bogus letter does she need for success probability about 1/2, and how many two-way choice points must each letter contain? What is her success probability with 2^{32} variants of each?
A. Problem 4: $k = 0.83 \times 2^{64/2} = 0.83 \times 2^{32} \approx 3.56 \times 10^9$ variants of each letter. With m independent places where one of two equivalent forms can be chosen, a letter has 2^m variants, so $m = 32$ choice points per letter are enough ($2^{32} > 0.83 \times 2^{32}$).
With $k = 2^{32}$: $P = 1 - e^{-k^2/N} = 1 - e^{-2^{64}/2^{64}} = 1 - e^{-1} =$ **0.63**.

35.6 Iterated hash functions: the Merkle–Damgård scheme ▸

▸ **Likely this year:** "Explain the Merkle–Damgård scheme with a diagram" (3–4 marks).

A hash function must accept messages of any length and still produce a fixed-length digest, and designing one function for arbitrary input lengths directly is hard. The standard solution is **iteration**: design a **compression function** f with *fixed-size* input, which compresses an $(n + m)$-bit input (an n-bit message block and an m-bit chaining value) to m bits, and apply it as many times as the message requires. A hash built this way is an **iterated cryptographic hash function**, and the **Merkle–Damgård scheme** is the standard way of doing it.



The scheme works in five steps.

1. The message length and padding are appended to the message to create an **augmented message** whose length is a multiple of n, the block size of the compression function.
2. The augmented message is split into t blocks $M_1, M_2, \ldots, M_t$ of n bits each.
3. Before the iteration starts, the chaining value H_0 is set to a fixed public value called the **IV** (initial value).
4. Iteration i combines the previous digest with the next block: $H_i = f(H_{i-1}, M_i)$ for $i = 1, \ldots, t$.
5. The last chaining value H_t is the hash of the original message, $h(M) = H_t$.

Why it is collision resistant if f is. Suppose two different messages $M \neq M'$ gave the same final digest $H_t = H'_t$. Then the last compression steps $f(H_{t-1}, M_t)$ and $f(H'_{t-1}, M'_t)$ have equal outputs. If their inputs differ, that is a collision of f. If their inputs are equal, then $H_{t-1} = H'_{t-1}$ and the same argument applies one step earlier, and so on backwards. Since $M \neq M'$, the inputs must differ at some step, so a collision of h always yields a collision of f. The length field is essential to this argument: it puts the message length in the last block, so two messages of different lengths already differ in their final inputs and cannot be lined up to share a suffix. As a result, the designer only has to make one fixed-size function collision resistant.

35.7 Compression functions: from scratch or from a block cipher ▸

▸ **Likely this year:** "Differentiate Rabin, Davies–Meyer and Miyaguchi–Preneel; why is Rabin weak?" (3 marks).

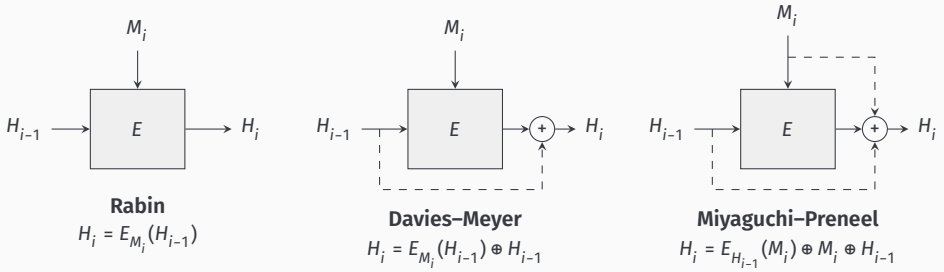
There are two ways to obtain the compression function f. In the first, f is **designed from scratch** specifically for hashing; in the second, an existing **symmetric-key block cipher** is used as f, so that the confidence gained in the cipher carries over.

Made from scratch: the MD and SHA families. The **Message Digest** family by Rivest has several versions (MD2, MD4, MD5). MD5 is a strengthened MD4 that divides the message into 512-bit blocks and creates a 128-bit digest; this digest turned out to be too short to resist collision attacks. The **Secure Hash Algorithm** family, the NIST standard also called the Secure Hash Standard (SHS), started with SHA (later SHA-1) and added SHA-224, SHA-256, SHA-384 and SHA-512 (together called SHA-2). All of them are Merkle–Damgård constructions.

Characteristic	MD5	SHA-1	SHA-224	SHA-256	SHA-384	SHA-512
Maximum message size (bits)	any*	$2^{64}-1$	$2^{64}-1$	$2^{64}-1$	$2^{128}-1$	$2^{128}-1$
Block size (bits)	512	512	512	512	1024	1024
Digest size (bits)	128	160	224	256	384	512
Number of rounds / steps	64	80	64	64	80	80
Word size (bits)	32	32	32	32	64	64

The SHA columns are the slide's table (slide 41); the MD5 column is added for comparison (*MD5 records the length modulo 2^{64}, so it accepts any length). The pattern to remember is that the 32-bit-word members use 512-bit blocks and the 64-bit-word members (SHA-384, SHA-512) use 1024-bit blocks; SHA-384 is SHA-512 with a different IV and a truncated digest.

Made from a block cipher. Let $E_K(P)$ be a block cipher with key K and plaintext P. One input of f is the message block M_i and the other is the chaining value H_{i-1}; the schemes differ in which goes into the key, which into the plaintext, and what is XORed onto the output.



In the Rabin and Davies–Meyer pictures the block M_i enters as the *key* (from above) and H_{i-1} as the plaintext; in Miyaguchi–Preneel it is the other way round: M_i is the plaintext and H_{i-1} the key (slide 45). Dashed lines are the feed-forward XORs.

Rabin scheme. Rabin plugs a block cipher directly into Merkle–Damgård: the message block is used as the key, the previous digest as the plaintext, and the ciphertext is the new digest, $H_i = E_{M_i}(H_{i-1})$. The digest size equals the cipher's block size, so with DES the digest is only 64 bits. Although the scheme is very simple, it is subject to a **meet-in-the-middle attack**. Because Eve knows every key (the message blocks are her choice), each step can be run *backwards*: $H_{i-1} = D_{M_i}(H_i)$. To forge a message with a given digest H_t, she encrypts forward from H_0 with about $2^{m/2}$ random choices of the first half of the blocks, stores the intermediate values, then decrypts backward from H_t with about $2^{m/2}$ random choices of the second half, and looks for a common intermediate value. A match gives a full message with digest H_t after only about $2 \times 2^{m/2}$ operations instead of 2^m; for DES that is about 2^{32}, which is feasible.

Davies–Meyer scheme. Davies–Meyer is the Rabin scheme plus a **feed-forward**: the previous digest is XORed onto the cipher output, $H_i = E_{M_i}(H_{i-1}) \oplus H_{i-1}$. Since H_{i-1} now appears both inside the encryption and in the XOR, knowing H_i and M_i no longer lets Eve compute H_{i-1} by decryption, so the backward half of the meet-in-the-middle attack is blocked. This is the protection the slide refers to.

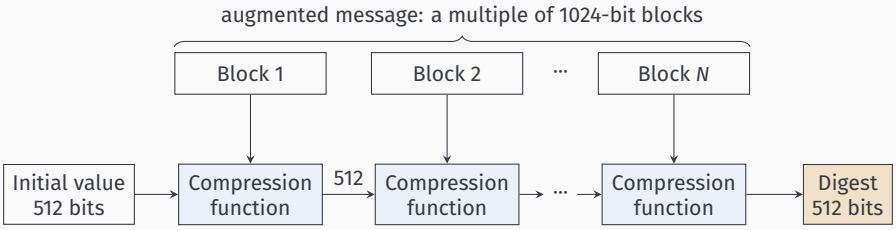
Matyas–Meyer–Oseas scheme (Forouzan, not on the slides). This is the dual of Davies–Meyer: the previous digest is the key (passed through a function g that adapts it to the key size) and the message block is the plaintext, with the block fed forward, $H_i = E_{g(H_{i-1})}(M_i) \oplus M_i$. It is useful when the key and block sizes of the cipher are equal.

Miyaguchi–Preneel scheme. To make the algorithm stronger, the plaintext, the cipher key and the ciphertext are all XORed together to create the new digest: $H_i = E_{H_{i-1}}(M_i) \oplus M_i \oplus H_{i-1}$. It is Matyas–Meyer–Oseas with the chaining value also fed forward. Whirlpool (below) is built on this scheme.

35.8 SHA-512: overview and parameters ▸

▸ **Likely this year:** "Explain SHA-512 with a block diagram: padding, word expansion, round function" (4–5 marks).

SHA-512 is the member of the SHA-2 family with the longest digest. It follows the Merkle–Damgård scheme exactly: the message is padded and given a length field, split into 1024-bit blocks, and each block is fed with the previous 512-bit digest into a compression function; the output of the last block is the 512-bit digest.



SHA-512 is **word oriented**: a word is 64 bits. Each 1024-bit message block is 16 words, and the 512-bit digest is 8 words, named A, B, C, D, E, F, G, H. The parameters are: maximum message length $2^{128} - 1$ bits, block 1024 bits, digest 512 bits, 80 rounds per block, 64-bit words, 80 round constants $K_0, \ldots, K_{79}$, and 80 expanded words $W_0, \ldots, W_{79}$ per block.

35.9 SHA-512: padding and length field ▸

▸ **Likely this year:** "Compute the number of padding bits for a message of length L; is padding needed if the length is already a multiple of 1024?" (2–3 marks).

Before processing, Merkle–Damgård requires the message to be a whole number of blocks, with its length recorded at the end. SHA-512 therefore appends two things to the original message M: **padding**, which is a single 1 bit followed by as many 0 bits as needed, and a **128-bit length field** holding $|M|$ as an unsigned integer (this is why the maximum message length is $2^{128} - 1$). The total $|M| + |P| + 128$ must be a multiple of 1024, so

$$|M| + |P| + 128 \equiv 0 \pmod{1024} \quad \Longrightarrow \quad \boxed{|P| = (-|M| - 128) \bmod 1024}, \qquad 0 \le |P| \le 1023.$$

Since the mod gives a value from 0 to 1023, the padding is always less than one block.

WORKED EXAMPLE : padding (Forouzan Example 12.1)

Q. What is the number of padding bits if the length of the original message is 2590 bits?
A. $|P| = (-2590 - 128) \bmod 1024 = (-2718) \bmod 1024$. Since $-2718 + 3 \times 1024 = 354$, $|P|$ = **354** bits: a 1 followed by 353 zeros.

Check: 2590 + 354 + 128 = 3072 = 3 × 1024, so the augmented message is exactly 3 blocks.

WORKED EXAMPLE : padding, new example

Q. A message of 5000 bits is hashed with SHA-512. Find the number of padding bits, the number of blocks, and the content of the length field.
A. $|P| = (-5000 - 128) \bmod 1024 = (-5128) \bmod 1024 = -5128 + 6 \times 1024 =$ **1016** bits (a 1 followed by 1015 zeros). Total = 5000 + 1016 + 128 = 6144 = **6 blocks**. The 128-bit length field holds 5000 = 1388 in hex, i.e. 115 zero bits followed by `1001110001000`.
Check: 6144/1024 = 6 exactly.

Edge cases (Forouzan Examples 12.2–12.3). A message whose length is *already* a multiple of 1024 still needs padding, because the length field must be added: for $|M| = 1024$, $|P| = (-1152) \bmod 1024 = 896$ and the result is 2 blocks. The formula gives the **minimum** padding $|P| = 0$ when $|M| \equiv 896 \pmod{1024}$ (the message plus the length field already fill the block) and the **maximum** $|P| = 1023$ when $|M| \equiv 897 \pmod{1024}$. In the actual standard (FIPS 180) the single 1 bit is always appended, so the padding is at least 1 bit; for $|M| \equiv 896$ this forces a full extra block of 1024 padding bits. A short check of the real rule: the 24-bit message "abc" gets $(-24 - 128) \bmod 1024 = 872$ padding bits and fits in one block.

Trap: padding arithmetic

Take the mod of a negative number correctly: add multiples of 1024 until the value lies in 0–1023. Do not forget the 128-bit length field (SHA-1 and SHA-256 use a 64-bit field and 512-bit blocks, giving $|P| = (-|M| - 64) \bmod 512$).

35.10 SHA-512: word expansion and the compression function ▸

Word expansion. Each round uses one 64-bit word, but a block has only 16 words, so the block is expanded to 80 words $W_0, \ldots, W_{79}$. The first 16 are the block itself; each later word mixes four earlier ones:

$$W_i = W_{i-16} \oplus \text{RotShift}_{1\text{-}8\text{-}7}(W_{i-15}) \oplus W_{i-7} \oplus \text{RotShift}_{19\text{-}61\text{-}6}(W_{i-2}), \qquad 16 \le i \le 79,$$

$$\text{RotShift}_{l\text{-}m\text{-}n}(x) = \text{RotR}_l(x) \oplus \text{RotR}_m(x) \oplus \text{ShL}_n(x),$$

where $\text{RotR}_i(x)$ rotates x right by i bits (bits leaving on the right re-enter on the left) and $\text{ShL}_i(x)$ shifts x left by i bits, filling with 0s (bits leaving are lost). This is the slides' (Forouzan's) form; see the correction box for the standard's exact operations. Since every later word depends on several earlier ones, a change in one message bit spreads into many of the 80 words.

Initial values and constants. The eight words of H_0 are the first 64 bits of the fractional parts of the square roots of the first eight primes (2 to 19); for example A_0 = `6A09E667F3BCC908` from $\sqrt{2}$. The 80 round constants $K_0, \ldots, K_{79}$ are the first 64 bits of the fractional parts of the cube roots of the first 80 primes (2 to 409); for example K_0 = `428A2F98D728AE22` from $\sqrt[3]{2}$. Taking them from irrational numbers shows that the designers hid no trapdoor in the constants ("nothing up my sleeve" numbers).

Compression function. Processing one block takes 80 rounds. At the start, the eight digest words from the previous block (or the IV) are saved in eight temporary variables. Round i mixes the current $A, \ldots, H$ with the word W_i and the constant K_i and produces a new set of eight words. After round 79, the saved values are added to the outputs word by word modulo 2^{64}; this **final adding** produces the digest that is passed to the next block (or is the final digest). The final adding is a feed-forward, exactly like Davies–Meyer, and it makes the compression function impossible to run backwards.

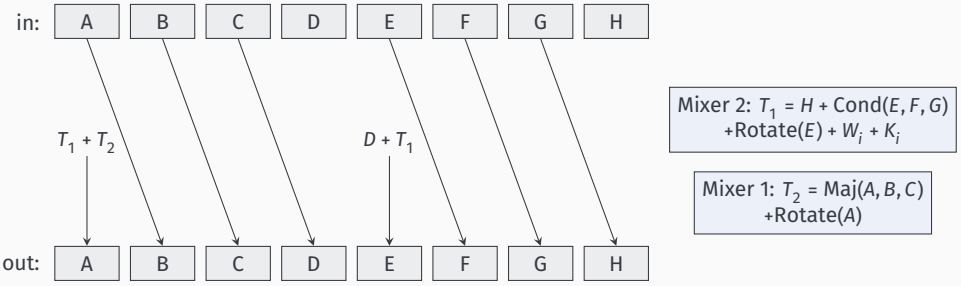
Structure of each round. Six of the eight words are simply moved one place right, and only two new words are computed, using two mixers (every + here is addition modulo 2^{64}):

$$T_1 = H + \text{Conditional}(E, F, G) + \text{Rotate}(E) + W_i + K_i, \qquad T_2 = \text{Majority}(A, B, C) + \text{Rotate}(A),$$

$$H \leftarrow G,\ G \leftarrow F,\ F \leftarrow E,\ E \leftarrow D + T_1,\ D \leftarrow C,\ C \leftarrow B,\ B \leftarrow A,\ A \leftarrow T_1 + T_2.$$

The three functions are bitwise (except Rotate, which moves bits within the word):

- $\text{Majority}(x, y, z) = (x \wedge y) \oplus (y \wedge z) \oplus (z \wedge x)$: each output bit is the majority value of the three input bits.
- $\text{Conditional}(x, y, z) = (x \wedge y) \oplus (\overline{x} \wedge z)$: each bit of x acts as a selector, taking the bit of y where $x = 1$ and the bit of z where $x = 0$ ("if x then y else z").
- $\text{Rotate}(x) = \text{RotR}_{28}(x) \oplus \text{RotR}_{34}(x) \oplus \text{RotR}_{39}(x)$.



In Forouzan's notation the new A is called X and the new E is Y. Because A and E change every round and the other six words are shifted, every input word affects all eight words within a few rounds.

★ MUST KNOW : SHA-512 in numbers

Block 1024 bits = 16 words of 64 bits; digest 512 bits = 8 words A–H; 80 rounds; 80 words W_0–W_{79}; 80 constants K_i; length field 128 bits; $|P| = (-|M| - 128) \bmod 1024$; max message $2^{128} - 1$ bits; final adding modulo 2^{64}.

35.11 Exercises on the SHA-512 operations ▸

▸ **Likely this year:** One of the slide exercises (RotR, ShL, Conditional, Majority, Rotate) on given hex words.

The slide exercises use a **128-bit** x (real SHA-512 words are 64 bits) and work in hex: a rotation or shift by 12 bits moves the string by 3 hex digits. All results below were recomputed.

$$x = 1234\ \ 5678\ \ ABCD\ \ 2345\ \ 3456\ \ 5678\ \ ABCD\ \ 2468$$

WORKED EXAMPLE : RotR, ShL, Conditional, Majority, Rotate (slides 52–56)

$\text{RotR}_{12}(x)$. Rotate right by 12 bits = 3 hex digits: the last three digits 468 move to the front.
4681 2345 678A BCD2 3453 4565 678A BCD2
$\text{ShL}_{12}(x)$. Shift left by 3 hex digits: the first three digits 123 are lost and three 0 digits enter on the right.
4567 8ABC D234 5345 6567 8ABC D246 8000
Conditional(x, y, z) with y = 2234 5678 ..., z = 3234 5678 ... (the three words differ only in the first digit). For the first digit, x = 0001 selects y where x = 1 and z where x = 0: $(0001 \wedge 0010) \oplus (1110 \wedge 0011) = 0000 \oplus 0010 = 0010 = 2$. For every other digit the three inputs are equal, and Conditional of three equal digits is that digit.
Result: 2234 5678 ABCD 2345 3456 5678 ABCD 2468
Majority(x, y, z), same x, y, z. First digit: $(0001 \wedge 0010) \oplus (0010 \wedge 0011) \oplus (0011 \wedge 0001) = 0000 \oplus 0010 \oplus 0001 = 0011 = 3$. The other digits are unchanged.
Result: 3234 5678 ABCD 2345 3456 5678 ABCD 2468
Rotate$(x) = \text{RotR}_{28}(x) \oplus \text{RotR}_{34}(x) \oplus \text{RotR}_{39}(x)$. First the three rotations, then the XOR:

$\text{RotR}_{28}(x)$	BCD2	4681	2345	678A	BCD2	3453	4565	678A
$\text{RotR}_{34}(x)$	2AF3	491A	048D	159E	2AF3	48D1	4D15	959E
$\text{RotR}_{39}(x)$	F157	9A48	D024	68AC	F157	9A46	8A68	ACAC
Rotate(x)	6776	95D3	F7EC	1AB8	6776	E6C4	8218	5EB8

RotR_{28} is a rotation by 7 hex digits, so it can be done digit-wise; RotR_{34} and RotR_{39} are not multiples of 4 and must be done in binary.

WORKED EXAMPLE : new exercise on 16-bit and 64-bit words

Q. (a) For the 16-bit words x = F0F0, y = AAAA, z = 5555, find Conditional(x, y, z) and Majority(x, y, z). (b) For the 64-bit word w = 0123 4567 89AB CDEF, find $\text{RotR}_8(w)$ and $\text{ShL}_8(w)$.
A. (a) Conditional takes y where x is 1 and z where x is 0. $x \wedge y$ = A0A0, $\overline{x} \wedge z$ = 0F0F $\wedge$ 5555 = 0505; Conditional = A0A0 $\oplus$ 0505 = **A5A5**.
Majority: $x \wedge y$ = A0A0, $y \wedge z$ = 0000 (since $z = \overline{y}$), $z \wedge x$ = 5050; XOR = **F0F0**. This equals x, as it must: y and z disagree in every bit, so x always casts the deciding vote.
(b) 8 bits = 2 hex digits. $\text{RotR}_8(w)$ = EF01 2345 6789 ABCD (EF wraps to the front); $\text{ShL}_8(w)$ = 2345 6789 ABCD EF00 (01 is lost, 00 enters).

35.12 Whirlpool

Whirlpool, designed by Rijmen and Barreto and selected by the NESSIE project (New European Schemes for Signatures, Integrity, and Encryption), is an iterated hash function based on the **Miyaguchi–Preneel** scheme, in which the compression function is a block cipher: $H_i = E_{H_{i-1}}(M_i) \oplus M_i \oplus H_{i-1}$. The cipher, called W, is a **modified AES** tailored for this purpose.

Parameters. Block size 512 bits, digest 512 bits, cipher key 512 bits (the previous digest), initial value H_0 all 0s, and maximum message length $2^{256} - 1$ bits. The message is padded (a 1 followed by 0s) to an *odd* multiple of 256 bits and then a **256-bit length field** is appended; an odd multiple of 256 plus 256 is an even multiple, i.e. a multiple of 512. For example, $|M| = 1000$ needs $|P| = (-1000 - 256) \bmod 512 = 280$ padding bits, since $1000 + 280 = 1280 = 5 \times 256$ (odd) and $1280 + 256 = 1536 = 3 \times 512$.

How the Whirlpool cipher differs from AES. It uses the same four kinds of transformation, but on a larger state and with all rounds alike:

- The block and key are 512 bits, held as an 8×8 matrix of bytes (AES: 128 bits as 4×4).
- There are 10 rounds, each with SubBytes, **ShiftColumns** (column j is rotated down by j positions, instead of AES's ShiftRows), **MixRows** (each row is multiplied by a fixed 8×8 circulant matrix, instead of AES's MixColumns) and AddRoundKey.
- Arithmetic is in GF(2^8) with modulus $x^8 + x^4 + x^3 + x^2 + 1$ (AES uses $x^8 + x^4 + x^3 + x + 1$), and the S-box is built from smaller 4-bit boxes.
- The round keys are generated by running the same round function on the key, with round constants taken from the S-box in place of a key, instead of AES's separate key-expansion routine.

Corrections to the notes / slides

- Slide 27: "0.69×2^{34} seconds or more than 500 years". In fact 0.69×2^{34} s $\approx$ 376 years (2^{34} s alone is $\approx$ 544 years). The conclusion (secure against preimage attacks) stands.
- Slides 54–55 swap the binary for the first digit: Conditional$(0001, 0010, 0011) = 0010 = 2$ and Majority$(0001, 0010, 0011) = 0011 = 3$; the slides print 0011 for Conditional and 0010 for Majority, though the hex results 2 and 3 are right. Both slides also label the third word x instead of z.
- The exercise x (slides 52–56) is 128 bits, though real SHA-512 words are 64 bits; the rotations on the slides are done on 128 bits, and the results above follow the slides.
- Slide 11 prints Problem 4 as $P \approx 1 - e^{-k^2/2N}$; that formula would give $k = 1.18\sqrt{N}$. The correct one is $P \approx 1 - e^{-k^2/N}$ (as on slide 35), which gives $k = 0.83\sqrt{N} = 16$.
- Slide 40 lists SHA under the "block cipher" approach. Forouzan classifies MD and SHA as compression functions *made from scratch* and Rabin, Davies–Meyer, Matyas–Meyer–Oseas and Miyaguchi–Preneel as block-cipher based; write it that way in the exam.
- Slides 48 and 51 (from Forouzan) simplify the standard: in FIPS 180 the word expansion uses addition modulo 2^{64} and a *right* shift (SHR), not XOR and ShL, and Rotate(E) uses $\text{RotR}_{14} \oplus \text{RotR}_{18} \oplus \text{RotR}_{41}$, not 28-34-39. The slide's "ShL...padding the left by 0s" is self-contradictory (a left shift fills on the right). For the exam, use the slide formulas.

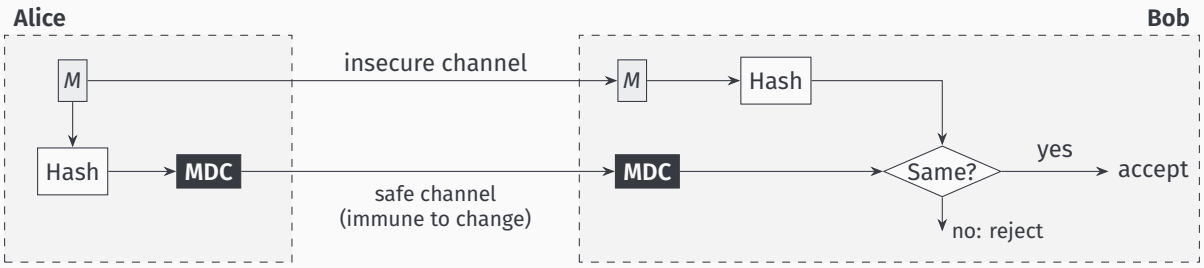
36 Message Authentication: MDC, MAC, HMAC and CMAC

36.1 Why a digest does not authenticate: MDC versus MAC ▸

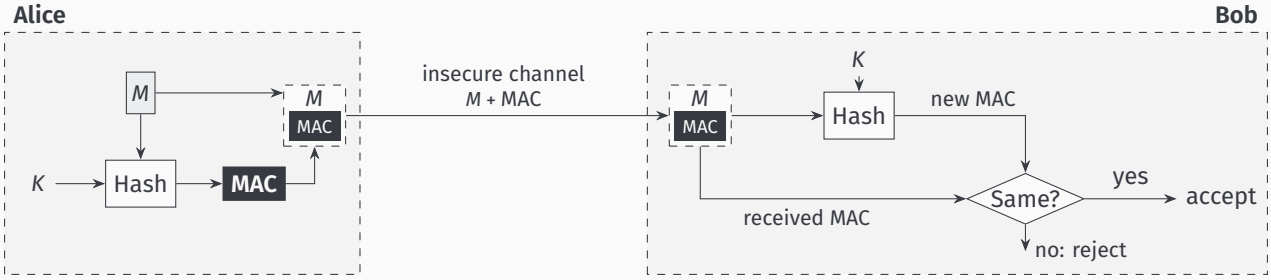
▸ **Likely this year:** "Why can a message digest not provide message authentication? Differentiate MDC and MAC with diagrams" (3–4 marks).

A message digest guarantees **integrity**: if Alice sends M and Bob recomputes $h(M)$, any change to M produces a different digest (§35). It does *not* tell Bob who created the message. The hash function is public and needs no secret, so Eve can write her own message M', compute $h(M')$ and send the pair $(M', h(M'))$; Bob's comparison succeeds, because the pair is internally consistent, and he has no way of telling it from a pair made by Alice. To prove that the message comes from Alice and not an impostor (**message authentication**, also called **data-origin authentication**), the check value must depend on something that only Alice and Bob know.

Modification detection code (MDC). A digest made by a keyless cryptographic hash function is called an MDC, because it can only detect modification. The MDC itself must therefore travel over a *safe* channel (a channel immune to change), while the message may travel over an insecure one. Bob hashes the received message and compares the result with the MDC from the safe channel; if they are equal the message is accepted, otherwise rejected.



Message authentication code (MAC). A MAC removes the need for a safe channel by mixing a **shared secret key** K into the digest. Alice computes MAC = $h(K \mid M)$, where | denotes concatenation, and sends M and the MAC together over the insecure channel. Bob separates the two, computes a new MAC from the received message and his copy of K, and compares it with the received MAC. If the two MACs match, the message is authentic (it was made by a holder of K) and has not been modified. Eve can still change M, but without K she cannot compute the MAC that goes with the new message, so the change is detected.



	MDC (modification detection code)	MAC (message authentication code)
Computed as	$h(M)$, keyless hash	$h(K \mid M)$ (or HMAC, CMAC) with a shared secret key K
Service	integrity only	integrity *and* data-origin authentication
Channel for the code	must be safe (immune to change)	the MAC travels with M over the insecure channel
Who can create a valid code	anyone	only holders of K (Alice and Bob)
Nonrepudiation	no	no: Bob holds K too, so he could have made the MAC himself

The last row is the reason digital signatures (§37) are needed: a MAC convinces Bob, but it cannot convince a judge that Alice, rather than Bob, produced it.

36.2 Prefix, postfix and envelope MACs from a hash ▸

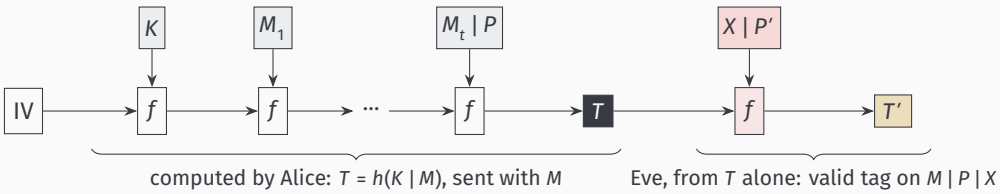
▸ **Likely this year:** "What is a prefix MAC? Why is $h(K \mid M)$ insecure with an iterated hash?" (2–3 marks).

The slide's MAC $h(K \mid M)$ is a **prefix MAC**: the secret key is placed at the beginning of the message before hashing. In a **postfix MAC** the key is appended at the end, $h(M \mid K)$, and the two can be combined into an **envelope** $h(K \mid M \mid K)$, with the same key at both ends or two different keys $h(K_1 \mid M \mid K_2)$. All three look reasonable, but each inherits a weakness from the iterated (Merkle–Damgård) structure of real hash functions.

Length extension breaks the prefix MAC. In an iterated hash the digest is simply the last chaining value H_t, and the computation of h on a longer message just continues from it: $H_{t+1} = f(H_t, X)$. Suppose Eve sees M and its tag $T = h(K \mid M)$. She knows the length of $K \mid M$ (key lengths are public), so she knows the padding block P that the hash appended. She can therefore take T as the chaining value, run the compression function on an extra block X of her choice (followed by the correct new padding), and obtain

$$T' = h(K \mid M \mid P \mid X)$$

without ever knowing K. The pair $(M \mid P \mid X,\ T')$ is a valid message–MAC pair, so Eve has forged a MAC on a longer message that begins with Alice's message (we checked this on SHA-256 by script: the forged tag equals the true $h(K \mid M \mid P \mid X)$).



Postfix MAC. Here the key comes last, so length extension does not work (Eve cannot hash past the unknown key). But if Eve finds an ordinary collision of the keyless hash, $h(M) = h(M')$ with M and M' of the same block length, the chaining values before the key are equal, so $h(M \mid K) = h(M' \mid K)$ for *every* key. The collision is found offline with about $2^{n/2}$ work (birthday attack), after which a MAC on M is also a MAC on M'. The envelope with two keys blocks both tricks, and the cleanest standard solution is the nested MAC (HMAC) below.

36.3 Security of a MAC ▸

▸ **Likely this year:** "How can Eve forge a MAC? On what does its security depend?" (2–3 marks).

Suppose Eve has intercepted M and $h(K \mid M)$ and wants to forge a message without knowing K. The slide lists three possible cases.

1. **Exhaustive key search.** If the key is short, Eve prepends every possible key K' to M and hashes $K' \mid M$ until the result equals the intercepted MAC. She then knows the key and can replace the message with any forged message and a correct MAC. The cost is about 2^k hash computations for a k-bit key, so the key must be long (128 bits or more).
2. **Preimage attack.** Since the key is normally too long for exhaustive search, Eve can instead attack the hash: she runs a preimage attack until she finds an X with $h(X)$ equal to the intercepted MAC. Because $X = K \mid M$ and she knows M, the front of X is the key, which she can then use to forge. This costs about 2^n for an n-bit digest (§35), so the MAC is only as strong as the preimage resistance of h.
3. **Forging from known pairs.** Given some pairs of messages and their MACs, Eve may be able to manipulate them into a new message with a valid MAC without finding the key at all. Length extension of the prefix MAC is exactly such an attack, and the CBC-MAC splice below is another.

Conclusion (slide 7). The security of a MAC depends on the security of the underlying hash algorithm (or block cipher), and on its construction. A brute-force forger needs the smaller of 2^k (key search) and 2^n (guessing a valid tag), which is why both the key and the tag must be long; the construction must in addition leave no algebraic shortcut such as length extension.

36.4 Nested MAC and HMAC ★▸

▸ **Likely this year:** "Explain HMAC with a block diagram; what are ipad and opad; why are two hashes used?" (5 marks). The most likely MAC question.

The weakness of $h(K \mid M)$ is that its output is the internal state of the hash, which anyone can continue. A **nested MAC** hides that state by hashing a second time with the key: first $h(K \mid M)$ is computed, and then the result is prefixed with the key again and hashed, giving $h\big(K \mid h(K \mid M)\big)$ (slide 8). Eve can still extend the inner hash in principle, but she never sees it; she sees only the outer digest, and extending *that* produces a hash of $K \mid \text{inner} \mid P \mid X$, which is not the nested MAC of any message. **HMAC** (hashed MAC, RFC 2104, standardized by NIST as FIPS 198) is the standardized nested MAC; it derives two different keys from K for the two passes.

Design objectives (Stallings). HMAC was designed to use an existing hash function (MD5, SHA-1, SHA-2) without modification, so that available fast code can be used; to allow the hash to be replaced by a newer one when needed; to keep the speed of the hash; to handle keys in a simple way; and to have a security proof relating its strength to that of the hash.

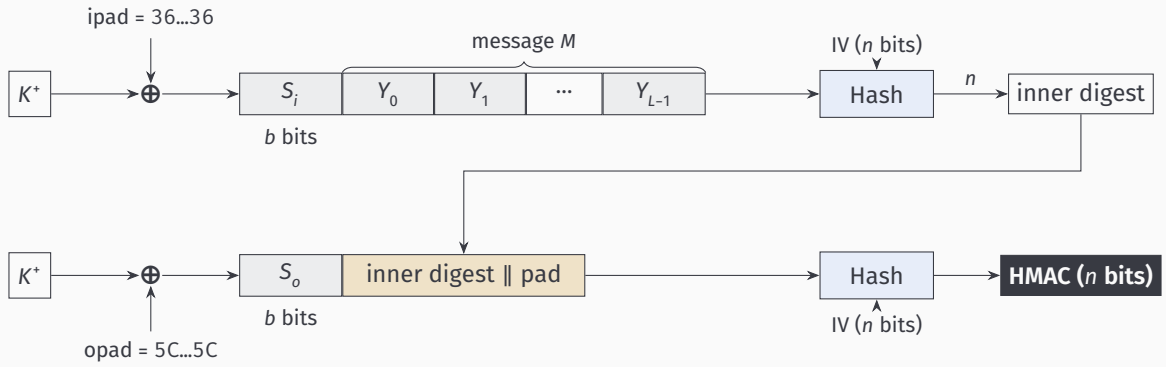
Notation. H is the hash function, with block size b bits (512 for MD5, SHA-1 and SHA-256; 1024 for SHA-512) and digest size n bits. M is the message, split by H into blocks $Y_0, \ldots, Y_{L-1}$. K is the secret key; if $|K| > b$ it is first replaced by $H(K)$. K^+ is K padded with zeros to exactly b bits (RFC 2104 and FIPS 198 append the zeros at the end; Stallings' figure pads on the left; either way the convention is fixed). The two constant pads are

$$\text{ipad} = \texttt{00110110} = \texttt{0x36} \text{ repeated } b/8 \text{ times}, \qquad \text{opad} = \texttt{01011100} = \texttt{0x5C} \text{ repeated } b/8 \text{ times}.$$

★ MUST KNOW : HMAC (FIPS 198, RFC 2104)

$$\text{HMAC}(K, M) = H\Big[\,(K^+ \oplus \text{opad}) \,\|\, H\big[\,(K^+ \oplus \text{ipad}) \,\|\, M\,\big]\Big]$$

Steps: (1) pad K with zeros to b bits, giving K^+; (2) $S_i = K^+ \oplus \text{ipad}$; (3) append M to S_i; (4) hash $S_i \| M$ to get the n-bit inner digest; (5) $S_o = K^+ \oplus \text{opad}$; (6) append the inner digest to S_o (the hash pads it to a full block); (7) hash $S_o \|$ inner digest; the n-bit result is the HMAC.



Why two different pads. Both pads have exactly four 1-bits per byte (`0x36` = `00110110`, `0x5C` = `01011100`), so each XOR flips half of the bits of K^+, and the two pads flip different halves (`0x36` $\oplus$ `0x5C` = `0x6A` = `01101010`). After the first compression step, S_i and S_o therefore act as two pseudo-randomly derived, unrelated keys, one for the inner and one for the outer hash, although only one key K is stored.

Why two nested hashes. The inner hash $H[S_i \| M]$ compresses the long message under the secret S_i; on its own it would be a prefix MAC and open to length extension. The outer hash takes the inner digest as its *message* and prefixes the second secret S_o. Its output reveals nothing usable about the inner chaining state, and extending it only yields a hash of $S_o \| \text{inner} \| \text{pad} \| X$, which is not the HMAC of any message. The outer pass is short (one block), so the protection costs almost nothing.

Efficiency. The blocks S_i and S_o depend only on the key. An implementation computes $f(\text{IV}, S_i)$ and $f(\text{IV}, S_o)$ once per key and stores them as the starting chaining values of the inner and outer hash. After that, each message costs the plain hash of M plus **one extra compression-function call** (the outer block), so for long messages HMAC runs at practically the speed of the hash itself.

Security. Bellare, Canetti and Krawczyk proved that forging HMAC is as hard as one of two things: computing outputs of the compression function when its IV is secret and random, or finding collisions in the hash when its IV is secret and random. In the second case the birthday attack cannot be run offline, because Eve cannot evaluate the keyed function herself; she must observe about $2^{n/2}$ MACs produced by Alice under one key. For MD5 ($n = 128$) that is 2^{64} MACs, more than 2^{72} bits of traffic, or about 150 000 years on a 1 Gbit/s link. Hence the security of HMAC depends on the cryptographic strength of the underlying hash function, but HMAC needs much less from the hash than plain hashing does: HMAC-MD5 is still unbroken although MD5 collisions are easy.

WORKED EXAMPLE : one HMAC-SHA-256 computation, block by block

Q. Describe the computation of HMAC-SHA-256 with key K = "key" (3 bytes, `6B 65 79`) on the 43-byte message M = "The quick brown fox jumps over the lazy dog". Give K^+, the first bytes of S_i and S_o, the number of compression-function calls, and the result.
A. SHA-256 has $b = 512$ bits = 64 bytes and $n = 256$ bits.
(1) K^+ = `6B 65 79` followed by 61 zero bytes (64 bytes).
(2) $S_i = K^+ \oplus$ `36 36` ...: `6B` $\oplus$ `36` = `5D`, `65` $\oplus$ `36` = `53`, `79` $\oplus$ `36` = `4F`, `00` $\oplus$ `36` = `36`, so S_i = `5D 53 4F 36 36` ...`36`.
(3) $S_o = K^+ \oplus$ `5C 5C` ... = `37 39 25 5C 5C` ...`5C`.
(4) Inner hash of $S_i \| M$: 64 + 43 = 107 bytes; with the `80` byte and the 8-byte length field it needs 107 + 1 + 8 = 116 ≤ 128 bytes, i.e. **2 blocks**.
(5) Outer hash of $S_o \|$ inner: 64 + 32 = 96 bytes, 96 + 9 = 105 ≤ 128, again **2 blocks**.
Total 4 compression calls; with $f(\text{IV}, S_i)$ and $f(\text{IV}, S_o)$ cached for this key, only 2 per message.
Result (256 bits):
HMAC = `f7bc83f4 30538424 b13298e6 aa6fb143 ef4d59a1 49461759 97479dbc 2d1a3cd8`
Check: the script's two-pass computation $H[S_o \| H[S_i \| M]]$ equals the library HMAC value.

36.5 CBC-MAC (Data Authentication Algorithm) and CMAC ▸

▸ **Likely this year:** "Explain CMAC with a diagram; how are the subkeys k_1, k_2 generated? Why is plain CBC-MAC insecure for variable-length messages?" (4–5 marks).

A MAC can also be built from a block cipher instead of a hash. The idea comes from **cipher block chaining** (CBC) mode, in which each plaintext block is XORed with the previous ciphertext block before encryption, $C_i = E_K(M_i \oplus C_{i-1})$. Because of the chaining, the last ciphertext block depends on every bit of every message block and on the key, so it can serve as a MAC: only a holder of K can compute it, and any change to any block changes it unpredictably.

CBC-MAC / Data Authentication Algorithm (DAA). The message is split into N blocks of n bits each ($n = 64$ for DES, 128 for AES), the last one padded with zeros. With an all-zero IV, $C_0 = 0^n$,

$$C_1 = E_K(M_1), \qquad C_i = E_K(M_i \oplus C_{i-1}) \ \ (i = 2, \ldots, N), \qquad \text{MAC} = \text{leftmost } t \text{ bits of } C_N.$$

Unlike CBC encryption, only the last block is output; the intermediate C_i are kept secret. The DAA of FIPS PUB 113 is exactly this with DES and a tag of $16 \le t \le 64$ bits.

Why plain CBC-MAC fails for variable-length messages. CBC-MAC is secure when all messages have one fixed length, but not otherwise. Suppose Eve obtains the tag $T = E_K(X)$ of a one-block message X. She forms the two-block message $X \,\|\, (X \oplus T)$ and computes its CBC-MAC symbolically:

$$C_1 = E_K(X) = T, \qquad C_2 = E_K\big((X \oplus T) \oplus C_1\big) = E_K(X \oplus T \oplus T) = E_K(X) = T.$$

The new message has the *same* tag T, so Eve has a valid forgery without knowing K (checked by script with AES). More generally, given (M, T) and (M', T'), the splice $M \,\|\, (M'_1 \oplus T) \,\|\, M'_2 \cdots$ has tag T': XORing T into the first block of

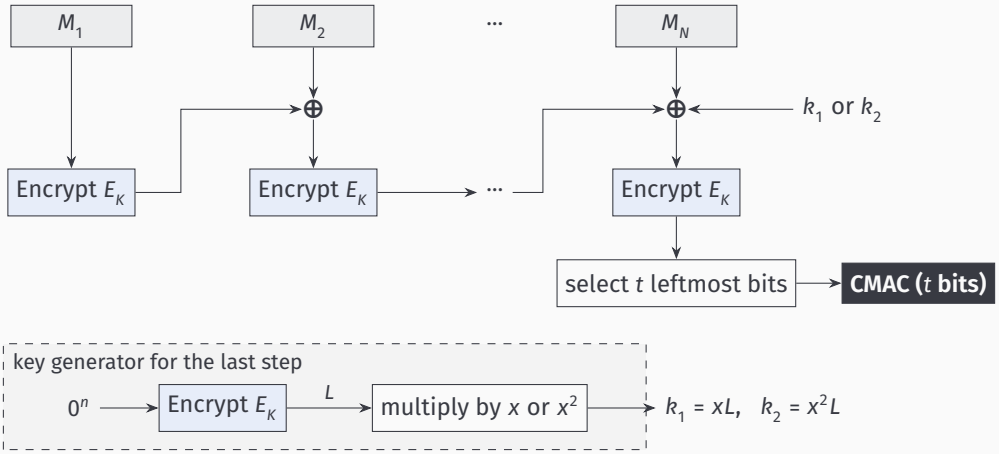
M' cancels the chaining value that M leaves behind. The root cause is that the last step is an ordinary step, so a tag is also a usable intermediate chaining value. The fix is to make the last step different from the others.

CMAC (cipher-based MAC, NIST SP 800-38B). CMAC (also called OMAC1; Forouzan and the slide call the family "CMAC or CBC-MAC") keeps the CBC chain but XORs a secret **subkey** into the last block, so the input to the final encryption is never a value Eve can predict from earlier tags. One cipher key K is used; the subkeys are derived from it.

1. **Subkey generation.** Encrypt the all-zero block: $L = E_K(0^n)$. Then $k_1 = x \cdot L$ and $k_2 = x \cdot k_1 = x^2 \cdot L$, where multiplication by x is done in GF(2^n) (§28). Multiplying by x is a left shift by one bit; if the bit shifted out (the MSB) was 1, the result is reduced by XORing the constant R_n:

$$x \cdot L = \begin{cases} L \ll 1 & \text{if MSB}(L) = 0, \\ (L \ll 1) \oplus R_n & \text{if MSB}(L) = 1, \end{cases} \qquad R_{128} = 0^{120}10000111 = \texttt{0x87}, \quad R_{64} = 0^{59}11011 = \texttt{0x1B}.$$

   The constants are the low terms of the field moduli $x^{128} + x^7 + x^2 + x + 1$ and $x^{64} + x^4 + x^3 + x + 1$: the shifted-out x^n is replaced by its residue.
2. **Chaining.** Split M into blocks $M_1, \ldots, M_N$ and run CBC with IV = 0^n on $M_1, \ldots, M_{N-1}$.
3. **Last block.** If M_N is a *complete* block, XOR it with k_1. If it is incomplete, pad it with a single 1 bit followed by 0s (10…0) to n bits and XOR it with k_2. Then $C_N = E_K(M'_N \oplus C_{N-1})$.
4. **Tag.** The CMAC is the leftmost t bits of C_N (Forouzan's figure calls the block size m and the tag size n).



Why the subkeys fix both problems. First, the tag is $T = E_K(M_N \oplus C_{N-1} \oplus k)$, with a secret k inside. If Eve appends blocks, M_N becomes a *middle* block, processed without k, so the chaining value after it is $E_K(M_N \oplus C_{N-1})$, a value Eve has never seen; XORing T into the next block therefore no longer cancels anything, and the splice fails. Second, the two different subkeys separate padded from unpadded messages: a message whose last block is incomplete and is padded with 10…0 and a message whose last block happens to *be* that padded block would otherwise get the same tag; with k_2 for the first and k_1 for the second they differ. Multiplication by x is used because it costs only a shift and a conditional XOR, yet makes L, k_1, k_2 distinct secret values.

WORKED EXAMPLE : CMAC subkeys with AES-128 (the NIST/RFC 4493 test key)

Q. With AES-128 and K = `2b7e1516 28aed2a6 abf71588 09cf4f3c`, $L = E_K(0^{128})$ = `7df76b0c 1ab899b3 3e42f047 b91b546f`. Find k_1 and k_2. What is the CMAC of the one-block message M = `6bc1bee2 2e409f96 e93d7e11 7393172a`?

A. MSB(L): the first hex digit 7 is 0111, so the MSB is 0 and $k_1 = L \ll 1$ with no reduction:
k_1 = `fbeed618 35713366 7c85e08f 7236a8de`.
MSB(k_1): f = 1111, so the MSB is 1: $k_2 = (k_1 \ll 1) \oplus$ 0x87. The shift gives `f7ddac30 6ae266cc f90bc11e e46d51bc`; the last byte becomes bc ⊕ 87 = 10111100 ⊕ 10000111 = 00111011 = 3b:
k_2 = `f7ddac30 6ae266cc f90bc11e e46d513b`.
M is one complete block, so $N = 1$, $C_0 = 0$ and the tag is $E_K(M \oplus k_1)$ = `070a16b4 6b4d4144 f79bdd9d d04a287c`.
Check: all three values agree with the script (AES via a library) and with the RFC 4493 test vectors.

	HMAC	CMAC
Built from	a hash function (SHA-1, SHA-2, MD5)	a block cipher (AES, 3DES)
Structure	two nested hashes, keys $K^+ \oplus$ ipad and $K^+ \oplus$ opad	CBC chain, IV = 0, subkey k_1 / k_2 XORed into the last block
Tag length	n (digest size), may be truncated	≤ block size, leftmost t bits
Stops length extension / splicing by	the outer hash	the secret subkey in the last step
Standard	RFC 2104, FIPS 198	NIST SP 800-38B

Corrections to the notes / slides

- Slide 8 titles the figure "Nested MAC (OR) HMAC". The figure is the generic nested MAC $h(K \mid h(K \mid M))$ with the same key twice; HMAC is the standardized nested MAC that pads the key to a block and uses $K^+ \oplus$ ipad inside and $K^+ \oplus$ opad outside.
- Slide 9 says "CMAC, or CBCMAC is also called Data Authentication Algorithm". Strictly, the DAA (FIPS 113) is plain CBC-MAC with DES; CMAC (SP 800-38B) is the improved version with the subkeys k_1, k_2 shown in the slide's figure. Plain CBC-MAC is insecure for variable-length messages.
- Slide 5: in a prefix MAC the key is *prepended* (placed at the beginning), not "appended to the beginning".

37 Digital Signatures

37.1 Conventional versus digital signatures ▸

▸ **Likely this year:** "Differentiate a conventional signature and a digital signature" (inclusion, verification method, relationship, duplicity; 3–4 marks).

A person signs a document to show that it originated from her or was approved by her, and the signature is proof to the recipient that the document comes from the correct entity. When Alice sends a message to Bob over a network, Bob needs the same assurance: that the message comes from Alice and not from Eve. Bob therefore asks Alice to sign the message electronically, and such an electronic signature, which proves the authenticity of Alice as the sender, is called a **digital signature**. It differs from a pen-and-paper signature in four ways.

Inclusion. A conventional signature is included in the document; it is part of it (the signature on a cheque is on the cheque, not on a separate sheet). A digital signature is sent as a *separate* document: the sender sends two documents, the message and the signature. The recipient receives both, verifies that the signature belongs to the supposed sender, and keeps the message if this is proven, otherwise rejects it.

Verification method. For a conventional signature, the recipient compares the signature on the document with a signature on file; if they are the same, the document is authentic, so the recipient needs a copy of the signature for comparison. For a digital signature no copy of the signature is stored anywhere. The recipient applies a *verification technique* (an algorithm using the signer's public key) to the combination of the message and the signature.

Relationship. A person uses the same conventional signature on many documents, a one-to-many relationship. A digital signature is computed *from* the message, so there is a one-to-one relationship between a signature and a message: each message has its own signature, and the signature of one message cannot be used for another. If Bob receives two messages from Alice, he cannot use the signature of the first to verify the second. This is precisely what stops cut-and-paste forgery: a signature lifted from one message fails verification on any other.

Duplicity. A copy of a signed paper document can be distinguished from the original on file. A copy of a digital document is bit-for-bit identical to the original, so there is no such distinction unless a factor of time, such as a **timestamp**, is included in the signed document. For example, if Alice sends a signed instruction asking Bob to pay Eve, Eve can intercept the message and signature and **replay** them later to be paid again; a timestamp or sequence number inside the signed message lets Bob reject the replay.

Aspect	Conventional signature	Digital signature
Inclusion	part of the document	a separate document; the pair (M, S) is sent
Verification	compared with a signature on file	a verifying algorithm with the signer's public key is applied to (M, S); nothing on file
Relationship	one-to-many: one signature, many documents	one-to-one: every message has its own signature
Duplicity	a copy can be told from the original	a copy is identical to the original; needs a timestamp to stop replay

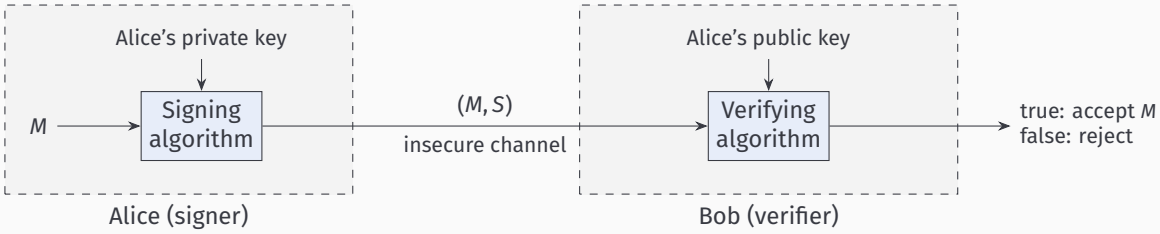
37.2 The signing and verifying process; why keys are needed ★▸

▸ **Likely this year:** "Draw the digital signature process. Why must Alice sign with her private key and not her public key? Which keys does a signature use compared with a cryptosystem?" (3–4 marks).

A digital signature scheme consists of a **signing algorithm** and a **verifying algorithm**. The sender applies the signing algorithm to the message and obtains the signature S; she sends the pair (M, S). The receiver applies the verifying algorithm to the pair; if the result is true the message is accepted, otherwise rejected.

Need for keys. A conventional signature behaves like a private key belonging to the signer: she uses it to sign, and no one else has it. The copy of the signature on file behaves like a public key: anyone can use it to check a document against the original. A digital signature makes this explicit. The signer applies her **private key** to the signing algorithm to sign the document; the verifier applies the **public key of the signer** to the verifying algorithm. Since everyone has access to Alice's public key, anyone, including Bob, can verify a document she signed.

Why Alice must not sign with her public key. A signature is valuable only if nobody else can produce it. The public key is known to everyone, so if Alice signed with it, anyone could run the same computation and forge her signature on any message. The signing step must therefore use something only Alice has (her private key), and the verifying step only what everyone has (her public key). The signer signs with her private key; the verifier verifies with the signer's public key.



Receiver's keys versus sender's keys. A digital signature needs a public-key system. A cryptosystem uses the private and public keys of the receiver; a digital signature uses the private and public keys of the sender. In encryption, anyone must be able to lock a message for Bob, but only Bob may unlock it, so the public operation is the encryption and it uses Bob's key. In signing, only Alice may produce the signature, but anyone must be able to check it, so the private operation is the signing and it uses Alice's key.

	Public-key encryption	Digital signature	MAC
Key that creates	receiver's public key (encrypt)	sender's private key (sign)	shared secret key
Key that checks	receiver's private key (decrypt)	sender's public key (verify)	the same shared key
Who can check	only the receiver	anyone	only key holders
Service	confidentiality	authentication, integrity, nonrepudiation	authentication, integrity

★ MUST KNOW : keys in a digital signature

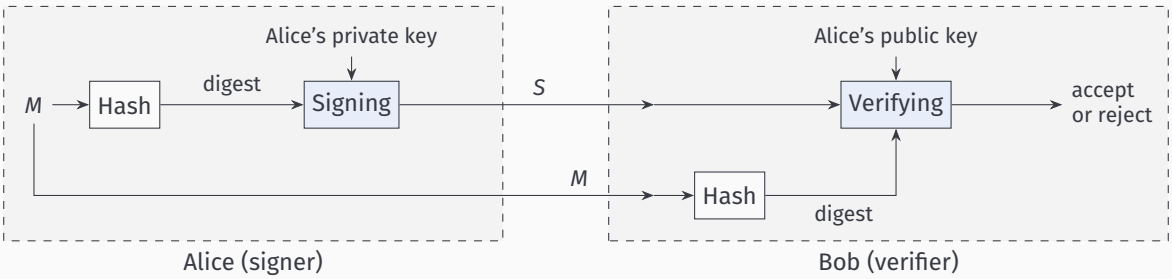
Alice **signs** with **her own private key**; anyone **verifies** with **Alice's public key**. A signature uses the *sender's* key pair; encryption uses the *receiver's* key pair. A signature gives message authentication, integrity and (with a trusted center) nonrepudiation, but **not** confidentiality.

37.3 Signing the digest ▸

▸ **Likely this year:** "Why is the digest signed instead of the message?" (2 marks), often with the diagram.

Public-key operations are slow, and in RSA the signed value must be smaller than n. Signing a long message directly would mean cutting it into blocks and signing every block, which multiplies the cost; it would also allow Eve to delete, repeat or reorder individually signed blocks without detection. In addition, as shown in the attacks below, signing raw messages lets Eve exploit the algebra of the scheme to produce valid signatures on messages Alice never signed.

The solution is to **sign the digest**. Alice computes the digest $h(M)$ (160–512 bits), signs only the digest with her private key, and sends M with the signature. Bob recomputes $h(M)$ from the received message and checks the signature against this digest with Alice's public key. Since the digest is short, one signing operation suffices for any length of message. Integrity is preserved because Eve cannot find a different message with the same digest (second preimage resistance); and because the hash destroys any algebraic relation between messages and signed values, the forgeries of §37.11 stop working. The hash must also be **collision resistant**: otherwise Eve prepares a fair and an unfair contract with the same digest (alternate collision attack, §35), lets Alice sign the fair one, and attaches the signature to the unfair one.



37.4 Services: authentication, integrity, nonrepudiation, but not confidentiality ★▸

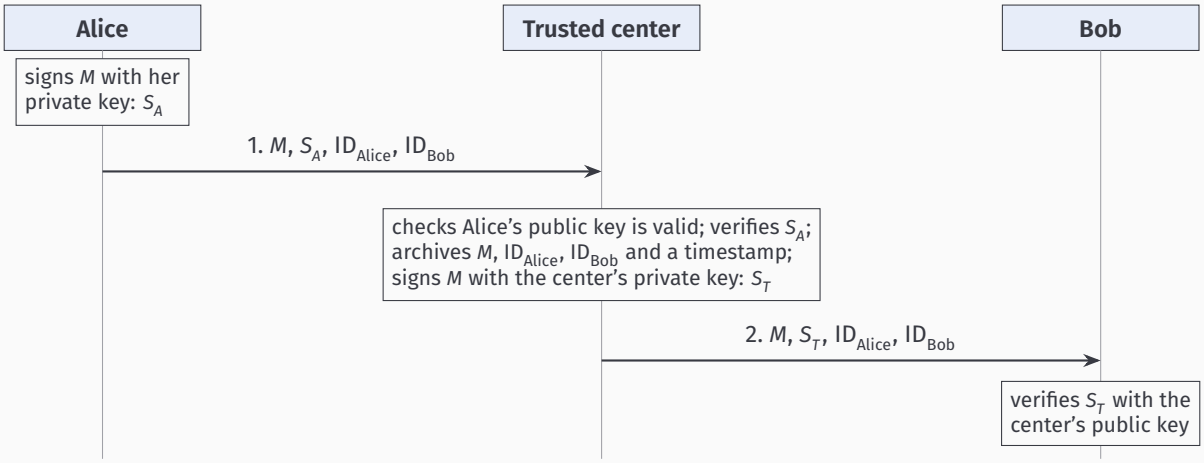
▸ **Likely this year:** "Which security services does a digital signature provide? How is nonrepudiation achieved? How is confidentiality added?" (4–5 marks), with the trusted-center diagram.

Of the four security services (message confidentiality, message authentication, message integrity and nonrepudiation), a digital signature directly provides the last three. For confidentiality, encryption is still needed.

Message authentication. A secure digital signature scheme, like a secure conventional signature (one that cannot easily be copied), provides message authentication, also called data-origin authentication. Bob verifies the message with Alice's public key, and Alice's public key cannot verify a signature made with Eve's private key, so a valid signature proves the message was signed by the holder of Alice's private key.

Message integrity. If the message is changed after signing, the signature no longer verifies, because each message has its own signature; Eve would need Alice's private key to make a new one. Today's schemes hash the message inside the signing and verifying algorithms, so any change to M changes the digest and the check fails.

Nonrepudiation. Suppose Alice signs a message asking her bank (Bob) to transfer \$10 000 to Ted's account, and later denies having sent it. Bob might keep the message and signature on file and later verify it again with Alice's public key, but this is not a reliable proof: Alice may have changed her key pair in the meantime, and she may claim that Bob's file is not authentic. The solution is a **trusted third party**, the *trusted center*, which both sides trust and which archives the evidence.

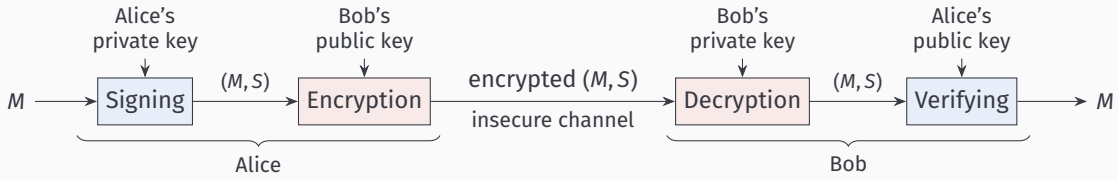


1. Alice creates a signature S_A from her message with her private key and sends the message, her identity, Bob's identity and S_A to the center. The identities bind the message to this particular sender and recipient.
2. The center, after checking that Alice's public key is valid, verifies through it that the message came from Alice. Checking validity at this moment is what defeats a later "I had changed my key" claim.
3. The center saves a copy of the message with the sender identity, the recipient identity and a **timestamp** in its archive. The archive is the independent evidence; the timestamp also distinguishes this message from a later replay of it.
4. The center uses its own private key to create another signature S_T from the message, and sends the message, S_T, Alice's identity and Bob's identity to Bob.
5. Bob verifies the message using the public key of the trusted center.

If Alice later denies the message, the center shows the archived copy, which it accepted only after verifying Alice's signature with her then-valid key; Alice cannot deny it, and Bob's copy carries the center's signature, so he cannot have invented it either. Nonrepudiation can be provided using a trusted party.

No confidentiality. A digital signature does not provide privacy: the message travels in clear (and in RSA anyone can even compute $M = S^e$ from the signature), since the goal is not to hide the message but to prove who sent it. If confidentiality is required, the message and the signature must be encrypted with a secret-key or public-key cryptosystem, adding another layer of encryption and decryption. Alice first signs with her private key, then

encrypts (M, S) with Bob's public key (or a shared session key); Bob decrypts with his private key and then verifies with Alice's public key.



The order **sign, then encrypt** matters. The signature is then on the plaintext, so Bob can later show (M, S) to a judge without revealing his private key. If Alice encrypted first and signed the ciphertext, Eve could strip Alice's signature and attach her own to the same ciphertext, claiming authorship of a message she cannot even read.

37.5 Attacks on digital signatures; types of forgery ▸

▸ **Likely this year:** "Explain key-only, known-message and chosen-message attacks; differentiate existential and selective forgery" (3–4 marks).

Attacks are classified by what Eve has access to, exactly as attacks on ciphers were.

- **Key-only attack.** Eve has access only to the public information released by Alice (her public key). To forge a message, she must create Alice's signature to convince Bob that the message comes from Alice. This is the analogue of the ciphertext-only attack on encipherment.
- **Known-message attack.** Eve has access to one or more message–signature pairs, i.e. some documents previously signed by Alice, and tries to create another message and forge Alice's signature on it. This is the analogue of the known-plaintext attack.
- **Chosen-message attack.** Eve somehow makes Alice sign one or more messages of Eve's choice, so she holds chosen-message/signature pairs. She later creates another message, with the content she wants, and forges Alice's signature on it. This is the analogue of the chosen-plaintext attack, and the strongest of the three.

If an attack succeeds, the result is a **forgery**, of one of two types.

Existential forgery. Eve is able to create a valid message–signature pair, but not one she can really use: the document is forged, but its content is calculated by the algebra (effectively random), so the message is likely to be syntactically or semantically unintelligible. Such forgery is probable for unprotected schemes, but Eve cannot benefit from it very much.

Selective forgery. Eve is able to forge Alice's signature on a message whose content *Eve selected*. This is beneficial to Eve and may be very detrimental to Alice; the probability of such forgery is low, but not negligible.

Forgery	What Eve obtains	Who chooses the content	Likelihood and harm
Existential	at least one valid (M, S)	the algebra (random-looking M)	likely for raw (unhashed) schemes; little benefit
Selective	a valid S on a message she chose beforehand	Eve	low probability, very harmful

Stallings adds two stronger outcomes: a **universal forgery** (Eve can sign *any* message) and a **total break** (Eve recovers the private key). A scheme is considered secure only if even the weakest outcome, existential forgery, is infeasible under the strongest attack, the chosen-message attack.

37.6 RSA digital signature scheme ★▸

▸ **Likely this year:** "Explain the RSA signature scheme with a diagram; sign and verify with small primes" (5 marks), Forouzan style.

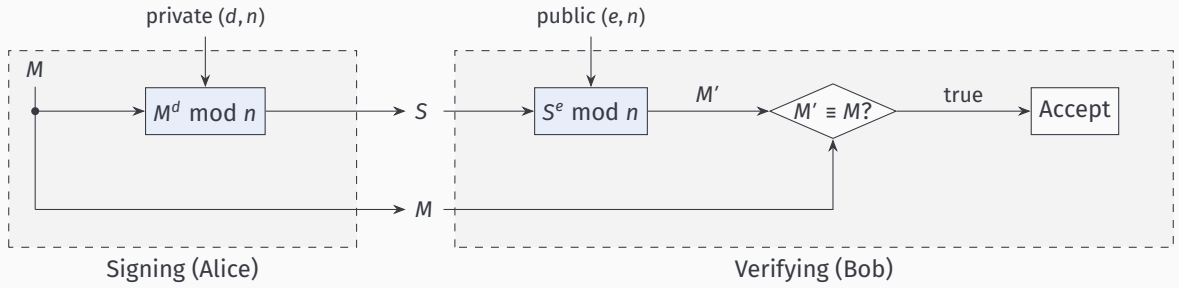
The RSA function $x \mapsto x^e \bmod n$ is a permutation of $\mathbb{Z}_n$ whose inverse $x \mapsto x^d$ only the key owner can compute (§33). For encryption the public direction locks; for a signature the roles are swapped: Alice applies her *private* exponent to the message, and anyone can undo it with her public exponent and compare.

★ MUST KNOW : RSA signature

Key generation (Alice): primes p, q; $n = pq$; $\phi(n) = (p-1)(q-1)$; e with $\gcd(e, \phi(n)) = 1$; $d = e^{-1} \bmod \phi(n)$. Public (e, n), private d (exactly as RSA).
Signing: $S = M^d \bmod n$; send (M, S). **Verifying:** $M' = S^e \bmod n$; accept if $M' \equiv M \pmod n$.
On the digest: $S = h(M)^d \bmod n$, accept if $S^e \bmod n = h(M)$.

Proof that verification succeeds. Since $ed = 1 + k\phi(n)$, $S^e \equiv (M^d)^e = M^{1+k\phi(n)} = M \cdot (M^{\phi(n)})^k \equiv M \pmod n$, by Euler's theorem when $\gcd(M, n) = 1$ and by the CRT argument of §33 otherwise. So a correctly signed message always

passes: $M' \equiv M \Rightarrow S^e \equiv M \Rightarrow M^{d\times e} \equiv M \pmod n$ (slide 33). Conversely, producing an S with $S^e \equiv M$ for a *given* M means taking an e-th root modulo n, which requires d (i.e. factoring n).



WORKED EXAMPLE : the slide example (slide 34)

Q. Alice chooses $p = 823$, $q = 953$ and $e = 313$. Find n, $\phi(n)$ and d. Sign $M = 19070$ and show Bob's verification.
Keys. $n = 823 \times 953 = 784\,319$; $\phi(n) = 822 \times 952 = 782\,544$; $\gcd(313, 782544) = 1$. $d = 313^{-1} \bmod 782544$:

q	r_1	r_2	r	t_1	t_2	t
2500	782544	313	44	0	1	−2500
7	313	44	5	1	−2500	17501
8	44	5	4	−2500	17501	−142508
1	5	4	1	17501	−142508	160009
4	4	1	0	−142508	160009	−782544
	1	0		**160009**	−782544	

$d =$ **160009**. Check: $313 \times 160009 = 50\,082\,817 = 64 \times 782544 + 1$ ✓.
Sign. $S = 19070^{160009} \bmod 784319 =$ **210625** (by square-and-multiply).
Verify. $M' = 210625^{313} \bmod 784319 = 19070 = M$, so Bob accepts: he has verified Alice's signature.

WORKED EXAMPLE : exam-size RSA signature

Q. Alice's RSA key uses $p = 7$, $q = 11$, $e = 7$. Find her private key, sign $M = 30$, and verify the signature.
Keys. $n = 77$, $\phi(n) = 6 \times 10 = 60$, $\gcd(7, 60) = 1$. $d = 7^{-1} \bmod 60$:

q	r_1	r_2	r	t_1	t_2	t
8	60	7	4	0	1	−8
1	7	4	3	1	−8	9
1	4	3	1	−8	9	−17
3	3	1	0	9	−17	60
	1	0		**−17**	60	

$d = -17 + 60 =$ **43**. Check: $7 \times 43 = 301 = 5 \times 60 + 1$ ✓. Public $(7, 77)$, private 43.
Sign $S = 30^{43} \bmod 77$, with $43 = 32 + 8 + 2 + 1$:

Power	Computation	mod 77	Power	Computation	mod 77
30^1		30	30^8	$37^2 = 1369 = 17 \cdot 77 + 60$	60
30^2	$900 = 11 \cdot 77 + 53$	53	30^{16}	$60^2 = 3600 = 46 \cdot 77 + 58$	58
30^4	$53^2 = 2809 = 36 \cdot 77 + 37$	37	30^{32}	$58^2 = 3364 = 43 \cdot 77 + 53$	53

$30^{32} \cdot 30^8 = 53 \cdot 60 = 3180 \equiv 23$; $23 \cdot 53 = 1219 \equiv 64$; $64 \cdot 30 = 1920 \equiv 72$. $\Rightarrow S =$ **72**.
Verify $M' = 72^7 \bmod 77$, $7 = 4 + 2 + 1$: $72^2 = 5184 \equiv 25$; $72^4 \equiv 25^2 = 625 \equiv 9$; $9 \cdot 25 = 225 \equiv 71$; $71 \cdot 72 = 5112 \equiv 30$.
$M' = 30 = M$ ✓: accept.

37.7 ElGamal digital signature scheme ★▸

▸ **Likely this year:** "Explain the ElGamal signature; prove that verification works; sign and verify with given p, e_1, d, r" (5–6 marks). The slide numbers are a likely source.

The ElGamal signature uses the same keys as the ElGamal cryptosystem (§34) but a different algorithm. Alice proves knowledge of d, the discrete logarithm of e_2, without revealing it: she commits to a fresh random exponent r through $S_1 = e_1^r$, and then publishes S_2, which ties M, d and r together by a linear equation in the exponent that only someone knowing d can satisfy.

★ MUST KNOW : ElGamal signature

Key generation: prime p, primitive root e_1 of p, private d; $e_2 = e_1^d \bmod p$. Public (e_1, e_2, p), private d.
Signing: choose a secret random r with $\gcd(r, p-1) = 1$ (new for every message);

$$S_1 = e_1^r \bmod p, \qquad S_2 = (M - d \times S_1) \times r^{-1} \bmod (p-1).$$

Verifying: check $0 < S_1 < p$ and $0 \le S_2 < p-1$; compute $V_1 = e_1^M \bmod p$ and $V_2 = e_2^{S_1} \times S_1^{S_2} \bmod p$; accept if $V_1 \equiv V_2$.

Why the arithmetic is modulo $p-1$. S_2 is used as an exponent of e_1, and by Fermat $e_1^{p-1} \equiv 1 \pmod p$, so exponents only matter modulo $p-1$. That is also why r must be invertible modulo $p-1$, i.e. $\gcd(r, p-1) = 1$; since $p-1$ is even, r must in particular be odd.

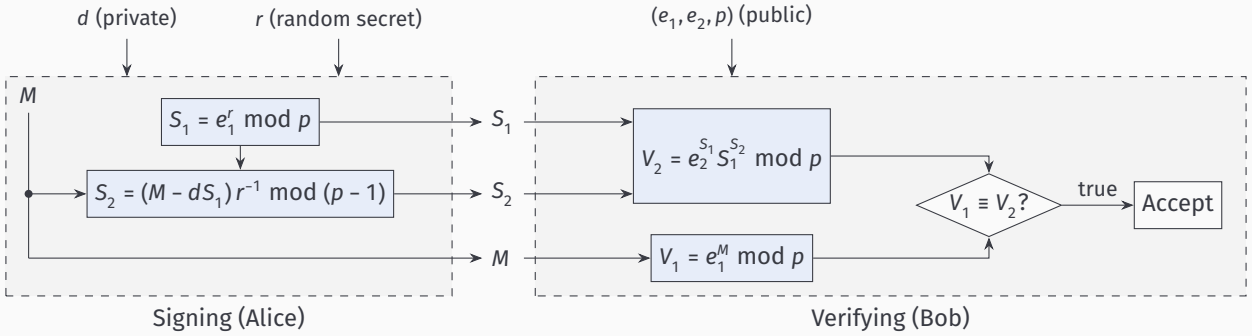
Proof that verification succeeds. Multiply the definition of S_2 by r: $S_2\, r \equiv M - d\, S_1 \pmod{p-1}$, i.e.

$$M \equiv d\, S_1 + r\, S_2 \pmod{p-1}.$$

Raising e_1 to both sides (allowed, since exponents are taken modulo $p-1$):

$$V_1 = e_1^M \equiv e_1^{dS_1 + rS_2} = (e_1^d)^{S_1} (e_1^r)^{S_2} = e_2^{S_1} \times S_1^{S_2} = V_2 \pmod p.$$

(Slide 37 writes the same chain backwards.) Eve, who does not know d, cannot solve $M \equiv dS_1 + rS_2$ for S_2; choosing S_1 and S_2 to satisfy $e_1^M \equiv e_2^{S_1} S_1^{S_2}$ directly is a discrete-logarithm problem.



Requirements on r. The random r must be (i) **coprime** to $p-1$, otherwise r^{-1} does not exist; (ii) **secret**, because from one signature and a known r Eve gets $d\, S_1 \equiv M - r\, S_2 \pmod{p-1}$ and hence d; and (iii) **fresh** for every message, because two signatures with the same r reveal r and then d (derived with numbers in §37.11). It should also be unpredictable, since a guessable r is a known r.

WORKED EXAMPLE : the slide examples (slides 38–39), recomputed

Q. Alice chooses $p = 3119$, $e_1 = 2$, $d = 127$. Find e_2. Sign $M = 320$ with $r = 307$ and $M = 3000$ with $r = 107$, and verify both signatures.
A. $e_2 = 2^{127} \bmod 3119 =$ **1702**; public $(2, 1702, 3119)$. Both r are odd primes not dividing $3118 = 2 \times 1559$, so $\gcd(r, 3118) = 1$.

M	r	$r^{-1} \bmod 3118$	$S_1 = 2^r \bmod 3119$	$(M - 127\, S_1) \bmod 3118$	S_2
320	307	2793	2083	$320 - 264541 \equiv 809$	$809 \cdot 2793 \equiv$ **2105**
3000	107	1661	2732	$3000 - 346964 \equiv 2134$	$2134 \cdot 1661 \equiv$ **2526**

M	$V_1 = 2^M \bmod 3119$	$1702^{S_1} \bmod 3119$	$S_1^{S_2} \bmod 3119$	V_2
320	3006	$1702^{2083} \equiv 2648$	$2083^{2105} \equiv 828$	$2648 \cdot 828 \equiv$ **3006**
3000	704	$1702^{2732} \equiv 1388$	$2732^{2526} \equiv 396$	$1388 \cdot 396 \equiv$ **704**

In both cases $V_1 \equiv V_2$, so Bob (and Ted) accept. Note that the second message must use its own $S_1 = 2732$ in S_2 and in V_2; see the correction box for the slide's slips.
Check of a reduction: $-264221 + 85 \times 3118 = 809$ and $2259537 - 724 \times 3118 = 2105$.

WORKED EXAMPLE : exam-size ElGamal signature

Q. In the ElGamal signature scheme, $p = 23$, $e_1 = 5$, $d = 6$. Find e_2. Sign $M = 10$ with $r = 7$ and verify.
Keys. $5^2 = 25 \equiv 2$, $5^4 \equiv 4$, so $e_2 = 5^6 = 5^4 \cdot 5^2 \equiv 8$. Public $(5, 8, 23)$.
r^{-1}. $\gcd(7, 22) = 1$; $22 = 3 \cdot 7 + 1$ gives $1 = 22 - 3 \cdot 7$, so $7^{-1} \equiv -3 \equiv \mathbf{19} \pmod{22}$ (check $7 \cdot 19 = 133 = 6 \cdot 22 + 1$).
Sign. $S_1 = 5^7 = 5^4 \cdot 5^2 \cdot 5 \equiv 4 \cdot 2 \cdot 5 = 40 \equiv \mathbf{17}$.
$S_2 = (10 - 6 \cdot 17) \cdot 19 = (-92) \cdot 19$; $-92 \equiv -92 + 110 = 18 \pmod{22}$; $18 \cdot 19 = 342 = 15 \cdot 22 + 12 \Rightarrow S_2 = \mathbf{12}$.
Verify. $V_1 = 5^{10} = 5^8 \cdot 5^2 \equiv 16 \cdot 2 = 32 \equiv \mathbf{9}$ ($5^8 = 4^2 = 16$).
$e_2^{S_1} = 8^{17}$: $8^2 \equiv 18$, $8^4 \equiv 18^2 = 324 \equiv 2$, $8^8 \equiv 4$, $8^{16} \equiv 16$, $8^{17} \equiv 16 \cdot 8 = 128 \equiv 13$.
$S_1^{S_2} = 17^{12}$: $17^2 = 289 \equiv 13$, $17^4 \equiv 169 \equiv 8$, $17^8 \equiv 64 \equiv 18$, $17^{12} = 17^8 \cdot 17^4 \equiv 18 \cdot 8 = 144 \equiv 6$.
$V_2 = 13 \cdot 6 = 78 \equiv \mathbf{9} = V_1$ ✓: accept. *Check*: $dS_1 + rS_2 = 102 + 84 = 186 = 8 \cdot 22 + 10 \equiv M \pmod{22}$.

37.8 Schnorr digital signature scheme ▸

▸ **Likely this year:** "Explain the Schnorr signature scheme; show that verification works" (4 marks).

The ElGamal signature is as long as $2|p|$ (2048 bits for a 1024-bit p), and all its exponent arithmetic is modulo the large number $p - 1$. Schnorr's idea is to work in a **subgroup of prime order** q of $\mathbb{Z}_p^*$, with q only as large as a hash digest (160 bits). Exponents then live modulo q, the two signature components are 160 bits each, and the hash of the message is mixed into S_1 itself.

Key generation (slide 43).

1. Alice selects a prime p, usually 1024 bits long.
2. She selects another prime q of the same size as the digest of the hash function (160 bits), such that q divides $p - 1$, i.e. $(p - 1) \equiv 0 \pmod{q}$.
3. She chooses e_1 to be a q-th root of 1 modulo p: she picks a primitive element e_0 of $\mathbb{Z}_p^*$ and computes $e_1 = e_0^{(p-1)/q} \bmod p$. Then $e_1^q = e_0^{p-1} \equiv 1$ while $e_1 \neq 1$, so e_1 has order exactly q (q is prime).
4. She chooses an integer d (with $1 \le d < q$) as her private key and computes $e_2 = e_1^d \bmod p$.
5. Public key (e_1, e_2, p, q); private key d.

Signing. Alice chooses a fresh random r ($1 \le r < q$), computes $e_1^r \bmod p$, concatenates it to the message and hashes:

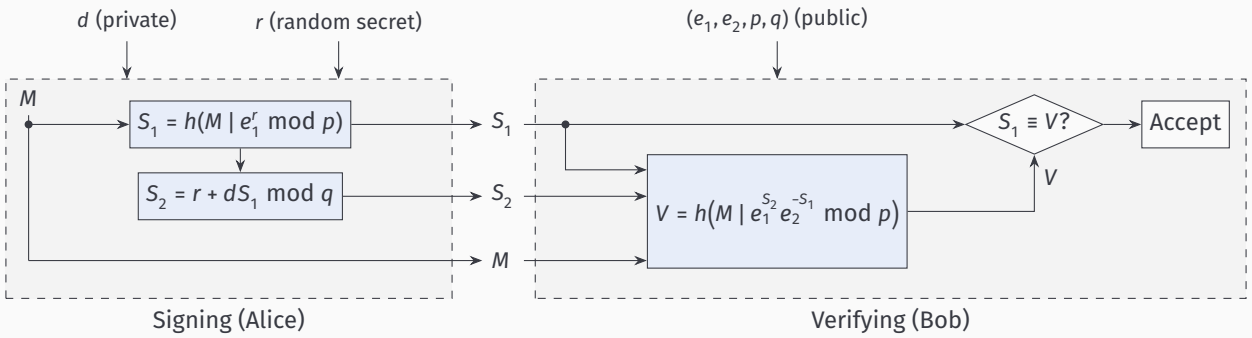
$$S_1 = h\big(M \mid e_1^r \bmod p\big), \qquad S_2 = r + d \times S_1 \bmod q.$$

Verifying. Bob computes $V = h\big(M \mid e_1^{S_2} e_2^{-S_1} \bmod p\big)$ and accepts if $V = S_1$.

Proof that verification succeeds. Since e_1 has order q, exponents of e_1 can be reduced modulo q, and $S_2 \equiv r + dS_1 \pmod{q}$. Therefore

$$e_1^{S_2} e_2^{-S_1} = e_1^{r+dS_1} (e_1^d)^{-S_1} = e_1^{r+dS_1-dS_1} = e_1^r \pmod{p},$$

so Bob hashes exactly the string $M \mid e_1^r \bmod p$ that Alice hashed, and $V = S_1$. Here $e_2^{-S_1}$ means $(e_2^{-1})^{S_1} \bmod p$. A forger would need S_2 with $e_1^{S_2} = X \cdot e_2^{S_1}$ for an S_1 that depends on X through the hash, which requires d.



WORKED EXAMPLE : a toy Schnorr signature

Q. Take $p = 23$, $q = 11$, $e_0 = 5$, $d = 7$. As a *toy* hash use $h(y) = y \bmod 11$, where $M \mid X$ means writing the decimal digits of X after those of M (a real scheme uses SHA-1/SHA-2; this h is not one-way). Find the keys, sign $M = 15$ with $r = 5$, and verify.
Keys. $q = 11$ divides $p - 1 = 22$, and 5 is a primitive root of 23. $e_1 = 5^{22/11} = 5^2 = 25 \equiv \mathbf{2}$; check $2^{11} = 2048 = 89 \cdot 23 + 1 \equiv 1$, so e_1 has order 11. $e_2 = 2^7 = 128 = 5 \cdot 23 + 13 \equiv \mathbf{13}$. Public $(2, 13, 23, 11)$, private 7.
Sign. $X = e_1^r = 2^5 = 32 \equiv 9$. $M \mid X = 15 \mid 9 = 159$; $S_1 = 159 \bmod 11 = \mathbf{5}$ ($159 = 14 \cdot 11 + 5$). $S_2 = 5 + 7 \cdot 5 = 40 \equiv \mathbf{7} \pmod{11}$.

Verify. $e_1^{S_2} = 2^7 \equiv 13$. $e_2^{-1} = 13^{-1} \bmod 23 = 16$ ($13 \cdot 16 = 208 = 9 \cdot 23 + 1$). $e_2^{-S_1} = 16^5$: $16^2 = 256 \equiv 3$, $16^4 \equiv 9$, $16^5 \equiv 9 \cdot 16 = 144 \equiv 6$.
$e_1^{S_2} e_2^{-S_1} = 13 \cdot 6 = 78 \equiv 9 = X$ ✓; $V = h(15 \mid 9) = h(159) = 5 = S_1$: **accept**.

37.9 Digital Signature Standard (DSS / DSA) ★▸

▸ **Likely this year:** "Explain DSS key generation, signing and verifying with a diagram; prove $V = S_1$; compare DSS with RSA" (5–6 marks), possibly with a small numerical.

The **Digital Signature Standard** (DSS) was adopted by NIST in 1994 as FIPS 186; its algorithm is the **Digital Signature Algorithm** (DSA). It combines the two previous ideas: the ElGamal form of S_2 (hash plus dS_1, divided by r) and Schnorr's order-q subgroup, so that S_1 and S_2 are both reduced modulo the 160-bit prime q.

Key generation (slide 47).

1. Alice chooses a prime p between 512 and 1024 bits long, the number of bits being a multiple of 64.
2. She chooses a 160-bit prime q that divides $p - 1$.
3. She uses two multiplicative structures: $\mathbb{Z}_p^*$, in which the powers of e_1 are computed, and the subgroup of order q generated by e_1, whose exponents are computed modulo q.
4. She creates e_1 as a q-th root of 1 modulo p ($e_1^q \equiv 1 \pmod p$): she chooses a primitive element e_0 of $\mathbb{Z}_p^*$ and computes $e_1 = e_0^{(p-1)/q} \bmod p$.
5. She chooses d ($1 \le d < q$) as the private key and computes $e_2 = e_1^d \bmod p$.
6. Public key (e_1, e_2, p, q); private key d.

(The current standard FIPS 186-4 uses $(|p|, |q|) = (2048, 224)$, $(2048, 256)$ or $(3072, 256)$ with SHA-2.)

★ MUST KNOW : DSS

Signing: fresh secret random r, $1 \le r < q$;

$$S_1 = (e_1^r \bmod p) \bmod q, \qquad S_2 = \big(h(M) + d \times S_1\big)\, r^{-1} \bmod q.$$

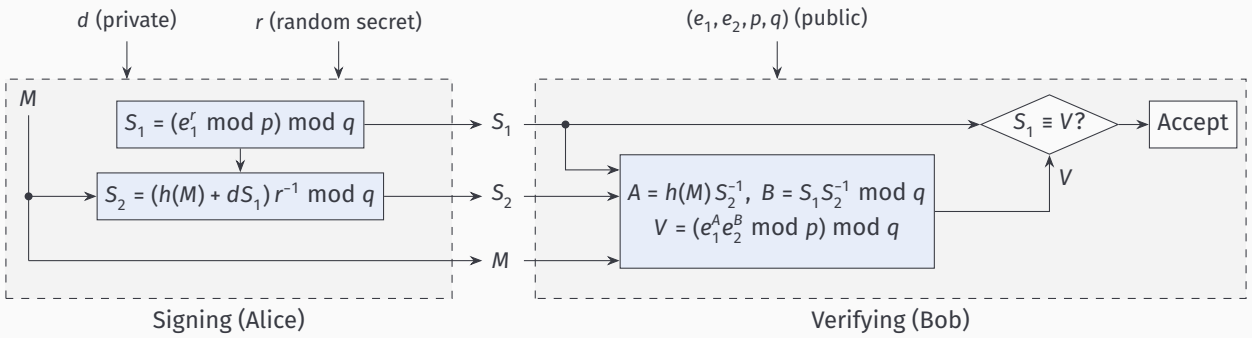
Verifying: check $0 < S_1 < q$, $0 < S_2 < q$; then

$$A = h(M)\, S_2^{-1} \bmod q, \quad B = S_1\, S_2^{-1} \bmod q, \quad V = \big(e_1^A\, e_2^B \bmod p\big) \bmod q; \quad \text{accept if } V = S_1.$$

Proof that verification succeeds. From the definition of S_2, $r \equiv \big(h(M) + dS_1\big)S_2^{-1} = h(M)S_2^{-1} + d\,S_1 S_2^{-1} = A + dB \pmod q$. Since e_1 has order q, exponents of e_1 reduce modulo q, so

$$e_1^A\, e_2^B = e_1^A\, (e_1^d)^B = e_1^{A+dB} = e_1^r \pmod p,$$

and reducing modulo q gives $V = (e_1^r \bmod p) \bmod q = S_1$. The slide's verification box $\big(e_1^{h(M)S_2^{-1}}\, e_2^{S_1 S_2^{-1}} \bmod p\big) \bmod q$ is the same formula with A and B written out.



WORKED EXAMPLE : the textbook DSS example, every value recomputed

Q. Alice chooses $q = 443$, $p = 48731$, $e_0 = 7$ and $d = 242$. Find e_1 and e_2. With $h(M) = 100$ and $r = 427$, find the signature and verify it.
Keys. $p - 1 = 48730 = 110 \times 443$, so $q \mid p - 1$; 7 is a primitive root of 48731. $e_1 = 7^{110} \bmod 48731 =$ **5260** (order 443), $e_2 = 5260^{242} \bmod 48731 =$ **3438**.
Sign. $e_1^r = 5260^{427} \bmod 48731 = 2717$; $S_1 = 2717 \bmod 443 = 2717 - 6 \cdot 443 =$ **59**.
$h(M) + dS_1 = 100 + 242 \cdot 59 = 14378 \equiv 14378 - 32 \cdot 443 = 202$; $r^{-1} = 427^{-1} \bmod 443 = 83$ ($427 \cdot 83 = 35441 = 80 \cdot 443 + 1$).
$S_2 = 202 \cdot 83 = 16766 \equiv 16766 - 37 \cdot 443 =$ **375**.

Verify. $S_2^{-1} = 375^{-1} \bmod 443 = 228$ ($375 \cdot 228 = 85500 = 193 \cdot 443 + 1$).
$A = 100 \cdot 228 = 22800 \equiv 207$, $B = 59 \cdot 228 = 13452 \equiv 162 \pmod{443}$.
$e_1^A = 5260^{207} \equiv 10322$, $e_2^B = 3438^{162} \equiv 25225$, $10322 \cdot 25225 \equiv 2717 \pmod{48731}$ (the same e_1^r, as the proof predicts).
$V = 2717 \bmod 443 = \mathbf{59} = S_1$ ✓: accept. (If $S_2 = 164$ were sent, Bob would get $V = 119 \neq 59$ and reject; see the correction box.)

WORKED EXAMPLE : exam-size DSS

Q. In DSS, $p = 23$, $q = 11$, $e_0 = 5$, $d = 7$. Find e_1 and e_2. Sign a message with $h(M) = 5$ using $r = 3$, and verify.
Keys. $q = 11 \mid 22$. $e_1 = 5^{22/11} = 25 \equiv \mathbf{2} \pmod{23}$ (order 11, since $2^{11} \equiv 1$). $e_2 = 2^7 = 128 \equiv \mathbf{13}$.
Sign. $S_1 = (2^3 \bmod 23) \bmod 11 = 8 \bmod 11 = \mathbf{8}$. $h(M) + dS_1 = 5 + 56 = 61 \equiv 6 \pmod{11}$; $3^{-1} \bmod 11 = 4$ ($3 \cdot 4 = 12$); $S_2 = 6 \cdot 4 = 24 \equiv \mathbf{2}$.
Verify. $S_2^{-1} = 2^{-1} \bmod 11 = 6$; $A = 5 \cdot 6 = 30 \equiv 8$, $B = 8 \cdot 6 = 48 \equiv 4 \pmod{11}$.
$e_1^A = 2^8 = 256 \equiv 3$; $e_2^B = 13^4$: $13^2 = 169 \equiv 8$, $8^2 = 64 \equiv 18$; $3 \cdot 18 = 54 \equiv 8 \pmod{23}$.
$V = 8 \bmod 11 = \mathbf{8} = S_1$ ✓. *Check*: $A + dB = 8 + 28 = 36 \equiv 3 = r \pmod{11}$.

DSS versus RSA. The comparison is a standard question.

	DSS (DSA)	RSA signature
Hard problem	discrete logarithm (in an order-q subgroup of $\mathbb{Z}_p^*$)	factoring $n = pq$
Use	signatures only; cannot encrypt or exchange keys	signatures, encryption and key transport
Signature size	$2 \times 160 = 320$ bits, independent of $\lvert p \rvert$	$\lvert n \rvert$ bits (1024–2048)
Randomness	fresh secret r per signature; reuse or leak reveals d	deterministic (unless a randomized padding such as PSS is used)
Signing cost	one exponentiation mod p with a 160-bit exponent; e_1^r and r^{-1} can be precomputed before the message is known	one full exponentiation with the large d
Verifying cost	two exponentiations mod p (slower)	one exponentiation with small e (very fast)

Hence DSS signing is faster than RSA signing for the same size of p, but RSA verification is faster, which suits certificates that are verified many times.

DSS versus ElGamal. Both use a random r and the same shape of S_2, but DSS reduces S_1 and S_2 modulo the 160-bit q instead of working modulo p and $p-1$. The signature therefore shrinks from $2\lvert p \rvert$ (2048 bits) to 320 bits, the inversion r^{-1} and the products are computed modulo a small q, and the hash is built into the standard, so the attacks on raw ElGamal below do not apply.

37.10 Elliptic Curve Digital Signature Algorithm (ECDSA) ▸

▸ **Likely this year:** "Explain the elliptic curve signature scheme; why does verification work?" (4 marks), or a small curve computation.

ECDSA (adopted in FIPS 186-2 in 2000) is DSS translated to the group of points of an elliptic curve (§34): exponentiation e_1^r becomes the scalar multiplication $r \times e_1$, and the reduction $(e_1^r \bmod p) \bmod q$ becomes "take the x-coordinate of the point modulo q". Because the best known attack on the ECDLP is generic, a 160-bit q gives the security of a 1024-bit DSA modulus, with much smaller keys.

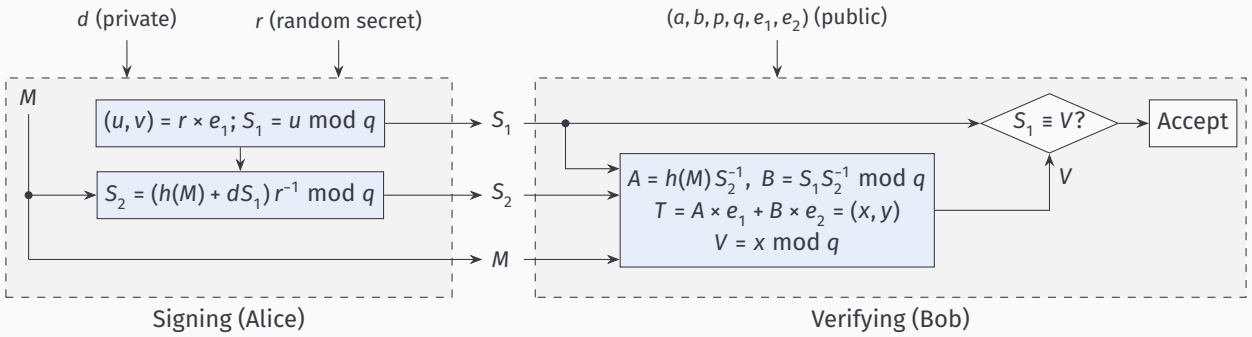
★ MUST KNOW : ECDSA

Key generation: curve $E_p(a, b)$; a point e_1 of prime order q ($q \times e_1 = O$); private d, $1 \le d < q$; $e_2 = d \times e_1$. Public (a, b, p, q, e_1, e_2), private d.
Signing: fresh secret random r, $1 \le r < q$; $P = r \times e_1 = (u, v)$; $S_1 = u \bmod q$, $S_2 = \big(h(M) + d \times S_1\big) r^{-1} \bmod q$ (if $S_1 = 0$ or $S_2 = 0$, pick a new r).
Verifying: $A = h(M)\, S_2^{-1} \bmod q$, $B = S_1\, S_2^{-1} \bmod q$, $T = A \times e_1 + B \times e_2 = (x, y)$; accept if $x \bmod q = S_1$.

Proof that verification succeeds. As for DSS, $A + dB \equiv \big(h(M) + dS_1\big) S_2^{-1} \equiv r \pmod{q}$. Since e_1 has order q, multiples of e_1 depend only on the multiplier modulo q, so

$$T = A \times e_1 + B \times (d \times e_1) = (A + dB) \times e_1 = r \times e_1 = P = (u, v),$$

hence $x \bmod q = u \bmod q = S_1$. Both nonzero checks matter: $S_2 = 0$ has no inverse, and $S_1 = 0$ would make S_2 independent of d.



Slide 40 shows the common pattern of DSS and ECDSA: S_1 is $f_1(e_1, r, p, q)$ followed by an "Extract" step modulo q (the residue mod q in DSS, the x-coordinate mod q in ECDSA), and $S_2 = f_2(h(M), d, r, q, S_1)$ is the same formula in both.

WORKED EXAMPLE : ECDSA on a toy curve

Q. Use $E_{17}(2,6)$: $y^2 = x^3 + 2x + 6 \pmod{17}$, with $e_1 = (1,3)$. Alice's private key is $d = 4$. Find the order q of e_1 and e_2. Sign a message with $h(M) = 5$ using $r = 3$, and verify.
Curve and order. $4a^3 + 27b^2 = 32 + 972 = 1004 \equiv 1 \neq 0 \pmod{17}$, and $(1,3)$ is on it: $1 + 2 + 6 = 9 = 3^2$. The curve has 10 affine points plus O, i.e. 11 points; 11 is prime, so every point other than O has order 11 and **q = 11** (confirmed by computing the multiples of $(1,3)$ in the script: $1e_1, \ldots, 10e_1$ are $(1,3)$, $(2,16)$, $(13,11)$, $(11,13)$, $(6,9)$, $(6,8)$, $(11,4)$, $(13,6)$, $(2,1)$, $(1,14)$ and $11e_1 = O$).

Step	Slope s (mod 17)	x_3	y_3	Result
$2e_1 = 2(1,3)$	$(3+2)\cdot 6^{-1} = 5\cdot 3 = 15$	$225 - 2 \equiv 2$	$15(1-2) - 3 \equiv 16$	$(2,16)$
$e_2 = 4e_1 = 2(2,16)$	$14 \cdot 32^{-1} = 14 \cdot 8 \equiv 10$	$100 - 4 \equiv 11$	$10(2-11) - 16 \equiv 13$	$(11,13)$
$P = 3e_1 = (2,16) + (1,3)$	$(-13)(-1)^{-1} = 13$	$169 - 3 \equiv 13$	$13(2-13) - 16 \equiv 11$	$(13,11)$
$2e_2 = 2(11,13)$	$365 \cdot 26^{-1} \equiv 8 \cdot 2 = 16$	$256 - 22 \equiv 13$	$16(11-13) - 13 \equiv 6$	$(13,6)$
$3e_2 = (13,6) + (11,13)$	$7 \cdot (-2)^{-1} = 7 \cdot 8 \equiv 5$	$25 - 24 = 1$	$5(13-1) - 6 \equiv 3$	$(1,3)$
$T = (2,16) + (1,3)$	as row 3	13	11	$(13,11)$

Inverses mod 17: $6 \cdot 3 = 18$, $15 \cdot 8 = 120 = 7 \cdot 17 + 1$ ($32 \equiv 15$, $-2 \equiv 15$), $9 \cdot 2 = 18$ ($26 \equiv 9$); $365 \equiv 8$.
Keys. $e_2 = 4 \times e_1 = (11,13)$. Public $(2, 6, 17, 11, (1,3), (11,13))$.
Sign. $P = 3 \times e_1 = (13,11)$, so $u = 13$ and $S_1 = 13 \bmod 11 = \mathbf{2}$. $3^{-1} \bmod 11 = 4$; $S_2 = (5 + 4 \cdot 2) \cdot 4 = 52 \equiv \mathbf{8} \pmod{11}$.
Verify. $S_2^{-1} = 8^{-1} \bmod 11 = 7$ ($56 = 5 \cdot 11 + 1$); $A = 5 \cdot 7 = 35 \equiv 2$, $B = 2 \cdot 7 = 14 \equiv 3$.
$A \times e_1 = 2e_1 = (2,16)$; $B \times e_2 = 3e_2 = (1,3)$; $T = (2,16) + (1,3) = (13,11)$. $x \bmod q = 13 \bmod 11 = \mathbf{2} = S_1$ ✓: accept.
Check: $A + dB = 2 + 12 = 14 \equiv 3 = r \pmod{11}$, so T must equal $3e_1 = P$.

37.11 Security attacks on digital signature schemes ▸

▸ **Likely this year:** Assignment topic: "Explain attacks on RSA / ElGamal / DSS signatures"; "show that reusing r reveals the private key" with numbers (5–6 marks).

The attacks below are of two kinds. The first kind exploits the *algebra* of a scheme applied to raw messages and gives existential or selective forgeries; hashing the message defeats it. The second kind exploits a *misused random number* r in ElGamal, Schnorr, DSS and ECDSA and gives a total break (the private key); hashing does not help against it, only a fresh secret r does.

1. RSA, key-only attack: existential forgery. Eve knows only (e, n). She picks any value S and computes $M = S^e \bmod n$. Then (M, S) is a valid pair, because verification computes exactly $S^e \bmod n = M$. Eve cannot choose M (it is whatever the algebra gives), so the forgery is existential. With Alice's key $(e, n) = (7, 77)$ from the worked example: Eve picks $S = 2$, so $M = 2^7 = 128 \equiv \mathbf{51}$; and indeed Alice's own signature on 51 would be $51^{43} \bmod 77 = 2$.

2. RSA, known-message attack: the multiplicative property. RSA signatures multiply: $(M_1 M_2)^d = M_1^d M_2^d$. If Eve has two genuine pairs (M_1, S_1) and (M_2, S_2), then $S = S_1 S_2 \bmod n$ is a valid signature on $M = M_1 M_2 \bmod n$. With the same key, Alice signed $M_1 = 3$ ($S_1 = 3^{43} \bmod 77 = 38$) and $M_2 = 5$ ($S_2 = 26$). Eve forms $M = 15$ and $S = 38 \cdot 26 = 988 \equiv \mathbf{64}$, and $64^7 \bmod 77 = 15$ ✓.

3. RSA, chosen-message attack: selective forgery. Now Eve wants Alice's signature on a *specific* M that Alice would refuse to sign. She picks a random x and asks Alice to sign the harmless-looking $M' = M \cdot x^e \bmod n$. Alice returns $S' = (Mx^e)^d = M^d x^{ed} \equiv M^d \cdot x$, and Eve computes $S = S' \cdot x^{-1} \bmod n = M^d$, a genuine signature on her chosen message. (Forouzan's version uses two queries: M_1 and $M_2 = M \cdot M_1^{-1}$, then $S = S_1 S_2$.) Numerically, target $M = 20$, $x = 2$: $x^e = 2^7 \equiv 51$, $M' = 20 \cdot 51 = 1020 \equiv 19$; Alice signs, $S' = 19^{43} \bmod 77 = 61$; $x^{-1} = 39$; $S = 61 \cdot 39 = 2379 \equiv \mathbf{69}$, and $69^7 \bmod 77 = 20$ ✓.

Hashing stops all three. When the digest is signed, verification checks $S^e \equiv h(M)$. In attack 1 Eve gets a value S^e

that must be the digest of some message, so she needs a preimage of h. In attacks 2 and 3 the product $h(M_1)h(M_2)$ (or $h(M)x^e$) is not the digest of any message she can name. Standard RSA signatures (PKCS#1 v1.5, PSS) also pad the digest with fixed structure and, in PSS, randomness.

4. ElGamal, key-only attack: existential forgery. Without hashing, Eve can produce a valid triple (M, S_1, S_2) from the public key alone. She chooses integers u and v with $\gcd(v, p-1) = 1$ and sets $S_1 = e_1^u e_2^v \bmod p$. Then

$$e_2^{S_1} S_1^{S_2} = e_2^{S_1} e_1^{uS_2} e_2^{vS_2} = e_1^{uS_2} e_2^{S_1+vS_2} \pmod{p}.$$

Choosing $S_2 = -S_1 v^{-1} \bmod (p-1)$ makes $S_1 + vS_2 \equiv 0$, so the e_2 factor disappears and the right side is $e_1^{uS_2}$; the message $M = uS_2 \bmod (p-1)$ then satisfies $e_1^M \equiv e_2^{S_1} S_1^{S_2}$. With the slide key ($p = 3119$, $e_1 = 2$, $e_2 = 1702$), $u = 5$, $v = 3$: $S_1 = 2^5 \cdot 1702^3 = 32 \cdot 1158 \equiv \mathbf{2747}$; $v^{-1} = 3^{-1} \bmod 3118 = 2079$; $S_2 = -2747 \cdot 2079 \equiv \mathbf{1163}$; $M = 5 \cdot 1163 = 5815 \equiv \mathbf{2697}$. Check: $V_1 = 2^{2697} \equiv 3072$ and $V_2 = 1702^{2747} \cdot 2747^{1163} \equiv 3072 \pmod{3119}$ ✓. With a hash, Eve would need a message with $h(M) = 2697$, a preimage.

5. ElGamal with a reused r: total break. Suppose Alice signs two messages M and M' with the same r. Then $S_1 = e_1^r$ is the same in both (which also tells Eve that r was reused), and

$$S_2 r \equiv M - dS_1, \quad S_2' r \equiv M' - dS_1 \pmod{p-1} \implies (S_2 - S_2')r \equiv M - M' \pmod{p-1}.$$

If $\gcd(S_2 - S_2', p-1) = 1$, Eve gets $r = (M - M')(S_2 - S_2')^{-1} \bmod (p-1)$; otherwise there are only gcd candidates, each tested against $e_1^r = S_1$. Knowing r, she solves the signing equation for d: $d = (M - rS_2)S_1^{-1} \bmod (p-1)$ (again with a few candidates, tested against $e_2 = e_1^d$, if $\gcd(S_1, p-1) \neq 1$). This is also why a leaked r is fatal: the second step alone gives d.

WORKED EXAMPLE : recovering d from a reused r (slide key)

Q. Alice ($p = 3119$, $e_1 = 2$, $e_2 = 1702$) signs $M = 320$ and $M' = 1111$ and Eve sees $(S_1, S_2) = (2083, 2105)$ and $(S_1, S_2') = (2083, 706)$. Find r and d.
A. Same S_1, so the same r was used. Work modulo $p - 1 = 3118$.
$M - M' = 320 - 1111 = -791 \equiv 2327$; $S_2 - S_2' = 2105 - 706 = 1399$, $\gcd(1399, 3118) = 1$, and $1399^{-1} \bmod 3118 = 1939$ (extended Euclid; $1399 \cdot 1939 = 2712661 = 870 \cdot 3118 + 1$).
$r = 2327 \cdot 1939 \bmod 3118 = \mathbf{307}$. Check: $2^{307} \bmod 3119 = 2083 = S_1$ ✓.
$M - rS_2 = 320 - 307 \cdot 2105 = -645915 \equiv 2629 \pmod{3118}$; $S_1^{-1} = 2083^{-1} \bmod 3118 = 1919$ ($2083 \cdot 1919 = 3997277 = 1282 \cdot 3118 + 1$).
$d = 2629 \cdot 1919 \bmod 3118 = \mathbf{127}$. Check: $2^{127} \bmod 3119 = 1702 = e_2$ ✓. Eve now owns Alice's private key (total break).

6. DSS, Schnorr and ECDSA with a reused or leaked nonce. The same algebra breaks the order-q schemes, and more easily, because q is prime and every nonzero difference is invertible.

- **DSS/ECDSA**: $S_2 r \equiv h + dS_1$ and $S_2' r \equiv h' + dS_1 \pmod{q}$ give $r = (h - h')(S_2 - S_2')^{-1}$ and then $d = (S_2 r - h)S_1^{-1} \bmod q$. With the exam-size DSS key ($q = 11$), signatures $(8, 2)$ on $h = 5$ and $(8, 7)$ on $h' = 9$ (both with $r = 3$): $r = (5-9)(2-7)^{-1} = 7 \cdot 6^{-1} = 7 \cdot 2 = 14 \equiv \mathbf{3}$, and $d = (2 \cdot 3 - 5) \cdot 8^{-1} = 1 \cdot 7 = \mathbf{7}$ ✓. In 2010 the private ECDSA signing key of the Sony PlayStation 3 was recovered exactly this way, because the same r was used for every signature.
- **Schnorr**: $S_2 = r + dS_1$ and $S_2' = r + dS_1' \pmod{q}$ give $d = (S_2 - S_2')(S_1 - S_1')^{-1} \bmod q$ directly. With the toy Schnorr key, $M = 15$ gave $(5, 7)$; $M' = 21$ with the same $r = 5$ gives $S_1' = h(21 \mid 9) = 219 \bmod 11 = 10$ and $S_2' = 5 + 70 \equiv 9$. Then $d = (7 - 9)(5 - 10)^{-1} = 9 \cdot 6^{-1} = 9 \cdot 2 = 18 \equiv \mathbf{7}$ ✓.
- A **leaked** r gives d from a single signature: $d = (S_2 r - h)S_1^{-1}$ (DSS) or $d = (S_2 - r)S_1^{-1}$ (Schnorr) modulo q. Even a few predictable bits of many nonces are enough for lattice attacks, so r must come from a strong random generator (or be derived deterministically from d and $h(M)$, as RFC 6979 does).

7. Attacks on the hash. Signing the digest moves part of the security into h. If Eve can find a collision $h(M) = h(M')$ between a fair and a fraudulent document, Alice's signature on M is also a signature on M' (alternate collision attack with $2^{n/2}$ work, §35). This is why signature schemes moved from MD5 and SHA-1 to SHA-2.

Attack	Model	Result	Defence
RSA: $M = S^e$	key-only	existential	sign $h(M)$: Eve would need a preimage
RSA: $S_1 S_2$ signs $M_1 M_2$	known-message	existential	sign $h(M)$ (products are not digests); padding
RSA blinding Mx^e	chosen-message	selective	sign $h(M)$; never sign raw data for others
ElGamal (u, v) forgery	key-only	existential	sign $h(M)$
Reused / leaked r	known-message	total break	fresh, secret, unpredictable r (hash does not help)
Hash collision	chosen-message	selective	collision-resistant hash ($n \geq 256$)

37.12 Comparison of the five schemes

▸ **Likely this year:** "Compare RSA, ElGamal, Schnorr, DSS and ECDSA signatures" (3–4 marks).

Scheme	Hard problem	Public key (private: d)	Signature size	Fresh random r
RSA	factoring n	(e, n)	$\lvert n\rvert$ = 1024–2048 bits	no
ElGamal	DLP modulo p	(e_1, e_2, p)	$2\lvert p\rvert$ = 2048 bits	yes, $\gcd(r, p-1) = 1$
Schnorr	DLP in order-q subgroup	(e_1, e_2, p, q)	2 × 160 = 320 bits	yes
DSS	DLP in order-q subgroup	(e_1, e_2, p, q)	2 × 160 = 320 bits	yes
ECDSA	ECDLP	(a, b, p, q, e_1, e_2)	2 × 160 = 320 bits	yes

Scheme	Signature	Accept if
RSA	$S = h(M)^d \bmod n$	$S^e \equiv h(M) \pmod n$
ElGamal	$S_1 = e_1^r \bmod p$, $S_2 = (M - dS_1)\, r^{-1} \bmod (p-1)$	$e_1^M \equiv e_2^{S_1} S_1^{S_2} \pmod p$
Schnorr	$S_1 = h(M \mid e_1^r \bmod p)$, $S_2 = r + dS_1 \bmod q$	$h(M \mid e_1^{S_2} e_2^{-S_1} \bmod p) = S_1$
DSS	$S_1 = (e_1^r \bmod p) \bmod q$, $S_2 = (h(M) + dS_1)\, r^{-1} \bmod q$	$(e_1^A e_2^B \bmod p) \bmod q = S_1$
ECDSA	$S_1 = x(r \times e_1) \bmod q$, S_2 as in DSS	$x(A \times e_1 + B \times e_2) \bmod q = S_1$

Here $A = h(M)S_2^{-1}$ and $B = S_1 S_2^{-1} \bmod q$. RSA is the only deterministic scheme and the only one that can also encrypt; the four discrete-logarithm schemes all fail completely if r is reused, and the three order-q schemes have 320-bit signatures because both components are reduced modulo the 160-bit q.

Trap: the usual slips in signature numericals

In ElGamal, S_2 is reduced modulo $p-1$ and r^{-1} is taken modulo $p-1$, never modulo p; in DSS, Schnorr and ECDSA everything after e_1^r (or $r \times e_1$) is modulo q. In ElGamal verification the base of the first factor is e_2 (public), not d. Each new message gets a new S_1, and that new S_1 must be used in S_2 and in V_2. Turn $M - dS_1$ into a non-negative residue before multiplying by r^{-1}.

Corrections to the notes / slides

- Slides 38–39: V_2 is printed as $d^{S_1} \times S_1^{S_2}$. It is $e_2^{S_1} \times S_1^{S_2}$ (the private d is unknown to Bob); the numbers on the slides do use $e_2 = 1702$.
- Slide 39: $S_2 = (3000 - 127 \times 2083) \times 107^{-1}$ uses the *old* S_1. With the new $r = 107$, $S_1 = 2^{107} \bmod 3119 = 2732$, and $S_2 = (3000 - 127 \times 2732) \times 107^{-1} = 2134 \times 1661 \bmod 3118 =$ **2526**, which is the value printed (with 2083 the result would be 1985). Likewise $V_2 = 1702^{2732} \times \mathbf{2732}^{2526} = 704 \bmod 3119$; as printed, $1702^{2732} \times 2083^{2526} \equiv 659$, which would not verify.
- Slide 38: $e_1 = 2$ is not a primitive root of 3119. Since $3119 \equiv 7 \pmod 8$, 2 is a quadratic residue and its order is $1559 = (p-1)/2$. The example still verifies (every exponent identity holds modulo the order, which divides $p-1$), but the key generation asks for a primitive root (e.g. 7 would do).
- Slide 47: "$e_1^p = 1 \bmod p$" should read $e_1^q \equiv 1 \pmod p$ (e_1 is a q-th root of 1). "$\langle \mathbb{Z}_q^*, \times \rangle$ is a subgroup of $\langle \mathbb{Z}_p^*, \times \rangle$" is loose: the subgroup of $\mathbb{Z}_p^*$ used is the order-q group generated by e_1, and its exponents are computed modulo q.
- Textbook DSS example ($p = 48731$, $q = 443$, $d = 242$, $r = 427$, $h(M) = 100$): if your copy shows $S_2 = 164$, it is wrong. With these values $S_1 = 59$ and $S_2 =$ **375**, and $V = 59$; the pair $(59, 164)$ gives $V = 119$ and fails.

38 Key Management and Key Distribution

Notation. $E_K(X)$ is X encrypted with the symmetric key K; a comma inside $E_K(\cdot)$ means concatenation. K_A is Alice's long-term *master key* shared with the key-distribution center, K_{AB} a *session key* between Alice and Bob, R_A a *nonce* (a random number used only once) chosen by Alice, and T a timestamp. In the diagrams a black circle marks the sender and the number of each message.

38.1 The symmetric-key distribution problem: the N^2 problem ▸

▸ **Likely this year:** "What is the N^2 problem? How many keys do N users need, with and without a KDC?" (2–3 marks).

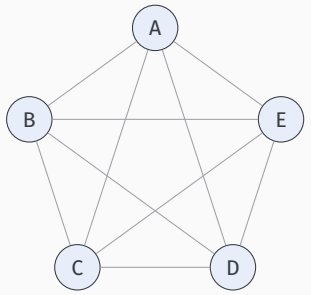
Symmetric-key cryptography is far faster than public-key cryptography, so it is used for all bulk data. Its weakness is that both parties must already hold the same secret key. If Alice wants to exchange confidential messages with N people, she needs N different keys, one per correspondent, since a key shared with Carol must not let Carol read Alice's traffic with Bob.

Counting the keys. Now let N people all communicate with each other. Each person needs a key with each of the other $N-1$ people, which gives $N(N-1)$ (person, partner) combinations. Every key, however, is shared by exactly two people and has therefore been counted twice, so

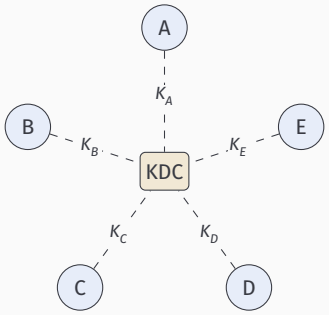
$$\text{number of keys} = \frac{N(N-1)}{2} = \binom{N}{2} \quad \text{(one key per pair)}, \qquad N(N-1) \quad \text{(a separate key for each direction)}.$$

Each person stores $N-1$ keys. Both counts grow like N^2, so this is called the N^2 **problem**. For one million people, each person holds 999 999 keys and the total is $10^6(10^6-1)/2 = 499\,999\,500\,000 \approx 5\times10^{11}$ keys, or about 10^{12} (one trillion) if each direction has its own key. A newcomer needs N new keys, all delivered securely.

The distribution problem. The number of keys is not the only problem: each key must reach both parties over a secure channel *before* any secure communication is possible, and the Internet is not one. Stallings lists four ways for Alice and Bob to obtain a key: (1) Alice chooses it and delivers it physically to Bob; (2) a third party chooses it and delivers it physically to both; (3) if they already share a key, one sends the new key encrypted under the old one; (4) if each has an encrypted link to a trusted third party, the third party sends the key to both over those links. Physical delivery does not scale to millions of users, and option 3 still needs a first key, and one compromised key then exposes every later one. Option 4 is the key-distribution center.



without a KDC: $\binom{5}{2} = 10$ pairwise keys



with a KDC: 5 master keys

Users N	Keys per user	Pairwise keys $N(N-1)/2$	Keys with a KDC (N)
10	9	45	10
100	99	4 950	100
1 000	999	499 500	1 000
10^6	999 999	$\approx 5\times10^{11}$	10^6

38.2 Key-distribution center (KDC) and session keys ▸

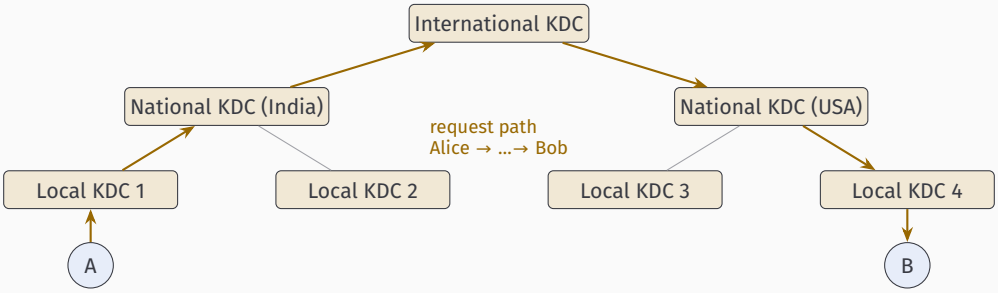
▸ **Likely this year:** "Explain the role of a KDC; why are session keys temporary; flat versus hierarchical KDCs" (3–4 marks).

A practical solution is a trusted third party called a **key-distribution center (KDC)**. Each member establishes one secret key, the master key, with the KDC: Alice shares K_{Alice} (written K_A), Bob shares K_B, and so on. The master key is set up once, for example in person when the member registers, and is used only between the member and the KDC, never between two members. As a result N members need only N master keys, and a newcomer needs one key instead of N.

Session keys. When Alice wants to talk to Bob, she asks the KDC for a **session key** K_{AB}, a fresh random key valid only for one conversation. The KDC uses the master keys to deliver K_{AB} secretly to both and to authenticate Alice and Bob to the center and to each other. When the conversation ends the session key is discarded. Session keys are kept temporary for four reasons. First, the less ciphertext is produced under one key, the less material a cryptanalyst has. Second, a compromised session key exposes only one session. Third, the master key is then used only to encrypt short random session keys, so it is rarely exposed and can stay valid for a long time. Fourth, the KDC need not store $N(N-1)/2$ pairwise keys, because it creates each one on demand. This two-level arrangement (long-term master keys protecting short-term session keys) is the **key hierarchy**.

The slides describe the flow conceptually: Alice requests a session key with Bob, the KDC informs Bob, and if Bob agrees a session key is created. In the actual protocols below, the KDC does not contact Bob itself; it gives Alice two copies of K_{AB}, one encrypted under K_A and one sealed under K_B in a **ticket**, and Alice forwards the ticket to Bob.

Multiple KDCs. A single KDC for the whole world would be a bottleneck and a single point of failure, and everyone would have to register with it. In **flat multiple KDCs** the world is divided into domains, each with one or more KDCs (more than one for redundancy). If Alice and Bob are in different domains, Alice contacts her KDC, which contacts the KDC of Bob's domain, and the two KDCs together create the session key. In **hierarchical multiple KDCs** the KDCs form a tree, for example local, national and international KDCs. A request from Alice to Bob in another country goes up from Alice's local KDC through her national KDC to an international KDC, and then down through Bob's national KDC to Bob's local KDC.

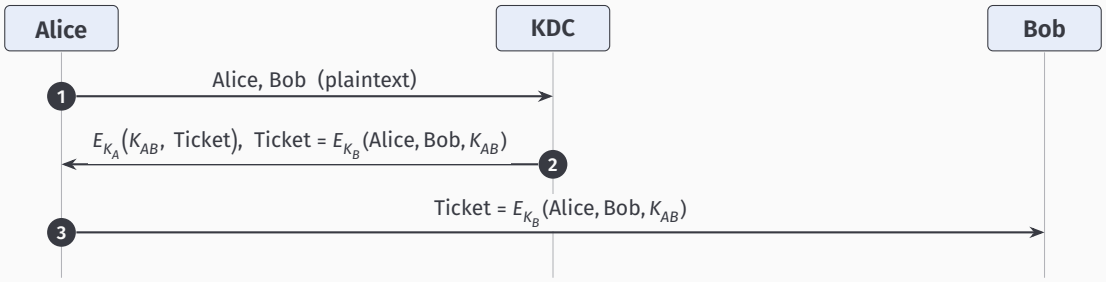


Advantages of the hierarchy. Most master keys are shared between a local KDC and its local users, so the effort of distributing master keys stays local. A faulty or subverted KDC damages only its own subtree. The KDCs themselves need far fewer keys: M KDCs in a flat full mesh need $M(M-1)/2$ inter-KDC keys, but a tree of M KDCs has only $M-1$ edges. **Disadvantage of every KDC scheme.** The KDC knows every session key, so it must be completely trusted, and it must be online for every new session.

38.3 Session-key creation: a simple KDC protocol and its flaw ▸

▸ **Likely this year:** "Show the simple KDC protocol; why is it vulnerable to replay?" (2–3 marks).

The simplest protocol uses three messages.



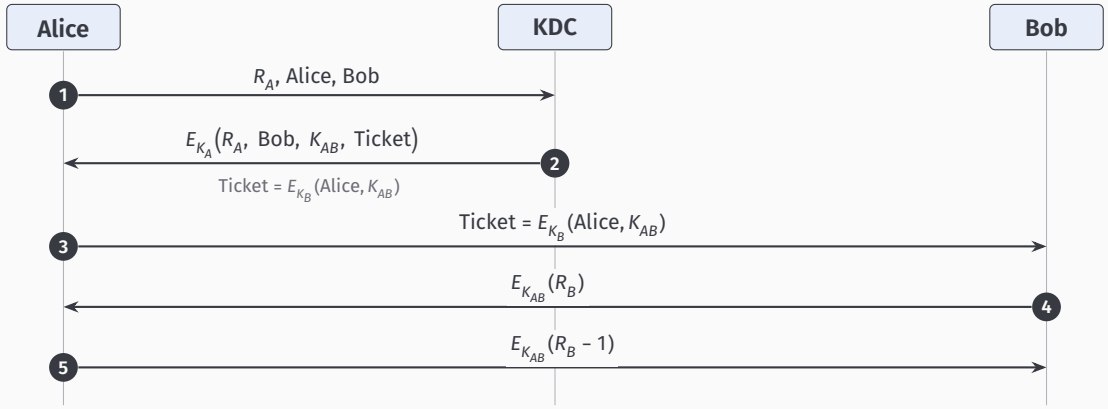
1. Alice sends the KDC a plaintext request containing her registered identity and Bob's identity. Nothing in it is secret.
2. The KDC creates K_{AB} and a **ticket** for Bob, encrypted with K_B and containing both identities and K_{AB}. It sends Alice the ticket together with a copy of K_{AB}, the whole message encrypted with K_A (the ticket is therefore encrypted twice). Only Alice can open the message and extract K_{AB}; she cannot read the ticket, because it is meant for Bob.
3. Alice forwards the ticket to Bob. Only Bob can open it, and since only the KDC could have made it, Bob learns that Alice wants to talk to him using K_{AB}. Alice trusts the KDC, so she is assured that only Bob can recover the key; Bob trusts the KDC, so he is assured that the key belongs to a session with Alice.

The flaw: no freshness. Nothing in the messages shows that they are new. Eve can record message 3 and the session that follows, and later **replay** them; Bob accepts the old ticket and acts on the old messages a second time (for example, a repeated money transfer). Worse, if Eve ever breaks an old session key, she can answer a new request of Alice by replaying the old message 2, and Alice will use a key Eve knows. There is a third weakness: message 1 is not protected, so Eve can replace "Bob" by "Eve"; the KDC then returns a ticket under K_{Eve}, Alice's part does not name the partner, and Alice unknowingly shares K_{AB} with Eve. The next protocol closes all three holes.

38.4 The Needham–Schroeder protocol ▸

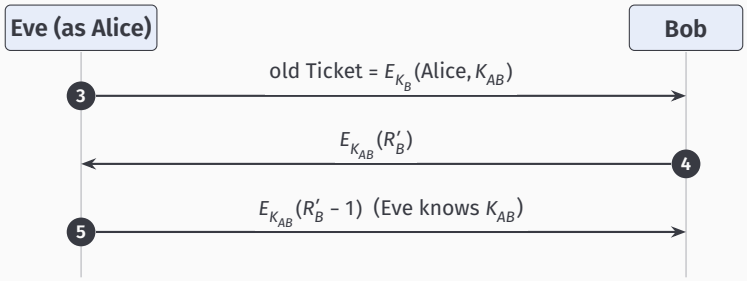
▸ **Likely this year:** "Draw and explain the Needham–Schroeder protocol; what is the purpose of each nonce; describe the Denning–Sacco attack" (5 marks).

Needham and Schroeder (1978) designed the protocol that is the foundation of many later ones, including Kerberos. It uses challenge–response with two nonces, R_A (Alice's challenge to the KDC) and R_B (Bob's challenge to Alice).



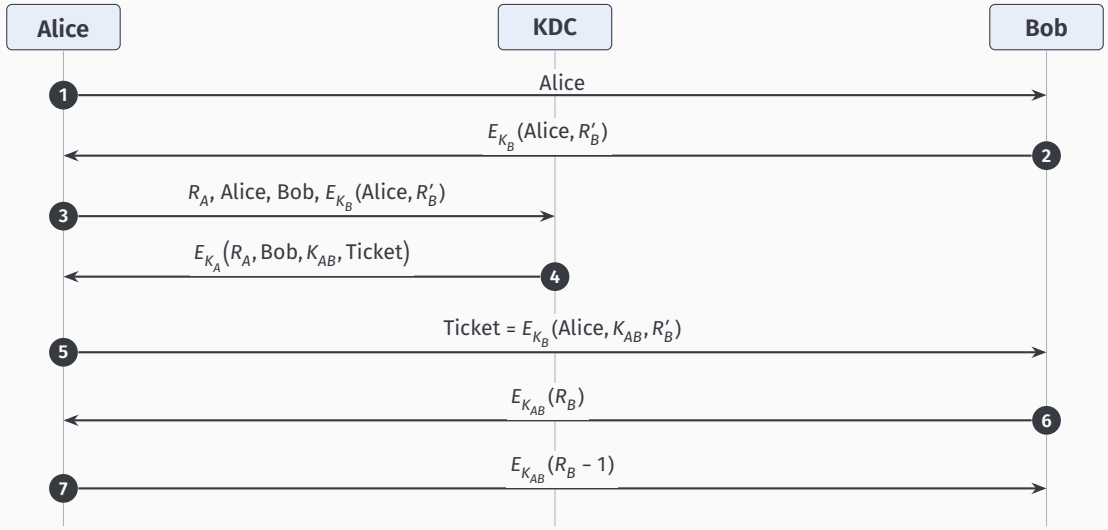
1. **Alice → KDC:** R_A**, Alice, Bob.** The nonce R_A is a fresh challenge to the KDC. The message may travel in plaintext.
2. **KDC → Alice:** $E_{K_A}(R_A,$ **Bob**, $K_{AB},$ **Ticket**). Returning R_A proves to Alice that this reply was made for *this* request, so an old message 2 (perhaps carrying a broken key) cannot be replayed. Including "Bob" lets Alice check that the key is for the partner she asked for, which defeats the substitution of "Eve" in message 1. The whole message is encrypted with K_A, so only Alice learns K_{AB}, and the fields cannot be cut apart and recombined. The ticket E_{K_B}(Alice, K_{AB}) gives Bob the key and tells him with whom he shares it.
3. **Alice → Bob: the ticket.** Bob decrypts it with K_B and obtains K_{AB} and Alice's identity.
4. **Bob → Alice:** $E_{K_{AB}}(R_B)$**.** Bob challenges the sender to prove that it holds K_{AB} now. A replayed ticket from someone who does not know K_{AB} fails here.
5. **Alice → Bob:** $E_{K_{AB}}(R_B - 1)$**.** Alice decrypts R_B, subtracts 1 and re-encrypts. The transformation is essential: if Alice simply returned $E_{K_{AB}}(R_B)$, Eve could reflect message 4 back to Bob without knowing the key. Only a holder of K_{AB} can produce $E_{K_{AB}}(R_B - 1)$.

The Denning–Sacco replay attack (1981). Alice receives a freshness guarantee through R_A, but Bob receives none: the ticket contains nothing chosen by Bob and no time. Suppose Eve recorded message 3 of an old session and has since learned that session's key K_{AB} (by cryptanalysis of the old traffic, or because the key was carelessly disposed of). She replays the old ticket to Bob, decrypts his challenge with the known key, and answers it:



Bob now believes he shares a fresh key with Alice, and Eve can impersonate Alice for as long as she likes. The challenge R_B does not help, because Eve really does know the (old) key. Two fixes exist.

- **Timestamps (Denning–Sacco).** The KDC puts a timestamp T in message 2 and in the ticket, E_{K_B}(Alice, K_{AB}, T). Bob accepts only if $|\text{Clock} - T| < \Delta t_1 + \Delta t_2$, where Δt_1 is the expected difference between the KDC's clock and his own and Δt_2 the expected network delay. An old ticket fails this test. The price is that all clocks must be synchronised; this is the route Kerberos takes.
- **Bob's nonce inside the ticket (revised protocol, 1987).** Bob first gives Alice a nonce sealed under his own key, and the KDC copies it into the ticket, so Bob can see that the ticket was issued after his nonce. This gives the seven-message version below.

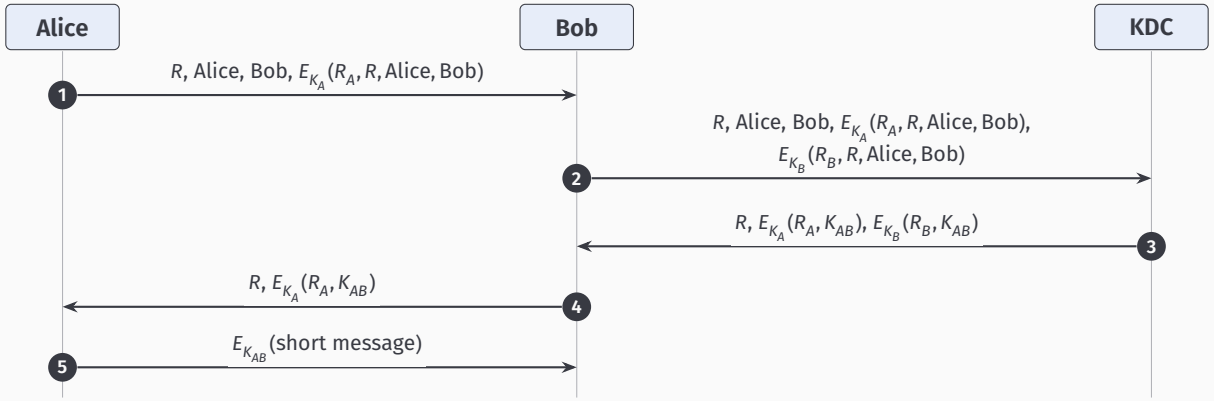


In messages 1–2 Bob issues the nonce R'_B, sealed so that Alice can neither read nor change it. Alice passes it to the KDC in message 3, the KDC places it inside Bob's ticket in message 4, and in message 5 Bob checks that the ticket contains the nonce he issued in this run. An old ticket carries an old nonce and is rejected. Messages 6–7 are the key confirmation of the original protocol.

38.5 The Otway–Rees protocol ▸

▸ **Likely this year:** "Draw the Otway–Rees protocol; what is the role of the common nonce?" (4–5 marks).

Otway and Rees (1987) give both parties their own freshness guarantee and let *Bob*, not Alice, talk to the KDC. A **common nonce** R identifies the run, and each party adds a private nonce (R_A, R_B) inside a part encrypted for the KDC.



1. Alice sends Bob the common nonce R, both identities, and a part for the KDC encrypted with K_A that contains her nonce R_A (her challenge to the KDC), a copy of R and the identities.
2. Bob creates the same kind of part with his own nonce R_B under K_B, and sends both encrypted parts to the KDC. The KDC decrypts them and checks that the R and the identities inside *both* parts agree with the plaintext. This proves that Alice and Bob each asked for a key with the other in this very run, so Eve cannot splice requests from different runs.
3. The KDC creates K_{AB} and returns R with two sealed parts, $E_{K_A}(R_A, K_{AB})$ for Alice and $E_{K_B}(R_B, K_{AB})$ for Bob. Bob opens his part and checks R_B: the key was made in answer to his current request.
4. Bob forwards Alice's part. Alice checks R_A, so she too knows the key is fresh.
5. (Forouzan) Alice sends a short message encrypted with K_{AB} to show that she has the key (key confirmation).

Why this defeats the Denning–Sacco replay. Bob obtains the key from a part that contains his own fresh R_B, so an old message 3 is rejected even if Eve knows the old K_{AB}. No synchronised clocks are needed. A known weakness is a *type-flaw* attack: Eve reflects Alice's encrypted part from message 1 back to her in message 4, and if Alice cannot tell the field types apart she accepts (R, Alice, Bob) as the key; tagging each field with its type stops this.

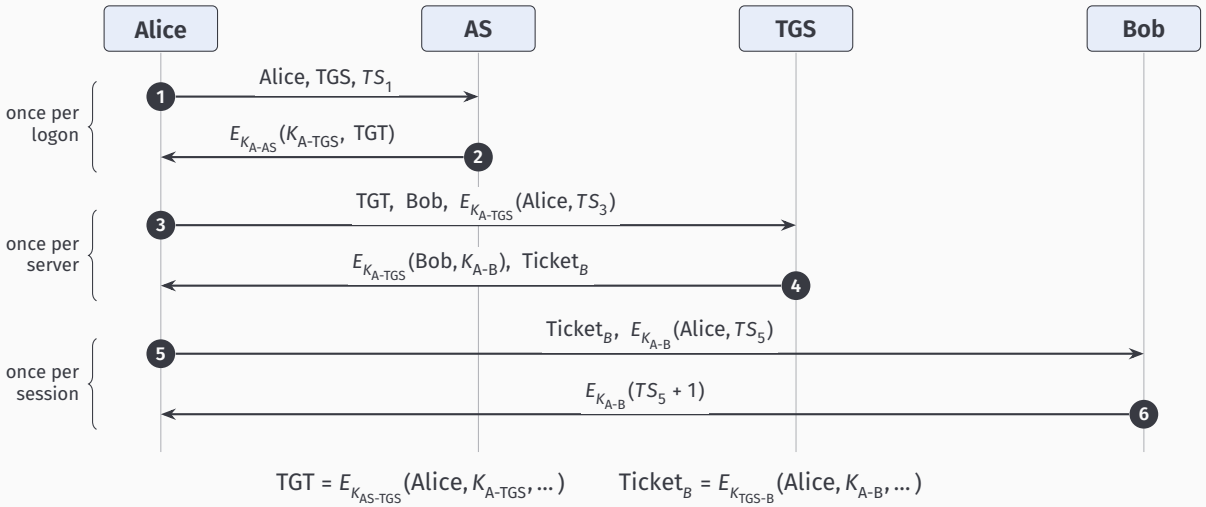
38.6 Kerberos ★▸

▸ **Likely this year:** "Draw and explain Kerberos: AS, TGS, server and the six messages" (8–10 marks); "why is the TGS separate from the AS?"; "differentiate Kerberos v4 and v5".

The problem. In a campus or company network, users sit at workstations and use services on many servers (mail, files, printing). Each server must know who is asking, but the network is open: a user may pretend to be another,

a workstation may forge its network address, and Eve may record and replay traffic. Building password checks into every server would scatter passwords everywhere and send them over the network. **Kerberos**, developed at MIT (Project Athena), is an authentication protocol and at the same time a KDC: a central server authenticates every user once, and then hands out tickets and session keys for the other servers, so that the password is typed once per logon and never travels over the network. Version 4 is described in Forouzan and Stallings; version 5 is the standard in use (for example in Windows domains).

The three servers. The **authentication server (AS)** is the KDC of Kerberos: every user registers with it and it holds a key derived from each user's password. It verifies the user and issues a session key and a ticket for the TGS. The **ticket-granting server (TGS)** issues tickets for the real servers and creates the session key between the user and the server. The **real server** (Bob) provides the service. In the notation below, $K_{A\text{-}AS}$ is Alice's long-term key with the AS, $K_{AS\text{-}TGS}$ and $K_{TGS\text{-}B}$ are long-term keys between servers, and $K_{A\text{-}TGS}$, $K_{A\text{-}B}$ are session keys.



1. **Alice → AS.** Alice sends her request in plaintext, using her registered identity.
2. **AS → Alice.** The AS replies with a message encrypted with Alice's permanent key $K_{A\text{-}AS}$. It contains the session key $K_{A\text{-}TGS}$ for talking to the TGS and a **ticket-granting ticket (TGT)** encrypted with the TGS's key $K_{AS\text{-}TGS}$. Alice does not store $K_{A\text{-}AS}$: when the message arrives she types her password, and the password with a fixed algorithm (a one-way function) produces $K_{A\text{-}AS}$ if the password is correct. The password and $K_{A\text{-}AS}$ are used for this one decryption and then destroyed; the password is never sent over the network and does not stay in the workstation.
3. **Alice → TGS.** Alice sends three items: the TGT, the name of the real server (Bob), and a timestamp encrypted with $K_{A\text{-}TGS}$ (the **authenticator**). The TGS opens the TGT with $K_{AS\text{-}TGS}$, takes $K_{A\text{-}TGS}$ from it, and checks the authenticator. The timestamp prevents a replay by Eve, who cannot make a new one without $K_{A\text{-}TGS}$.
4. **TGS → Alice.** The TGS sends two tickets, each containing the new session key $K_{A\text{-}B}$: Alice's copy is encrypted with $K_{A\text{-}TGS}$, Bob's copy (Ticket_B) with Bob's key $K_{TGS\text{-}B}$. Eve cannot extract $K_{A\text{-}B}$, since she knows neither key. Even if she replays message 3 before its timestamp expires, she only receives the same two tickets, which she cannot decrypt.
5. **Alice → Bob.** Alice sends Bob's ticket with a timestamp encrypted under $K_{A\text{-}B}$.
6. **Bob → Alice.** Bob confirms receipt by adding 1 to the timestamp, encrypted with $K_{A\text{-}B}$. Only the real Bob could open the ticket and read the timestamp, so Alice now knows she is talking to Bob (mutual authentication).

To use another server (say a printer), Alice repeats only messages 3–6 with the same TGT; messages 1–2 are needed once per logon.

Every field of version 4 (Stallings). The diagram shows the core; the full messages carry identities, addresses and lifetimes, each with a job. AD_A is the network address of Alice's workstation, and an authenticator Auth is $E_{\text{session key}}(\text{Alice}, \text{AD}_A, TS)$.

Msg	Contents	Why each field is there
(1)	Alice, TGS, TS_1	Asks for a TGT; TS_1 lets the AS check that Alice's clock is synchronised.
(2)	$E_{K_{A\text{-}AS}}(K_{A\text{-}TGS}, \text{TGS}, TS_2, \text{Lifetime}_2, \text{TGT})$	Only the holder of the password can read it. $K_{A\text{-}TGS}$ is Alice's copy of the session key; "TGS" confirms whom the ticket is for; TS_2 and Lifetime_2 tell her when it expires.
	$\text{TGT} = E_{K_{AS\text{-}TGS}}(K_{A\text{-}TGS}, \text{Alice}, \text{AD}_A, \text{TGS}, TS_2, \text{Lifetime}_2)$	Readable only by the TGS, so Alice cannot alter it. "Alice" names the owner, AD_A ties the ticket to her workstation, and the lifetime bounds how long a stolen ticket is useful.
(3)	Bob, TGT, $\text{Auth} = E_{K_{A\text{-}TGS}}(\text{Alice}, \text{AD}_A, TS_3)$	A TGT can be copied off the network; the authenticator proves that the sender knows $K_{A\text{-}TGS}$ *now*. The TGS checks that the name and address match the TGT and that TS_3 is current.
(4)	$E_{K_{A\text{-}TGS}}(K_{A\text{-}B}, \text{Bob}, TS_4, \text{Ticket}_B)$	Encrypted with the session key, so no password is needed for further servers. "Bob" and TS_4 confirm the server and the issue time.
	$\text{Ticket}_B = E_{K_{TGS\text{-}B}}(K_{A\text{-}B}, \text{Alice}, \text{AD}_A, \text{Bob}, TS_4, \text{Lifetime}_4)$	Readable only by Bob; reusable for repeated access to Bob until it expires.
(5)	Ticket_B, $\text{Auth} = E_{K_{A\text{-}B}}(\text{Alice}, \text{AD}_A, TS_5)$	Bob opens the ticket with $K_{TGS\text{-}B}$, obtains $K_{A\text{-}B}$, and checks the fresh authenticator.
(6)	$E_{K_{A\text{-}B}}(TS_5 + 1)$	Proves Bob's identity (only he could read TS_5); the +1 makes it differ from anything Alice sent, so it cannot be reflected.

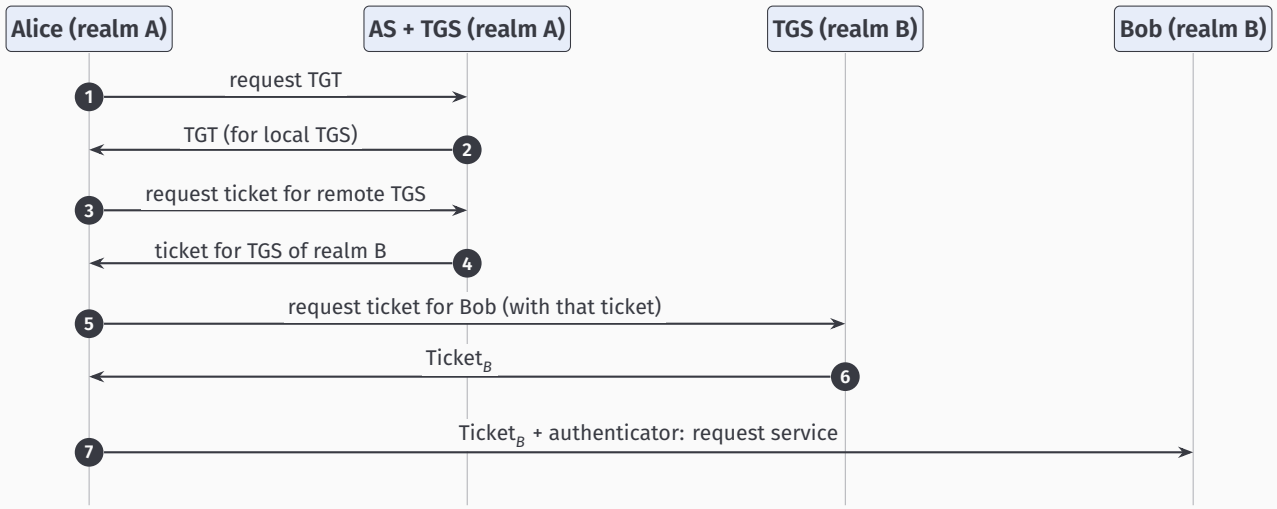
What each key protects.

Key	Shared by	Origin and lifetime	Protects
$K_{A\text{-}AS}$	Alice, AS	long-term, derived from Alice's password	message 2
$K_{AS\text{-}TGS}$	AS, TGS	long-term, set by the administrator	the TGT
$K_{A\text{-}TGS}$	Alice, TGS	session key made by the AS, lives as long as the TGT	authenticator of 3, message 4
$K_{TGS\text{-}B}$	TGS, Bob	long-term, set by the administrator	Ticket_B
$K_{A\text{-}B}$	Alice, Bob	session key made by the TGS	authenticator of 5, message 6, data

Tickets versus authenticators. A **ticket** is made by a Kerberos server, encrypted with the key of the server it is meant for, and can be reused until its lifetime ends. An **authenticator** is made by the client, encrypted with the session key from the ticket, has a very short lifetime and is used once. The ticket delivers the session key securely; the authenticator proves that the person presenting the ticket is its owner, since a ticket alone could have been copied from the network.

Why the TGS is separate from the AS. Alice contacts several servers in one day. If the AS issued server tickets directly, every reply would be encrypted with the password-derived key $K_{A\text{-}AS}$, so Alice would have to type her password for every new server, or the workstation would have to keep $K_{A\text{-}AS}$ in memory all day, exposing the long-term key. Splitting the roles solves this: Kerberos separates user verification (AS, once per logon) from ticket issuing (TGS, once per server). The password is typed once, $K_{A\text{-}AS}$ is used once and destroyed, and all later requests use the TGT and the session key $K_{A\text{-}TGS}$, which expires with the TGT. This is **single sign-on**. A stolen $K_{A\text{-}TGS}$ is useful only until the TGT expires, while the password itself is never exposed again.

Realms and inter-realm authentication. A **realm** is one Kerberos server (AS and TGS) together with the users and application servers registered with it: the server holds every user's password-derived key and shares a secret key with every application server. Organisations with separate administrations run separate realms. For Alice in realm A to use Bob in realm B, the Kerberos servers of the two realms must share a secret key, which makes the TGS of realm B a "server" that the TGS of realm A can issue tickets for.



In version 4, N realms that all trust each other need $N(N-1)/2$ inter-realm keys (45 for 10 realms), the N^2 problem again at the level of realms; version 5 allows a hierarchy of realms with fewer keys.

Kerberos version 4 versus version 5. Version 5 (RFC 4120) removes environmental limits and technical flaws of version 4.

Aspect	Version 4	Version 5
Encryption	DES only	any algorithm; ciphertext tagged with type and key length
Network addresses	IP addresses only	any type, tagged with type and length
Message encoding	sender's byte order, with a flag	ASN.1 with Basic Encoding Rules (unambiguous)
Ticket lifetime	8-bit field in units of 5 min, about 21 h maximum	explicit start and end times; tickets can be renewed
Forwarding credentials	not allowed	allowed (a server can act for the client)
Inter-realm	$N(N-1)/2$ realm-to-realm keys	hierarchy of realms, fewer keys
Double encryption	tickets encrypted a second time in messages 2 and 4	removed
Mode of encryption	PCBC (non-standard, weak)	CBC with an explicit integrity check
Password guessing	anyone can request message 2	pre-authentication (encrypted timestamp first)
Overhead	smaller	larger

Weaknesses of Kerberos.

- **Clock synchronisation.** Freshness rests on timestamps, so all clocks must agree within a few minutes. Within that window a captured authenticator can be replayed unless servers keep a cache of recently seen authenticators, and an attacker who can change a server's clock can make old tickets look valid.
- **Password guessing.** Message 2 is encrypted with a key derived from the password. In version 4 anyone can ask the AS for Alice's message 2 and then try dictionary words offline: a guess is right when the decryption gives meaningful fields (the name of the TGS). Version 5 pre-authentication stops the free requests, but an eavesdropped exchange can still be attacked offline, so strong passwords remain essential.
- **Single point of failure and attack.** The Kerberos server holds every user's key. If it is down, nobody can log in; if it is compromised, every key is exposed. It is therefore replicated and physically protected.
- **Workstation compromise.** Tickets and session keys are cached on the workstation; malware there can steal a TGT and use it until it expires.

★ MUST KNOW : Kerberos in six messages

(1) A → AS: Alice, TGS. (2) AS → A: $E_{K_{\text{A-AS}}}(K_{\text{A-TGS}}, \text{TGT})$. (3) A → TGS: TGT, Bob, $E_{K_{\text{A-TGS}}}(T)$. (4) TGS → A: $E_{K_{\text{A-TGS}}}(\text{Bob}, K_{\text{A-B}})$, $E_{K_{\text{TGS-B}}}(\text{Alice}, K_{\text{A-B}})$. (5) A → B: $E_{K_{\text{TGS-B}}}(\text{Alice}, K_{\text{A-B}})$, $E_{K_{\text{A-B}}}(T)$. (6) B → A: $E_{K_{\text{A-B}}}(T+1)$.
AS verifies the user once (password → $K_{\text{A-AS}}$), TGS issues server tickets (single sign-on), timestamps stop replays.

38.7 Comparing the KDC-based protocols ▸

▸ **Likely this year:** "Compare Needham–Schroeder, Otway–Rees and Kerberos" (3 marks).

	Needham–Schroeder	Otway–Rees	Kerberos (v4)
Messages	5 (7 in the revised version)	4 (+1 key confirmation)	6 (2 once per logon)
Who talks to the KDC	Alice	Bob, carrying both requests	Alice (AS, then TGS)
Freshness by	nonces R_A, R_B	nonces R_A, R_B and common nonce R	timestamps and lifetimes
Replay of an old ticket	possible (Denning–Sacco)	prevented by R_B	prevented by timestamp
Synchronised clocks	not needed	not needed	needed
Proof of Bob's identity	message 4 (only Bob can read the ticket)	implicit (only Alice confirms the key)	explicit, message 6
Single sign-on	no	no	yes, through the TGS

38.8 Symmetric-key agreement: Diffie–Hellman ★▸

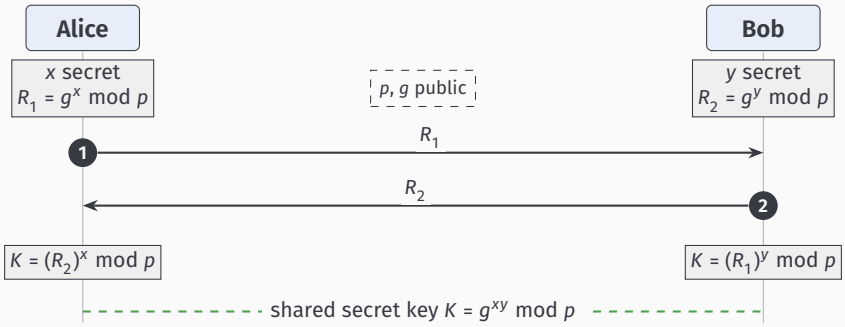
▸ **Likely this year:** "Explain Diffie–Hellman key exchange; prove that both parties get the same key; compute R_1, R_2, K for given p, g, x, y" (5–6 marks).

A KDC needs a trusted third party that is online for every session and a master key registered in advance. In **symmetric-key agreement**, Alice and Bob create a session key between themselves, over a public channel, without a KDC and without any earlier shared secret. The slides discuss two methods: Diffie–Hellman and station-to-station.

The idea. Diffie and Hellman (1976) combined two facts. Modular exponentiation is a one-way function: $g^x \bmod p$ is fast to compute (square-and-multiply, §22), but recovering x from it is the discrete logarithm problem (§27), infeasible for a large prime. And exponents commute: $(g^x)^y = (g^y)^x = g^{xy}$. Each party therefore keeps an exponent secret, publishes g raised to it, and raises the other's public value to its own secret exponent.

Setup. Before establishing a key, the two parties choose two numbers p and g. p is a large prime, on the order of 300 decimal digits (1024 bits; 2048 bits today). g is a generator of order $p - 1$ in the group $\langle \mathbb{Z}_p^*, \times \rangle$, that is, a primitive root of p (§26), so that its powers run through all of $\mathbb{Z}_p^*$. Neither number needs to be secret: they can be sent over the Internet and shared by many users.

1. Alice chooses a large random number x with $0 < x < p - 1$ and calculates $R_1 = g^x \bmod p$.
2. Bob chooses another large random number y with $0 < y < p - 1$ and calculates $R_2 = g^y \bmod p$.
3. Alice sends R_1 to Bob. She does not send x.
4. Bob sends R_2 to Alice. He does not send y.
5. Alice calculates $K = (R_2)^x \bmod p$.
6. Bob calculates $K = (R_1)^y \bmod p$.



Proof that both obtain the same key. Reducing modulo p before raising to a power does not change the result modulo p, because $a \equiv b \pmod p$ implies $a^k \equiv b^k \pmod p$. Hence

$$\text{Alice: } K = (R_2)^x \bmod p = (g^y \bmod p)^x \bmod p = g^{yx} \bmod p,$$
$$\text{Bob: } K = (R_1)^y \bmod p = (g^x \bmod p)^y \bmod p = g^{xy} \bmod p,$$

and $g^{yx} = g^{xy}$. Both reach the same value without Bob knowing x and without Alice knowing y: the symmetric (shared) key in the Diffie–Hellman method is $K = g^{xy} \bmod p$.

What Eve faces. Eve sees p, g, $R_1 = g^x$ and $R_2 = g^y$. Computing g^{xy} from these is the **(computational) Diffie–Hellman problem**; the only known general method is to solve a discrete logarithm, finding x from R_1 (or y from R_2), which is infeasible for a properly chosen 1024-bit or larger p. Note that Diffie–Hellman encrypts nothing: it is a key *agreement*, in which both parties contribute to the key, and the resulting K (usually hashed) then keys a symmetric cipher such as AES.

★ MUST KNOW : Diffie–Hellman

Public: prime p, generator g of $\mathbb{Z}_p^*$. Alice: $R_1 = g^x \bmod p$. Bob: $R_2 = g^y \bmod p$. $K = (R_2)^x \bmod p = (R_1)^y \bmod p = g^{xy} \bmod p$. Security rests on the discrete logarithm; no authentication, so man-in-the-middle is possible.

WORKED EXAMPLE : the slide example (Forouzan)

Q. Alice and Bob use Diffie–Hellman with $g = 7$ and $p = 23$. Alice chooses $x = 3$ and Bob chooses $y = 6$. Find the values exchanged and the shared key.
A. Powers of 7 modulo 23 (each line from the ones above):

Power	Computation	mod 23
7^1		7
7^2	$49 = 2 \cdot 23 + 3$	3
7^3	$7^2 \cdot 7 = 3 \cdot 7$	21
7^4	$(7^2)^2 = 3^2$	9
7^6	$7^4 \cdot 7^2 = 9 \cdot 3 = 27 = 23 + 4$	4

$R_1 = 7^3 \bmod 23 =$ **21** (sent to Bob), $R_2 = 7^6 \bmod 23 =$ **4** (sent to Alice).
Alice: $K = 4^3 \bmod 23 = 64 \bmod 23 = 64 - 46 =$ **18**.
Bob: $K = 21^6 \bmod 23$; $21^2 = 441 = 19 \cdot 23 + 4 \equiv 4$, $21^4 \equiv 4^2 = 16$, $21^6 = 21^4 \cdot 21^2 \equiv 16 \cdot 4 = 64 \equiv$ **18**.

Check: $g^{xy} = 7^{18} \bmod 23 = (7^6)^3 = 4^3 \equiv 18$ ✓. (By Fermat, $7^{22} \equiv 1$, so $7^{18} \equiv 7^{-4} = 9^{-1} \bmod 23 = 18$, since $9 \cdot 18 = 162 = 7 \cdot 23 + 1$.) The modulus is 23; the slide's "$7^{18} \bmod 35$" is a typo.

WORKED EXAMPLE : new, Stallings' example with full square-and-multiply

Q. Users A and B use the Diffie–Hellman key exchange with the common prime $q = 353$ and primitive root $\alpha = 3$. A's private key is $X_A = 97$ and B's is $X_B = 233$. Compute the public keys Y_A, Y_B and the common secret key, showing both sides.

A. In Stallings' notation $Y_A = \alpha^{X_A} \bmod q$ (our R_1), $Y_B = \alpha^{X_B} \bmod q$ (our R_2) and $K = Y_B^{X_A} = Y_A^{X_B} \bmod q$. The exponents in binary: $97 = 1100001_2 = 64 + 32 + 1$ and $233 = 11101001_2 = 128 + 64 + 32 + 8 + 1$. Squaring table modulo 353 (each entry is the square of the one above it):

Power	base 3	base 248	base 40
b^1	3	248	40
b^2	9	61504 → 82	1600 → 188
b^4	81	6724 → 17	35344 → 44
b^8	6561 → 207	289 → 289	1936 → 171
b^{16}	42849 → 136	83521 → 213	29241 → 295
b^{32}	18496 → 140	45369 → 185	87025 → 187
b^{64}	19600 → 185	34225 → 337	34969 → 22
b^{128}	34225 → 337	(not needed)	484 → 131

("6561 → 207" means $81^2 = 6561 = 18 \cdot 353 + 207$.) Multiplying the powers selected by the 1-bits, reducing after each product:
$Y_A = 3^{97} = 3^{64} \cdot 3^{32} \cdot 3$: $185 \cdot 140 = 25900 = 73 \cdot 353 + 131$; $131 \cdot 3 = 393 \equiv$ **40**.
$Y_B = 3^{233} = 3^{128} \cdot 3^{64} \cdot 3^{32} \cdot 3^8 \cdot 3$: $337 \cdot 185 = 62345 \equiv 217$; $217 \cdot 140 = 30380 \equiv 22$; $22 \cdot 207 = 4554 \equiv 318$; $318 \cdot 3 = 954 \equiv$ **248**.
A computes $K = 248^{97} = 248^{64} \cdot 248^{32} \cdot 248$: $337 \cdot 185 \equiv 217$; $217 \cdot 248 = 53816 = 152 \cdot 353 + 160 \equiv$ **160**.
B computes $K = 40^{233} = 40^{128} \cdot 40^{64} \cdot 40^{32} \cdot 40^8 \cdot 40$: $131 \cdot 22 = 2882 \equiv 58$; $58 \cdot 187 = 10846 \equiv 256$; $256 \cdot 171 = 43776 \equiv 4$; $4 \cdot 40 =$ **160**.
Check: both sides give $K = 160$ ✓. Eve sees 40 and 248; to get 160 she must find 97 or 233 by a discrete logarithm, which for $q = 353$ takes at most 352 trials but for a 2048-bit q is infeasible.

Trap: Diffie–Hellman slips

The key is $g^{xy} \bmod p$, not g^{x+y} and not $R_1 \cdot R_2$. Each party raises the *other*'s public value to its *own* secret exponent. x and y are never sent. Reduce modulo p (not $p - 1$) after every multiplication; exponents may be reduced modulo $p - 1$ (Fermat).

38.9 Security of Diffie–Hellman: discrete logarithm and man-in-the-middle ★▸

▸ **Likely this year:** "Explain the man-in-the-middle (bucket brigade) attack on Diffie–Hellman with a diagram; compute the two keys Eve shares" (5 marks); "what are the requirements on p and g?".

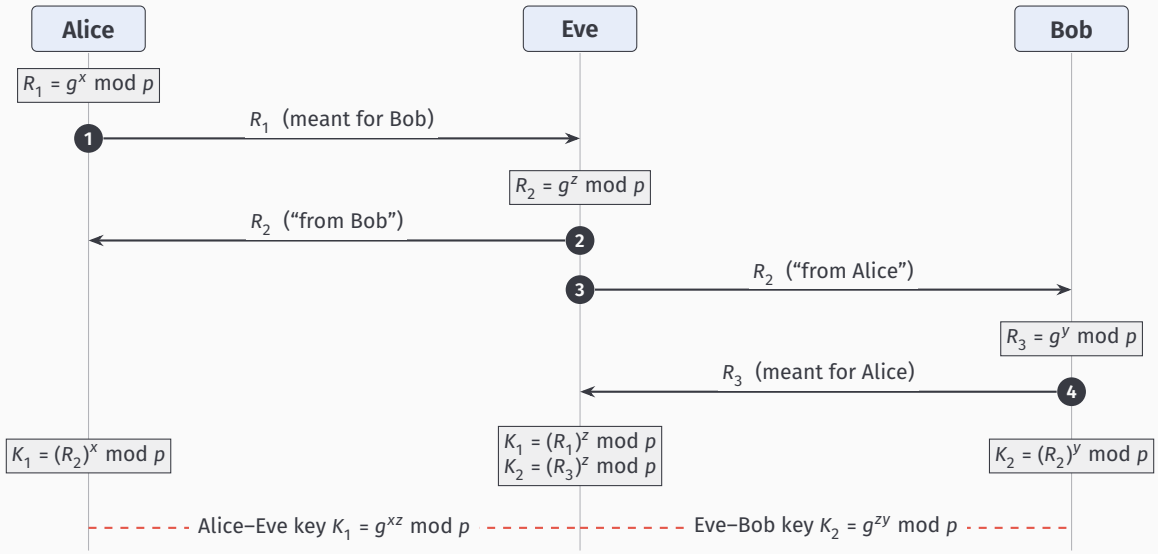
The slides list two attacks: the discrete logarithm attack, which breaks the mathematics, and the man-in-the-middle attack, which does not need to.

Discrete logarithm attack. Eve intercepts R_1 and R_2. If she can find x from $R_1 = g^x \bmod p$ (or y from R_2), she computes $K = R_2^x \bmod p$ exactly as Alice does. With the slide numbers this is trivial: trying $7^1 = 7$, $7^2 \equiv 3$, $7^3 \equiv 21 = R_1$ gives $x = 3$ and $K = 4^3 \bmod 23 = 18$. Forouzan's recommendations make the attack infeasible, each for a reason:

1. **The prime p must be very large** (more than 300 decimal digits), so that exhaustive search and the faster index-calculus algorithms are out of reach.
2. $p - 1$ **must have at least one large prime factor** (more than 60 decimal digits). The Pohlig–Hellman algorithm splits a discrete logarithm modulo p into separate logarithms in the subgroups whose orders are the prime-power factors of $p - 1$ and joins the answers with the CRT, so its cost is set by the largest prime factor. A *safe prime* $p = 2q + 1$ with q prime is the usual choice.
3. **The generator must be chosen from the group** $\langle \mathbb{Z}_p^*, \times \rangle$ with order $p - 1$ (or with a large prime order). If g has a small order d, then R_1 and K take only d values. For example, 2 has order 11 modulo 23 ($2^{11} = 2048 = 89 \cdot 23 + 1$), so with $g = 2$ only 11 keys are possible. For the same reason a party should reject a received value of 1 or $p - 1$, which would force the key into $\{1, p - 1\}$.
4. **Alice and Bob must destroy x and y after computing K, and use them only once.** If x were kept and later stolen, every recorded R_2 would reveal an old key; if it were reused, all sessions would share related keys. Fresh exponents give *forward secrecy*: breaking into a machine later reveals nothing about past sessions.

Man-in-the-middle attack. The protocol has a deeper weakness: Eve does not have to find x or y. She can fool Alice and Bob by creating two keys, one between herself and Alice and another between herself and Bob.

1. Alice chooses x, calculates $R_1 = g^x \bmod p$, and sends R_1 to Bob.
2. Eve intercepts R_1. She chooses z, calculates $R_2 = g^z \bmod p$, and sends R_2 to both Alice and Bob.
3. Bob chooses y, calculates $R_3 = g^y \bmod p$, and sends R_3 to Alice. Eve intercepts R_3, and it never reaches Alice.
4. Alice and Eve calculate $K_1 = g^{xz} \bmod p$, which becomes a key shared between Alice and Eve. Alice, however, thinks that it is a key shared between Bob and herself.
5. Eve and Bob calculate $K_2 = g^{zy} \bmod p$, which becomes a key shared between Eve and Bob. Bob, however, thinks that it is a key shared between Alice and himself.



Why each party computes Eve's key. Alice raises the value she received, $R_2 = g^z$, to her x: $K_1 = (g^z)^x = g^{xz}$. Eve raises Alice's $R_1 = g^x$ to her z: $(g^x)^z = g^{xz} = K_1$. In the same way Bob gets $(g^z)^y = g^{zy}$ and Eve gets $(g^y)^z = g^{zy} = K_2$. From now on Eve decrypts every message from Alice with K_1, reads or changes it, re-encrypts it with K_2 and forwards it to Bob, and the reverse for Bob's replies. Neither victim notices anything, because each has completed a perfectly valid Diffie–Hellman exchange, only with the wrong party. The attack is called man-in-the-middle because Eve comes in between and intercepts R_1, sent by Alice to Bob, and R_3, sent by Bob to Alice. It is also known as a **bucket brigade attack** because it resembles a short line of volunteers passing a bucket of water from person to person.

WORKED EXAMPLE : man-in-the-middle with the slide numbers

Q. In the exchange $g = 7$, $p = 23$, $x = 3$, $y = 6$, Eve performs a man-in-the-middle attack with $z = 9$. Find the value Eve sends, the key Eve shares with each party, and compare with the honest key.
A. $R_1 = 21$ and $R_3 = 7^6 \bmod 23 = 4$ as before. Eve: $7^8 = (7^4)^2 = 81 = 3 \cdot 23 + 12 \equiv 12$, so $R_2 = 7^9 = 12 \cdot 7 = 84 = 3 \cdot 23 + 15 \equiv$ **15**.

Key	Victim computes	Eve computes
K_1 (Alice–Eve)	15^3: $15^2 = 225 \equiv 18$, $18 \cdot 15 = 270 = 11 \cdot 23 + 17 \Rightarrow$ **17**	$21^9 \equiv (-2)^9 = -512 \equiv -6 \equiv$ **17**
K_2 (Eve–Bob)	$15^6 = (15^3)^2 = 17^2 = 289 = 12 \cdot 23 + 13 \Rightarrow$ **13**	$4^9 = 2^{18} \equiv 2^7 = 128 \equiv$ **13** ($2^{11} \equiv 1$)

Alice uses $K_1 = 17$ and Bob uses $K_2 = 13$, each believing it is the shared key; the honest key would have been 18.
Check: $K_1 = 7^{xz} = 7^{27} \equiv 7^5 \equiv 17$ and $K_2 = 7^{zy} = 7^{54} \equiv 7^{10} \equiv 13$ (exponents reduced mod 22) ✓.

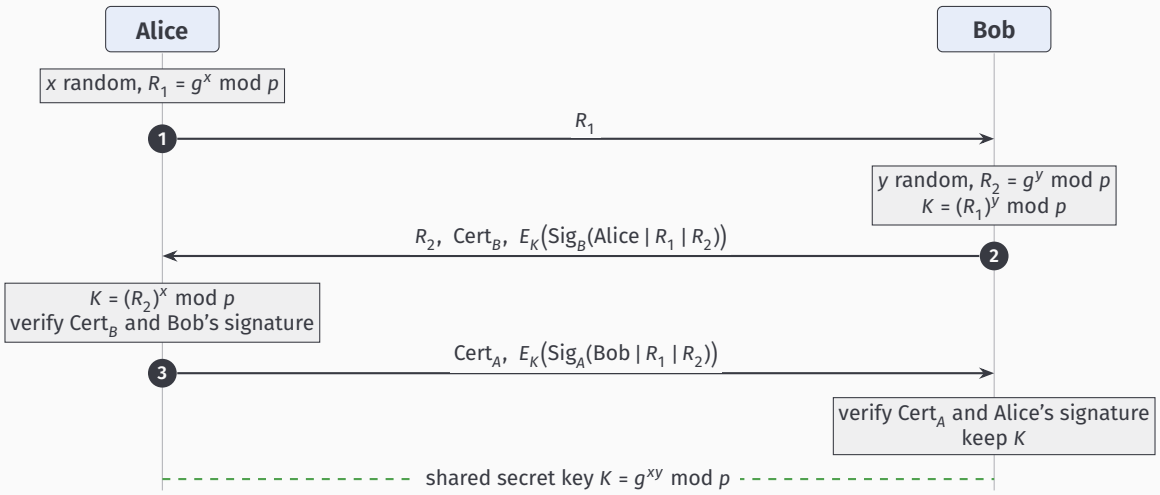
Why Diffie–Hellman alone gives no authentication. The values R_1 and R_2 carry nothing that ties them to Alice or Bob: any number in $\mathbb{Z}_p^*$ looks like a valid public value, and Eve can produce one as easily as Bob. Diffie–Hellman therefore guarantees only that Alice shares K with *whoever sent the value she received*, not who that is. The cure is to authenticate the exchanged values, by digital signatures with certificates (station-to-station, below), by certified long-term Diffie–Hellman values, or by a MAC under a key or password the two already share.

38.10 Station-to-station protocol ▸

▸ **Likely this year:** "Explain the station-to-station protocol and show how it prevents the man-in-the-middle attack" (4–5 marks).

The **station-to-station (STS)** protocol (Diffie, van Oorschot and Wiener, 1992) is a method based on Diffie–Hellman. It uses digital signatures with public-key certificates to establish a session key between Alice and Bob while

authenticating both of them. Each party has a signature key pair and a certificate, issued by a trusted certification authority (§39), that binds its identity to its public verification key.



The slide's figure numbers nine steps; they map onto the three messages as follows.

- **Message 1 (steps 1–2).** After calculating R_1, Alice sends R_1 to Bob, exactly as in Diffie–Hellman.
- **Message 2 (steps 3–5).** After calculating R_2 and the session key K, Bob concatenates Alice's ID, R_1 and R_2, and signs the result with his private key. He sends R_2, the signature and his own public-key certificate to Alice; the signature is encrypted with the session key.
- **Message 3 (steps 6–8).** After calculating the session key, Alice verifies Bob's certificate with the CA's public key, decrypts the signature with K and verifies it with Bob's public key from the certificate. If it is valid, she concatenates Bob's ID, R_1 and R_2, signs the result with her own private key, encrypts the signature with K and sends it with her certificate.
- **Step 9.** If Alice's signature is verified, Bob keeps the session key.

Why each field is there. The signature covers *both* Diffie–Hellman values, so it binds this particular exchange to the signer: a signature from another run (with other R values) is useless to Eve. The signature also names the *intended partner* (Bob signs "Alice", Alice signs "Bob"), so Eve cannot take Bob's signed statement and present it in a session with a third party. Encrypting the signature with K proves that the sender has actually computed K (key confirmation) and hides the identity proof from eavesdroppers. The certificates let each side obtain the other's genuine public key without an earlier meeting, and they are trusted because they are issued by trusted authorities.

Derivation: why the man-in-the-middle attack fails. Suppose Eve intercepts R_1 and sends Bob her own $R_E = g^z$. Bob computes $K_2 = g^{zy}$ and signs (Alice | R_E | R_2). Eve now has only three options towards Alice, and each fails.

- **Forward Bob's message 2 unchanged.** Alice sent R_1, so she computes $K = R_2^x = g^{xy} \neq K_2$; the decryption gives garbage, and even a correctly decrypted signature covers R_E, not Alice's R_1, so the check against (Alice | R_1 | R_2) fails.
- **Substitute her own value $R'_E = g^{z'}$ for R_2.** Alice then needs Bob's signature on (Alice | R_1 | R'_E). Bob never signed this string, and Eve cannot produce the signature without Bob's private key; a signature with Eve's own key fails verification with the public key in Bob's certificate, and she cannot forge a certificate binding her key to Bob's name without the CA's private key.
- **Leave R_1 and R_2 untouched.** Then Alice and Bob share $K = g^{xy}$, which Eve cannot compute (Diffie–Hellman problem), so she is reduced to a passive eavesdropper.

The same argument applies to Alice's signature in message 3, which Eve cannot forge either. Hence the station-to-station protocol prevents man-in-the-middle attacks: whatever Eve does, she is either detected or learns nothing.

	Diffie–Hellman	Station-to-station
Messages	2 (R_1, R_2)	3 (R_1; R_2 + certificate + signature; certificate + signature)
Authentication	none	mutual, by signatures and certificates
Man-in-the-middle	possible	prevented
Key confirmation	none	yes (signatures encrypted under K)
Infrastructure	only public p, g	signature key pairs and a CA (PKI)
Cost per party	2 exponentiations	2 exponentiations + 1 signature + 2 verifications (certificate, signature)

39 Public-Key Distribution, X.509 and PKI

39.1 The public-key distribution problem; public announcement ▸

▸ **Likely this year:** "Explain the methods of public-key distribution: announcement, trusted center, controlled trusted center, CA" (4–5 marks).

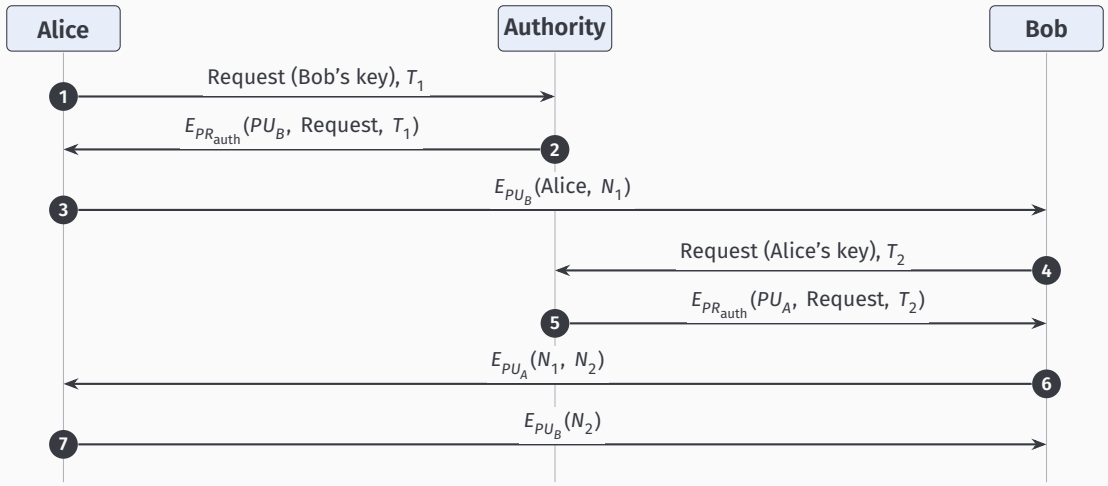
In asymmetric-key cryptography the private key never leaves its owner and the public key is meant to be known, so secrecy is not the problem. **Authenticity** is: Alice must be sure that the key she holds as "Bob's" really belongs to Bob. If Eve can make Alice accept Eve's public key as Bob's, Alice encrypts for Eve, who reads the message and re-encrypts it under Bob's real key (a man-in-the-middle), and Alice also accepts Eve's signatures as Bob's. Forouzan describes four ways of distributing public keys, each closing a hole of the previous one.

Public announcement. Bob simply announces his public key: on his website, in a newspaper, on a mailing list or at the end of his e-mails. This is simple and needs no third party. It is not secure, because the announcement itself is not authenticated: Eve can announce her own key as Bob's (**forgery**), and until Bob notices, Eve reads every message sent to "Bob" and signs in his name. There is also no reliable way to withdraw a key once it has been announced.

39.2 Trusted center and controlled trusted center ▸

Trusted center. A higher level of security is obtained with a trusted center that keeps a **directory** of public keys, a table of (name, public key) entries. Each user selects a private and public key, keeps the private key and delivers the public key to the center in a secure way, for example in person, so that the center can check the identity. The directory is published, and users can look keys up. Users may replace their keys, for example after a compromise. The weak point is the response: if the lookup reply travels unprotected, Eve can intercept it and substitute her own key.

Controlled trusted center (public-key authority). Here the distribution itself is controlled: the center's replies include a timestamp and are signed with the center's private key, whose public counterpart every user knows. Stallings gives the full scenario in seven messages (PU = public key, PR = private key, N_1, N_2 nonces); messages 1–2 are Forouzan's controlled trusted center.



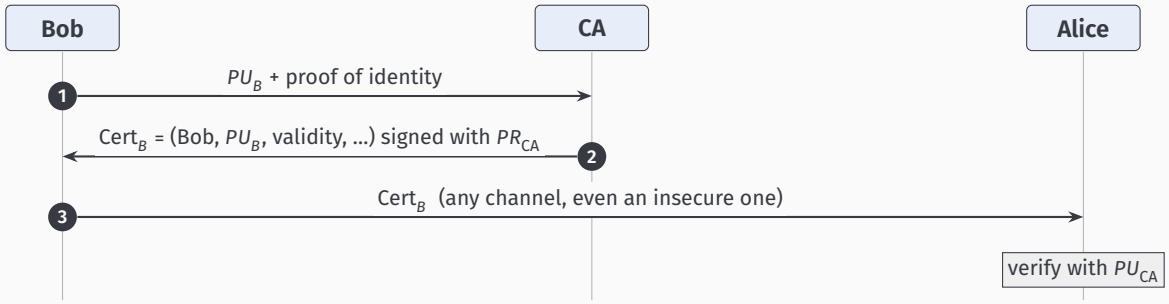
In message 2 the **signature** (encryption with PR_{auth}) lets Alice verify with the authority's public key that the reply really comes from the authority and was not altered. The copy of the **request** lets her check that the reply answers her question, so Eve cannot change "Bob" into "Eve" on the way. The **timestamp** T_1 shows that the reply is fresh, so Eve cannot replay an old reply carrying an old (perhaps revoked) key of Bob. Messages 3–7 then authenticate the two parties: only Bob can read N_1 in message 3 and return it in message 6, and only Alice can read N_2 in message 6 and return it in message 7. (This is the Needham–Schroeder public-key protocol; Lowe showed in 1995 that message 6 should also contain Bob's identity.) **Disadvantage.** The authority must be online for every request, so it is a bottleneck and an attractive target, and its directory must be protected.

39.3 Certification authority and certificates ▸

▸ **Likely this year:** "What is a public-key certificate? Why is it signed by the CA? How does it solve the problems of announcement and the trusted center?" (3–4 marks).

The idea is to let the authority sign the binding between a name and a key *once, in advance*, instead of answering every request online. A certificate binds an identity to a public key and is signed with the private key of a certification authority (CA). Bob goes to the CA with his public key and proof of identity (for example a passport); the CA checks the identity and that Bob holds the matching private key (Bob signs the request with it). The CA then writes a certificate containing Bob's identity, his public key, a validity period, a serial number and the CA's name,

and signs it with the CA's private key. Bob can now publish the certificate anywhere or send it inside a protocol (as in station-to-station). Alice verifies it with the CA's public key, which she obtained securely in advance (for example pre-installed in the browser or operating system); if the signature is valid, the key in the certificate is Bob's.



Why the certificate is signed by the CA. The signature is computed over all the fields, so changing the name or the key breaks it; and producing a valid signature requires the CA's private key, so Eve cannot make a certificate that binds her key to Bob's name. A certificate is therefore *self-protecting*: it can be sent over insecure channels, cached and copied, even delivered by Eve, without losing its value. This removes both earlier weaknesses: a forged announcement fails the signature check, and the CA need not be online, because the certificate was made in advance. Alice needs only *one* key obtained securely, the CA's, instead of the key of every correspondent. The CA never sees private keys, so unlike a KDC it cannot read anyone's traffic; a dishonest or compromised CA can, however, issue false certificates, which is why CAs are carefully audited.

Stallings states the requirements: any participant can read a certificate to learn the owner's name and public key; any participant can verify that it came from the CA and is not counterfeit; only the CA can create and update certificates; and any participant can check that it is still current (validity period and revocation, below).

39.4 X.509 certificates ★▸

▸ **Likely this year:** "Draw and explain the X.509 certificate format" (5 marks); "what does $X\langle\langle A\rangle\rangle$ mean".

Certificates from different CAs are only useful if everyone parses them the same way. **X.509** is the ITU-T recommendation (part of the X.500 directory series, first issued in 1988, version 3 in 1996) that defines the certificate structure, written in ASN.1. It is used by IPSec, SSL/TLS, S/MIME and the Internet PKI.

Field	Meaning and purpose
Version number	Version of X.509 used. Numbering starts at 0, so the current version 3 is coded as 2; it tells the reader which fields to expect.
Serial number	Unique identifier of the certificate among those issued by this CA; it is what a revocation list names.
Signature algorithm ID	Algorithm (and parameters) used by the CA to sign the certificate, e.g. SHA-256 with RSA.
Issuer name	X.500 name of the CA that issued the certificate: a hierarchy of strings (country, state, organisation, department, ...).
Validity period	Earliest ("not before") and latest ("not after") time at which the certificate is valid.
Subject name	X.500 name of the entity the public key belongs to, again a hierarchy of strings.
Subject public key info	The owner's public key, the heart of the certificate, with its algorithm (RSA, DSS, ...) and parameters.
Issuer unique identifier	Optional (v2): distinguishes two issuers with the same name.
Subject unique identifier	Optional (v2): distinguishes two subjects with the same name.
Extensions	Optional (v3): extra information, each with an identifier, a criticality flag and a value (see below).
Signature	Three parts: all the other fields; their digest encrypted with the CA's *private* key; the identifier of the algorithm used.

Verification. Alice hashes the to-be-signed fields, verifies the signature with the issuer's public key, compares, checks that the current date lies within the validity period and that the serial number is not revoked, and only then uses the subject public key.

Versions and extensions. Version 1 has the basic fields. Version 2 added the two unique identifiers, so that names could be reused. Version 3 added extensions, because experience showed that more information was needed. Stallings groups them into three classes: *key and policy information* (key identifiers, key usage such as "signature only" or "key encipherment", certificate policies), *subject and issuer attributes* (alternative names such as an e-mail address or DNS name), and *certification path constraints* (whether the subject may itself act as a CA and how long a chain below it may be). A reader that does not understand a *critical* extension must reject the certificate.

Notation. Stallings writes $Y\langle\langle X\rangle\rangle$ for the certificate of user X issued by CA Y, and $Y\{I\}$ for the signing of I by Y (that is,

I with an encrypted hash appended). So $CA\langle\langle A\rangle\rangle = CA\{V, SN, AI, CA, UCA, A, UA, A_p, T^A\}$, listing version, serial number, algorithm, issuer, issuer ID, subject, subject ID, subject public key and validity period.

Trap: who signs with what

A certificate is signed with the *CA's private* key and verified with the *CA's public* key. It contains the *subject's public* key, never a private key. "Issuer" is the CA, "subject" is the owner of the key.

39.5 Certificate renewal and revocation ▸

▸ **Likely this year:** "Why are certificates revoked? Explain the CRL format and delta CRL" (3–4 marks).

Renewal. Each certificate has a validity period. If there is no problem with it, the CA issues a new certificate before the old one expires, much as a bank renews a credit card; the new certificate has a new serial number and validity period and may carry a new key pair (key update).

Revocation. Sometimes a certificate must be cancelled before it expires. Forouzan gives three reasons: the subject's private key may have been compromised; the CA may no longer be willing to certify the subject (for example, the certificate relates to an organisation the subject has left); or the CA's own private key may have been compromised, in which case the CA must revoke all its unexpired certificates. Revocation is done by periodically publishing a **certificate revocation list (CRL)**, a list of revoked certificates signed by the CA. When a user wants to use a certificate, she first checks the CA's directory for the latest CRL.

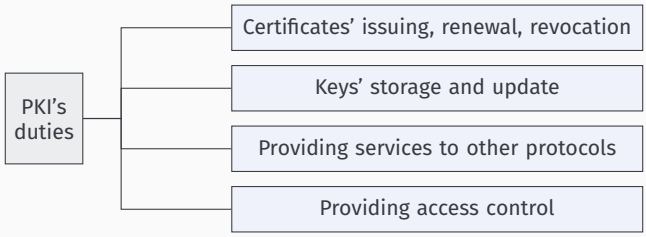
CRL field	Purpose
Signature algorithm ID	Algorithm used to sign the CRL.
Issuer name	CA that issued the list.
This update date	When this list was issued.
Next update date	When the next list will appear; a list older than this is stale, so a user can tell that she must fetch a new one.
Revoked certificates (repeated)	For each one, the user certificate serial number and the revocation date.
Signature	CA's signature over the list, so that Eve cannot delete an entry and revive a revoked certificate.

A revoked certificate stays on the list only until its own validity period ends; after that it is invalid anyway. **Delta CRL.** To make revocation more efficient, the CA publishes a full (base) CRL at fixed intervals, say monthly, and in between a **delta CRL** that contains only the changes made since the last base CRL. A delta CRL is created and posted whenever revocations happen between the "this update" and "next update" dates. It is much smaller, so it can be issued often, and a user combines the base CRL with the latest delta. **OCSP** (Online Certificate Status Protocol) goes further: the user asks an online responder about one certificate and gets a signed answer (good, revoked or unknown). It is as fresh as possible and avoids downloading lists, but the responder must always be online, and it learns which certificates (and so which sites) the user checks.

39.6 Public-key infrastructure (PKI) ▸

▸ **Likely this year:** "What is a PKI? List and explain its duties" (3 marks).

A **public-key infrastructure (PKI)** is a model for creating, distributing and revoking certificates based on X.509; the IETF's version is called **PKIX**. Its components are the end entities (users and devices), the CAs, *registration authorities* (which check identities on behalf of a CA), CRL issuers and repositories (directories from which certificates and CRLs are fetched).



- **Certificates' issuing, renewal and revocation.** This is the main duty: verifying identities, creating certificates, renewing them before expiry, and publishing CRLs or answering OCSP queries.
- **Keys' storage and update.** A PKI should be a storage place for the private keys of members who need to keep them somewhere safe (for example a backup of a decryption key, without which encrypted data would be lost), and it is responsible for updating keys when members ask.
- **Providing services to other protocols.** Internet security protocols such as IPSec, SSL/TLS and S/MIME rely on a PKI to supply and validate the certificates they use.

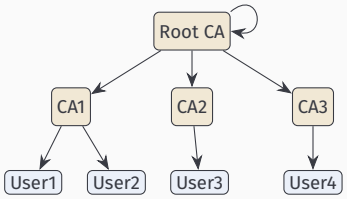
- **Providing access control.** A PKI can give different levels of access to the information in its database; an organisation's PKI may give top management access to the whole database and employees access only to their own records.

39.7 Trust models and certificate chains ▸

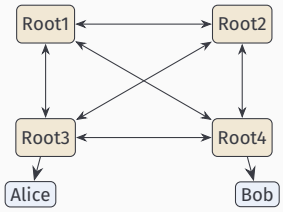
▸ **Likely this year:** "Explain the hierarchical, mesh and web-of-trust models; show how User1 verifies User3's public key" (5 marks).

No single CA can certify everyone in the world, so there are many CAs, each responsible for creating, storing, issuing and revoking a limited number of certificates. A user, however, holds securely the key of only one or a few CAs. A **trust model** defines the rules by which a user can verify a certificate issued by a CA she does not know directly.

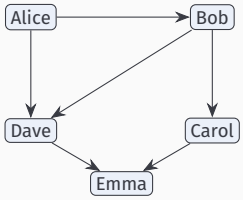
Certificate chains. Suppose Alice trusts CA X_1 but Bob's certificate $X_2\langle\langle B\rangle\rangle$ was issued by X_2. If X_1 has certified X_2, Alice fetches $X_1\langle\langle X_2\rangle\rangle$, verifies it with X_1's key, extracts X_2's public key, and with it verifies $X_2\langle\langle B\rangle\rangle$. The chain is written $X_1\langle\langle X_2\rangle\rangle X_2\langle\langle B\rangle\rangle$, and in general $X_1\langle\langle X_2\rangle\rangle X_2\langle\langle X_3\rangle\rangle \cdots X_N\langle\langle B\rangle\rangle$: each certificate is verified with the key taken from the one before, and the first with a key Alice already trusts (a **trust anchor**).



(a) hierarchical



(b) mesh (cross-certified roots)



(c) web of trust (PGP)

An arrow $X \to Y$ means "X has signed a certificate for Y".

(a) Hierarchical model. The CAs form a tree. The **root CA** has a self-signed, self-issued certificate Root⟪Root⟫ and must be trusted by every user for the system to work; it certifies the level-1 CAs, which certify users or lower CAs. Every user holds the root's public key. To obtain a verified copy of User3's key, User1 receives the chain Root⟪CA2⟫ CA2⟪User3⟫ and (i) validates Root⟪CA2⟫ with the root's public key, (ii) extracts CA2's public key from it, (iii) validates CA2⟪User3⟫ with CA2's key, and (iv) extracts User3's public key. Web browsers use a variant: they ship with the certificates of many independent roots, without a single higher authority above them, and trust a site's chain if it ends at any of them.

(b) Mesh model. A single hierarchy is not enough for a very large community, so several hierarchies are connected by letting their roots **cross-certify** each other: every root issues a certificate for every other root. With n roots this takes $n(n-1)$ certificates (12 for the four roots shown). Alice under Root3 obtains Bob's key (under Root4) by taking Root3⟪Root4⟫ from her root's directory, verifying it with Root3's key, and then verifying Bob's chain down from Root4, e.g. Root4⟪Bob⟫ or Root4⟪CA⟫ CA⟪Bob⟫.

(c) Web of trust. PGP uses no CAs at all. Every user may sign the public keys of people she knows, and each user keeps her own key ring. Alice obtains Bob's key directly (for example in person) and, if she considers Bob a trustworthy *introducer*, she accepts Carol's key because Bob has signed it. PGP lets each user assign a trust level to each introducer, and a key is accepted as valid if it is signed by one fully trusted introducer or by enough partially trusted ones (by default two). This needs no infrastructure and suits small communities, but trust is subjective, chains are hard to evaluate for strangers, and revocation depends on each user spreading the news.

	Hierarchical	Mesh	Web of trust
Trust anchor	one root CA	the user's own root; roots trust each other	keys the user verified personally
Who certifies	CAs only	CAs; roots cross-certify	any user
Path	unique, down from the root	up to own root, across, down	any chain of introducers, judged by the user
Strength	simple, unique paths	connects independent hierarchies	no infrastructure, free
Weakness	root is a single point of failure	$n(n-1)$ cross-certificates	does not scale; weak revocation
Example	a national or corporate PKI	linked government PKIs	PGP

39.8 KDC versus public-key distribution ▸

▸ **Likely this year:** "Differentiate symmetric-key distribution (KDC) and public-key distribution (CA)" (3 marks).

	KDC (symmetric keys)	CA / PKI (public keys)
What must be protected	secrecy of master and session keys	authenticity of the name–key binding
Keys for N users	N master keys + session keys on demand	N key pairs + the CA's key pair
Third party online	yes, for every session	no; certificates are made in advance
What the third party knows	every session key, so it can read all traffic	only public keys; it cannot read traffic
Effect of compromise	all keys and traffic exposed	false certificates can be issued; past traffic stays safe
Revocation	delete the master key at the KDC (immediate)	CRL, delta CRL or OCSP needed
Speed	fast symmetric operations	slower public-key operations
Non-repudiation	no	possible (signatures)
Examples	Needham–Schroeder, Otway–Rees, Kerberos	X.509, PKIX, PGP, TLS certificates

Corrections to the notes / slides

- Slide 12: "The value of K is the same for both Alice and Bob; $g^{xy} \bmod p = 7^{18} \bmod 35 = 18$". The modulus is $p = 23$: $7^{18} \bmod 23 = 18$. (With 35 the value would be $7^{18} \bmod 35 = 14$.)
- Slide 2: "if one million people need to communicate … almost one trillion keys are needed". That count assumes two keys per pair, $N(N-1) \approx 10^{12}$. With one key per pair, $N(N-1)/2 = 499\,999\,500\,000 \approx 5 \times 10^{11}$ (half a trillion). Likewise "the number of required keys for N entities is N^2" means it grows like N^2; the exact count is $N(N-1)/2$.
- Slides 6 and 7 are identical (the symmetric-key agreement slide is repeated).
- Slide 5 says "the KDC informs Bob about Alice's request". This is the conceptual flow; in the actual protocols (Needham–Schroeder, Kerberos) the KDC gives Alice a ticket sealed under Bob's key and Alice forwards it, and in Otway–Rees Bob himself contacts the KDC.
- Slide 19: "Eve cannot forge the private key of Bob to create the signature" should read: Eve cannot forge Bob's *signature*, because she does not know his private key.
- Slide 8: 1024 bits is about 309 decimal digits ("on the order of 300" is fine); 2048 bits (617 digits) is the minimum recommended today.

40 Entity Authentication

40.1 What entity authentication is ▸

▸ **Likely this year:** "Define entity authentication; differentiate data-origin (message) authentication and entity authentication" (2–3 marks).

Entity authentication is a technique designed to let one party prove the identity of another party. An **entity** can be a person, a process, a client or a server. The entity whose identity needs to be proved is called the claimant; the party that tries to prove the identity of the claimant is called the verifier. When Bob tries to prove the identity of Alice, Alice is the claimant and Bob is the verifier. The proof always rests on a **witness**: something that only the true Alice can show or compute, and that Bob can check.

Data-origin versus entity authentication. Message authentication (data-origin authentication, provided by a MAC or a digital signature) also tells Bob who created something, so the two ideas are easily confused. The slides give two differences.

- **Real time.** Message authentication might not happen in real time; entity authentication does. When Bob authenticates an e-mail from Alice, she may or may not be present; the MAC or signature travelled with the message and can be checked days later. When Alice requests entity authentication, no real message is communicated until Bob has authenticated her: she must be online and take part in the process, and only afterwards can messages flow. Data-origin authentication is needed when an e-mail is sent; entity authentication is needed when Alice draws cash from an ATM.
- **One message versus a session.** Message authentication authenticates one message, so the process must be repeated for each new message. Entity authentication authenticates the claimant for the entire duration of a session.

A consequence of the first difference is that entity authentication must defeat **replay**. A signed e-mail replayed a week later is still a genuine message from Alice, so its signature still checks; but a recorded login replayed a week later must *not* make Bob believe Alice is present now. Every protocol in this section therefore contains something that changes from one run to the next (a one-time password, a nonce, a timestamp, a fresh random commitment).

	Data-origin (message) authentication	Entity authentication
Question answered	Who created *this message*?	Who is at the other end *now*?
Real time	Not necessarily; the sender may be offline	Yes; the claimant takes part online
Scope	One message; repeated for every message	Whole session, done once at the start
Replay	A replayed message is still authentic	A replayed proof must be rejected (freshness)
Mechanisms	MAC, HMAC, digital signature	Passwords, challenge–response, zero-knowledge, biometrics
Example	Alice e-mails Bob	Alice withdraws cash at an ATM; login

40.2 Verification categories and multi-factor authentication ▸

▸ **Likely this year:** "What are the three kinds of witness? Give examples" (2 marks); "why is ATM card + PIN stronger than either alone?"

The claimant must identify herself to the verifier with one of three kinds of witnesses.

- **Something known**: a secret known only by the claimant that the verifier can check. Examples are a password, a PIN, a secret key and a private key.
- **Something possessed**: an object that proves the claimant's identity. Examples are a passport, a driver's licence, an identification card, a credit card and a smart card.
- **Something inherent**: an inherent characteristic of the claimant. Examples are a conventional signature, fingerprints, voice, facial characteristics, the retinal pattern and handwriting.

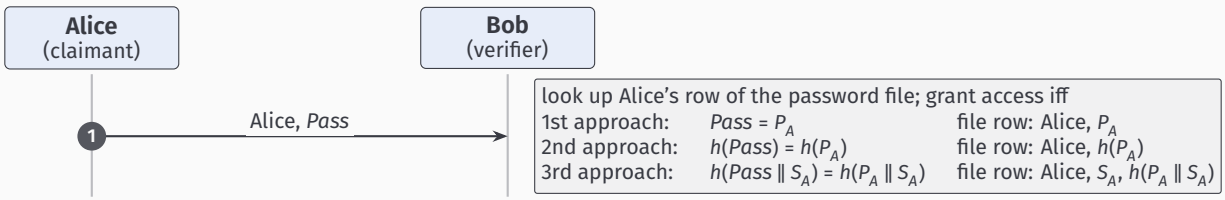
Stallings splits the third category in two, giving four means: something the individual *knows*, *possesses*, *is* (static biometrics: fingerprint, retina, face) and *does* (dynamic biometrics: voice pattern, handwriting, typing rhythm). The two classifications agree; Stallings only separates physiological from behavioural features.

Multi-factor authentication. Each category fails in its own way: a known secret can be guessed or observed, a possessed object can be stolen or lost, an inherent feature can be copied and never changed. Combining witnesses from *different* categories forces Eve to mount two unrelated attacks at once; stealing the card does not reveal the PIN, and watching the PIN being typed does not produce the card. Two-factor authentication means two witnesses from two different categories; two passwords are still one factor, because one keylogger captures both. Everyday examples are an ATM card with a PIN (possessed + known), a password with a one-time code sent to the registered phone (known + possessed), UPI, where the phone with its bound SIM is possessed and the UPI PIN is known, and Aadhaar authentication with a fingerprint plus an OTP (inherent + possessed).

40.3 Fixed passwords: the four approaches ▸

▸ **Likely this year:** "Explain hashing and salting of passwords; what is a dictionary attack and how does salting make it harder?" (4–5 marks), often with the slide figure.

The simplest and oldest method of entity authentication is **password-based authentication**, where the password is something that the claimant knows. A password is used when a user needs to access a system to use its resources (login). Each user has a **user identification** (user ID) that is public and a **password** that is private. Password schemes fall into two groups: the **fixed password**, which is used over and over again for every access, and the **one-time password**, which is used only once. Several fixed-password schemes have been built, each one repairing a weakness of the previous one.



P_A is Alice's stored password, *Pass* the password sent by the claimant, S_A Alice's salt and ‖ concatenation. The message is the same in all three approaches; they differ only in what the verifier stores and computes.

First approach: user ID and password file. The system keeps a table (file) sorted by user ID, with the password of each user stored in clear. To access the system, Alice sends her user ID and password; the system uses the ID to find her row and compares the stored password P_A with the received *Pass*. If they match, access is granted; otherwise it is denied. The approach is open to four attacks.

- **Eavesdropping**: Eve watches Alice while she types her password (or listens on the line). As a measure, most systems do not echo the characters typed.
- **Stealing a password**: Eve physically steals the password, for example a note on which Alice wrote it. This is prevented if Alice does not write the password down but commits it to memory.
- **Accessing the password file**: Eve hacks into the system and reads the ID/password file, which gives her every

user's password at once. The file can be read/write protected, but one bug or one dishonest administrator defeats the protection.

- **Guessing**: Eve logs into the system and tries different combinations of characters as Alice's password. This works well if Alice chose something trivial such as her birthday, her child's name or the name of her favourite actor. A long random password is recommended. Online guessing can also be throttled by locking the account or adding a delay after a few failures.

Second approach: hashing the password. The third attack is the most damaging because one break exposes every user, so the system stops storing passwords and stores their hashes. When the password is created, the system stores $h(P_A)$ instead of P_A; at login it computes $h(Pass)$ and compares it with the stored value. Since h is a one-way (preimage-resistant) hash function, Eve cannot recover P_A from $h(P_A)$ even if she reads the whole file, and she cannot log in with $h(P_A)$ itself, because the system will hash whatever she sends.

The dictionary attack. Hashing protects strong passwords but not the ones people actually choose, because the set of likely passwords is small. In a **dictionary attack**, Eve is interested in finding *one* password, regardless of the user ID. If passwords are 6 digits, she lists all numbers from 000000 to 999999 and hashes every one, which gives a list of one million (word, hash) pairs. She then obtains the password file and searches its second column for any hash in her list; a match gives her the password of that user. All of this can be programmed and run **offline** on Eve's own computer, so account lock-outs and delays do not slow it down; only after a match does she go online and log in. The cost is one hash per candidate password, and since the same list serves every user and every system that uses the same h, it can be computed once and reused.

WORKED EXAMPLE : cost of a dictionary attack

Q. Eve has stolen a file of unsalted password hashes and can compute 10^9 hashes per second. How long does an exhaustive dictionary attack take if passwords are (a) 6 decimal digits, (b) 8 lowercase letters, (c) 8 characters from the 95 printable ASCII characters?
A. The work is (number of candidates) ÷ (rate), and one pass covers all users.
(a) 10^6 candidates: $10^6/10^9$ = **0.001 s**.
(b) 26^8 = 208 827 064 576 ≈ 2.09×10^{11}: ≈ 209 s ≈ **3.5 minutes**.
(c) 95^8 = 6 634 204 312 890 625 ≈ 6.63×10^{15}: 6.63×10^6 s ÷86 400 ≈ **77 days**.
Check: each extra character multiplies the work by the alphabet size; going from 26 to 95 symbols multiplies it by $(95/26)^8 \approx 3.2 \times 10^4$, which matches $6.63 \times 10^{15}/2.09 \times 10^{11}$.

Third approach: salting the password. When the password is created, a random string called the **salt** is concatenated to the password, and the salted password is hashed. The ID, the salt and the hash are stored in the file: (Alice, S_A, $h(P_A \| S_A)$). When a user asks for access, the system extracts her salt, concatenates it with the received password, hashes the result, and compares it with the stored hash. The salt is *not* secret; it is stored in clear next to the hash, because the system needs it at every login.

Why salting makes the dictionary attack harder. Without salt, two users with the same password have the same hash, and a single precomputed list $h(w)$ for every dictionary word w attacks every user in every system at once. With salt, the hash of a word depends on the salt, so a precomputed list must contain $h(w \| S)$ for every word *and every possible salt*: a b-bit salt multiplies the size of the list by 2^b. The slide's example: if the password is 6 digits and the salt 4 digits, hashing is done over a 10-digit value, and a complete precomputed list has 10^{10} entries instead of 10^6. Stallings lists three effects of the salt.

- It prevents duplicate passwords from being visible in the password file: two users with the same password get different salts and therefore different hashes.
- It greatly increases the difficulty of offline dictionary attacks: with a b-bit salt, the number of possible hashed values for each password grows by a factor of 2^b, so precomputed tables become 2^b times larger, and the users of one file must be attacked one at a time.
- It becomes nearly impossible to find out whether a person with accounts on two or more systems has used the same password on all of them.

In **UNIX** the salt is 12 bits, so each password can be stored in 2^{12} = 4096 different ways; the classical `crypt` also iterates its DES-based one-way function 25 times to make each guess slower. **What salting does not do**: if Eve attacks one particular user, she reads that user's salt from the file and hashes her dictionary with it, which costs exactly as much as an unsalted attack. Salting therefore defeats precomputation and attacks on many users at once, but a weak password of a single targeted user still falls; against that, the only defences are long random passwords and a deliberately slow hash (many iterations, as in `crypt`, PBKDF2 or bcrypt).

WORKED EXAMPLE : what a 12-bit salt buys

Q. A system with 1000 users stores salted hashes with a 12-bit salt. Eve's dictionary has 10^6 words and she computes 10^6 hashes per second. Compare (a) the size of a precomputed table covering all salts with the unsalted table; (b) the time to test the dictionary against all 1000 users with and without salt; (c) the time to

attack Alice alone.
A. (a) Unsalted: 10^6 entries. Salted: $10^6 \times 2^{12} = 10^6 \times 4096 =$ **4.096×10^9** entries, 4096 times larger.
(b) Unsalted: one pass of 10^6 hashes; each digest is looked up among the 1000 stored hashes, so **1 s** for all users. Salted: the users (almost all) have different salts, so each needs its own pass: $1000 \times 10^6 = 10^9$ hashes = 1000 s ≈ **16.7 minutes**.
(c) Alice's salt is in the file, so one pass of 10^6 hashes: still **1 s**.
Check: salting multiplied the bulk attack by the number of distinct salts (1000) and precomputation by 2^{12}, but did not change the single-user attack.

Fourth approach: combining two identification techniques. Two identification techniques are combined, the standard example being an ATM card with a PIN (personal identification number). The card belongs to the category "something possessed" and the PIN to "something known". The PIN is a password that enhances the security of the card: if the card is stolen, it cannot be used unless the PIN is known. The PIN, however, is traditionally very short (4 digits, 10^4 values) so that it is easily remembered, which makes it vulnerable to guessing; the card is therefore retained or blocked after three wrong PINs, which turns a $1/10^4$ chance per try into at most a $3/10^4$ chance overall.

Approach	File stores	Remaining weakness
1. Password file	ID, P_A	Stolen file reveals every password; eavesdropping; guessing
2. Hashing	ID, $h(P_A)$	Offline dictionary attack with one precomputed list for all users
3. Salting	ID, S_A, $h(P_A \| S_A)$	Weak password of one targeted user still falls; eavesdropping
4. Two techniques	card data, PIN check	Short PIN guessable (hence retry limit); card can be cloned

All four approaches share one weakness: the same password crosses the line at every login, so an eavesdropper who records it once can replay it forever. One-time passwords remove exactly this.

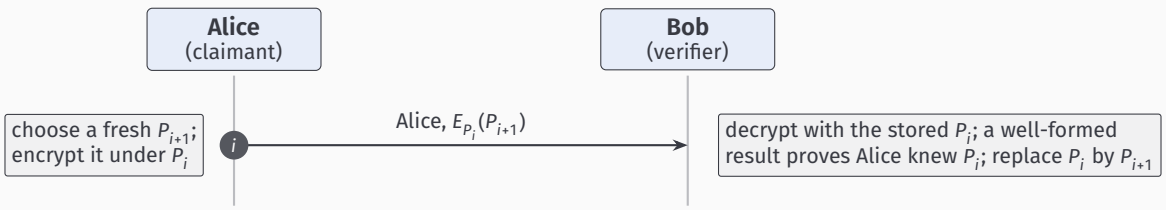
40.4 One-time passwords and Lamport's hash chain ★▸

▸ **Likely this year:** "Explain the Lamport one-time password scheme with a diagram; why can an eavesdropper not use what she sees?" (4–5 marks), possibly with a small hash chain to compute.

A **one-time password** is a password that is used only once. Since each value is accepted a single time, recording it is useless: a one-time password makes eavesdropping and reuse (replay) of the password useless, and it also makes salting unnecessary, because a stolen value cannot be used again. Forouzan gives three approaches.

First approach: an agreed list. The user and the system agree upon a list of passwords, and each password on the list can be used only once. There are two drawbacks. First, the system and the user must keep a long list of passwords, and whoever steals either copy of the list owns all future logins. Second, if the user does not use the passwords in sequence, the system needs to perform a long search to find the match.

Second approach: sequentially updated passwords. The user and the system agree on an original password P_1, which is valid only for the first access. During the first access, the user generates a new password P_2 and sends it encrypted with P_1 as the key; P_2 becomes the password for the second access. During the second access the user generates P_3 and encrypts it with P_2, and so on: P_i is used to create (carry) P_{i+1}.

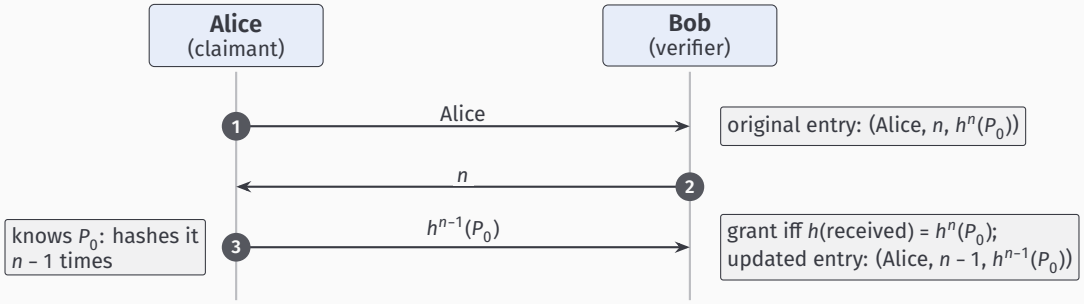


The weakness is that the passwords form a chain that can be followed *forwards*. If Eve guesses or steals P_1 (or any later P_i), she decrypts the recorded message of that access to get P_{i+1}, then the next one, and so on: she finds all subsequent passwords. Since P_1 is usually a human-chosen password, Eve can even run a dictionary attack on the recorded $E_{P_1}(P_2)$ offline. In addition, the system must keep the current P_i in a usable form (it needs it as a decryption key), so stealing the file gives the current password, and a lost message leaves the user and the system out of step.

Third approach: Lamport's hash chain. Leslie Lamport's scheme reverses the direction of the chain so that following it requires inverting a hash function. The user and the system agree upon an original password P_0 and a counter n. The system calculates $h^n(P_0)$, where h^n means applying a hash function n times:

$$h^n(x) = h(h^{n-1}(x)), \quad h^{n-1}(x) = h(h^{n-2}(x)), \quad \ldots, \quad h^2(x) = h(h(x)), \quad h^1(x) = h(x).$$

The system stores the identity of Alice, the value of n and the value of $h^n(P_0)$; it never stores P_0. Each login is three messages.



1. Alice sends her identity, which tells the system which entry to use.
2. The system sends the current counter n, telling Alice which element of the chain is expected.
3. Alice computes $h^{n-1}(P_0)$ from P_0 and sends it. The system applies the hash function once to the received value and compares the result with the stored $h^n(P_0)$. If there is a match, access is granted; otherwise it is denied. The system then decrements n and replaces $h^n(P_0)$ by the value just received, $h^{n-1}(P_0)$.

At the second access the counter Alice receives is $n - 1$, her third message is $h^{n-2}(P_0)$, and the system hashes it to get $h^{n-1}(P_0)$, which it compares with the updated entry. The counter is decremented at every access; when it reaches 0 the user can no longer log in and everything must be set up again with a new P_0, which is why n is normally chosen large, such as 1000.

Why an eavesdropper cannot go forward. The values are *used* in the order $h^{n-1}, h^{n-2}, \ldots, h^1, h^0$, i.e. backwards along the direction in which the hash is computed. After seeing $h^k(P_0)$, Eve needs the next value $h^{k-1}(P_0)$, which is a *preimage* of what she holds: $h(h^{k-1}(P_0)) = h^k(P_0)$. Finding it would break the preimage resistance of h. Going the easy direction only produces $h^{k+1}(P_0), h^{k+2}(P_0), \ldots$, which have already been used and are rejected. Replaying the value she saw also fails, because the system has replaced its entry by that value and will hash whatever it receives once more. For the same reason the password file is harmless: it contains only the last used value $h^k(P_0)$, from which nothing usable can be computed. Compared with the second approach, nothing secret is stored and nothing is encrypted; the only tool is a one-way function.

The small-n attack. Alice must check that the counter she receives is smaller than the last one she answered. If Eve impersonates the system and sends a much smaller counter m, Alice answers $h^{m-1}(P_0)$, and since hashing forwards is easy, Eve can compute $h^m(P_0), h^{m+1}(P_0), \ldots$ and so answer every counter above $m - 1$ that the real system will ask for. A one-way chain can be extended forwards by anyone, so the claimant must never reveal an element closer to P_0 than necessary.

S/KEY. S/KEY (Bellcore, N. Haller, RFC 1760, 1995) is the practical implementation of Lamport's scheme, later standardised as the OTP system of RFC 2289. The user's secret pass-phrase is concatenated with a public **seed** sent by the server (playing the role of a salt, so that one pass-phrase gives different chains on different servers) and hashed with MD4 (MD5 or SHA-1 in RFC 2289); each output is folded to 64 bits. The server sends the sequence number and the seed as the challenge, and the 64-bit one-time password is typed as six short English words from a 2048-word dictionary, which is easy to read from a printed list or a small calculator.

WORKED EXAMPLE : a Lamport chain with a toy hash

Q. Let $h(x) = x^2 \bmod 9991$ (here $9991 = 97 \times 103$; with the factors secret, finding a square root, i.e. a preimage, is as hard as factoring). Alice's original password is $P_0 = 1234$ and $n = 5$. (a) Compute the chain and the entry the system stores. (b) Show the first two logins. (c) Eve recorded the value of the first login and replays it at the second; what happens?
A. (a) Hash repeatedly:

Value	Computation	Result
$h^0(P_0)$	P_0	1234
$h^1(P_0)$	$1234^2 = 1\,522\,756 = 152 \cdot 9991 + 4124$	4124
$h^2(P_0)$	$4124^2 = 17\,007\,376 = 1702 \cdot 9991 + 2694$	2694
$h^3(P_0)$	$2694^2 = 7\,257\,636 = 726 \cdot 9991 + 4170$	4170
$h^4(P_0)$	$4170^2 = 17\,388\,900 = 1740 \cdot 9991 + 4560$	4560
$h^5(P_0)$	$4560^2 = 20\,793\,600 = 2081 \cdot 9991 + 2329$	2329

The system stores **(Alice, 5, 2329)**.
(b) *Login 1*: Bob sends $n = 5$; Alice sends $h^4(P_0) = 4560$; Bob computes $4560^2 \bmod 9991 = 2329$ = stored value, so access is granted, and the entry becomes (Alice, 4, 4560).
Login 2: Bob sends 4; Alice sends $h^3(P_0) = 4170$; $4170^2 \bmod 9991 = 4560$ = stored, granted; entry (Alice, 3, 4170).
(c) At login 2 Eve sends 4560. Bob computes $4560^2 \bmod 9991 = 2329 \neq 4560$, so access is **denied**. To succeed Eve needs a square root of 4560 modulo 9991 (one of 874, 4170, 5821, 9117), which requires the factors of 9991.
Check: the five logins use 4560, 4170, 2694, 4124, 1234 in turn; after the fifth the counter is 0 and a new chain

is needed.

★ MUST KNOW : Lamport

Store (ID, n, $h^n(P_0)$). Login: system sends n, user sends $h^{n-1}(P_0)$, system checks h(received) = $h^n(P_0)$, then stores (n – 1, $h^{n-1}(P_0)$). Security rests on preimage resistance: the next value is a preimage of the last one seen.

40.5 Challenge–response authentication ▸

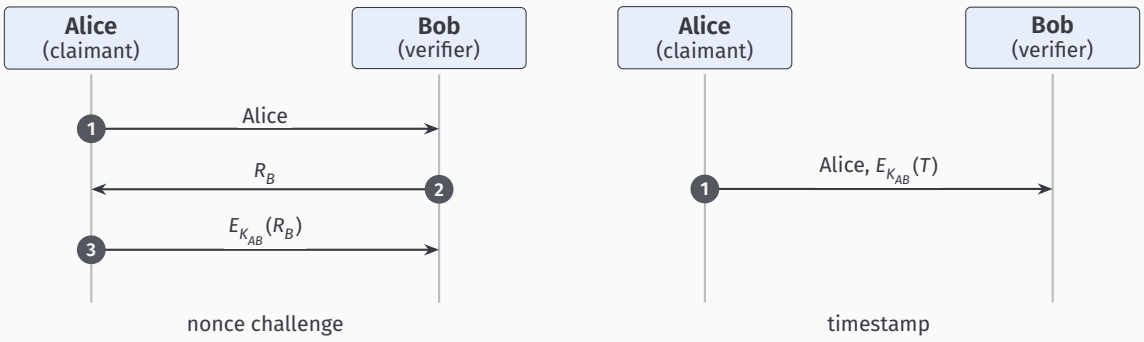
▸ **Likely this year:** "Explain challenge–response authentication using a symmetric-key cipher / keyed hash / public-key cipher / digital signature with diagrams; what are replay and reflection attacks and how are they prevented?" (5 marks).

In password authentication the claimant proves her identity by sending her secret, the password, so whoever sees it can reuse it (or, with one-time passwords, the scheme needs long lists or chains). In **challenge–response authentication**, the claimant proves that she *knows* a secret without sending it. The verifier sends a time-varying challenge; the claimant applies a function to the challenge and her secret and sends the result, the response; the verifier checks the response. Since the challenge is different every time, so is the correct response, and a recorded response is useless at the next login.

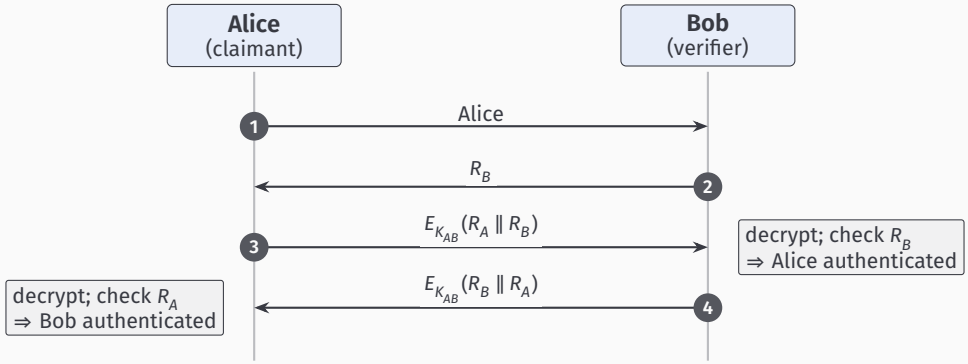
The **challenge** is a **time-varying value** of one of three kinds. A **nonce** is a random number used only once; it needs no shared state but costs one extra message, because the verifier has to send it. A **timestamp** is the current time, which the claimant can generate herself, so one message suffices, but the clocks must be synchronised and the verifier must remember the timestamps already accepted within its acceptance window. A **sequence number** is a counter that both sides keep, which needs stored state per partner. The secret used for the response can be a symmetric key shared with the verifier, a key for a keyed-hash function, or the claimant's private key (used either to decrypt or to sign).

Using a symmetric-key cipher: nonce challenge. Alice and Bob share a secret key K_{AB}. Alice sends her identity; Bob sends a nonce R_B; Alice encrypts it and returns $E_{K_{AB}}(R_B)$. Bob decrypts and compares the result with the nonce he sent. Only someone holding K_{AB} could have produced a ciphertext that decrypts to this fresh R_B, so Alice is authenticated; since R_B was never used before, the response cannot be a replay. The key itself never crosses the line.

Using a symmetric-key cipher: timestamp. The challenge is replaced by a timestamp T that Alice generates, so the whole protocol is one message: Alice sends her identity and $E_{K_{AB}}(T)$. Bob decrypts and accepts if T is within the acceptance window around his clock and has not been seen before. The timestamp gives freshness without a message from Bob, but the clocks must agree and Bob must remember recent timestamps, since otherwise Eve can replay the message within the window.

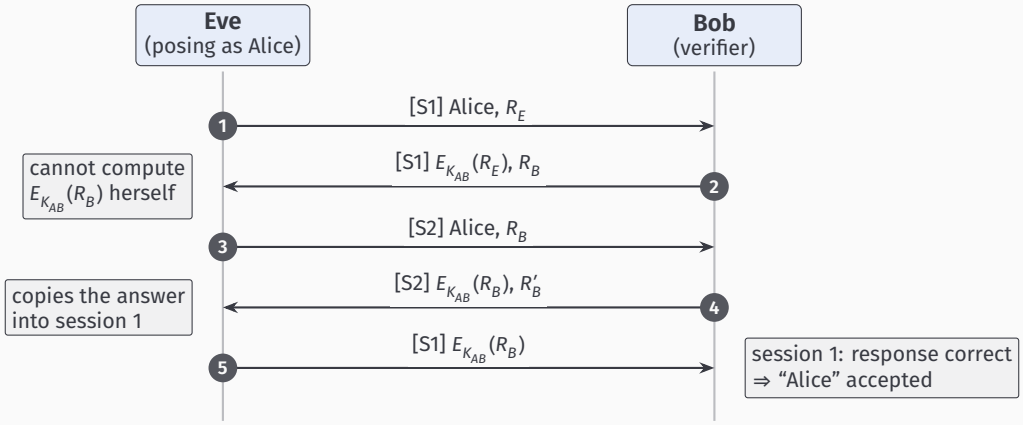


Bidirectional (mutual) authentication. Often Bob must also prove his identity to Alice; otherwise Eve could pose as the server (a fake ATM or a phishing site) and collect whatever Alice sends. Each side then sends its own challenge and answers the other's. In the four-message version, Alice announces herself, Bob sends R_B, and Alice answers R_B while challenging Bob with her own R_A; Bob answers with both nonces in reverse order.



Message 3 proves that Alice holds K_{AB} (it contains Bob's fresh nonce) and carries her challenge; message 4 proves that Bob holds K_{AB} (it contains Alice's fresh nonce). The two encrypted messages have different forms (nonce order swapped), and Bob encrypts nothing until Alice has proved herself; both choices are there to stop the reflection attack below. The timestamp version needs only two messages: Alice sends Alice, $E_{K_{AB}}(T_A, \text{Bob})$ and Bob answers $E_{K_{AB}}(T_B, \text{Alice})$; the name of the intended receiver inside each ciphertext stops Eve from simply sending Alice's own message back to her.

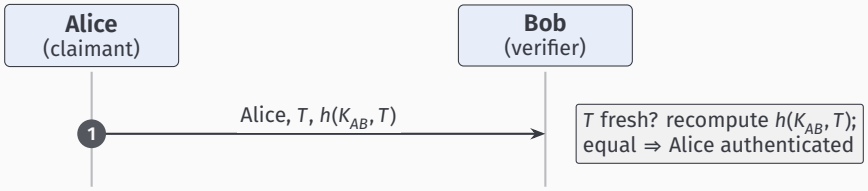
The reflection attack. A natural three-message mutual protocol lets the initiator challenge first: (1) Alice → Bob: Alice, R_A; (2) Bob → Alice: $E_{K_{AB}}(R_A)$, R_B; (3) Alice → Bob: $E_{K_{AB}}(R_B)$. Both directions use the same key and the same format, "encrypt the other side's nonce". In a **reflection attack** Eve, without the key, makes Bob answer his own challenge in a second, parallel session.



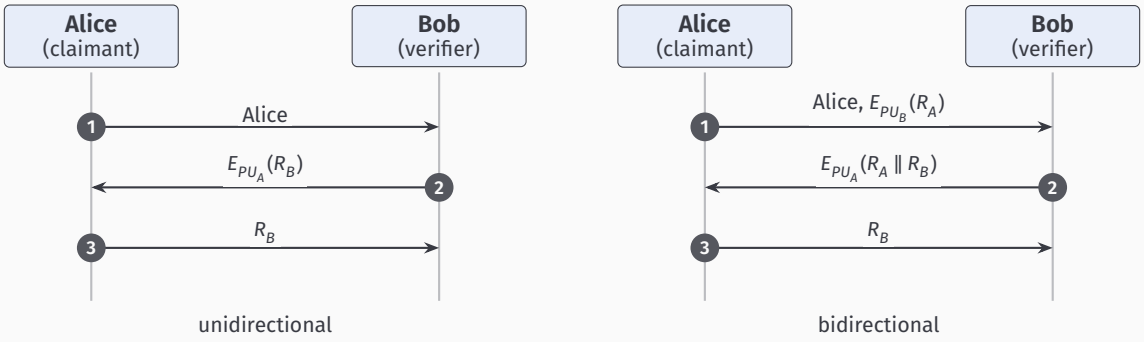
Eve opens session S1 with any number R_E and receives Bob's challenge R_B, which she cannot answer. She then opens session S2 and presents R_B as *her* challenge; Bob dutifully encrypts it. Eve copies $E_{K_{AB}}(R_B)$ into session S1 as Alice's answer and abandons S2. Bob is fooled by his own response, reflected back at him. The attack needs two conditions: the verifier answers challenges for anyone before they have authenticated, and a response from one direction is valid in the other. Removing either one prevents it.

- **Make the claimant prove first.** If Bob encrypts nothing until the other party has answered his challenge (as in the four-message protocol above), Eve never obtains an encryption from Bob to reflect.
- **Put identities or roles inside the response.** If Alice's response is $E_{K_{AB}}(R_B, \text{Bob})$ and Bob's is $E_{K_{AB}}(R_A, \text{Alice})$ (name of the intended receiver), a response made by Bob says "Alice" and is rejected when it comes back claiming to be for Bob. Changing the order of fields, as in message 4 above, has the same effect.
- **Use different keys in the two directions**, K_{AB} for Alice's responses and K_{BA} for Bob's, so that nothing Bob produces is ever a valid response from Alice.

Using keyed-hash functions. Instead of encryption, a keyed-hash function (MAC, e.g. HMAC) of the challenge can be used. Alice sends her identity, the timestamp T in clear, and $h(K_{AB}, T)$; Bob takes the plaintext T, checks that it is fresh, computes $h(K_{AB}, T)$ himself and compares. With a nonce, the third message is $h(K_{AB}, R_B)$ instead of $E_{K_{AB}}(R_B)$. Nothing needs to be decrypted, since Bob recomputes and compares; a hash is cheaper than a cipher; and the integrity of the challenge and response is preserved, because changing T in transit changes the expected digest.

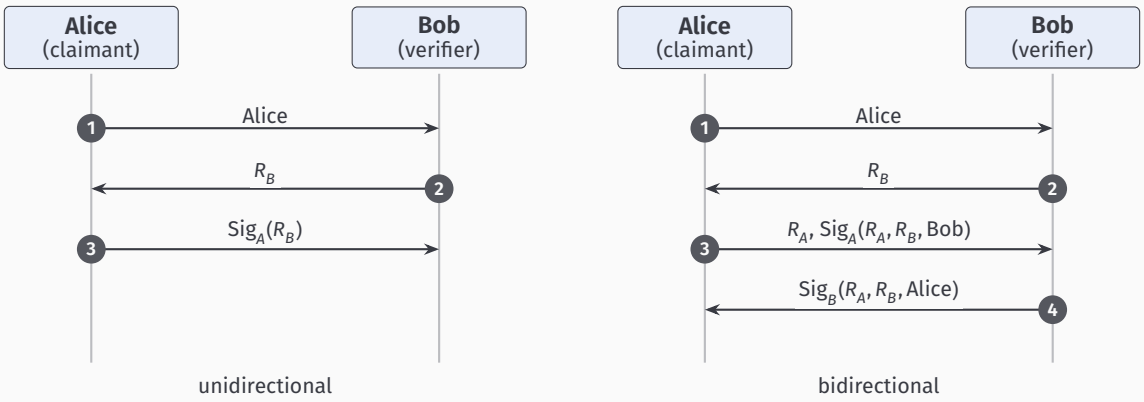


Using an asymmetric-key cipher. Here the secret is Alice's private key, so Bob needs only her public key and no shared secret has to be set up in advance. In the unidirectional version, Bob encrypts a nonce with Alice's public key PU_A and sends $E_{PU_A}(R_B)$; only the holder of the private key PR_A can decrypt it, so Alice proves herself by returning R_B. In the bidirectional version, Alice sends her identity with her own challenge encrypted for Bob, $E_{PU_B}(R_A)$; Bob decrypts it and returns $E_{PU_A}(R_A, R_B)$, proving that he could decrypt (he knows R_A) and challenging Alice; Alice decrypts and returns R_B.



Two cautions make these protocols safe in practice. First, in the unidirectional version Alice decrypts whatever she is sent, so a dishonest "verifier" can submit a ciphertext that someone else encrypted for Alice and obtain its plaintext: Alice becomes a **decryption oracle**. Alice should use a separate key pair for authentication, or Bob should also send $h(R_B)$ so that Alice only answers a challenge whose plaintext Bob already knows. Second, the bidirectional version is essentially the Needham–Schroeder public-key protocol, and Lowe (1995) showed that if Alice ever starts a run with a dishonest party Eve, Eve can re-encrypt R_A for Bob, pass Bob's reply on to Alice, and let Alice unknowingly answer Bob's challenge, so that Bob believes he is talking to Alice. Putting the responder's identity inside message 2, $E_{PU_A}(R_A, R_B, \text{Bob})$, lets Alice see that the reply came from Bob and not from Eve, which blocks the attack.

Using digital signatures. Alice proves possession of her private key by signing the challenge; Bob verifies the signature with her public key. In the unidirectional version Bob sends R_B and Alice returns $\text{Sig}_A(R_B)$, her signature on it. Since Alice would then sign any value Bob chooses, she adds her own nonce: in the bidirectional version she returns R_A and $\text{Sig}_A(R_A, R_B, \text{Bob})$, and Bob answers with $\text{Sig}_B(R_A, R_B, \text{Alice})$. The fresh R_B in Alice's signature defeats replay, her own R_A ensures she never signs a string chosen entirely by someone else, and the name of the intended verifier prevents Eve from taking a response signed for Bob and presenting it to Carol.



Secret used	Challenge	Response	What the fields protect against
Symmetric key K_{AB}	nonce R_B	$E_{K_{AB}}(R_B)$	fresh nonce: replay
Symmetric key K_{AB}	timestamp T	$E_{K_{AB}}(T)$	time window + memory of recent T: replay
Symmetric key, mutual	R_B, then R_A	$E_{K_{AB}}(R_A \| R_B)$, $E_{K_{AB}}(R_B \| R_A)$	claimant proves first, different formats: reflection
Keyed hash	T or R_B	$h(K_{AB}, T)$	freshness: replay; digest also protects integrity of T
Public-key encryption	$E_{PU_A}(R_B)$	R_B	only PR_A decrypts; responder's name in mutual version: Lowe's attack
Digital signature	R_B	R_A, $\mathrm{Sig}_A(R_A, R_B, \mathrm{Bob})$	R_B: replay; R_A: chosen-message signing; name: reuse with another verifier

Trap: replay versus reflection

A **replay attack** resends a message recorded in an *earlier* run; it is stopped by freshness (nonce, timestamp, sequence number). A **reflection attack** sends a party's own message back to it in a *concurrent* run; nonces alone do not stop it, because the reflected response is fresh. It is stopped by asymmetry: identities inside responses, different formats or keys per direction, or making the claimant answer first.

40.6 Zero-knowledge authentication: the idea ▸

▸ **Likely this year:** "What is zero-knowledge authentication? State its properties" (2–3 marks), usually before a Fiat–Shamir question.

In password authentication the claimant must send her secret to the verifier, where it is exposed to eavesdropping by Eve; in addition, a dishonest verifier could reveal the password to others or use it to impersonate the claimant. Challenge–response hides the secret from Eve, but the verifier still sees values computed directly from the key (and in the symmetric case already holds the key). In **zero-knowledge authentication**, the claimant proves to the verifier that she knows a secret without revealing anything that might endanger the confidentiality of the secret. The interactions are designed so that they cannot lead to revealing or guessing the secret: after the exchange, the verifier only knows that the claimant does or does not have the secret, nothing more. The result is a yes/no situation, a single bit of information. A zero-knowledge protocol must have three properties.

- **Completeness**: if the claimant knows the secret and both sides follow the protocol, the verifier always accepts.
- **Soundness**: if the claimant does not know the secret, she is accepted only with a negligible probability, which the verifier can make as small as he likes by repeating the protocol.
- **Zero-knowledge**: the verifier learns nothing beyond the fact that the claim is true. Formally, everything he sees in a run (the transcript) could have been produced by himself without the claimant, so it cannot contain information about the secret.

The cave example. A cave has an entrance that splits into a left and a right passage, which meet at a door deep inside that opens only with a magic word. Alice claims to know the word. Bob waits outside while Alice walks to the door along a passage of her choice. Bob then comes to the fork and calls out "left" or "right", and Alice must come out of that passage. If she knows the word, she can always do it, passing through the door if needed. If she does not, she succeeds only when Bob happens to call the passage she entered, with probability 1/2; after k repetitions her chance of fooling Bob is $(1/2)^k$. Bob learns nothing about the word, and a video of the session would not convince anyone else, since Alice and a friend could have agreed the calls in advance. Entering a passage is the commitment, Bob's call is the challenge, and coming out is the response; Fiat–Shamir is this cave in modular arithmetic.

40.7 Fiat–Shamir protocol ★▸

▸ **Likely this year:** "Explain Fiat–Shamir with a diagram; prove that an honest claimant passes; show that a dishonest one passes a round with probability 1/2; how many rounds for a given confidence?" plus a small numerical (5–6 marks).

Setup. A trusted third party chooses two large primes p and q and calculates $n = p \times q$. The value of n is announced to the public; p and q are kept secret. Alice, the claimant, chooses a secret number s between 1 and $n - 1$ (exclusive), which must be coprime to n, and calculates

$$v = s^2 \bmod n.$$

She keeps s as her private key and registers v as her public key with the third party. Computing v from s is one squaring; computing s from v means finding a square root modulo n, which is as hard as factoring n. This is the one-way function behind the protocol.

One round. Verification of Alice by Bob is done in the following steps.

1. Alice chooses a random number r between 0 and $n-1$ (r is called the commitment) and calculates $x = r^2 \bmod n$ (x is called the witness).
2. Alice sends x to Bob as the witness.

3. Bob, the verifier, sends the challenge c to Alice; c is either 0 or 1, chosen at random.
4. Alice calculates the response $y = rs^c \bmod n$: $y = r$ if $c = 0$ and $y = rs$ if $c = 1$.
5. Alice sends the response y to Bob to show that she knows her private key s.
6. Bob calculates $y^2 \bmod n$ and $xv^c \bmod n$. If they are congruent, the round is passed (Alice is *probably* honest); otherwise she is rejected.



Why an honest Alice always passes (completeness). Substituting $y = rs^c$, $x = r^2$ and $v = s^2$:

$$y^2 = (rs^c)^2 = r^2 s^{2c} = r^2 (s^2)^c = xv^c \pmod{n}.$$

Why a dishonest claimant passes a round with probability $1/2$**.** Alice can be honest (she knows s) or dishonest (she does not); call the dishonest one Eve. Eve can still pass a round by predicting the challenge, and two situations can happen.

- **Eve guesses** $c = 1$**.** She picks r and sends $x = r^2/v = r^2 v^{-1} \bmod n$, which she can compute because v is public. If c turns out to be 1, she sends $y = r$, and $y^2 = r^2 = (r^2 v^{-1})v = xv^1$: she passes. If c turns out to be 0, she needs y with $y^2 = x$, a square root of $x = r^2 v^{-1}$ modulo n, which she cannot compute (it is rs^{-1}, and needs s); she quits or sends a wrong value, and Bob aborts.
- **Eve guesses** $c = 0$**.** She sends $x = r^2$. If $c = 0$, she sends $y = r$ and passes ($y^2 = x = xv^0$). If $c = 1$, she needs y with $y^2 = xv = r^2 s^2$, i.e. $y = rs$, which again needs s; she fails.

The key point is that Eve must commit to x before she sees c, and *no* witness lets her answer both challenges. Suppose some x let her answer both, with $y_0^2 \equiv x$ and $y_1^2 \equiv xv$. Dividing,

$$(y_1 y_0^{-1})^2 \equiv \frac{xv}{x} = v \pmod{n},$$

so $y_1 y_0^{-1}$ is a square root of v, which works exactly like s (it passes every round). Thus anyone who can answer both challenges for one witness can compute a square root of v modulo n, which is as hard as factoring n. Without s, therefore, Eve can prepare for at most one value of c, and since Bob chooses c uniformly at random, a dishonest claimant passes a round with probability $1/2$. The rounds are independent (fresh r and c each time), so after k rounds

$$\Pr[\text{Eve passes all } k \text{ rounds}] = (1/2)^k, \qquad (1/2)^{20} = 9.54 \times 10^{-7}.$$

Bob assigns a probability of $1/2$ to each round; to push the cheating probability below ε he needs $k \geq \log_2(1/\varepsilon)$ rounds, i.e. 20 rounds for $\varepsilon = 10^{-6}$ ($\log_2 10^6 = 19.93$).

Why it is zero-knowledge. Eve's cheating strategy is also a recipe for faking transcripts: anyone can pick c and y at random and set $x = y^2 v^{-c} \bmod n$, and the triple (x, c, y) has exactly the distribution of a real round with honest Alice (x a random square, c a random bit, y a random square root of xv^c). Since Bob could have produced every transcript himself, seeing them teaches him nothing about s. Two rules are needed for this: Alice must use a **fresh** r in every round, because answering both challenges for one r hands out $y_0 = r$ and $y_1 = rs$, and then $s = y_1 y_0^{-1} \bmod n$; and Bob must choose c **unpredictably**, because a predictable challenge is exactly what Eve needs. An r coprime to n must be used, and Bob must reject $x = 0$: with $r = 0$ the witness is 0 and $y = 0$ passes for either challenge.

WORKED EXAMPLE : Fiat–Shamir, honest rounds and a reused r

Q. The trusted centre chooses $p = 7$, $q = 11$. Alice's private key is $s = 9$. (a) Find n and Alice's public key. (b) Show a round with $r = 13$, $c = 1$ and a round with $r = 24$, $c = 0$. (c) Show what Bob learns if Alice wrongly reuses $r = 13$ and is asked both $c = 0$ and $c = 1$.
A. (a) $n = 7 \times 11 = 77$; $\gcd(9, 77) = 1$; $v = 9^2 \bmod 77 = 81 \bmod 77 =$ **4**.
(b)

r	$x = r^2 \bmod 77$	c	$y = rs^c \bmod 77$	$y^2 \bmod 77$	$xv^c \bmod 77$
13	$169 - 154 = 15$	1	$13 \cdot 9 = 117 \to 40$	$1600 - 1540 = 60$	$15 \cdot 4 = 60$ ✓
24	$576 - 539 = 37$	0	24	$576 \to 37$	37 ✓

Both rounds pass: $y^2 \equiv xv^c$.
(c) Bob receives $y_0 = 13$ and $y_1 = 40$ for the same r. Then $13^{-1} \bmod 77 = 6$ (since $13 \cdot 6 = 78 \equiv 1$), and $s = y_1 y_0^{-1} = 40 \cdot 6 = 240 \bmod 77 = \mathbf{9}$, Alice's private key.
Check: $9^2 = 81 \equiv 4 = v$ ✓.

WORKED EXAMPLE : a cheating claimant

Q. With $n = 77$ and Alice's public key $v = 4$, Eve (who does not know s) bets that Bob will send $c = 1$ and uses $r = 17$. What witness does she send? Show the round for $c = 1$ and for $c = 0$.
A. $v^{-1} \bmod 77 = 58$ (since $4 \cdot 58 = 232 = 3 \cdot 77 + 1$). $r^2 = 289 \equiv 58$, so $x = r^2 v^{-1} = 58 \cdot 58 = 3364 \bmod 77 = \mathbf{53}$.
$c = 1$: Eve sends $y = r = 17$. Bob: $y^2 = 289 \bmod 77 = 58$; $xv = 53 \cdot 4 = 212 \bmod 77 = 58$. Equal, so Eve **passes**.
$c = 0$: Bob needs $y^2 \equiv x = 53$. Eve has only $r = 17$: $17^2 \equiv 58 \neq 53$, so she **fails**. A valid answer would be a square root of 53 modulo 77 (19, 30, 47 or 58; e.g. $rs^{-1} = 17 \cdot 60 \bmod 77 = 19$), which Eve can find only by factoring n; for a 2048-bit n that is infeasible.
Check: Eve passes exactly when her guess equals Bob's coin, probability 1/2.

★ MUST KNOW : Fiat–Shamir

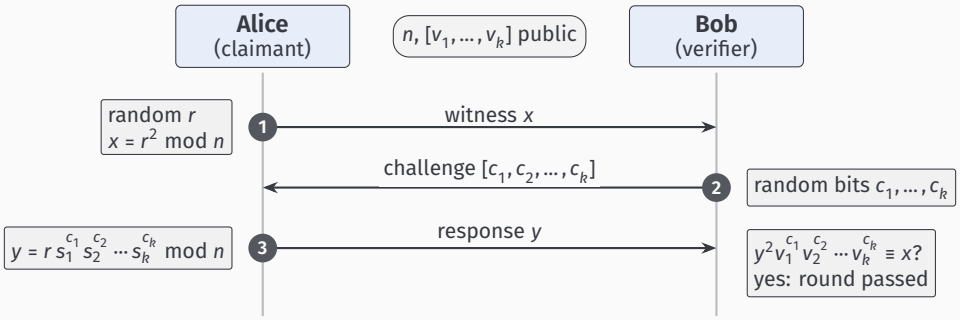
$n = pq$ public; secret s, public $v = s^2 \bmod n$. Round: $x = r^2$; $c \in \{0, 1\}$; $y = rs^c$; accept iff $y^2 \equiv xv^c \pmod n$. Proof: $y^2 = r^2 s^{2c} = xv^c$. Cheater passes a round with probability 1/2, k rounds with $(1/2)^k$; 20 rounds give 9.54×10^{-7}.

40.8 Feige–Fiat–Shamir protocol ▸

▸ **Likely this year:** "How does Feige–Fiat–Shamir differ from Fiat–Shamir? Prove its verification equation" (3–4 marks).

Fiat–Shamir needs many rounds because each round tests only one bit. The **Feige–Fiat–Shamir** protocol tests k bits per round: it is similar to Fiat–Shamir except that it uses a vector of private keys $[s_1, s_2, \ldots, s_k]$, a vector of public keys $[v_1, v_2, \ldots, v_k]$ and a vector of challenges $[c_1, c_2, \ldots, c_k]$. The private keys are chosen randomly but must be relatively prime to n, and the public keys are chosen such that

$$v_i = (s_i^2)^{-1} \bmod n.$$



Proof of the verification. Since $s_i^2 v_i \equiv 1$ by the choice of v_i,

$$y^2 v_1^{c_1} \cdots v_k^{c_k} = r^2 (s_1^{c_1})^2 \cdots (s_k^{c_k})^2 v_1^{c_1} \cdots v_k^{c_k} = x\,(s_1^2 v_1)^{c_1} (s_2^2 v_2)^{c_2} \cdots (s_k^2 v_k)^{c_k} = x \cdot 1^{c_1} \cdots 1^{c_k} = x \pmod n.$$

Note the sign convention: Fiat–Shamir uses $v = s^2$ and checks $y^2 = xv^c$, whereas Feige–Fiat–Shamir uses the inverse $v_i = s_i^{-2}$ and moves the v's to the left: $y^2 \prod v_i^{c_i} = x$. The two are equivalent.

Cheating probability and why it is faster. Eve can still prepare for one guessed challenge vector c': she sends $x = r^2 \prod v_i^{c_i}$ and, if $c = c'$, answers $y = r$. But now the guess must be right in all k bits, so she passes a round with probability $1/2^k$ instead of 1/2. One round therefore gives the assurance of k Fiat–Shamir rounds: for $k = 5$, each round has cheating probability 1/32, and 4 rounds give $2^{-20} = 9.54 \times 10^{-7}$, the same as 20 Fiat–Shamir rounds, with one fifth of the message exchanges. The price is a larger key (k secrets and k public values) and up to k extra multiplications per round. The original paper also lets Alice attach a random sign, $x = \pm r^2$; Forouzan and the slides omit it.

WORKED EXAMPLE : Feige–Fiat–Shamir with $k = 3$

Q. $n = 143$ ($= 11 \times 13$). Alice's private keys are $[3, 5, 7]$. (a) Compute her public keys. (b) She picks $r = 10$ and Bob sends $[1, 0, 1]$. Compute x, y and Bob's check.
A. (a) $v_i = (s_i^2)^{-1} \bmod 143$:
$s_1^2 = 9$, $9^{-1} = \mathbf{16}$ ($9 \cdot 16 = 144 \equiv 1$); $s_2^2 = 25$, $25^{-1} = \mathbf{103}$ ($25 \cdot 103 = 2575 = 18 \cdot 143 + 1$); $s_3^2 = 49$, $49^{-1} = \mathbf{108}$ ($49 \cdot 108 = 5292 = 37 \cdot 143 + 1$).
(b) $x = 10^2 \bmod 143 = \mathbf{100}$. With $c = [1, 0, 1]$: $y = r s_1 s_3 = 10 \cdot 3 \cdot 7 = 210 \bmod 143 = \mathbf{67}$.
Bob: $y^2 = 4489 = 31 \cdot 143 + 56 \to 56$; $56 \cdot v_1 = 56 \cdot 16 = 896 \to 38$; $38 \cdot v_3 = 38 \cdot 108 = 4104 = 28 \cdot 143 + 100 \to \mathbf{100} = x$ ✓.
Check: a cheater would have needed to guess all three bits, probability $1/8$.

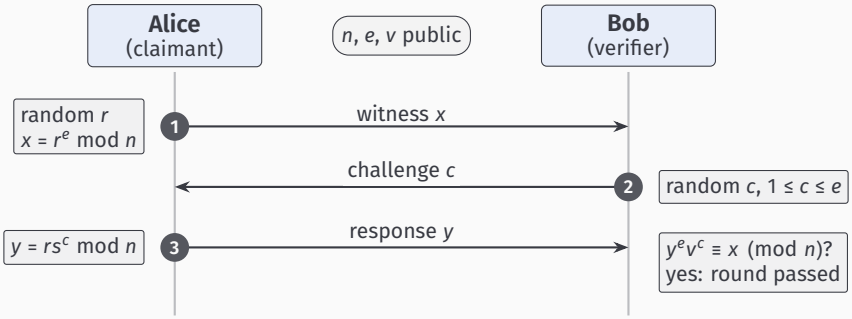
40.9 Guillou–Quisquater protocol ▸

▸ **Likely this year:** "Explain Guillou–Quisquater; prove $y^e v^c = x$; why does it need fewer rounds?" (3–4 marks).

The **Guillou–Quisquater** protocol is an extension of Fiat–Shamir in which fewer rounds can be used to prove the identity of the claimant: instead of squaring, it raises to a public exponent e, so the challenge can take e values rather than two. A trusted third party chooses two large primes p and q, calculates $n = p \times q$, and chooses an exponent e coprime with $\phi = (p - 1)(q - 1)$. The values of n and e are announced to the public; p and q are kept secret. The trusted party chooses two numbers for each entity, v, which is public, and s, which is secret, with the relationship

$$s^e \times v \equiv 1 \pmod{n}, \qquad \text{i.e. } v = (s^e)^{-1} \bmod n.$$

Recovering s from v means taking an e-th root modulo n, which is the RSA problem. (In the identity-based form, v is derived from Alice's identity and the party computes $s = v^{-d}$ with $d = e^{-1} \bmod \phi$, which is why it must know ϕ.)



Proof of the verification.

$$y^e \times v^c = (r \times s^c)^e \times v^c = r^e \times s^{ce} \times v^c = r^e \times (s^e \times v)^c = x \times 1^c = x \pmod{n}.$$

Why fewer rounds. A cheater can again prepare for one guessed challenge c' only: she picks y and sends $x = y^e v^{c'}$, which passes if and only if $c = c'$. Answering two different challenges $c_1 \neq c_2$ for the same x would give $(y_1 y_2^{-1})^e \equiv v^{c_2 - c_1}$, from which (since $\gcd(e, c_2 - c_1) = 1$ for a prime e) an e-th root of v can be computed, i.e. the RSA problem would be solved. Since c has e possible values, a cheater passes a round with probability $1/e$. With $e = 65537$, one round already gives 1.5×10^{-5} and two rounds $1/65537^2 = 2.33 \times 10^{-10}$. Each party stores only one secret and one public value, and very few messages are exchanged, which is why Guillou and Quisquater designed it for smart cards with little memory and bandwidth; the price is an exponentiation to the power e in place of one squaring.

WORKED EXAMPLE : Guillou–Quisquater

Q. The trusted party picks $p = 11$, $q = 17$, $e = 7$. Alice's secret is $s = 5$. (a) Find n and Alice's public key v. (b) Alice picks $r = 10$ and Bob sends $c = 3$. Compute x, y and verify. (c) How many rounds are needed for a cheating probability below 10^{-6}?
A. (a) $n = 187$, $\phi = 10 \times 16 = 160$, $\gcd(7, 160) = 1$. $s^e = 5^7 \bmod 187$: $5^2 = 25$, $5^4 = 625 \to 64$, $5^7 = 5^4 \cdot 5^2 \cdot 5 = 64 \cdot 25 \cdot 5$: $1600 \to 104$, $104 \cdot 5 = 520 \to 146$. Extended Euclid on $(187, 146)$ (quotients $1, 3, 1, 1, 3, 1, 1, 2$) ends with $t_1 = -73$, so $v = 146^{-1} = -73 + 187 = \mathbf{114}$. Check: $146 \cdot 114 = 16\,644 = 89 \cdot 187 + 1$ ✓.
(b) $x = 10^7 \bmod 187$: $10^2 = 100$, $10^4 = 10\,000 \to 89$, $89 \cdot 100 = 8900 \to 111$, $111 \cdot 10 = 1110 \to \mathbf{175}$.
$y = rs^c = 10 \cdot 5^3 = 1250 \bmod 187 = \mathbf{128}$.
Bob: y^7: $128^2 = 16\,384 \to 115$, $115^2 = 13\,225 \to 135$, $135 \cdot 115 = 15\,525 \to 4$, $4 \cdot 128 = 512 \to 138$. v^3: $114^2 = 12\,996 \to 93$, $93 \cdot 114 = 10\,602 \to 130$.
$y^e v^c = 138 \cdot 130 = 17\,940 = 95 \cdot 187 + 175 \to \mathbf{175} = x$ ✓.
(c) Per round $1/7$; we need $7^k > 10^6$: $7^7 = 823\,543 < 10^6 < 7^8 = 5\,764\,801$, so **8 rounds** (Fiat–Shamir needs 20).

	Fiat–Shamir	Feige–Fiat–Shamir	Guillou–Quisquater
Secret / public key	s; $v = s^2 \bmod n$	$s_1, \dots, s_k$; $v_i = (s_i^2)^{-1} \bmod n$	s; v with $s^e v \equiv 1 \pmod n$
Public parameters	n	n	n, e
Witness	$x = r^2$	$x = r^2$	$x = r^e$
Challenge	one bit $c \in \{0, 1\}$	k bits $c_1, \dots, c_k$	$c \in \{1, \dots, e\}$
Response	$y = rs^c$	$y = r \prod s_i^{c_i}$	$y = rs^c$
Verification	$y^2 \equiv xv^c$	$y^2 \prod v_i^{c_i} \equiv x$	$y^e v^c \equiv x$
Cheating per round	$1/2$	$1/2^k$	$1/e$
Rounds for $< 10^{-6}$	20	$\lceil 20/k \rceil$ (4 for $k = 5$)	2 for $e = 65537$
Hard problem	square roots mod n (factoring)	square roots mod n (factoring)	e-th roots mod n (RSA)

All three protocols are zero-knowledge for the same reason (a cheater's transcript is indistinguishable from a real one), and all rely on the claimant using a fresh random r in every round. Replacing Bob's random challenge by a hash of the witness and a message turns them into signature schemes, which is known as the Fiat–Shamir heuristic.

40.10 Biometrics ▸

▸ **Likely this year:** "Explain biometric authentication: enrollment, verification and identification, physiological and behavioural techniques, FAR and FRR" (4–5 marks); a small FAR/FRR computation.

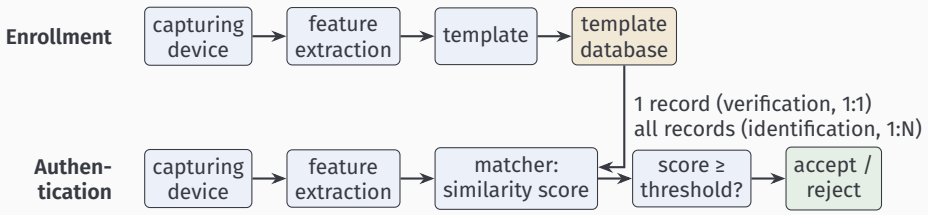
Biometrics is the measurement of physiological or behavioural features that identify a person; it is authentication by *something inherent*. A **physiological** feature is part of the body's structure (fingerprint, iris, face); a **behavioural** feature is the way a person does something (signing, typing). Unlike a password or a card, the feature cannot be forgotten, lost or lent to a colleague, and it is very hard to guess; this is what the slides mean by features that "cannot be guessed, stolen, or shared".

Components. A biometric system needs **capturing devices** (readers or sensors: a fingerprint scanner, a camera, a microphone, a signature pad) that sense the feature; **processors** that extract the distinctive characteristics from the raw sample, turn them into a compact **template**, and compare templates; and **storage devices** that keep the templates created at enrollment, either in a central database or on a smart card carried by the user.

Enrollment. Before any biometric technique can be used for authentication, the corresponding feature of each person in the community must be available in the database; this is called **enrollment**. The person presents the feature several times under supervision, after their identity has been checked by other means, and the processor stores the resulting template with the person's identity.

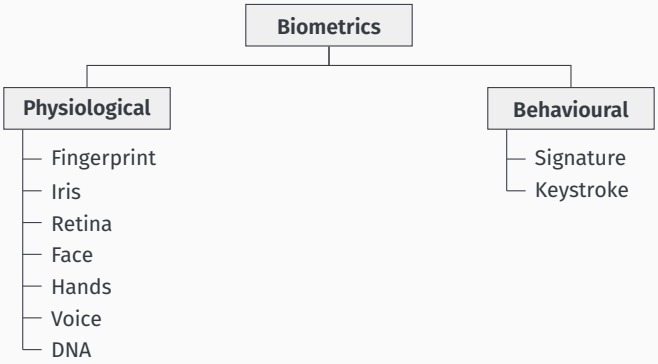
Authentication: verification and identification. Authentication is done in one of two ways.

- In **verification**, a person's feature is matched against a *single* record in the database (one-to-one matching) to find whether she is who she claims to be. The claimed identity (an ID, a card) selects the record. This is useful, for example, when a bank needs to verify a customer's signature on a cheque.
- In **identification**, a person's feature is matched against *all* records in the database (one-to-many matching) to find whether she has a record there, and whose. No identity is claimed. This is useful, for example, when a company needs to allow only its employees into the building.



Matching is approximate. A biometric sample is never exactly the same twice: finger pressure and angle, lighting, a cold or a new pair of glasses all change the reading. The system therefore cannot store a hash and test for equality, as it does with passwords. Instead the matcher computes a **similarity score** between the fresh sample and the template and accepts if the score is at least a chosen **threshold**. This is the source of the two kinds of error measured below, and it is why a stolen template is so damaging: it is stored in a usable, unhashed form.

Techniques. Biometric techniques are divided into physiological and behavioural ones.



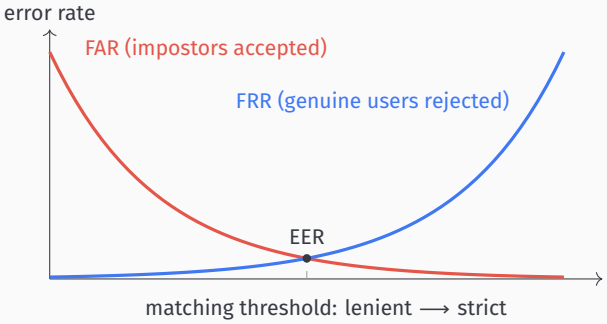
Technique	What is measured	Remarks
Fingerprint	ridges and valleys of the fingertip and their minutiae (ridge endings, bifurcations)	cheap and mature; cuts, dirt and worn skin cause rejections; prints can be lifted and copied
Iris	texture of the coloured ring around the pupil, from a camera	very distinctive, stable for life, contactless; costlier camera
Retina	pattern of blood vessels at the back of the eye	very accurate and hard to fake, but intrusive (eye close to the scanner)
Face	geometry of the face (distances between eyes, nose, mouth)	contactless, works from ordinary cameras; affected by lighting, expression, ageing, twins
Hands	hand geometry: lengths and widths of fingers and palm	simple and well accepted; not distinctive enough for 1:N identification
Voice	characteristics of the voice, from the shape of the vocal tract	usable over a telephone; affected by colds and noise, open to recordings
DNA	the DNA sequence of a cell sample	unique except for identical twins; needs a laboratory, not real time; forensics
Signature	dynamics of signing: speed, pressure, stroke order, on a pad	familiar; varies over time and can be imitated with practice
Keystroke	typing rhythm: how long keys are held, time between keys	no extra hardware; can monitor a whole session; varies with fatigue and keyboard

The slides (Forouzan) list voice under physiological, because it depends on the vocal tract; Stallings puts voice pattern under dynamic (behavioural) biometrics, because it also depends on how a person speaks. In the exam, follow the slide tree and mention that voice has both aspects.

Accuracy: FRR and FAR. The accuracy of biometric techniques is measured with two parameters.

- The **false rejection rate** (FRR) measures how often a person who should be recognised is not recognised by the system: $\text{FRR} = \dfrac{\text{number of false rejections}}{\text{number of genuine attempts}} \times 100\%$.
- The **false acceptance rate** (FAR) measures how often a person who should not be recognised is recognised by the system: $\text{FAR} = \dfrac{\text{number of false acceptances}}{\text{number of impostor attempts}} \times 100\%$.

A false rejection is an inconvenience (the genuine user tries again); a false acceptance is a security failure (an impostor gets in). The two rates are controlled by the same threshold and move in opposite directions. Raising the threshold demands a closer match, so fewer impostors pass (FAR falls) but more genuine users fail (FRR rises); lowering it does the reverse. The threshold at which the two rates are equal is the **equal error rate** (EER), a single number often used to compare systems: the lower the EER, the more accurate the technique. A bank vault is run at a high threshold (very low FAR, tolerating a higher FRR), a phone unlock or an attendance system at a lower one.



WORKED EXAMPLE : computing FAR and FRR

Q. A fingerprint system is tested with 1000 attempts by enrolled users, of which 30 are rejected, and 5000 attempts by impostors, of which 10 are accepted. (a) Find the FRR and the FAR. (b) The same matcher is used for identification against a database of 500 people, with the per-comparison FAR found in (a). What is the probability that an impostor matches at least one record?
A. (a) FRR = 30/1000 = **3%**; FAR = 10/5000 = **0.2%**.
(b) Each of the 500 comparisons is a false match with probability 0.002, so Pr[at least one] = $1 - (1 - 0.002)^{500}$ = $1 - 0.368$ = **0.632**.
Check: the expected number of false matches is $500 \times 0.002 = 1$, and $1 - e^{-1} = 0.632$. A FAR that is fine for 1:1 verification is useless for 1:N identification, which needs a far lower FAR.

Applications. Biometrics is used for physical access control (the employee-only building of the slides; laboratories, data centres); logical access to phones and laptops; banking (cheque-signature verification as on the slides, biometric ATMs, Aadhaar-enabled payments); government identity (e-passports and border control, India's Aadhaar with fingerprints and iris); forensics (fingerprints, DNA); and attendance systems.

Advantages. The feature is always with the user and cannot be forgotten, lost or lent, so authentication is tied to the person rather than to an object or a secret; it is also convenient and fast.

Disadvantages. A biometric **cannot be changed if stolen**: a compromised password is replaced in a minute, but a person has ten fingers and two irises for life, so a leaked template is permanent damage. Features can be copied (prints lifted from a glass, photographs of a face, recordings of a voice), which requires liveness detection. Matching is approximate, so FAR and FRR are never both zero. Sensors cost money, some people cannot enrol (worn fingerprints, injuries), and a central database of templates raises **privacy** concerns: it can be used to track people across systems, some features reveal health information, and the database is a high-value target. For these reasons a biometric is best used as one factor of a multi-factor scheme, with the template kept locally (on the phone or the smart card) rather than in a central store.

Corrections to the notes / slides

- Slide 13 (from Forouzan): "If the original password is 6 digits and the salt is 4 digits ... Eve now needs to make a list of 10 million items." A 10-digit value has 10^{10} (ten billion) possible values, not 10^7; the precomputed list has 10^{10} entries. Also note that an attack on *one* user, whose salt is read from the file, still needs only 10^6 hashes.
- Slide 15: "This kind of password makes eavesdropping and salting useless" means that eavesdropping gains nothing and salting becomes *unnecessary*, not that the one-time password defeats salting.
- Slide 23 says only that s lies between 1 and $n - 1$; it must also be coprime to n, since otherwise $\gcd(v, n) > 1$ reveals a factor of n. Slide 24 lets r be 0: Bob must reject $x = 0$, because the pair $x = 0$, $y = 0$ passes for both challenges.
- Slides 24–27 follow Forouzan's naming: r is the "commitment" and x the "witness". Most other texts call x itself the commitment (or witness). Slide 27 also has "passes the rest" for "passes the test".
- Slide 28, second line of the proof: the last factor is printed $(s_2^2)^{c_2}(v_2^{c_k})$; it should be $(s_k^2)^{c_k}(v_k^{c_k})$.
- Slide 32: "features that cannot be guessed, stolen, or shared" is too strong. They cannot be forgotten or lent, but they can be copied (lifted fingerprints, photographs, recordings), and once copied they cannot be changed.
- Slide 34 divides both FRR and FAR by "the total number of attempts". FRR should be divided by the number of *genuine* attempts and FAR by the number of *impostor* attempts; dividing by the total mixes the two populations. Slide 33 also misspells "Behavioural" as "Behaviorial".

Part VI
Beyond the Core

41 Other Cryptographic Techniques: DNA, Quantum, Information Hiding

▸ **Likely this year:** Last lecture of the course and never examined before: expect one "explain" question (BB84 with a table, or DNA encryption steps, or steganography vs watermarking vs cryptography) worth 4–6 marks, and short notes on the others.

Every cipher so far rests on a *mathematical* assumption: that a key search is too long (DES, AES) or that factoring or discrete logarithms are hard (RSA, ElGamal, ECC). Lecture 18 surveys three directions that leave this pattern. **DNA cryptography** uses molecules as the storage and computing medium; **quantum cryptography** replaces the hardness assumption by a law of physics; and **information hiding** (steganography and watermarking) protects a

message by hiding that it exists at all, or by binding it invisibly to a cover object.

41.1 DNA cryptography: why use DNA at all ▸

▸ **Likely this year:** "What is DNA cryptography? List the advantages of DNA computing over conventional computing" (3 marks).

DNA cryptography is the use of DNA (deoxyribonucleic acid) either as the *carrier* of encrypted information or as the *computing medium* for a cryptographic operation. It combines cryptology with modern biotechnology, and the slides call it a "promising and rapidly emerging field" that "may bring forward a new hope for unbreakable algorithms". The reason for interest is that a test tube of DNA behaves very differently from a silicon processor, in three ways listed on slide 3.

Speed through parallelism. A conventional processor executes one instruction stream; the slide puts it at about 10^8 instructions per second, while combining DNA strands is predicted to reach about 10^9 operations per second. The real source of speed is not the rate of a single reaction (each reaction takes minutes or hours) but **massive parallelism**: a small tube holds trillions of DNA strands, and one chemical step acts on all of them at the same time. A problem that needs a search over many candidates can therefore try all candidates at once, one candidate per molecule.

Storage density. DNA stores information at about 1 bit per nm^3, whereas conventional media need about 10^{12} nm^3 per bit, so DNA is about 10^{12} times denser. The same conclusion follows from the mass of DNA: each nucleotide carries 2 bits (four possible bases) and weighs about 330 g/mol, so one gram of single-stranded DNA holds

$$\frac{6.022 \times 10^{23}}{330} \times 2 \approx 3.65 \times 10^{21} \text{ bits} \approx 4.56 \times 10^{20} \text{ bytes} \approx 456 \text{ exabytes}.$$

Power. The slide claims DNA computing "does not require power" while the computation takes place, because the chemical reactions that build and pair DNA run on their own chemical energy. This is true of the hybridisation step itself, but the surrounding laboratory steps (heating cycles for PCR, electrophoresis) do use power.

New hard problems. The security of a DNA scheme can rest on a biological rather than a mathematical difficulty: for example, a secret short DNA sequence (a *primer*) is needed to copy and read a message strand hidden among billions of decoy strands, and without it Eve does not know which molecule to amplify. Three kinds of scheme exist in the literature: **DNA one-time pads** (Gehani, LaBean and Reif), in which the pad is a library of random DNA strands; **DNA steganography** (Clelland, Risca and Bancroft, 1999), which hid the message "JUNE6 INVASION: NORMANDY" in a DNA strand mixed into human DNA and printed it as a microdot; and **pseudo-DNA cryptography**, which uses only the DNA *encoding rules* on an ordinary computer. The algorithm on slides 7–11 is of the third kind.

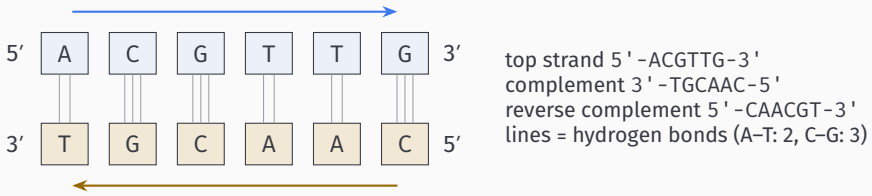
41.2 DNA structure and complementary base pairing ▸

▸ **Likely this year:** "Explain the structure of DNA and the complementary rule" (2 marks).

A DNA molecule is a chain of **nucleotides**. Each nucleotide has three parts: a phosphate group, a deoxyribose sugar, and one of four nitrogenous **bases**: **adenine (A), cytosine (C), guanine (G) and thymine (T)**. The phosphate of one nucleotide bonds to the sugar of the next, so the sugar–phosphate chain forms the **backbone** of a strand, and the bases stick out from it. The sequence of bases along the strand is the information; the backbone is the same everywhere.

The backbone has a direction. Its two ends are chemically different and are called the **5′ end** and the **3′ end** (after the numbering of the carbon atoms of the sugar), and a sequence is always written from 5′ to 3′. A DNA molecule normally consists of two strands wound around each other in a **double helix** (Watson and Crick, 1953). The two strands run in opposite directions (they are *antiparallel*), and they are held together by hydrogen bonds between facing bases.

Complementary base pairing. The bases only pair in two ways: A pairs with T (two hydrogen bonds) and C pairs with G (three hydrogen bonds). A and G are larger two-ring bases (purines) and C and T are one-ring bases (pyrimidines), so every pair is one large plus one small base and the helix has a constant width. As a result, one strand completely determines the other, which is called its **complement**. Since the partner runs the other way, the partner read 5′ →3′ is the **reverse complement**.



top strand 5′-ACGTTG-3′
complement 3′-TGCAAC-5′
reverse complement 5′-CAACGT-3′
lines = hydrogen bonds (A–T: 2, C–G: 3)

41.3 Fundamentals of DNA computing ▸

▸ **Likely this year:** "Explain Adleman's experiment" or "list the basic operations of DNA computing" (3–4 marks).

DNA computing replaces electronic operations on bits by laboratory operations on strands. The basic toolbox is the following.

- **Synthesis.** Short single strands (*oligonucleotides*, about 10–100 bases) of any chosen sequence are made chemically. This is how data are written.
- **Denaturation and annealing (hybridisation).** Heating to about 95 °C breaks the hydrogen bonds and separates the two strands; on cooling, every single strand searches the tube and binds to its complement. This is the "compare" operation of DNA computing, done by all molecules in parallel.
- **Ligation.** The enzyme ligase joins two strands lying end to end on a common complementary strand into one continuous backbone, so short pieces are concatenated into long ones.
- **PCR (polymerase chain reaction).** Two short *primers* mark the start and end of a target sequence; each heating–cooling cycle copies every strand that has both primer sites, so the number of copies doubles per cycle and n cycles give 2^n copies ($2^{30} \approx 1.07 \times 10^9$ after 30 cycles). Only strands with the right ends are amplified, which acts as a filter.
- **Gel electrophoresis.** DNA is negatively charged, so in an electric field it moves through a gel towards the positive electrode, and shorter strands move faster. This separates strands by length.
- **Affinity separation and sequencing.** Strands containing a given subsequence are pulled out with complementary probes fixed to magnetic beads; finally the surviving strands are read base by base (sequenced).

Adleman's experiment (1994). Leonard Adleman (the "A" of RSA) solved a small instance of the **directed Hamiltonian path problem**: given a directed graph with 7 vertices and designated start and end vertices, is there a path that visits every vertex exactly once? The problem is NP-complete, and a computer solves it by trying candidate paths one by one. Adleman instead built all paths at once.

1. Each vertex i was encoded as a random 20-base strand O_i. Each edge $i \to j$ was encoded as a 20-base strand made of the last 10 bases of O_i followed by the first 10 bases of O_j, so that the complement of O_j can hold edge $i \to j$ and edge $j \to k$ side by side.
2. Many copies of all edge strands and all vertex complements were mixed with ligase. Hybridisation and ligation joined edges into long strands, each one a random path through the graph; trillions of random paths formed in one step.
3. PCR with primers for the start and end vertices amplified only the paths that begin and end correctly.
4. Gel electrophoresis kept only strands of length $7 \times 20 = 140$ bases, i.e. paths through exactly 7 vertices.
5. Affinity separation was run once per vertex, each time keeping only strands that contain that vertex. A strand that survives all 7 rounds visits every vertex, and since it has only 7 vertices' worth of length, it visits each exactly once.
6. If any DNA remained, it was amplified and sequenced: the answer is "yes" and the sequence is the path.

The chemistry took about a week, which shows both sides of DNA computing: the search itself was fully parallel, but the input/output steps are slow manual laboratory work. Moreover, the number of molecules needed grows with the number of candidate paths, i.e. exponentially in the graph size, so a 70-vertex graph would need more DNA than is practical. Boneh, Dunworth and Lipton (1995) used the same ideas on paper to show how a DNA computer could test all 2^{56} DES keys in parallel.

41.4 Encoding binary data and ASCII text as DNA ★▸

▸ **Likely this year:** "How many binary encodings of A, C, G, T exist and how many respect complementarity? Encode a word in DNA" (3 marks).

Since there are four bases, each base carries $\log_2 4 = 2$ bits. The standard rule (slide 5) is

$$\mathbf{A} = 00, \qquad \mathbf{C} = 01, \qquad \mathbf{G} = 10, \qquad \mathbf{T} = 11,$$

so an 8-bit ASCII character becomes 4 bases. For example H = 72 = `01 00 10 00` = CAGA.

Why 24 encodings, of which only 8 are complementary. The four 2-bit values can be assigned to the four bases in $4! = 24$ ways. The binary digits 0 and 1 are complements of each other, so `00`–`11` and `01`–`10` are the two complementary pairs of 2-bit values. An encoding respects the biology if complementary bases get complementary values, i.e. the pair {A, T} gets one complementary pair of values and {C, G} the other. There are 2 ways to choose which value pair goes to {A, T}, 2 ways to order it inside (A = `00`, T = `11` or the reverse), and 2 ways to order the other pair inside {C, G}: $2 \times 2 \times 2 = 8$ suitable encodings. The standard rule is one of them, and with it **the Watson–Crick complement of a strand is exactly the bitwise NOT of its binary code**: A = `00` ↔ `11` = T and C = `01` ↔ `10` = G.

★ MUST KNOW : DNA encoding

A = 00, C = 01, G = 10, T = 11; 2 bits per base, 4 bases per ASCII character. Base pairs A–T and C–G. $4! = 24$ encodings, $2 \times 2 \times 2 = 8$ complementary ones. Slide 9's "complementary rule" is A→C→G→T→A, i.e. *add 1 mod 4* to each 2-bit group.

The slide's ASCII-to-DNA table (slide 6). Slide 9 explains that a key selects one of many substitution tables mapping each printable ASCII character (codes 32–127) to a 4-base word; slide 6 shows one such table. It is *not* the binary rule above (that rule would give A ↦ CAAC, whereas the table gives TACT): it is a keyed code book. Transcribed exactly:

S	DNA	S	DNA	S	DNA	S	DNA	S	DNA	S	DNA
sp	ACAT	0	CAAA	@	CCAC	P	TTCA	`	TCCG	p	GACA
!	AGGT	1	CACC	A	TACT	Q	TTTA	a	GAGC	q	GACT
"	AAAG	2	CCGT	B	TCCT	R	TAGA	b	GTGC	r	GGAT
#	AGAC	3	CGAG	C	TACG	S	TGAG	c	GACG	s	GGTG
$	AAGC	4	CCTT	D	TGCC	T	TAAA	d	GTAA	t	GCTT
%	AACT	5	CCGT	E	TCTA	U	TGAC	e	GTAC	u	GACC
&	AGAA	6	CTGT	F	TAGT	V	TGAG	f	GCCT	v	GACT
'	AATC	7	CTCT	G	TTAA	W	TAAC	g	GCTA	w	GCCC
(	ATTG	8	CCGT	H	TGGC	X	TCCT	h	GAGT	x	GATC
)	AATT	9	CTCA	I	TGTT	Y	TGAA	i	GATG	y	GTCG
*	AATG	:	CTAG	J	TTCC	Z	TAAG	j	GATT	z	GTGA
+	AAGA	;	CCGC	K	TACT	[	TCAT	k	GGGC	{	GGCT
,	AGAG	<	CACA	L	TATG	\	TAAG	l	GTTG	|	GGTG
-	AAGC	=	CATA	M	TAGT	]	TCCA	m	GTGA	}	GAAC
.	ACAC	>	CTAC	N	TGTC	^	TGTT	n	GACT	~	GATG
/	ACGT	?	CCAG	O	TATT	_	TCCG	o	GCCG	DEL	GAGT

Rows run down each column in ASCII order (codes 32–47, 48–63, ..., 112–127); sp = space, DEL = code 127 (printed `\x7f` on the slide). **Shaded cells share their 4-base word with another character**: only 80 of the 96 words are distinct (e.g. TACT is both A and K; CCGT is 2, 5 and 8), so the table as printed cannot be decoded uniquely; see the corrections box.

Using this table, HI ↦ TGGC TGTT. A code book like this is a monoalphabetic substitution on characters; on its own it would fall to frequency analysis, which is why the scheme combines it with a one-time-pad XOR.

41.5 The DNA cryptography algorithm (slides 7–11) ★▸

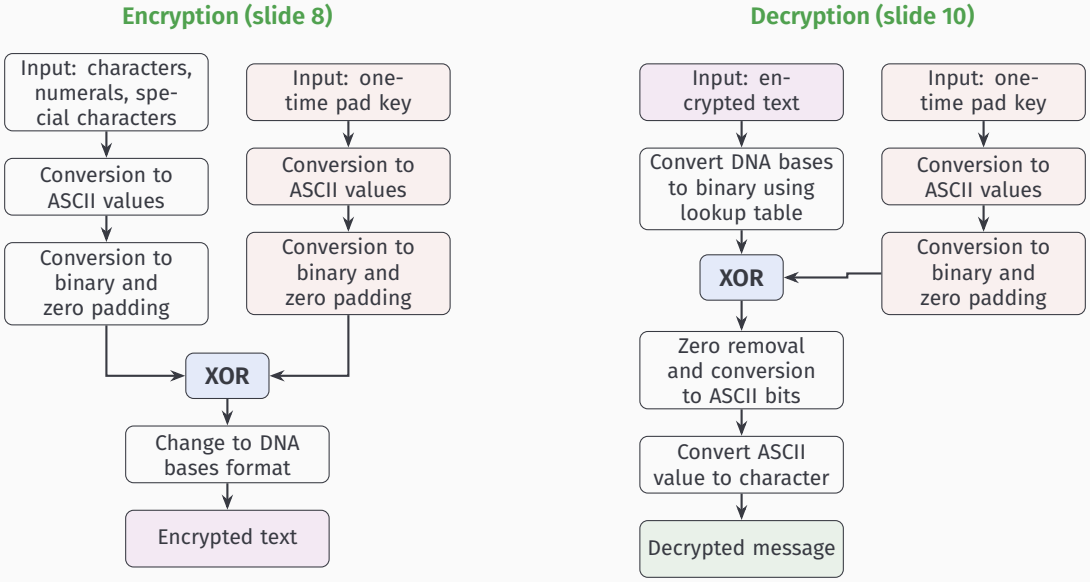
▸ **Likely this year:** "Explain the DNA cryptography algorithm with its encryption and decryption flowcharts, and encrypt a short word" (5–6 marks).

Table-based set-up (slide 7). The sender generates a DNA encoding table (a table like slide 6), and the receiver generates another table with the same technique. Instead of sending the whole table, each party sends a short *clue* (the seed or key number from which the table is generated), so the other party can rebuild the table locally. The plaintext is divided into two equal halves, with random padding if its length is odd; one half is converted into a DNA sequence with the sender's table and the other half with the receiver's table. Here DNA is used purely as an information carrier for end-to-end communication.

The algorithm (slide 9). The bit-level version, which is what is examined, runs as follows.

1. Convert each plaintext character to its ASCII code and write it as an 8-bit binary number.
2. Represent the binary data in DNA form with A = 00, C = 01, G = 10, T = 11 (4 bases per character).
3. Apply the slide's complementary rule to every base: A → C, C → G, G → T, T → A.
4. Convert the resulting DNA sequence back to binary.
5. Generate the key and convert it to binary. On slide 9 it is a random number between 1 and 256 that also selects the ASCII-to-DNA table (key 1 gives one table, key 2 another, and so on); in the flowchart of slide 8 it is a **one-time-pad key** that is converted character by character to ASCII and then to 8-bit binary (with zero padding to full bytes).
6. XOR the key bits with the data bits.
7. Change the result to DNA bases (A = 00, ...) to obtain the encrypted text (slide 8).

Decryption is the reverse operation (slides 10–11): convert the received bases to binary with the lookup table, XOR with the same key, convert to DNA and undo the complementary rule (C → A, G → C, T → G, A → T), remove the zero padding, and convert each 8-bit group back to its ASCII character.



The flowcharts omit the complementary-rule step of slide 9. Since that step is a fixed public substitution, including it changes the ciphertext but not the security; the worked example below follows slide 9 (with the rule) and slide 8 (one-time-pad key, final conversion to DNA).

Why decryption recovers the plaintext. Write x for the 8-bit code of a character, σ for steps 2–4 and k for the key byte. In binary, the rule A→C→G→T→A sends 00 → 01 → 10 → 11 → 00, i.e. it *adds 1 modulo 4 to each 2-bit group*; this is a bijection, and its inverse subtracts 1 modulo 4 (C→A, G→C, T→G, A→T). The ciphertext byte is $c = \sigma(x) \oplus k$. The receiver computes $c \oplus k = \sigma(x) \oplus k \oplus k = \sigma(x)$, since $k \oplus k = 0$, and then $\sigma^{-1}(\sigma(x)) = x$. Both steps are exactly invertible, so the plaintext is recovered bit for bit.

WORKED EXAMPLE : DNA encryption and decryption of "HI"

Q. Using the DNA cryptography algorithm (A = 00, C = 01, G = 10, T = 11; complementary rule A→C, C→G, G→T, T→A) and the one-time-pad key "OK", encrypt the plaintext "HI" and show that decryption recovers it.
Encryption. Steps 1–4 on the plaintext, then steps 5–7 with the key:

Char	ASCII	Binary (step 1)	DNA (step 2)	Rule (step 3)	Binary $\sigma(x)$ (step 4)
H	72	01001000	CAGA	GCTC	10011101
I	73	01001001	CAGC	GCTG	10011110

Key char	ASCII	Key bits k	$\sigma(x)$	$c = \sigma(x) \oplus k$	DNA (step 7)
O	79	01001111	10011101	11010010	TCAG
K	75	01001011	10011110	11010101	TCCC

Ciphertext: **TCAGTCCC**.
Decryption. Bases to binary, XOR with the key, back to DNA, inverse rule (C→A, G→C, T→G, A→T), binary, ASCII:

DNA	Binary	$\oplus k$	DNA	Inverse rule	Binary	Char
TCAG	11010010	10011101	GCTC	CAGA	01001000 = 72	H
TCCC	11010101	10011110	GCTG	CAGC	01001001 = 73	I

Recovered plaintext: **HI**. *Check*: each base of GCTC is one step after the base of CAGA in the cycle A→C→G→T→A, and 10011101 ⊕ 01001111 = 11010010 bit by bit.

Security analysis. By Kerckhoffs's principle Eve knows the algorithm, so she can undo steps 2–4 and 7 herself: the DNA encoding and the complementary rule are public bijections with no key, and they add no secrecy. All the security therefore comes from the XOR with the key. If the key is a true one-time pad (random, as long as the message, never reused), c is uniformly distributed whatever the plaintext, and the scheme has the perfect secrecy of the one-time pad (§15). If instead the key is a single number between 1 and 256 XORed into every byte, there are only 256 keys, so Eve tries them all; and one known plaintext character gives the key at once as $k = \sigma(x) \oplus c$. Slide 11's claim of a key space of about 10^{23} ($\approx 2^{76}$) is far larger than 256 and is not derived on the slides; even taken at face value it is below the 2^{128} expected of modern symmetric ciphers.

Advantages. DNA offers enormous storage density (about 10^{12} times that of conventional media), massive parallelism for search-type problems, very low energy per operation, and long-term stability of stored molecules.

Biological schemes (DNA one-time pads, DNA steganography) add a physical barrier: without the right primers Eve cannot even locate the message strand.

Limitations. Laboratory DNA cryptography needs expensive equipment and trained staff; synthesis, PCR and sequencing are slow (hours to days per operation) and have non-negligible error rates (mismatched hybridisation, synthesis and sequencing errors), so error-correcting redundancy is needed. The parallelism does not change exponential problems into polynomial ones, it only gives a large constant factor bought with an exponential amount of DNA. Finally, there is no theoretical security model comparable to number-theoretic or information-theoretic proofs, and the purely digital "pseudo-DNA" schemes are only as strong as the classical cipher (here the XOR) inside them. For these reasons DNA cryptography is still a research area, not a practical replacement for AES or RSA.

41.6 Quantum cryptography: security from physics ▸

▸ **Likely this year:** "What is quantum cryptography and how does it differ from classical cryptography?" (2–3 marks).

Every public-key scheme in this course is secure only as long as a mathematical problem stays hard, and none of these problems has been proved hard; a better algorithm or a new kind of computer could break them, even years after the ciphertext was recorded. **Quantum cryptography** uses the laws of quantum physics instead, to build a cryptosystem that cannot be compromised *without the knowledge of the sender or the receiver*. The word *quantum* refers to the behaviour of the smallest particles of matter and energy, such as single photons. Richard Feynman proposed in 1982 a *quantum computer*, a computer that uses the effects of quantum mechanics to its advantage; quantum cryptography is the related idea of using the same effects to protect communication.

The practical form of quantum cryptography is **quantum key distribution (QKD)**. It does not encrypt the message itself; it lets two remote parties create a shared random key over a channel that Eve can attack, with the guarantee that any eavesdropping shows up as errors. The key is then used in a one-time pad or in AES. The difference from classical cryptography is therefore the *basis of security*: physics (an eavesdropper must measure, and measuring disturbs) rather than mathematics (an eavesdropper must solve a hard problem).

41.7 Properties of quantum information ▸

▸ **Likely this year:** "State the properties of quantum information used in quantum cryptography; state and justify the no-cloning theorem" (3–4 marks).

Qubit and superposition. In quantum cryptography information is carried by a **quantum bit (qubit)**, which in practice is a single photon. A classical bit is 0 or 1; a qubit can be in a **superposition** $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ of the two basis states, where α, β are (complex) amplitudes with $|\alpha|^2 + |\beta|^2 = 1$. For a photon the basis states are polarisations: horizontal (0°) is $|0\rangle$ and vertical (90°) is $|1\rangle$, and a photon polarised at 45° is the superposition $(|0\rangle + |1\rangle)/\sqrt{2}$.

Measurement disturbs the state (the slide's "Heisenberg uncertainty principle"). Measuring $|\psi\rangle$ in the basis $\{|0\rangle, |1\rangle\}$ gives 0 with probability $|\alpha|^2$ and 1 with probability $|\beta|^2$, and afterwards the qubit *is* in the state that was observed: the original amplitudes are lost. Hence the polarisation of a photon is only known at the point when it is measured, and an eavesdropper who measures a photon changes it.

Photon polarisation (the slide's second pillar). A photon polarised at angle θ that meets a polarising filter (or polarising beam splitter) set at angle φ is found along the filter with probability $\cos^2(\theta - \varphi)$, and is then polarised along the filter. If the filter is parallel ($\theta - \varphi = 0^\circ$) the photon is detected with certainty; if perpendicular (90°) it is certainly not; at 45° each outcome has probability $\cos^2 45^\circ = 1/2$, so the result is a fair coin flip that carries no information about θ. Two directions at 90° form a **basis**; BB84 uses the **rectilinear** basis + (0°, 90°) and the **diagonal** basis × (45°, 135°), which are at 45° to each other.

Basis	Bit 0	Bit 1	Measured in the other basis
Rectilinear +	0° (→)	90° (↑)	random 0/1, probability $\frac{1}{2}$ each
Diagonal ×	45° (↗)	135° (↖)	random 0/1, probability $\frac{1}{2}$ each

No-cloning theorem (Wootters–Zurek, Dieks, 1982): an unknown quantum state cannot be copied. Suppose a machine (a unitary operation U) could copy every state: $U(|\psi\rangle|0\rangle) = |\psi\rangle|\psi\rangle$ and $U(|\varphi\rangle|0\rangle) = |\varphi\rangle|\varphi\rangle$. Unitary operations preserve inner products, so taking the inner product of the two equations gives $\langle\varphi|\psi\rangle \cdot 1 = \langle\varphi|\psi\rangle^2$. Therefore $\langle\varphi|\psi\rangle$ is 0 or 1: the machine works only for states that are identical or orthogonal (i.e. for one known basis), never for all four BB84 states. Eve therefore cannot copy a photon, keep the copy, and forward the original untouched; she must measure, and measuring disturbs.

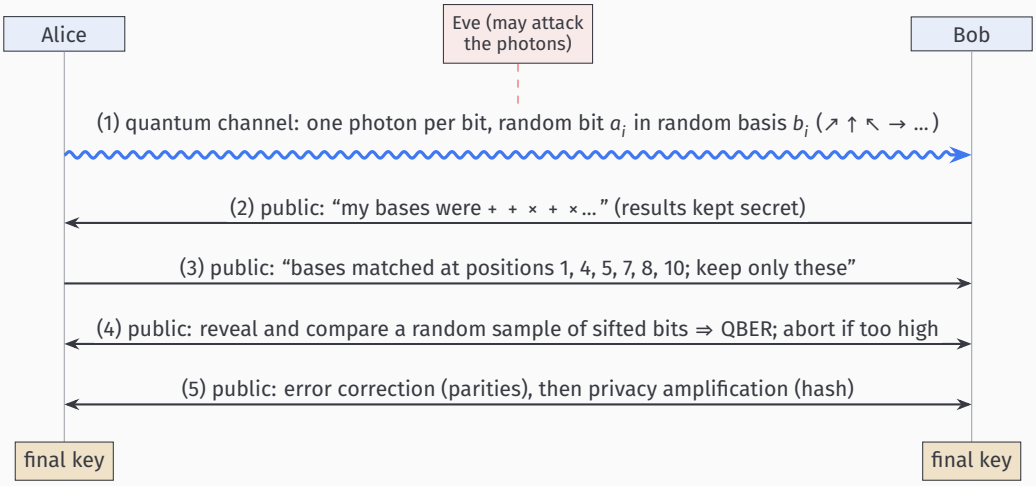
Entanglement. Two photons can be prepared in a joint state such as $(|00\rangle + |11\rangle)/\sqrt{2}$, in which neither photon has its own polarisation but the two are perfectly correlated: if both are measured in the same basis, the results are random but always equal, however far apart the photons are. Entanglement carries no message by itself, but it can give two parties identical random bits, which is the basis of the E91 protocol.

41.8 BB84 quantum key distribution ★▸

▸ **Likely this year:** "Explain the BB84 protocol with an example; how are eavesdroppers detected?" (5–6 marks). Draw the table.

BB84 (Bennett and Brassard, 1984) lets Alice and Bob agree on a secret key using two channels: a **quantum channel** (optical fibre or free space) that carries single polarised photons and that Eve may attack in any way, and a **public classical channel** that Eve can read but not alter, i.e. it must be *authenticated*.

1. **Preparation.** Alice generates a random bit string $a_1, \ldots, a_n$ and, independently, a random basis string $b_1, \ldots, b_n \in \{+, \times\}$.
2. **Transmission.** For each i she sends one photon polarised according to the table above: bit 0 as 0° or 45°, bit 1 as 90° or 135°, in basis b_i.
3. **Measurement.** Bob does not know b_i, so for each photon he chooses his own random basis b_i' and measures, recording the result.
4. **Sifting.** Over the public channel Bob announces his bases b_i' (*not* his results). Alice replies with the positions where $b_i' = b_i$. Both discard all other positions. Where the bases match, Bob's result equals Alice's bit with certainty (filter parallel or perpendicular); where they differ, his result is a coin flip and useless. Since the bases agree with probability $1/2$, about $n/2$ bits survive: the **sifted key**.
5. **Error estimation.** Alice and Bob publicly compare a random sample of the sifted bits and compute the **quantum bit error rate** (QBER), the fraction that disagree. Without Eve and noise it is 0. If it exceeds a threshold (about 11% for BB84), they abort, because Eve may know too much. The compared bits are discarded, since Eve has now seen them.
6. **Information reconciliation.** The remaining bits may still contain a few errors (noise or Eve). Alice and Bob correct them by exchanging parity information over the public channel.
7. **Privacy amplification.** Eve may know some bits or parities. Alice and Bob apply the same hash function to the corrected key and keep a shorter output, of which Eve's knowledge is negligible. The result is the final secret key.



Each public message matters. Message (2) reveals only bases, which say nothing about bit values because each basis carries both values equally often. Message (3) reveals positions, again not values. Message (4) is the eavesdropping test: it works because, as shown next, Eve cannot measure without causing errors. Message (5) removes the effect of noise and of whatever Eve learned. The public channel must be authenticated, otherwise Eve could sit in the middle and run BB84 separately with each party (a man-in-the-middle attack).

WORKED EXAMPLE : BB84 with 10 photons

Q. Alice sends 10 photons. Her bits are `0110100110` and her bases + × + + × × + × + ×. Bob measures with bases + + × + × + + × ××. Show the polarisations, Bob's results, the sifted key, and the final key if positions 4 and 8 are used for the error check. (Where Bob's basis differs, his result is random; the values shown are one possible outcome.)

Photon i	1	2	3	4	5	6	7	8	9	10
Alice's bit a_i	0	1	1	0	1	0	0	1	1	0
Alice's basis b_i	+	×	+	+	×	×	+	×	+	×
Polarisation sent	0°	135°	90°	0°	135°	45°	0°	135°	90°	45°
Bob's basis b'_i	+	+	×	+	×	+	+	×	×	×
Bob's result	0	0^r	1^r	0	1	1^r	0	1	0^r	0
Bases match?	✓	–	–	✓	✓	–	✓	✓	–	✓
Sifted key	0			0	1		0	1		0

(^{r} = random result, basis mismatch.) Sifted key (positions 1, 4, 5, 7, 8, 10): **001010**, 6 of 10 bits, close to the expected $n/2 = 5$. Alice and Bob reveal positions 4 and 8: Alice has 0, 1 and Bob has 0, 1, so QBER = 0/2 and no eavesdropping is indicated. They discard those two and keep positions 1, 5, 7, 10: final key **0100** (before privacy amplification). *Check*: at photon 3 Bob happened to get 1, equal to Alice's bit, but it is still discarded, because neither side can know a mismatched-basis result is right.

41.9 Why eavesdropping is detected: the intercept–resend attack ★▸

▸ **Likely this year:** "Show that an intercept–resend eavesdropper causes a 25% error rate in the sifted key" (3–4 marks).

The simplest attack is **intercept–resend**: Eve catches each photon, measures it in a basis of her own choice, and sends Bob a new photon prepared in the state she observed. She has to guess the basis, because the bases are announced only after Bob has measured, and she cannot copy the photon to measure later (no-cloning). Consider a bit that survives sifting, so Alice and Bob used the same basis.

- With probability 1/2 Eve guesses Alice's basis. She gets Alice's bit exactly, resends the same polarisation, and Bob (same basis as Alice) gets the correct bit. No error.
- With probability 1/2 Eve uses the wrong basis. Her result is random and she resends a photon at 45° to Alice's basis. Bob measures in Alice's basis, so his result is a coin flip: wrong with probability 1/2.

Therefore

$$\Pr[\text{error in a sifted bit}] = \tfrac{1}{2}\times 0 + \tfrac{1}{2}\times\tfrac{1}{2} = \boxed{\tfrac{1}{4} = 25\%}.$$

If Eve attacks only a fraction f of the photons, the error rate is $f/4$ (e.g. $f = 0.2$ gives 5%). In return Eve learns each sifted bit with probability $\frac{1}{2}\cdot 1 + \frac{1}{2}\cdot\frac{1}{2} = \frac{3}{4}$ (right basis: certainly; wrong basis: a coin flip). Since each compared bit exposes her with probability 1/4, the probability that m compared bits all agree although she attacked every photon is $(3/4)^m$, and Alice and Bob detect her with probability $1 - (3/4)^m$.

★ MUST KNOW : BB84 numbers

Bases agree with probability $\frac{1}{2}$ ⇒ sifted key ≈ $n/2$ bits. Full intercept–resend ⇒ QBER = $\frac{1}{2}\cdot\frac{1}{2}$ = 25% (fraction f: $f/4$). Eve escapes m test bits with probability $(3/4)^m$.

WORKED EXAMPLE : detecting Eve

Q. Eve performs intercept–resend on every photon. (a) What QBER do Alice and Bob observe? (b) If they compare 20 sifted bits, what is the probability that Eve is not detected? (c) How many bits must they compare to make that probability less than 10^{-9}? (d) In another run they compare 200 bits and find 47 mismatches; estimate the fraction of photons Eve attacked.

A. (a) 25%, as derived above.

(b) Each bit agrees with probability 3/4: $(3/4)^{20}$ = **0.00317**, so Eve is caught with probability 0.9968.

(c) Need $(3/4)^m < 10^{-9}$, i.e. $m > 9/\log_{10}(4/3) = 72.03$; indeed $(3/4)^{72} = 1.01\times 10^{-9}$, so m = **73** bits.

(d) QBER = 47/200 = 0.235; since QBER = $f/4$, $f \approx 4\times 0.235$ = **0.94**, about 94% of the photons. The run is aborted.

Check: a simulation of 2×10^5 photons with Eve on all of them kept 49.95% of the bits and gave QBER = 0.2500.

E91 (Ekert, 1991). In the entanglement-based protocol a source sends one photon of each entangled pair to Alice and the other to Bob. Each measures in a randomly chosen basis; when their bases agree, the results are perfectly correlated and form the key. On the rounds with other basis combinations they test a **Bell inequality**: correlations stronger than any classical (pre-determined) system allows prove that the photons are still entangled and therefore that nobody has measured them on the way. An eavesdropper breaks the entanglement, the Bell test fails, and the parties abort. The advantage is that even the source need not be trusted.

41.10 Quantum algorithms: Shor and Grover ★▸

▸ **Likely this year:** "Explain Shor's and Grover's algorithms and their impact on RSA, ECC and AES" (3–5 marks).

The slides call Shor's and Grover's algorithms "the two basic algorithms of quantum cryptography". More precisely, they are quantum algorithms for *cryptanalysis*: they run on a quantum computer and attack classical

ciphers. QKD does not use them; they are the reason classical cryptography must change.

Grover's algorithm (1996) finds, with high probability, the unique input to a black-box function that produces a particular output value. Classically, finding the one marked item among N needs about $N/2$ evaluations on average (N in the worst case), because each test rules out one candidate. Grover's algorithm needs only about $\frac{\pi}{4}\sqrt{N}$ evaluations: it starts from an equal superposition of all N inputs and, in each iteration, increases the amplitude of the marked input a little, so that after about $\frac{\pi}{4}\sqrt{N}$ iterations a measurement returns it with high probability. It is known to be optimal: no quantum algorithm can search an unstructured space faster than order $\sqrt{N}$. For a cipher with a k-bit key, $N = 2^k$, so a key search costs about $2^{k/2}$: Grover halves the effective key length. AES-128 drops to about $\frac{\pi}{4}2^{64} \approx 1.45 \times 10^{19}$ evaluations (64-bit security), while AES-256 keeps 2^{128}. The fix is simply to **double the key length**; symmetric cryptography survives.

Shor's algorithm (1994) gives a dramatic improvement in factoring: given an integer N, it finds its prime factors in polynomial time (about $O(n^3)$ elementary quantum operations for an n-bit N), whereas the best classical algorithm, the general number field sieve, is sub-exponential, $\exp\!\big(1.923\,(\ln N)^{1/3}(\ln\ln N)^{2/3}\big)$. The same method computes discrete logarithms in $\mathbb{Z}_p^*$ and on elliptic curves. Hence it breaks RSA, Diffie–Hellman, ElGamal, DSA and all of ECC.

How Shor factors: reduction to order finding. The quantum computer is used for one task only: given a with $\gcd(a, N) = 1$, find the **order** r of a modulo N, the smallest $r > 0$ with $a^r \equiv 1 \pmod N$. It does so by preparing a superposition of all exponents x, computing $a^x \bmod N$ in superposition (a periodic function of x with period r), and applying the quantum Fourier transform, which reveals the period. The rest is classical. If r is even,

$$a^r - 1 = (a^{r/2} - 1)(a^{r/2} + 1) \equiv 0 \pmod N,$$

so N divides the product. N does not divide $a^{r/2} - 1$ (otherwise $r/2$ would be a smaller order), and if also $a^{r/2} \not\equiv -1 \pmod N$, N does not divide $a^{r/2} + 1$ either. Then each factor shares a proper divisor with N, and $\gcd(a^{r/2} \pm 1, N)$ are non-trivial factors. If r is odd or $a^{r/2} \equiv -1$, a new random a is tried; at least half of the choices work.

WORKED EXAMPLE : the classical part of Shor's algorithm

Q. Factor $N = 15$ with $a = 7$ as Shor's algorithm does, and show why $a = 5$ fails for $N = 21$.
A. Powers of 7 modulo 15: $7^1 = 7$, $7^2 = 49 \equiv 4$, $7^3 \equiv 4 \times 7 = 28 \equiv 13$, $7^4 \equiv 13 \times 7 = 91 \equiv 1$. The order is $r = 4$ (this is the step the quantum computer performs). r is even and $a^{r/2} = 7^2 \equiv 4 \not\equiv -1 \pmod{15}$. Then $\gcd(4 - 1, 15) = \mathbf{3}$ and $\gcd(4 + 1, 15) = \mathbf{5}$, and $15 = 3 \times 5$.
For $N = 21$, $a = 5$: powers $5, 4, 20, 16, 17, 1$, so $r = 6$, but $5^3 \equiv 20 \equiv -1 \pmod{21}$, giving $\gcd(19, 21) = 1$ and $\gcd(21, 21) = 21$, both trivial. A new a is chosen: $a = 2$ has order 6 and $2^3 = 8$, so $\gcd(7, 21) = 7$ and $\gcd(9, 21) = 3$.
Check: $3 \times 5 = 15$ and $3 \times 7 = 21$.

Scheme	Classical hard problem	Quantum attack	Effect	Remedy
RSA	factoring	Shor, polynomial	broken	post-quantum KEM / signature
DH, ElGamal, DSA	discrete log mod p	Shor, polynomial	broken	post-quantum KEM / signature
ECDH, ECDSA, EC-ElGamal	elliptic-curve discrete log	Shor, polynomial	broken	post-quantum KEM / signature
AES-128	2^{128} key search	Grover, 2^{64}	weakened	use AES-256 (2^{128} left)
3DES (112-bit key)	2^{112} key search	Grover, 2^{56}	weak	retire; use AES-256
AES-256	2^{256} key search	Grover, 2^{128}	secure	none needed
SHA-256	preimage 2^{256}	Grover, 2^{128}	secure	SHA-384/512 for margin
One-time pad, QKD key	none (information-theoretic)	none	secure	none needed

How close is the threat? The slide mentions D-Wave as one of the companies making quantum computers a reality. D-Wave machines are *quantum annealers* built for optimisation problems; they cannot run Shor's algorithm. Gate-based machines have so far run Shor's algorithm only on toy numbers such as 15 and 21, and published estimates for factoring a 2048-bit RSA modulus need on the order of a million noisy physical qubits. Nevertheless the migration has already started, because of the **harvest-now, decrypt-later** threat: ciphertext recorded today can be decrypted once a large quantum computer exists, so data that must stay secret for decades needs quantum-safe protection now.

41.11 Post-quantum cryptography ▸

▸ **Likely this year:** "What is post-quantum cryptography? Differentiate it from quantum cryptography" (2–3 marks).

Post-quantum cryptography (PQC) consists of *classical* algorithms, running on ordinary computers, whose security rests on problems that are believed hard even for quantum computers, i.e. problems with no known Shor-like structure (no hidden period). After a public competition, NIST published the first standards in August 2024.

- **Lattice-based.** Security rests on finding short vectors in high-dimensional lattices, or on *learning with errors* (solving linear equations modulo q to which small random errors have been added). Standards: **ML-KEM** (FIPS

203, from CRYSTALS-Kyber) for key encapsulation, **ML-DSA** (FIPS 204, from CRYSTALS-Dilithium) for signatures, and Falcon (FN-DSA) to follow. Keys are about 1–2 KB and operations are as fast as or faster than RSA.

- **Hash-based signatures.** Security rests only on the preimage and collision resistance of a hash function (§35), the best-understood assumption of all. **SLH-DSA** (FIPS 205, from SPHINCS$^+$) is stateless; XMSS and LMS are stateful (the signer must never reuse a one-time key). Signatures are large (about 8 to 50 KB).
- **Code-based.** Security rests on decoding a random linear error-correcting code. The McEliece cryptosystem (1978) has never been broken but has public keys of hundreds of kilobytes to about a megabyte; HQC was selected by NIST in 2025 as a second, non-lattice KEM.
- **Multivariate and isogeny-based** schemes (systems of quadratic equations; maps between elliptic curves) were also studied; two candidates, Rainbow and SIKE, were broken by classical attacks in 2022, which shows why the standards were chosen conservatively.

	Post-quantum cryptography	Quantum cryptography (QKD)
Security from	mathematical problems believed hard for quantum computers	laws of physics (measurement disturbs, no-cloning)
Hardware	ordinary computers; a software upgrade	single-photon sources, detectors, dedicated fibre or satellite link
Distance	any (works over the Internet)	typically 100–200 km of fibre per link (a few hundred km in record experiments) without trusted relay nodes
Services	key exchange and digital signatures	key exchange only; still needs an authenticated classical channel
Proof	conjectured hardness	information-theoretic, if the devices behave as modelled

41.12 Challenges of quantum cryptography ▸

▸ **Likely this year:** "List and explain the challenges of quantum cryptography" (3 marks).

Expensive and complex. QKD needs specialised hardware (single-photon sources, single-photon detectors, often cooled, and stable optical links) at both ends of every link, and it cannot be added to existing equipment by a software update.

Difficult over long distances. Photons are absorbed in the fibre. A typical loss of 0.2 dB per km means a fraction $10^{-0.02L}$ of the photons survives L km: 10% after 50 km, 1% after 100 km, 10^{-4} after 200 km and 10^{-10} after 500 km, where a source sending 10^9 pulses per second would deliver only one photon every 10 seconds. Classical optical amplifiers cannot help, because amplifying an unknown quantum state would copy it, which the no-cloning theorem forbids. Long links therefore use chains of *trusted nodes* (each of which sees the key) or satellites; true quantum repeaters are still experimental.

Authentication is still classical. BB84 needs an authenticated public channel, otherwise Eve runs it separately with Alice and with Bob. Authentication needs a pre-shared secret key (or classical signatures), so QKD is really *key expansion*: it turns a short shared secret into a long one.

Low key rate and implementation attacks. Sifting, error estimation and privacy amplification discard most of the raw bits, so key rates are modest and fall quickly with distance. The security proof assumes ideal devices, and real ones leak: a weak laser pulse sometimes contains two photons, of which Eve can keep one (the photon-number-splitting attack, countered with *decoy states*), and bright light can "blind" detectors so that Eve controls their clicks.

Decoherence and probabilistic output (slide 15, which concern quantum *computers*). Qubits keep their quantum state only for a short time before interaction with the environment destroys the superposition (*decoherence*), which limits how long a quantum computation can run and is why large-scale error correction is needed. Quantum algorithms are mainly probabilistic: a measurement returns the right answer only with some probability, so the algorithm is repeated and each candidate answer is checked classically (for Shor, by multiplying the factors).

41.13 Information hiding: what Lecture 18 adds to steganography ▸

▸ **Likely this year:** "Define steganography; why can data be hidden in digital media?" (2 marks). Basics are in §19.

The basics of steganography (hiding the *existence* of a message rather than its meaning, the historical techniques, the two drawbacks, LSB hiding in RGB pixels, watermarking versus steganography, and the OpenStego demo) are in §19 and are not repeated here. The word comes from the Greek *steganos* ("covered") and *graphein* ("writing"): steganography is the art of hiding the existence of a message between sender and intended recipient, so that nobody even suspects that a (possibly encrypted) message exists. Lecture 18 adds the digital-media view: it works because digital media contain **unused or redundant bits**, whose change is imperceptible to human eyes and ears. Steganography is hiding data within data; it is often combined with cryptography, and the two are not mutually exclusive.

Vocabulary. The innocent file used as a carrier is the **cover** (cover image, cover audio); the information to hide is the **secret message** or **payload**; the secret shared by sender and receiver that controls where and how bits are hidden is the **stego-key**; the result of **embedding** is the **stego object**, from which the receiver performs **extraction**. The attacker's discipline, detecting (and possibly extracting or destroying) hidden data, is **steganalysis**, the counterpart of cryptanalysis. A stego system is judged by three competing requirements: **imperceptibility** (the stego object looks and sounds like the cover), **capacity** (bits hidden per cover) and **undetectability** (statistical tests cannot tell stego objects from covers); a watermarking system replaces undetectability by **robustness**.

41.14 Steganography in digital media ▸

▸ **Likely this year:** "Explain text, image, audio and video steganography" (4 marks).

Text steganography. Text has very little redundancy, so information is hidden in its *format* or *wording*. Format-based methods use the number of tabs or white spaces (for example one or two spaces after a full stop, or invisible trailing spaces), capital letters, or tiny shifts of lines and words, much as Morse code encodes letters in dots and dashes. Random and statistical generation methods produce a text whose statistics encode the message, and linguistic methods choose between synonyms or equivalent sentence structures. Capacity is low and the hiding is fragile, since retyping or reformatting the text destroys it.

Image steganography. Pixels are the building blocks of an image, and in a 24-bit image every pixel has three values (red, green, blue), each from 0 to 255; 8-bit (grey-scale or palette) and 24-bit images are the common covers. The information is hidden in the pixel intensities, either directly (spatial domain) or in transform coefficients (frequency domain). Even a whole image can be hidden inside another (slide 21: a baboon picture hidden in the "Lena" cover, whose stego version looks unchanged): one way is to replace the 4 least significant bits of each cover pixel by the 4 most significant bits of the secret pixel, which keeps the cover's appearance (its high bits are untouched) and keeps a recognisable coarse version of the secret image.

Audio steganography. Taking audio as the carrier (WAVE, MIDI, AVI/MPEG formats) has the advantage of large data redundancy, so a large amount of information can be embedded. The methods on the slide are: *LSB encoding* (replace the LSB of each audio sample); *parity encoding* (split the samples into regions and, for each message bit, flip one LSB in the region if needed so that the parity of the region's LSBs equals the bit, choosing the sample whose change is least audible); *phase coding* (replace the phase of an initial segment by a phase pattern representing the data and adjust later segments to keep relative phases, exploiting that the ear is insensitive to absolute phase); and *spread spectrum* (below).

Video steganography. Any file or information is hidden in a digital video (H.264, MP4, MPEG, AVI); the sequence of pictures is the carrier, so capacity is very large and the video format is popular and unsuspicious. There are two main classes: embedding in the uncompressed raw video and compressing it later (the data must then survive compression), and embedding directly into the compressed data stream (for example in DCT coefficients or motion vectors), which avoids the recompression loss.

Network (protocol) steganography (not on the slides). Data can also be hidden in unused or loosely specified fields of packet headers (such as the IP identification field or TCP initial sequence numbers) or in the timing and ordering of packets. Such *covert channels* are a concern for firewalls and intrusion detection, because the traffic looks legitimate.

Medium	Where the bits go	Typical methods	Capacity
Text	spacing, line/word positions, word choice	format-based, random/statistical generation, linguistic	very low
Image	pixel LSBs, transform coefficients, noisy regions	LSB, BPCS, DCT/DWT, spread spectrum	medium
Audio	sample LSBs, phase, spread noise	LSB, parity, phase coding, spread spectrum, echo	high
Video	frames, DCT coefficients, motion vectors	raw-frame embedding, compressed-domain embedding	very high
Network	header fields, packet timing	covert storage and timing channels	low

41.15 Steganography techniques ★▸

▸ **Likely this year:** "Classify steganography techniques; explain LSB, transform-domain and spread-spectrum hiding" (4–5 marks), or an LSB capacity numerical.

Image steganography is broadly divided into two categories (slide 24). In **spatial-domain** methods the message is hidden directly in the pixel values of the base image. In **frequency-domain** (transform-domain) methods the cover is first converted into the frequency domain with the discrete cosine transform (DCT), the discrete Fourier transform (DFT) or the discrete wavelet transform (DWT), the message is hidden in the coefficients, and the inverse transform produces the stego image that is transmitted. The standard textbook classification (Katzenbeisser and Petitcolas) has six classes; the techniques on slides 25–28 fit into them as follows.

1. Substitution systems (spatial): LSB. The least significant bits of the cover are replaced by message bits. In a typical grey-scale image (8 bits per pixel) the 8th (least significant) bit of some or all bytes is replaced by a

message bit; in a full-colour image (24 bits per pixel, 8 per colour) one bit of each of the red, green and blue components can be used, i.e. 3 bits per pixel. The change is imperceptible because it alters a value by at most 1 out of 255. This can be quantified with the **peak signal-to-noise ratio** PSNR = $10\log_{10}(255^2/\text{MSE})$, where MSE is the mean squared change per value. Since each LSB changes by at most 1, MSE ≤ 1 and PSNR ≥ 48.1 dB; with random message bits only half the LSBs actually change, so MSE = 0.5 and PSNR = 51.1 dB, far above the roughly 40 dB beyond which differences are generally invisible.

Weaknesses of LSB. LSB hiding is *fragile*: JPEG compression, resizing or filtering rewrites the low bits and destroys the message (which is why stego images are saved as PNG or BMP, §19). It is also *statistically detectable*: replacing an LSB only ever moves a value within its pair $\{2i, 2i+1\}$, so after embedding random bits the counts of $2i$ and $2i+1$ in the histogram become nearly equal, whereas in natural images they differ; the chi-square attack of Westfeld and Pfitzmann tests exactly this.

WORKED EXAMPLE : LSB embedding and capacity

Q. (a) Hide the character H with 1-bit LSB substitution in eight grey-scale pixels with values 154, 155, 200, 201, 99, 100, 37, 38, and compute the PSNR of this block. (b) How many bytes can be hidden in a 512 × 512 24-bit colour image using one LSB per colour component, and using two?
A. (a) H = 72 = `01001000`. Each pixel's LSB is set to the next message bit (value & ~1, then OR the bit):

Pixel	154	155	200	201	99	100	37	38
Old LSB	0	1	0	1	1	0	1	0
Message bit	0	1	0	0	1	0	0	0
New value	154	155	200	**200**	99	100	**36**	38

Only 2 of the 8 pixels change, each by 1: MSE = 2/8 = 0.25, PSNR = $10\log_{10}(255^2/0.25)$ = **54.2 dB**. Reading the LSBs of the new values gives `01001000` = H.
(b) Pixels = 512 × 512 = 262 144; with 3 bits per pixel, 786 432 bits = **98 304 bytes** (96 KiB). With 2 LSBs per component (6 bits per pixel), **196 608** bytes. (An 8-bit grey-scale image of the same size holds 262 144 bits = 32 768 bytes at 1 bit per pixel.)
Check: capacity is 1/8 of the image size per LSB plane: 512 × 512 × 3 bytes /8 = 98 304.

BPCS: bit-plane complexity segmentation (slide 26, called "binary pattern complexity"). An 8-bit image is split into 8 bit-planes, and each plane into small blocks (typically 8 × 8). The *complexity* of a block is measured by the number of changes between adjacent 0 and 1 bits relative to the maximum possible; blocks above a threshold are "noisy" and the eye cannot see their exact pattern. These noisy blocks are replaced by blocks of secret data (a data block that happens to be too simple is first XORed with a checkerboard so that it looks noisy, and a flag records this). Because whole noisy blocks on several bit-planes can be replaced, BPCS hides far more than LSB, often a large fraction of the image size.

2. Transform-domain systems: DCT, DFT, DWT. The cover is transformed, and the message is hidden in selected coefficients. In JPEG images the data are hidden in the LSBs of the quantised DCT coefficients (as in the JSteg tool), so the hidden data survive JPEG storage because they are part of what JPEG keeps. The **discrete wavelet transform** (slide 26) transforms the image from the spatial domain to the frequency domain by decomposing it hierarchically with *wavelets*, small waves of different frequency and limited duration, which suits non-stationary signals such as images. One level of DWT splits the image into an approximation sub-band (low frequencies, LL) and three detail sub-bands (horizontal, vertical, diagonal high frequencies), i.e. it identifies the low- and high-frequency information at each location. Hiding in the high-frequency detail bands is least visible; hiding in lower bands is more robust. Since a change to one coefficient spreads over a whole region of pixels after the inverse transform, transform-domain hiding is more robust to compression and filtering than LSB, at the cost of lower capacity.

3. Spread-spectrum systems (slide 27). The secret data are spread over a wide frequency band, with such a small signal-to-noise ratio in every band that the presence of data is hard to detect. Let the cover samples (or coefficients) be $x_1, \ldots, x_N$, let the stego-key generate a pseudo-random sequence $p_1, \ldots, p_N$ of ±1, and let a message bit be $b = \pm 1$. Embedding adds a tiny multiple of the sequence: $y_i = x_i + \alpha\, b\, p_i$ with small strength α. The receiver, who knows p, correlates:

$$\frac{1}{N}\sum_{i=1}^{N} y_i p_i = \frac{1}{N}\sum_{i=1}^{N} x_i p_i + \alpha b \frac{1}{N}\sum_{i=1}^{N} p_i^2 = \underbrace{\frac{1}{N}\sum x_i p_i}_{\approx 0} + \alpha b,$$

because $p_i^2 = 1$ and the cover is uncorrelated with the random signs, so the first term shrinks like $1/\sqrt{N}$. The sign of the result is b. Each sample changes only by $\pm\alpha$ (imperceptible), but N samples add up to a clear signal. Even if parts of the data are removed from several bands, enough remains in the others to recover it, so the data cannot be removed without entirely destroying the cover; without the key, Eve sees only faint noise. This is why spread spectrum is the classic **watermarking** technique.

4. Statistical methods hide one bit per block of the cover by changing a statistical property of the block (for example forcing its mean or the parity of its LSBs above or below a threshold), and the receiver tests the property. **5. Distortion techniques** store the message as a sequence of small modifications of the cover; the receiver needs the original cover and decodes from the differences. **6. Cover-generation methods** do not modify any cover; they generate one that encodes the message (for example grammatically generated text or "spam-like" messages whose word choices carry the bits). The slide's comparison table names the same families as spatial domain, transform domain, model-based and ad hoc.

Combined methods (slide 28). In **multilevel steganography** at least two methods are used together, so that one method (the upper level) serves as a carrier for the second (the lower level); someone who finds the upper-level message still does not see the lower-level one. In **hybrid cryptography and steganography** the data are first encrypted with RSA or any other cipher and then hidden in an image or other file, so detection reveals only ciphertext (the "best of both" of §19).

Technique	Capacity	Robustness	Detectability
LSB substitution	high	very low (destroyed by compression)	moderate (histogram pairs)
BPCS	very high	very low	moderate
Transform domain (DCT/DWT)	medium	medium to high	low
Spread spectrum	low	high	very low without the key
Statistical / distortion	low	low to medium	low
Cover generation	low	n/a (no cover to attack)	depends on generator quality

The pattern is a **trade-off**: hiding more bits (capacity) means larger or more numerous changes, which hurts imperceptibility and makes detection easier, while spreading few bits redundantly over the whole cover gives robustness but little capacity. Steganography favours capacity and undetectability; watermarking favours robustness.

41.16 Cryptography versus information hiding ★▸

▸ **Likely this year:** "Differentiate cryptography, steganography and watermarking" (3–5 marks): reproduce the table.

Slide 18 compares the three techniques by application, principle, visibility and requirement. Cryptography transforms the message itself; the two information-hiding techniques embed something into a separate cover file, and they differ in what that something is and what must survive.

	Cryptography	**Information hiding**	
Technique		**Steganography**	**Watermarking**
Application	secret information	secret communication	copyright
Principle	plaintext + key → *encryption* → ciphertext	secret message + cover file + key → *embedding* → stego file	signature + cover file + key → *encoding* → marked file
Visibility	visible (ciphertext is obviously secret)	invisible / inaudible	visible or invisible
Requirement	robustness	undetectability, capacity	robustness

Slide 19 gives a detailed comparison of steganography and cryptography. It is transcribed below; entries marked † are wrong or misleading and are corrected after the table.

Comparison	Steganography	Cryptography
Basic	it is known as cover writing	it means secret writing
Goal	secret communication	data protection
Structure of the message	not altered	altered only for the transmission
Popularity	less popular	more commonly used
Relies on	key	no parameters†
Supported security principles	confidentiality and authentication	confidentiality, data integrity, authentication, and non-repudiation
Techniques	spatial domain, transform domain, model-based, and ad hoc	transposition, substitution, stream ciphers, block ciphers
Implemented on	audio, video, image, text	only on text files†
Types of attack	steganalysis	cryptanalysis

†Cryptography relies *entirely* on its key: by Kerckhoffs's principle the algorithm is public and only the key is secret. Cryptography is applied to any bit string (images, audio, disks, network packets), not only to text. In an exam, write "relies on: secrecy of the method and a stego-key" for steganography and "relies on: secrecy of the key" for cryptography.

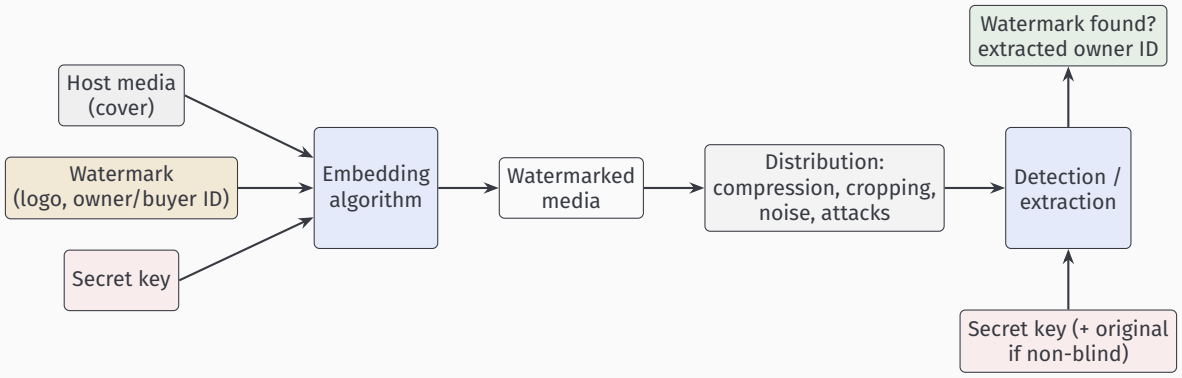
The key conceptual difference. Cryptography hides the *meaning* and leaves the existence obvious; once Eve sees ciphertext she knows a secret is being exchanged and attacks the cipher (cryptanalysis). Steganography hides the *existence*; Eve's task is first to notice that anything is hidden (steganalysis), and once she does, an unencrypted payload is lost. Combining them (encrypt, then hide) gives both protections.

41.17 Digital watermarking ★▸

▸ **Likely this year:** "What is digital watermarking? Explain its types, properties and applications; differentiate it from steganography" (5–6 marks).

Watermarking is embedding some information within digital media (an image, audio, video or document) so that the media looks unchanged. The hidden information is not important by itself, but it *says something about the media*: who owns it, who bought this copy, or whether it has been altered. Watermarking is closely related to steganography, since both hide a message inside a digital signal and watermarking uses the same techniques, but they are separated by their goal: steganography protects the hidden message, whereas watermarking protects the cover (authenticity, proof of ownership). Paper watermarks (patterns pressed into banknotes and letter paper) can often be removed or imitated with little effort, which led to the digital watermark, which is an internal part of the digital media and so cannot easily be detected or removed.

The process has two parts: **embedding** the watermark into the host media and **extraction** (or *detection*) of the watermark from the possibly modified media.



A common embedding is the additive spread-spectrum rule of the previous subsection, $y_i = x_i + \alpha w_i$, where the watermark sequence w is generated from the key and placed in perceptually significant transform coefficients (Cox et al.); detection correlates the received media with w and declares the watermark present if the correlation exceeds a threshold. Because the mark sits in the significant coefficients, removing it requires damaging exactly the parts of the image that matter.

Properties. A watermarking system is judged by:

- **Imperceptibility (fidelity):** an invisible watermark must not perceptibly degrade the media.
- **Robustness:** the watermark must survive common processing (compression, filtering, scaling, printing and scanning) and deliberate attempts at removal, as long as the media remains useful.
- **Capacity (payload):** the number of bits the watermark carries, from 1 bit ("marked or not") to an owner or buyer identifier.
- **Security:** without the key, an attacker cannot detect, read, remove or forge the watermark, even knowing the algorithm (Kerckhoffs's principle again).
- **Low false-positive rate:** unmarked media must almost never trigger the detector, otherwise ownership claims are worthless.

These conflict. A larger strength α makes the mark more robust but more perceptible; more payload bits leave fewer cover samples per bit, so each bit's correlation margin shrinks and robustness falls. Every design picks a point in this imperceptibility–robustness–capacity triangle.

Types of watermark.

- **Visible or invisible.** A visible watermark is a translucent logo or text over the content (slide 30: the original image and a version covered with a faint repeated "KH" pattern); it advertises ownership and deters misuse but can be cropped or painted out. An invisible watermark is hidden in the content and found only by the detector.
- **Robust, fragile or semi-fragile.** A robust watermark survives processing and is used for copyright and fingerprinting. A fragile watermark is destroyed by any change (for example a hash of the image hidden in its own LSBs), so a missing or broken mark proves tampering and can even show where. A semi-fragile watermark survives benign operations such as JPEG compression but breaks under content changes, which suits authentication of photographs.
- **Blind or non-blind.** Blind detection needs only the key and the marked media; non-blind (informed) detection

also needs the original, which makes it more reliable but impractical for monitoring large amounts of content.
- **Spatial or frequency domain**, exactly as for steganography.

Attacks on watermarks. *Removal attacks* add noise, denoise, compress or filter until the detector fails; *geometric attacks* rotate, scale or crop slightly so that the detector loses synchronisation; *collusion attacks* average several copies carrying different buyer fingerprints so that the individual marks cancel; *ambiguity (protocol) attacks* let a pirate "find" his own fake watermark in the original, so that two parties claim ownership. The last is countered by making the watermark depend on a hash of the original and registering it with a trusted time-stamp.

Applications (slides 32–33).

- **Owner identification.** The owner's identity is embedded so that it can be read from any copy; a strong (robust) watermark is important to prove ownership during conflict resolution.
- **Copy protection.** Illegal copying is prevented by embedding a *copy-protect bit* (for example "copy never", "copy once") that compliant recorders and players read and obey.
- **Proofs and samples.** Professional photographers watermark the digital proofs sent to clients until the client agrees to purchase, at which point the original, unaltered images are released; this allows them to distribute demos and samples without giving away the originals.
- **Broadcast monitoring.** Television and radio broadcasts (especially paid media such as sports, news and advertisements) carry watermarks so that automatic monitors can verify when and where each item was actually aired.
- **Data authentication.** The sender embeds a (fragile) watermark in the host data, and the receiver extracts and verifies it; a mismatch shows that the data were altered.
- **Fingerprinting (traitor tracing)** (textbook addition). Each buyer's copy carries a different watermark, so a leaked copy identifies the buyer who leaked it.

Slide 31 compares watermarking with cryptography; most rows are copied from slide 19, and three are wrong (†, corrected below).

Comparison	Cryptography	Watermarking
Basic	it means secret writing	it is known as cover writing
Goal	data protection	copyright protection
Structure of the message	altered only for the transmission	not altered
Popularity	more commonly used	less popular
Relies on	no parameters†	key
Supported security principles	confidentiality, data integrity, authentication, and non-repudiation	confidentiality and authentication†
Techniques	transposition, substitution, stream cipher, block ciphers	application-based: using a universal logo without encryption in the embedding algorithm
Implemented on	only on text files†	audio, video, image, text
Types of attack	steganalysis†	cryptanalysis†

†The attack row is swapped: cryptography is attacked by *cryptanalysis*; watermarks are attacked by removal, geometric, collusion and ambiguity attacks. Watermarking gives ownership proof, authentication and (fragile marks) integrity, not confidentiality, since the mark is often public or even visible. Cryptography relies on its key and applies to any data, as noted for slide 19.

The comparison most often asked is **watermarking versus steganography** (both hide data in a cover, which is why they are confused):

	Steganography	Watermarking
Goal	secret communication: hide that a message exists	prove ownership, identify the buyer, or authenticate the cover
What is valuable	the hidden message; the cover is just a carrier	the cover; the watermark is information *about* it
Relation to the cover	none (any cover will do)	tied to the cover (owner ID, buyer ID, hash of the content)
Visibility	always invisible / inaudible	visible or invisible
Main requirement	undetectability and high capacity	robustness (or deliberate fragility for tamper detection)
Payload	large	small (one bit up to an identifier)
Attacker wins by	detecting that something is hidden	removing or forging the mark without ruining the cover
Is detection a failure?	yes, detection alone defeats it	no; a visible mark is known to everyone

The last slide

The lecture ends with the slide "Hss aol ilza mvy fvby lehtz", a Caesar cipher (the shift cipher of the classical-cipher sections, with key k: $C = (P + k) \bmod 26$), a fitting reminder that the oldest technique of the course still turns up at the end.

WORKED EXAMPLE : decrypting the closing slide

Q. Decrypt "Hss aol ilza mvy fvby lehtz".
A. *Finding the key.* The first word has three letters with a double letter at the end; the likely English word is "All", which gives k = H − A = 7 − 0 = 7. The most frequent ciphertext letter is l (3 times), and l − e = 11 − 4 = 7 agrees. *Decryption* $P = (C - 7) \bmod 26$:

C	h	s	a	o	l	i	z	m	v	y	f	b	e	t
C (number)	7	18	0	14	11	8	25	12	21	24	5	1	4	19
$(C - 7) \bmod 26$	0	11	19	7	4	1	18	5	14	17	24	20	23	12
P	a	l	t	h	e	b	s	f	o	r	y	u	x	m

Plaintext: **"All the best for your exams"**. *Check*: re-encrypting "exams" with +7 gives "lehtz".

Corrections to the notes / slides

- **Slide 6 (ASCII-to-DNA table):** 14 four-base words are assigned to two or three characters (AAGC: $ and –; CCGT: 2, 5, 8; GACT: n, q, v; GAGT: h, DEL; GATG: i, ~; GGTG: s, |; GTGA: m, z; TAAG: Z, \; TACT: A, K; TAGT: F, M; TCCG: _, `; TCCT: B, X; TGAG: S, V; TGTT: I, ^). Only 80 of 96 words are distinct, so the table cannot be decoded uniquely (HI ↦ TGGC TGTT, and TGTT could be I or ^). A usable code book must be one-to-one.
- **Slide 9:** the "complementary rule" A→C, C→G, G→T, T→A is *not* the Watson–Crick complement of slide 5 (A↔T, C↔G); it is a cyclic shift (add 1 mod 4 to each 2-bit group). Use the slide's rule when asked about this algorithm. "DND strands" should read "DNA strands".
- **Slides 9 and 11:** a key "between 1 and 256" that "determines the permutation of the four characters A, T, G, C" is inconsistent: four letters have only 4! = 24 permutations, and 256 keys are brute-forced instantly. Slide 11's key space of about 10^{23} ($\approx 2^{76}$) does not follow from a 1–256 key. With a true one-time-pad key (slide 8) the scheme is perfectly secret, but only because of the XOR.
- **Slides 8 and 10:** the flowcharts omit the complementary-rule step of slide 9; decryption must also undo it (C→A, G→C, T→G, A→T).
- **Slide 3:** "DNA computing does not require power" is overstated (PCR heating cycles and electrophoresis need power), and the speed advantage comes from parallelism; individual reactions take minutes to hours.
- **Slide 13:** the "photon filter … or else the photon will be destroyed" is an oversimplification: a photon at 45° to the filter passes with probability $\cos^2 45^\circ = 1/2$ and is then re-polarised; only at 90° is it always blocked. A qubit is any two-level quantum system; a single photon is the carrier used in QKD.
- **Slides 14–15:** Shor's and Grover's algorithms are quantum *attacks* on classical cryptography, not algorithms "of quantum cryptography". D-Wave's annealers cannot run Shor's algorithm. Decoherence and probabilistic output are challenges of quantum *computers*, not of QKD.
- **Slide 19:** cryptography relies on a key (not "no parameters") and is applied to any data, not "only on text files".
- **Slide 31:** the attack row is swapped (cryptography → cryptanalysis; watermarking → removal/geometric/-collusion attacks), and watermarking does not provide confidentiality; the rows were copied from slide 19.
- **Slide 26:** the standard name of "Binary Pattern Complexity (BPC)" is *bit-plane complexity segmentation* (BPCS). **Slide 16:** "fist place" → "first place"; the hidden message need not be encrypted.

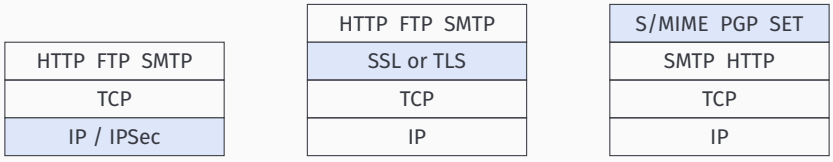
42 Network Security Protocols and System Security

The cryptographic tools of the earlier sections (block ciphers and modes, RSA and Diffie–Hellman, hashes, MACs and signatures) become useful only when a protocol fixes exactly which tool protects which bytes, how the keys get there, and in what order the operations are applied. This part of the syllabus studies the standard protocols at three layers of the TCP/IP stack, e-mail security (PGP, S/MIME), IP security (IPSec), web security (SSL/TLS, HTTPS, SET), and then the defence of the computer systems themselves (intruders, malicious software, firewalls, trusted systems). None of it is on this year's slides, so the notes follow Stallings and Forouzan.

42.1 Where security can be placed in the stack ▸

▸ **Likely this year:** "Compare security at the network, transport and application layers" (3 marks).

Security can be added at three places, and the choice decides who must change and what is protected.



(a) network level *(b) transport level* *(c) application level*

Network level (IPSec). Security is applied to every IP packet, so it is transparent to applications and to users: no application has to be rewritten, and all traffic between two sites (web, mail, file transfer) is protected at once. **Transport level (SSL/TLS).** Security sits just above TCP and is used by the applications that ask for it (a browser opening an `https` connection); it protects one connection at a time. **Application level (PGP, S/MIME, SET).** Security is built into the application and can be tailored to it: an e-mail is signed by its author and stays signed in the recipient's mailbox, and SET can show the bank the card number while hiding it from the merchant. The price is that every application must implement its own security.

	Network (IPSec)	Transport (SSL/TLS)	Application (PGP, S/MIME, SET)
Protects	every IP packet between two hosts or gateways	one TCP connection	one message or transaction
Transparent to applications	yes	no (the application calls TLS)	no (built into the application)
Protection after delivery	none (removed on arrival)	none	yes (a signed mail stays signed)
Typical use	VPNs, site-to-site links	HTTPS, secure mail transport	signed/encrypted e-mail, card payments

43 E-mail Security: PGP and S/MIME

43.1 Why e-mail needs its own security ▸

▸ **Likely this year:** "Why can e-mail not use a handshake like TLS?" (2 marks), as part of a PGP question.

E-mail is a **one-way, store-and-forward** service: Alice writes the message, it is passed from server to server, and Bob reads it hours later. Since Alice and Bob are never online together, there is no handshake in which they could agree on algorithms or a session key. Alice must therefore decide everything herself and put it *into* the message: the identifiers of the algorithms, a fresh session key encrypted for Bob, her signature and the information Bob needs to find her public key. Two practical constraints of the mail system shape the design as well: classic SMTP carries only 7-bit ASCII text, so binary ciphertext must be converted to printable characters, and mail servers limit the size of a message. PGP and S/MIME are the two standard solutions; both use the same cryptographic idea (hybrid encryption plus a hash-and-sign signature) and differ mainly in formats and in how public keys are trusted.

43.2 PGP: the five services ★▸

▸ **Likely this year:** "Explain the operational description of PGP with a diagram (authentication, confidentiality, both)" (5–8 marks).

Pretty Good Privacy (PGP), written by Phil Zimmermann in 1991, is a freely available package that provides confidentiality and authentication for e-mail and file storage. It selected the best available algorithms as building blocks, runs on every platform, and is not controlled by any government or standards body, which explains its spread. PGP offers five services.

Service	Algorithms (classic PGP)	What is done
Digital signature	DSS/SHA-1 or RSA/SHA-1	a SHA-1 hash of the message is signed with the sender's private key and attached
Message encryption	CAST-128, IDEA or 3DES; session key by RSA or ElGamal	the message is encrypted with a one-time session key, which is encrypted with the recipient's public key and attached
Compression	ZIP	the message is compressed for storage and transmission
E-mail compatibility	radix-64 conversion	the binary result is converted to printable ASCII
Segmentation	—	long messages are split to fit mail size limits and reassembled

Authentication only. The sender creates the message M and computes its 160-bit SHA-1 hash $H(M)$. The hash is encrypted with RSA under the sender's private key PR_a and prepended to the message. The receiver decrypts the

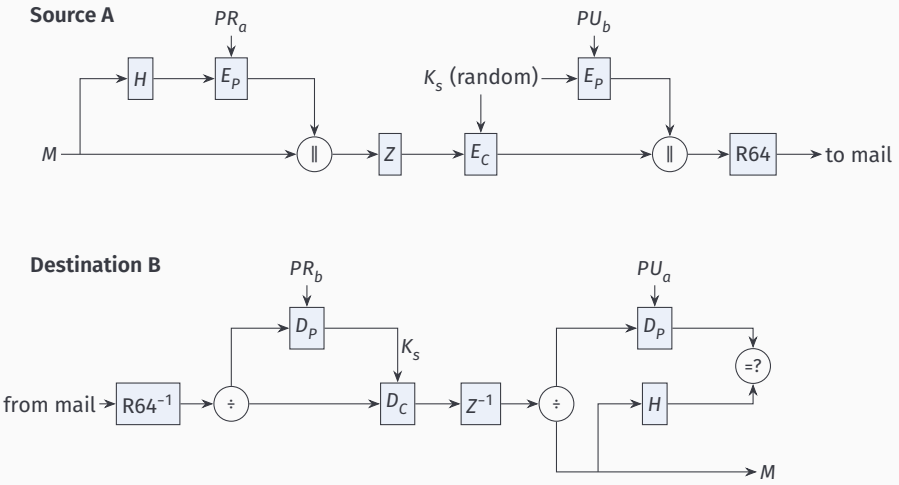
signature with the sender's public key PU_a to recover the hash, computes $H(M)$ of the received message himself, and accepts if the two are equal. Since only Alice holds PR_a, a match proves that she created the signature, and since the hash is collision resistant (§35), it proves that M was not changed. Signing the hash instead of M keeps the RSA operation short whatever the length of the message.

Confidentiality only. The sender generates a random 128-bit **session key** K_s, used for this message only, and encrypts the message with CAST-128 (or IDEA or 3DES) in 64-bit CFB mode (§32). The session key is encrypted with RSA under the recipient's public key PU_b and prepended. The receiver recovers K_s with his private key PR_b and decrypts the message. This is **hybrid encryption**: the slow public-key operation is applied to 128 bits only, the bulk is encrypted by the fast symmetric cipher, and the one-time key removes the need for any earlier key exchange, which a store-and-forward service could not have.

Both services. The sender first signs the plaintext, then encrypts the message together with the signature under K_s, and attaches $E(PU_b, K_s)$:

$$A \to B: \quad E(PU_b, K_s) \,\|\, E_{K_s}\Big(Z\big(E(PR_a, H(M)) \,\|\, M\big)\Big),$$

where Z is ZIP compression; the whole result is then radix-64 encoded.



E_P/D_P are public-key encryption/decryption (RSA), E_C/D_C the symmetric cipher, Z compression, R64 radix-64 conversion, ‖ concatenation and ÷ separation of the components.

At the receiver the steps run in reverse: radix-64 decoding, recovery of K_s with PR_b, symmetric decryption, decompression, and finally verification of the signature with PU_a against a freshly computed $H(M)$.

43.3 PGP: the order of operations and why ★▸

▸ **Likely this year:** "Why does PGP sign before compressing and compress before encrypting?" (3 marks).

The sender applies the operations in the order **sign → compress → encrypt → radix-64 → segment**, and each position has a reason.

Sign before compressing. First, the signature is then computed on the uncompressed message, so the recipient can store the plain message together with the signature and verify it later without keeping a compressed copy or recompressing. Second, PGP's compression is not deterministic: different versions of ZIP trade speed against compression ratio and produce different compressed outputs from the same input. A signature on the compressed form would verify only with the same compressor version; a signature on the original message is independent of the compressor.

Compress before encrypting. Compression removes redundancy, and redundancy is what cryptanalysis feeds on, so encrypting the compressed text strengthens the encryption. It also shortens what has to be encrypted and sent. (Compressing *after* encryption would be useless anyway, since good ciphertext looks random and does not compress.)

Radix-64 after encrypting. The signature, the encrypted session key and the ciphertext are arbitrary binary data, which 7-bit mail systems may corrupt. Radix-64 (the same encoding as MIME base64) maps every 3 bytes to 4 printable characters from a 64-symbol alphabet (A–Z, a–z, 0–9, +, /), and a 24-bit CRC is appended to detect transmission errors. It expands the data by 4/3, about 33%, but ZIP compresses ordinary text by about a factor of 2, so the final message is usually *shorter* than the plaintext: $\frac{1}{2} \times \frac{4}{3} \approx 0.67$.

Segmentation last. Many mail systems limit the message size (e.g. 50 000 bytes), so after all other processing PGP splits a long message into segments that are mailed separately. The session key component and the signature component appear only once, at the start of the first segment; the receiver strips the mail headers and reassembles the whole block before any other processing.

WORKED EXAMPLE : radix-64 and the size of a PGP message

Q. (a) Radix-64 encode the three bytes 4D 61 6E ("Man"). (b) A 60 000-byte text is signed (signature component 300 bytes), compressed by ZIP with ratio 2, encrypted (CFB, no expansion), given a 260-byte session key component, and radix-64 encoded. Ignoring line breaks and the CRC, how many characters are sent, and how many segments if the mailer accepts at most 20 000 characters?

A. (a) The 24 bits 01001101 01100001 01101110 are regrouped into four 6-bit groups 010011 010110 000101 101110 = 19, 22, 5, 46. In the radix-64 alphabet (0–25 → A–Z, 26–51 → a–z, 52–61 → 0–9) these are **T, W, F, u**, so the output is TWFu.

(b) Signed: 60 000 + 300 = 60 300 bytes; compressed: 60 300/2 = 30 150; with the session key component: 30 410 bytes; radix-64: $\lceil 30\,410/3 \rceil \times 4 = 10\,137 \times 4 =$ **40 548 characters**; segments: $\lceil 40\,548/20\,000 \rceil =$ **3**.

Check: 40 548/60 000 = 0.676, close to the predicted $\frac{1}{2} \cdot \frac{4}{3} = 0.67$.

Message format. A PGP message consists of three components. The *message component* holds the data with a filename and a timestamp. The *signature component* holds a timestamp, the **key ID** of the sender's public key, the **leading two octets of the message digest** in the clear, and the digest encrypted with the sender's private key. The two clear octets let the receiver check, before doing anything else, that he used the right public key to decrypt the digest: if the decrypted digest does not start with them, the wrong key was used. The *session key component* holds the key ID of the recipient's public key and the encrypted session key. Compression and symmetric encryption cover the signature and message components; radix-64 covers everything.

43.4 PGP: key IDs and key rings ▸

▸ **Likely this year:** "Explain the PGP private-key ring and public-key ring; what is a key ID?" (4 marks).

Key IDs. A user may own several key pairs (to change keys from time to time, or to separate work and private use), and he will hold the public keys of many correspondents. The recipient therefore needs to know which public key was used. Sending the whole public key would waste space, so PGP sends a **key ID**, the least significant 64 bits of the public key: ID = $PU \bmod 2^{64}$. With 64 bits, the chance that two keys of one user share an ID is negligible.

Private-key ring. Each user stores his own key pairs in a table with one row per pair: timestamp (when the pair was created), key ID, public key, **encrypted private key**, and user ID (normally the e-mail address); the table can be indexed by user ID or key ID. The private key is never stored in the clear. When a pair is generated, PGP asks for a passphrase, computes its 160-bit SHA-1 hash, and encrypts the private key with CAST-128 using 128 bits of that hash as the key; the passphrase and the hash are then discarded. To sign, the user types the passphrase again, PGP recomputes the hash and decrypts the private key. A thief who copies the ring therefore still needs the passphrase, so the security of the private key rests on the passphrase.

Public-key ring. The public keys of other users are stored with timestamp, key ID, public key, user ID and the three trust fields described next: **owner trust**, the **signatures** on the key with their **signature trust**, and the computed **key legitimacy**. When a message arrives, PGP uses the key ID in the session key component to fetch the right private key from the private-key ring, and the key ID in the signature component to fetch the sender's public key from the public-key ring.

43.5 PGP: the web of trust ▸

▸ **Likely this year:** "Explain the PGP trust model: owner trust, signature trust, key legitimacy" (4 marks), with a small example.

Problem. Public-key encryption is only as safe as the binding between a public key and its owner. If Eve can plant a key with Bob's user ID on Alice's ring, Alice will encrypt to Eve and accept Eve's signatures as Bob's (a man-in-the-middle). PGP has no central certificate authority. Instead, any user may sign another user's public key to vouch that it belongs to that user (a key signature acts as a certificate), and each user decides whom he trusts as an **introducer**. The resulting graph of mutual signatures is the **web of trust**.

Mechanism. Each entry of Alice's public-key ring carries three kinds of trust information.

- **Owner trust** says how far Alice trusts the owner of this key to sign *other* people's keys correctly. It is assigned by Alice herself: unknown, usually not trusted, usually (partially) trusted, always (fully) trusted, or ultimate for her own keys.
- **Signature trust** is attached to each signature on the key and is simply a copy of the owner trust of the signer, if the signer is on Alice's ring (otherwise unknown).
- **Key legitimacy** says how far Alice believes the key really belongs to its user ID. PGP computes it from the signature trusts: one signature of ultimate trust makes it complete; otherwise PGP adds weights, $1/X$ for each always-trusted signature and $1/Y$ for each usually-trusted one (X and Y are user parameters, typically $X = 1$, $Y = 2$), and when the sum reaches 1 the key is complete, below 1 it is marginal.

The important distinction is that **legitimacy** concerns the key (is it really Bob's?) while **owner trust** concerns the person (does Bob check keys carefully before signing them?). A key can be fully legitimate while its owner is not trusted as an introducer. A user revokes a compromised key by issuing a **key revocation certificate** signed with

the corresponding private key and spreading it as widely as possible.

WORKED EXAMPLE : key legitimacy in a web of trust

Q. Alice gives owner trust *always* to Bob and *usually* to Carol and Dave, and has not assigned any trust to Frank. Weights are 1 for an always-trusted and $\frac{1}{2}$ for a usually-trusted signer. Determine the legitimacy of the keys of (a) Gita, signed by Bob; (b) Hari, signed by Carol and Dave; (c) Ivan, signed by Carol; (d) Jaya, signed by Frank, whose own key on Alice's ring is legitimate.
A. (a) $1 \geq 1$: **complete**. (b) $\frac{1}{2} + \frac{1}{2} = 1$: **complete**. (c) $\frac{1}{2} < 1$: **marginal**. (d) Frank's owner trust is unknown, so his signature has weight 0: Jaya's key is **not legitimate**, although Frank's own key is. Legitimacy of the signer's key is not enough; Alice must trust him as an introducer.
Check: only (a) and (b) reach the threshold 1.

43.6 S/MIME and MIME ▸

▸ **Likely this year:** "What are the limitations of SMTP/RFC 822 that MIME removes? List the MIME header fields and content types" (4 marks).

Problem. The original Internet mail format (RFC 822) and SMTP were designed for short 7-bit ASCII text. As a result, SMTP cannot carry executable files or other binary objects, cannot carry national-language characters (8-bit codes), and may reject messages above a size limit; gateways translating between ASCII and EBCDIC or to X.400 networks use inconsistent mappings; and non-standard SMTP implementations delete or add carriage returns and line feeds, truncate or wrap lines longer than 76 characters, remove trailing white space, pad lines to the same length and convert tabs to spaces. Any of these silently corrupts binary data such as ciphertext or a signature.

MIME (Multipurpose Internet Mail Extensions) solves this within RFC 822 by adding five header fields (MIME-Version, Content-Type, Content-Transfer-Encoding, Content-ID and Content-Description), a set of standard content types, and transfer encodings (7bit, 8bit, binary, quoted-printable for mostly-ASCII text, and **base64** for arbitrary binary data) that make any content survive the mail system.

Type	Subtypes	Content
text	plain, enriched	unformatted or formatted text
multipart	mixed, parallel, alternative, digest	several parts in one message (in order, in any order, alternative versions, digest of messages)
message	rfc822, partial, external-body	an encapsulated message, a fragment of a large one, a pointer to one
image, video, audio	jpeg, gif; mpeg; basic	media
application	postscript, octet-stream	other data, e.g. arbitrary binary

S/MIME (Secure/MIME), based on RSA Data Security's technology and standardised by the IETF, adds security to MIME by defining new content types that carry signed and/or encrypted data in the PKCS#7 (Cryptographic Message Syntax) format. It provides four functions.

- **Enveloped data** (`application/pkcs7-mime; smime-type=envelopedData`): the sender generates a random session key, encrypts the content with it (3DES or AES), and for *each* recipient encrypts the session key with that recipient's public RSA key and builds a RecipientInfo block (recipient's certificate identifier, algorithm identifier, encrypted session key). The RecipientInfo blocks and the encrypted content are then base64 encoded. This gives confidentiality, and one message serves many recipients with one bulk encryption.
- **Signed data** (`smime-type=signedData`): the sender hashes the content (SHA), encrypts the digest with his private key, and builds a SignerInfo block (signer's certificate, digest and signature algorithm identifiers, encrypted digest). Content and signature are base64 encoded *together*, so only an S/MIME-capable reader can see the content.
- **Clear-signed data** (`multipart/signed`): the signature is computed the same way, but the message has two parts: the content in its normal MIME form, readable by anyone, and a detached signature (`application/pkcs7-signature`) in base64. Recipients without S/MIME can still read the message, though they cannot verify it.
- **Signed and enveloped data**: the two operations can be nested in either order, signing the plaintext and then enveloping it, or enveloping and then signing the ciphertext.

Certificates. S/MIME uses **X.509 version 3** certificates issued by certification authorities. Its key management is a hybrid of a strict CA hierarchy and PGP's web of trust: each client is configured with a list of trusted root certificates and certificate revocation lists, and a user agent must generate key pairs, request certificates from a CA, and store and retrieve certificates of its correspondents.

43.7 PGP versus S/MIME ▸

▸ **Likely this year:** "Differentiate PGP and S/MIME" (3–4 marks).

	PGP	S/MIME
Origin	one author (Zimmermann), free software; OpenPGP standard	RSA Data Security / IETF standard, built into mail clients
Trust model	web of trust: users sign each other's keys	hierarchical CAs with X.509v3 certificates
Certificate format	PGP's own key certificates	X.509v3
Message format	PGP's own binary format	MIME with PKCS#7 / CMS content types
Binary-to-text encoding	radix-64 with CRC	base64 (MIME transfer encoding)
Compression	ZIP, built in	optional (compressed-data type)
Readable without the software	no	yes for clear-signed messages
Typical users	individuals	organisations and industry

Both use the same cryptographic pattern: hash-and-sign with the sender's private key, then hybrid encryption with a one-time session key wrapped in the recipient's public key.

44 IP Security (IPSec)

44.1 Why security at the IP level; applications and services ▸

▸ **Likely this year:** "What are the applications and benefits of IPSec? List the services provided by AH and ESP" (4 marks).

Problem. IP itself offers no security: the source address of a packet can be forged, and any router or eavesdropper on the path can read or change the contents. Securing each application separately (as PGP does for mail) leaves all other traffic unprotected. **IPSec** adds authentication, integrity and confidentiality to IP itself, so that every packet between two hosts or two networks can be protected whatever application produced it. It is optional in IPv4 and was designed together with IPv6.

Applications. IPSec is used for secure branch-office connectivity over the Internet (a **virtual private network**, VPN, instead of leased lines), secure remote access for employees through an ISP, extranet and intranet connections with partners, and as a basis for electronic commerce, since all traffic between the sites is protected without changes to the applications.

Benefits. When IPSec is implemented in a firewall or router, it protects all traffic crossing the perimeter, while traffic inside the site carries no security overhead; and if all outside traffic must use IP through the firewall, IPSec there cannot be bypassed. Because it lies below the transport layer, it is **transparent to applications and to end users**: no software on the hosts has to change and users need no training or keys. It can also protect individual users where needed, and it secures routing (router advertisements and redirects really come from authorised routers).

Architecture and services. IPSec consists of two traffic protocols, the **Authentication Header** (AH), which authenticates packets, and the **Encapsulating Security Payload** (ESP), which encrypts and optionally authenticates them, plus a key-management protocol (IKE) and the documents that define the algorithms. Together they provide the following services.

Service	AH	ESP (encryption only)	ESP (encryption + authentication)
Access control	✓	✓	✓
Connectionless integrity	✓		✓
Data origin authentication	✓		✓
Rejection of replayed packets	✓	✓	✓
Confidentiality		✓	✓
Limited traffic-flow confidentiality		✓	✓

44.2 Security associations, SAD and SPD ★▸

▸ **Likely this year:** "What is a security association? What parameters identify it? Explain SAD and SPD" (4 marks).

Idea. Before two parties can protect packets they must agree on the protocol (AH or ESP), the algorithms, the keys, the lifetime and so on. IPSec stores such an agreement as a **security association** (SA): a one-way relationship between a sender and a receiver that affords security services to the traffic carried on it. Since it is one-way, a two-way conversation needs two SAs, one in each direction, and an SA covers one protocol only, AH or ESP. An SA is uniquely identified by three parameters:

- the **Security Parameters Index** (SPI), a 32-bit value chosen by the receiver and carried in every AH and ESP header, which lets the receiver select the SA for an arriving packet;
- the **IP destination address**, the address of the SA's endpoint (an end host or a firewall/router);

- the **security protocol identifier**, which says whether the SA is AH or ESP.

Security Association Database (SAD). Each IPSec host keeps one entry per SA with the parameters needed to process its packets: the sequence number counter and its overflow flag, the anti-replay window, the AH information (authentication algorithm and keys), the ESP information (encryption and authentication algorithms, keys, IVs), the lifetime of the SA, the IPSec mode (transport or tunnel) and the path MTU.

Security Policy Database (SPD). The SPD decides *which* traffic gets *which* protection. Each entry is defined by **selectors**, values of IP and upper-layer fields (destination and source IP addresses, user ID, data sensitivity level, transport protocol, source and destination ports), and specifies an action: **discard** the packet, **bypass** IPSec, or **protect** it with the SA(s) the entry points to. For every outbound packet IPSec compares the packet's fields with the SPD, finds the matching entry, determines the SA (creating one with IKE if none exists yet) and applies its processing. For an inbound packet the receiver uses the SPI, destination address and protocol to find the SA in the SAD, processes the packet, and then checks that the result is allowed by the SPD.

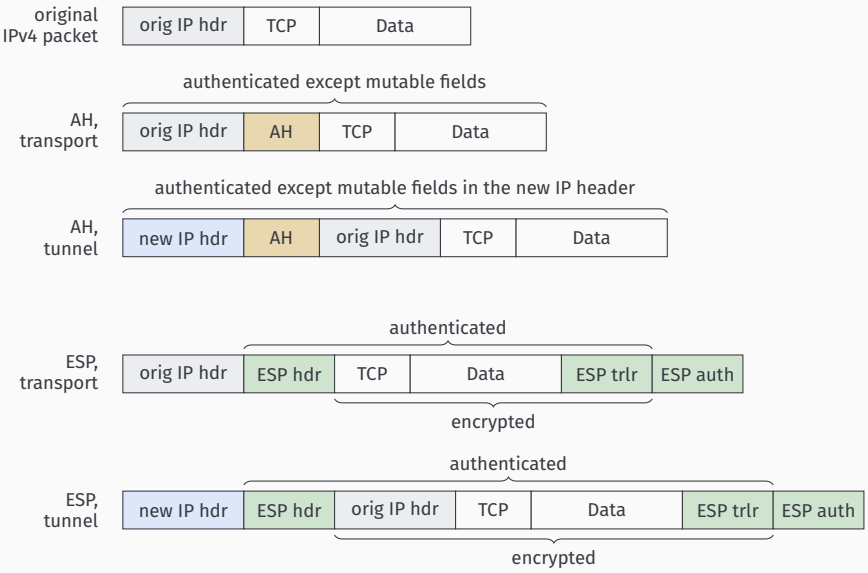
44.3 Transport mode and tunnel mode ★▸

▸ **Likely this year:** "Differentiate transport and tunnel mode; draw the packet formats for AH and ESP in both modes" (5 marks).

Both AH and ESP can be used in two modes, which differ in what is protected and in where the IPSec processing is done.

Transport mode protects the *payload* of the IP packet, i.e. the upper-layer segment (TCP or UDP header plus data). The AH or ESP header is inserted between the original IP header and the payload, and the original IP header stays visible. It is used for **end-to-end** communication between two hosts that both run IPSec, such as a client and a server.

Tunnel mode protects the *entire* original IP packet. After the AH or ESP fields are added, the whole packet (including its original header) becomes the payload of a **new outer IP packet** with a new header, normally addressed from one security gateway to another. Routers on the path see only the outer header, so the inner source and destination addresses are hidden. Tunnel mode is used when one or both ends of the SA are **security gateways** (firewalls or routers): hosts behind the gateways need no IPSec at all, and the gateways build a VPN between the sites.

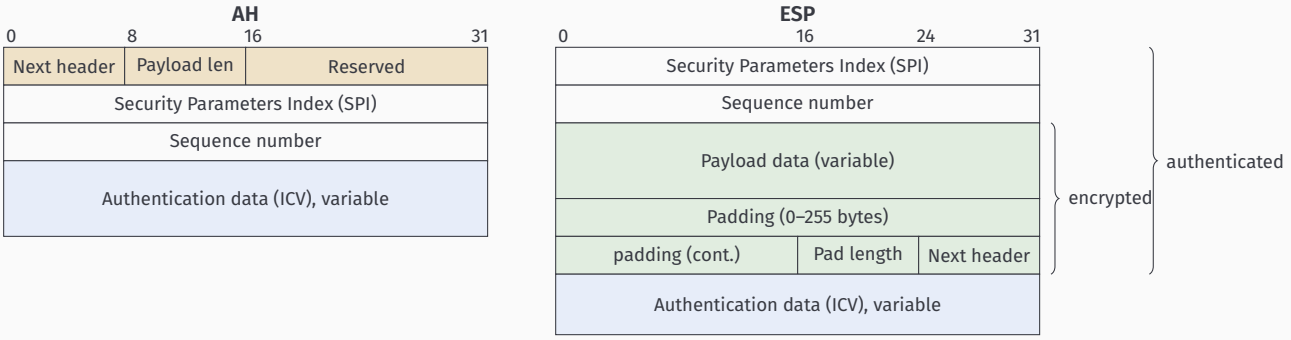


	Transport mode SA	Tunnel mode SA
AH	authenticates the IP payload and selected (immutable) parts of the IP header	authenticates the entire inner IP packet and selected parts of the outer IP header
ESP	encrypts the IP payload	encrypts the entire inner IP packet (inner header included)
ESP with authentication	encrypts and authenticates the IP payload, but not the IP header	encrypts and authenticates the entire inner IP packet
Endpoints	hosts (end-to-end)	usually security gateways (VPN)
Addresses seen on the path	real source and destination	only the gateways' addresses

44.4 Authentication Header (AH) ★▸

▸ **Likely this year:** "Draw the AH format and explain each field; which IP header fields are authenticated?" (4–5 marks).

Purpose. AH provides data integrity and data origin authentication of IP packets, plus anti-replay protection. It prevents address spoofing, since the source address is authenticated, and it stops replay through the sequence number. It is based on a MAC (HMAC, §35), so the two parties must share a secret key. AH provides *no* confidentiality.



Fields.

- **Next header** (8 bits) identifies the header that follows AH (e.g. TCP in transport mode, IP in tunnel mode).
- **Payload length** (8 bits) is the length of AH in 32-bit words, minus 2.
- **Reserved** (16 bits) for future use.
- **SPI** (32 bits) identifies the SA.
- **Sequence number** (32 bits) is a monotonically increasing counter used for anti-replay.
- **Authentication data** (variable, a multiple of 32 bits) holds the **integrity check value** (ICV), a MAC of the packet; the defaults were HMAC-MD5-96 and HMAC-SHA-1-96, i.e. the HMAC truncated to its first 96 bits.

What the ICV covers. The MAC must be the same at the sender and at the receiver, but some IP header fields legitimately change on the way. AH therefore computes the MAC over the IP header fields that are **immutable** in transit (version, header length, total length, identification, protocol, source address, destination address) or **mutable but predictable** at the endpoint (the destination address when source routing is used, which is set to its final value). The **mutable, unpredictable** fields (type of service, flags, fragment offset, time to live, header checksum) are set to zero for the calculation at both ends. The MAC also covers the AH header itself, with the authentication data field set to zero, and the entire upper-layer data, which is immutable. Because the source and destination addresses are inside the MAC, an attacker cannot change them without detection; this is also why AH does not work through NAT, which rewrites addresses.

WORKED EXAMPLE : AH length field and packet size

Q. AH uses HMAC-SHA-1-96. Find the value of the Payload length field and the size of the packet when a 37-byte TCP segment (20-byte header, 17 data bytes) is sent from a host with a 20-byte IPv4 header in transport mode and in tunnel mode.
A. Fixed AH fields: 1 + 1 + 2 + 4 + 4 = 12 bytes; ICV = 96 bits = 12 bytes; AH = 24 bytes = 6 words, so Payload length = 6 − 2 = **4**.
Transport: 20 + 24 + 37 = **81** bytes. Tunnel: new IP header + AH + inner packet = 20 + 24 + (20 + 37) = **101** bytes.
Check: tunnel mode costs exactly one extra 20-byte IP header.

44.5 Anti-replay service ▸

▸ **Likely this year:** "Explain the IPSec anti-replay window with an example" (3–4 marks).

Problem. In a replay attack the attacker records an authenticated packet and sends it again later; the MAC is still valid, so authentication alone cannot reject it. IP gives no ordering guarantee, so the receiver also cannot simply demand increasing numbers: packets may legitimately arrive out of order.

Sender side. When an SA is created, the sender's sequence counter is set to 0; for each packet it is incremented and written into the Sequence number field, so the first packet carries 1. The counter must not wrap around past $2^{32} - 1$: before that the SA is terminated and a new SA with a new key is negotiated, so no number repeats under one key.

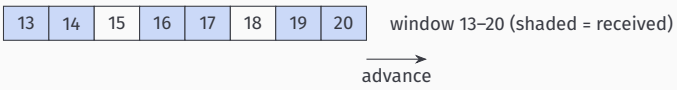
Receiver side. The receiver keeps a **window** of size W (default 64) whose right edge N is the highest sequence number received so far in a valid packet; for each number from $N - W + 1$ to N it remembers whether a valid packet with that number has arrived (marked slot). For an arriving packet:

1. if its number is **inside the window** and the slot is unmarked, the MAC is checked; if it is valid, the slot is marked;
2. if its number is **to the right of the window**, the MAC is checked; if it is valid, the window is advanced so that this number becomes the right edge, and its slot is marked;
3. if its number is **to the left of the window**, or the slot is already marked, or the MAC fails, the packet is discarded (and the event is logged).

The MAC is checked before the window moves, so a forged packet with a huge sequence number cannot push the window forward.

WORKED EXAMPLE : tracing the anti-replay window

Q. $W = 8$ and the right edge is $N = 20$, so the window is 13–20; valid packets 13, 14, 16, 17, 19, 20 have arrived. Packets arrive with numbers 18, 16, 25, 15, 21 (all with valid MACs). What happens to each?



Packet	Position	Action and new window
18	inside, slot unmarked	accept, mark 18; window 13–20
16	inside, slot marked	**replay**: discard
25	right of window	accept; window slides to 18–25 (marked 18, 19, 20, 25)
15	left of the new window	discard, although 15 was never received
21	inside, slot unmarked	accept, mark 21

Check: packet 15 shows the cost of a finite window: a genuine but very late packet is dropped; a larger W tolerates more reordering.

44.6 Encapsulating Security Payload (ESP) ★▸

▸ **Likely this year:** "Draw the ESP format; why is padding needed? Compare AH and ESP" (5 marks).

Purpose. ESP provides **confidentiality** (encryption of the payload), limited traffic-flow confidentiality, and optionally the same authentication, integrity and anti-replay services as AH, all with one header and trailer (format above).

Fields. SPI and **sequence number** are as in AH. **Payload data** (variable) is the transport-layer segment (transport mode) or the whole inner IP packet (tunnel mode), encrypted; if the cipher needs an IV (e.g. AES-CBC, §32), it is carried in the clear at the start of this field. **Padding** (0–255 bytes) is followed by the **pad length** (8 bits, the number of padding bytes) and the **next header** (8 bits, the type of the encrypted payload). **Authentication data** (variable) is the ICV, computed over the ESP packet from the SPI to the next-header field, i.e. *after* encryption.

Why padding. Padding is added for three reasons: a block cipher needs the plaintext (payload, padding, pad length and next header) to be a whole number of blocks; ESP requires the pad-length and next-header fields to end on a 32-bit boundary, so the ciphertext is a multiple of 4 bytes; and extra padding can hide the true length of the payload, which gives partial traffic-flow confidentiality.

Encrypt-then-MAC. Because the ICV is computed over the ciphertext, the receiver checks it first and discards bogus or replayed packets *before* spending time on decryption, which reduces the impact of denial-of-service attacks, and the two operations can be done in parallel. Unlike AH, ESP's authentication does *not* cover the IP header in front of the ESP header (in tunnel mode the inner header is covered, being inside the payload).

WORKED EXAMPLE : ESP padding and packet size

Q. ESP uses AES-CBC (16-byte blocks, 16-byte explicit IV) and HMAC-SHA-1-96 (12-byte ICV). The same 37-byte TCP segment as above is sent. Find the padding and the total packet size in transport and in tunnel mode (IPv4 header 20 bytes).
A. The encrypted part is payload + padding + pad length (1) + next header (1), a multiple of 16.
Transport: payload = 37; 37 + 2 = 39; padding = (−39) mod 16 = **9**; encrypted part = 48 bytes. Total = 20 (IP) +8 (SPI, seq.) +16 (IV) +48 + 12 (ICV) = **104** bytes.
Tunnel: payload = inner packet = 20 + 37 = 57; 57 + 2 = 59; padding = (−59) mod 16 = **5**; encrypted part = 64. Total = 20 + 8 + 16 + 64 + 12 = **120** bytes.
Check: 48 = 3 × 16 and 64 = 4 × 16.

44.7 AH versus ESP ▸

▸ **Likely this year:** "Differentiate AH and ESP" (3 marks).

	AH	ESP
Services	integrity, origin authentication, anti-replay	confidentiality, optional integrity/authentication, anti-replay, limited traffic-flow confidentiality
Encryption	none	payload (and inner header in tunnel mode)
Authentication covers	payload, AH header and immutable/predictable fields of the (outer) IP header	ESP header, payload and trailer only, not the IP header in front
Header/trailer	header only	header, trailer and ICV
Works through NAT	no (addresses are authenticated)	yes (transport/UDP encapsulation)
IP protocol number	51	50

Since ESP with authentication offers almost everything AH does, plus confidentiality, ESP is by far the more widely used; AH is chosen only when the outer IP header itself must be authenticated or when encryption is not allowed.

44.8 Combining security associations ▸

▸ **Likely this year:** "Explain transport adjacency and iterated tunnelling; give the basic combinations of SAs" (4 marks).

One SA implements either AH or ESP, but not both, and a packet may need different services between different pairs of nodes (end-to-end authentication between hosts, plus encryption between the two gateways). IPSec therefore allows a packet to be processed by a sequence of SAs, called an **SA bundle**. SAs are combined in two ways. In **transport adjacency** more than one security protocol is applied to the same IP packet without tunnelling, for example ESP and then AH in transport mode between the same two hosts; since both act at the same endpoints, only one level of combination is possible. In **iterated tunnelling** several layers of protocols are applied by IP tunnelling, one inside the other; each tunnel may start and end at a different IPSec node along the path, so many levels of nesting are possible.

Authentication plus confidentiality can be obtained in three ways: ESP with its authentication option; transport adjacency with an inner ESP SA (no authentication) and an outer AH SA, which has the advantage that the authentication now also covers the IP addresses; or a bundle of an inner AH transport SA and an outer ESP tunnel SA, which authenticates the plaintext before encryption, so the authentication data are protected by encryption and can be stored with the message for later checking.

Basic combinations (Stallings). Case 1: end-to-end security between two IPSec hosts (AH or ESP in transport mode, or one inside the other). Case 2: security only between two gateways, a simple VPN with one tunnel SA, the hosts doing nothing. Case 3: case 2 plus case 1, a gateway-to-gateway tunnel that carries end-to-end transport SAs between the hosts. Case 4: a remote host reaches the organisation's firewall over the Internet with a tunnel SA, and an inner SA protects the traffic all the way to the server inside.

44.9 Key management: Oakley, ISAKMP and IKE ★▸

▸ **Likely this year:** "How does Oakley improve Diffie–Hellman? Explain cookies (clogging attack) and nonces (replay)" (5 marks).

Problem. AH and ESP need shared secret keys: two pairs per two-way SA bundle (one for integrity, one for encryption in each direction). With **manual** key management an administrator configures each system with its own and its peers' keys, which only works for small static networks. **Automated** management creates keys on demand; the default for IPSec is **ISAKMP/Oakley**, later called the Internet Key Exchange (**IKE**). Oakley is a key-determination protocol based on Diffie–Hellman (§38); ISAKMP is the framework that defines the message formats and procedures for negotiating, establishing, modifying and deleting SAs.

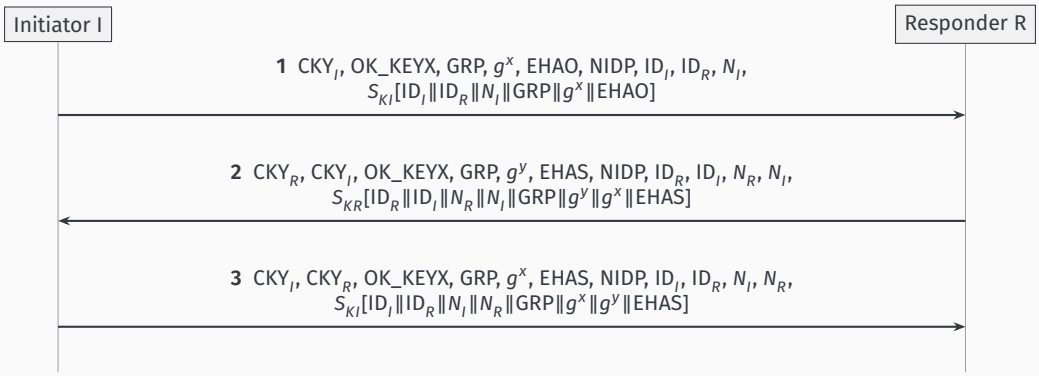
Weaknesses of plain Diffie–Hellman. Diffie–Hellman lets two parties create a shared secret $g^{xy} \bmod p$ over a public channel, but (1) it provides no information about the identities of the parties, so a man in the middle can run one exchange with each side (§38); and (2) it is computationally intensive, which opens a **clogging attack**: the attacker sends a flood of requests with forged source addresses, each forcing the victim to compute a modular exponentiation, until its processor is exhausted.

Oakley's improvements. Oakley keeps the advantages of Diffie–Hellman (keys are created only when needed, no pre-existing infrastructure beyond global parameters) and adds five features:

- **Cookies** to thwart clogging attacks.
- Negotiation of a **group**, i.e. the global parameters of the exchange (768- or 1024-bit modular exponentiation, or elliptic-curve groups).
- **Nonces** to ensure against replay attacks.
- Exchange of the Diffie–Hellman public values.
- **Authentication** of the exchange, by digital signatures, public-key encryption or a symmetric key, to thwart man-in-the-middle attacks.

Cookies. Each side sends a pseudorandom number, the cookie, in its first message, and the other side must echo it in the first message of the Diffie–Hellman exchange. A cookie is sent to the claimed source address,

so an attacker who forges the address never receives it and cannot echo it. As a result the attacker can only make the victim generate cookies, never perform the expensive exponentiation. A cookie must satisfy three requirements: it must depend on the specific parties (so that a cookie obtained with the attacker's real address cannot be reused from forged addresses and ports); it must be impossible for anyone but the issuer to generate a cookie the issuer will accept (so it depends on a local secret, which cannot be deduced from any cookie); and generating and checking it must be fast. The recommended construction is a fast hash over the source and destination IP addresses, the source and destination ports and a locally generated secret value, CKY = $H(\text{IP}_s \| \text{IP}_d \| \text{port}_s \| \text{port}_d \| \text{secret})$; the issuer stores nothing and simply recomputes the hash when the cookie comes back.



The aggressive Oakley exchange (Stallings) above uses three messages. CKY are cookies, OK_KEYX marks a key-exchange message, GRP names the Diffie–Hellman group, g^x and g^y are the public values, EHAO and EHAS are the encryption, hash and authentication algorithms offered and selected, NIDP says that the rest of the message is not encrypted, ID are identities, N are nonces, and $S_K[\cdot]$ is a signature with the sender's private key.

- **Message 1.** I sends its cookie, the group and its public value g^x, the algorithms it offers, both identities and a fresh nonce N_I, and signs the identities, nonce, group, g^x and offer with its private key. The signature binds g^x to I's identity, so a man in the middle cannot substitute his own value.
- **Message 2.** R returns its cookie and echoes I's, sends g^y, the selected algorithms and its nonce N_R, echoes N_I, and signs all of this *including* N_I and g^x. Signing I's fresh nonce proves that the reply is new and not a replay, and signing g^x confirms which exchange it answers.
- **Message 3.** I echoes R's cookie, which proves that I really receives messages at its claimed address, and signs both nonces and both public values, which proves to R that I is live and holds the private key.

Both sides then compute $K = g^{xy} \bmod p$ and derive the keys from it. The aggressive mode saves messages but makes R compute g^y before it has seen its own cookie echoed; the six-message **main mode** first exchanges cookies and the SA proposal (messages 1–2), then the Diffie–Hellman values and nonces (3–4), and only then the identities and authentication, encrypted under the new key (5–6), which also hides the identities from eavesdroppers.

ISAKMP and the IKE phases. An ISAKMP message starts with a header holding the initiator and responder cookies (64 bits each), the type of the next payload, version numbers, the exchange type, flags, a message ID and the length, followed by a chain of payloads (SA, proposal, transform, key exchange, identification, certificate, certificate request, hash, signature, nonce, notification, delete). IKE works in two phases. **Phase 1** (main or aggressive mode) authenticates the peers and builds a protected **IKE SA** between them. **Phase 2** ("quick mode") runs under the protection of the IKE SA and quickly negotiates the actual AH/ESP SAs, and can be repeated to create more SAs or to rekey without a new Diffie–Hellman exchange. The current version, IKEv2, does the same with four messages (IKE_SA_INIT and IKE_AUTH) and lets a busy responder answer the first message statelessly with a cookie, the same anti-clogging idea.

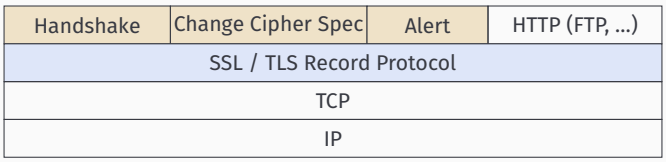
45 Web Security: SSL/TLS, HTTPS and SET

45.1 Web security threats and the SSL/TLS architecture ★▸

▸ **Likely this year:** "Explain the SSL architecture (protocol stack); differentiate an SSL session and an SSL connection" (4 marks).

Problem. The web is a client/server application over the Internet: anyone can publish a server, users are often unskilled, and web servers are complex and a doorway into an organisation's systems. The threats are to *integrity* (modification of user data or messages in transit), *confidentiality* (eavesdropping on the network, theft of data from the server or client), *authentication* (impersonation of legitimate users or servers, forged data) and *availability* (denial of service). Encryption, MACs and certificates answer the first three; denial of service is hard to prevent by cryptography.

Secure Sockets Layer (SSL, Netscape; version 3.0 in 1996) and its IETF successor **Transport Layer Security** (TLS 1.0, 1999, essentially SSL 3.1) provide a secure channel on top of TCP. SSL is not one protocol but two layers of protocols:



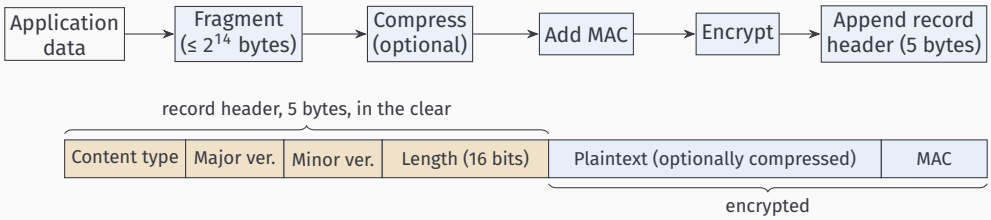
The **Record Protocol** provides confidentiality and integrity to whatever higher-layer data it carries, in particular HTTP. Three higher-layer protocols manage the channel: the **Handshake Protocol** authenticates the parties and negotiates algorithms and keys, the **Change Cipher Spec Protocol** switches to the newly negotiated parameters, and the **Alert Protocol** reports errors and closure.

Session and connection. An SSL **connection** is a transport (in the OSI sense) that provides a suitable type of service; connections are peer-to-peer, transient, and each is associated with one session. An SSL **session** is an association between a client and a server created by the Handshake Protocol; it defines a set of cryptographic security parameters that can be shared among *multiple connections*. Sessions exist to avoid the expensive negotiation of new parameters for every connection: a browser that opens ten connections to a site runs the full public-key handshake once and resumes the session for the others. The *session state* holds the session identifier, the peer's X.509 certificate, the compression method, the cipher spec (bulk encryption and MAC algorithms), the 48-byte **master secret** and an "is resumable" flag. The *connection state* holds the server and client randoms, the server and client write MAC secrets, the server and client write keys, the IVs and the sequence numbers; each direction has its own keys.

45.2 The Record Protocol ★▸

▸ **Likely this year:** "Explain the operation of the SSL Record Protocol with a diagram" (4 marks).

The Record Protocol takes an application message and processes it in five steps.



1. **Fragmentation.** The message is split into blocks of at most 2^{14} = 16 384 bytes.
2. **Compression** (optional, lossless). It must not increase the length by more than 1024 bytes; in practice it is not used.
3. **MAC.** A MAC is computed over the (compressed) fragment with the shared MAC write secret. In SSLv3 it is a precursor of HMAC, $\text{hash}(K \| \text{pad}_2 \| \text{hash}(K \| \text{pad}_1 \| \text{seq_num} \| \text{type} \| \text{length} \| \text{fragment}))$; TLS uses true HMAC over seq_num‖type‖version‖length‖fragment. The 64-bit **sequence number** is not transmitted; both sides count records, so a deleted, reordered or replayed record makes the MAC fail.
4. **Encryption.** The fragment and MAC are encrypted with the write key, using a block cipher in CBC mode (DES, 3DES, IDEA, AES; §32) with padding, or a stream cipher (RC4). Encryption may add at most 1024 bytes, so a record body is at most $2^{14} + 2048$ bytes.
5. **Header.** A 5-byte header is prepended: **content type** (8 bits: change_cipher_spec, alert, handshake or application_data), **major and minor version** (8 bits each; 3.0 for SSLv3, 3.1 for TLS 1.0) and the **length** of the record body in bytes (16 bits).

The receiver decrypts, verifies the MAC, decompresses and reassembles before delivering the data to the application. SSL/TLS up to version 1.2 is **MAC-then-encrypt**, the opposite of IPSec ESP; this order later enabled padding-oracle attacks on CBC (Lucky 13, POODLE) and is one reason TLS 1.3 abandoned it.

45.3 Change Cipher Spec and Alert protocols ▸

▸ **Likely this year:** "What is the purpose of the Change Cipher Spec and Alert protocols?" (2 marks).

The **Change Cipher Spec Protocol** is the simplest of the three: it consists of a single message of one byte with value 1. Its effect is to copy the *pending* state, negotiated during the handshake, into the *current* state, so that all following records use the new cipher suite and keys. Each direction switches separately, when its sender sends this message.

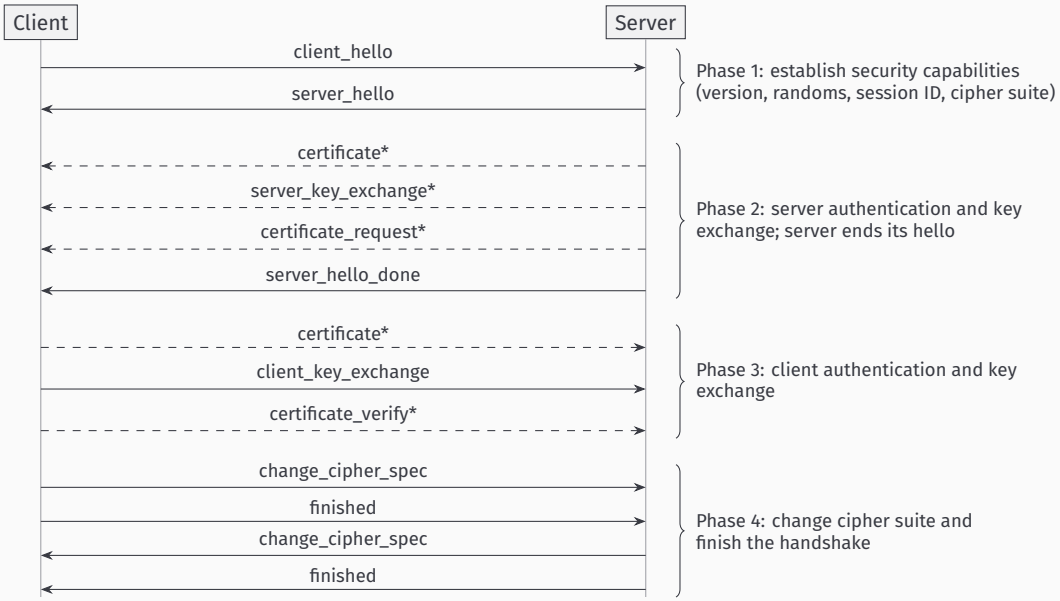
The **Alert Protocol** conveys SSL-related alerts to the peer. Each alert message has two bytes: a **level**, warning (1) or fatal (2), and a **code**. Fatal alerts (unexpected_message, bad_record_mac, decompression_failure,

handshake_failure, illegal_parameter) make SSL terminate the connection immediately; other connections of the same session may continue, but no new connections may be opened on the session. Warnings include close_notify (the sender will send no more data; each side must send it before closing the write side, which detects truncation attacks), no_certificate, bad_certificate, unsupported_certificate, certificate_revoked, certificate_expired and certificate_unknown.

45.4 The Handshake Protocol ★▸

▸ **Likely this year:** "Draw and explain the SSL/TLS handshake in four phases" (6–8 marks).

Problem. Before any data are protected, the client and server must agree on a protocol version and a cipher suite, the server (and optionally the client) must be authenticated, and both must end up with the same fresh secret keys, all over a channel that the attacker controls. The Handshake Protocol does this in four phases; each message has a 1-byte type, a 3-byte length and the content.



Messages marked * are optional or depend on the key-exchange method (dashed).

Phase 1: establish security capabilities. The client sends **client_hello** with: the highest SSL/TLS **version** it understands; a **random** of 32 bytes (a 4-byte timestamp and 28 bytes from a secure generator), which serves as a nonce against replay; a **session ID** (zero for a new session, nonzero to resume an existing one); the **cipher suite** list in decreasing order of preference, each entry naming a key-exchange method (RSA, fixed Diffie–Hellman, ephemeral Diffie–Hellman, anonymous Diffie–Hellman, Fortezza) and a cipher spec (bulk cipher, MAC algorithm, cipher type, hash size, key material, IV size); and the compression methods it supports. The server replies with **server_hello**: the lower of the two highest versions, its own independent random, the session ID, and the *one* cipher suite and compression method it selected from the client's lists.

Phase 2: server authentication and key exchange. The server sends its **certificate** (an X.509 chain), required for every method except anonymous Diffie–Hellman. It sends **server_key_exchange** only when the certificate does not already contain what the client needs: for anonymous or ephemeral Diffie–Hellman (the server's p, g and $g^y \bmod p$) or when the server's RSA key is for signing only. These parameters are **signed** by the server over hash(ClientHello.random‖ServerHello.random‖ServerParams); including both randoms ties the signature to this handshake, so an old signed message cannot be replayed. A server that wants client authentication sends **certificate_request** (acceptable certificate types and CAs). Finally **server_hello_done**, which is always sent, says that the server's part of the hello is complete, and the server waits.

Phase 3: client authentication and key exchange. The client checks that the server's certificate is valid (signature chain, dates, host name) and that the server's parameters are acceptable. If a certificate was requested, it sends its **certificate** (or a no_certificate alert). It then sends **client_key_exchange**, which is mandatory: with RSA, the client generates a 48-byte **pre-master secret** and sends it encrypted under the server's public key; with ephemeral or anonymous Diffie–Hellman it sends its public value $g^x \bmod p$; with fixed Diffie–Hellman the value is in its certificate and the message is empty. If the client sent a certificate with signing capability, it also sends **certificate_verify**, a signature over a hash of all preceding handshake messages and the master secret. This proves that the client owns the private key of its certificate: anyone could send a copy of the certificate, but only the owner can sign the transcript.

Phase 4: finish. The client sends **change_cipher_spec** (through the Change Cipher Spec Protocol, not the handshake) and copies the pending cipher spec into the current one, then immediately sends **finished** under the new

algorithms and keys. Finished contains a keyed hash of the master secret and the whole handshake transcript; in TLS 1.0/1.1

$$\text{verify_data} = \text{PRF}\big(\text{master_secret}, \text{“client finished”}, \text{MD5}(\text{handshake_msgs}) \,\|\, \text{SHA-1}(\text{handshake_msgs})\big)[0..11].$$

The server does the same: its own change_cipher_spec and finished. Since each side checks the other's Finished against its own view of the transcript, **any tampering with the earlier, unprotected handshake messages is detected**, for example an attacker who deleted the strong cipher suites from client_hello to force a weak one (a downgrade attack). After Phase 4 the client and server exchange application data.

45.5 Master secret and key derivation ▸

▸ **Likely this year:** “How are the master secret and the session keys generated in SSL/TLS?” (3–4 marks).

The keys are created in two stages. First a **pre-master secret** is shared: with RSA the client generates it and sends it encrypted under the server's public key; with Diffie–Hellman both sides compute $g^{xy} \bmod p$. Then both sides compute the 48-byte **master secret** from it and the two randoms. In SSLv3,

$$\begin{aligned}\text{master_secret} = {} & \text{MD5}(\text{pre_master} \| \text{SHA}(\text{‘A’} \| \text{pre_master} \| \text{ClientHello.random} \| \text{ServerHello.random})) \,\| \\ & \text{MD5}(\text{pre_master} \| \text{SHA}(\text{‘BB’} \| \text{pre_master} \| \text{ClientHello.random} \| \text{ServerHello.random})) \,\| \\ & \text{MD5}(\text{pre_master} \| \text{SHA}(\text{‘CCC’} \| \text{pre_master} \| \text{ClientHello.random} \| \text{ServerHello.random})),\end{aligned}$$

three 16-byte MD5 outputs. TLS replaces this by a **pseudorandom function** built from HMAC:

$$\text{master_secret} = \text{PRF}(\text{pre_master_secret}, \text{“master secret”}, \text{ClientHello.random} \| \text{ServerHello.random})[0..47],$$

$$P_\text{hash}(\text{secret}, \text{seed}) = \text{HMAC}(\text{secret}, A(1) \| \text{seed}) \,\|\, \text{HMAC}(\text{secret}, A(2) \| \text{seed}) \,\|\, \cdots,$$

$$A(0) = \text{seed}, \qquad A(i) = \text{HMAC}(\text{secret}, A(i-1)),$$

which expands a short secret into as many bytes as needed (TLS 1.0/1.1 XOR a P_MD5 and a P_SHA-1 stream computed on the two halves of the secret; TLS 1.2 uses P_SHA-256). Finally the master secret is expanded into the **key block**, PRF(master_secret, “key expansion”, ServerHello.random‖ClientHello.random), which is cut into the client and server write MAC secrets, the client and server write keys, and the client and server IVs.

Why both randoms. The master secret depends on both parties' fresh randoms, so neither side alone controls the keys, and every session has new keys even if a pre-master secret were reused. In particular, an attacker who replays a recorded client_key_exchange to the server gets nowhere: the server has chosen a new random, so the replay leads to a different master secret and fails at the Finished check.

WORKED EXAMPLE : size of the TLS key block

Q. A TLS 1.0 session uses the cipher suite TLS_RSA_WITH_AES_128_CBC_SHA. How many bytes of key block must the PRF produce, and how many iterations of P_SHA-1 and P_MD5 does that need?
A. MAC secrets: HMAC-SHA-1, 20 bytes each, $2 \times 20 = 40$. Write keys: AES-128, 16 bytes each, $2 \times 16 = 32$. IVs: one AES block each, $2 \times 16 = 32$. Key block = $40 + 32 + 32$ = **104 bytes**. P_SHA-1 gives 20 bytes per iteration: $\lceil 104/20 \rceil$ = **6**; P_MD5 gives 16: $\lceil 104/16 \rceil$ = **7**; both streams are truncated to 104 bytes and XORed.
Check: for TLS_RSA_WITH_3DES_EDE_CBC_SHA the block is $2 \times 20 + 2 \times 24 + 2 \times 8 = 104$ bytes as well (24-byte 3DES keys, 8-byte IVs).

45.6 From SSL to TLS 1.3 ▸

▸ **Likely this year:** “What are the differences between SSL and TLS?” / “What did TLS 1.3 change?” (3 marks).

TLS 1.0–1.2 versus SSLv3. The version number is 3.1–3.3; the MAC is standard HMAC with the version in its input; keys come from the HMAC-based PRF instead of SSL's ad hoc MD5/SHA construction; there are more alert codes; Fortezza is dropped from the cipher suites; the certificate_verify and finished computations use the PRF; and the padding before encryption may be any amount up to 255 bytes (to hide message lengths) instead of the minimum.

TLS 1.3 (RFC 8446, 2018) is a redesign. It removes RSA key transport and static Diffie–Hellman, keeping only ephemeral (EC)DHE, so that every session has **forward secrecy** (stealing the server's private key later does not decrypt recorded sessions). It removes compression, CBC MAC-then-encrypt and RC4, allowing only AEAD ciphers (AES-GCM, ChaCha20-Poly1305). The handshake takes one round trip: the client sends its key share in client_hello, and everything after server_hello, including the server certificate, is encrypted. Keys are derived with HKDF, renegotiation is gone, Change Cipher Spec survives only as a dummy for middlebox compatibility, and a resumed session can send data in its first flight (0-RTT), at the cost of possible replay of that data.

45.7 HTTPS ▸

▸ **Likely this year:** "What is HTTPS? Which parts of an HTTP exchange are protected?" (2 marks).

HTTPS is HTTP over SSL/TLS. The URL begins with `https://` and the server listens on port **443** instead of 80. When HTTPS is used, the URL of the requested document, the document contents, the contents of browser forms, the cookies sent in both directions and the contents of the HTTP headers are all encrypted; only the server's IP address and port (and, in older TLS, the host name in the certificate) are visible. The client opens a TCP connection, runs the TLS handshake, and only then sends the first HTTP request. The browser checks that the server's certificate is valid and issued for the host name in the URL; this check is what stops a man in the middle, who can present a certificate only for his own name. At the end, each side sends a TLS close_notify alert before closing the TCP connection, so that an attacker who forges a TCP close cannot silently truncate a page.

45.8 SET: purpose and participants ▸

▸ **Likely this year:** "What is SET? List its participants and requirements" (4 marks).

Secure Electronic Transaction (SET), specified by MasterCard and Visa in 1996, is an open encryption and security specification for protecting **credit card transactions** on the Internet. It is not a payment system itself but a set of security protocols and formats that lets users use the existing card payment infrastructure securely over an open network. SET provides a secure channel among all parties, trust through X.509v3 certificates, and **privacy**, because information is available to a party only when and where it is needed. Its business requirements are: confidentiality of payment and ordering information; integrity of all transmitted data; authentication that the cardholder is a legitimate user of the card account; authentication that the merchant can accept card payments through a relationship with a financial institution; use of the best security practices; and independence from transport-level security and interoperability across platforms. Confidentiality used DES, integrity RSA signatures with SHA-1, and authentication X.509v3 certificates.

Participants.

- The **cardholder**, an authorised holder of a payment card issued by an issuer, who buys over the Internet.
- The **merchant**, who sells goods or services and must have a relationship with an acquirer to accept cards.
- The **issuer**, the financial institution (bank) that issued the card and is responsible for the cardholder's payments.
- The **acquirer**, the financial institution that keeps the merchant's account, authorises card transactions (the card is valid and the purchase within the limit) and transfers payments to the merchant's account; it is later reimbursed by the issuer over the payment network.
- The **payment gateway**, operated by the acquirer or a third party, which processes the merchant's SET payment messages and interfaces between SET on the Internet and the existing bank-card networks.
- The **certification authority** (CA), trusted to issue X.509v3 certificates to cardholders, merchants and payment gateways, in a hierarchy of CAs.

Sequence of events. (1) The customer opens a card account and (2) receives a certificate signed by the bank, which binds his RSA public key to the card. (3) Merchants have two certificates, one for signing and one for key exchange, and hold a copy of the payment gateway's certificate. (4) The customer places an order. (5) The merchant sends a copy of its certificate, so the customer can verify that it is a valid store. (6) The customer sends the order and the payment information together with his certificate; the payment information is encrypted so that the merchant cannot read it. (7) The merchant forwards the payment information to the payment gateway and requests authorisation. (8) The merchant confirms the order to the customer, (9) provides the goods or service, and (10) later requests payment from the gateway, which handles all payment processing.

45.9 SET: the dual signature ★▸

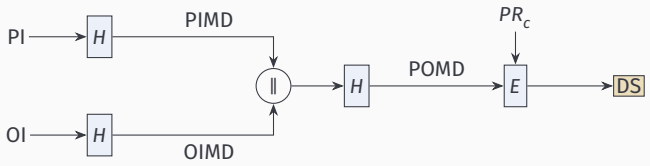
▸ **Likely this year:** "What is a dual signature? How is it constructed and how do the merchant and the bank verify it?" (5 marks).

Problem. A purchase involves two messages for two recipients: the **order information** (OI), meant for the merchant, and the **payment information** (PI, the card details), meant for the bank. The merchant does not need the card number, and the bank does not need to know what was bought, so each should see only its own message. Yet the two must be **linked**: the customer needs to be able to prove that this payment is for this order and no other. If the customer signed OI and PI separately, a dishonest merchant could take the signed PI from one order and claim it pays for a different, more expensive order.

Construction. The customer hashes each message separately, hashes the concatenation of the two digests, and signs that:

$$\text{PIMD} = H(\text{PI}), \qquad \text{OIMD} = H(\text{OI}), \qquad \text{POMD} = H(\text{PIMD} \,\|\, \text{OIMD}), \qquad \text{DS} = E(PR_c, \text{POMD}),$$

where H is SHA-1, PR_c is the customer's private signature key, and DS is the **dual signature**.



Verification by the merchant. The merchant receives OI, PIMD (not PI) and DS, and has the customer's public key PU_c from his certificate. He computes

$$H\big(\text{PIMD} \,\|\, H(\text{OI})\big) \quad \text{and} \quad D(PU_c, \text{DS}),$$

and accepts if they are equal. The second value is POMD as signed by the customer. The first equals POMD exactly when the OI he received is the one the customer hashed, because $H(\text{OI}) = \text{OIMD}$; PIMD is used as received, so the merchant never needs PI.

Verification by the bank. The bank (through the payment gateway) receives PI, OIMD (not OI) and DS, and computes

$$H\big(H(\text{PI}) \,\|\, \text{OIMD}\big) \quad \text{and} \quad D(PU_c, \text{DS}),$$

which are equal exactly when the PI is the one the customer signed, so the bank never needs OI.

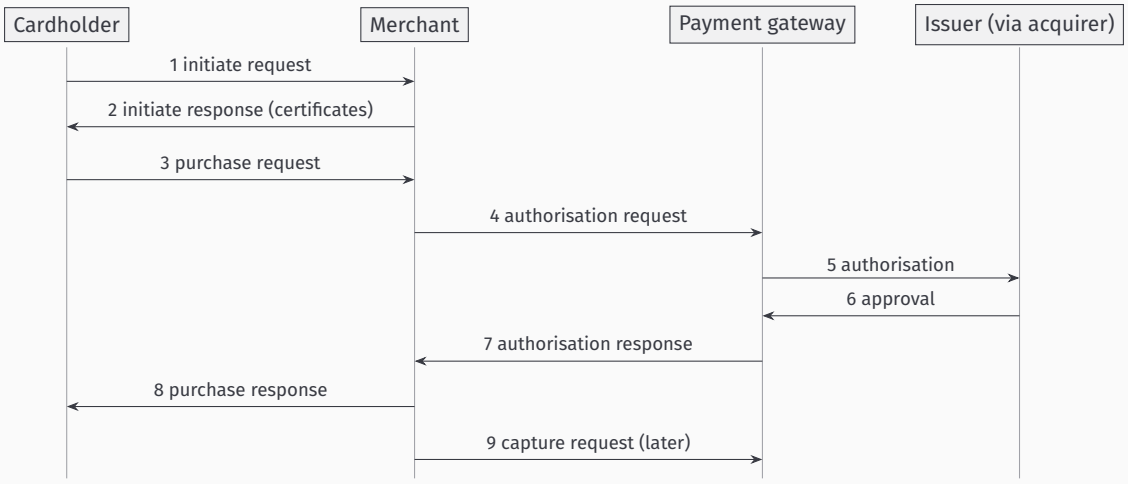
Why the link holds. Both checks compare against the same signed value POMD, which depends on both PIMD and OIMD. If the merchant presented the payment with a different order OI′, the bank and a judge would compute $H(\text{PIMD}\|H(\text{OI}'))$, and since H is collision resistant this differs from POMD, so the signature does not match. Only the customer, holding PR_c, can produce a DS for a new pair. Thus the dual signature lets each party verify its own message while seeing only the digest of the other, and binds the order and the payment together.

WORKED EXAMPLE : a toy dual signature

Q. Let the hash be the toy function $H(x) = (7x + 3) \bmod 101$, let the concatenation of two digests $a\|b$ be the number $1000a + b$, and let the customer's RSA key be $n = 187$, $e = 7$, $d = 23$ (§33). For PI = 2345 and OI = 678, compute the dual signature and show both verifications.
A. PIMD = $(7 \cdot 2345 + 3) \bmod 101 = 16\,418 \bmod 101 = 56$. OIMD = $(7 \cdot 678 + 3) \bmod 101 = 4749 \bmod 101 = 2$.
PIMD‖OIMD = 56 002; POMD = $(7 \cdot 56\,002 + 3) \bmod 101 = 392\,017 \bmod 101 = 36$.
DS = $36^{23} \bmod 187 = \mathbf{60}$.
Merchant (has OI, PIMD = 56, DS): $H(678) = 2$, $H(56\,002) = 36$; $60^7 \bmod 187 = 36$. Equal: accepted.
Bank (has PI, OIMD = 2, DS): $H(2345) = 56$, $H(56\,002) = 36$; $60^7 \bmod 187 = 36$. Equal: accepted.
Check: a changed order OI′ = 679 gives $H(679) = 9$ and $H(56\,009) = 85 \neq 36$, so the substitution is detected.

45.10 SET: purchase request and payment authorisation ▸

▸ **Likely this year:** "Explain the SET purchase request (and payment authorisation) with a diagram" (5 marks).



Initiate request and response. After browsing and choosing, the cardholder asks the merchant for the certificates of the merchant and of the payment gateway. The merchant answers with a message signed with its private signature key that contains a transaction ID and both certificates. The cardholder verifies the certificates through their CA signatures and creates OI and PI, each containing the transaction ID.

Purchase request. The cardholder generates a one-time symmetric key K_s and sends

$$\underbrace{E_{K_s}(\text{PI} \,\|\, \text{DS} \,\|\, \text{OIMD}) \,\|\, E(PU_g, K_s)}_{\text{digital envelope, forwarded to the gateway}} \;\|\; \underbrace{\text{PIMD} \,\|\, \text{OI} \,\|\, \text{DS}}_{\text{for the merchant}} \;\|\; \text{Cert}_c,$$

where PU_g is the gateway's key-exchange public key. The merchant cannot open the envelope, so he never sees the card details. He verifies the cardholder's certificate and the dual signature as above, processes the order, forwards the envelope to the gateway, and returns a signed **purchase response** with the transaction ID.

Payment authorisation. The merchant's **authorisation request** to the gateway contains the cardholder's envelope, an authorisation block with the transaction ID signed with the merchant's private key and encrypted under a new one-time key (in its own digital envelope for the gateway), and the certificates. The gateway verifies all certificates, opens both envelopes, verifies the merchant's signature and the **dual signature**, checks that the transaction ID in the merchant's block matches the one in PI, and obtains authorisation from the issuer over the bank network. It returns a signed **authorisation response** with a capture token. Later the merchant sends a **capture request** with the token to obtain the money.

Advantages and disadvantages. SET gave strong privacy (the merchant never learns the card number, the bank never learns the order) and strong authentication of all parties by certificates. It failed in practice because every cardholder needed software and a certificate, and the many public-key operations made it slow and expensive. Card payments today use TLS between browser and merchant plus issuer-side checks (3-D Secure) and tokenisation, but the dual-signature idea remains the standard example of linking two messages for different recipients.

46 System Security: Intruders, Malicious Software, Firewalls and Trusted Systems

46.1 Intruders ▸

▸ **Likely this year:** "Classify intruders; list intrusion techniques" (3 marks).

The protocols above protect data on the wire; the remaining threats attack the computer systems themselves. One of the two most publicised threats is the **intruder**, a hacker or cracker who gains, or tries to gain, unauthorised access (the other is malicious software). Anderson's classification has three classes.

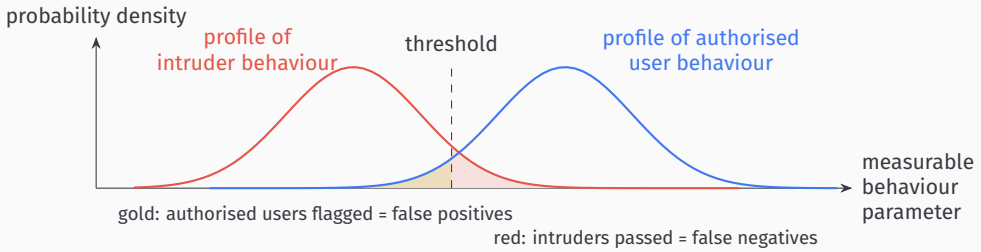
- A **masquerader** is an individual who is not authorised to use the computer and who penetrates the system's access controls to exploit a legitimate user's account. Masqueraders are usually outsiders.
- A **misfeasor** is a legitimate user who accesses data, programs or resources for which he is not authorised, or who is authorised but misuses his privileges. Misfeasors are usually insiders.
- A **clandestine user** is an individual who seizes supervisory (administrator) control of the system and uses it to evade auditing and access controls or to suppress audit collection. He may be an outsider or an insider.

Intrusion techniques. The intruder's goal is to gain access to a system or to increase his privileges, and the information he needs most is a password. The system protects its password file either by a one-way function (only hashed passwords are stored) or by access control. The intruder therefore tries default passwords shipped with the system, all short passwords, words from dictionaries and lists of likely passwords, and personal data of the users (names, birthdays, phone numbers, car numbers); he also installs a Trojan horse that captures passwords, or taps the line between a remote user and the host.

46.2 Intrusion detection ▸

▸ **Likely this year:** "Differentiate statistical anomaly detection and rule-based detection; what is a honeypot?" (4 marks).

Idea. Prevention eventually fails, so a second line of defence watches for intruders. Intrusion detection is motivated by three facts: an intruder detected quickly can be ejected before he does damage; an effective detection system deters intrusions; and it collects information about intrusion techniques that strengthens prevention. It rests on the assumption that the behaviour of an intruder differs from that of a legitimate user in ways that can be quantified. The two behaviour profiles overlap, however, so every detector trades **false positives** (legitimate users flagged as intruders) against **false negatives** (intruders missed): a looser threshold catches more intruders and annoys more users.



Approaches. Statistical anomaly detection collects data on the behaviour of legitimate users over a period of time and applies statistical tests to observed behaviour to decide with high confidence whether it is legitimate. It has two forms: *threshold detection*, which sets user-independent thresholds on the frequency of events (e.g. failed logins per minute), and *profile-based* detection, which builds a profile of each user's activity (login times,

programs used, CPU and I/O usage) and detects changes in it, using counters, gauges, interval timers and tests such as mean and standard deviation, multivariate correlation and Markov models. **Rule-based detection** defines a set of rules that decide whether a given behaviour is that of an intruder: *anomaly* rules are derived from past usage patterns, and *penetration identification* is an expert system that searches for known suspicious sequences (attack signatures). Statistical detection is effective against **masqueraders**, who are unlikely to imitate the behaviour of the accounts they steal, but not against misfeasors, who act within their normal profile; rule-based penetration identification catches misfeasors and known attacks, since it recognises the events themselves.

Audit records. Detection runs on audit records of user activity. *Native* audit records come from the operating system's accounting software, so no extra software is needed, but they may lack the needed information; *detection-specific* records are generated for the detector alone, which makes them vendor-independent at the cost of running two accounting packages. Denning's record has six fields: **subject** (initiator of the action), **action** (login, read, execute, ...), **object** (file, program, device), **exception condition** (any error raised), **resource usage** (CPU time, lines printed, records read) and **time-stamp**. For example, the command `COPY GAME.EXE TO <Library>GAME.EXE` issued by Smith produces records such as (Smith, execute, `<Library>COPY.EXE`, 0, CPU = 00002, 11058721678) and (Smith, write, `<Library>GAME.EXE`, write-violation, RECORDS = 0, 11058721679).

Honeypots. A **honeypot** is a decoy system designed to lure a potential attacker away from critical systems. It is filled with fabricated information that looks valuable but that no legitimate user would ever access, so *any* access to it is suspect, and it is instrumented with monitors and event loggers. It diverts the attacker, collects information about his activity, and encourages him to stay long enough for the administrators to respond.

46.3 Password management ▸

▸ **Likely this year:** "Explain the UNIX password scheme with salt; why is the salt used?" (4 marks), possibly with a cost calculation.

The UNIX scheme. Each user selects a password of up to eight printable characters, which is converted into a 56-bit value (7-bit ASCII) and used as the *key* of an encryption routine. The routine, `crypt(3)`, is DES modified by a 12-bit **salt**, which in older systems is derived from the time the password was assigned; it encrypts a 64-bit block of zeros, feeds the output back as input, and repeats 25 times. The 64-bit result is translated into an 11-character string, and the password file stores, for each user ID, this hashed password together with a *plaintext copy* of the salt. At login the user gives his ID and password; the system uses the ID to find the salt and the stored hash, runs crypt on the typed password with that salt, and compares the result with the stored value.

Why the salt. The salt serves three purposes. It prevents duplicate passwords from being visible in the password file: two users with the same password almost certainly get different salts and hence different hashes. It effectively lengthens the password without the user remembering more: each password has $2^{12} = 4096$ possible hashed forms, so a precomputed dictionary must be 4096 times larger. And the modified DES defeats off-the-shelf DES hardware for brute-force guessing. Since the hash is one-way, the scheme does not need to keep the file secret, but in practice the hashes are moved to a **shadow password file** readable only by the system, and modern systems use salted, deliberately slow hashes (bcrypt, PBKDF2, scrypt, Argon2) because a stolen file can be attacked offline.

Password selection strategies. Users choose guessable passwords, so four strategies are used: *user education* (often ignored); *computer-generated* passwords (random, hence hard to remember and likely to be written down); *reactive checking*, in which the system periodically runs its own password cracker and cancels the passwords it guesses (costly, and weak passwords stay vulnerable until found); and **proactive checking**, in which the system checks a password at the moment it is chosen against rules (length, character mix) and a dictionary of bad passwords, rejecting weak ones while still letting the user choose a memorable password.

WORKED EXAMPLE : what the salt costs the attacker

Q. An attacker has a stolen UNIX password file with 1000 users, all with different salts, and a dictionary of 2^{20} candidate passwords. Compare the number of crypt evaluations needed to test the dictionary against every user (a) if there were no salt, (b) with 12-bit salts; and (c) the size of a table that precomputes the dictionary for every salt.

A. (a) Without salt one pass of 2^{20} hashes, each compared with all 1000 stored hashes, tests every user at once: $\mathbf{2^{20}} \approx 10^6$ evaluations.

(b) With salt each user's hashes must be computed with his own salt: $1000 \times 2^{20} \approx \mathbf{1.05 \times 10^9}$ evaluations, 1000 times more.

(c) $2^{20} \times 2^{12} = \mathbf{2^{32}} \approx 4.3 \times 10^9$ entries, 4096 times the size of the plain dictionary.

Check: the salt does not make any single password harder to guess (2^{20} per user); it removes the attacker's ability to share work across users.

46.4 Malicious software: taxonomy ★▸

▸ **Likely this year:** "Define virus, worm, Trojan horse, logic bomb, trapdoor, zombie" / "Differentiate virus and worm" (4–5 marks).

Malicious software (malware) is software designed to damage or misuse a system. Stallings divides it into programs that **need a host program** (they are fragments that cannot exist independently: trapdoors, logic bombs, Trojan horses, viruses) and **independent** programs that the operating system can schedule and run (worms, zombies), and, across that, into programs that **replicate** (viruses, worms) and those that do not.

Type	Needs host	Replicates	Definition
Trapdoor (backdoor)	yes	no	a secret entry point into a program that lets someone who knows it gain access without the usual security procedures; legitimately used by programmers for debugging (maintenance hooks)
Logic bomb	yes	no	code embedded in a legitimate program that "explodes" when certain conditions are met (a date, the presence or absence of a file, a particular user), then deletes data or halts the machine
Trojan horse	yes	no	a useful or apparently useful program containing hidden code that, when invoked, performs an unwanted or harmful function, typically one the attacker could not perform directly
Virus	yes	yes	a program that infects other programs by modifying them to include a copy of itself, which then infects further programs
Worm	no	yes	a program that replicates itself and sends copies from computer to computer across network connections
Zombie (bot)	no	no*	a program that secretly takes over an Internet-attached computer and uses it to launch attacks that are hard to trace to its creator (DDoS, spam); many bots form a botnet
Ransomware	no*	varies	malware that encrypts the victim's files and demands payment for the decryption key

*Bots and ransomware are usually installed by a worm or a Trojan horse, which does the spreading.

The difference to remember between a virus and a worm is that a virus needs a host program and spreads when infected programs are run or copied, whereas a worm is a standalone program that actively propagates itself across the network. Other common classes are spyware and keyloggers, which steal information, and rootkits, which hide the attacker's presence by modifying the operating system.

46.5 Viruses: phases, structure and types ★▸

▸ **Likely this year:** "Explain the phases of a virus and the types of viruses" (5 marks).

Phases. During its lifetime a typical virus goes through four phases.

- **Dormant phase**: the virus is idle and will eventually be activated by some event, such as a date, the presence of another program or file, or the disk exceeding some capacity. Not all viruses have this phase.
- **Propagation phase**: the virus places an identical copy of itself into other programs or into certain system areas on the disk. Each infected program now contains a clone of the virus, which will itself enter a propagation phase.
- **Triggering phase**: the virus is activated to perform the function for which it was intended, caused by a system event, which can include the count of how many times this copy has replicated itself.
- **Execution phase**: the function is performed. It may be harmless (a message on the screen) or damaging (destruction of programs and data files).

Structure. A virus can be prepended or appended to an executable program, or embedded in it some other way. The key is that when the infected program is invoked, the virus code runs first and the original program runs afterwards, so the user notices nothing.

Structure of a simple virus (Stallings)

```
program V :=
{goto main;
  1234567;
  subroutine infect-executable :=
    {loop: file := get-random-executable-file;
     if (first-line-of-file = 1234567) then goto loop
     else prepend V to file; }
  subroutine do-damage := {whatever damage is to be done}
  subroutine trigger-pulled := {return true if some condition holds}
main: main-program :=
    {infect-executable;
     if trigger-pulled then do-damage;
     goto next;}
next:
}
```

The marker `1234567` lets the virus recognise programs it has already infected, so that it does not infect them twice. Such a virus is easy to detect because an infected program is longer than the original; a *compression virus* therefore compresses the host program so that the infected version has the same length.

Types.

- A **parasitic virus**, the traditional and most common form, attaches itself to executable files and replicates by finding other executables to infect whenever an infected program runs.
- A **memory-resident virus** lodges in main memory as part of a resident system program and from then on infects every program that executes.
- A **boot sector virus** infects the master boot record or boot record and spreads when the system is booted from the disk containing it.
- A **stealth virus** is designed to hide itself from antivirus software, for example by compression or by intercepting disk I/O so that a read of an infected area returns the original, uninfected program.
- A **polymorphic virus** mutates with every infection, making detection by a fixed "signature" impossible. It creates functionally equivalent copies with different bit patterns, by inserting superfluous instructions or reordering independent ones, or more effectively by *encryption*: a mutation engine encrypts the body of the virus with a fresh random key at each infection and stores the key and a small decryption routine with it; only that routine stays visible, and it is itself varied.
- A **metamorphic virus** also mutates with every infection, but rewrites itself completely each time, possibly changing its behaviour as well as its appearance.
- A **macro virus** is written in the macro language of an application such as Word or Excel and infects documents rather than executable code. It is particularly threatening because it is platform-independent, documents are shared far more often than programs, it spreads easily by e-mail, and file-system access controls are of little use since users are expected to modify documents.
- An **e-mail virus** (Melissa, 1999) arrives as an attachment, typically a macro, and when it is opened sends itself to everyone in the user's address book, so it spreads in hours instead of months.

46.6 Worms, bots and ransomware ▸

▸ **Likely this year:** "How does a worm propagate? Give an example" (3 marks).

A **worm** actively seeks out more machines to infect, and each infected machine serves as a launching pad for further attacks. To replicate it uses a network vehicle: the **e-mail** facility (mailing a copy of itself), a **remote execution** capability (running a copy on another system), or a **remote login** capability (logging in to a remote system and copying itself there). It has the same four phases as a virus; in its propagation phase it searches for other systems to infect (by examining host tables or address books), establishes a connection with a remote system, and copies itself there and causes the copy to run, often by exploiting a software vulnerability so that no user action is needed. The **Morris worm** (1988) exploited UNIX `finger`, `sendmail` and password guessing; **Code Red** (2001) and **SQL Slammer** (2003) infected hundreds of thousands of servers within hours by scanning random addresses.

Bots (zombies) wait for commands from a controller over a command-and-control channel, and a **botnet** of thousands of them is used for distributed denial-of-service attacks, spam and click fraud; the attacks come from innocent machines, which makes them hard to trace. **Ransomware** turns cryptography against the victim: the malware generates a symmetric key, encrypts the user's files (AES), and encrypts that key under the attacker's RSA *public* key, which is embedded in the malware. Only the attacker's private key can recover the file key, so neither analysing the malware nor the victim's machine helps; WannaCry (2017) combined this with a worm that spread through a Windows file-sharing vulnerability. The defences are offline backups and prompt patching.

46.7 Virus countermeasures ▸

▸ **Likely this year:** "Explain the generations of antivirus software; what is generic decryption?" (3–4 marks).

The ideal solution is **prevention**, by not allowing a virus into the system, but in general this is impossible, so antivirus software aims at **detection** (determine that an infection has occurred), **identification** (determine which virus it is) and **removal** (remove all traces and restore the program to its original state); if removal fails, the infected program is discarded and a clean backup reloaded. Stallings identifies four generations of antivirus software.

- **First generation: simple scanners** require a virus signature (the virus's fixed structure and bit pattern) to identify it, so they are limited to known viruses; others merely record program lengths and look for changes.
- **Second generation: heuristic scanners** use heuristic rules to search for probable infection, such as code fragments typical of viruses (e.g. the start of a decryption loop of a polymorphic virus), and **integrity checking**: a checksum of each program is stored and recomputed. A plain checksum or hash can be recomputed by a virus that changes the program, so the check uses a *keyed* hash (a MAC, §35) whose key is kept away from the programs.
- **Third generation: activity traps** are memory-resident programs that identify a virus by its *actions* rather than its structure, so no signature is needed.
- **Fourth generation: full-featured protection** packages combine scanning, activity traps and access control, which limits the ability of a virus to penetrate the system and to update files.

Generic decryption (GD) catches polymorphic viruses: the suspect file is run in a **CPU emulator**, a software computer under the control of an **emulation control module**, while a **virus signature scanner** watches the emulated memory. A polymorphic virus must decrypt its body to run, so after enough emulated instructions the plaintext virus appears in memory and is recognised, without risk to the real machine. **Behaviour-blocking software** integrates with the operating system and monitors programs in real time for malicious actions (opening, deleting or modifying files, formatting disks, changing executables or system settings, initiating network connections), blocking them before they take effect. IBM's **digital immune system** adds automation: a suspicious program on one client is sent to a central virus-analysis machine, which runs it safely, derives a prescription (signature and removal), and distributes it to all clients.

46.8 Firewalls: design goals and control techniques ★▸

▸ **Likely this year:** "What are the design goals of a firewall? List the four control techniques" (4 marks).

Problem. An organisation with hundreds of hosts on the Internet cannot secure every one of them individually: each runs different software with different flaws. A **firewall** is inserted between the premises network and the Internet to establish a controlled link and an outer security wall or perimeter, so that security is enforced at a single choke point. Bellovin and Cheswick give three design goals.

1. **All traffic** from inside to outside, and vice versa, must pass through the firewall; this is achieved by physically blocking all access to the local network except via the firewall.
2. **Only authorised traffic**, as defined by the local security policy, is allowed to pass.
3. The firewall itself is **immune to penetration**, which implies a trusted system with a secure operating system.

Control techniques. Service control decides which types of Internet services can be accessed, inbound or outbound (by IP address and port number, by proxy software, or by hosting the service on the firewall). **Direction control** decides in which direction particular service requests may be initiated and allowed to flow. **User control** controls access to a service according to which user is attempting it, usually for local users, and for remote users with secure authentication such as IPSec. **Behaviour control** controls how particular services are used, e.g. filtering spam out of e-mail or giving external users access to only part of the web server.

Capabilities. The single choke point keeps unauthorised users out, prohibits vulnerable services from entering or leaving, and protects against IP spoofing and routing attacks; it gives one location for monitoring, audits and alarms; it is a convenient platform for functions such as network address translation and usage logging; and it can serve as the endpoint of IPSec tunnels for a VPN.

46.9 Types of firewall ★▸

▸ **Likely this year:** "Explain packet filter, stateful inspection, application-level and circuit-level gateways; design a packet-filter rule set" (5–8 marks).

Packet-filtering router. A packet filter applies a set of rules to each incoming and outgoing IP packet and then forwards or discards it. The rules match fields of the packet: source and destination IP address, source and destination port (which identifies the application, e.g. 25 for SMTP, 80 for HTTP), the IP protocol field, TCP flags, and the router interface the packet arrived on or leaves by. The rules are checked in order and the first match decides; if no rule matches, a **default policy** applies. *Default discard* (that which is not expressly permitted is prohibited) is the conservative choice: everything is blocked at first and services are added case by case. *Default forward* (that which is not expressly prohibited is permitted) is easier for users but means the administrator must

react to each new threat.

#	Dir	Source address	Src port	Dest. address	Dst port	Flag	Action	Purpose
1	in	192.168.1.0/24	*	*	*		deny	spoofed inside address from outside
2	in	*	> 1023	192.168.1.10	80, 443		allow	requests to our web server
3	out	192.168.1.10	80, 443	*	> 1023	ACK	allow	web server's replies
4	in	*	*	192.168.1.20	25		allow	mail to our SMTP server
5	out	192.168.1.20	25	*	*	ACK	allow	SMTP server's replies
6	out	192.168.1.0/24	> 1023	*	80, 443		allow	inside users browse the web
7	in	*	80, 443	192.168.1.0/24	> 1023	ACK	allow	replies to their requests
8	*	*	*	*	*		deny	default: discard everything else

The **ACK flag** in rules 3, 5 and 7 is essential. The first packet of a TCP connection (SYN) has no ACK, and every later packet has it, so a rule that requires ACK only admits packets of connections that were opened from the other side. Without it, rule 7 would let an outsider open a connection to *any* high port inside, simply by sending from source port 80. Rule 1 is anti-spoofing: a packet arriving from the Internet cannot legitimately carry an inside source address.

Advantages and disadvantages. Packet filters are simple, transparent to users and very fast. But they do not examine upper-layer data, so they cannot block application-specific attacks or commands; their logging is limited; most do not support user authentication; they are open to attacks on TCP/IP itself; and rule sets are easy to get wrong. Three attacks and their countermeasures are standard. In **IP address spoofing**, an outsider sends packets with an internal source address; the filter discards packets with an inside source address that arrive on the external interface (rule 1). In a **source routing attack** the sender specifies the route, hoping to bypass checks; the filter discards all packets that use the source-route option. In a **tiny fragment attack** the intruder fragments a packet so finely that the TCP header falls into a second fragment, which escapes rules that test TCP fields; the filter requires the first fragment to contain a minimum amount of the transport header, and if it rejects the first fragment it remembers the packet and discards all later fragments.

Stateful inspection firewall. A client normally uses a port number above 1023 for its side of a TCP connection, so a simple packet filter must admit inbound traffic to all high-numbered ports (rule 7), a large hole. A stateful inspection firewall tightens this by keeping a **directory (state table) of outbound TCP connections**, one entry per established connection (source and destination addresses and ports, connection state), and admits inbound packets to high ports only if they fit an entry in the directory. Some also track TCP sequence numbers to stop session hijacking.

Application-level gateway (proxy server). The gateway relays application-level traffic. The user contacts it with an application such as Telnet or FTP, the gateway asks for the remote host and for authentication, and then opens its own connection to the remote application and relays the application data between the two connections. If the gateway has no proxy code for an application, that service is simply not available, and each proxy can be restricted to the commands the administrator considers acceptable. *Advantages*: it is more secure than a packet filter, since it only has to scrutinise a few allowed applications instead of all combinations at TCP/IP level, and it can log and audit all traffic at application level. *Disadvantage*: the extra processing on every connection, since the gateway splices two connections and examines all traffic in both directions.

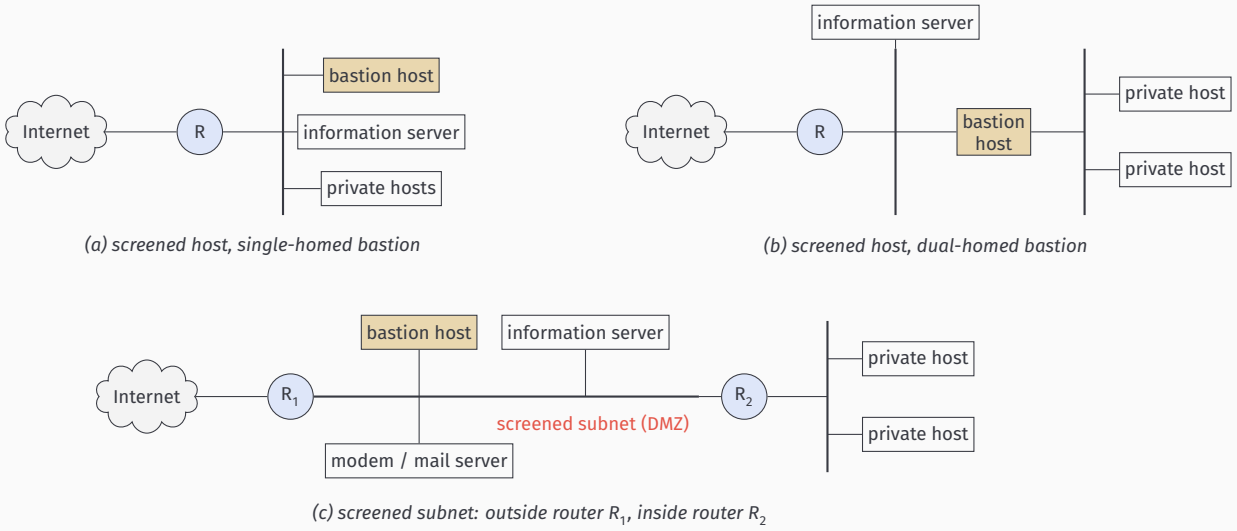
Circuit-level gateway. The gateway does not permit an end-to-end TCP connection either; it sets up two TCP connections, one with the inside host and one with the outside host, but once both exist it relays TCP segments *without examining their contents*. Its security function is deciding which connections are allowed. A typical use is with trusted inside users: application-level proxies for inbound connections and circuit-level relaying for outbound ones, which avoids the processing overhead of examining outgoing data. SOCKS is the standard circuit-level gateway.

Type	Layer examined	Speed	Main strength / weakness
Packet filter	IP and TCP/UDP headers	fastest	simple and transparent / blind to application data, no state
Stateful inspection	headers + connection state	fast	closes the high-port hole / still blind to application data
Application-level gateway	application data	slowest	per-command control, logging, authentication / overhead, one proxy per application
Circuit-level gateway	TCP connection set-up	fast	controls which connections exist / does not inspect contents

46.10 Bastion host and firewall configurations ★▸

▸ **Likely this year:** "Explain screened host and screened subnet firewall configurations with diagrams; what is a bastion host / DMZ?" (5 marks).

A **bastion host** is a system identified by the firewall administrator as a critical strong point in the network's security; it usually hosts the application-level or circuit-level gateways. It runs a secure (trusted) version of its operating system; only the essential services (proxies for Telnet, DNS, FTP, SMTP, user authentication) are installed; it may demand extra authentication before allowing access to a proxy; each proxy supports only a subset of its application's commands, allows access only to specific hosts, logs all traffic, is a small program that can be checked for flaws, is independent of the other proxies, does no disk access beyond reading its configuration, and runs as an unprivileged user in a private directory.



(a) screened host, single-homed bastion

(b) screened host, dual-homed bastion

(c) screened subnet: outside router R_1, inside router R_2

Screened host firewall, single-homed bastion (a). The firewall consists of a packet-filtering router and a bastion host. The router is configured so that from the Internet only packets destined for the bastion host are allowed in, and from the internal network only packets from the bastion host are allowed out; the bastion host performs authentication and proxy functions. This is more secure than either component alone, because it combines packet-level and application-level filtering and an intruder must generally penetrate two separate systems; the router can also let some traffic, such as public web information, go directly to an information server. *Weakness*: if the router is completely compromised, traffic can flow directly between the Internet and the private hosts.

Screened host firewall, dual-homed bastion (b). The bastion host has two network interfaces, one on the router's side and one on the internal network, so traffic between the Internet and the private hosts physically has to pass through the bastion host. This removes the weakness of (a): even a compromised router cannot bypass the bastion. An information server can still be placed on the outer segment for direct access.

Screened subnet firewall (c), the most secure configuration. Two packet-filtering routers are used, one between the bastion host and the Internet and one between the bastion host and the internal network, creating an isolated subnetwork, the **screened subnet** or **DMZ** (demilitarised zone), which holds the bastion host and public servers. Both the Internet and the internal network can reach the DMZ, but traffic across it is blocked. *Advantages*: there are three levels of defence (outer router, bastion host, inner router); the outer router advertises only the DMZ to the Internet, so the internal network is invisible from outside; and the inner router advertises only the DMZ to the internal network, so inside systems cannot build direct routes to the Internet.

Limitations of firewalls. A firewall cannot protect against attacks that bypass it (an inside system with its own dial-out or mobile connection, a modem pool for travelling staff); it does not protect against internal threats (a disgruntled employee, or one who unwittingly helps an outside attacker); it cannot practically scan every file transferred for viruses, given the variety of formats; a laptop or USB drive infected outside can be carried in and attached inside; and it cannot inspect traffic that is encrypted end to end.

46.11 Trusted systems: Bell–LaPadula and the reference monitor ★▸

▸ **Likely this year:** "Explain the Bell–LaPadula model (no read up, no write down) and the reference monitor; how does a trusted system defeat a Trojan horse?" (5–6 marks).

Problem. Ordinary access control (an access matrix of subjects, objects and rights, stored per object as an access control list or per subject as capability tickets) lets the owner of a file pass its contents to anyone, and any program a user runs acts with all of the user's rights. Military and many commercial settings need a stronger guarantee: information classified at one level must never flow to a lower level, whatever users or programs do. Data and users are therefore given **security levels**, for example unclassified (U) < confidential (C) < secret (S) < top secret (TS); a subject's level is its **clearance** and an object's level is its **classification**. A system that enforces such levels is a **multilevel secure** system.

Bell–LaPadula model (1973). A multilevel secure system must enforce two rules:

- **No read up** (simple security property): a subject can only read an object of less or equal security level.
- **No write down** ($*$-property): a subject can only write into an object of greater or equal security level.

Why both are needed. No read up alone stops a secret-cleared user from reading top-secret data, but it does not stop information from leaking downwards: a secret-level process could read a secret file (allowed) and write its contents into an unclassified file, which any unclassified user may then read. The $*$-property closes that path: whatever a subject at level L has read is at level $\leq L$, and it can only write at level $\geq L$, so information can only flow upwards. Writing up is harmless for confidentiality (it is a blind append). Together the two rules guarantee that information never moves from a higher to a lower level.

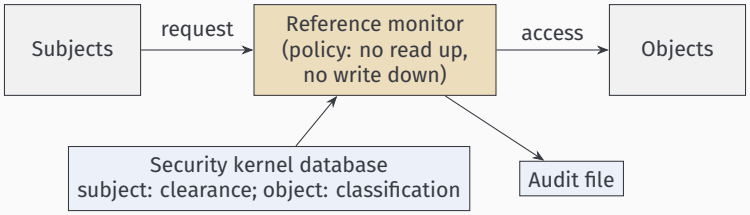
WORKED EXAMPLE : applying Bell–LaPadula

Q. Levels are U < C < S < TS. Subjects: General (TS), Officer (S), Clerk (C). Objects: War plan (TS), Report (S), Memo (C), Notice (U). Give the read (r) and write (w) operations allowed to each subject.
A. Read is allowed if subject level ≥ object level, write if subject level ≤ object level; equal levels give both.

Subject \ Object	War plan (TS)	Report (S)	Memo (C)	Notice (U)
General (TS)	r, w	r	r	r
Officer (S)	w	r, w	r	r
Clerk (C)	w	w	r, w	r

Check: the Officer may not write the Memo (write down), so nothing he has read from the Report can reach the Clerk; he may append to the War plan (write up) but not read it.

Reference monitor. The rules are enforced by a **reference monitor**, a controlling element in the hardware and operating system of a computer that regulates the access of subjects to objects on the basis of their security parameters. It consults the **security kernel database**, which lists the access privileges (security clearance) of each subject and the protection attributes (classification level) of each object, and it records important security events, such as detected violations and authorised changes to the kernel database, in an **audit file**.



The reference monitor must have three properties.

- **Complete mediation**: the security rules are enforced on *every* access, not only, for example, when a file is opened.
- **Isolation**: the reference monitor and its database are protected from unauthorised modification.
- **Verifiability**: the reference monitor's correctness must be provable, i.e. it must be possible to demonstrate mathematically that it enforces the security rules and provides complete mediation and isolation.

A system that can provide such verification is called a **trusted system**. These requirements are demanding: complete mediation in software would be too slow, so it needs hardware support; isolation requires that an attacker cannot change the monitor's logic or data; and formal verification is only feasible for a small, simple kernel. Evaluation criteria such as the US Orange Book (TCSEC, classes D to A1) and the international Common Criteria grade systems by how far these properties are achieved.

Trojan horse defence. Stallings' example shows why access control lists are not enough. Bob keeps a sensitive string in a file that only he may read and write. Alice, a legitimate but hostile user, installs a Trojan horse program and creates a private "back-pocket" file, giving herself read/write access and Bob write access to it. She persuades Bob to run her program, advertised as a useful utility. Running with Bob's rights, the program reads Bob's file and copies the string into the back-pocket file; both operations satisfy the ACLs, and Alice later reads her own file. In a trusted system, subjects get security levels at logon, say *sensitive* for Bob's processes and files and *public* for Alice's. When Bob runs the Trojan horse it acquires Bob's level, so it may read the sensitive string (no read up is satisfied), but its attempt to write into the public back-pocket file violates the $*$-property and is **refused by the reference monitor**, even though the ACL permits it. The security policy takes precedence over the discretionary ACL mechanism, and the leak is stopped.

Corrections to the notes / slides

- No class slides cover this chapter; the notes follow Stallings (which gives most of the detail on PGP, S/MIME, IPSec, SSL, SET, intruders, malware, firewalls and trusted systems) and Forouzan (PGP, S/MIME, IPSec, SSL/TLS). Answer in the books' terms, but know that the details have aged.
- Algorithms: the lists for PGP (CAST-128, IDEA, 3DES, SHA-1), S/MIME (3DES, RC2/40, SHA-1) and IPSec (HMAC-MD5-96, 3DES) are the books' historical defaults; current versions use AES and SHA-2. SSL 2.0/3.0 and TLS 1.0/1.1 are deprecated; TLS 1.2 and 1.3 are current. IKEv1 (main/aggressive mode) has been replaced by IKEv2. SET was never widely deployed and was dropped from later Stallings editions.
- Stallings words the $*$-property as "a subject can only write into an object of greater or equal security level". Some texts state a strong $*$-property (write only at the *same* level), which also protects integrity; the exam usually wants the standard version.

47 Assignment Topics: Primality, Factoring, RSA Attacks, OAEP, Hash Families, Real-World Protocols

The lecture slides on primes, RSA and hashing mark several topics as "Tutorial" and leave them to the assignments: the AKS primality test, the factorization algorithms (Pollard $p-1$, Pollard rho, quadratic sieve, number field sieve), the attacks on RSA, OAEP padding, and the hash functions other than SHA-512 and Whirlpool. Two further assignment topics apply the course to systems the student uses daily: WhatsApp's end-to-end encryption and the security of internet banking and UPI. The sections below extend §27 (Miller–Rabin, trial division, Fermat factoring), §33 (RSA and its attack list) and §35 (Merkle–Damgård, SHA-512, Whirlpool) without repeating them. The assignment's central question for RSA, "verify whether any attack can be launched successfully, and if not, justify why", is answered attack by attack in §50.

Assignment topic	What is usually asked	Section
AKS primality test	theorem, pseudo code, complexity, why not used	48
Factorization methods	Pollard $p-1$, rho, QS, NFS; complexities; numericals	49
Attacks on RSA	each attack with a demonstration; can it succeed?	50
OAEP	structure, diagram, why it fixes textbook RSA	51
Other hash algorithms	MD4/MD5, SHA-0/1, SHA-2 family, SHA-3, others	52
Attacks on digital signatures	forgery types, attacks	37 (separate chapter)
WhatsApp E2EE	X3DH, Double Ratchet, groups, safety numbers	53
Internet banking and UPI	controls, UPI PIN path, frauds, "which app is safer"	54

48 The AKS Primality Test

48.1 The problem and the theorem behind AKS ▸

▸ **Likely this year:** "State the identity on which AKS is based and prove it for prime n" (3–4 marks).

Trial division decides primality correctly but needs up to $\sqrt{n} = 2^{b/2}$ divisions for a b-bit n, which is exponential in the input size. Miller–Rabin (§27) is fast but probabilistic: its "prime" answer can be wrong with probability up to 4^{-t}. Whether primality can be decided *deterministically in polynomial time* (in the number of bits $b = \log_2 n$) was an open question until 2002, when Manindra Agrawal and his students Neeraj Kayal and Nitin Saxena (IIT Kanpur) answered yes. Their algorithm, **AKS**, rests on a polynomial version of Fermat's little theorem.

★ MUST KNOW : the AKS identity

Let $n \geq 2$ and let a be an integer with $\gcd(a, n) = 1$. Then

$$n \text{ is prime} \iff (X + a)^n \equiv X^n + a \pmod{n}$$

as polynomials in X, i.e. every coefficient of $(X + a)^n - (X^n + a)$ is divisible by n.

Proof. By the binomial theorem,

$$(X + a)^n - (X^n + a) = \sum_{i=1}^{n-1} \binom{n}{i} a^{n-i} X^i \; + \; (a^n - a).$$

If n is prime, every middle coefficient vanishes modulo n: for $0 < i < n$, $\binom{n}{i} = \dfrac{n!}{i!\,(n-i)!}$, the numerator contains the prime factor n, and the denominator is a product of numbers smaller than n, none of which contains the prime n; so $n \mid \binom{n}{i}$. The constant term vanishes because $a^n \equiv a \pmod{n}$ by Fermat's little theorem. Hence the identity holds.

If n is composite, let q be a prime factor of n and q^k the highest power of q dividing n. Then

$$\binom{n}{q} = \frac{n(n-1)\cdots(n-q+1)}{q!}.$$

In the numerator only the factor n is divisible by q (the other $q-1$ factors are $n-1, \dots, n-q+1$, which are not multiples of q), so the numerator contains exactly q^k; the denominator $q!$ contains exactly one q. Therefore q^{k-1} is the exact power of q in $\binom{n}{q}$, so $q^k \nmid \binom{n}{q}$ and $n \nmid \binom{n}{q}$. Since $\gcd(a,n) = 1$, a^{n-q} is coprime to q, so the coefficient $\binom{n}{q}a^{n-q}$ of X^q is not $\equiv 0 \pmod{q^k}$, and the identity fails. This proves both directions.

Small check. For $n = 5$ the row $\binom{5}{i}$ is $1, 5, 10, 10, 5, 1$, and all inner entries are multiples of 5, so $(X+1)^5 \equiv X^5 + 1 \pmod 5$. For $n = 6$ the row is $1, 6, 15, 20, 15, 6, 1$, which modulo 6 is $1, 0, 3, 2, 3, 0, 1$, so $(X+1)^6 \equiv X^6 + 3X^4 + 2X^3 + 3X^2 + 1 \not\equiv X^6 + 1 \pmod 6$; the surviving coefficients sit at X^2, X^3, X^4, as the proof predicts for $q = 2, 3$ (and their mirror images).

Why the identity alone is not an efficient test. The polynomial $(X+a)^n$ has $n+1$ coefficients, so even writing it down takes time proportional to n, which is exponential in b. AKS therefore checks the identity in a much smaller ring: it reduces both sides modulo $X^r - 1$ for a small r (polynomial in b), keeping only r coefficients. A prime n still passes, since a true congruence stays true after reduction. A composite n might now pass for a particular a, and the work of the AKS paper is to show that, with the right r, a composite cannot pass for *all* a up to a small bound.

48.2 The AKS algorithm (pseudo code) ★▸

▸ **Likely this year:** "Write the pseudo code of AKS and explain each step; state its complexity" (5–6 marks).

AKS(n), $n > 1$; all logarithms are base 2

Step 1. If $n = a^b$ for integers $a > 1$ and $b > 1$, output **COMPOSITE**.
Step 2. Find the smallest r such that $\mathrm{ord}_r(n) > (\log_2 n)^2$.
Step 3. If $1 < \gcd(a,n) < n$ for some $a \le r$, output **COMPOSITE**.
Step 4. If $n \le r$, output **PRIME**.
Step 5. For $a = 1$ to $\lfloor \sqrt{\phi(r)}\,\log_2 n \rfloor$: if $(X+a)^n \not\equiv X^n + a \pmod{X^r - 1,\ n}$, output **COMPOSITE**.
Step 6. Output **PRIME**.

Here $\mathrm{ord}_r(n)$ is the multiplicative order of n modulo r (the least k with $n^k \equiv 1 \pmod r$, defined when $\gcd(n,r) = 1$; see §26), and "mod $(X^r - 1, n)$" means that powers of X are reduced with $X^r = 1$ (so X^i becomes $X^{i \bmod r}$) and coefficients are reduced modulo n.

Step 1 (perfect powers). The correctness proof of step 5 needs n to be divisible by a prime p without being a power of p; powers such as $343 = 7^3$ are therefore removed first. The test is cheap: for each $b \le \log_2 n$, compute the integer b-th root of n by binary search and check whether its b-th power equals n.

Step 2 (choosing r). Try $r = 2, 3, 4, \dots$ and compute $\mathrm{ord}_r(n)$ (skip r with $\gcd(r,n) > 1$). The order condition is what makes the final counting argument work: it guarantees that the powers $n, n^2, n^3, \dots$ produce more than $(\log_2 n)^2$ distinct residues modulo r. A lemma in the paper shows that such an r exists with $r \le \max\{3, \lceil \log_2^5 n \rceil\}$, so the search is polynomial and the polynomials in step 5 have polynomially many coefficients.

Step 3 (small factors). Compute $\gcd(a,n)$ for $a = 2, \dots, r$. A value strictly between 1 and n is a proper factor, so n is composite. After this step every prime factor of n exceeds r.

Step 4 (small n). If $n \le r$, step 3 has examined every $a < n$ and found no common factor, so n is prime. This step decides only small inputs; for larger n, r is much smaller than n.

Step 5 (the polynomial test). Each congruence is checked in the ring of polynomials of degree less than r with coefficients modulo n, by square-and-multiply on polynomials: $(X+a)^n$ needs about $2\log_2 n$ polynomial multiplications, each followed by reduction modulo $X^r - 1$ and modulo n. A prime always passes (theorem above). The heart of the AKS paper is the converse: if all $\lfloor \sqrt{\phi(r)}\log_2 n \rfloor$ checks pass, then for a prime factor p of n both n and p are "introspective" (they satisfy $f(X)^m \equiv f(X^m)$ for every product f of the polynomials $X + a$ modulo p), and counting the resulting polynomials in two ways gives a lower bound larger than an upper bound unless n is a power of p. Step 1 excluded proper powers, so $n = p$ is prime.

WORKED EXAMPLE : AKS on $n = 31$

Q. Run the AKS algorithm on $n = 31$. Find r, the loop bound of step 5, and show the step-5 check for $a = 1$.
Step 1. $b \le \log_2 31 = 4.954$, so $b \in \{2, 3, 4\}$: $5^2 = 25 < 31 < 36 = 6^2$; $3^3 = 27 < 31 < 64 = 4^3$; $2^4 = 16 < 31 < 81 = 3^4$. Not a perfect power.
Step 2. $(\log_2 31)^2 = 4.954^2 = 24.54$, so we need $\mathrm{ord}_r(31) \ge 25$. Since $\mathrm{ord}_r(31) \le \phi(r) \le r - 1$, only $r \ge 26$ can work: $\phi(26) = 12$, $\phi(27) = 18$, $\phi(28) = 12$ are too small. For $r = 29$: $31 \equiv 2 \pmod{29}$, and $2^2 = 4$, $2^4 = 16$, $2^7 = 128 \equiv 12$,

$2^{14} \equiv 12^2 = 144 \equiv 28 \equiv -1$, so no proper divisor of 28 is the order and $\mathrm{ord}_{29}(31) = 28 > 24.54$. Hence **r = 29**.
Step 3. $\gcd(a, 31) = 1$ for $a = 2, \dots, 29$. Pass. **Step 4.** $31 > 29$, so continue.
Step 5. $\phi(29) = 28$; bound $= \lfloor\sqrt{28} \times 4.954\rfloor = \lfloor 5.2915 \times 4.9542\rfloor = \lfloor 26.22\rfloor = \mathbf{26}$. Modulo $X^{29} - 1$, $X^{31} = X^{29}X^2 \equiv X^2$, so each check reads $(X + a)^{31} \equiv X^2 + a \pmod{X^{29} - 1, 31}$. For $a = 1$, reduce $(X + 1)^{31} = \sum_i \binom{31}{i} X^i$ with $X^{29} \to 1$, $X^{30} \to X$, $X^{31} \to X^2$:

Coefficient of	Contributions	Sum	mod 31
X^0	$\binom{31}{0} + \binom{31}{29} = 1 + 465$	466	1
X^1	$\binom{31}{1} + \binom{31}{30} = 31 + 31$	62	0
X^2	$\binom{31}{2} + \binom{31}{31} = 465 + 1$	466	1
$X^3, \dots, X^{28}$	$\binom{31}{i}$, $3 \le i \le 28$	multiples of 31	0

So $(X + 1)^{31} \equiv X^2 + 1$ ✓. A script confirms that all 26 checks $a = 1, \dots, 26$ pass. **Step 6: 31 is PRIME.**
Contrast. For $n = 171371 = 409 \times 419$ the script finds $r = 313$; step 3 passes because both prime factors exceed 313, $n > r$, and step 5 fails already at $a = 1$, so AKS outputs COMPOSITE without ever finding a factor.

48.3 Complexity, and why Miller–Rabin is used instead ▸

▸ **Likely this year:** "State the complexity of AKS; why is it not used to generate RSA primes?" (2–3 marks).

Step 5 dominates. It has about $\sqrt{r}\log_2 n$ iterations; each computes $(X + a)^n$ with $O(\log n)$ multiplications of polynomials of degree below r whose coefficients have $\log_2 n$ bits, and with fast multiplication one such product costs $\tilde{O}(r \log n)$ bit operations. In total this is $\tilde{O}(r^{3/2}\log^3 n)$, and with $r = O(\log^5 n)$ it is $\tilde{O}(\log^{10.5} n)$. ($\tilde{O}$ hides factors that are powers of $\log\log n$.) The first version of the paper (2002) stated the cruder bound $\tilde{O}(\log^{12} n)$, which is the figure usually quoted. **Lenstra and Pomerance** (2005) modified the polynomial used in step 5 and brought the running time down to $\tilde{O}(\log^6 n)$. Every version is polynomial in the number of bits, which is the theoretical breakthrough: PRIMES is in P.

Why it is not used in practice. Even $\log^6 n$ is large: for a 1024-bit n it is $1024^6 = 2^{60}$ bit operations before the hidden constants and $\log\log$ factors. One round of Miller–Rabin is one modular exponentiation, about $1024^3 = 2^{30}$ bit operations with schoolbook arithmetic, and 40 random bases leave an error probability below $4^{-40} \approx 10^{-24}$, far smaller than the chance of a hardware fault during the computation. Key generation tests hundreds of candidates per prime (§27), so only the fast probabilistic test is affordable. When a proof of primality is wanted, elliptic-curve primality proving (ECPP) is also faster than AKS in practice and outputs a certificate that can be checked quickly.

Test	Type	Cost for a b-bit n	Used for
Trial division	deterministic	$O(\sqrt{n}) = O(2^{b/2})$	small n; first filter
Fermat test	probabilistic	$O(t\,b^3)$	fooled by Carmichael numbers
Miller–Rabin	probabilistic, error $\le 4^{-t}$	$O(t\,b^3)$	RSA key generation
AKS (2002 analysis)	deterministic	$\tilde{O}(b^{12})$	theory: PRIMES is in P
AKS (Lenstra–Pomerance)	deterministic	$\tilde{O}(b^6)$	theory

49 Factorization Algorithms

49.1 Measuring the cost: input size and L-notation ▸

▸ **Likely this year:** "Arrange trial division, Pollard rho, QS and NFS by complexity" (2–3 marks).

RSA is broken as soon as $n = pq$ is factored, since then $\phi(n) = (p - 1)(q - 1)$ and $d = e^{-1} \bmod \phi(n)$ follow at once (§33). The cost of a factoring algorithm is measured against the *size* of the input, $b = \log_2 n$ bits (or $\ln n$), not against n itself. An algorithm costing $O(\sqrt{n}) = O(2^{b/2})$ is *exponential*; one costing $O(b^k)$ would be *polynomial*. The best known methods lie in between, and the standard way to express this is the **L-notation**

$$L_n[\alpha, c] = \exp\Big((c + o(1))\,(\ln n)^{\alpha}(\ln\ln n)^{1-\alpha}\Big), \qquad 0 \le \alpha \le 1.$$

For $\alpha = 0$ this is $(\ln n)^c$, polynomial; for $\alpha = 1$ it is n^c, exponential. Values $0 < \alpha < 1$ are called **sub-exponential**, and the smaller α, the faster the algorithm for large n. The quadratic sieve runs in $L_n[1/2, 1]$ and the number field sieve in $L_n[1/3, 1.923]$. Trial division and Fermat's method were covered in §27; this section adds their cost analysis and the four "tutorial" methods.

49.2 Trial division and Fermat's method: when each is fast

Trial division divides by $2, 3, 5, \dots$ up to $\sqrt{n}$ and costs up to $\sqrt{n}$ divisions: for a 2048-bit RSA modulus that is 2^{1024} divisions. It finds a small factor f after about f divisions, so it is the right first step for random numbers but useless for RSA moduli whose factors both have 1024 bits.

Fermat's method searches $x = \lceil\sqrt{n}\rceil, \lceil\sqrt{n}\rceil + 1, \ldots$ until $x^2 - n = y^2$ is a square, and then $n = (x - y)(x + y)$. With $n = pq$ and $p > q$ it stops at $x = (p + q)/2$, so the number of steps is about

$$\frac{p+q}{2} - \sqrt{pq} = \frac{(\sqrt{p} - \sqrt{q})^2}{2} = \frac{(p-q)^2}{2(\sqrt{p} + \sqrt{q})^2} \approx \frac{(p-q)^2}{8\sqrt{n}}.$$

If $|p - q|$ is below $n^{1/4}$, a single step suffices; if p and q are random 1024-bit primes, $|p - q|$ is typically about 2^{1023} and the count is about $2^{2046}/2^{1027} = 2^{1019}$ steps. This is why the RSA recommendation says "p and q must not be close" and why standards such as FIPS 186 require $|p - q| > 2^{(b/2)-100}$.

WORKED EXAMPLE : Fermat factorization (new)

Q. Factor $n = 6077$ by Fermat's method.
$\sqrt{6077} = 77.96$, so start at $x = 78$.

x	x^2	$w = x^2 - 6077$	square?
78	6084	7	no
79	6241	164	no
80	6400	323	no (17×19)
81	6561	484	**yes**, 22^2

$n = (81-22)(81+22) = \mathbf{59} \times \mathbf{103}$. Check: $59 \times 103 = 6077$ ✓; the formula predicts $(103-59)^2/(8\sqrt{6077}) = 1936/623.7 \approx 3.1$ steps beyond the start, and we needed 3.

49.3 Pollard's $p - 1$ method ★▸

▸ **Likely this year:** "Factor n using Pollard $p - 1$ with bound B" (4–5 marks), and "why must $p - 1$ have a large prime factor?"

Idea. Fermat's little theorem says $a^{p-1} \equiv 1 \pmod{p}$ for a prime $p \nmid a$, and therefore $a^M \equiv 1 \pmod{p}$ for *every* multiple M of $p - 1$. Eve does not know p, but if $p - 1$ is B**-smooth**, meaning all its prime-power factors are at most B, then $p - 1$ divides $M = B!$ (or $\operatorname{lcm}(1, \ldots, B)$). Then $p \mid a^{B!} - 1$, and since $p \mid n$ as well,

$$p \mid \gcd(a^{B!} - 1,\ n).$$

The other prime q divides this gcd only if $q - 1$ is also B-smooth (or by luck), so usually the gcd is exactly p. All the arithmetic is done modulo n, so the numbers stay small.

Pollard $p - 1$ (Forouzan's form)

Input n and a bound B. Set $a \leftarrow 2$. For $e = 2, 3, \ldots, B$: $a \leftarrow a^e \bmod n$ (so after step e, $a = 2^{e!} \bmod n$). Then $d \leftarrow \gcd(a - 1, n)$. If $1 < d < n$, output the factor d; otherwise the method **fails** for this B.

When it fails. If $d = 1$, no prime factor p has $p - 1 \mid B!$, so B was too small; increase B. If $d = n$, every prime factor was caught at the same time; decrease B (or check the gcd after every step, which finds the first e at which one factor appears). The cost is about $B \log B$ modular multiplications, so the method is practical only when some $p - 1$ is smooth for a modest B. For RSA primes chosen at random, $p - 1$ almost always has a prime factor of hundreds of bits, and B would have to be that large.

WORKED EXAMPLE : Pollard $p - 1$ on $n = 1403$ with $B = 5$

Q. Use Pollard's $p-1$ method with $B = 5$ to find a factor of $n = 1403$. Explain why it succeeds, and what happens with $B = 3$ and $B = 11$.
$a_1 = 2$ and $a_e = a_{e-1}^e \bmod 1403$, so $a_e = 2^{e!} \bmod 1403$:

e	$a_e = a_{e-1}^e \bmod 1403$	a_e	$\gcd(a_e - 1, 1403)$
2	2^2	4	1
3	$4^3 = 64$	64	1
4	$64^4 \bmod 1403$	142	1
5	$142^5 \bmod 1403$ ($= 2^{120}$)	794	**61**

$d = \gcd(793, 1403) = 61$, so $\mathbf{1403} = \mathbf{61} \times \mathbf{23}$. *Why*: $61 - 1 = 60 = 2^2 \cdot 3 \cdot 5$ divides $5! = 120$, so $2^{120} \equiv 1 \pmod{61}$; but $23 - 1 = 22 = 2 \cdot 11$ does not divide 120, so $2^{120} \not\equiv 1 \pmod{23}$ and 23 stays out of the gcd.
Failures: with $B = 3$, $3! = 6$ is a multiple of neither 60 nor 22, and the gcd is 1. With $B = 11$, both 60 and 22 divide $11!$, so $2^{11!} \equiv 1$ modulo both primes and the gcd is $n = 1403$ itself.
Second example: $n = 299 = 13 \times 23$ gives $\gcd(2^{24} - 1, 299) = 13$ at $B = 4$ (since $12 \mid 24$), but $B = 11$ again yields gcd = 299. Forouzan's example $n = 57247159$ with $B = 8$ gives 421, because $420 = 2^2 \cdot 3 \cdot 5 \cdot 7$; the cofactor is 135979, with $135978 = 2 \cdot 3 \cdot 131 \cdot 173$ not 8-smooth.

Implication for RSA. The recommendation "both $p-1$ and $q-1$ should have at least one large prime factor" (§33) is exactly the defence against this method. A prime with this property (and similar conditions on $p+1$, against Williams' $p+1$ method) is called a *strong prime*. For random primes of 1024 bits the probability that $p-1$ is smooth enough to be caught is negligible, so modern standards accept random primes; the more general *elliptic curve method* (ECM) replaces the group $\mathbb{Z}_p^*$ of fixed order $p-1$ by random elliptic-curve groups of varying order, removing the dependence on the shape of $p-1$, but it still only finds factors of up to about 60–80 digits.

49.4 Pollard's rho method ★▸

▸ **Likely this year:** "Factor 8051 (or a similar n) using Pollard rho with $f(x)=x^2+1$, $x_0=2$; why $O(n^{1/4})$?" (4–5 marks).

Idea. Let p be the unknown smallest prime factor of n. Iterate a simple polynomial map $x_{i+1}=f(x_i)=x_i^2+1 \bmod n$ starting from $x_0=2$. Because f is a polynomial, reducing modulo p commutes with it: the residues $x_i \bmod p$ follow the same rule $x_{i+1}\equiv x_i^2+1 \pmod p$ inside the much smaller set $\mathbb{Z}_p$. A sequence in a finite set in which each term determines the next must eventually repeat and then cycle forever; drawn out, a tail leading into a loop looks like the Greek letter ρ. A repetition $x_i\equiv x_j \pmod p$ means $p \mid x_i - x_j$, so

$$\gcd(|x_i - x_j|,\ n) \text{ is a multiple of } p,$$

and it is usually p itself (it equals n only if the sequence repeats modulo q at exactly the same step).

Why about $\sqrt{p}$ steps (birthday argument). The residues $x_i \bmod p$ behave like random elements of $\mathbb{Z}_p$. By birthday Problem 3 (§35), among k random values from p possibilities two coincide with probability $1-e^{-k^2/2p}$, so a repetition is expected after about $\sqrt{\pi p/2}\approx 1.25\sqrt{p}$ terms. Modulo n the sequence would need about $\sqrt{n}$ terms to repeat, so the gcd detects the much earlier repetition modulo p. Since $p\le\sqrt{n}$, the method costs $O(\sqrt{p})\le O(n^{1/4})$ steps, the square root of trial division's $O(\sqrt{n})$.

Floyd's cycle detection. Storing all x_i and comparing every pair would cost memory and k^2 gcds. Floyd's method moves a "tortoise" $x\leftarrow f(x)$ and a "hare" $y\leftarrow f(f(y))$, so that after i steps $x=x_i$ and $y=x_{2i}$, and computes one gcd per step. If the sequence modulo p has a tail of length μ and a cycle of length λ, then $x_i\equiv x_{2i} \pmod p$ as soon as $i\ge\mu$ and $\lambda\mid i$, which happens for some $i\le\mu+\lambda$. The method therefore needs constant memory and $O(\sqrt{p})$ steps of three evaluations of f and one gcd each.

Pollard rho with Floyd cycle detection

$x\leftarrow 2$, $y\leftarrow 2$, $d\leftarrow 1$. While $d=1$: $x\leftarrow f(x)$, $y\leftarrow f(f(y))$, $d\leftarrow\gcd(|x-y|,n)$, with $f(x)=x^2+1 \bmod n$. If $d=n$, **fail** (restart with $f(x)=x^2+c$ for another $c\neq 0,-2$); otherwise output the factor d.

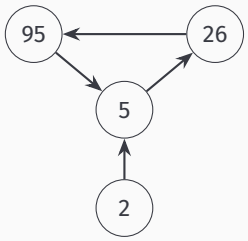
WORKED EXAMPLE : Pollard rho on $n=8051$

Q. Use Pollard's rho method with $f(x)=x^2+1$ and $x_0=y_0=2$ to factor $n=8051$.

i	$x_i=f(x_{i-1})$	$y_i=f(f(y_{i-1}))=x_{2i}$	$\lvert x_i-y_i\rvert$	$\gcd(\lvert x_i-y_i\rvert,8051)$
1	$2^2+1=5$	$f(5)=26$	21	1
2	$5^2+1=26$	$f(677)=677^2+1 \bmod 8051=7474$	7448	1
3	$26^2+1=677$	$f(2839)=871$ $(x_5=2839,\ x_6=871)$	194	**97**

(The hare passes through $x_3=677$, $x_4=7474$, $x_5=2839$, $x_6=871$.) $d=\gcd(194,8051)=97$, so **8051 = 97 × 83**. Check: $97\times 83=8051$ ✓.

Why at step 3: modulo 97 the sequence is $2,5,26,95,5,26,95,\ldots$ (tail $\mu=1$, cycle $\lambda=3$), so $x_3\equiv x_6\equiv 95 \pmod{97}$ ($677=6\cdot 97+95$, $871=8\cdot 97+95$). Modulo 83 it is $2,5,26,13,4,17,41,\ldots$, still without repetition, so 83 is not in the gcd.



$x_i \bmod 97$ for $n=8051$:
tail $x_0=2$, then the cycle
$5\to 26\to 95\to 5$ (length 3).
Floyd: $x_3\equiv x_6 \pmod{97}$.

A larger run of the same procedure: $n=455459$ splits as 613×743 at step 9 (gcd 743).

49.5 The quadratic sieve ▸

▸ **Likely this year:** "Explain the idea of the quadratic sieve (factor base, smooth numbers, linear algebra mod 2)" (4 marks), possibly with $n = 1649$.

Idea: congruent squares. Fermat's method looks for $x^2 - n = y^2$ exactly. The quadratic sieve (QS, Pomerance 1981) only needs

$$x^2 \equiv y^2 \pmod{n} \quad \text{with} \quad x \not\equiv \pm y \pmod{n}.$$

Then $n \mid (x-y)(x+y)$ but n divides neither factor, so $\gcd(x-y, n)$ is a proper factor. For $n = pq$, a random pair with $x^2 \equiv y^2$ has $x \equiv \pm y$ modulo each prime independently, so a useful pair appears with probability $1/2$.

Building the square from smooth numbers. Take x near $\sqrt{n}$ and compute $Q(x) = x^2 - n$. Since $Q(x) \approx 2\sqrt{n}\,(x - \sqrt{n})$ is small, it has a good chance of being **smooth**, i.e. of factoring completely over a fixed set of small primes, the **factor base**. Only primes p for which n is a quadratic residue modulo p can ever divide $Q(x)$, since $p \mid x^2 - n$ means $x^2 \equiv n \pmod{p}$; the factor base keeps just those. Each smooth $Q(x)$ is a **relation** $x^2 \equiv Q(x) \pmod{n}$, recorded by the vector of its prime exponents *modulo 2*. A product of relations is a square on the right-hand side exactly when the exponent vectors add up to zero mod 2, so a suitable subset is found by **Gaussian elimination over GF(2)**; with more relations than primes in the factor base a dependency must exist. Multiplying the chosen relations gives $X^2 \equiv Y^2 \pmod{n}$.

Why "sieve". Testing each $Q(x)$ by trial division would be slow. Instead, for each factor-base prime p one solves $x^2 \equiv n \pmod{p}$ once; then $p \mid Q(x)$ exactly for x in two residue classes modulo p, so the multiples can be marked along an interval of x values, exactly like the sieve of Eratosthenes, using additions of $\log p$ instead of divisions. This makes collecting relations fast, and the method runs in $L_n[1/2, 1]$. It was the fastest method for numbers up to about 100 digits (RSA-129 was factored with a QS variant in 1994).

WORKED EXAMPLE : congruent squares for $n = 1649$

Q. Factor $n = 1649$ with the quadratic-sieve idea, using $x = 41, 42, \ldots$ ($\lceil\sqrt{1649}\rceil = 41$).
Quadratic residues: $1649 \equiv 2 \pmod 3$ (a non-residue), $\equiv 4 \pmod 5$ and $\equiv 4 \pmod 7$ (residues), so the factor base up to 7 is $\{2, 5, 7\}$.

x	$Q(x) = x^2 - 1649$	factorization	smooth over $\{2,5,7\}$?	exponents mod 2
41	32	2^5	yes	$(1,0,0)$
42	115	$5 \cdot 23$	no	
43	200	$2^3 \cdot 5^2$	yes	$(1,0,0)$
44	287	$7 \cdot 41$	no	
47	560	$2^4 \cdot 5 \cdot 7$	yes	$(0,1,1)$

The vectors of $x = 41$ and $x = 43$ add to $(0,0,0)$, so their product is a square: $41^2 \cdot 43^2 \equiv 32 \cdot 200 = 6400 = 2^8 \cdot 5^2 = 80^2$.
$X = 41 \cdot 43 \bmod 1649 = 1763 - 1649 = 114$, $Y = 80$, and $114^2 \equiv 80^2 \pmod{1649}$ with $114 \not\equiv \pm 80$.
$\gcd(114 - 80, 1649) = \gcd(34, 1649) = 17$ and $\gcd(114 + 80, 1649) = \gcd(194, 1649) = 97$, so $\mathbf{1649 = 17 \times 97}$ ✓.

A larger textbook run: for $n = 87463$ the factor base is $\{2, 3, 13, 17, 19, 29\}$, and $Q(296) = 153 = 3^2 \cdot 17$ and $Q(316) = 12393 = 3^6 \cdot 17$ combine to $(296 \cdot 316)^2 \equiv (3^4 \cdot 17)^2$, i.e. $6073^2 \equiv 1377^2$, giving $\gcd(6073 - 1377, 87463) = 587$ and $87463 = 149 \times 587$.

49.6 The number field sieve ▸

▸ **Likely this year:** "Name the fastest factoring algorithm and its complexity; what is the current record?" (2 marks).

The **number field sieve** (NFS; proposed by Pollard in 1988 and developed by A. K. and H. W. Lenstra, Manasse, Pomerance and others in 1990–93) also produces congruent squares, but the numbers it needs to be smooth are much smaller than the $\approx \sqrt{n}$ values of QS. It chooses a polynomial f of small degree d (typically 5 or 6 for RSA sizes) and an integer m with $f(m) \equiv 0 \pmod{n}$, and works simultaneously in the integers and in the ring $\mathbb{Z}[\alpha]$ of the algebraic number field generated by a root α of f. It sieves for pairs (a, b) such that $a - bm$ is smooth over ordinary primes and $a - b\alpha$ is smooth over "prime ideals" of the number field. Linear algebra mod 2 again selects a subset whose product is a square on both sides, and the ring map $\alpha \mapsto m$ carries the two squares into $\mathbb{Z}_n$ as $X^2 \equiv Y^2 \pmod{n}$. Since the values to be smooth are about $n^{1/d}$ instead of $\sqrt{n}$, smooth values are far more common, and the running time becomes

$$L_n\!\left[1/3,\ (64/9)^{1/3}\right] = L_n[1/3,\ 1.923].$$

NFS is the fastest known classical algorithm for numbers without special form and holds all RSA factoring records. RSA-768 (232 digits) was factored in 2009, and **RSA-250 (250 digits, 829 bits)** in 2020 with the CADO-NFS software, using about 2700 core-years. No 1024-bit RSA modulus has been publicly factored by these methods.

49.7 Comparison and implications for RSA key sizes ▸

▸ **Likely this year:** "Compare the factoring methods (complexity, when each works); what key length is safe today?" (3–4 marks).

Method	Complexity	Depends on	Works well when
Trial division	$O(\sqrt{n}) = L_n[1, 1/2]$	smallest factor p (~ p steps)	n has a small factor
Fermat	$\approx (p-q)^2/(8\sqrt{n})$ steps	gap $\lvert p-q\rvert$	$p \approx q$
Pollard $p-1$	$\approx B \log B$ mult.	largest prime of $p-1$	$p-1$ is B-smooth
Pollard rho	$O(\sqrt{p}) \le O(n^{1/4})$	smallest factor p	p up to ~ 20 digits
ECM	$L_p[1/2, \sqrt{2}]$	smallest factor p	p up to ~ 70 digits
Quadratic sieve	$L_n[1/2, 1]$	size of n only	n up to ~ 100 digits
Number field sieve	$L_n[1/3, 1.923]$	size of n only	large n (RSA moduli)

The first five methods exploit a weakness of one factor (small, close to the other, or with smooth $p-1$); a correctly generated RSA modulus has none, so only the general-purpose sieves apply, and their cost depends only on the size of n. Evaluating the formulas without the $o(1)$ term gives rough work estimates in bit-security terms (they overstate a little; the accepted equivalences in the last column are NIST's):

Size of n	Trial division	Pollard rho	QS $L[1/2, 1]$	NFS $L[1/3, 1.923]$	Accepted security
512 bits	2^{256}	2^{128}	$\approx 2^{66}$	$\approx 2^{64}$	broken (1999)
1024 bits	2^{512}	2^{256}	$\approx 2^{99}$	$\approx 2^{87}$	80 bits (deprecated)
2048 bits	2^{1024}	2^{512}	$\approx 2^{146}$	$\approx 2^{117}$	112 bits
3072 bits	2^{1536}	2^{768}	$\approx 2^{184}$	$\approx 2^{139}$	128 bits

Consequences. The 1024-bit minimum of the lecture slides (§33) is now considered too small for new keys; 2048 bits is the current minimum and 3072 bits matches AES-128. Since NFS is sub-exponential, doubling the key length far less than squares the work, which is why RSA keys grow quickly for each extra bit of security (15360 bits for 256-bit security), and why elliptic-curve keys (256 bits for 128-bit security) are preferred where size matters. Shor's algorithm would factor n in polynomial time on a large quantum computer, which is the reason for the current migration to post-quantum algorithms.

50 Attacks on RSA: Can Any of Them Succeed?

50.1 The question, the toy key and the reference key ▸

▸ **Likely this year:** "Explain the attacks on RSA. Verify whether any of them can be launched successfully; if not, justify why" (8–10 marks, the assignment question).

§33 lists the attacks on RSA in one line each. The assignment asks for more: for each attack, *why* it works, a demonstration that it really does work on a weak key, and a justification of why it fails against a correctly chosen key. Every attack below exploits one of three things: a mathematical shortcut that exists only for special parameters (small e, small d, shared n, special primes), the *deterministic and multiplicative* structure of textbook RSA, or a leak from the implementation. The demonstrations use the textbook toy key

$$p = 53, \quad q = 61, \quad n = 3233, \quad \phi(n) = 3120, \quad e = 17, \quad d = 2753 \quad (17 \times 2753 = 46801 = 15 \cdot 3120 + 1),$$

and the verdicts are given for the **reference key** that standards require today: n of 2048 bits or more with random primes p, q of equal length generated independently by each user, $e = 65537$, $d \approx n$ in size, messages encoded with OAEP (§51), and a constant-time, blinded implementation.

50.2 Attack 1: factoring n

Idea. The private key is a function of the factors: $\phi(n) = (p-1)(q-1)$ and $d = e^{-1} \bmod \phi(n)$. Eve, who knows (e, n), repeats Bob's key generation as soon as she factors n.

Demonstration. For $n = 3233$, trial division up to $\lfloor\sqrt{3233}\rfloor = 56$ finds $53 \mid 3233$, so $q = 61$, $\phi = 52 \times 60 = 3120$ and $d = 17^{-1} \bmod 3120 = 2753$: the key is completely broken. A related real-world failure is a *shared prime*: if two keys were generated with a poor random-number generator and share a factor, a single gcd breaks both. For $n_1 = 3233 = 53 \cdot 61$ and $n_2 = 4087 = 61 \cdot 67$, Euclid gives $\gcd(3233, 4087) = 61$ immediately. Surveys of public keys on the internet in 2012 found thousands of device keys broken this way.

Can it be launched? Not against the reference key. The special-purpose methods of §49 need a weakness of one factor (small, $p \approx q$, smooth $p-1$), and random 1024-bit primes have none of them with any measurable probability; the general NFS needs roughly 2^{112} operations for 2048 bits, and the record is 829 bits. Shared primes are avoided by seeding key generation with enough entropy. Factoring remains the ultimate attack, which is why key sizes grow over time and why quantum computers would end RSA.

50.3 Attack 2: common modulus ★▸

▸ **Likely this year:** "Two users share n; the same message is sent with e_1, e_2. Recover it" (5 marks).

Setting. To save work, a trusted centre picks one modulus n and gives each user a different pair (e_i, d_i). Alice sends the same M to two users with exponents e_1, e_2, $\gcd(e_1, e_2) = 1$; Eve sees $C_1 = M^{e_1} \bmod n$ and $C_2 = M^{e_2} \bmod n$.

Derivation. Since $\gcd(e_1, e_2) = 1$, the extended Euclidean algorithm gives integers s, t with $se_1 + te_2 = 1$ (one of them negative). Then

$$C_1^s C_2^t \equiv M^{se_1} M^{te_2} = M^{se_1 + te_2} = M^1 \pmod{n}.$$

A negative exponent is computed with the inverse: if $s < 0$, use $(C_1^{-1})^{-s}$, which exists unless $\gcd(C_1, n) > 1$ (and that would factor n). No private key is needed.

WORKED EXAMPLE : common modulus attack (new)

Q. Two users share $n = 3233$ with public exponents $e_1 = 7$ and $e_2 = 11$. The same message is sent to both; Eve intercepts $C_1 = 1844$ and $C_2 = 1268$. Recover M.
Step 1: s, t. Extended Euclid on $(11, 7)$:

q	r_1	r_2	r	s_1	s_2	s	t_1	t_2	t
1	11	7	4	1	0	1	0	1	−1
1	7	4	3	0	1	−1	1	−1	2
1	4	3	1	1	−1	2	−1	2	−3
3	3	1	0	−1	2	−7	2	−3	11
	1	0		**2**	−7		**−3**	11	

$2 \cdot 11 + (-3) \cdot 7 = 1$, so the exponent of C_1 (for $e_1 = 7$) is -3 and that of C_2 (for $e_2 = 11$) is 2.
Step 2: inverse. $C_1^{-1} = 1844^{-1} \bmod 3233 = 3098$ (check: $1844 \times 3098 \equiv 1$).
Step 3: combine. $(C_1^{-1})^3 = 3098^3 \bmod 3233 = 3171$; $C_2^2 = 1268^2 \bmod 3233 = 1023$; $M = 3171 \times 1023 \bmod 3233 =$ **1234**.
Check. $1234^7 \bmod 3233 = 1844$ and $1234^{11} \bmod 3233 = 1268$ ✓.

The insider attack. A shared modulus is broken even if no message is ever sent twice, because any user who holds one pair (e, d) can factor n. Write $ed - 1 = 2^s t$ with t odd; $ed - 1$ is a multiple of $\phi(n)$, so $a^{ed-1} \equiv 1$ for every a coprime to n. Squaring a^t repeatedly therefore reaches 1, and the value just before the first 1 is a square root of 1; with probability at least $1/2$ it is a *nontrivial* one, which factors n exactly as in the Miller–Rabin discussion (§27). With the toy key, $ed - 1 = 46800 = 2^4 \times 2925$. Base $a = 2$: $2^{2925} \equiv 1514$, $1514^2 \equiv 3232 = -1$, a trivial root, so try again. Base $a = 3$: $3^{2925} \equiv 2256$, $2256^2 \equiv 794$, $794^2 \equiv 1$; so 794 is a nontrivial square root of 1, and $\gcd(793, 3233) = 61$, $\gcd(795, 3233) = 53$. This is also why a leaked d forces Bob to change n, not just d.

Can it be launched? Only if a modulus is shared, which the recommendations forbid ("the modulus n must not be shared"). With every user generating his own p, q, the attack does not apply.

50.4 Attack 3: Håstad's broadcast attack (low encryption exponent) ★▸

▸ **Likely this year:** "The same message is sent with $e = 3$ to three users; show how Eve recovers it" (5 marks).

Setting. A small public exponent such as $e = 3$ makes encryption cheap. Alice sends the same unpadded M to three receivers with pairwise coprime moduli n_1, n_2, n_3, all with $e = 3$: $C_i = M^3 \bmod n_i$.

Derivation. By the CRT (§27) Eve computes the unique $X < N = n_1 n_2 n_3$ with $X \equiv C_i \pmod{n_i}$ for $i = 1, 2, 3$; this X is $M^3 \bmod N$. Since $M < n_i$ for each i, $M^3 < n_1 n_2 n_3 = N$, so no reduction took place: $X = M^3$ *as an integer*, and $M = \sqrt[3]{X}$ is an ordinary cube root, which is easy. In general, e ciphertexts of the same message under exponent e suffice. (If two moduli were not coprime, their gcd would factor both.)

WORKED EXAMPLE : Håstad's attack with $e = 3$ (new)

Q. Alice sends the same M with $e = 3$ to three users with $n_1 = 187$, $n_2 = 667$, $n_3 = 1927$. The ciphertexts are $C_1 = 111$, $C_2 = 167$, $C_3 = 1814$. Recover M.
$N = 187 \times 667 \times 1927 = 240\,352\,783$; $N_i = N/n_i$; $y_i = N_i^{-1} \bmod n_i$:

i	n_i	C_i	N_i	$N_i \bmod n_i$	y_i	$C_i N_i y_i$
1	187	111	1 285 309	58	158	22 541 749 242
2	667	167	360 349	169	371	22 326 142 993
3	1927	1814	124 729	1401	1154	261 102 200 524
					sum	305 970 092 759

$X = 305\,970\,092\,759 \bmod 240\,352\,783 = 1\,000\,000$ (since $1273 \times 240\,352\,783 = 305\,969\,092\,759$).

$M = \sqrt[3]{1\,000\,000} = \mathbf{100}$. **Check.** $100^3 = 10^6$: $10^6 \bmod 187 = 111$, $\bmod 667 = 167$, $\bmod 1927 = 1814$ ✓. (All three $\phi(n_i) = 160, 616, 1840$ are coprime to 3, so $e = 3$ is a legal exponent for each user.)

Can it be launched? Not against the reference setup, for two independent reasons. With $e = 65537$, Eve would need 65537 ciphertexts of the same message under different moduli. More fundamentally, OAEP puts fresh randomness into every encryption, so the three receivers encrypt three *different* numbers and the CRT combination is not a perfect cube. (Håstad's general theorem still breaks fixed, known padding such as appending a user number, so the padding must be random.)

50.5 Attacks 4 and 5: short messages and plaintext–key relations ▸

▸ **Likely this year:** "Why is textbook RSA insecure for short messages? Why are $e = 1$ and small M dangerous?" (3 marks).

Short-message (dictionary) attack. Textbook RSA is *deterministic*: the same M always gives the same C. If M comes from a small set (a 4-digit PIN, "yes"/"no", a bid amount), Eve encrypts every candidate with the public key and compares with C. Demonstration: a 2-digit PIN encrypted with $(17, 3233)$ gives $C = 2557$; Eve computes $m^{17} \bmod 3233$ for $m = 0, \ldots, 99$ and only $m = 42$ matches ($41 \mapsto 3199$, $42 \mapsto 2557$, $43 \mapsto 706$), so $M = 42$ after at most 100 encryptions.

Plaintext–key relation (no modular reduction). If $M^e < n$, the reduction modulo n never happens and $C = M^e$ over the integers, so $M = \sqrt[e]{C}$ is an ordinary root. With $e = 3$ and $n = 1927$: $M = 12$ gives $C = 1728 < 1927$, and $\sqrt[3]{1728} = 12$. This is realistic: a 128-bit AES key encrypted with $e = 3$ under a 2048-bit modulus is a 384-bit cube that never wraps. The extreme case $e = 1$ is formally allowed ($\gcd(1, \phi) = 1$) but gives $C = M$. Unconcealed messages ($M^e \equiv M$, such as $0, 1, n - 1$) are a related relation; see §33.

Can they be launched? Not with OAEP and $e = 65537$. OAEP's random r makes encryption probabilistic, so encrypting a candidate PIN gives a different ciphertext each time and the comparison is useless (Eve would also have to guess r, of 256 bits in practice). The encoded block is as long as n (its top byte is zero, the rest random-looking), so $(\mathrm{EM})^e$ with $e = 65537$ always wraps around n many times.

50.6 Attack 6: the cycling attack ▸

▸ **Likely this year:** "Explain the cycling attack with an example" (3–4 marks).

Idea. Encryption $x \mapsto x^e \bmod n$ is a permutation of $\mathbb{Z}_n$, so repeated encryption of the ciphertext must eventually return to it: $C, C^e, C^{e^2}, \ldots, C^{e^k} = C$. The term just before C reappears is the one whose encryption is C, i.e. the plaintext M. Eve needs only the public key.

How long is the cycle? Let $\lambda(n) = \mathrm{lcm}(p - 1, q - 1)$. Then $x^{\lambda(n)} \equiv 1$ modulo p and modulo q for x coprime to n, so $x^{1+j\lambda(n)} \equiv x \pmod n$. Hence if $e^k \equiv 1 \pmod{\lambda(n)}$, every message returns after k encryptions: the cycle length divides $\mathrm{ord}_{\lambda(n)}(e)$. For the toy key $\lambda(3233) = \mathrm{lcm}(52, 60) = 780$ and $\mathrm{ord}_{780}(17) = 12$, so *every* ciphertext cycles within 12 steps.

WORKED EXAMPLE : cycling attack on $(17, 3233)$

Q. Eve intercepts $C = 2790$ sent with the public key $(17, 3233)$. Recover M by cycling.
Re-encrypt repeatedly, $C_{i+1} = C_i^{17} \bmod 3233$:

i	0	1	2	3	4	5	6	7	8	9	10	11	12
C_i	2790	1452	1930	2505	2973	171	405	1346	1082	2611	588	**65**	2790

$C_{12} = C_0$, so $C_{11}^{17} \equiv C$, and $\mathbf{M = 65}$. **Check.** $65^{17} \bmod 3233 = 2790$ ✓. A smaller key shows the same thing in four steps: with $(3, 55)$ and $C = 13$, the cycle is $13 \to 52 \to 28 \to 7 \to 13$, so $M = 7$.

Can it be launched? No. For random 1024-bit primes, $\lambda(n)$ has large prime factors and the order of e modulo $\lambda(n)$ is astronomically large, so the expected cycle length is far beyond 2^{100} steps. Moreover, the cycling attack is a disguised factoring algorithm: at each step $\gcd(C_i - C, n)$ may reveal p (when the cycle closes modulo p before modulo q), and it is never faster than the general factoring methods for random primes (Rivest and Silverman, 1999). The toy key fails only because $p - 1 = 52$ and $q - 1 = 60$ are tiny.

50.7 Attack 7: Wiener's low private exponent attack ▸

▸ **Likely this year:** "Why must d not be small? Show Wiener's attack on a small example" (4–5 marks).

Setting. Decryption costs grow with the size of d, so a designer may be tempted to pick a small d (e.g. for a smart card) and compute $e = d^{-1} \bmod \phi(n)$. Wiener (1990) showed that if $q < p < 2q$ and

$$d < \tfrac{1}{3} n^{1/4},$$

then d can be recovered from the public key alone.

Derivation. From $ed = 1 + k\phi(n)$ for some integer k,

$$\frac{e}{n} - \frac{k}{d} = \frac{ed - kn}{nd} = \frac{1 - k(n - \phi(n))}{nd}.$$

Here $n - \phi(n) = p + q - 1 < 3\sqrt{n}$, and $k < d$ (since $k\phi(n) = ed - 1 < ed < \phi(n)\,d$), so $\left|\frac{e}{n} - \frac{k}{d}\right| < \frac{3k\sqrt{n}}{nd} < \frac{3}{\sqrt{n}}$. If $d < \frac{1}{3}n^{1/4}$, then $\frac{1}{2d^2} > \frac{9}{2\sqrt{n}} > \frac{3}{\sqrt{n}}$, so $\left|\frac{e}{n} - \frac{k}{d}\right| < \frac{1}{2d^2}$. A classical theorem of Legendre says that any fraction this close to e/n is one of the **convergents** of the continued fraction of e/n. Eve therefore expands e/n, and for each convergent k/d tests:

$$\phi = \frac{ed - 1}{k} \text{ must be an integer;} \qquad x^2 - (n - \phi + 1)\,x + n = 0 \text{ must have integer roots } p, q.$$

The second test works because $p + q = n - \phi(n) + 1$ and $pq = n$ (§33).

WORKED EXAMPLE : Wiener's attack

Q. Bob's public key is $(e, n) = (17993, 90581)$. Show that Wiener's attack recovers d.
Continued fraction of e/n. $17993 = 0{\cdot}90581 + 17993$; $90581 = 5{\cdot}17993 + 616$; $17993 = 29{\cdot}616 + 129$; $616 = 4{\cdot}129 + 100$; $129 = 1 \cdot 100 + 29$; $100 = 3 \cdot 29 + 13$; $29 = 2 \cdot 13 + 3$; $13 = 4 \cdot 3 + 1$; $3 = 3 \cdot 1$.
So $e/n = [0; 5, 29, 4, 1, 3, 2, 4, 3]$, with convergents $\frac{0}{1}, \frac{1}{5}, \frac{29}{146}, \frac{117}{589}, \ldots$
Test $k/d = 1/5$. $\phi = (17993 \cdot 5 - 1)/1 = 89964$ (an integer). $p + q = n - \phi + 1 = 90581 - 89964 + 1 = 618$. Solve $x^2 - 618x + 90581 = 0$: discriminant $618^2 - 4{\cdot}90581 = 381924 - 362324 = 19600 = 140^2$, so $x = (618 \pm 140)/2 = 379, 239$.
Result. $n = 379 \times 239$ and $\mathbf{d = 5}$. **Check.** $\phi(n) = 378 \times 238 = 89964$ and $17993 \times 5 = 89965 = 89964 + 1$ ✓; the condition holds since $\frac{1}{3}n^{1/4} = 5.78 > 5$.

Can it be launched? No, because with $e = 65537$ the private exponent $d = e^{-1} \bmod \phi(n)$ is essentially a random number of the same size as n (2048 bits), far above $n^{1/4}$ (512 bits). Boneh and Durfee extended the attack to $d < n^{0.292}$, so the rule is simply never to choose d small. The legitimate way to speed up decryption is the CRT (§33), which keeps d large.

50.8 Attack 8: chosen-ciphertext attack (multiplicative property) ▸

▸ **Likely this year:** "Show how Eve decrypts C using Bob's willingness to decrypt other ciphertexts" (4 marks).

Idea. Textbook RSA is *malleable*: $(MX)^e = M^e X^e$, so Eve can turn a ciphertext of M into a ciphertext of MX without knowing M. Suppose Bob decrypts any ciphertext except the target C (a "decryption oracle", e.g. a server that returns decrypted data or signs blindly). Eve picks a random X, sends $Y = C \cdot X^e \bmod n$, receives $Z = Y^d = (M^e X^e)^d = MX \bmod n$, and computes $M = ZX^{-1} \bmod n$. Bob sees only the random-looking Y.

Demonstration. Key $(17, 3233)$, $d = 2753$; target $C = 1892$. Eve picks $X = 2$: $X^e = 2^{17} \bmod 3233 = 1752$, $Y = 1892 \times 1752 \bmod 3233 = 959$. Bob returns $Z = 959^{2753} \bmod 3233 = 1710$. Eve computes $X^{-1} = 2^{-1} \bmod 3233 = 1617$ and $M = 1710 \times 1617 \bmod 3233 = \mathbf{855}$. Check: $855^{17} \bmod 3233 = 1892$ ✓.

Can it be launched? Not against OAEP. The decryption of Y is $2 \cdot \mathrm{EM} \bmod n$, which decodes to garbage and fails OAEP's redundancy check, so Bob rejects Y and returns nothing (see the tampering demonstration in §51). The attack is not only theoretical: Bleichenbacher's 1998 "million message" attack recovered TLS session keys protected by the older PKCS#1 v1.5 padding, using only the server's "padding correct/incorrect" replies as the oracle, and variants (ROBOT, 2017) still worked on many servers. Hence OAEP, uniform error messages, and TLS 1.3's removal of RSA key transport.

50.9 Attack 9: timing and other implementation attacks ▸

▸ **Likely this year:** "What is a timing attack on RSA? How do blinding and constant-time exponentiation stop it?" (3–4 marks).

Idea. Square-and-multiply (§22) performs a squaring for every bit of d and an extra multiplication only for the 1-bits. The decryption time therefore depends on the bits of d (and on the ciphertext, through the cost of modular reductions). Kocher (1996) showed that by timing many decryptions of known ciphertexts and correlating the times with the predicted cost of the next bit, an attacker recovers d bit by bit; Brumley and Boneh (2003) did it remotely against an OpenSSL web server. For $d = 2753 = 101011000001_2$ the left-to-right method does 11 squarings and only 4 multiplications, one after each 1-bit following the leading 1; a different d of the same length takes a different time. Power analysis (measuring the current drawn by a smart card) leaks the same pattern even more directly, and fault attacks on CRT decryption factor n from a single faulty result.

Countermeasures.

- **Constant-time exponentiation**: perform the same operations for every bit, e.g. the Montgomery ladder (one squaring and one multiplication per bit, whatever its value), with no secret-dependent branches or memory accesses.
- **Blinding**: before decrypting, multiply C by r^e for a fresh random r; decrypt $C' = Cr^e$ to get $P' = C'^d = Pr$; output $P = P'r^{-1}$. The timed computation now runs on a random input unrelated to what the attacker chose.

Demonstration with $r = 7$: $r^e = 7^{17} \bmod 3233 = 2369$, $C' = 1892 \times 2369 \bmod 3233 = 1210$, $P' = 1210^{2753} \bmod 3233 = 2752$, $r^{-1} = 462$, $P = 2752 \times 462 \bmod 3233 = 855$ ✓.

- Random delays are weak, since averaging over many measurements removes them. Against fault attacks, the implementation re-encrypts the result and checks it before output.

Can it be launched? Against an unprotected implementation, yes, even over a network; against the reference implementation (constant time plus blinding), no. This attack shows that a mathematically sound key is not sufficient: the code must not leak.

50.10 Verdict: no attack succeeds against correctly used RSA ★▸

▸ **Likely this year:** The conclusion of the assignment answer: a table of attack, required weakness, and defence.

Attack	Works only if	Why it fails for the reference key	Succeeds?
Factoring n	n small, $p \approx q$, $p-1$ smooth, shared prime	2048-bit random primes: NFS $\approx 2^{112}$, record 829 bits	No
Common modulus	n shared by users	every user has his own n	No
Håstad broadcast	small e, same unpadded M to e users	$e = 65537$; OAEP randomizes each M	No
Short message	deterministic encryption	OAEP's random r	No
$M^e < n$, $e = 1$	tiny e or tiny unpadded M	OAEP fills the block; $e = 65537$	No
Cycling	small $\mathrm{ord}_{\lambda(n)}(e)$	huge order for random primes; as hard as factoring	No
Wiener	$d < \frac{1}{3}n^{1/4}$	$d \approx n$ when $e = 65537$	No
Chosen ciphertext	malleable textbook RSA + oracle	OAEP redundancy check rejects modified C	No
Timing / power / fault	leaky implementation	constant-time code, blinding, result check	No

★ MUST KNOW : the assignment's conclusion

Each attack was demonstrated successfully on a deliberately weak toy key, which shows that the attacks are real. None of them can be launched against RSA used as recommended (2048-bit or longer n from independent random primes, $e = 65537$, large d, OAEP padding, constant-time blinded decryption), because each one needs a weakness that these choices exclude. The only generic attack left is factoring n, whose best known cost (NFS, $L_n[1/3, 1.923]$) is infeasible at these sizes.

51 Optimal Asymmetric Encryption Padding (OAEP)

51.1 Why textbook RSA must be padded ▸

▸ **Likely this year:** "What are the weaknesses of textbook RSA that OAEP removes?" (3 marks).

Textbook RSA, $C = M^e \bmod n$, has three structural weaknesses that the attacks of §50 exploit. It is **deterministic**: the same M always gives the same C, so Eve can confirm guesses (short-message attack) and recognise repeated messages. It is **malleable**: from C anyone can compute a valid encryption CX^e of the related message MX (chosen-ciphertext attack). And small or structured messages leak through **algebraic relations** ($M^e < n$, broadcast, related messages). A probabilistic scheme such as ElGamal (§34) avoids the first problem by encrypting with a fresh random number every time. OAEP (Optimal Asymmetric Encryption Padding, Bellare and Rogaway, 1994) gives RSA the same property and more: before exponentiation, the message is mixed with a random value through two hash-based functions, so that the number actually encrypted looks random, carries a built-in integrity check, and is as long as the modulus.

51.2 OAEP encoding and decoding ★▸

▸ **Likely this year:** "Explain OAEP with a neat diagram; show that the receiver recovers M" (5–6 marks).

Ingredients. Let the encoded block have $m + k$ bits, slightly fewer than n. The message M is padded with a fixed run of zeros to m bits, giving M_{pad}; the zeros are redundancy that the receiver will check. Alice draws a fresh random string r of k bits. Two public **mask-generation functions** built from a hash function are used: G expands k bits to m bits, and H compresses m bits to k bits.

Encoding and encryption (Alice).

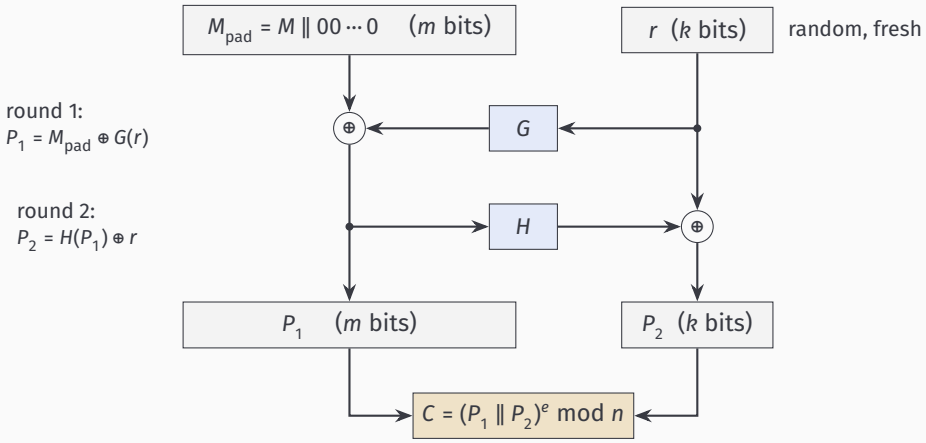
$$P_1 = M_{\text{pad}} \oplus G(r), \qquad P_2 = H(P_1) \oplus r, \qquad C = (P_1 \,\|\, P_2)^e \bmod n.$$

Decryption and decoding (Bob). Bob computes $P = C^d \bmod n$, splits it into P_1 (m bits) and P_2 (k bits), and undoes the two rounds in reverse order:

$$r = H(P_1) \oplus P_2, \qquad M_{\text{pad}} = P_1 \oplus G(r).$$

If the padding zeros of M_{pad} are not all zero, Bob **rejects** the ciphertext; otherwise he strips them and outputs M.

Why decoding works. Since $x \oplus x = 0$, $H(P_1) \oplus P_2 = H(P_1) \oplus H(P_1) \oplus r = r$; Bob can compute $H(P_1)$ because P_1 is part of what he decrypted. With r known, $P_1 \oplus G(r) = M_{\text{pad}} \oplus G(r) \oplus G(r) = M_{\text{pad}}$. The structure is a two-round Feistel network (§29) with G and H as round functions and no key, which is why it is invertible by anyone who has both halves, and only by them.



51.3 What OAEP achieves ▸

▸ **Likely this year:** "Why is RSA-OAEP randomized, all-or-nothing and non-malleable?" (3–4 marks).

Randomized. A fresh r is drawn for every encryption, and $G(r)$ changes all of P_1, so the same M gives a different $P_1 \| P_2$, hence a different C, every time. Guess-and-compare attacks and broadcast attacks lose their basis: Eve would have to guess r as well.

All-or-nothing. To learn even one bit of M, Eve needs the corresponding bit of $G(r)$, hence all of r (a hash output reveals nothing about a partially known input); to get r she needs P_2 and $H(P_1)$, hence all of P_1. So unless she inverts RSA on the *whole* block, she learns nothing about M. Partial information that RSA might leak about $P_1 \| P_2$ (a few bits, a Jacobi symbol) is useless.

Non-malleable, plaintext-aware. Any ciphertext that Eve constructs without knowing a message, such as CX^e, decrypts to a block whose P_1 and P_2 are unrelated to any valid encoding. Decoding then yields an M_{pad} whose padding bits are effectively random, so the redundancy check fails except with probability 2^{-z} for z padding bits, and Bob rejects. A decryption oracle therefore tells Eve nothing she did not already know. Modelling G and H as random oracles, RSA-OAEP was proved secure against adaptive chosen-ciphertext attacks (IND-CCA2) by Fujisaki, Okamoto, Pointcheval and Stern (2001).

OAEP as standardised (PKCS#1 v2.2, RFC 8017). Real OAEP uses one function, **MGF1** (a hash in counter mode, e.g. SHA-256), for both G and H. The data block is DB = lHash ‖ PS ‖ `0x01` ‖ M, where lHash is the hash of an optional label and PS is a string of zero bytes (the redundancy); seed = random r of h bytes (h = hash length). Then maskedDB = $DB \oplus$ MGF1(seed), maskedSeed = seed $\oplus$ MGF1(maskedDB), and the encoded block is EM = `0x00` ‖ maskedSeed ‖ maskedDB, which is encrypted with RSA. A k-byte modulus carries at most $k - 2h - 2$ message bytes: 190 bytes for 2048-bit RSA with SHA-256, plenty for a session key. All decoding failures must produce the same error at the same time; Manger (2001) showed that distinguishing "leading byte not zero" from other failures again creates a usable oracle.

WORKED EXAMPLE : a toy OAEP with explicit G and H (new)

Q. Toy parameters: 4-bit messages padded with 4 zero bits ($m = 8$), 4-bit random r ($k = 4$), $G(r) = (37r + 11) \bmod 256$ written as 8 bits, and $H(x) = x_L \oplus x_R$ (the XOR of the two 4-bit halves of x). RSA key $n = 61 \times 97 = 5917$, $e = 7$, $d = 823$. (a) Encrypt $M = 1011$ with $r = 0110$ and again with $r = 1101$. (b) Decrypt the first ciphertext. (c) Eve multiplies the first ciphertext by 2^e; show that Bob rejects it.

(a) First encryption, $r = 0110 = 6$. $M_{\text{pad}} = 1011\,0000$. $G(6) = 37 \cdot 6 + 11 = 233 = 1110\,1001$.
$P_1 = 1011\,0000 \oplus 1110\,1001 = 0101\,1001$. $H(P_1) = 0101 \oplus 1001 = 1100$. $P_2 = 1100 \oplus 0110 = 1010$.
$P_1 \| P_2 = 0101\,1001\,1010 = 1434$. $C = 1434^7 \bmod 5917$: $1434^2 \equiv 3157$, $1434^4 \equiv 3157^2 \equiv 2421$, $1434^7 = 1434^4 \cdot 1434^2 \cdot 1434 \equiv 2421 \cdot 3157 \cdot 1434 \equiv 4250 \cdot 1434 \equiv$ **5907**.
Second encryption, $r = 1101 = 13$. $G(13) = 492 \bmod 256 = 236 = 1110\,1100$; $P_1 = 0101\,1100$; $H(P_1) = 0101 \oplus 1100 = 1001$; $P_2 = 1001 \oplus 1101 = 0100$; $P_1 \| P_2 = 0101\,1100\,0100 = 1476$; $C = 1476^7 \bmod 5917 =$ **2726**.
Same message, different ciphertexts. (Textbook RSA would give $11^7 \bmod 5917 = 2490$ both times.)
(b) Decryption of $C = 5907$. $5907^{823} \bmod 5917 = 1434 = 0101\,1001\,1010$, so $P_1 = 0101\,1001$, $P_2 = 1010$.
$r = H(P_1) \oplus P_2 = 1100 \oplus 1010 = 0110$; $G(r) = 1110\,1001$; $M_{\text{pad}} = 0101\,1001 \oplus 1110\,1001 = 1011\,\underline{0000}$.
Padding bits are zero, so Bob accepts and outputs **M** = **1011** ✓.

(c) Tampering. $C' = 5907 \cdot 2^7 \bmod 5917 = 4637$. Bob gets $4637^{823} \bmod 5917 = 2868 = 2 \times 1434 = 1011\,0011\,0100$: $P_1 = 1011\,0011$, $P_2 = 0100$.
$r = H(P_1) \oplus P_2 = (1011 \oplus 0011) \oplus 0100 = 1000 \oplus 0100 = 1100$; $G(12) = 455 \bmod 256 = 199 = 1100\,0111$;
$M_{\text{pad}} = 1011\,0011 \oplus 1100\,0111 = 0111\,\underline{0100}$: the padding bits are not zero, so Bob **rejects**. With textbook RSA the same trick would have produced the valid-looking plaintext $2M = 22$.

52 Hash Function Families: MD, SHA-1, SHA-2, SHA-3 and Others

§35 explained what a cryptographic hash must achieve (preimage, second-preimage and collision resistance, with generic costs 2^n and $2^{n/2}$), the Merkle–Damgård (MD) iteration, and the two hashes on the slides, SHA-512 and Whirlpool. The assignment asks for the remaining members of the families. They share the MD skeleton (pad, append length, split into blocks, compress with feed-forward), so the differences to learn are the block and digest sizes, the step function, and the attack history; SHA-3 is the exception, built on a different principle, the sponge.

52.1 MD4 and MD5 ▸

▸ **Likely this year:** "Explain MD5: padding, buffers, rounds, auxiliary functions; why is it no longer secure?" (5 marks).

Ronald Rivest designed **MD4** (1990) for speed on 32-bit processors: 512-bit blocks, a 128-bit digest held in four 32-bit words, and a compression function of 3 rounds of 16 steps (48 steps). Collisions in MD4 were found in 1995–96 and today can be produced by hand, so MD4 is only of historical interest. **MD5** (1992, RFC 1321) is MD4 strengthened with a fourth round, a distinct additive constant in every step, and a few changes to the mixing.

MD5 processing.

- **Padding and length.** A 1 bit and then 0 bits are appended until the length is 448 mod 512, followed by the original length as a 64-bit number, so the result is a multiple of 512 bits; the padding is between 1 and 512 bits. For "abc" (24 bits): 24 + 424 (padding) +64 (length) = 512 bits, exactly one block.
- **Buffer.** Four 32-bit words A, B, C, D start from the fixed IV `67452301`, `EFCDAB89`, `98BADCFE`, `10325476` (the bytes 01 23 45 … in little-endian order).
- **Compression: 4 rounds × 16 steps = 64 steps.** Each block is viewed as sixteen 32-bit words $X[0..15]$, each used once per round in a round-specific order. Step i updates one word:

$$A \leftarrow B + \big((A + g(B,C,D) + X[k] + T[i]) \lll s\big),$$

  then the roles of the words rotate, $(A,B,C,D) \leftarrow (D,A,B,C)$. Here $+$ is addition modulo 2^{32}, $\lll s$ is a left rotation by a step-specific amount, and $T[i] = \lfloor 2^{32} \mid \sin i \mid \rfloor$ (i in radians), e.g. $T[1] =$ `D76AA478`; like the SHA constants, these are "nothing up my sleeve" numbers.
- **Auxiliary functions**, one per round, each acting bitwise on three words:

$$F(X,Y,Z) = (X \wedge Y) \vee (\neg X \wedge Z), \quad G(X,Y,Z) = (X \wedge Z) \vee (Y \wedge \neg Z), \quad H(X,Y,Z) = X \oplus Y \oplus Z, \quad I(X,Y,Z) = Y \oplus (X \vee \neg Z).$$

  F is the "if X then Y else Z" selector (SHA's Conditional), G the same with Z as selector, H the parity, and I a further nonlinear mix.
- **Feed-forward.** After step 64 the input chaining words are added back modulo 2^{32} (as in SHA-512's final adding), and the four words form the next chaining value or, after the last block, the 128-bit digest.

Test values: MD5("") = `d41d8cd9 8f00b204 e9800998 ecf8427e` and MD5("abc") = `90015098 3cd24fb0 d6963f7d 28e17f72`; changing "abc" to "abd" flips 63 of the 128 digest bits, about half, as the avalanche property requires.

Why MD5 is broken. Its 128-bit digest already limits collision resistance to 2^{64} by the birthday bound (§35), but the real break is structural. In August 2004 Wang, Feng, Lai and Yu published actual MD5 collisions found by differential cryptanalysis, and the method was soon fast enough to find a collision in seconds on a laptop. Chosen-prefix collisions followed (Stevens, Lenstra and de Weger, 2007): two *different, chosen* documents can be extended so that their MD5 values agree. In 2008 researchers used one to create a rogue certificate-authority certificate trusted by browsers, and in 2012 the **Flame** espionage malware used an MD5 chosen-prefix collision to forge a Microsoft code-signing certificate, so that it could spread disguised as a Windows Update. MD5 remains unbroken only for preimages (best attack about 2^{123}), but it must not be used for signatures, certificates or anything else where collisions matter.

52.2 SHA-0 and SHA-1 ▸

▸ **Likely this year:** "Compare MD5 and SHA-1"; "what was the SHAttered attack?" (3–4 marks).

The NSA-designed **SHA-0** was published by NIST in 1993 (FIPS 180) and withdrawn almost immediately for an undisclosed weakness; its replacement **SHA-1** (FIPS 180-1, 1995) differs in a single detail. Both follow MD5's pattern with a longer digest: 512-bit blocks, a 64-bit length field, a 160-bit digest in five 32-bit words $A, \ldots, E$, and

80 steps arranged as four stages of 20 that use Ch (the selector), Parity, Maj (majority) and Parity again, with constants 5A827999, 6ED9EBA1, 8F1BBCDC, CA62C1D6 ($\lfloor 2^{30}\sqrt{2}\rfloor$, $\lfloor 2^{30}\sqrt{3}\rfloor$, $\lfloor 2^{30}\sqrt{5}\rfloor$, $\lfloor 2^{30}\sqrt{10}\rfloor$). Each step computes $T = (A \lll 5) + f_t(B,C,D) + E + K_t + W_t$ and shifts $E \leftarrow D$, $D \leftarrow C$, $C \leftarrow B \lll 30$, $B \leftarrow A$, $A \leftarrow T$. The 16 block words are expanded to 80 by

$$W_t = \big(W_{t-3} \oplus W_{t-8} \oplus W_{t-14} \oplus W_{t-16}\big) \lll 1, \qquad 16 \le t \le 79.$$

The rotation by 1 is the only change from SHA-0. Without it each bit position of the expanded words depends only on the same position of the block, so differences do not spread across positions, which made SHA-0 collisions practical (found in 2004).

Attack history. Wang, Yin and Yu (2005) showed that SHA-1 collisions cost about 2^{69} operations instead of the generic 2^{80}, which made SHA-1 "theoretically broken". In February 2017 the **SHAttered** attack (CWI Amsterdam and Google) produced the first actual collision: two different PDF files with the same SHA-1 digest, after about 2^{63} SHA-1 computations (roughly 6500 CPU-years and 110 GPU-years). In 2020 a chosen-prefix collision (Leurent and Peyrin, "SHA-1 is a Shambles") was used to impersonate a PGP key. Browsers stopped accepting SHA-1 certificates in 2017, Git is migrating to SHA-256, and NIST has announced that SHA-1 is to be phased out completely by 2030.

52.3 The SHA-2 family ▸

▸ **Likely this year:** "List the SHA-2 variants with block size, word size, rounds and digest size" (3 marks).

SHA-2 (published in 2001, standardised in FIPS 180-2 in 2002 and extended in FIPS 180-4) contains two genuinely different compression functions: **SHA-256** on 32-bit words (512-bit blocks, 64 rounds) and **SHA-512** on 64-bit words (1024-bit blocks, 80 rounds, §35). The other four members are *truncations* of these two, each with its own initial value: SHA-224 from SHA-256, and SHA-384, SHA-512/224 and SHA-512/256 from SHA-512. The separate IVs make the truncated digest unrelated to a prefix of the full one, so the variants act as independent functions. SHA-512/256 gives SHA-256's security level but runs faster than SHA-256 on 64-bit processors. No attack on the full SHA-2 functions better than the generic birthday and brute-force bounds is known.

Algorithm	Block	Word	Rounds	Digest	Max. message	Security (coll./preim.)	Length ext.
SHA-224	512	32	64	224	$2^{64}-1$	112 / 224	32 bits hidden
SHA-256	512	32	64	256	$2^{64}-1$	128 / 256	vulnerable
SHA-384	1024	64	80	384	$2^{128}-1$	192 / 384	resistant
SHA-512	1024	64	80	512	$2^{128}-1$	256 / 512	vulnerable
SHA-512/224	1024	64	80	224	$2^{128}-1$	112 / 224	resistant
SHA-512/256	1024	64	80	256	$2^{128}-1$	128 / 256	resistant

(Sizes in bits.) Test value: SHA-256("abc") = `ba7816bf 8f01cfea 414140de 5dae2223 b00361a3 96177a9c b410ff61 f20015ad`; for "abd" 122 of the 256 bits change.

The length-extension attack. In every MD hash the digest *is* the final chaining value H_t. Suppose someone authenticates messages with the naive MAC $T = h(K \,\|\, M)$ for a secret key K. Eve knows T and the length of $K\|M$ (not K). She sets the chaining value to T, appends the padding block that h would have added (computable from the length alone; call it *pad*), and continues hashing any extra text X. The result equals $h(K \,\|\, M \,\|\, \text{pad} \,\|\, X)$, a valid tag for the longer message $M\|\text{pad}\|X$, obtained without K. A script confirms this for SHA-256 with a 10-byte key, M = "amount=100", X = "&to=eve" and 44 bytes of glue padding. The truncated variants resist it because Eve does not see the whole chaining value (SHA-384 hides 128 of 512 bits; SHA-224 hides only 32, which can be guessed). HMAC (§36) defeats it for all MD hashes by hashing a second time with an outer key, so the attacker never sees the inner chaining value.

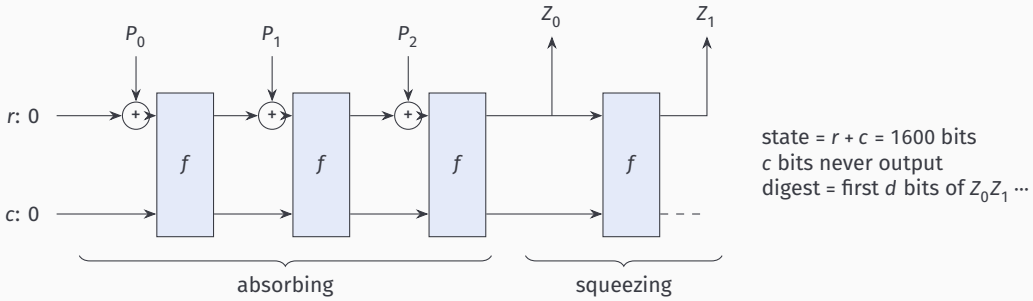
52.4 SHA-3 (Keccak) and the sponge construction ▸

▸ **Likely this year:** "Explain the sponge construction of SHA-3; why is SHA-3 not vulnerable to length extension?" (5 marks).

After the attacks on MD5 and SHA-1, NIST ran an open competition (2007–2012) for a hash with a *different* design, as insurance in case the MD/SHA-2 approach were broken. From 64 submissions and five finalists (BLAKE, Grøstl, JH, Keccak, Skein), **Keccak** by Bertoni, Daemen, Peeters and Van Assche was chosen, and it was standardised as SHA-3 in FIPS 202 (2015). SHA-3 complements SHA-2 rather than replacing it.

The sponge. Instead of a compression function, Keccak has a fixed *permutation* f = Keccak-f[1600] on a 1600-bit state, viewed as a 5×5 array of 64-bit *lanes*. The state is split into the **rate** r (the part that touches message and output) and the **capacity** c (the part that is never input or output directly), with $r + c = 1600$.

- **Padding.** A few domain-separation bits (01 for SHA-3, 1111 for SHAKE) and the *pad10*1* rule (a 1, zeros, a final 1) extend the message to a multiple of r bits $P_0, P_1, \ldots$
- **Absorbing.** Start from the all-zero state. For each block, XOR it into the first r bits and apply f: $S \leftarrow f\big(S \oplus (P_i \,\|\, 0^c)\big)$.
- **Squeezing.** Output the first r bits of the state; if more output is needed, apply f again and output r more bits; finally truncate to the digest length d.



The permutation f: 24 rounds of five steps. Each round applies, in order:

- θ (theta): each bit is XORed with the parities of two neighbouring columns, so every bit soon depends on many others (diffusion).
- ρ (rho): each of the 25 lanes is rotated by its own fixed offset, spreading bits along the 64 positions.
- π (pi): the 25 lanes are permuted to new positions in the 5×5 array.
- χ (chi): along each 5-bit row, $a_x \leftarrow a_x \oplus (\neg a_{x+1} \wedge a_{x+2})$. This is the **only nonlinear step**, and without it the whole function would be linear and easy to invert.
- ι (iota): a round constant is XORed into one lane, so that the rounds differ and the symmetry of the other steps is broken.

Parameters. The capacity is set to twice the digest length, $c = 2d$, because generic attacks on a sponge cost about $2^{c/2}$, so $c = 2d$ keeps them at the birthday level $2^{d/2}$. The rest of the state is the rate.

Function	Digest d	Rate r	Capacity c	Collision security	Preimage security
SHA3-224	224	1152	448	112	224
SHA3-256	256	1088	512	128	256
SHA3-384	384	832	768	192	384
SHA3-512	512	576	1024	256	512
SHAKE128	any	1344	256	$\min(d/2, 128)$	$\min(d, 128)$
SHAKE256	any	1088	512	$\min(d/2, 256)$	$\min(d, 256)$

SHAKE128 and SHAKE256 are **extendable-output functions** (XOFs): the squeezing phase simply continues for as many bits as requested, which is useful for key derivation and mask generation. Test value: SHA3-256("abc") = `3a985da7 4fe225b2 045c172d 6bd390bd 855f086e 3e9d525b 46bfe245 11431532`.

Why SHA-3 is immune to length extension. The attack on MD hashes worked because the digest *is* the complete internal state, so Eve could resume the computation from it. A sponge outputs only the first d bits of the 1600-bit state, and the c capacity bits are never output. To absorb another block after $h(K \| M)$, Eve would need the full state after the last application of f; she knows at most d of its bits and would have to guess the remaining $c \geq 2d$ bits (512 for SHA3-256), which is hopeless. For this reason the simple keyed construction $h(K \| M)$ is safe with SHA-3, and NIST's KMAC is built exactly this way, whereas MD hashes need the two-pass HMAC.

52.5 RIPEMD-160, Whirlpool and BLAKE2

RIPEMD-160 (Dobbertin, Bosselaers and Preneel, 1996) was designed in Europe as a strengthened successor of RIPEMD. It is an MD hash with 512-bit blocks and a 160-bit digest, but its compression function runs *two parallel lines* of 5 rounds × 16 steps with different functions and constants and combines them at the end, so an attacker must control two different computations at once. No practical collision is known; it survives mainly in Bitcoin, where an address is derived from RIPEMD-160(SHA-256(public key)).

Whirlpool is described in §35: a Miyaguchi–Preneel hash whose compression function is a 512-bit AES-like block cipher (10 rounds), with a 512-bit digest. It has no known practical weakness but is little used.

BLAKE2 (Aumasson, Neves, Wilcox-O'Hearn and Winnerlein, 2012) is a refinement of the SHA-3 finalist BLAKE, whose core is the add–rotate–XOR quarter-round of the ChaCha stream cipher. BLAKE2b uses 64-bit words, 1024-bit blocks and 12 rounds and outputs 1 to 64 bytes; BLAKE2s uses 32-bit words, 512-bit blocks and 10 rounds and outputs up to 32 bytes. Its iteration (HAIFA-style) feeds a byte counter and a "last block" flag into every compression, which blocks length extension, and it has a built-in keyed mode that serves as a MAC. It is faster than MD5 in software while offering SHA-3-level security, and is used in Argon2 (password hashing), WireGuard and libsodium.

52.6 Comparison of the hash families ★▸

▸ **Likely this year:** "Compare MD5, SHA-1, SHA-2 and SHA-3" (5 marks): reproduce this table and add one line on each attack.

Hash	Year	Construction	Block	Word	Steps	Digest	Status
MD4	1990	MD	512	32	48	128	broken (collisions by hand)
MD5	1992	MD	512	32	64	128	broken (2004; Flame 2012)
SHA-0	1993	MD	512	32	80	160	broken; withdrawn
SHA-1	1995	MD	512	32	80	160	broken (SHAttered 2017)
RIPEMD-160	1996	MD, two lines	512	32	2×80	160	no practical attack; legacy
SHA-256	2001	MD	512	32	64	256	secure
SHA-512	2001	MD	1024	64	80	512	secure
Whirlpool	2000	MP, AES-like	512	8	10 rounds	512	secure, rarely used
BLAKE2b	2012	HAIFA	1024	64	12 rounds	≤ 512	secure, fast
SHA3-256	2015	sponge	$r = 1088$	64	24 rounds	256	secure

(MD = Merkle–Damgård, MP = Miyaguchi–Preneel; sizes in bits.) For orientation, the first 16 hex digits of each hash of "abc" (script-checked): MD5 `900150983cd24fb0`, SHA-1 `a9993e364706816a`, SHA-256 `ba7816bf8f01cfea`, SHA-512 `ddaf35a193617aba`, SHA3-256 `3a985da74fe225b2`, RIPEMD-160 `8eb208f7e05d987a`, BLAKE2b-512 `ba80a53f981c4d0d`.

★ MUST KNOW : hash families

MD5: 512-bit blocks, 128-bit digest, $4 \times 16 = 64$ steps, functions F, G, H, I; collisions since 2004. SHA-1: 160-bit digest, 80 steps, the $\lll 1$ in the word expansion is its only difference from SHA-0; first collision SHAttered 2017. SHA-2: SHA-256 (32-bit words, 64 rounds) and SHA-512 (64-bit words, 80 rounds) plus truncations; secure. SHA-3: sponge, 1600-bit state, $r + c = 1600$, $c = 2d$, 24 rounds of $\theta\rho\pi\chi\iota$; immune to length extension because the capacity is never output.

Attacks on digital signature schemes. The assignment also covers attacks on signature schemes (key-only, known-message and chosen-message attacks; existential versus selective forgery; the RSA multiplicative and "choose the signature first" forgeries; the danger of reusing the random number in ElGamal or DSS). These are treated with the signature schemes themselves in §37 (attacks in §37.11). The link with this chapter is that a signature is computed on a *hash* of the message, so a collision in the hash is a signature forgery: Eve gets the harmless document of a colliding pair signed and transfers the signature to the harmful one, which is exactly what the MD5 rogue-certificate and Flame attacks did.

53 WhatsApp End-to-End Encryption (the Signal Protocol)

53.1 The problem and the keys ★▸

▸ **Likely this year:** "How does WhatsApp provide end-to-end encryption? Explain with a diagram" (8–10 marks): keys, X3DH, Double Ratchet, safety numbers.

Every WhatsApp message passes through WhatsApp's servers, which store it until the recipient comes online. Encrypting each link with TLS-style transport security would protect the message on the wire but leave it readable on the server. **End-to-end encryption** (E2EE) means that only the two endpoints hold the keys, so the server relays ciphertext it cannot read. This raises three design problems. The server must not be trusted, yet it is the only party that can hand out public keys. Two users must agree on a key even when the recipient is *offline* (asynchronous messaging), so an interactive Diffie–Hellman exchange is impossible. And a conversation lasts months, so the theft of a key at one moment must expose neither the earlier messages (**forward secrecy**) nor, for long, the later ones (**post-compromise security**). WhatsApp solves them with the **Signal Protocol** (designed by Open Whisper Systems; fully deployed in WhatsApp in 2016), whose two parts are the **X3DH** key agreement and the **Double Ratchet**.

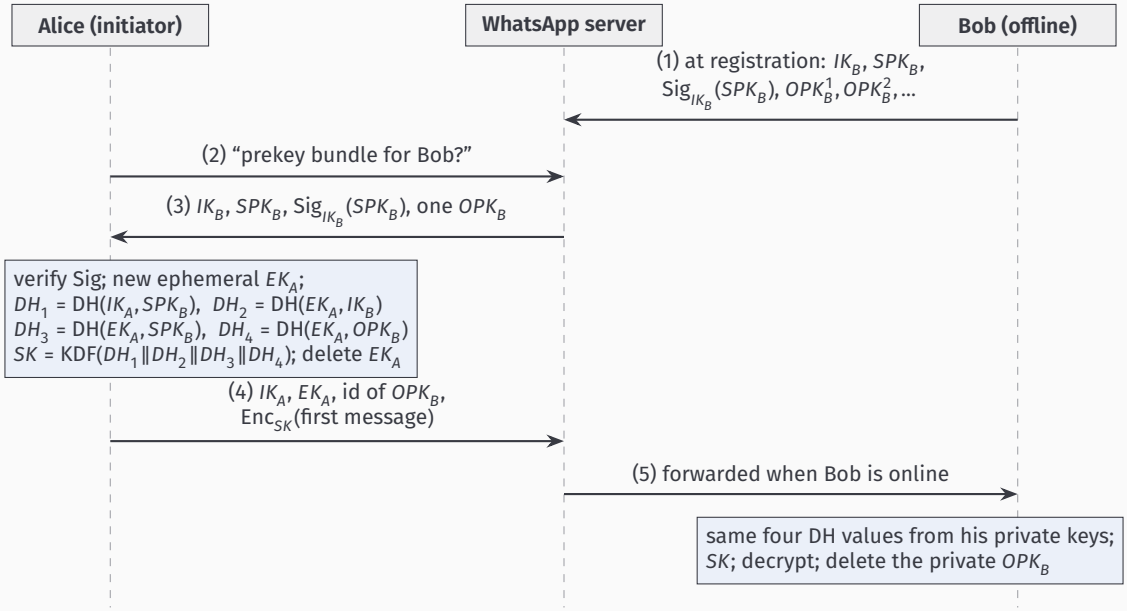
Keys. All public-key operations are elliptic-curve Diffie–Hellman on **Curve25519** (ECC, §34); $\mathrm{DH}(X, Y)$ below denotes the shared secret computed from the private key of one pair and the public key of the other.

- **Identity key** IK: a long-term key pair generated at installation; it identifies the user's device.
- **Signed prekey** SPK: a medium-term key pair, *signed* with IK and replaced periodically.
- **One-time prekeys** $OPK_1, OPK_2, \ldots$: a batch of key pairs, each to be used in exactly one session.

At registration the phone uploads the public halves (and the signature on SPK) to the server; private keys never leave the phone. The server hands out each OPK only once and deletes it, and the phone uploads new ones when the stock runs low.

53.2 X3DH: agreeing on a key with an offline user ★▸

▸ **Likely this year:** "Explain X3DH; why are four Diffie–Hellman computations used?" (5–6 marks).



The messages. (1) Bob publishes a *prekey bundle* in advance, which is what makes the protocol asynchronous: Alice can use it while Bob's phone is off. (2–3) Alice fetches the bundle and verifies the signature on SPK_B with IK_B; this stops the server from substituting a signed prekey of its own while keeping Bob's identity key. (4) Alice generates an **ephemeral** key pair EK_A, computes four DH values, derives the shared key SK with a key-derivation function (HKDF; WhatsApp derives the initial root key and chain key from it), deletes EK_A's private key, and sends her public keys with the first message encrypted under SK. The two identity keys are included as associated data, so the ciphertext is bound to these two identities. (5) Bob, possibly hours later, computes the same four values with his private keys, $\mathrm{DH}(SPK_B, IK_A)$, $\mathrm{DH}(IK_B, EK_A)$, $\mathrm{DH}(SPK_B, EK_A)$, $\mathrm{DH}(OPK_B, EK_A)$, obtains the same SK, and deletes the used one-time prekey.

Why each of the four DH computations is there.

- $DH_1 = \mathrm{DH}(IK_A, SPK_B)$ uses Alice's *identity* key, so only the real Alice can compute SK: it **authenticates Alice to Bob**.
- $DH_2 = \mathrm{DH}(EK_A, IK_B)$ uses Bob's *identity* key, so only the real Bob can compute SK: it **authenticates Bob to Alice**. Together, DH_1 and DH_2 give mutual authentication without any signature on the messages.
- $DH_3 = \mathrm{DH}(EK_A, SPK_B)$ involves no long-term key, so it gives **forward secrecy**: an attacker who later steals both identity keys still cannot recompute SK, because EK_A was deleted and SPK_B has been rotated away.
- $DH_4 = \mathrm{DH}(EK_A, OPK_B)$ uses a key that exists for one session only, so even before SPK_B is rotated, a later theft of Bob's keys does not expose this session, and a replayed first message cannot produce a second valid session. If the server has run out of one-time prekeys, the protocol runs without DH_4.

A single $\mathrm{DH}(IK_A, IK_B)$ would not do: it is the same value in every session, so one stolen identity key would decrypt all past and future sessions. Since no message is signed, either party could have produced the transcript, which gives WhatsApp conversations a degree of **deniability**.

WORKED EXAMPLE : toy X3DH in $\mathbb{Z}_{23}^*$ (new)

Q. Replace Curve25519 by modular Diffie–Hellman with $p = 23$ and generator $g = 5$. Alice has identity key $a = 6$ and ephemeral key $x = 15$; Bob has identity key $b = 9$, signed prekey $s = 3$ and one-time prekey $o = 13$. Compute the public keys and show that both sides obtain the same four DH values.
Public keys ($g^k \bmod 23$): $IK_A = 5^6 = 8$, $EK_A = 5^{15} = 19$, $IK_B = 5^9 = 11$, $SPK_B = 5^3 = 10$, $OPK_B = 5^{13} = 21$.

	Alice computes	Bob computes	Exponent of g	Value
DH_1	$SPK_B^a = 10^6$	$IK_A^s = 8^3 = 512$	$as = 18$	6
DH_2	$IK_B^x = 11^{15}$	$EK_A^b = 19^9$	$xb = 135 \equiv 3 \pmod{22}$	10
DH_3	$SPK_B^x = 10^{15}$	$EK_A^s = 19^3 \equiv (-4)^3 = -64$	$xs = 45 \equiv 1$	5
DH_4	$OPK_B^x = 21^{15}$	$EK_A^o = 19^{13}$	$xo = 195 \equiv 19$	7

Both sides get $(6, 10, 5, 7)$, because each DH value is g raised to the product of the two private keys involved, and exponents may be reduced modulo $p - 1 = 22$. $SK = \mathrm{KDF}(6\|10\|5\|7)$. **Check.** $8^3 = 512 = 22 \cdot 23 + 6$ ✓; $-64 + 3 \cdot 23 = 5$ ✓.

53.3 The Double Ratchet: a fresh key for every message ★▸

▸ **Likely this year:** "Explain the Double Ratchet; derive how it gives forward secrecy and post-compromise security" (5 marks).

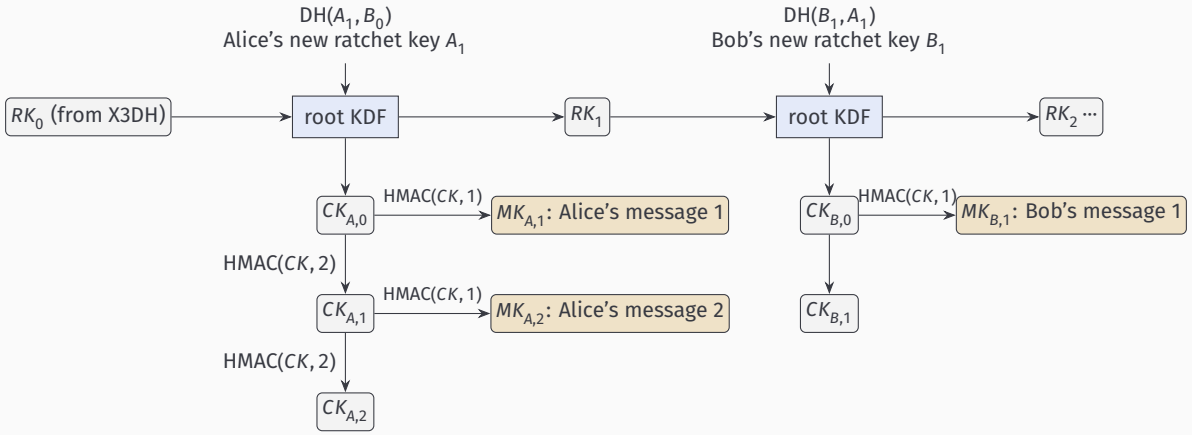
After X3DH the two phones share a **root key** RK and a **chain key** CK. Using one key for the whole conversation would mean that stealing it (by malware, or by seizing the phone) exposes everything. The Double Ratchet instead derives a new 32-byte key for every message, by combining two "ratchets" (mechanisms that only turn forward).

Symmetric-key (KDF) ratchet. Each sending direction has a chain key. For every message the sender computes

$$MK = \text{HMAC-SHA256}(CK, \texttt{0x01}), \qquad CK \leftarrow \text{HMAC-SHA256}(CK, \texttt{0x02}),$$

uses the message key MK once and deletes it. In WhatsApp, MK is expanded with HKDF into 80 bytes: a 32-byte AES-256 key, a 32-byte HMAC-SHA256 key and a 16-byte IV, and the message is encrypted with **AES-256 in CBC mode** and authenticated with **HMAC-SHA256** (encrypt-then-MAC). *What it gives*: since HMAC is one-way, the current CK_i yields $CK_{i+1}, MK_{i+1}, \ldots$ but not CK_{i-1} or MK_{i-1}, which would require inverting HMAC. Old message keys are deleted, so a thief who copies the current state cannot decrypt earlier messages: **forward secrecy**. It does not give post-compromise security, because all *future* keys of that chain follow from the stolen CK.

Diffie–Hellman ratchet. Every message also carries the sender's current **ratchet public key**, a Curve25519 key. Whenever the conversation changes direction, the new sender generates a fresh ratchet key pair and computes a DH value with the other side's latest ratchet public key; this DH output is fed with the root key into the KDF, $(RK, CK) \leftarrow \text{HKDF}(RK,\ \text{DH(own new ratchet key, peer's ratchet key)})$, which gives a new root key and a new chain key for the next run of messages. *What it gives*: suppose Eve copied all of Bob's secrets at some moment. She can follow the symmetric chain, but at the next DH ratchet step the new chain key depends on a DH value computed from a private key generated *after* the theft, which she never saw. As long as she is passive, she is locked out after one round trip: **post-compromise security** (break-in recovery, "self-healing").



Horizontal: the DH ratchet (a new DH value at each change of speaker). Vertical: the symmetric ratchet (one HMAC step per message).

Ratchet	Turns when	Built from	Gives
Symmetric (KDF chain)	every message	HMAC-SHA256 (one-way)	forward secrecy, per-message keys
DH ratchet	every change of speaker	fresh Curve25519 DH + root KDF	post-compromise security

The combination is needed because each alone is insufficient: the symmetric ratchet cannot recover from a theft, and a pure DH ratchet would need a new DH exchange for every message, which is impossible when one side sends several messages without a reply. Because message keys can be derived ahead along a chain, messages that arrive out of order or are lost can still be decrypted (skipped keys are kept briefly, then deleted).

53.4 Groups, attachments and verifying keys

Group messages: sender keys. Encrypting every group message separately for each member (pairwise) would multiply the upload by the group size. Instead, each member, when first sending to a group, generates a random 32-byte **sender chain key** and a Curve25519 **signature key pair**, and distributes both to every other member over the existing pairwise Signal sessions. Each group message is then encrypted once, with a message key from that sender's chain (the same HMAC ratchet, giving forward secrecy), signed with the signature key so members know who sent it, and uploaded once; the server fans the single ciphertext out to all members. When a member leaves, all members discard their sender keys and distribute new ones, so the departed member cannot read later messages. Sender keys have no DH ratchet, so post-compromise security inside a group comes only from such re-keying.

Attachments. A photo or document is encrypted with a fresh random AES-256 key and authenticated with HMAC-SHA256, and the ciphertext is uploaded to a storage server. The key, a hash of the encrypted file and a pointer to it are then sent in an ordinary end-to-end encrypted message, so the storage server holds only ciphertext.

Safety numbers against a man-in-the-middle. Every protection above assumes that Alice received Bob's real identity key. The server distributes identity keys, so a malicious or compromised server could give Alice a key of its own and relay between two separate sessions (the classic Diffie–Hellman man-in-the-middle, §38). The prekey signature does not help, since the attacker would sign with its own identity key. The defence is out-of-band verification: WhatsApp displays a **security code** (Signal: *safety number*), a 60-digit number (also shown as a QR code) formed from the fingerprints of both users' identity keys. If Alice and Bob compare it in person or scan each other's QR code and it matches, no man-in-the-middle is present. The app also warns when a contact's security code changes (new phone or reinstall), which is when an interception would show up.

What E2EE does not cover. The server still learns metadata (who talks to whom, when, phone numbers, IP addresses). Messages are readable on the endpoints, so malware on the phone defeats E2EE. Cloud backups are protected only if the user enables WhatsApp's optional end-to-end encrypted backup. The connection between phone and server is separately protected by a transport protocol (Noise Pipes with Curve25519 and AES-GCM), which hides even the metadata of the encrypted messages from network observers.

54 Internet Banking and UPI Security

54.1 Threats and the controls of internet banking ▸

▸ **Likely this year:** "Discuss the security measures in internet banking" (5 marks).

Online banking is attacked less through its cryptography than through its users and their devices: the attacker steals or tricks out the credentials (phishing, malware, SIM swap) or persuades the customer to authorise a payment himself. The controls are therefore layered so that no single stolen secret is enough.

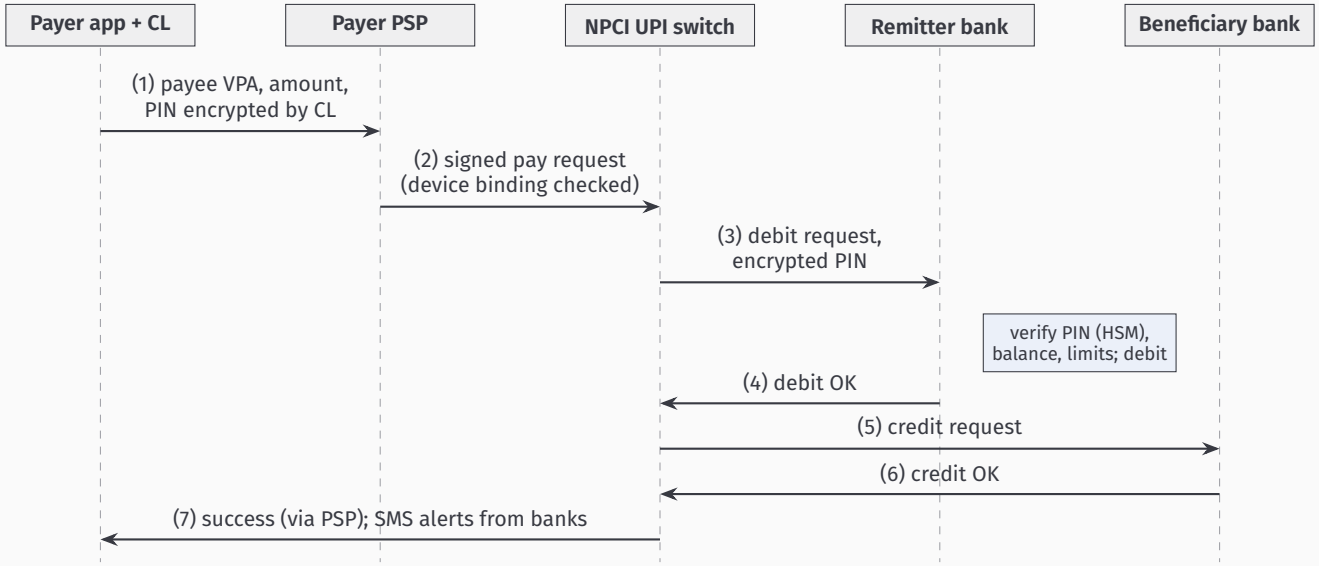
- **TLS (HTTPS) with certificates.** All traffic is encrypted and integrity-protected by TLS, and the bank's server proves its identity with an X.509 certificate issued by a certification authority (§39, §45). This defeats eavesdropping and man-in-the-middle attacks on the network, and it lets the user check the padlock and the exact domain before typing a password, which is the technical defence against look-alike phishing sites.
- **Multi-factor authentication.** Login and transactions require something the user *knows* (password, transaction password) plus something he *has* (a one-time password (OTP) sent by SMS or generated by an app, or the registered phone itself), so a phished password alone is useless. Indian regulation requires such an additional factor of authentication for online card and banking payments.
- **Transaction controls.** Per-transaction and daily limits, OTP confirmation and a cooling period with a reduced limit after adding a new beneficiary, and SMS/e-mail alerts for every login and debit. These limit the damage of a compromised account and let the customer react quickly.
- **Session controls.** Automatic logout after a short idle time, one active session at a time, display of the last login time, CAPTCHA and account lock-out after a few wrong passwords (against password guessing).
- **Virtual keyboards.** The password is entered by clicking a randomly arranged on-screen keyboard, so a key-logger that records keystrokes captures nothing useful.
- **Device binding and risk scoring** (mobile banking). The app is registered to one phone and SIM; a login from a new device, location or pattern triggers extra verification or a block.

Regulation backs these controls: the RBI's directions on digital payment security require banks to implement multi-factor authentication, fraud monitoring, limits and secure application development, and RBI rules on customer liability make the customer's liability zero for an unauthorised electronic transaction caused by a third-party breach if he reports it within three working days. Frauds can also be reported on the national cyber-crime helpline 1930.

54.2 UPI: architecture and the path of a payment ★▸

▸ **Likely this year:** "Explain how a UPI transaction is secured; draw the flow" (6–8 marks).

The **Unified Payments Interface** (UPI), launched by the National Payments Corporation of India (NPCI) in 2016, lets any bank account be debited from a phone in real time. Four kinds of parties are involved: the customer's UPI app, provided by a **Payment Service Provider** (PSP: a bank, or a third-party app such as Google Pay or PhonePe operating through partner PSP banks); the **NPCI UPI switch**, which routes every transaction; the payer's **remitter (issuer) bank**, which holds the account and verifies the PIN; and the payee's **beneficiary bank**. Users are addressed by a **Virtual Payment Address** (VPA, or UPI ID, such as `name@bank`) instead of an account number and IFSC, so the account number is never revealed to the payee.



The messages. (1) The payer enters the payee's VPA (or scans a QR code that contains it) and the amount, then types the UPI PIN into a screen drawn by NPCI's **Common Library** (CL), which encrypts it on the phone. (2) The PSP checks that the request comes from the registered device and SIM and forwards a digitally signed request to NPCI; links between PSPs, NPCI and banks are encrypted and mutually authenticated. (3) NPCI translates the payee VPA into an account (through the payee's PSP) and asks the remitter bank to debit the payer, passing the encrypted PIN. The bank verifies the PIN in a hardware security module, checks balance and limits, and debits. (4–6) NPCI instructs the beneficiary bank to credit the payee and collects the confirmation. (7) The result returns to the app, and both banks send SMS alerts. The whole exchange completes in seconds.

54.3 Why the UPI design is secure ▸

▸ **Likely this year:** "How does UPI achieve two-factor authentication? Why can the app not see the PIN?" (3–4 marks).

Device binding (possession factor). During registration the app sends an SMS from the phone to the PSP; since an SMS reveals the sending SIM's number, this proves that the SIM registered with the bank account is in this phone. The app then binds the account to the combination of mobile number, SIM and device. A thief who learns the PIN cannot use it from his own phone, and removing or changing the SIM forces a fresh registration.

UPI PIN (knowledge factor). The 4- or 6-digit UPI PIN is set using the debit card's last six digits and expiry date plus an OTP from the bank (or an Aadhaar-based OTP), so setting it requires both the card details and the registered phone. Every payment needs the PIN, so UPI is two-factor: *the registered device* and *the PIN*.

The PIN never leaves the phone in clear. The PIN is captured only by the CL, a library supplied and certified by NPCI that every UPI app must embed, and it is encrypted inside the CL with a public key published by NPCI before the app's own code ever handles it. The PSP app and its servers therefore see only ciphertext, and the PIN is decrypted and verified only inside hardware security modules on the NPCI and issuer-bank side. As a result, a malicious or breached UPI app or PSP server cannot collect customers' PINs, and all UPI apps have the same protection here.

VPA instead of account details, and limits. Payees learn a VPA, not an account number and IFSC, which reduces the information available for fraud. NPCI and the banks impose per-transaction and daily limits (for ordinary transfers, ₹1 lakh per transaction, with higher limits for specified categories), and newly registered devices may get lower limits for a while.

54.4 Frauds and countermeasures ▸

▸ **Likely this year:** "Discuss common UPI frauds and how they are prevented" (5 marks).

Since the cryptographic core (device binding plus an encrypted PIN verified by the bank) is hard to break, nearly all UPI frauds trick the user into authorising the payment himself or into handing over the second factor.

Attack	How it works	Countermeasures
Phishing / vishing	Fake bank, KYC or refund messages and calls, or fake "customer care" numbers found by web search, obtain the PIN, OTP or card details.	Banks never ask for PIN or OTP; verify domains and official numbers; awareness campaigns; report to 1930.
Collect-request fraud	The fraudster sends a *collect* (payment request) labelled "refund" or "prize"; the victim enters the PIN, believing it receives money, and pays instead.	The UPI PIN is needed only to pay, never to receive. Apps show the direction and payee clearly, flag spam requests; NPCI has restricted person-to-person collect requests.
QR-code scams	"Scan this QR to receive money"; scanning a QR always starts a payment. Fake QR stickers pasted over a shop's QR divert payments.	Check the payee name shown before entering the PIN; merchants use sound-box confirmations.
SIM swap	The attacker obtains a duplicate SIM, receives the victim's OTPs and SMS, and re-registers banking or UPI on his phone.	Operators hold SMS for a period after a SIM change; banks flag new-device registrations; card details must also be known to set a PIN.
Remote-access / malware	The victim is persuaded to install a screen-sharing app, or malware reads SMS and draws overlays, exposing PIN and OTP or operating the app.	Apps block screenshots and screen sharing on the PIN screen, detect overlays and rooted devices; install apps only from official stores.
Money mules	Stolen money is moved quickly through accounts rented from third parties.	Bank fraud monitoring, account freezing, KYC.

54.5 Model answer: which UPI app is safer, and why? ▸

▸ **Likely this year:** An opinion question with justification (5 marks); the marks are for the reasoning, not for naming an app.

Model answer

Common core. Every UPI app (BHIM, a bank's own app, Google Pay, PhonePe, Paytm and others) runs on the same NPCI rails. Each must be certified by NPCI, uses the same Common Library to capture and encrypt the UPI PIN, the same SMS-based device binding, and the same verification of the PIN by the customer's own bank; the money stays in the bank account, not in the app. The core cryptographic protection is therefore identical: no app can read the PIN, and a breach of one app's servers does not reveal PINs.

Where apps differ: app-level controls. (i) App lock with biometrics or a device PIN before the app opens; (ii) the quality of fraud and risk engines that flag suspicious payees, spam collect requests and new-device logins; (iii) protection against screen sharing, overlays and rooted or tampered phones; (iv) how quickly security updates are released, and whether the app has been independently audited or runs a bug bounty; (v) privacy: permissions requested and use of transaction data; (vi) dispute resolution and how fast a compromised account can be blocked; (vii) attack surface: a feature-rich "super-app" (shopping, loans, mini-apps) contains more code that can fail, while BHIM or a bank's own app is simpler.

Conclusion. No app is intrinsically safer at the protocol level; an app is safer only to the extent that its own controls are stronger. A reasoned choice is an app from a regulated bank or a large NPCI-certified provider, installed from the official store, kept updated, with app lock enabled. Since almost all UPI fraud is social engineering, the user's behaviour (never sharing the PIN or OTP, never entering the PIN to "receive" money, checking the payee name) matters more than the choice of app.

Corrections to the notes / slides

- The slides list Pollard $p-1$, Pollard rho, NFS, QS, AKS and OAEP only as "Tutorial" items without content; the material above follows Forouzan and Stallings and the original papers.
- AKS's complexity is often quoted as $O(\log^{12} n)$: that is the first (2002) analysis. The same algorithm is $\tilde{O}(\log^{10.5} n)$ by a sharper count, and the Lenstra–Pomerance variant is $\tilde{O}(\log^{6} n)$.
- "Pollard rho takes $O(n^{1/4})$ steps" is a worst-case bound for $n = pq$; precisely, it needs about $\sqrt{p}$ steps for the smallest prime factor p.
- Wiener's bound $d < \frac{1}{3}n^{1/4}$ assumes $q < p < 2q$ (primes of similar size), as in the derivation above.

Part VII
Revision

55 Formula and Algorithm Sheet

Everything below is derived and explained in the chapter named in the right-hand column; this sheet is for the last revision, not for learning. Notation follows Forouzan, as the slides do: e_1, e_2 are public values, d a private key, r a fresh random secret, h a hash function, $\|$ concatenation.

Classical ciphers

Shift (Caesar)	$C = (P + k) \bmod 26$, $P = (C - k) \bmod 26$; 26 keys (25 useful)	§8
Multiplicative / affine	$C = (P \cdot k_1 + k_2) \bmod 26$, $P = ((C - k_2) \cdot k_1^{-1}) \bmod 26$; $k_1 \in \mathbb{Z}_{26}^*$ (12 values); affine key space $12 \times 26 = 312$	§17
Monoalphabetic	key = a permutation of the alphabet, $26! \approx 4 \times 10^{26}$ keys; broken by letter frequencies	§11
Playfair	digrams (split doubles with x, pad odd length); same row → right, same column → below, rectangle → same row, other corner	§12
Vigenère / autokey	$C_i = (P_i + K_{i \bmod m}) \bmod 26$; autokey: k_1 given, then $k_i = P_{i-1}$	§14
Hill	$C = PK \bmod 26$, $P = CK^{-1} \bmod 26$; K invertible iff $\gcd(\det K, 26) = 1$	§13
One-time pad	$C_i = P_i \oplus K_i$ (or + mod 26); key random, as long as the message, used once ⇒ perfect secrecy	§15

Number theory

Euclid	$\gcd(a, b) = \gcd(b, a \bmod b)$, $\gcd(a, 0) = a$	§24
Extended Euclid table	$q = \lfloor r_1 / r_2 \rfloor$, $r = r_1 - qr_2$, $t = t_1 - qt_2$; start $(r_1, r_2, t_1, t_2) = (n, b, 0, 1)$; if final $r_1 = 1$ then $b^{-1} = t_1 \bmod n$	§24
Linear congruence	$ax \equiv b \pmod n$, $d = \gcd(a, n)$: solvable iff $d \mid b$; then d solutions $x_0 + k\frac{n}{d}$, $x_0 = (\frac{b}{d})(\frac{a}{d})^{-1} \bmod \frac{n}{d}$	§21
Diophantine $ax + by = c$	solvable iff $d = \gcd(a, b) \mid c$; $x = x_0 + \frac{b}{d}k$, $y = y_0 - \frac{a}{d}k$	§24
Totient	$\phi(p) = p - 1$; $\phi(p^e) = p^e - p^{e-1}$; $\phi(mn) = \phi(m)\phi(n)$ if coprime; $\phi(n) = n \prod_{p \mid n}(1 - \frac{1}{p})$	§25
Fermat	p prime, $p \nmid a$: $a^{p-1} \equiv 1$, $a^{-1} \equiv a^{p-2} \pmod p$; any a: $a^p \equiv a$	§26
Euler	$\gcd(a, n) = 1$: $a^{\phi(n)} \equiv 1$, $a^{-1} \equiv a^{\phi(n)-1} \pmod n$; reduce exponents mod $\phi(n)$	§26
Order, primitive root	$\mathrm{ord}_n(a) \mid \phi(n)$; g is a primitive root iff $\mathrm{ord}(g) = \phi(n)$, i.e. $g^{\phi(n)/q} \neq 1$ for every prime $q \mid \phi(n)$; there are $\phi(\phi(n))$ of them; they exist only for $n = 2, 4, p^k, 2p^k$	§26
CRT	$M = \prod m_i$, $M_i = M / m_i$, $M_i^{-1} \bmod m_i$; $x = \sum a_i M_i M_i^{-1} \bmod M$ (moduli pairwise coprime)	§27
Square-and-multiply	scan exponent bits left to right: always square, multiply by the base when the bit is 1; cost $\leq 2 \log_2 e$ multiplications	§22
Miller–Rabin	$n - 1 = m2^k$, m odd; $T = a^m$; if $T = \pm 1$ prime; square up to $k - 1$ times: +1 composite, −1 prime; else composite; error $\leq 4^{-t}$	§27
GF(2^n)	add = XOR; multiply polynomials mod an irreducible $m(x)$; AES: $m(x) = x^8 + x^4 + x^3 + x + 1$ (11B); inverse by extended Euclid on polynomials	§28

Symmetric ciphers

Padding	$\lvert M \rvert + \lvert \mathrm{Pad} \rvert \equiv 0 \pmod n$	§29
Feistel round	$L_i = R_{i-1}$, $R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$; decryption = same structure, keys reversed	§29
DES	64-bit block, 56-bit key (64 with parity), 16 rounds, 48-bit round keys; f: expansion $32 \to 48$, $\oplus K_i$, 8 S-boxes $6 \to 4$ (row = $b_1 b_6$, column = $b_2 b_3 b_4 b_5$), straight P-box $32 \to 32$; key: PC-1 $64 \to 56$, halves shifted left 1 bit in rounds 1, 2, 9, 16, else 2 (28 in all), PC-2 $56 \to 48$	§30
Multiple DES	2DES: meet-in-the-middle $\approx 2^{57}$ work; 3DES-EDE: $C = E_{K_3}(D_{K_2}(E_{K_1}(P)))$, 2-key version $K_3 = K_1$	§30
AES	128-bit block; key 128/192/256 → 10/12/14 rounds; round = SubBytes, ShiftRows (row r rotated left by r), MixColumns (matrix rows 02 03 01 01 rotated), AddRoundKey; last round has no MixColumns; key expansion $w_i = w_{i-1} \oplus w_{i-4}$, and for $i \bmod 4 = 0$: $w_i = t \oplus w_{i-4}$, $t = \mathrm{SubWord}(\mathrm{RotWord}(w_{i-1})) \oplus \mathrm{RCon}_{i/4}$	§31
Modes	ECB $C_i = E(P_i)$; CBC $C_i = E(P_i \oplus C_{i-1})$, $P_i = D(C_i) \oplus C_{i-1}$, $C_0 = IV$; CFB $C_i = P_i \oplus \mathrm{MSB}_s(E(S_{i-1}))$; OFB $O_i = E(O_{i-1})$, $C_i = P_i \oplus O_i$; CTR $C_i = P_i \oplus E(\mathrm{ctr}_i)$	§32

Public-key encryption

RSA	$n = pq$, $\phi = (p-1)(q-1)$, $\gcd(e,\phi) = 1$, $d = e^{-1} \bmod \phi$; $C = P^e \bmod n$, $P = C^d \bmod n$; CRT: $P_p = C^{d \bmod (p-1)} \bmod p$, P_q likewise, combine	§33
ElGamal	$e_2 = e_1^d \bmod p$; $C_1 = e_1^r$, $C_2 = Pe_2^r \bmod p$; $P = C_2(C_1^d)^{-1} \bmod p$	§34
Elliptic curve over GF(p)	$y^2 = x^3 + ax + b$, $4a^3 + 27b^2 \not\equiv 0$; $\lambda = \frac{y_2-y_1}{x_2-x_1}$ (add) or $\frac{3x_1^2+a}{2y_1}$ (double); $x_3 = \lambda^2 - x_1 - x_2$, $y_3 = \lambda(x_1 - x_3) - y_1$; $-(x,y) = (x,-y)$	§34
EC ElGamal	$e_2 = d \times e_1$; $C_1 = r \times e_1$, $C_2 = P + r \times e_2$; $P = C_2 - d \times C_1$	§34

Hashing and message authentication

Ideal n-bit hash	preimage, second preimage $\approx 2^n$; collision $\approx 2^{n/2}$; birthday: $k \approx 1.18\sqrt{N}$ (any two equal), $0.69N$ (equal to a given value)	§35
SHA-512	1024-bit blocks, 64-bit words, 80 rounds, 512-bit digest; $\lvert M\rvert + \lvert P\rvert + 128 \equiv 0 \pmod{1024}$	§35
HMAC	$h[(K^+ \oplus \text{opad}) \parallel h[(K^+ \oplus \text{ipad}) \parallel M]]$, ipad = 36, opad = 5C repeated	§36
CMAC	CBC-MAC with $IV = 0$; last block $\oplus k_1$ (full) or padded $\oplus k_2$; $k_1 = 2 \cdot E_K(0)$, $k_2 = 2 \cdot k_1$ in GF(2^n); keep n leftmost bits	§36

Digital signatures

RSA	$S = M^d \bmod n$; accept iff $S^e \bmod n = M$ (sign $h(M)$ in practice)	§37
ElGamal	$S_1 = e_1^r \bmod p$, $S_2 = (M - dS_1)r^{-1} \bmod (p-1)$; accept iff $e_1^M \equiv e_2^{S_1} S_1^{S_2} \pmod p$	§37
Schnorr	$q \mid p-1$, e_1 of order q, $e_2 = e_1^d$; $S_1 = h(M \parallel e_1^r \bmod p)$, $S_2 = r + dS_1 \bmod q$; accept iff $h(M \parallel e_1^{S_2} e_2^{-S_1} \bmod p) = S_1$	§37
DSS (DSA)	$S_1 = (e_1^r \bmod p) \bmod q$, $S_2 = (h(M) + dS_1)r^{-1} \bmod q$; $w = S_2^{-1}$, accept iff $(e_1^{h(M)w} e_2^{S_1 w} \bmod p) \bmod q = S_1$	§37
ECDSA	$e_2 = d \times e_1$, e_1 of order q; $P = r \times e_1$, $S_1 = x_P \bmod q$, $S_2 = (h(M) + dS_1)r^{-1} \bmod q$; $T = (h(M)S_2^{-1}) \times e_1 + (S_1 S_2^{-1}) \times e_2$, accept iff $x_T \bmod q = S_1$	§37

Key management and entity authentication

Key counts	N users: $N(N-1)/2$ pairwise secret keys; with a KDC, N master keys	§38
Diffie–Hellman	$R_1 = g^x$, $R_2 = g^y \bmod p$; $K = R_2^x = R_1^y = g^{xy} \bmod p$; MITM: Eve's z gives g^{xz} (Alice–Eve) and g^{yz} (Eve–Bob)	§38
Lamport one-time password	server stores $(n, h^n(P))$; user sends $h^{n-1}(P)$; accept iff h(received) = stored; then store $(n-1, h^{n-1}(P))$	§40
Fiat–Shamir	$n = pq$, $v = s^2 \bmod n$; $x = r^2$, $c \in \{0,1\}$, $y = rs^c$; accept iff $y^2 \equiv xv^c$; cheating 2^{-k} in k rounds	§40
Feige–Fiat–Shamir	secrets $s_1 .. s_k$, public $v_i = (s_i^2)^{-1} \bmod n$; $x = r^2$, challenge bits c_i, $y = r \prod s_i^{c_i}$; accept iff $y^2 \prod v_i^{c_i} \equiv x$; cheating 2^{-k} per round	§40
Guillou–Quisquater	$s^e v \equiv 1 \pmod n$; $x = r^e$, $c \in [1, e]$, $y = rs^c$; accept iff $y^e v^c \equiv x$	§40
BB84	keep positions where bases agree ($\approx \frac{1}{2}$); intercept–resend Eve causes 25% errors in the sifted key	§41